FILE: Dockerfile.backend

```
# Backend Dockerfile for FastAPI service (TradingAgents-CN v1.0.0-preview)
# 前后端分离架构 - 后端服务
# 支持多架构: amd64, arm64
# 使用 Debian Bookworm (稳定版) 而不是 Trixie (测试版)
FROM python:3.10-slim-bookworm
# 获取构建架构信息
ARG TARGETARCH
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    TZ=Asia/Shanghai
WORKDIR /app
# 创建必需的目录并安装系统依赖
# - curl: 健康检查
# - pandoc: 从 GitHub 下载最新版本（避免 Debian 仓库问题）
# - wkhtmltopdf: 从官方下载（用于 PDF 生成）
# - 中文字体: 用于 PDF 中文显示
RUN mkdir -p /app/logs /app/data /app/config && \
    # 配置 apt 重试和超时
    echo 'Acquire::Retries "3";' > /etc/apt/apt.conf.d/80-retries && \
    echo 'Acquire::http::Timeout "30";' >> /etc/apt/apt.conf.d/80-retries && \
    echo 'Acquire::https::Timeout "30";' >> /etc/apt/apt.conf.d/80-retries && \
    # 更新软件源，允许失败后继续
    (apt-get update || apt-get update || apt-get update) && \
    apt-get install -y --no-install-recommends \
        ca-certificates \
        curl \
        fontconfig \
        fonts-noto-cjk \
        wget \
        xvfb && \
    # 根据架构设置变量
    if [ "$TARGETARCH" = "arm64" ]; then \
        PANDOC_ARCH="arm64"; \
        WKHTMLTOPDF_ARCH="arm64"; \
    else \
        PANDOC_ARCH="amd64"; \
        WKHTMLTOPDF_ARCH="amd64"; \
    fi && \
    # 下载并安装 pandoc
    wget -q https://github.com/jgm/pandoc/releases/download/3.8.2.1/pandoc-3.8.2.1-1-${PANDOC_ARCH}.deb && \
    dpkg -i pandoc-3.8.2.1-1-${PANDOC_ARCH}.deb && \
    rm pandoc-3.8.2.1-1-${PANDOC_ARCH}.deb && \
    # 下载并安装 wkhtmltopdf
    wget -q https://github.com/wkhtmltopdf/packaging/releases/download/0.12.6.1-3/wkhtmltox_0.12.6.1-3.bookworm_${WKHTMLTOPDF_ARCH}.deb && \
    apt-get install -y --no-install-recommends \
        ./wkhtmltox_0.12.6.1-3.bookworm_${WKHTMLTOPDF_ARCH}.deb && \
    rm wkhtmltox_0.12.6.1-3.bookworm_${WKHTMLTOPDF_ARCH}.deb && \
    # 更新字体缓存
    fc-cache -fv && \
    rm -rf /var/lib/apt/lists/*
# 复制pyproject.toml和README.md（pip安装需要）
COPY pyproject.toml README.md ./
# 安装Python依赖
# 优化说明：
# 1. 使用清华镜像加速下载
# 2. 优先使用预编译的二进制wheel包（--prefer-binary）
# 3. 避免从源码编译，大幅提升ARM架构构建速度
# 4. 安装 PDF 导出工具: pdfkit
RUN pip install --upgrade pip -i https://pypi.tuna.tsinghua.edu.cn/simple && \
    pip install --prefer-binary . -i https://pypi.tuna.tsinghua.edu.cn/simple && \
    pip install --prefer-binary pdfkit -i https://pypi.tuna.tsinghua.edu.cn/simple
# 复制后端代码和必需模块
COPY app ./app
COPY tradingagents ./tradingagents
COPY config ./config
COPY scripts ./scripts
COPY docs ./docs
COPY install ./install
# 复制Docker环境配置文件
COPY .env.docker ./.env
# 暴露后端端口（与docker-compose.v1.0.0.yml一致）
EXPOSE 8000
# Docker环境标识
ENV DOCKER_CONTAINER=true
# 启动FastAPI服务
CMD ["python", "-m", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

FILE: Dockerfile.frontend

```
# Frontend Dockerfile for Vue 3 + Vite app (TradingAgents-CN v1.0.0-preview)
# 前后端分离架构 - 前端服务
# 构建阶段：使用Node.js 22.x（与项目开发环境一致）
FROM node:22-alpine AS build
ENV NODE_ENV=production
WORKDIR /app/frontend
# 启用Corepack并使用Yarn 1.22.22（项目使用的包管理器）
RUN corepack enable && corepack prepare yarn@1.22.22 --activate
# 复制package.json、yarn.lock和.yarnrc（配置国内镜像源）
COPY frontend/package.json frontend/yarn.lock frontend/.yarnrc ./
# 安装依赖（使用yarn.lock确保版本一致）
# 增加网络超时时间到5分钟，适应跨平台构建的网络延迟
RUN yarn install --frozen-lockfile --production=false --network-timeout 300000
# 复制前端源代码
COPY frontend/. ./
# 复制根目录的静态资源与文档到构建环境
# - assets: 提供 /assets/* 静态资源（前端使用绝对路径）
# - docs: 提供 Article.vue 中通过 ?raw 引用的 Markdown 文档
```

```
COPY assets /app/frontend/public/assets
COPY docs /app/docs
# 构建生产版本（跳过类型检查以加快构建速度）
RUN yarn vite build
# 运行阶段：使用Nginx提供静态文件服务
FROM nginx:alpine AS runtime
WORKDIR /usr/share/nginx/html
# 从构建阶段复制构建产物
COPY --from=build /app/frontend/dist .
# 复制Nginx配置（支持SPA路由）
COPY docker/nginx.conf /etc/nginx/conf.d/default.conf
# 暴露端口80
EXPOSE 80
# 启动Nginx
CMD ["nginx", "-g", "daemon off;"]
```

FILE: VERSION

```
v1.0.0-preview
```

FILE: app/__init__.py

```
"""TradingAgents-CN Web API package."""
```

FILE: app/__main__.py

```
"""
TradingAgents-CN Backend Entry Point
支持 python -m app 启动方式
"""
import uvicorn
import sys
import os
from pathlib import Path
# ============================================================================
# 全局 UTF-8 编码设置（必须在最开始，支持 emoji 和中文）
# ============================================================================
if sys.platform == 'win32':
    try:
        # 1. 设置环境变量，让 Python 全局使用 UTF-8
        os.environ['PYTHONIOENCODING'] = 'utf-8'
        os.environ['PYTHONUTF8'] = '1'
        # 2. 设置标准输出和错误输出为 UTF-8
        import io
        sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', errors='replace')
        sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8', errors='replace')
        # 3. 尝试设置控制台代码页为 UTF-8 (65001)
        try:
            import ctypes
            ctypes.windll.kernel32.SetConsoleCP(65001)
            ctypes.windll.kernel32.SetConsoleOutputCP(65001)
        except Exception:
            pass
    except Exception as e:
        # 如果设置失败，打印警告但继续运行
        print(f"Warning: Failed to set UTF-8 encoding: {e}", file=sys.stderr)
# 添加项目根目录到Python路径
project_root = Path(__file__).parent.parent
sys.path.insert(0, str(project_root))
# 检查并打印.env文件加载信息
def check_env_file():
    """检查并打印.env文件加载信息"""
    import logging
    logger = logging.getLogger("app.startup")

    logger.info("🔍 检查环境配置文件...")
    # 检查当前工作目录
    current_dir = Path.cwd()
    logger.info(f"📂 当前工作目录: {current_dir}")
    # 检查项目根目录
    logger.info(f"📂 项目根目录: {project_root}")

    # 检查可能的.env文件位置（按优先级排序）
    env_locations = [
        project_root / ".env",         # 优先：项目根目录（标准位置）
        current_dir / ".env",          # 次选：当前工作目录
        Path(__file__).parent / ".env"  # 最后：app目录下（不推荐）
    ]
    env_found = False
    for env_path in env_locations:
        if env_path.exists():
            if not env_found:  # 只显示第一个找到的文件详情
                logger.info(f"✅ 找到.env文件: {env_path}")
                logger.info(f"📏 文件大小: {env_path.stat().st_size} bytes")
                env_found = True
                # 读取并显示部分内容（隐藏敏感信息）
                try:
                    with open(env_path, 'r', encoding='utf-8') as f:
                        lines = f.readlines()
                    logger.info(f"📄 .env文件内容预览 (共{len(lines)}行):")
                    for i, line in enumerate(lines[:10]):  # 只显示前10行
                        line = line.strip()
                        if line and not line.startswith('#'):
                            # 隐藏敏感信息
                            if any(keyword in line.upper() for keyword in ['SECRET', 'PASSWORD', 'TOKEN', 'KEY']):
                                key = line.split('=')[0] if '=' in line else line
                                logger.info(f"  {key}=***")
                            else:
```

```
                    logger.info(f"  {line}")
            if len(lines) > 10:
                logger.info(f"  ... (还有{len(lines) - 10}行)")
        except Exception as e:
            logger.warning(f"⚠️ 读取.env文件时出错: {e}")
    else:
        # 如果已经找到一个，只记录其他位置也有文件（可能重复）
        logger.debug(f"ℹ️ 其他位置也有.env文件: {env_path}")
if not env_found:
    logger.warning("⚠️ 未找到.env文件，将使用默认配置")
    logger.info(f"💡 提示: 请在项目根目录 ({project_root}) 创建 .env 文件")

logger.info("-" * 50)
try:
    from app.core.config import settings
    from app.core.dev_config import DEV_CONFIG
except Exception as e:
    import traceback
    print(f"❌ 导入配置模块失败: {e}")
    print("📋 详细错误信息:")
    print("-" * 50)
    traceback.print_exc()
    print("-" * 50)
    sys.exit(1)
def main():
    """主启动函数"""
    import logging
    logger = logging.getLogger("app.startup")

    logger.info("🚀 Starting TradingAgents-CN Backend...")
    logger.info(f"📍 Host: {settings.HOST}")
    logger.info(f"🔌 Port: {settings.PORT}")
    logger.info(f"🐛 Debug Mode: {settings.DEBUG}")
    logger.info(f"📚 API Docs: http://{settings.HOST}:{settings.PORT}/docs" if settings.DEBUG else "📚 API Docs: Disabled in production")

    # 打印关键配置信息
    logger.info("🔧 关键配置信息:")
    logger.info(f"📊 MongoDB: {settings.MONGODB_HOST}:{settings.MONGODB_PORT}/{settings.MONGODB_DATABASE}")
    logger.info(f"🔴 Redis: {settings.REDIS_HOST}:{settings.REDIS_PORT}/{settings.REDIS_DB}")
    logger.info(f"🔐 JWT Secret: {'已配置' if settings.JWT_SECRET != 'change-me-in-production' else '⚠️ 使用默认值'}")
    logger.info(f"📝 日志级别: {settings.LOG_LEVEL}")

    # 检查环境变量加载状态
    logger.info("🌍 环境变量加载状态:")
    env_vars_to_check = [
        ('MONGODB_HOST', settings.MONGODB_HOST, 'localhost'),
        ('MONGODB_PORT', str(settings.MONGODB_PORT), '27017'),
        ('MONGODB_DATABASE', settings.MONGODB_DATABASE, 'tradingagents'),
        ('REDIS_HOST', settings.REDIS_HOST, 'localhost'),
        ('REDIS_PORT', str(settings.REDIS_PORT), '6379'),
        ('JWT_SECRET', '***' if settings.JWT_SECRET != 'change-me-in-production' else settings.JWT_SECRET, 'change-me-in-production')
    ]

    for env_name, current_value, default_value in env_vars_to_check:
        status = "✅ 已设置" if current_value != default_value else "⚠️ 默认值"
        logger.info(f"  {env_name}: {current_value} ({status})")

    logger.info("-" * 50)
    # 获取uvicorn配置
    uvicorn_config = DEV_CONFIG.get_uvicorn_config(settings.DEBUG)
    # 设置简化的日志配置
    logger.info("🔧 正在设置日志配置...")
    try:
        from app.core.logging_config import setup_logging as app_setup_logging
        app_setup_logging(settings.LOG_LEVEL)
    except Exception:
        # 回退到开发环境简化日志配置
        DEV_CONFIG.setup_logging(settings.DEBUG)
    logger.info("✅ 日志配置设置完成")
    # 在日志系统初始化后检查.env文件
    logger.info("📋 Configuration Loading Phase:")
    check_env_file()
    try:
        uvicorn.run(
            "app.main:app",
            host=settings.HOST,
            port=settings.PORT,
            **uvicorn_config
        )
    except KeyboardInterrupt:
        logger.info("⏹️ Server stopped by user")
    except Exception as e:
        import traceback
        logger.error(f"❌ Failed to start server: {e}")
        logger.error("📋 详细错误信息:")
        logger.error("-" * 50)
        traceback.print_exc()
        logger.error("-" * 50)
        sys.exit(1)
if __name__ == "__main__":
    main()
```

FILE: app/constants/model_capabilities.py

```
"""
模型能力分级系统
定义模型的能力等级、适用角色、特性标签等元数据，
用于智能匹配分析深度和模型选择。
🔄 聚合渠道支持：
- 支持 302.AI、OpenRouter、One API 等聚合渠道
- 聚合渠道的模型名称格式：{provider}/{model}（如 openai/gpt-4）
- 系统会自动映射到原厂模型的能力配置
```

```
"""
from enum import IntEnum, Enum
from typing import Dict, List, Any, Tuple

class ModelCapabilityLevel(IntEnum):
    """模型能力等级（1-5级）"""
    BASIC = 1        # 基础：适合1-2级分析，轻量快速
    STANDARD = 2     # 标准：适合1-3级分析，日常使用
    ADVANCED = 3     # 高级：适合1-4级分析，复杂推理
    PROFESSIONAL = 4  # 专业：适合1-5级分析，专业级分析
    FLAGSHIP = 5     # 旗舰：适合所有级别，最强能力

class ModelRole(str, Enum):
    """模型角色类型"""
    QUICK_ANALYSIS = "quick_analysis"  # 快速分析（数据收集、工具调用）
    DEEP_ANALYSIS = "deep_analysis"    # 深度分析（推理、决策）
    BOTH = "both"                      # 两者都适合

class ModelFeature(str, Enum):
    """模型特性标签"""
    TOOL_CALLING = "tool_calling"     # 支持工具调用（必需）
    LONG_CONTEXT = "long_context"     # 支持长上下文
    REASONING = "reasoning"           # 强推理能力
    VISION = "vision"                 # 支持视觉输入
    FAST_RESPONSE = "fast_response"   # 快速响应
    COST_EFFECTIVE = "cost_effective"  # 成本效益高

# 能力等级描述
CAPABILITY_DESCRIPTIONS = {
    1: "基础模型 - 适合快速分析和简单任务，响应快速，成本低",
    2: "标准模型 - 适合日常分析和常规任务，平衡性能和成本",
    3: "高级模型 - 适合深度分析和复杂推理，质量较高",
    4: "专业模型 - 适合专业级分析和多轮辩论，高质量输出",
    5: "旗舰模型 - 最强能力，适合全面分析和关键决策"
}

# 分析深度要求的最低能力等级
ANALYSIS_DEPTH_REQUIREMENTS = {
    "快速": {
        "min_capability": 1,
        "quick_model_min": 1,
        "deep_model_min": 1,
        "required_features": [ModelFeature.TOOL_CALLING],
        "description": "1级快速分析：任何模型都可以，优先选择快速响应的模型"
    },
    "基础": {
        "min_capability": 1,
        "quick_model_min": 1,
        "deep_model_min": 2,
        "required_features": [ModelFeature.TOOL_CALLING],
        "description": "2级基础分析：快速模型可用基础级，深度模型建议标准级以上"
    },
    "标准": {
        "min_capability": 2,
        "quick_model_min": 1,
        "deep_model_min": 2,
        "required_features": [ModelFeature.TOOL_CALLING],
        "description": "3级标准分析：快速模型可用基础级，深度模型需要标准级以上"
    },
    "深度": {
        "min_capability": 3,
        "quick_model_min": 2,
        "deep_model_min": 3,
        "required_features": [ModelFeature.TOOL_CALLING, ModelFeature.REASONING],
        "description": "4级深度分析：快速模型需标准级，深度模型需高级以上，需要推理能力"
    },
    "全面": {
        "min_capability": 4,
        "quick_model_min": 2,
        "deep_model_min": 4,
        "required_features": [ModelFeature.TOOL_CALLING, ModelFeature.REASONING],
        "description": "5级全面分析：快速模型需标准级，深度模型需专业级以上，强推理能力"
    }
}

# 常见模型的默认能力配置（用于初始化和参考）
DEFAULT_MODEL_CAPABILITIES: Dict[str, Dict[str, Any]] = {
    # ==================== 阿里百炼 (DashScope) ====================
    "qwen-turbo": {
        "capability_level": 1,
        "suitable_roles": [ModelRole.QUICK_ANALYSIS],
        "features": [ModelFeature.TOOL_CALLING, ModelFeature.FAST_RESPONSE, ModelFeature.COST_EFFECTIVE],
        "recommended_depths": ["快速", "基础"],
        "performance_metrics": {"speed": 5, "cost": 5, "quality": 3},
        "description": "通义千问轻量版，快速响应，适合数据收集"
    },
    "qwen-plus": {
        "capability_level": 2,
        "suitable_roles": [ModelRole.BOTH],
        "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT],
        "recommended_depths": ["快速", "基础", "标准"],
        "performance_metrics": {"speed": 4, "cost": 4, "quality": 4},
        "description": "通义千问标准版，平衡性能和成本"
    },
    "qwen-max": {
        "capability_level": 4,
        "suitable_roles": [ModelRole.BOTH],
        "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.REASONING],
        "recommended_depths": ["标准", "深度", "全面"],
        "performance_metrics": {"speed": 3, "cost": 2, "quality": 5},
        "description": "通义千问旗舰版，强大推理能力"
    },
    "qwen3-max": {
        "capability_level": 5,
        "suitable_roles": [ModelRole.DEEP_ANALYSIS],
        "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.REASONING],
        "recommended_depths": ["深度", "全面"],
        "performance_metrics": {"speed": 2, "cost": 1, "quality": 5},
        "description": "通义千问长文本版，超长上下文"
    },
```

```
# ==================== OpenAI ====================
"gpt-3.5-turbo": {
    "capability_level": 1,
    "suitable_roles": [ModelRole.QUICK_ANALYSIS],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.FAST_RESPONSE, ModelFeature.COST_EFFECTIVE],
    "recommended_depths": ["快速", "基础"],
    "performance_metrics": {"speed": 5, "cost": 5, "quality": 3},
    "description": "GPT-3.5 Turbo，快速且经济"
},
"gpt-4": {
    "capability_level": 3,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.REASONING],
    "recommended_depths": ["基础", "标准", "深度"],
    "performance_metrics": {"speed": 3, "cost": 3, "quality": 4},
    "description": "GPT-4，强大的推理能力"
},
"gpt-4-turbo": {
    "capability_level": 4,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.REASONING, ModelFeature.VISION],
    "recommended_depths": ["标准", "深度", "全面"],
    "performance_metrics": {"speed": 4, "cost": 2, "quality": 5},
    "description": "GPT-4 Turbo，更快更强"
},
"gpt-4o-mini": {
    "capability_level": 2,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.FAST_RESPONSE, ModelFeature.COST_EFFECTIVE],
    "recommended_depths": ["快速", "基础", "标准"],
    "performance_metrics": {"speed": 5, "cost": 5, "quality": 3},
    "description": "GPT-4o Mini，经济实惠"
},
"o1-mini": {
    "capability_level": 4,
    "suitable_roles": [ModelRole.DEEP_ANALYSIS],
    "features": [ModelFeature.REASONING],
    "recommended_depths": ["深度", "全面"],
    "performance_metrics": {"speed": 2, "cost": 3, "quality": 5},
    "description": "O1 Mini，强推理模型"
},
"o1": {
    "capability_level": 5,
    "suitable_roles": [ModelRole.DEEP_ANALYSIS],
    "features": [ModelFeature.REASONING],
    "recommended_depths": ["全面"],
    "performance_metrics": {"speed": 1, "cost": 1, "quality": 5},
    "description": "O1，最强推理能力"
},
"o4-mini": {
    "capability_level": 4,
    "suitable_roles": [ModelRole.DEEP_ANALYSIS],
    "features": [ModelFeature.REASONING],
    "recommended_depths": ["深度", "全面"],
    "performance_metrics": {"speed": 2, "cost": 3, "quality": 5},
    "description": "O4 Mini，新一代推理模型"
},

# ==================== DeepSeek ====================
"deepseek-chat": {
    "capability_level": 3,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.COST_EFFECTIVE],
    "recommended_depths": ["基础", "标准", "深度"],
    "performance_metrics": {"speed": 4, "cost": 5, "quality": 4},
    "description": "DeepSeek Chat，性价比高"
},

# ==================== 百度文心 (Qianfan) ====================
"ernie-3.5": {
    "capability_level": 2,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING],
    "recommended_depths": ["快速", "基础", "标准"],
    "performance_metrics": {"speed": 4, "cost": 4, "quality": 3},
    "description": "文心一言3.5，标准版本"
},
"ernie-4.0": {
    "capability_level": 3,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.REASONING],
    "recommended_depths": ["基础", "标准", "深度"],
    "performance_metrics": {"speed": 3, "cost": 3, "quality": 4},
    "description": "文心一言4.0，高级版本"
},
"ernie-4.0-turbo": {
    "capability_level": 4,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.REASONING, ModelFeature.FAST_RESPONSE],
    "recommended_depths": ["标准", "深度", "全面"],
    "performance_metrics": {"speed": 4, "cost": 2, "quality": 5},
    "description": "文心一言4.0 Turbo，旗舰版本"
},

# ==================== 智谱AI (GLM) ====================
"glm-3-turbo": {
    "capability_level": 1,
    "suitable_roles": [ModelRole.QUICK_ANALYSIS],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.FAST_RESPONSE, ModelFeature.COST_EFFECTIVE],
    "recommended_depths": ["快速", "基础"],
    "performance_metrics": {"speed": 5, "cost": 5, "quality": 3},
    "description": "智谱GLM-3 Turbo，快速版本"
},
"glm-4": {
    "capability_level": 3,
```

```
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.REASONING],
    "recommended_depths": ["基础", "标准", "深度"],
    "performance_metrics": {"speed": 3, "cost": 3, "quality": 4},
    "description": "智谱GLM-4，标准版本"
},
"glm-4-plus": {
    "capability_level": 4,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.REASONING],
    "recommended_depths": ["标准", "深度", "全面"],
    "performance_metrics": {"speed": 3, "cost": 2, "quality": 5},
    "description": "智谱GLM-4 Plus，旗舰版本"
},

# ==================== Anthropic Claude ====================
"claude-3-haiku": {
    "capability_level": 2,
    "suitable_roles": [ModelRole.QUICK_ANALYSIS],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.FAST_RESPONSE],
    "recommended_depths": ["快速", "基础", "标准"],
    "performance_metrics": {"speed": 5, "cost": 4, "quality": 3},
    "description": "Claude 3 Haiku，快速版本"
},
"claude-3-sonnet": {
    "capability_level": 3,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.VISION],
    "recommended_depths": ["基础", "标准", "深度"],
    "performance_metrics": {"speed": 4, "cost": 3, "quality": 4},
    "description": "Claude 3 Sonnet，平衡版本"
},
"claude-3-opus": {
    "capability_level": 4,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.REASONING, ModelFeature.VISION],
    "recommended_depths": ["标准", "深度", "全面"],
    "performance_metrics": {"speed": 3, "cost": 2, "quality": 5},
    "description": "Claude 3 Opus，旗舰版本"
},
"claude-3.5-sonnet": {
    "capability_level": 5,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.REASONING, ModelFeature.VISION],
    "recommended_depths": ["标准", "深度", "全面"],
    "performance_metrics": {"speed": 4, "cost": 2, "quality": 5},
    "description": "Claude 3.5 Sonnet，最新旗舰"
},
# ==================== Google Gemini ====================
"gemini-pro": {
    "capability_level": 3,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.REASONING],
    "recommended_depths": ["基础", "标准", "深度"],
    "performance_metrics": {"speed": 4, "cost": 4, "quality": 4},
    "description": "Gemini Pro，经典稳定版本"
},
"gemini-1.5-pro": {
    "capability_level": 4,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.REASONING, ModelFeature.VISION],
    "recommended_depths": ["标准", "深度", "全面"],
    "performance_metrics": {"speed": 4, "cost": 3, "quality": 5},
    "description": "Gemini 1.5 Pro，长上下文旗舰"
},
"gemini-1.5-flash": {
    "capability_level": 2,
    "suitable_roles": [ModelRole.QUICK_ANALYSIS],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.FAST_RESPONSE, ModelFeature.COST_EFFECTIVE],
    "recommended_depths": ["快速", "基础", "标准"],
    "performance_metrics": {"speed": 5, "cost": 5, "quality": 3},
    "description": "Gemini 1.5 Flash，快速响应版本"
},
"gemini-2.0-flash": {
    "capability_level": 4,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.REASONING, ModelFeature.FAST_RESPONSE],
    "recommended_depths": ["标准", "深度", "全面"],
    "performance_metrics": {"speed": 5, "cost": 3, "quality": 5},
    "description": "Gemini 2.0 Flash，新一代快速旗舰"
},
"gemini-2.5-flash-lite-preview-06-17": {
    "capability_level": 2,
    "suitable_roles": [ModelRole.QUICK_ANALYSIS],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.FAST_RESPONSE, ModelFeature.COST_EFFECTIVE],
    "recommended_depths": ["快速", "基础"],
    "performance_metrics": {"speed": 5, "cost": 5, "quality": 3},
    "description": "Gemini 2.5 Flash Lite，轻量预览版"
},
# ==================== 月之暗面 (Moonshot) ====================
"moonshot-v1-8k": {
    "capability_level": 2,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING],
    "recommended_depths": ["快速", "基础", "标准"],
    "performance_metrics": {"speed": 4, "cost": 4, "quality": 3},
    "description": "Moonshot V1 8K，标准版本"
},
"moonshot-v1-32k": {
    "capability_level": 3,
    "suitable_roles": [ModelRole.BOTH],
    "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT],
    "recommended_depths": ["基础", "标准", "深度"],
    "performance_metrics": {"speed": 3, "cost": 3, "quality": 4},
    "description": "Moonshot V1 32K，长上下文版本"
},
```

```
    "moonshot-v1-128k": {
        "capability_level": 4,
        "suitable_roles": [ModelRole.DEEP_ANALYSIS],
        "features": [ModelFeature.TOOL_CALLING, ModelFeature.LONG_CONTEXT, ModelFeature.REASONING],
        "recommended_depths": ["标准", "深度", "全面"],
        "performance_metrics": {"speed": 2, "cost": 2, "quality": 5},
        "description": "Moonshot V1 128K，超长上下文旗舰"
    },
}

def get_model_capability_badge(level: int) -> Dict[str, str]:
    """获取能力等级徽章样式"""
    badges = {
        1: {"text": "基础", "color": "#909399", "icon": "⚡"},
        2: {"text": "标准", "color": "#409EFF", "icon": "📊"},
        3: {"text": "高级", "color": "#67C23A", "icon": "🎯"},
        4: {"text": "专业", "color": "#E6A23C", "icon": "🔥"},
        5: {"text": "旗舰", "color": "#F56C6C", "icon": "👑"}
    }
    return badges.get(level, badges[2])

def get_role_badge(role: ModelRole) -> Dict[str, str]:
    """获取角色徽章样式"""
    badges = {
        ModelRole.QUICK_ANALYSIS: {"text": "快速分析", "color": "success", "icon": "⚡"},
        ModelRole.DEEP_ANALYSIS: {"text": "深度推理", "color": "warning", "icon": "🧠"},
        ModelRole.BOTH: {"text": "通用", "color": "primary", "icon": "🎯"}
    }
    return badges.get(role, badges[ModelRole.BOTH])

def get_feature_badge(feature: ModelFeature) -> Dict[str, str]:
    """获取特性徽章样式"""
    badges = {
        ModelFeature.TOOL_CALLING: {"text": "工具调用", "color": "info", "icon": "🔧"},
        ModelFeature.LONG_CONTEXT: {"text": "长上下文", "color": "success", "icon": "📚"},
        ModelFeature.REASONING: {"text": "强推理", "color": "warning", "icon": "🧠"},
        ModelFeature.VISION: {"text": "视觉", "color": "primary", "icon": "👁"},
        ModelFeature.FAST_RESPONSE: {"text": "快速", "color": "success", "icon": "⚡"},
        ModelFeature.COST_EFFECTIVE: {"text": "经济", "color": "success", "icon": "💰"}
    }
    return badges.get(feature, {"text": str(feature), "color": "info", "icon": "✨"})

# ==================== 聚合渠道配置 ====================
# 聚合渠道的默认配置
AGGREGATOR_PROVIDERS = {
    "302ai": {
        "display_name": "302.AI",
        "description": "302.AI 聚合平台，提供多厂商模型统一接口",
        "website": "https://302.ai",
        "api_doc_url": "https://doc.302.ai",
        "default_base_url": "https://api.302.ai/v1",
        "model_name_format": "{provider}/{model}",  # 如: openai/gpt-4
        "supported_providers": ["openai", "anthropic", "google", "deepseek", "qwen"]
    },
    "openrouter": {
        "display_name": "OpenRouter",
        "description": "OpenRouter 聚合平台，支持多种 AI 模型",
        "website": "https://openrouter.ai",
        "api_doc_url": "https://openrouter.ai/docs",
        "default_base_url": "https://openrouter.ai/api/v1",
        "model_name_format": "{provider}/{model}",
        "supported_providers": ["openai", "anthropic", "google", "meta", "mistral"]
    },
    "oneapi": {
        "display_name": "One API",
        "description": "One API 开源聚合平台",
        "website": "https://github.com/songquanpeng/one-api",
        "api_doc_url": "https://github.com/songquanpeng/one-api",
        "default_base_url": "http://localhost:3000/v1",  # 需要用户自行部署
        "model_name_format": "{model}",  # One API 通常不需要前缀
        "supported_providers": ["openai", "anthropic", "google", "azure", "claude"]
    },
    "newapi": {
        "display_name": "New API",
        "description": "New API 聚合平台",
        "website": "https://github.com/Calcium-Ion/new-api",
        "api_doc_url": "https://github.com/Calcium-Ion/new-api",
        "default_base_url": "http://localhost:3000/v1",
        "model_name_format": "{model}",
        "supported_providers": ["openai", "anthropic", "google", "azure", "claude"]
    }
}

def is_aggregator_model(model_name: str) -> bool:
    """
    判断是否为聚合渠道模型名称

    Args:
        model_name: 模型名称

    Returns:
        是否为聚合渠道模型
    """
    return "/" in model_name

def parse_aggregator_model(model_name: str) -> Tuple[str, str]:
    """
    解析聚合渠道模型名称

    Args:
        model_name: 模型名称（如 openai/gpt-4）

    Returns:
        (provider, model) 元组
    """
    if "/" in model_name:
        parts = model_name.split("/", 1)
        return parts[0], parts[1]
    return "", model_name
```

FILE: app/core/__init__.py

"""
Core module for TradingAgents FastAPI backend
"""

FILE: app/core/config.py

```
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict
from typing import List
import os
import warnings
# Legacy env var aliases (deprecated): map API_HOST/PORT/DEBUG -> HOST/PORT/DEBUG
_LEGACY_ENV_ALIASES = {
    "API_HOST": "HOST",
    "API_PORT": "PORT",
    "API_DEBUG": "DEBUG",
}
for _legacy, _new in _LEGACY_ENV_ALIASES.items():
    if _new not in os.environ and _legacy in os.environ:
        os.environ[_new] = os.environ[_legacy]
        warnings.warn(
            f"Environment variable {_legacy} is deprecated; use {_new} instead.",
            DeprecationWarning,
            stacklevel=2,
        )
class Settings(BaseSettings):
    # 基础配置
    DEBUG: bool = Field(default=True)
    HOST: str = Field(default="0.0.0.0")
    PORT: int = Field(default=8000)
    ALLOWED_ORIGINS: List[str] = Field(default_factory=lambda: ["*"])
    ALLOWED_HOSTS: List[str] = Field(default_factory=lambda: ["*"])
    # MongoDB配置
    MONGODB_HOST: str = Field(default="localhost")
    MONGODB_PORT: int = Field(default=27017)
    MONGODB_USERNAME: str = Field(default="")
    MONGODB_PASSWORD: str = Field(default="")
    MONGODB_DATABASE: str = Field(default="tradingagents")
    MONGODB_AUTH_SOURCE: str = Field(default="admin")
    MONGO_MAX_CONNECTIONS: int = Field(default=100)
    MONGO_MIN_CONNECTIONS: int = Field(default=10)
    # MongoDB超时参数（毫秒）- 用于处理大量历史数据
    MONGO_CONNECT_TIMEOUT_MS: int = Field(default=30000)  # 连接超时：30秒（原为10秒）
    MONGO_SOCKET_TIMEOUT_MS: int = Field(default=60000)   # 套接字超时：60秒（原为20秒）
    MONGO_SERVER_SELECTION_TIMEOUT_MS: int = Field(default=5000)  # 服务器选择超时：5秒
    @property
    def MONGO_URI(self) -> str:
        """构建MongoDB URI"""
        if self.MONGODB_USERNAME and self.MONGODB_PASSWORD:
            return f"mongodb://{self.MONGODB_USERNAME}:{self.MONGODB_PASSWORD}@{self.MONGODB_HOST}:{self.MONGODB_PORT}/{self.MONGODB_DATABASE}?authSource={self.MONGODB_AUTH_SOURCE}"
        else:
            return f"mongodb://{self.MONGODB_HOST}:{self.MONGODB_PORT}/{self.MONGODB_DATABASE}"
    @property
    def MONGO_DB(self) -> str:
        """获取数据库名称"""
        return self.MONGODB_DATABASE
    # Redis配置
    REDIS_HOST: str = Field(default="localhost")
    REDIS_PORT: int = Field(default=6379)
    REDIS_PASSWORD: str = Field(default="")
    REDIS_DB: int = Field(default=0)
    REDIS_MAX_CONNECTIONS: int = Field(default=20)
    REDIS_RETRY_ON_TIMEOUT: bool = Field(default=True)
    @property
    def REDIS_URL(self) -> str:
        """构建Redis URL"""
        if self.REDIS_PASSWORD:
            return f"redis://:{self.REDIS_PASSWORD}@{self.REDIS_HOST}:{self.REDIS_PORT}/{self.REDIS_DB}"
        else:
            return f"redis://{self.REDIS_HOST}:{self.REDIS_PORT}/{self.REDIS_DB}"
    # JWT配置
    JWT_SECRET: str = Field(default="change-me-in-production")
    JWT_ALGORITHM: str = Field(default="HS256")
    ACCESS_TOKEN_EXPIRE_MINUTES: int = Field(default=60)
    REFRESH_TOKEN_EXPIRE_DAYS: int = Field(default=30)
    # 队列配置
    QUEUE_MAX_SIZE: int = Field(default=10000)
    QUEUE_VISIBILITY_TIMEOUT: int = Field(default=300)  # 5分钟
    QUEUE_MAX_RETRIES: int = Field(default=3)
    WORKER_HEARTBEAT_INTERVAL: int = Field(default=30)  # 30秒
    # 队列轮询/清理间隔（秒）
    QUEUE_POLL_INTERVAL_SECONDS: float = Field(default=1.0)
    QUEUE_CLEANUP_INTERVAL_SECONDS: float = Field(default=60.0)
    # 并发控制
    DEFAULT_USER_CONCURRENT_LIMIT: int = Field(default=3)
    GLOBAL_CONCURRENT_LIMIT: int = Field(default=50)
    DEFAULT_DAILY_QUOTA: int = Field(default=1000)
    # 速率限制
    RATE_LIMIT_ENABLED: bool = Field(default=True)
    DEFAULT_RATE_LIMIT: int = Field(default=100)  # 每分钟请求数
    # 日志配置
    LOG_LEVEL: str = Field(default="INFO")
    LOG_FORMAT: str = Field(default="%(asctime)s - %(name)s - %(levelname)s - %(message)s")
    LOG_FILE: str = Field(default="logs/tradingagents.log")
    # 代理配置
    # 用于配置需要绕过代理的域名（国内数据源）
    # 多个域名用逗号分隔
    # ⚠️  Windows 不支持通配符 *，必须使用完整域名
    # 详细说明: docs/proxy_configuration.md
    HTTP_PROXY: str = Field(default="")
    HTTPS_PROXY: str = Field(default="")
    NO_PROXY: str = Field(
```

```
    default="localhost,127.0.0.1,eastmoney.com,push2.eastmoney.com,82.push2.eastmoney.com,82.push2delay.eastmoney.com,gtimg.cn,sinaimg.cn,api.tushare.pro,baostock.com"
)
# 文件上传配置
MAX_UPLOAD_SIZE: int = Field(default=10 * 1024 * 1024)  # 10MB
UPLOAD_DIR: str = Field(default="uploads")
# 缓存配置
CACHE_TTL: int = Field(default=3600)  # 1小时
SCREENING_CACHE_TTL: int = Field(default=1800)  # 30分钟
# 安全配置
BCRYPT_ROUNDS: int = Field(default=12)
SESSION_EXPIRE_HOURS: int = Field(default=24)
CSRF_SECRET: str = Field(default="change-me-csrf-secret")
# 外部服务配置
STOCK_DATA_API_URL: str = Field(default="")
STOCK_DATA_API_KEY: str = Field(default="")
# SSE 配置
SSE_POLL_TIMEOUT_SECONDS: float = Field(default=1.0)
SSE_HEARTBEAT_INTERVAL_SECONDS: int = Field(default=10)
SSE_TASK_MAX_IDLE_SECONDS: int = Field(default=300)
SSE_BATCH_POLL_INTERVAL_SECONDS: float = Field(default=2.0)
SSE_BATCH_MAX_IDLE_SECONDS: int = Field(default=600)
# 监控配置
METRICS_ENABLED: bool = Field(default=True)
HEALTH_CHECK_INTERVAL: int = Field(default=60)  # 60秒
# 配置真相来源（方案A）：file|db|hybrid
# - file：以文件/env 为准（推荐，生产缺省）
# - db：以数据库为准（仅兼容旧版，不推荐）
# - hybrid：文件/env 优先，DB 作为兜底
CONFIG_SOT: str = Field(default="file")
# 基础信息同步任务配置（可配置调度）
SYNC_STOCK_BASICS_ENABLED: bool = Field(default=True)
# 优先使用 CRON 表达式，例如 "30 6 * * *" 表示每日 06:30
SYNC_STOCK_BASICS_CRON: str = Field(default="")
# 若未提供 CRON，则使用简单时间字符串 "HH:MM"（24小时制）
SYNC_STOCK_BASICS_TIME: str = Field(default="06:30")
# 时区
TIMEZONE: str = Field(default="Asia/Shanghai")
# 实时行情入库任务
QUOTES_INGEST_ENABLED: bool = Field(default=True)
QUOTES_INGEST_INTERVAL_SECONDS: int = Field(
    default=360,
    description="实时行情采集间隔（秒）。默认360秒（6分钟），免费用户建议>=300秒，付费用户可设置5-60秒"
)
# 休市期/启动兜底补数（填充上一笔快照）
QUOTES_BACKFILL_ON_STARTUP: bool = Field(default=True)
QUOTES_BACKFILL_ON_OFFHOURS: bool = Field(default=True)
# 实时行情接口轮换配置
QUOTES_ROTATION_ENABLED: bool = Field(
    default=True,
    description="启用接口轮换机制（Tushare → AKShare东方财富 → AKShare新浪财经）"
)
QUOTES_TUSHARE_HOURLY_LIMIT: int = Field(
    default=2,
    description="Tushare rt_k接口每小时调用次数限制（免费用户2次，付费用户可设置更高）"
)
QUOTES_AUTO_DETECT_TUSHARE_PERMISSION: bool = Field(
    default=True,
    description="自动检测Tushare rt_k接口权限，付费用户自动切换到高频模式（5秒）"
)
# Tushare基础配置
TUSHARE_TOKEN: str = Field(default="", description="Tushare API Token")
TUSHARE_ENABLED: bool = Field(default=True, description="启用Tushare数据源")
TUSHARE_TIER: str = Field(default="standard", description="Tushare积分等级 (free/basic/standard/premium/vip)")
TUSHARE_RATE_LIMIT_SAFETY_MARGIN: float = Field(default=0.8, ge=0.1, le=1.0, description="速率限制安全边际")
# Tushare统一数据同步配置
TUSHARE_UNIFIED_ENABLED: bool = Field(default=True)
TUSHARE_BASIC_INFO_SYNC_ENABLED: bool = Field(default=True)
TUSHARE_BASIC_INFO_SYNC_CRON: str = Field(default="0 2 * * *")  # 每日凌晨2点
TUSHARE_QUOTES_SYNC_ENABLED: bool = Field(default=True)
TUSHARE_QUOTES_SYNC_CRON: str = Field(default="*/5 9-15 * * 1-5")  # 交易时间每5分钟
TUSHARE_HISTORICAL_SYNC_ENABLED: bool = Field(default=True)
TUSHARE_HISTORICAL_SYNC_CRON: str = Field(default="0 16 * * 1-5")  # 工作日16点
TUSHARE_FINANCIAL_SYNC_ENABLED: bool = Field(default=True)
TUSHARE_FINANCIAL_SYNC_CRON: str = Field(default="0 3 * * 0")  # 周日凌晨3点
TUSHARE_STATUS_CHECK_ENABLED: bool = Field(default=True)
TUSHARE_STATUS_CHECK_CRON: str = Field(default="0 * * * *")  # 每小时
# Tushare数据初始化配置
TUSHARE_INIT_HISTORICAL_DAYS: int = Field(default=365, ge=1, le=3650, description="初始化历史数据天数")
TUSHARE_INIT_BATCH_SIZE: int = Field(default=100, ge=10, le=1000, description="初始化批处理大小")
TUSHARE_INIT_AUTO_START: bool = Field(default=False, description="应用启动时自动检查并初始化数据")
# AKShare统一数据同步配置
AKSHARE_UNIFIED_ENABLED: bool = Field(default=True, description="启用AKShare统一数据同步")
AKSHARE_BASIC_INFO_SYNC_ENABLED: bool = Field(default=True, description="启用基础信息同步")
AKSHARE_BASIC_INFO_SYNC_CRON: str = Field(default="0 3 * * *", description="基础信息同步CRON表达式")  # 每日凌晨3点
AKSHARE_QUOTES_SYNC_ENABLED: bool = Field(default=True, description="启用行情同步")
AKSHARE_QUOTES_SYNC_CRON: str = Field(default="*/30 9-15 * * 1-5", description="行情同步CRON表达式")  # 交易时间每30分钟（避免频率限制）
AKSHARE_HISTORICAL_SYNC_ENABLED: bool = Field(default=True, description="启用历史数据同步")
AKSHARE_HISTORICAL_SYNC_CRON: str = Field(default="0 17 * * 1-5", description="历史数据同步CRON表达式")  # 工作日17点
AKSHARE_FINANCIAL_SYNC_ENABLED: bool = Field(default=True, description="启用财务数据同步")
AKSHARE_FINANCIAL_SYNC_CRON: str = Field(default="0 4 * * 0", description="财务数据同步CRON表达式")  # 周日凌晨4点
AKSHARE_STATUS_CHECK_ENABLED: bool = Field(default=True, description="启用状态检查")
AKSHARE_STATUS_CHECK_CRON: str = Field(default="30 * * * *", description="状态检查CRON表达式")  # 每小时30分
# AKShare数据初始化配置
AKSHARE_INIT_HISTORICAL_DAYS: int = Field(default=365, ge=1, le=3650, description="初始化历史数据天数")
AKSHARE_INIT_BATCH_SIZE: int = Field(default=100, ge=10, le=1000, description="初始化批处理大小")
AKSHARE_INIT_AUTO_START: bool = Field(default=False, description="应用启动时自动检查并初始化数据")
# ==================== 分析师数据获取配置 ====================
# 市场分析师数据范围配置
# 默认60天：可覆盖MA60等所有常用技术指标（MA5/10/20/60, MACD, RSI, BOLL）
MARKET_ANALYST_LOOKBACK_DAYS: int = Field(default=60, ge=5, le=365, description="市场分析回溯天数（用于技术分析）")
# ==================== BaoStock统一数据同步配置 ====================
# BaoStock统一数据同步总开关
```

```
    BAOSTOCK_UNIFIED_ENABLED: bool = Field(default=True, description="启用BaoStock统一数据同步")
    # BaoStock数据同步任务配置
    BAOSTOCK_BASIC_INFO_SYNC_ENABLED: bool = Field(default=True, description="启用基础信息同步")
    BAOSTOCK_BASIC_INFO_SYNC_CRON: str = Field(default="0 4 * * *", description="基础信息同步CRON表达式")  # 每日凌晨4点
    BAOSTOCK_DAILY_QUOTES_SYNC_ENABLED: bool = Field(default=True, description="启用日K线同步（注意：BaoStock不支持实时行情）")
    BAOSTOCK_DAILY_QUOTES_SYNC_CRON: str = Field(default="0 16 * * 1-5", description="日K线同步CRON表达式")  # 工作日收盘后16:00
    BAOSTOCK_HISTORICAL_SYNC_ENABLED: bool = Field(default=True, description="启用历史数据同步")
    BAOSTOCK_HISTORICAL_SYNC_CRON: str = Field(default="0 18 * * 1-5", description="历史数据同步CRON表达式")  # 工作日18点
    BAOSTOCK_STATUS_CHECK_ENABLED: bool = Field(default=True, description="启用状态检查")
    BAOSTOCK_STATUS_CHECK_CRON: str = Field(default="45 * * * *", description="状态检查CRON表达式")  # 每小时45分
    # BaoStock数据初始化配置
    BAOSTOCK_INIT_HISTORICAL_DAYS: int = Field(default=365, ge=1, le=3650, description="初始化历史数据天数")
    BAOSTOCK_INIT_BATCH_SIZE: int = Field(default=50, ge=10, le=500, description="初始化批处理大小")
    BAOSTOCK_INIT_AUTO_START: bool = Field(default=False, description="应用启动时自动检查并初始化数据")
    # 数据目录配置
    TRADINGAGENTS_DATA_DIR: str = Field(default="./data")
    @property
    def log_dir(self) -> str:
        """获取日志目录"""
        return os.path.dirname(self.LOG_FILE)
    # ==================== 港股数据配置 ====================
    # 港股数据源配置（按需获取+缓存模式）
    HK_DATA_CACHE_HOURS: int = Field(default=24, ge=1, le=168, description="港股数据缓存时长（小时）")
    HK_DEFAULT_DATA_SOURCE: str = Field(default="yfinance", description="港股默认数据源（yfinance/akshare）")
    # ==================== 美股数据配置 ====================
    # 美股数据源配置（按需获取+缓存模式）
    US_DATA_CACHE_HOURS: int = Field(default=24, ge=1, le=168, description="美股数据缓存时长（小时）")
    US_DEFAULT_DATA_SOURCE: str = Field(default="yfinance", description="美股默认数据源（yfinance/finnhub）")
    # ===== 新闻数据同步服务配置 =====
    NEWS_SYNC_ENABLED: bool = Field(default=True)
    NEWS_SYNC_CRON: str = Field(default="0 */2 * * *")  # 每2小时
    NEWS_SYNC_HOURS_BACK: int = Field(default=24)
    NEWS_SYNC_MAX_PER_SOURCE: int = Field(default=50)
    @property
    def is_production(self) -> bool:
        """是否为生产环境"""
        return not self.DEBUG
    # Ignore any extra environment variables present in .env or process env
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
settings = Settings()
# 自动将代理配置设置到环境变量
# 这样 requests 库可以直接读取 os.environ['NO_PROXY']
if settings.HTTP_PROXY:
    os.environ['HTTP_PROXY'] = settings.HTTP_PROXY
if settings.HTTPS_PROXY:
    os.environ['HTTPS_PROXY'] = settings.HTTPS_PROXY
if settings.NO_PROXY:
    os.environ['NO_PROXY'] = settings.NO_PROXY
def get_settings() -> Settings:
    """获取配置实例"""
    return settings
```

FILE: app/core/config_bridge.py

```
"""
配置桥接模块
将统一配置系统的配置桥接到环境变量，供 TradingAgents 核心库使用
"""
import os
import json
import logging
from pathlib import Path
from typing import Optional
logger = logging.getLogger("app.config_bridge")
def bridge_config_to_env():
    """
    将统一配置桥接到环境变量
    这个函数会：
    1. 从数据库读取大模型厂家配置（API 密钥、超时、温度等）
    2. 将配置写入环境变量
    3. 将默认模型写入环境变量
    4. 将数据源配置写入环境变量（API 密钥、超时、重试等）
    5. 将系统运行时配置写入环境变量
    这样 TradingAgents 核心库就能通过环境变量读取到用户配置的数据
    """
    try:
        from app.core.unified_config import unified_config
        from app.services.config_service import config_service
        logger.info("🔧 开始桥接配置到环境变量...")
        bridged_count = 0
        # 强制启用 MongoDB 存储（用于 Token 使用统计）
        # 从 .env 文件读取配置，如果未设置则默认启用
        use_mongodb_storage = os.getenv("USE_MONGODB_STORAGE", "true")
        os.environ["USE_MONGODB_STORAGE"] = use_mongodb_storage
        logger.info(f" ✓ 桥接 USE_MONGODB_STORAGE: {use_mongodb_storage}")
        bridged_count += 1
        # 桥接 MongoDB 连接字符串
        mongodb_conn_str = os.getenv("MONGODB_CONNECTION_STRING")
        if mongodb_conn_str:
            os.environ["MONGODB_CONNECTION_STRING"] = mongodb_conn_str
            logger.info(f" ✓ 桥接 MONGODB_CONNECTION_STRING (长度: {len(mongodb_conn_str)})")
            bridged_count += 1
        # 桥接 MongoDB 数据库名称
        mongodb_db_name = os.getenv("MONGODB_DATABASE_NAME", "tradingagents")
        os.environ["MONGODB_DATABASE_NAME"] = mongodb_db_name
        logger.info(f" ✓ 桥接 MONGODB_DATABASE_NAME: {mongodb_db_name}")
        bridged_count += 1
        # 1. 桥接大模型配置（基础 API 密钥）
        # 🔧 [优先级] .env 文件 > 数据库厂家配置
        # 💧 修改：从数据库的 llm_providers 集合读取厂家配置，而不是从 JSON 文件
```

```
# 只有当环境变量不存在或为占位符时，才使用数据库中的配置
try:
    # 使用同步 MongoDB 客户端读取厂家配置
    from pymongo import MongoClient
    from app.core.config import settings
    from app.models.config import LLMProvider

    # 创建同步 MongoDB 客户端
    client = MongoClient(settings.MONGO_URI)
    db = client[settings.MONGO_DB]
    providers_collection = db.llm_providers

    # 查询所有厂家配置
    providers_data = list(providers_collection.find())
    providers = [LLMProvider(**data) for data in providers_data]

    logger.info(f"📊 从数据库读取到 {len(providers)} 个厂家配置")

    for provider in providers:
        if not provider.is_active:
            logger.debug(f"⏭️ 厂家 {provider.name} 未启用，跳过")
            continue
        env_key = f"{provider.name.upper()}_API_KEY"
        existing_env_value = os.getenv(env_key)
        # 检查环境变量是否已存在且有效（不是占位符）
        if existing_env_value and not existing_env_value.startswith("your_"):
            logger.info(f"✓ 使用 .env 文件中的 {env_key} (长度: {len(existing_env_value)})")
            bridged_count += 1
        elif provider.api_key and not provider.api_key.startswith("your_"):
            # 只有当环境变量不存在或为占位符时，才使用数据库配置
            os.environ[env_key] = provider.api_key
            logger.info(f"✓ 使用数据库厂家配置的 {env_key} (长度: {len(provider.api_key)})")
            bridged_count += 1
        else:
            logger.debug(f"⏭️ {env_key} 未配置有效的 API Key")
    # 关闭同步客户端
    client.close()
except Exception as e:
    logger.error(f"❌ 从数据库读取厂家配置失败: {e}", exc_info=True)
    logger.warning("⚠️ 将尝试从 JSON 文件读取配置作为后备方案")
    # 后备方案：从 JSON 文件读取
    llm_configs = unified_config.get_llm_configs()
    for llm_config in llm_configs:
        # provider 现在是字符串类型，不再是枚举
        env_key = f"{llm_config.provider.upper()}_API_KEY"
        existing_env_value = os.getenv(env_key)
        # 检查环境变量是否已存在且有效（不是占位符）
        if existing_env_value and not existing_env_value.startswith("your_"):
            logger.info(f"✓ 使用 .env 文件中的 {env_key} (长度: {len(existing_env_value)})")
            bridged_count += 1
        elif llm_config.enabled and llm_config.api_key:
            # 只有当环境变量不存在或为占位符时，才使用数据库配置
            if not llm_config.api_key.startswith("your_"):
                os.environ[env_key] = llm_config.api_key
                logger.info(f"✓ 使用 JSON 文件中的 {env_key} (长度: {len(llm_config.api_key)})")
                bridged_count += 1
            else:
                logger.warning(f"⚠️ {env_key} 在 .env 和 JSON 文件中都是占位符，跳过")
        else:
            logger.debug(f"⏭️ {env_key} 未配置")
# 2. 桥接默认模型配置
default_model = unified_config.get_default_model()
if default_model:
    os.environ['TRADINGAGENTS_DEFAULT_MODEL'] = default_model
    logger.info(f"✓ 桥接默认模型: {default_model}")
    bridged_count += 1
quick_model = unified_config.get_quick_analysis_model()
if quick_model:
    os.environ['TRADINGAGENTS_QUICK_MODEL'] = quick_model
    logger.info(f"✓ 桥接快速分析模型: {quick_model}")
    bridged_count += 1
deep_model = unified_config.get_deep_analysis_model()
if deep_model:
    os.environ['TRADINGAGENTS_DEEP_MODEL'] = deep_model
    logger.info(f"✓ 桥接深度分析模型: {deep_model}")
    bridged_count += 1
# 3. 桥接数据源配置（基础 API 密钥）
# 🔗 [优先级] .env 文件 > 数据库配置
# 🔥 修改：从数据库的 system_configs 集合读取数据源配置，而不是从 JSON 文件
try:
    # 使用同步 MongoDB 客户端读取系统配置
    from pymongo import MongoClient
    from app.core.config import settings
    from app.models.config import SystemConfig

    # 创建同步 MongoDB 客户端
    client = MongoClient(settings.MONGO_URI)
    db = client[settings.MONGO_DB]
    config_collection = db.system_configs

    # 查询最新的系统配置
    config_data = config_collection.find_one(
        {"is_active": True},
        sort=[("version", -1)]
    )

    if config_data and config_data.get('data_source_configs'):
        system_config = SystemConfig(**config_data)
        data_source_configs = system_config.data_source_configs
        logger.info(f"📊 从数据库读取到 {len(data_source_configs)} 个数据源配置")
    else:
        logger.warning("⚠️ 数据库中没有数据源配置，使用 JSON 文件配置")
        data_source_configs = unified_config.get_data_source_configs()

    # 关闭同步客户端
    client.close()
except Exception as e:
    logger.error(f"❌ 从数据库读取数据源配置失败: {e}", exc_info=True)
    logger.warning("⚠️ 将尝试从 JSON 文件读取配置作为后备方案")
    data_source_configs = unified_config.get_data_source_configs()
for ds_config in data_source_configs:
```

```
        if ds_config.enabled and ds_config.api_key:
            # Tushare Token
            # 🔥 优先级：数据库配置 > .env 文件（用户在 Web 后台修改后立即生效）
            if ds_config.type.value == 'tushare':
                existing_token = os.getenv('TUSHARE_TOKEN')

                # 优先使用数据库配置
                if ds_config.api_key and not ds_config.api_key.startswith("your_"):
                    os.environ['TUSHARE_TOKEN'] = ds_config.api_key
                    logger.info(f"  ✓ 使用数据库中的 TUSHARE_TOKEN (长度: {len(ds_config.api_key)})")
                    if existing_token and existing_token != ds_config.api_key:
                        logger.info(f"  ℹ️  已覆盖 .env 文件中的 TUSHARE_TOKEN")
                # 降级到 .env 文件配置
                elif existing_token and not existing_token.startswith("your_"):
                    logger.info(f"  ✓ 使用 .env 文件中的 TUSHARE_TOKEN (长度: {len(existing_token)})")
                    logger.info(f"  ℹ️  数据库中未配置有效的 TUSHARE_TOKEN，使用 .env 降级方案")
                else:
                    logger.warning(f"  ⚠️  TUSHARE_TOKEN 在数据库和 .env 中都未配置有效值")
                    continue
                bridged_count += 1
            # FinnHub API Key
            # 🔥 优先级：数据库配置 > .env 文件
            elif ds_config.type.value == 'finnhub':
                existing_key = os.getenv('FINNHUB_API_KEY')

                # 优先使用数据库配置
                if ds_config.api_key and not ds_config.api_key.startswith("your_"):
                    os.environ['FINNHUB_API_KEY'] = ds_config.api_key
                    logger.info(f"  ✓ 使用数据库中的 FINNHUB_API_KEY (长度: {len(ds_config.api_key)})")
                    if existing_key and existing_key != ds_config.api_key:
                        logger.info(f"  ℹ️  已覆盖 .env 文件中的 FINNHUB_API_KEY")
                # 降级到 .env 文件配置
                elif existing_key and not existing_key.startswith("your_"):
                    logger.info(f"  ✓ 使用 .env 文件中的 FINNHUB_API_KEY (长度: {len(existing_key)})")
                    logger.info(f"  ℹ️  数据库中未配置有效的 FINNHUB_API_KEY，使用 .env 降级方案")
                else:
                    logger.warning(f"  ⚠️  FINNHUB_API_KEY 在数据库和 .env 中都未配置有效值")
                    continue
                bridged_count += 1
    # 4. 桥接数据源细节配置（超时、重试、缓存等）
    bridged_count += _bridge_datasource_details(data_source_configs)
    # 5. 桥接系统运行时配置
    bridged_count += _bridge_system_settings()
    # 6. 重新初始化 tradingagents 库的 MongoDB 存储
    # 因为全局 config_manager 实例是在模块导入时创建的，那时环境变量还没有被桥接
    try:
        from tradingagents.config.config_manager import config_manager
        from tradingagents.config.mongodb_storage import MongoDBStorage
        logger.info("🔄 重新初始化 tradingagents MongoDB 存储...")
        # 调试：检查环境变量
        use_mongodb = os.getenv("USE_MONGODB_STORAGE", "false")
        mongodb_conn = os.getenv("MONGODB_CONNECTION_STRING", "未设置")
        mongodb_db = os.getenv("MONGODB_DATABASE_NAME", "tradingagents")
        logger.info(f"  📊 USE_MONGODB_STORAGE: {use_mongodb}")
        logger.info(f"  📊 MONGODB_CONNECTION_STRING: {mongodb_conn[:30]}..." if len(mongodb_conn) > 30 else f"  📊 MONGODB_CONNECTION_STRING: {mongodb_conn}")
        logger.info(f"  📊 MONGODB_DATABASE_NAME: {mongodb_db}")
        # 直接创建 MongoDBStorage 实例，而不是调用 _init_mongodb_storage()
        # 这样可以捕获更详细的错误信息
        if use_mongodb.lower() == "true":
            try:
                # 🔍 详细日志：显示完整的连接字符串（用于调试）
                logger.info(f"  🔍 实际传入的连接字符串: {mongodb_conn}")
                logger.info(f"  🔍 实际传入的数据库名称: {mongodb_db}")
                config_manager.mongodb_storage = MongoDBStorage(
                    connection_string=mongodb_conn,
                    database_name=mongodb_db
                )
                if config_manager.mongodb_storage.is_connected():
                    logger.info("✅ tradingagents MongoDB 存储已启用")
                else:
                    logger.warning("⚠️  tradingagents MongoDB 连接失败，将使用 JSON 文件存储")
                    config_manager.mongodb_storage = None
            except Exception as e:
                logger.error(f"❌ 创建 MongoDBStorage 实例失败: {e}")
                import traceback
                logger.error(traceback.format_exc())
                config_manager.mongodb_storage = None
        else:
            logger.info("ℹ️  USE_MONGODB_STORAGE 未启用，将使用 JSON 文件存储")
    except Exception as e:
        logger.error(f"❌ 重新初始化 tradingagents MongoDB 存储失败: {e}")
        import traceback
        logger.error(traceback.format_exc())
    # 7. 同步定价配置到 tradingagents 的 config/pricing.json
    # 注意：这里需要从数据库读取配置，因为文件中的配置没有定价信息
    # 使用异步方式同步定价配置
    import asyncio
    try:
        loop = asyncio.get_running_loop()
        # 在异步上下文中，创建后台任务
        task = loop.create_task(_sync_pricing_config_from_db())
        task.add_done_callback(_handle_sync_task_result)
        logger.info("🔄 定价配置同步任务已创建（后台执行）")
    except RuntimeError:
        # 不在异步上下文中，使用 asyncio.run
        asyncio.run(_sync_pricing_config_from_db())
    logger.info(f"✅ 配置桥接完成，共桥接 {bridged_count} 项配置")
    return True
except Exception as e:
    logger.error(f"❌ 配置桥接失败: {e}", exc_info=True)
    logger.warning("⚠️  TradingAgents 将使用 .env 文件中的配置")
    return False

def _bridge_datasource_details(data_source_configs) -> int:
    """
    桥接数据源细节配置到环境变量

    Args:
```

```
        data_source_configs: 数据源配置列表
    Returns:
        int: 桥接的配置项数量
    """
    bridged_count = 0
    for ds_config in data_source_configs:
        if not ds_config.enabled:
            continue
        # 注意：字段名是 type 而不是 source_type
        source_type = ds_config.type.value.upper()
        # 超时时间
        if ds_config.timeout:
            env_key = f"{source_type}_TIMEOUT"
            os.environ[env_key] = str(ds_config.timeout)
            logger.debug(f"  ✓ 桥接 {env_key}: {ds_config.timeout}")
            bridged_count += 1
        # 速率限制
        if ds_config.rate_limit:
            env_key = f"{source_type}_RATE_LIMIT"
            os.environ[env_key] = str(ds_config.rate_limit / 60.0)  # 转换为每秒请求数
            logger.debug(f"  ✓ 桥接 {env_key}: {ds_config.rate_limit / 60.0}")
            bridged_count += 1
        # 最大重试次数（从 config_params 中获取）
        if ds_config.config_params and 'max_retries' in ds_config.config_params:
            env_key = f"{source_type}_MAX_RETRIES"
            os.environ[env_key] = str(ds_config.config_params['max_retries'])
            logger.debug(f"  ✓ 桥接 {env_key}: {ds_config.config_params['max_retries']}")
            bridged_count += 1
        # 缓存 TTL（从 config_params 中获取）
        if ds_config.config_params and 'cache_ttl' in ds_config.config_params:
            env_key = f"{source_type}_CACHE_TTL"
            os.environ[env_key] = str(ds_config.config_params['cache_ttl'])
            logger.debug(f"  ✓ 桥接 {env_key}: {ds_config.config_params['cache_ttl']}")
            bridged_count += 1
        # 是否启用缓存（从 config_params 中获取）
        if ds_config.config_params and 'cache_enabled' in ds_config.config_params:
            env_key = f"{source_type}_CACHE_ENABLED"
            os.environ[env_key] = str(ds_config.config_params['cache_enabled']).lower()
            logger.debug(f"  ✓ 桥接 {env_key}: {ds_config.config_params['cache_enabled']}")
            bridged_count += 1
    if bridged_count > 0:
        logger.info(f"  ✓ 桥接数据源细节配置: {bridged_count} 项")
    return bridged_count
def _bridge_system_settings() -> int:
    """
    桥接系统运行时配置到环境变量
    Returns:
        int: 桥接的配置项数量
    """
    try:
        # 使用同步的 MongoDB 客户端
        from pymongo import MongoClient
        from app.core.config import settings
        # 创建同步客户端
        client = MongoClient(
            settings.MONGO_URI,
            serverSelectionTimeoutMS=5000,
            connectTimeoutMS=5000
        )
        try:
            db = client[settings.MONGO_DB]
            # 从 system_configs 集合中读取激活的配置
            config_doc = db.system_configs.find_one({"is_active": True})
            if not config_doc or 'system_settings' not in config_doc:
                logger.debug("  ⚠️  系统设置为空，跳过桥接")
                return 0
            system_settings = config_doc['system_settings']
        except Exception as e:
            logger.debug(f"  ⚠️  无法从数据库获取系统设置: {e}")
            import traceback
            logger.debug(traceback.format_exc())
            return 0
        finally:
            client.close()
        if not system_settings:
            logger.debug("  ⚠️  系统设置为空，跳过桥接")
            return 0
        logger.debug(f"  📦 获取到 {len(system_settings)} 个系统设置")
        bridged_count = 0
        # TradingAgents 运行时配置
        ta_settings = {
            'ta_hk_min_request_interval_seconds': 'TA_HK_MIN_REQUEST_INTERVAL_SECONDS',
            'ta_hk_timeout_seconds': 'TA_HK_TIMEOUT_SECONDS',
            'ta_hk_max_retries': 'TA_HK_MAX_RETRIES',
            'ta_hk_rate_limit_wait_seconds': 'TA_HK_RATE_LIMIT_WAIT_SECONDS',
            'ta_hk_cache_ttl_seconds': 'TA_HK_CACHE_TTL_SECONDS',
            'ta_use_app_cache': 'TA_USE_APP_CACHE',
        }
        # Token 使用统计配置
        token_tracking_settings = {
            'enable_cost_tracking': 'ENABLE_COST_TRACKING',
            'auto_save_usage': 'AUTO_SAVE_USAGE',
        }
        for setting_key, env_key in ta_settings.items():
            # 检查 .env 文件中是否已经设置了该环境变量
            env_value = os.getenv(env_key)
            if env_value is not None:
                # .env 文件中已设置，优先使用 .env 的值
                logger.info(f"  ✓ 使用 .env 文件中的 {env_key}: {env_value}")
                bridged_count += 1
            elif setting_key in system_settings:
                # .env 文件中未设置，使用数据库中的值
```

```
            value = system_settings[setting_key]
            os.environ[env_key] = str(value).lower() if isinstance(value, bool) else str(value)
            logger.info(f"  ✓ 桥接 {env_key}: {value}")
            bridged_count += 1
        else:
            logger.debug(f"  ⚠️  配置键 {setting_key} 不存在于系统设置中")
    # 桥接 Token 使用统计配置
    for setting_key, env_key in token_tracking_settings.items():
        if setting_key in system_settings:
            value = system_settings[setting_key]
            os.environ[env_key] = str(value).lower() if isinstance(value, bool) else str(value)
            logger.info(f"  ✓ 桥接 {env_key}: {value}")
            bridged_count += 1
        else:
            logger.debug(f"  ⚠️  配置键 {setting_key} 不存在于系统设置中")
    # 时区配置
    if 'app_timezone' in system_settings:
        os.environ['APP_TIMEZONE'] = system_settings['app_timezone']
        logger.debug(f"  ✓ 桥接 APP_TIMEZONE: {system_settings['app_timezone']}")
        bridged_count += 1
    # 货币偏好
    if 'currency_preference' in system_settings:
        os.environ['CURRENCY_PREFERENCE'] = system_settings['currency_preference']
        logger.debug(f"  ✓ 桥接 CURRENCY_PREFERENCE: {system_settings['currency_preference']}")
        bridged_count += 1
    if bridged_count > 0:
        logger.info(f"  ✓ 桥接系统运行时配置: {bridged_count} 项")
    # 同步到文件系统（供 unified_config 使用）
    try:
        print(f"🔄 [config_bridge] 准备同步系统设置到文件系统")
        print(f"🔄 [config_bridge] system_settings 包含 {len(system_settings)} 项")
        # 检查关键字段
        if "quick_analysis_model" in system_settings:
            print(f"  ✓ [config_bridge] 包含 quick_analysis_model: {system_settings['quick_analysis_model']}")
        else:
            print(f"  ⚠️  [config_bridge] 不包含 quick_analysis_model")
        if "deep_analysis_model" in system_settings:
            print(f"  ✓ [config_bridge] 包含 deep_analysis_model: {system_settings['deep_analysis_model']}")
        else:
            print(f"  ⚠️  [config_bridge] 不包含 deep_analysis_model")
        from app.core.unified_config import unified_config
        result = unified_config.save_system_settings(system_settings)
        if result:
            logger.info(f"  ✓ 系统设置已同步到文件系统")
            print(f"✅ [config_bridge] 系统设置同步成功")
        else:
            logger.warning(f"  ⚠️  系统设置同步返回 False")
            print(f"⚠️  [config_bridge] 系统设置同步返回 False")
    except Exception as e:
        logger.warning(f"  ⚠️  同步系统设置到文件系统失败: {e}")
        print(f"❌ [config_bridge] 同步系统设置到文件系统失败: {e}")
        import traceback
        print(traceback.format_exc())
    return bridged_count
  except Exception as e:
    logger.warning(f"  ⚠️  桥接系统设置失败: {e}")
    return 0
def get_bridged_api_key(provider: str) -> Optional[str]:
  """
  获取桥接的 API 密钥

  Args:
    provider: 提供商名称 (如: openai, deepseek, dashscope)

  Returns:
    API 密钥，如果不存在返回 None
  """
  env_key = f"{provider.upper()}_API_KEY"
  return os.environ.get(env_key)
def get_bridged_model(model_type: str = "default") -> Optional[str]:
  """
  获取桥接的模型名称

  Args:
    model_type: 模型类型 (default, quick, deep)

  Returns:
    模型名称，如果不存在返回 None
  """
  if model_type == "quick":
    return os.environ.get('TRADINGAGENTS_QUICK_MODEL')
  elif model_type == "deep":
    return os.environ.get('TRADINGAGENTS_DEEP_MODEL')
  else:
    return os.environ.get('TRADINGAGENTS_DEFAULT_MODEL')
def clear_bridged_config():
  """
  清除桥接的配置
  用于测试或重新加载配置
  """
  keys_to_clear = [
    # 模型配置
    'TRADINGAGENTS_DEFAULT_MODEL',
    'TRADINGAGENTS_QUICK_MODEL',
    'TRADINGAGENTS_DEEP_MODEL',
    # 数据源 API 密钥
    'TUSHARE_TOKEN',
    'FINNHUB_API_KEY',
    # 系统配置
    'APP_TIMEZONE',
    'CURRENCY_PREFERENCE',
  ]
  # 清除所有可能的 API 密钥
```

```
    providers = ['OPENAI', 'ANTHROPIC', 'GOOGLE', 'DEEPSEEK', 'DASHSCOPE', 'QIANFAN']
    for provider in providers:
        keys_to_clear.append(f'{provider}_API_KEY')
    # 清除数据源细节配置
    data_sources = ['TUSHARE', 'AKSHARE', 'FINNHUB']
    for ds in data_sources:
        keys_to_clear.extend([
            f'{ds}_TIMEOUT',
            f'{ds}_RATE_LIMIT',
            f'{ds}_MAX_RETRIES',
            f'{ds}_CACHE_TTL',
            f'{ds}_CACHE_ENABLED',
        ])
    # 清除 TradingAgents 运行时配置
    ta_runtime_keys = [
        'TA_HK_MIN_REQUEST_INTERVAL_SECONDS',
        'TA_HK_TIMEOUT_SECONDS',
        'TA_HK_MAX_RETRIES',
        'TA_HK_RATE_LIMIT_WAIT_SECONDS',
        'TA_HK_CACHE_TTL_SECONDS',
        'TA_USE_APP_CACHE',
    ]
    keys_to_clear.extend(ta_runtime_keys)
    for key in keys_to_clear:
        if key in os.environ:
            del os.environ[key]
            logger.debug(f"  清除环境变量: {key}")
    logger.info("✅ 已清除所有桥接的配置")
def reload_bridged_config():
    """
    重新加载桥接的配置
    用于配置更新后重新桥接
    """
    logger.info("🔄 重新加载配置桥接...")
    clear_bridged_config()
    return bridge_config_to_env()
def _sync_pricing_config(llm_configs):
    """
    同步定价配置到 tradingagents 的 config/pricing.json
    Args:
        llm_configs: LLM 配置列表
    """
    try:
        # 获取项目根目录的 config 目录
        project_root = Path(__file__).parent.parent.parent
        config_dir = project_root / "config"
        config_dir.mkdir(exist_ok=True)
        pricing_file = config_dir / "pricing.json"
        # 构建定价配置列表
        pricing_configs = []
        for llm_config in llm_configs:
            if llm_config.enabled:
                pricing_config = {
                    # provider 现在是字符串类型，不再是枚举
                    "provider": llm_config.provider,
                    "model_name": llm_config.model_name,
                    "input_price_per_1k": llm_config.input_price_per_1k or 0.0,
                    "output_price_per_1k": llm_config.output_price_per_1k or 0.0,
                    "currency": llm_config.currency or "CNY"
                }
                pricing_configs.append(pricing_config)
        # 保存到文件
        with open(pricing_file, 'w', encoding='utf-8') as f:
            json.dump(pricing_configs, f, ensure_ascii=False, indent=2)
        logger.info(f"  ✓ 同步定价配置到 {pricing_file}: {len(pricing_configs)} 个模型")
    except Exception as e:
        logger.warning(f"  ⚠️  同步定价配置失败: {e}")
def sync_pricing_config_now():
    """
    立即同步定价配置（用于配置更新后实时同步）
    注意：这个函数会在后台异步执行同步操作
    """
    import asyncio
    try:
        # 如果在异步上下文中，创建后台任务
        try:
            loop = asyncio.get_running_loop()
            # 在异步上下文中，创建一个后台任务（不等待完成）
            task = loop.create_task(_sync_pricing_config_from_db())
            # 添加回调来记录错误
            task.add_done_callback(_handle_sync_task_result)
            logger.info("🔄 定价配置同步任务已创建（后台执行）")
            return True
        except RuntimeError:
            # 不在异步上下文中，使用 asyncio.run
            asyncio.run(_sync_pricing_config_from_db())
            return True
    except Exception as e:
        logger.error(f"❌ 立即同步定价配置失败: {e}")
        import traceback
        logger.error(traceback.format_exc())
        return False
def _handle_sync_task_result(task):
    """处理同步任务的结果"""
    try:
        task.result()
    except Exception as e:
        logger.error(f"❌ 定价配置同步任务执行失败: {e}")
        import traceback
        logger.error(traceback.format_exc())
async def _sync_pricing_config_from_db():
    """
```

```
    从数据库同步定价配置（异步版本）
    """
    try:
        from app.core.database import get_mongo_db
        from app.models.config import LLMConfig
        db = get_mongo_db()
        # 获取最新的激活配置
        config = await db['system_configs'].find_one(
            {'is_active': True},
            sort=[('version', -1)]
        )
        if not config:
            logger.warning("⚠️ 未找到激活的配置")
            return
        # 获取项目根目录的 config 目录
        project_root = Path(__file__).parent.parent.parent
        config_dir = project_root / "config"
        config_dir.mkdir(exist_ok=True)
        pricing_file = config_dir / "pricing.json"
        # 构建定价配置列表
        pricing_configs = []
        for llm_config in config.get('llm_configs', []):
            if llm_config.get('enabled', False):
                # 从数据库读取的是字典，直接使用字符串 provider
                provider = llm_config.get('provider')
                # 如果 provider 是枚举类型，转换为字符串
                if hasattr(provider, 'value'):
                    provider = provider.value
                pricing_config = {
                    "provider": provider,
                    "model_name": llm_config.get('model_name'),
                    "input_price_per_1k": llm_config.get('input_price_per_1k') or 0.0,
                    "output_price_per_1k": llm_config.get('output_price_per_1k') or 0.0,
                    "currency": llm_config.get('currency') or "CNY"
                }
                pricing_configs.append(pricing_config)
        # 保存到文件
        with open(pricing_file, 'w', encoding='utf-8') as f:
            json.dump(pricing_configs, f, ensure_ascii=False, indent=2)
        logger.info(f"✅ 同步定价配置到 {pricing_file}: {len(pricing_configs)} 个模型")
    except Exception as e:
        logger.error(f"❌ 从数据库同步定价配置失败: {e}")
        import traceback
        logger.error(traceback.format_exc())

# 导出函数
__all__ = [
    'bridge_config_to_env',
    'get_bridged_api_key',
    'get_bridged_model',
    'clear_bridged_config',
    'reload_bridged_config',
    'sync_pricing_config_now',
]
```

FILE: app/core/config_compat.py

```
"""
配置系统兼容层
为旧的 tradingagents 库提供配置兼容接口，
使其能够使用新的配置系统而无需修改代码。
⚠️ 此模块仅用于向后兼容，新代码应直接使用 ConfigService
"""
import os
import asyncio
from typing import Dict, Any, Optional, List
from functools import lru_cache
import warnings
from app.core.config import settings

class ConfigManagerCompat:
    """
    ConfigManager 兼容类

    提供与旧 ConfigManager 相同的接口，但使用新的配置系统。
    """

    def __init__(self):
        """初始化兼容层"""
        self._warned = False
        self._emit_deprecation_warning()

    def _emit_deprecation_warning(self):
        """发出废弃警告（仅一次）"""
        if not self._warned:
            warnings.warn(
                "ConfigManagerCompat is a compatibility layer for legacy code. "
                "Please migrate to app.services.config_service.ConfigService. "
                "See docs/DEPRECATION_NOTICE.md for details.",
                DeprecationWarning,
                stacklevel=3
            )
            self._warned = True

    def get_data_dir(self) -> str:
        """
        获取数据目录

        Returns:
            str: 数据目录路径
        """
        # 优先从环境变量读取
        data_dir = os.getenv("DATA_DIR")
```

```
    if data_dir:
        return data_dir

    # 默认值
    return "./data"

def load_settings(self) -> Dict[str, Any]:
    """
    加载系统设置

    Returns:
        Dict[str, Any]: 系统设置字典
    """
    try:
        # 尝试从新配置系统加载
        from app.services.config_service import config_service

        # 在同步上下文中运行异步代码
        loop = asyncio.get_event_loop()
        if loop.is_running():
            # 如果事件循环正在运行，返回默认值
            return self._get_default_settings()
        else:
            config = loop.run_until_complete(config_service.get_system_config())
            if config and config.system_settings:
                return config.system_settings
    except Exception:
        pass

    # 返回默认设置
    return self._get_default_settings()

def save_settings(self, settings_dict: Dict[str, Any]) -> bool:
    """
    保存系统设置

    Args:
        settings_dict: 系统设置字典

    Returns:
        bool: 是否保存成功
    """
    try:
        from app.services.config_service import config_service

        loop = asyncio.get_event_loop()
        if loop.is_running():
            # 如果事件循环正在运行，无法保存
            warnings.warn("Cannot save settings in running event loop", RuntimeWarning)
            return False
        else:
            loop.run_until_complete(
                config_service.update_system_settings(settings_dict)
            )
            return True
    except Exception as e:
        warnings.warn(f"Failed to save settings: {e}", RuntimeWarning)
        return False

def get_models(self) -> List[Dict[str, Any]]:
    """
    获取模型配置列表

    Returns:
        List[Dict[str, Any]]: 模型配置列表
    """
    try:
        from app.services.config_service import config_service

        loop = asyncio.get_event_loop()
        if loop.is_running():
            return []
        else:
            config = loop.run_until_complete(config_service.get_system_config())
            if config and config.llm_configs:
                return [
                    {
                        "provider": llm.provider,
                        "model_name": llm.model_name,
                        "api_key": llm.api_key or "",
                        "base_url": llm.base_url,
                        "max_tokens": llm.max_tokens,
                        "temperature": llm.temperature,
                        "enabled": llm.enabled,
                    }
                    for llm in config.llm_configs
                ]
    except Exception:
        pass

    return []

def get_model_config(self, provider: str, model_name: str) -> Optional[Dict[str, Any]]:
    """
    获取指定模型的配置

    Args:
        provider: 提供商名称
        model_name: 模型名称

    Returns:
        Optional[Dict[str, Any]]: 模型配置，如果不存在则返回 None
    """
    models = self.get_models()
    for model in models:
        if model["provider"] == provider and model["model_name"] == model_name:
```

```
            return model
        return None

    def _get_default_settings(self) -> Dict[str, Any]:
        """
        获取默认系统设置

        Returns:
            Dict[str, Any]: 默认设置
        """
        return {
            "max_debate_rounds": 1,
            "max_risk_discuss_rounds": 1,
            "online_tools": True,
            "online_news": True,
            "realtime_data": False,
            "memory_enabled": True,
            "debug": False,
        }
class TokenTrackerCompat:
    """
    TokenTracker 兼容类

    提供与旧 TokenTracker 相同的接口。
    """

    def __init__(self):
        """初始化兼容层"""
        self._usage_data = {}

    def track_usage(
        self,
        provider: str,
        model_name: str,
        input_tokens: int,
        output_tokens: int,
        cost: float = 0.0
    ):
        """
        记录 Token 使用量

        Args:
            provider: 提供商名称
            model_name: 模型名称
            input_tokens: 输入 Token 数
            output_tokens: 输出 Token 数
            cost: 成本
        """
        key = f"{provider}:{model_name}"

        if key not in self._usage_data:
            self._usage_data[key] = {
                "provider": provider,
                "model_name": model_name,
                "total_input_tokens": 0,
                "total_output_tokens": 0,
                "total_cost": 0.0,
                "call_count": 0,
            }

        self._usage_data[key]["total_input_tokens"] += input_tokens
        self._usage_data[key]["total_output_tokens"] += output_tokens
        self._usage_data[key]["total_cost"] += cost
        self._usage_data[key]["call_count"] += 1
        # 注意：此兼容层仅提供内存缓存，不持久化到数据库
        # 如需持久化，请使用 app.services.llm_service 中的相关功能

    def get_usage_summary(self) -> Dict[str, Any]:
        """
        获取使用统计摘要

        Returns:
            Dict[str, Any]: 使用统计摘要
        """
        return self._usage_data.copy()

    def reset_usage(self):
        """重置使用统计"""
        self._usage_data.clear()
# 创建全局实例（用于向后兼容）
config_manager_compat = ConfigManagerCompat()
token_tracker_compat = TokenTrackerCompat()

# 便捷函数
def get_config_manager() -> ConfigManagerCompat:
    """
    获取配置管理器兼容实例

    Returns:
        ConfigManagerCompat: 配置管理器兼容实例
    """
    return config_manager_compat

def get_token_tracker() -> TokenTrackerCompat:
    """
    获取 Token 跟踪器兼容实例

    Returns:
        TokenTrackerCompat: Token 跟踪器兼容实例
    """
    return token_tracker_compat
```

FILE: app/core/database.py

```
"""
```

```
数据库连接管理模块
增强版本，支持连接池、健康检查和错误恢复
"""
import logging
import asyncio
from typing import Optional
from motor.motor_asyncio import AsyncIOMotorClient, AsyncIOMotorDatabase
from pymongo import MongoClient
from pymongo.database import Database
from redis.asyncio import Redis, ConnectionPool
from pymongo.errors import ServerSelectionTimeoutError, ConnectionFailure
from redis.exceptions import ConnectionError as RedisConnectionError
from .config import settings

logger = logging.getLogger(__name__)

# 全局连接实例
mongo_client: Optional[AsyncIOMotorClient] = None
mongo_db: Optional[AsyncIOMotorDatabase] = None
redis_client: Optional[Redis] = None
redis_pool: Optional[ConnectionPool] = None

# 同步 MongoDB 连接（用于非异步上下文）
_sync_mongo_client: Optional[MongoClient] = None
_sync_mongo_db: Optional[Database] = None

class DatabaseManager:
    """数据库连接管理器"""
    def __init__(self):
        self.mongo_client: Optional[AsyncIOMotorClient] = None
        self.mongo_db: Optional[AsyncIOMotorDatabase] = None
        self.redis_client: Optional[Redis] = None
        self.redis_pool: Optional[ConnectionPool] = None
        self._mongo_healthy = False
        self._redis_healthy = False
    async def init_mongodb(self):
        """初始化MongoDB连接"""
        try:
            logger.info("🔄 正在初始化MongoDB连接...")
            # 创建MongoDB客户端，配置连接池
            self.mongo_client = AsyncIOMotorClient(
                settings.MONGO_URI,
                maxPoolSize=settings.MONGO_MAX_CONNECTIONS,
                minPoolSize=settings.MONGO_MIN_CONNECTIONS,
                maxIdleTimeMS=30000,  # 30秒空闲超时
                serverSelectionTimeoutMS=settings.MONGO_SERVER_SELECTION_TIMEOUT_MS,  # 服务器选择超时
                connectTimeoutMS=settings.MONGO_CONNECT_TIMEOUT_MS,  # 连接超时
                socketTimeoutMS=settings.MONGO_SOCKET_TIMEOUT_MS,  # 套接字超时
            )
            # 获取数据库实例
            self.mongo_db = self.mongo_client[settings.MONGO_DB]
            # 测试连接
            await self.mongo_client.admin.command('ping')
            self._mongo_healthy = True
            logger.info("✅ MongoDB连接成功建立")
            logger.info(f"📊 数据库: {settings.MONGO_DB}")
            logger.info(f"🔗 连接池: {settings.MONGO_MIN_CONNECTIONS}-{settings.MONGO_MAX_CONNECTIONS}")
            logger.info(f"⏱️ 超时配置: connectTimeout={settings.MONGO_CONNECT_TIMEOUT_MS}ms, socketTimeout={settings.MONGO_SOCKET_TIMEOUT_MS}ms")
        except Exception as e:
            logger.error(f"❌ MongoDB连接失败: {e}")
            self._mongo_healthy = False
            raise
    async def init_redis(self):
        """初始化Redis连接"""
        try:
            logger.info("🔄 正在初始化Redis连接...")
            # 创建Redis连接池
            self.redis_pool = ConnectionPool.from_url(
                settings.REDIS_URL,
                max_connections=settings.REDIS_MAX_CONNECTIONS,
                retry_on_timeout=settings.REDIS_RETRY_ON_TIMEOUT,
                decode_responses=True,
                socket_connect_timeout=5,  # 5秒连接超时
                socket_timeout=10,  # 10秒套接字超时
            )
            # 创建Redis客户端
            self.redis_client = Redis(connection_pool=self.redis_pool)
            # 测试连接
            await self.redis_client.ping()
            self._redis_healthy = True
            logger.info("✅ Redis连接成功建立")
            logger.info(f"🔗 连接池大小: {settings.REDIS_MAX_CONNECTIONS}")
        except Exception as e:
            logger.error(f"❌ Redis连接失败: {e}")
            self._redis_healthy = False
            raise
    async def close_connections(self):
        """关闭所有数据库连接"""
        logger.info("🔄 正在关闭数据库连接...")
        # 关闭MongoDB连接
        if self.mongo_client:
            try:
                self.mongo_client.close()
                self._mongo_healthy = False
                logger.info("✅ MongoDB连接已关闭")
            except Exception as e:
                logger.error(f"❌ 关闭MongoDB连接时出错: {e}")
        # 关闭Redis连接
        if self.redis_client:
            try:
                await self.redis_client.close()
                self._redis_healthy = False
                logger.info("✅ Redis连接已关闭")
            except Exception as e:
                logger.error(f"❌ 关闭Redis连接时出错: {e}")
        # 关闭Redis连接池
```

```
        if self.redis_pool:
            try:
                await self.redis_pool.disconnect()
                logger.info("✅ Redis连接池已关闭")
            except Exception as e:
                logger.error(f"❌ 关闭Redis连接池时出错: {e}")
    async def health_check(self) -> dict:
        """数据库健康检查"""
        health_status = {
            "mongodb": {"status": "unknown", "details": None},
            "redis": {"status": "unknown", "details": None}
        }
        # 检查MongoDB
        try:
            if self.mongo_client:
                result = await self.mongo_client.admin.command('ping')
                health_status["mongodb"] = {
                    "status": "healthy",
                    "details": {"ping": result, "database": settings.MONGO_DB}
                }
                self._mongo_healthy = True
            else:
                health_status["mongodb"]["status"] = "disconnected"
        except Exception as e:
            health_status["mongodb"] = {
                "status": "unhealthy",
                "details": {"error": str(e)}
            }
            self._mongo_healthy = False
        # 检查Redis
        try:
            if self.redis_client:
                result = await self.redis_client.ping()
                health_status["redis"] = {
                    "status": "healthy",
                    "details": {"ping": result}
                }
                self._redis_healthy = True
            else:
                health_status["redis"]["status"] = "disconnected"
        except Exception as e:
            health_status["redis"] = {
                "status": "unhealthy",
                "details": {"error": str(e)}
            }
            self._redis_healthy = False
        return health_status
    @property
    def is_healthy(self) -> bool:
        """检查所有数据库连接是否健康"""
        return self._mongo_healthy and self._redis_healthy
# 全局数据库管理器实例
db_manager = DatabaseManager()
async def init_database():
    """初始化数据库连接"""
    global mongo_client, mongo_db, redis_client, redis_pool
    try:
        # 初始化MongoDB
        await db_manager.init_mongodb()
        mongo_client = db_manager.mongo_client
        mongo_db = db_manager.mongo_db
        # 初始化Redis
        await db_manager.init_redis()
        redis_client = db_manager.redis_client
        redis_pool = db_manager.redis_pool
        logger.info("🎉 所有数据库连接初始化完成")
        # 🔥 初始化数据库视图和索引
        await init_database_views_and_indexes()
    except Exception as e:
        logger.error(f"💥 数据库初始化失败: {e}")
        raise
async def init_database_views_and_indexes():
    """初始化数据库视图和索引"""
    try:
        db = get_mongo_db()
        # 1. 创建股票筛选视图
        await create_stock_screening_view(db)
        # 2. 创建必要的索引
        await create_database_indexes(db)
        logger.info("✅ 数据库视图和索引初始化完成")
    except Exception as e:
        logger.warning(f"⚠️ 数据库视图和索引初始化失败: {e}")
        # 不抛出异常，允许应用继续启动
async def create_stock_screening_view(db):
    """创建股票筛选视图"""
    try:
        # 检查视图是否已存在
        collections = await db.list_collection_names()
        if "stock_screening_view" in collections:
            logger.info("📋 视图 stock_screening_view 已存在，跳过创建")
            return
        # 创建视图：将 stock_basic_info、market_quotes 和 stock_financial_data 关联
        pipeline = [
            # 第一步：关联实时行情数据 (market_quotes)
            {
                "$lookup": {
                    "from": "market_quotes",
                    "localField": "code",
                    "foreignField": "code",
                    "as": "quote_data"
                }
            },
```

```
        # 第二步：展开 quote_data 数组
        {
            "$unwind": {
                "path": "$quote_data",
                "preserveNullAndEmptyArrays": True
            }
        },
        # 第三步：关联财务数据 (stock_financial_data)
        {
            "$lookup": {
                "from": "stock_financial_data",
                "let": {"stock_code": "$code", "stock_source": "$source"},
                "pipeline": [
                    {
                        "$match": {
                            "$expr": {
                                "$and": [
                                    {"$eq": ["$code", "$$stock_code"]},
                                    {"$eq": ["$data_source", "$$stock_source"]}
                                ]
                            }
                        }
                    },
                    {"$sort": {"report_period": -1}},
                    {"$limit": 1}
                ],
                "as": "financial_data"
            }
        },
        # 第四步：展开 financial_data 数组
        {
            "$unwind": {
                "path": "$financial_data",
                "preserveNullAndEmptyArrays": True
            }
        },
        # 第五步：重新组织字段结构
        {
            "$project": {
                # 基础信息字段
                "code": 1,
                "name": 1,
                "industry": 1,
                "area": 1,
                "market": 1,
                "list_date": 1,
                "source": 1,
                # 市值信息
                "total_mv": 1,
                "circ_mv": 1,
                # 估值指标
                "pe": 1,
                "pb": 1,
                "pe_ttm": 1,
                "pb_mrq": 1,
                # 财务指标
                "roe": "$financial_data.roe",
                "roa": "$financial_data.roa",
                "netprofit_margin": "$financial_data.netprofit_margin",
                "gross_margin": "$financial_data.gross_margin",
                "report_period": "$financial_data.report_period",
                # 交易指标
                "turnover_rate": 1,
                "volume_ratio": 1,
                # 实时行情数据
                "close": "$quote_data.close",
                "open": "$quote_data.open",
                "high": "$quote_data.high",
                "low": "$quote_data.low",
                "pre_close": "$quote_data.pre_close",
                "pct_chg": "$quote_data.pct_chg",
                "amount": "$quote_data.amount",
                "volume": "$quote_data.volume",
                "trade_date": "$quote_data.trade_date",
                # 时间戳
                "updated_at": 1,
                "quote_updated_at": "$quote_data.updated_at",
                "financial_updated_at": "$financial_data.updated_at"
            }
        }
    ]
    # 创建视图
    await db.command({
        "create": "stock_screening_view",
        "viewOn": "stock_basic_info",
        "pipeline": pipeline
    })
    logger.info("✅ 视图 stock_screening_view 创建成功")
except Exception as e:
    logger.warning(f"⚠️ 创建视图失败: {e}")
async def create_database_indexes(db):
    """创建数据库索引"""
    try:
        # stock_basic_info 的索引
        basic_info = db["stock_basic_info"]
        await basic_info.create_index([("code", 1), ("source", 1)], unique=True)
        await basic_info.create_index([("industry", 1)])
        await basic_info.create_index([("total_mv", -1)])
        await basic_info.create_index([("pe", 1)])
        await basic_info.create_index([("pb", 1)])
        # market_quotes 的索引
        market_quotes = db["market_quotes"]
        await market_quotes.create_index([("code", 1)], unique=True)
        await market_quotes.create_index([("pct_chg", -1)])
        await market_quotes.create_index([("amount", -1)])
```

```
        await market_quotes.create_index([("updated_at", -1)])
        logger.info("✅ 数据库索引创建完成")
    except Exception as e:
        logger.warning(f"⚠️ 创建索引失败: {e}")
async def close_database():
    """关闭数据库连接"""
    global mongo_client, mongo_db, redis_client, redis_pool
    await db_manager.close_connections()
    # 清空全局变量
    mongo_client = None
    mongo_db = None
    redis_client = None
    redis_pool = None
def get_mongo_client() -> AsyncIOMotorClient:
    """获取MongoDB客户端"""
    if mongo_client is None:
        raise RuntimeError("MongoDB客户端未初始化")
    return mongo_client
def get_mongo_db() -> AsyncIOMotorDatabase:
    """获取MongoDB数据库实例"""
    if mongo_db is None:
        raise RuntimeError("MongoDB数据库未初始化")
    return mongo_db
def get_mongo_db_sync() -> Database:
    """
    获取同步版本的MongoDB数据库实例
    用于非异步上下文（如普通函数调用）
    """
    global _sync_mongo_client, _sync_mongo_db
    if _sync_mongo_db is not None:
        return _sync_mongo_db
    # 创建同步 MongoDB 客户端
    if _sync_mongo_client is None:
        _sync_mongo_client = MongoClient(
            settings.MONGO_URI,
            maxPoolSize=settings.MONGO_MAX_CONNECTIONS,
            minPoolSize=settings.MONGO_MIN_CONNECTIONS,
            maxIdleTimeMS=30000,
            serverSelectionTimeoutMS=5000
        )
    _sync_mongo_db = _sync_mongo_client[settings.MONGO_DB]
    return _sync_mongo_db
def get_redis_client() -> Redis:
    """获取Redis客户端"""
    if redis_client is None:
        raise RuntimeError("Redis客户端未初始化")
    return redis_client
async def get_database_health() -> dict:
    """获取数据库健康状态"""
    return await db_manager.health_check()
# 兼容性别名
init_db = init_database
close_db = close_database
def get_database():
    """获取数据库实例"""
    if db_manager.mongo_client is None:
        raise RuntimeError("MongoDB客户端未初始化")
    return db_manager.mongo_client.tradingagents
```

FILE: app/core/dev_config.py

```
"""
开发环境配置
优化开发体验，减少不必要的文件监控
"""
import logging
from typing import List, Optional
class DevConfig:
    """开发环境配置类"""

    # 文件监控配置
    RELOAD_DIRS: List[str] = ["app"]

    # 排除的文件和目录
    RELOAD_EXCLUDES: List[str] = [
        # Python缓存文件
        "__pycache__",
        "*.pyc",
        "*.pyo",
        "*.pyd",

        # 版本控制
        ".git",
        ".gitignore",

        # 测试和缓存
        ".pytest_cache",
        ".coverage",
        "htmlcov",

        # 日志文件
        "*.log",
        "logs",

        # 临时文件
        "*.tmp",
        "*.temp",
        "*.swp",
        "*.swo",
```

```
        # 系统文件
        ".DS_Store",
        "Thumbs.db",
        "desktop.ini",

        # IDE文件
        ".vscode",
        ".idea",
        "*.sublime-*",

        # 数据文件
        "*.db",
        "*.sqlite",
        "*.sqlite3",

        # 配置文件（避免敏感信息重载）
        ".env",
        ".env.local",
        ".env.production",

        # 文档和静态文件
        "*.md",
        "*.txt",
        "*.json",
        "*.yaml",
        "*.yml",
        "*.toml",

        # 前端文件
        "node_modules",
        "dist",
        "build",
        "*.js",
        "*.css",
        "*.html",

        # 其他
        "requirements*.txt",
        "Dockerfile*",
        "docker-compose*"
    ]

    # 只监控的文件类型
    RELOAD_INCLUDES: List[str] = [
        "*.py"
    ]

    # 重载延迟（秒）
    RELOAD_DELAY: float = 0.5

    # 日志配置
    LOG_LEVEL: str = "info"

    # 是否显示访问日志
    ACCESS_LOG: bool = True

    @classmethod
    def get_uvicorn_config(cls, debug: bool = True) -> dict:
        """获取uvicorn配置"""
        # 统一禁用reload，避免日志配置冲突
        return {
            "reload": False,  # 禁用自动重载，手动重启
            "log_level": cls.LOG_LEVEL,
            "access_log": cls.ACCESS_LOG,
            # 确保使用我们自定义的日志配置
            "log_config": None  # 禁用uvicorn默认日志配置，使用我们的配置
        }

    @classmethod
    def setup_logging(cls, debug: bool = True):
        """设置简化的日志配置"""
        # 设置统一的日志格式
        logging.basicConfig(
            level=logging.INFO,
            format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
            datefmt='%Y-%m-%d %H:%M:%S',
            force=True  # 强制重新配置，覆盖之前的设置
        )
        if debug:
            # 开发环境：减少噪音日志
            logging.getLogger("watchfiles").setLevel(logging.ERROR)
            logging.getLogger("watchfiles.main").setLevel(logging.ERROR)
            logging.getLogger("watchfiles.watcher").setLevel(logging.ERROR)
            # 确保重要日志正常显示
            logging.getLogger("webapi").setLevel(logging.INFO)
            logging.getLogger("app.core.database").setLevel(logging.INFO)
            logging.getLogger("uvicorn.error").setLevel(logging.INFO)
            # 测试webapi logger是否工作
            webapi_logger = logging.getLogger("webapi")
            webapi_logger.info("🔧 DEV_CONFIG: webapi logger 测试消息")
        else:
            # 生产环境：更严格的日志控制
            logging.getLogger("watchfiles").setLevel(logging.ERROR)
            logging.getLogger("uvicorn").setLevel(logging.WARNING)
# 开发环境快捷配置
DEV_CONFIG = DevConfig()
```

FILE: app/core/logging_config.py

```
import logging
import logging.config
import sys
from pathlib import Path
import os
```

```python
import platform
from app.core.logging_context import LoggingContextFilter, trace_id_var
# 🔥 在 Windows 上使用 concurrent-log-handler 避免文件占用问题
_IS_WINDOWS = platform.system() == "Windows"
if _IS_WINDOWS:
    try:
        from concurrent_log_handler import ConcurrentRotatingFileHandler
        _USE_CONCURRENT_HANDLER = True
    except ImportError:
        _USE_CONCURRENT_HANDLER = False
        logging.warning("concurrent-log-handler 未安装，在 Windows 上可能遇到日志轮转问题")
else:
    _USE_CONCURRENT_HANDLER = False
try:
    import tomllib as toml_loader  # Python 3.11+
except Exception:
    try:
        import tomli as toml_loader  # Python 3.10 fallback
    except Exception:
        toml_loader = None
def resolve_logging_cfg_path() -> Path:
    """根据环境选择日志配置文件路径（可能不存在）
    优先 docker 配置，其次默认配置。
    """
    profile = os.environ.get("LOGGING_PROFILE", "").lower()
    is_docker_env = os.environ.get("DOCKER", "").lower() in {"1", "true", "yes"} or Path("/.dockerenv").exists()
    cfg_candidate = "config/logging_docker.toml" if profile == "docker" or is_docker_env else "config/logging.toml"
    return Path(cfg_candidate)
class SimpleJsonFormatter(logging.Formatter):
    """Minimal JSON formatter without external deps."""
    def format(self, record: logging.LogRecord) -> str:
        import json
        obj = {
            "time": self.formatTime(record, "%Y-%m-%d %H:%M:%S"),
            "name": record.name,
            "level": record.levelname,
            "trace_id": getattr(record, "trace_id", "-"),
            "message": record.getMessage(),
        }
        return json.dumps(obj, ensure_ascii=False)
def _parse_size(size_str: str) -> int:
    """解析大小字符串（如 '10MB'）为字节数"""
    if isinstance(size_str, int):
        return size_str
    if isinstance(size_str, str) and size_str.upper().endswith("MB"):
        try:
            return int(float(size_str[:-2]) * 1024 * 1024)
        except Exception:
            return 10 * 1024 * 1024
    return 10 * 1024 * 1024
def setup_logging(log_level: str = "INFO"):
    """
    设置应用日志配置：
    1) 优先尝试从 config/logging.toml 读取并转化为 dictConfig
    2) 失败或不存在时，回退到内置默认配置
    """
    # 1) 若存在 TOML 配置且可解析，则优先使用
    try:
        cfg_path = resolve_logging_cfg_path()
        print(f"🔍 [setup_logging] 日志配置文件路径: {cfg_path}")
        print(f"🔍 [setup_logging] 配置文件存在: {cfg_path.exists()}")
        print(f"🔍 [setup_logging] TOML加载器可用: {toml_loader is not None}")
        if cfg_path.exists() and toml_loader is not None:
            with cfg_path.open("rb") as f:
                toml_data = toml_loader.load(f)
            print(f"🔍 [setup_logging] 成功加载TOML配置")
            # 读取基础字段
            logging_root = toml_data.get("logging", {})
            level = logging_root.get("level", log_level)
            fmt_cfg = logging_root.get("format", {})
            fmt_console = fmt_cfg.get(
                "console", "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
            )
            fmt_file = fmt_cfg.get(
                "file", "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
            )
            # 确保文本格式包含 trace_id（若未显式包含）
            if "%(trace_id)" not in str(fmt_console):
                fmt_console = str(fmt_console) + " trace=%(trace_id)s"
            if "%(trace_id)" not in str(fmt_file):
                fmt_file = str(fmt_file) + " trace=%(trace_id)s"
            handlers_cfg = logging_root.get("handlers", {})
            file_handler_cfg = handlers_cfg.get("file", {})
            file_dir = file_handler_cfg.get("directory", "./logs")
            file_level = file_handler_cfg.get("level", "DEBUG")
            max_bytes = file_handler_cfg.get("max_size", "10MB")
            # 支持 "10MB" 形式
            if isinstance(max_bytes, str) and max_bytes.upper().endswith("MB"):
                try:
                    max_bytes = int(float(max_bytes[:-2]) * 1024 * 1024)
                except Exception:
                    max_bytes = 10 * 1024 * 1024
            elif not isinstance(max_bytes, int):
                max_bytes = 10 * 1024 * 1024
            backup_count = int(file_handler_cfg.get("backup_count", 5))
            Path(file_dir).mkdir(parents=True, exist_ok=True)
            # 从TOML配置读取各个日志文件路径
            main_handler_cfg = handlers_cfg.get("main", {})
            webapi_handler_cfg = handlers_cfg.get("webapi", {})
            worker_handler_cfg = handlers_cfg.get("worker", {})
            print(f"🔍 [setup_logging] handlers配置: {list(handlers_cfg.keys())}")
            print(f"🔍 [setup_logging] main_handler_cfg: {main_handler_cfg}")
            print(f"🔍 [setup_logging] webapi_handler_cfg: {webapi_handler_cfg}")
```

```
print(f"🔍 [setup_logging] worker_handler_cfg: {worker_handler_cfg}")
# 主日志文件（tradingagents.log）
main_log = main_handler_cfg.get("filename", str(Path(file_dir) / "tradingagents.log"))
main_enabled = main_handler_cfg.get("enabled", True)
main_level = main_handler_cfg.get("level", "INFO")
main_max_bytes = _parse_size(main_handler_cfg.get("max_size", "100MB"))
main_backup_count = int(main_handler_cfg.get("backup_count", 5))
print(f"🔍 [setup_logging] 主日志文件配置:")
print(f"  - 文件路径: {main_log}")
print(f"  - 是否启用: {main_enabled}")
print(f"  - 日志级别: {main_level}")
print(f"  - 最大大小: {main_max_bytes} bytes")
print(f"  - 备份数量: {main_backup_count}")
# WebAPI日志文件
webapi_log = webapi_handler_cfg.get("filename", str(Path(file_dir) / "webapi.log"))
webapi_enabled = webapi_handler_cfg.get("enabled", True)
webapi_level = webapi_handler_cfg.get("level", "DEBUG")
webapi_max_bytes = _parse_size(webapi_handler_cfg.get("max_size", "100MB"))
webapi_backup_count = int(webapi_handler_cfg.get("backup_count", 5))
print(f"🔍 [setup_logging] WebAPI日志文件: {webapi_log}, 启用: {webapi_enabled}")
# Worker日志文件
worker_log = worker_handler_cfg.get("filename", str(Path(file_dir) / "worker.log"))
worker_enabled = worker_handler_cfg.get("enabled", True)
worker_level = worker_handler_cfg.get("level", "DEBUG")
worker_max_bytes = _parse_size(worker_handler_cfg.get("max_size", "100MB"))
worker_backup_count = int(worker_handler_cfg.get("backup_count", 5))
print(f"🔍 [setup_logging] Worker日志文件: {worker_log}, 启用: {worker_enabled}")
# 错误日志文件
error_handler_cfg = handlers_cfg.get("error", {})
error_log = error_handler_cfg.get("filename", str(Path(file_dir) / "error.log"))
error_enabled = error_handler_cfg.get("enabled", True)
error_level = error_handler_cfg.get("level", "WARNING")
error_max_bytes = _parse_size(error_handler_cfg.get("max_size", "100MB"))
error_backup_count = int(error_handler_cfg.get("backup_count", 5))
# JSON 开关：保持向后兼容（json/mode 仅控制台）；新增 file_json/file_mode 控制文件 handler
use_json_console = bool(fmt_cfg.get("json", False)) or str(fmt_cfg.get("mode", "")).lower() == "json"
use_json_file = (
    bool(fmt_cfg.get("file_json", False))
    or bool(fmt_cfg.get("json_file", False))
    or str(fmt_cfg.get("file_mode", "")).lower() == "json"
)
# 构建处理器配置
handlers_config = {
    "console": {
        "class": "logging.StreamHandler",
        "formatter": "json_console_fmt" if use_json_console else "console_fmt",
        "level": level,
        "filters": ["request_context"],
        "stream": sys.stdout,
    },
}
print(f"🔍 [setup_logging] 开始构建handlers配置")
# 🔥 选择日志处理器类（Windows 使用 ConcurrentRotatingFileHandler）
handler_class = "concurrent_log_handler.ConcurrentRotatingFileHandler" if _USE_CONCURRENT_HANDLER else "logging.handlers.RotatingFileHandler"
# 主日志文件（tradingagents.log）
if main_enabled:
    print(f"✅ [setup_logging] 添加 main_file handler: {main_log} (使用 {handler_class})")
    handlers_config["main_file"] = {
        "class": handler_class,
        "formatter": "json_file_fmt" if use_json_file else "file_fmt",
        "level": main_level,
        "filename": main_log,
        "maxBytes": main_max_bytes,
        "backupCount": main_backup_count,
        "encoding": "utf-8",
        "filters": ["request_context"],
    }
else:
    print(f"⚠️  [setup_logging] main_file handler 未启用")
# WebAPI日志文件
if webapi_enabled:
    handlers_config["file"] = {
        "class": handler_class,
        "formatter": "json_file_fmt" if use_json_file else "file_fmt",
        "level": webapi_level,
        "filename": webapi_log,
        "maxBytes": webapi_max_bytes,
        "backupCount": webapi_backup_count,
        "encoding": "utf-8",
        "filters": ["request_context"],
    }
# Worker日志文件
if worker_enabled:
    handlers_config["worker_file"] = {
        "class": handler_class,
        "formatter": "json_file_fmt" if use_json_file else "file_fmt",
        "level": worker_level,
        "filename": worker_log,
        "maxBytes": worker_max_bytes,
        "backupCount": worker_backup_count,
        "encoding": "utf-8",
        "filters": ["request_context"],
    }
# 添加错误日志处理器（如果启用）
if error_enabled:
    handlers_config["error_file"] = {
        "class": "logging.handlers.RotatingFileHandler",
        "formatter": "json_file_fmt" if use_json_file else "file_fmt",
        "level": error_level,
        "filename": error_log,
        "maxBytes": error_max_bytes,
        "backupCount": error_backup_count,
        "encoding": "utf-8",
```

```
            "filters": ["request_context"],
        }
    # 构建logger handlers列表
    main_handlers = ["console"]
    if main_enabled:
        main_handlers.append("main_file")
    if error_enabled:
        main_handlers.append("error_file")
    print(f"🔍 [setup_logging] main_handlers: {main_handlers}")
    webapi_handlers = ["console"]
    if webapi_enabled:
        webapi_handlers.append("file")
    if main_enabled:
        webapi_handlers.append("main_file")
    if error_enabled:
        webapi_handlers.append("error_file")
    print(f"🔍 [setup_logging] webapi_handlers: {webapi_handlers}")
    worker_handlers = ["console"]
    if worker_enabled:
        worker_handlers.append("worker_file")
    if main_enabled:
        worker_handlers.append("main_file")
    if error_enabled:
        worker_handlers.append("error_file")
    print(f"🔍 [setup_logging] worker_handlers: {worker_handlers}")
    logging_config = {
        "version": 1,
        "disable_existing_loggers": False,
        "filters": {
            "request_context": {"()": "app.core.logging_context.LoggingContextFilter"}
        },
        "formatters": {
            "console_fmt": {
                "format": fmt_console,
                "datefmt": "%Y-%m-%d %H:%M:%S",
            },
            "file_fmt": {
                "format": fmt_file,
                "datefmt": "%Y-%m-%d %H:%M:%S",
            },
            "json_console_fmt": {
                "()": "app.core.logging_config.SimpleJsonFormatter"
            },
            "json_file_fmt": {
                "()": "app.core.logging_config.SimpleJsonFormatter"
            },
        },
        "handlers": handlers_config,
        "loggers": {
            "tradingagents": {
                "level": "INFO",
                "handlers": main_handlers,
                "propagate": False
            },
            "webapi": {
                "level": "INFO",
                "handlers": webapi_handlers,
                "propagate": False
            },
            "worker": {
                "level": "DEBUG",
                "handlers": worker_handlers,
                "propagate": False
            },
            "uvicorn": {
                "level": "INFO",
                "handlers": webapi_handlers,
                "propagate": False
            },
            "fastapi": {
                "level": "INFO",
                "handlers": webapi_handlers,
                "propagate": False
            },
            "app": {
                "level": "INFO",
                "handlers": main_handlers,
                "propagate": False
            },
        },
        "root": {"level": level, "handlers": main_handlers},
    }
    print(f"🔍 [setup_logging] 最终handlers配置: {list(handlers_config.keys())}")
    print(f"🔍 [setup_logging] 开始应用 dictConfig")
    logging.config.dictConfig(logging_config)
    print(f"☑ [setup_logging] dictConfig 应用成功")
    logging.getLogger("webapi").info(f"Logging configured from {cfg_path}")
    # 测试主日志文件是否可写
    if main_enabled:
        test_logger = logging.getLogger("tradingagents")
        test_logger.info(f"🔍 测试主日志文件写入: {main_log}")
        print(f"🔍 [setup_logging] 已向 tradingagents logger 写入测试日志")
    return
except Exception as e:
    # TOML 存在但加载失败，回退到默认配置
    logging.getLogger("webapi").warning(f"Failed to load logging.toml, fallback to defaults: {e}")
# 2) 默认内置配置（与原先一致）
log_dir = Path("logs")
log_dir.mkdir(exist_ok=True)
# 🔥 选择日志处理器类（Windows 使用 ConcurrentRotatingFileHandler）
handler_class = "concurrent_log_handler.ConcurrentRotatingFileHandler" if _USE_CONCURRENT_HANDLER else "logging.handlers.RotatingFileHandler"
logging_config = {
    "version": 1,
```

```
        "disable_existing_loggers": False,
        "filters": {"request_context": {"()": "app.core.logging_context.LoggingContextFilter"}},
        "formatters": {
            "default": {
                "format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s trace=%(trace_id)s",
                "datefmt": "%Y-%m-%d %H:%M:%S",
            },
            "detailed": {
                "format": "%(asctime)s - %(name)s - %(levelname)s - %(pathname)s:%(lineno)d - %(message)s trace=%(trace_id)s",
                "datefmt": "%Y-%m-%d %H:%M:%S",
            },
        },
        "handlers": {
            "console": {
                "class": "logging.StreamHandler",
                "formatter": "default",
                "level": log_level,
                "filters": ["request_context"],
                "stream": sys.stdout,
            },
            "file": {
                "class": handler_class,
                "formatter": "detailed",
                "level": "DEBUG",
                "filters": ["request_context"],
                "filename": "logs/webapi.log",
                "maxBytes": 10485760,
                "backupCount": 5,
                "encoding": "utf-8",
            },
            "worker_file": {
                "class": handler_class,
                "formatter": "detailed",
                "level": "DEBUG",
                "filters": ["request_context"],
                "filename": "logs/worker.log",
                "maxBytes": 10485760,
                "backupCount": 5,
                "encoding": "utf-8",
            },
            "error_file": {
                "class": handler_class,
                "formatter": "detailed",
                "level": "WARNING",
                "filters": ["request_context"],
                "filename": "logs/error.log",
                "maxBytes": 10485760,
                "backupCount": 5,
                "encoding": "utf-8",
            },
        },
        "loggers": {
            "webapi": {"level": "INFO", "handlers": ["console", "file", "error_file"], "propagate": True},
            "worker": {"level": "DEBUG", "handlers": ["console", "worker_file", "error_file"], "propagate": False},
            "uvicorn": {"level": "INFO", "handlers": ["console", "file", "error_file"], "propagate": False},
            "fastapi": {"level": "INFO", "handlers": ["console", "file", "error_file"], "propagate": False},
        },
        "root": {"level": log_level, "handlers": ["console"]},
    }
    logging.config.dictConfig(logging_config)
    logging.getLogger("webapi").info("Logging configured successfully (built-in)")
```

FILE: app/core/logging_context.py

```
import logging
import contextvars
# Shared contextvar for trace id across the whole process
trace_id_var: contextvars.ContextVar[str] = contextvars.ContextVar("trace_id", default="-")
class LoggingContextFilter(logging.Filter):
    """Injects trace_id from contextvars into LogRecord.
    Always sets record.trace_id to a string (default '-') so formatters are safe.
    """
    def filter(self, record: logging.LogRecord) -> bool:
        try:
            record.trace_id = trace_id_var.get()
        except Exception:
            record.trace_id = "-"
        return True
```

FILE: app/core/rate_limiter.py

```
"""
速率限制器
用于控制API调用频率，避免超过数据源的限流限制
"""
import asyncio
import time
import logging
from collections import deque
from typing import Optional
logger = logging.getLogger(__name__)
class RateLimiter:
    """
    滑动窗口速率限制器

    使用滑动窗口算法精确控制API调用频率
    """

    def __init__(self, max_calls: int, time_window: float, name: str = "RateLimiter"):
        """
        初始化速率限制器
```

```
        Args:
            max_calls: 时间窗口内最大调用次数
            time_window: 时间窗口大小（秒）
            name: 限制器名称（用于日志）
        """
        self.max_calls = max_calls
        self.time_window = time_window
        self.name = name
        self.calls = deque()  # 存储调用时间戳
        self.lock = asyncio.Lock()  # 确保线程安全

        # 统计信息
        self.total_calls = 0
        self.total_waits = 0
        self.total_wait_time = 0.0

        logger.info(f"🔧 {self.name} 初始化: {max_calls}次/{time_window}秒")

    async def acquire(self):
        """
        获取调用许可
        如果超过速率限制，会等待直到可以调用
        """
        async with self.lock:
            now = time.time()

            # 移除时间窗口外的旧调用记录
            while self.calls and self.calls[0] <= now - self.time_window:
                self.calls.popleft()

            # 如果当前窗口内调用次数已达上限，需要等待
            if len(self.calls) >= self.max_calls:
                # 计算需要等待的时间
                oldest_call = self.calls[0]
                wait_time = oldest_call + self.time_window - now + 0.01  # 加一点缓冲

                if wait_time > 0:
                    self.total_waits += 1
                    self.total_wait_time += wait_time

                    logger.debug(f"⏳ {self.name} 达到速率限制，等待 {wait_time:.2f}秒")
                    await asyncio.sleep(wait_time)

                    # 重新获取当前时间
                    now = time.time()

                    # 再次清理旧记录
                    while self.calls and self.calls[0] <= now - self.time_window:
                        self.calls.popleft()

            # 记录本次调用
            self.calls.append(now)
            self.total_calls += 1

    def get_stats(self) -> dict:
        """获取统计信息"""
        return {
            "name": self.name,
            "max_calls": self.max_calls,
            "time_window": self.time_window,
            "current_calls": len(self.calls),
            "total_calls": self.total_calls,
            "total_waits": self.total_waits,
            "total_wait_time": self.total_wait_time,
            "avg_wait_time": self.total_wait_time / self.total_waits if self.total_waits > 0 else 0
        }

    def reset_stats(self):
        """重置统计信息"""
        self.total_calls = 0
        self.total_waits = 0
        self.total_wait_time = 0.0
        logger.info(f"🔄 {self.name} 统计信息已重置")
class TushareRateLimiter(RateLimiter):
    """
    Tushare专用速率限制器

    根据Tushare的积分等级自动调整限流策略
    """

    # Tushare积分等级对应的限流配置
    TIER_LIMITS = {
        "free": {"max_calls": 100, "time_window": 60},      # 免费用户: 100次/分钟
        "basic": {"max_calls": 200, "time_window": 60},     # 基础用户: 200次/分钟
        "standard": {"max_calls": 400, "time_window": 60},  # 标准用户: 400次/分钟
        "premium": {"max_calls": 600, "time_window": 60},   # 高级用户: 600次/分钟
        "vip": {"max_calls": 800, "time_window": 60},       # VIP用户: 800次/分钟
    }

    def __init__(self, tier: str = "standard", safety_margin: float = 0.8):
        """
        初始化Tushare速率限制器

        Args:
            tier: 积分等级 (free/basic/standard/premium/vip)
            safety_margin: 安全边际（0-1），实际限制为理论限制的百分比
        """
        if tier not in self.TIER_LIMITS:
            logger.warning(f"⚠️      未知的Tushare积分等级: {tier}，使用默认值 'standard'")
            tier = "standard"

        limits = self.TIER_LIMITS[tier]

        # 应用安全边际
        max_calls = int(limits["max_calls"] * safety_margin)
```

```
        time_window = limits["time_window"]

        super().__init__(
            max_calls=max_calls,
            time_window=time_window,
            name=f"TushareRateLimiter({tier})"
        )

        self.tier = tier
        self.safety_margin = safety_margin

        logger.info(f"✅ Tushare速率限制器已配置: {tier}等级, "
                    f"{max_calls}次/{time_window}秒 (安全边际: {safety_margin*100:.0f}%)")
class AKShareRateLimiter(RateLimiter):
    """
    AKShare专用速率限制器

    AKShare没有明确的限流规则，使用保守的限流策略
    """

    def __init__(self, max_calls: int = 60, time_window: float = 60):
        """
        初始化AKShare速率限制器

        Args:
            max_calls: 时间窗口内最大调用次数（默认60次/分钟）
            time_window: 时间窗口大小（秒）
        """
        super().__init__(
            max_calls=max_calls,
            time_window=time_window,
            name="AKShareRateLimiter"
        )
class BaoStockRateLimiter(RateLimiter):
    """
    BaoStock专用速率限制器

    BaoStock没有明确的限流规则，使用保守的限流策略
    """

    def __init__(self, max_calls: int = 100, time_window: float = 60):
        """
        初始化BaoStock速率限制器

        Args:
            max_calls: 时间窗口内最大调用次数（默认100次/分钟）
            time_window: 时间窗口大小（秒）
        """
        super().__init__(
            max_calls=max_calls,
            time_window=time_window,
            name="BaoStockRateLimiter"
        )
# 全局速率限制器实例
_tushare_limiter: Optional[TushareRateLimiter] = None
_akshare_limiter: Optional[AKShareRateLimiter] = None
_baostock_limiter: Optional[BaoStockRateLimiter] = None

def get_tushare_rate_limiter(tier: str = "standard", safety_margin: float = 0.8) -> TushareRateLimiter:
    """获取Tushare速率限制器（单例）"""
    global _tushare_limiter
    if _tushare_limiter is None:
        _tushare_limiter = TushareRateLimiter(tier=tier, safety_margin=safety_margin)
    return _tushare_limiter

def get_akshare_rate_limiter() -> AKShareRateLimiter:
    """获取AKShare速率限制器（单例）"""
    global _akshare_limiter
    if _akshare_limiter is None:
        _akshare_limiter = AKShareRateLimiter()
    return _akshare_limiter

def get_baostock_rate_limiter() -> BaoStockRateLimiter:
    """获取BaoStock速率限制器（单例）"""
    global _baostock_limiter
    if _baostock_limiter is None:
        _baostock_limiter = BaoStockRateLimiter()
    return _baostock_limiter

def reset_all_limiters():
    """重置所有速率限制器"""
    global _tushare_limiter, _akshare_limiter, _baostock_limiter
    _tushare_limiter = None
    _akshare_limiter = None
    _baostock_limiter = None
    logger.info("🔄 所有速率限制器已重置")
```

FILE: app/core/redis_client.py

```
"""
Redis客户端配置和连接管理
"""
import redis.asyncio as redis
import logging
from typing import Optional
from .config import settings
logger = logging.getLogger(__name__)
# 全局Redis连接池
redis_pool: Optional[redis.ConnectionPool] = None
redis_client: Optional[redis.Redis] = None

async def init_redis():
    """初始化Redis连接"""
    global redis_pool, redis_client
    try:
        # 创建连接池
```

```
        redis_pool = redis.ConnectionPool.from_url(
            settings.REDIS_URL,
            max_connections=settings.REDIS_MAX_CONNECTIONS,  # 使用配置文件中的值
            retry_on_timeout=settings.REDIS_RETRY_ON_TIMEOUT,
            decode_responses=True,
            socket_keepalive=True,  # 启用 TCP keepalive
            socket_keepalive_options={
                1: 60,  # TCP_KEEPIDLE: 60秒后开始发送keepalive探测
                2: 10,  # TCP_KEEPINTVL: 每10秒发送一次探测
                3: 3,   # TCP_KEEPCNT: 最多发送3次探测
            },
            health_check_interval=30,  # 每30秒检查一次连接健康状态
        )
        # 创建Redis客户端
        redis_client = redis.Redis(connection_pool=redis_pool)
        # 测试连接
        await redis_client.ping()
        logger.info(f"✅ Redis连接成功建立 (max_connections={settings.REDIS_MAX_CONNECTIONS})")
    except Exception as e:
        logger.error(f"❌ Redis连接失败: {e}")
        raise
async def close_redis():
    """关闭Redis连接"""
    global redis_pool, redis_client

    try:
        if redis_client:
            await redis_client.close()
        if redis_pool:
            await redis_pool.disconnect()
        logger.info("✅ Redis连接已关闭")
    except Exception as e:
        logger.error(f"❌ 关闭Redis连接时出错: {e}")
def get_redis() -> redis.Redis:
    """获取Redis客户端实例"""
    if redis_client is None:
        raise RuntimeError("Redis客户端未初始化")
    return redis_client
class RedisKeys:
    """Redis键名常量"""

    # 队列相关
    USER_PENDING_QUEUE = "user:{user_id}:pending"
    USER_PROCESSING_SET = "user:{user_id}:processing"
    GLOBAL_PENDING_QUEUE = "global:pending"
    GLOBAL_PROCESSING_SET = "global:processing"

    # 任务相关
    TASK_PROGRESS = "task:{task_id}:progress"
    TASK_RESULT = "task:{task_id}:result"
    TASK_LOCK = "task:{task_id}:lock"

    # 批次相关
    BATCH_PROGRESS = "batch:{batch_id}:progress"
    BATCH_TASKS = "batch:{batch_id}:tasks"
    BATCH_LOCK = "batch:{batch_id}:lock"

    # 用户相关
    USER_SESSION = "session:{session_id}"
    USER_RATE_LIMIT = "rate_limit:{user_id}:{endpoint}"
    USER_DAILY_QUOTA = "quota:{user_id}:{date}"

    # 系统相关
    QUEUE_STATS = "queue:stats"
    SYSTEM_CONFIG = "system:config"
    WORKER_HEARTBEAT = "worker:{worker_id}:heartbeat"

    # 缓存相关
    SCREENING_CACHE = "screening:{cache_key}"
    ANALYSIS_CACHE = "analysis:{cache_key}"
class RedisService:
    """Redis服务封装类"""

    def __init__(self):
        self.redis = get_redis()

    async def set_with_ttl(self, key: str, value: str, ttl: int = 3600):
        """设置带TTL的键值"""
        await self.redis.setex(key, ttl, value)

    async def get_json(self, key: str):
        """获取JSON格式的值"""
        import json
        value = await self.redis.get(key)
        if value:
            return json.loads(value)
        return None

    async def set_json(self, key: str, value: dict, ttl: int = None):
        """设置JSON格式的值"""
        import json
        json_str = json.dumps(value, ensure_ascii=False)
        if ttl:
            await self.redis.setex(key, ttl, json_str)
        else:
            await self.redis.set(key, json_str)

    async def increment_with_ttl(self, key: str, ttl: int = 3600):
        """递增计数器并设置TTL"""
        pipe = self.redis.pipeline()
        pipe.incr(key)
        pipe.expire(key, ttl)
        results = await pipe.execute()
        return results[0]
```

```
    async def add_to_queue(self, queue_key: str, item: dict):
        """添加项目到队列"""
        import json
        await self.redis.lpush(queue_key, json.dumps(item, ensure_ascii=False))

    async def pop_from_queue(self, queue_key: str, timeout: int = 1):
        """从队列弹出项目"""
        import json
        result = await self.redis.brpop(queue_key, timeout=timeout)
        if result:
            return json.loads(result[1])
        return None

    async def get_queue_length(self, queue_key: str):
        """获取队列长度"""
        return await self.redis.llen(queue_key)

    async def add_to_set(self, set_key: str, value: str):
        """添加到集合"""
        await self.redis.sadd(set_key, value)

    async def remove_from_set(self, set_key: str, value: str):
        """从集合移除"""
        await self.redis.srem(set_key, value)

    async def is_in_set(self, set_key: str, value: str):
        """检查是否在集合中"""
        return await self.redis.sismember(set_key, value)

    async def get_set_size(self, set_key: str):
        """获取集合大小"""
        return await self.redis.scard(set_key)

    async def acquire_lock(self, lock_key: str, timeout: int = 30):
        """获取分布式锁"""
        import uuid
        lock_value = str(uuid.uuid4())
        acquired = await self.redis.set(lock_key, lock_value, nx=True, ex=timeout)
        if acquired:
            return lock_value
        return None

    async def release_lock(self, lock_key: str, lock_value: str):
        """释放分布式锁"""
        lua_script = """
        if redis.call("get", KEYS[1]) == ARGV[1] then
            return redis.call("del", KEYS[1])
        else
            return 0
        end
        """
        return await self.redis.eval(lua_script, 1, lock_key, lock_value)
# 全局Redis服务实例
redis_service: Optional[RedisService] = None

def get_redis_service() -> RedisService:
    """获取Redis服务实例"""
    global redis_service
    if redis_service is None:
        redis_service = RedisService()
    return redis_service
```

FILE: app/core/response.py

```
"""
统一API响应格式工具
"""
from datetime import datetime
from typing import Any, Optional, Dict
from app.utils.timezone import now_tz

def ok(data: Any = None, message: str = "ok") -> Dict[str, Any]:
    """标准成功响应
    返回结构：{"success": True, "data": data, "message": message, "timestamp": ...}
    """
    return {
        "success": True,
        "data": data,
        "message": message,
        "timestamp": now_tz().isoformat()
    }

def fail(message: str = "error", code: int = 500, data: Any = None) -> Dict[str, Any]:
    """标准失败响应（一般错误仍建议用 HTTPException 抛出，此函数用于业务失败场景）"""
    return {
        "success": False,
        "data": data,
        "message": message,
        "code": code,
        "timestamp": now_tz().isoformat()
    }
```

FILE: app/core/startup_validator.py

```
"""
启动配置验证器
验证系统启动所需的必需配置项，提供友好的错误提示。
"""
import os
import logging
from typing import List, Dict, Any, Optional
from dataclasses import dataclass
from enum import Enum
```

```
logger = logging.getLogger(__name__)

class ConfigLevel(Enum):
    """配置级别"""
    REQUIRED = "required"     # 必需配置，缺少则无法启动
    RECOMMENDED = "recommended"  # 推荐配置，缺少会影响功能
    OPTIONAL = "optional"     # 可选配置，缺少不影响基本功能

@dataclass
class ConfigItem:
    """配置项"""
    key: str                  # 配置键名
    level: ConfigLevel        # 配置级别
    description: str          # 配置描述
    example: Optional[str] = None  # 配置示例
    help_url: Optional[str] = None  # 帮助链接
    validator: Optional[callable] = None  # 自定义验证函数

@dataclass
class ValidationResult:
    """验证结果"""
    success: bool             # 是否验证成功
    missing_required: List[ConfigItem]  # 缺少的必需配置
    missing_recommended: List[ConfigItem]  # 缺少的推荐配置
    invalid_configs: List[tuple[ConfigItem, str]]  # 无效的配置（配置项，错误信息）
    warnings: List[str]       # 警告信息

class StartupValidator:
    """启动配置验证器"""

    # 必需配置项
    REQUIRED_CONFIGS = [
        ConfigItem(
            key="MONGODB_HOST",
            level=ConfigLevel.REQUIRED,
            description="MongoDB主机地址",
            example="localhost"
        ),
        ConfigItem(
            key="MONGODB_PORT",
            level=ConfigLevel.REQUIRED,
            description="MongoDB端口",
            example="27017",
            validator=lambda v: v.isdigit() and 1 <= int(v) <= 65535
        ),
        ConfigItem(
            key="MONGODB_DATABASE",
            level=ConfigLevel.REQUIRED,
            description="MongoDB数据库名称",
            example="tradingagents"
        ),
        ConfigItem(
            key="REDIS_HOST",
            level=ConfigLevel.REQUIRED,
            description="Redis主机地址",
            example="localhost"
        ),
        ConfigItem(
            key="REDIS_PORT",
            level=ConfigLevel.REQUIRED,
            description="Redis端口",
            example="6379",
            validator=lambda v: v.isdigit() and 1 <= int(v) <= 65535
        ),
        ConfigItem(
            key="JWT_SECRET",
            level=ConfigLevel.REQUIRED,
            description="JWT密钥（用于生成认证令牌）",
            example="your-super-secret-jwt-key-change-in-production",
            validator=lambda v: len(v) >= 16
        ),
    ]

    # 推荐配置项
    RECOMMENDED_CONFIGS = [
        ConfigItem(
            key="DEEPSEEK_API_KEY",
            level=ConfigLevel.RECOMMENDED,
            description="DeepSeek API密钥（推荐，性价比高）",
            example="sk-xxx",
            help_url="https://platform.deepseek.com/"
        ),
        ConfigItem(
            key="DASHSCOPE_API_KEY",
            level=ConfigLevel.RECOMMENDED,
            description="阿里百炼API密钥（推荐，国产稳定）",
            example="sk-xxx",
            help_url="https://dashscope.aliyun.com/"
        ),
        ConfigItem(
            key="TUSHARE_TOKEN",
            level=ConfigLevel.RECOMMENDED,
            description="Tushare Token（推荐，专业A股数据）",
            example="xxx",
            help_url="https://tushare.pro/register?reg=tacn"
        ),
    ]

    def __init__(self):
        self.result = ValidationResult(
            success=True,
            missing_required=[],
            missing_recommended=[],
            invalid_configs=[],
            warnings=[]
        )
    def _is_valid_api_key(self, api_key: str) -> bool:
        """
```

```
    判断 API Key 是否有效（不是占位符）
    Args:
        api_key: 待验证的 API Key
    Returns:
        bool: True 表示有效，False 表示无效或占位符
    """
    if not api_key:
        return False
    # 去除首尾空格和引号
    api_key = api_key.strip().strip('"').strip("'")
    # 检查是否为空
    if not api_key:
        return False
    # 检查是否为占位符（前缀）
    if api_key.startswith('your_') or api_key.startswith('your-'):
        return False
    # 检查是否为占位符（后缀）
    if api_key.endswith('_here') or api_key.endswith('-here'):
        return False
    # 检查长度（大多数 API Key 都 > 10 个字符）
    if len(api_key) <= 10:
        return False
    return True
def validate(self) -> ValidationResult:
    """
    验证配置

    Returns:
        ValidationResult: 验证结果
    """
    logger.info("🔍 开始验证启动配置...")

    # 验证必需配置
    self._validate_required_configs()

    # 验证推荐配置
    self._validate_recommended_configs()

    # 检查安全配置
    self._check_security_configs()

    # 设置验证结果
    self.result.success = len(self.result.missing_required) == 0 and len(self.result.invalid_configs) == 0

    # 输出验证结果
    self._print_validation_result()

    return self.result

def _validate_required_configs(self):
    """验证必需配置"""
    for config in self.REQUIRED_CONFIGS:
        value = os.getenv(config.key)

        if not value:
            self.result.missing_required.append(config)
            logger.error(f"❌ 缺少必需配置: {config.key}")
        elif config.validator and not config.validator(value):
            self.result.invalid_configs.append((config, "配置值格式不正确"))
            logger.error(f"❌ 配置格式错误: {config.key}")
        else:
            logger.debug(f"☑ {config.key}: 已配置")

def _validate_recommended_configs(self):
    """验证推荐配置"""
    for config in self.RECOMMENDED_CONFIGS:
        value = os.getenv(config.key)
        if not value:
            self.result.missing_recommended.append(config)
            logger.warning(f"⚠ 缺少推荐配置: {config.key}")
        elif not self._is_valid_api_key(value):
            # API Key 存在但是占位符，视为未配置
            self.result.missing_recommended.append(config)
            logger.warning(f"⚠ {config.key} 配置为占位符，视为未配置")
        else:
            logger.debug(f"☑ {config.key}: 已配置")

def _check_security_configs(self):
    """检查安全配置"""
    # 检查JWT密钥是否使用默认值
    jwt_secret = os.getenv("JWT_SECRET", "")
    if jwt_secret in ["change-me-in-production", "your-super-secret-jwt-key-change-in-production"]:
        self.result.warnings.append(
            "⚠ JWT_SECRET 使用默认值，生产环境请务必修改！"
        )

    # 检查CSRF密钥是否使用默认值
    csrf_secret = os.getenv("CSRF_SECRET", "")
    if csrf_secret in ["change-me-csrf-secret", "your-csrf-secret-key-change-in-production"]:
        self.result.warnings.append(
            "⚠ CSRF_SECRET 使用默认值，生产环境请务必修改！"
        )

    # 检查是否在生产环境使用DEBUG模式
    debug = os.getenv("DEBUG", "true").lower() in ("true", "1", "yes", "on")
    if not debug:
        logger.info("ℹ 生产环境模式")
    else:
        logger.info("ℹ 开发环境模式（DEBUG=true）")

def _print_validation_result(self):
    """输出验证结果"""
    logger.info("\n" + "=" * 70)
    logger.info("TradingAgents-CN Configuration Validation Result")
```

```
        logger.info("=" * 70)

        # 必需配置
        if self.result.missing_required:
            logger.info("\nMissing required configurations:")
            for config in self.result.missing_required:
                logger.info(f"  - {config.key}")
                logger.info(f"    Description: {config.description}")
                if config.example:
                    logger.info(f"    Example: {config.example}")
                if config.help_url:
                    logger.info(f"    Help: {config.help_url}")
        else:
            logger.info("\nAll required configurations are complete")
        # 无效配置
        if self.result.invalid_configs:
            logger.info("\nInvalid configurations:")
            for config, error in self.result.invalid_configs:
                logger.info(f"  - {config.key}: {error}")
                if config.example:
                    logger.info(f"    Example: {config.example}")
        # 推荐配置
        if self.result.missing_recommended:
            logger.info("\nMissing recommended configurations (won't affect startup):")
            for config in self.result.missing_recommended:
                logger.info(f"  - {config.key}")
                logger.info(f"    Description: {config.description}")
                if config.help_url:
                    logger.info(f"    Get it from: {config.help_url}")
        # 警告信息
        if self.result.warnings:
            logger.info("\nSecurity warnings:")
            for warning in self.result.warnings:
                logger.info(f"  - {warning}")
        # 总结
        logger.info("\n" + "=" * 70)
        if self.result.success:
            logger.info("Configuration validation passed, system can start")
            if self.result.missing_recommended:
                logger.info("Tip: Configure recommended items for better functionality")
        else:
            logger.info("Configuration validation failed, please check the above items")
            logger.info("Configuration guide: docs/configuration_guide.md")
        logger.info("=" * 70 + "\n")

    def raise_if_failed(self):
        """如果验证失败则抛出异常"""
        if not self.result.success:
            error_messages = []

            if self.result.missing_required:
                error_messages.append(
                    f"缺少必需配置: {', '.join(c.key for c in self.result.missing_required)}"
                )

            if self.result.invalid_configs:
                error_messages.append(
                    f"配置格式错误: {', '.join(c.key for c, _ in self.result.invalid_configs)}"
                )

            raise ConfigurationError(
                "配置验证失败:\n" + "\n".join(f"  • {msg}" for msg in error_messages) +
                "\n\n请检查 .env 文件并参考 docs/configuration_guide.md"
            )
class ConfigurationError(Exception):
    """配置错误异常"""
    pass
def validate_startup_config() -> ValidationResult:
    """
    验证启动配置（便捷函数）

    Returns:
        ValidationResult: 验证结果

    Raises:
        ConfigurationError: 如果验证失败
    """
    validator = StartupValidator()
    result = validator.validate()
    validator.raise_if_failed()
    return result
```

FILE: app/core/unified_config.py

```
"""
统一配置管理系统
整合 config/、tradingagents/config/ 和 webapi 的配置管理
"""
import json
import os
from pathlib import Path
from typing import Dict, List, Optional, Any, Union
from datetime import datetime
import asyncio
from dataclasses import dataclass, asdict
from app.models.config import (
    LLMConfig, DataSourceConfig, DatabaseConfig, SystemConfig,
    ModelProvider, DataSourceType, DatabaseType
)
@dataclass
class ConfigPaths:
    """配置文件路径"""
```

```
    root_config_dir: Path = Path("config")
    tradingagents_config_dir: Path = Path("tradingagents/config")
    webapi_config_dir: Path = Path("data/config")

    # 具体配置文件
    models_json: Path = root_config_dir / "models.json"
    settings_json: Path = root_config_dir / "settings.json"
    pricing_json: Path = root_config_dir / "pricing.json"
    verified_models_json: Path = root_config_dir / "verified_models.json"
class UnifiedConfigManager:
    """统一配置管理器"""

    def __init__(self):
        self.paths = ConfigPaths()
        self._cache = {}
        self._last_modified = {}

    def _get_file_mtime(self, file_path: Path) -> float:
        """获取文件修改时间"""
        try:
            return file_path.stat().st_mtime
        except FileNotFoundError:
            return 0.0

    def _is_cache_valid(self, cache_key: str, file_path: Path) -> bool:
        """检查缓存是否有效"""
        if cache_key not in self._cache:
            return False

        current_mtime = self._get_file_mtime(file_path)
        cached_mtime = self._last_modified.get(cache_key, 0)

        return current_mtime <= cached_mtime

    def _load_json_file(self, file_path: Path, cache_key: str = None) -> Dict[str, Any]:
        """加载JSON文件，支持缓存"""
        if cache_key and self._is_cache_valid(cache_key, file_path):
            return self._cache[cache_key]

        try:
            with open(file_path, 'r', encoding='utf-8') as f:
                data = json.load(f)

            if cache_key:
                self._cache[cache_key] = data
                self._last_modified[cache_key] = self._get_file_mtime(file_path)

            return data
        except FileNotFoundError:
            return {}
        except json.JSONDecodeError as e:
            print(f"配置文件格式错误 {file_path}: {e}")
            return {}

    def _save_json_file(self, file_path: Path, data: Dict[str, Any], cache_key: str = None):
        """保存JSON文件"""
        # 确保目录存在
        file_path.parent.mkdir(parents=True, exist_ok=True)

        with open(file_path, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2, ensure_ascii=False)

        if cache_key:
            self._cache[cache_key] = data
            self._last_modified[cache_key] = self._get_file_mtime(file_path)

    # ==================== 模型配置管理 ====================

    def get_legacy_models(self) -> List[Dict[str, Any]]:
        """获取传统格式的模型配置"""
        return self._load_json_file(self.paths.models_json, "models")

    def get_llm_configs(self) -> List[LLMConfig]:
        """获取标准化的LLM配置"""
        legacy_models = self.get_legacy_models()
        llm_configs = []
        for model in legacy_models:
            try:
                # 直接使用 provider 字符串，不再映射到枚举
                provider = model.get("provider", "openai")
                # 方案A：敏感密钥不从文件加载，统一走环境变量/厂家目录
                llm_config = LLMConfig(
                    provider=provider,
                    model_name=model.get("model_name", ""),
                    api_key="",
                    api_base=model.get("base_url"),
                    max_tokens=model.get("max_tokens", 4000),
                    temperature=model.get("temperature", 0.7),
                    enabled=model.get("enabled", True),
                    description=f"{model.get('provider', '')} {model.get('model_name', '')}"
                )
                llm_configs.append(llm_config)
            except Exception as e:
                print(f"转换模型配置失败: {model}, 错误: {e}")
                continue
        return llm_configs

    def save_llm_config(self, llm_config: LLMConfig) -> bool:
        """保存LLM配置到传统格式"""
        try:
            legacy_models = self.get_legacy_models()
            # 直接使用 provider 字符串，不再需要映射
            # 方案A：保存到文件时不写入密钥
            legacy_model = {
                "provider": llm_config.provider,
```

```
            "model_name": llm_config.model_name,
            "api_key": "",
            "base_url": llm_config.api_base,
            "max_tokens": llm_config.max_tokens,
            "temperature": llm_config.temperature,
            "enabled": llm_config.enabled
        }

        # 查找并更新现有配置，或添加新配置
        updated = False
        for i, model in enumerate(legacy_models):
            if (model.get("provider") == legacy_model["provider"] and
                model.get("model_name") == legacy_model["model_name"]):
                legacy_models[i] = legacy_model
                updated = True
                break

        if not updated:
            legacy_models.append(legacy_model)

        self._save_json_file(self.paths.models_json, legacy_models, "models")
        return True

    except Exception as e:
        print(f"保存LLM配置失败: {e}")
        return False

# ==================== 系统设置管理 ====================

def get_system_settings(self) -> Dict[str, Any]:
    """获取系统设置"""
    return self._load_json_file(self.paths.settings_json, "settings")

def save_system_settings(self, settings: Dict[str, Any]) -> bool:
    """保存系统设置（保留现有字段，添加新字段映射）"""
    try:
        print(f"📝 [unified_config] save_system_settings 被调用")
        print(f"📝 [unified_config] 接收到的 settings 包含 {len(settings)} 项")
        # 检查关键字段
        if "quick_analysis_model" in settings:
            print(f"  ✓ [unified_config] 包含 quick_analysis_model: {settings['quick_analysis_model']}")
        else:
            print(f"  ⚠️ [unified_config] 不包含 quick_analysis_model")
        if "deep_analysis_model" in settings:
            print(f"  ✓ [unified_config] 包含 deep_analysis_model: {settings['deep_analysis_model']}")
        else:
            print(f"  ⚠️ [unified_config] 不包含 deep_analysis_model")
        # 读取现有配置
        print(f"📖 [unified_config] 读取现有配置文件: {self.paths.settings_json}")
        current_settings = self.get_system_settings()
        print(f"📖 [unified_config] 现有配置包含 {len(current_settings)} 项")
        # 合并配置（新配置覆盖旧配置）
        merged_settings = current_settings.copy()
        merged_settings.update(settings)
        print(f"🔀 [unified_config] 合并后配置包含 {len(merged_settings)} 项")
        # 添加字段名映射（新字段名 -> 旧字段名）
        if "quick_analysis_model" in settings:
            merged_settings["quick_think_llm"] = settings["quick_analysis_model"]
            print(f"  ✓ [unified_config] 映射 quick_analysis_model -> quick_think_llm: {settings['quick_analysis_model']}")
        if "deep_analysis_model" in settings:
            merged_settings["deep_think_llm"] = settings["deep_analysis_model"]
            print(f"  ✓ [unified_config] 映射 deep_analysis_model -> deep_think_llm: {settings['deep_analysis_model']}")
        # 打印最终要保存的配置
        print(f"💾 [unified_config] 即将保存到文件:")
        if "quick_think_llm" in merged_settings:
            print(f"  ✓ quick_think_llm: {merged_settings['quick_think_llm']}")
        if "deep_think_llm" in merged_settings:
            print(f"  ✓ deep_think_llm: {merged_settings['deep_think_llm']}")
        if "quick_analysis_model" in merged_settings:
            print(f"  ✓ quick_analysis_model: {merged_settings['quick_analysis_model']}")
        if "deep_analysis_model" in merged_settings:
            print(f"  ✓ deep_analysis_model: {merged_settings['deep_analysis_model']}")
        # 保存合并后的配置
        print(f"💾 [unified_config] 保存到文件: {self.paths.settings_json}")
        self._save_json_file(self.paths.settings_json, merged_settings, "settings")
        print(f"✅ [unified_config] 配置保存成功")
        return True
    except Exception as e:
        print(f"❌ [unified_config] 保存系统设置失败: {e}")
        import traceback
        print(traceback.format_exc())
        return False

def get_default_model(self) -> str:
    """获取默认模型（向后兼容）"""
    settings = self.get_system_settings()
    # 优先返回快速分析模型，保持向后兼容
    return settings.get("quick_analysis_model", settings.get("default_model", "qwen-turbo"))
def set_default_model(self, model_name: str) -> bool:
    """设置默认模型（向后兼容）"""
    settings = self.get_system_settings()
    settings["quick_analysis_model"] = model_name
    return self.save_system_settings(settings)
def get_quick_analysis_model(self) -> str:
    """获取快速分析模型"""
    settings = self.get_system_settings()
    # 优先读取新字段名，如果不存在则读取旧字段名（向后兼容）
    return settings.get("quick_analysis_model") or settings.get("quick_think_llm", "qwen-turbo")
def get_deep_analysis_model(self) -> str:
    """获取深度分析模型"""
    settings = self.get_system_settings()
    # 优先读取新字段名，如果不存在则读取旧字段名（向后兼容）
    return settings.get("deep_analysis_model") or settings.get("deep_think_llm", "qwen-max")
def set_analysis_models(self, quick_model: str, deep_model: str) -> bool:
```

```
    """设置分析模型"""
    settings = self.get_system_settings()
    settings["quick_analysis_model"] = quick_model
    settings["deep_analysis_model"] = deep_model
    return self.save_system_settings(settings)

# ==================== 数据源配置管理 ====================

def get_data_source_configs(self) -> List[DataSourceConfig]:
    """获取数据源配置 - 优先从数据库读取，回退到硬编码（同步版本）"""
    try:
        # 🔥 优先从数据库读取配置（使用同步连接）
        from app.core.database import get_mongo_db_sync
        db = get_mongo_db_sync()
        config_collection = db.system_configs
        # 获取最新的激活配置
        config_data = config_collection.find_one(
            {"is_active": True},
            sort=[("version", -1)]
        )
        if config_data and config_data.get('data_source_configs'):
            # 从数据库读取到配置
            data_source_configs = config_data.get('data_source_configs', [])
            print(f"✅ [unified_config] 从数据库读取到 {len(data_source_configs)} 个数据源配置")
            # 转换为 DataSourceConfig 对象
            result = []
            for ds_config in data_source_configs:
                try:
                    result.append(DataSourceConfig(**ds_config))
                except Exception as e:
                    print(f"⚠️ [unified_config] 解析数据源配置失败: {e}, 配置: {ds_config}")
                    continue
            # 按优先级排序（数字越大优先级越高）
            result.sort(key=lambda x: x.priority, reverse=True)
            return result
        else:
            print("⚠️ [unified_config] 数据库中没有数据源配置，使用硬编码配置")
    except Exception as e:
        print(f"⚠️ [unified_config] 从数据库读取数据源配置失败: {e}，使用硬编码配置")
    # 🔥 回退到硬编码配置（兼容性）
    settings = self.get_system_settings()
    data_sources = []
    # AKShare (默认启用)
    akshare_config = DataSourceConfig(
        name="AKShare",
        type=DataSourceType.AKSHARE,
        endpoint="https://akshare.akfamily.xyz",
        enabled=True,
        priority=1,
        description="AKShare开源金融数据接口"
    )
    data_sources.append(akshare_config)
    # Tushare (如果有配置)
    if settings.get("tushare_token"):
        tushare_config = DataSourceConfig(
            name="Tushare",
            type=DataSourceType.TUSHARE,
            api_key=settings.get("tushare_token"),
            endpoint="http://api.tushare.pro",
            enabled=True,
            priority=2,
            description="Tushare专业金融数据接口"
        )
        data_sources.append(tushare_config)
    # 按优先级排序
    data_sources.sort(key=lambda x: x.priority, reverse=True)
    return data_sources
async def get_data_source_configs_async(self) -> List[DataSourceConfig]:
    """获取数据源配置 - 优先从数据库读取，回退到硬编码（异步版本）"""
    try:
        # 🔥 优先从数据库读取配置（使用异步连接）
        from app.core.database import get_mongo_db
        db = get_mongo_db()
        config_collection = db.system_configs
        # 获取最新的激活配置
        config_data = await config_collection.find_one(
            {"is_active": True},
            sort=[("version", -1)]
        )
        if config_data and config_data.get('data_source_configs'):
            # 从数据库读取到配置
            data_source_configs = config_data.get('data_source_configs', [])
            print(f"✅ [unified_config] 从数据库读取到 {len(data_source_configs)} 个数据源配置")
            # 转换为 DataSourceConfig 对象
            result = []
            for ds_config in data_source_configs:
                try:
                    result.append(DataSourceConfig(**ds_config))
                except Exception as e:
                    print(f"⚠️ [unified_config] 解析数据源配置失败: {e}, 配置: {ds_config}")
                    continue
            # 按优先级排序（数字越大优先级越高）
            result.sort(key=lambda x: x.priority, reverse=True)
            return result
        else:
            print("⚠️ [unified_config] 数据库中没有数据源配置，使用硬编码配置")
    except Exception as e:
        print(f"⚠️ [unified_config] 从数据库读取数据源配置失败: {e}，使用硬编码配置")
    # 🔥 回退到硬编码配置（兼容性）
    settings = self.get_system_settings()
    data_sources = []
    # AKShare (默认启用)
    akshare_config = DataSourceConfig(
```

```
        name="AKShare",
        type=DataSourceType.AKSHARE,
        endpoint="https://akshare.akfamily.xyz",
        enabled=True,
        priority=1,
        description="AKShare开源金融数据接口"
    )
    data_sources.append(akshare_config)
    # Tushare (如果有配置)
    if settings.get("tushare_token"):
        tushare_config = DataSourceConfig(
            name="Tushare",
            type=DataSourceType.TUSHARE,
            api_key=settings.get("tushare_token"),
            endpoint="http://api.tushare.pro",
            enabled=True,
            priority=2,
            description="Tushare专业金融数据接口"
        )
        data_sources.append(tushare_config)
    # Finnhub (如果有配置)
    if settings.get("finnhub_api_key"):
        finnhub_config = DataSourceConfig(
            name="Finnhub",
            type=DataSourceType.FINNHUB,
            api_key=settings.get("finnhub_api_key"),
            endpoint="https://finnhub.io/api/v1",
            enabled=True,
            priority=3,
            description="Finnhub股票数据接口"
        )
        data_sources.append(finnhub_config)
    return data_sources

# ==================== 数据库配置管理 ====================

def get_database_configs(self) -> List[DatabaseConfig]:
    """获取数据库配置"""
    configs = []

    # MongoDB配置
    mongodb_config = DatabaseConfig(
        name="MongoDB主库",
        type=DatabaseType.MONGODB,
        host=os.getenv("MONGODB_HOST", "localhost"),
        port=int(os.getenv("MONGODB_PORT", "27017")),
        database=os.getenv("MONGODB_DATABASE", "tradingagents"),
        enabled=True,
        description="MongoDB主数据库"
    )
    configs.append(mongodb_config)

    # Redis配置
    redis_config = DatabaseConfig(
        name="Redis缓存",
        type=DatabaseType.REDIS,
        host=os.getenv("REDIS_HOST", "localhost"),
        port=int(os.getenv("REDIS_PORT", "6379")),
        database=os.getenv("REDIS_DB", "0"),
        enabled=True,
        description="Redis缓存数据库"
    )
    configs.append(redis_config)

    return configs

# ==================== 统一配置接口 ====================

async def get_unified_system_config(self) -> SystemConfig:
    """获取统一的系统配置"""
    try:
        config = SystemConfig(
            config_name="统一系统配置",
            config_type="unified",
            llm_configs=self.get_llm_configs(),
            default_llm=self.get_default_model(),
            data_source_configs=self.get_data_source_configs(),
            default_data_source="AKShare",
            database_configs=self.get_database_configs(),
            system_settings=self.get_system_settings()
        )
        return config
    except Exception as e:
        print(f"获取统一配置失败: {e}")
        # 返回默认配置
        return SystemConfig(
            config_name="默认配置",
            config_type="default",
            llm_configs=[],
            data_source_configs=[],
            database_configs=[],
            system_settings={}
        )

def sync_to_legacy_format(self, system_config: SystemConfig) -> bool:
    """同步配置到传统格式"""
    try:
        # 同步模型配置
        for llm_config in system_config.llm_configs:
            self.save_llm_config(llm_config)
        # 读取现有的 settings.json
        current_settings = self.get_system_settings()
        # 同步系统设置（保留现有字段，只更新需要的字段）
        settings = current_settings.copy()
        # 映射新字段名到旧字段名
```

```
        if "quick_analysis_model" in system_config.system_settings:
            settings["quick_think_llm"] = system_config.system_settings["quick_analysis_model"]
            settings["quick_analysis_model"] = system_config.system_settings["quick_analysis_model"]
        if "deep_analysis_model" in system_config.system_settings:
            settings["deep_think_llm"] = system_config.system_settings["deep_analysis_model"]
            settings["deep_analysis_model"] = system_config.system_settings["deep_analysis_model"]
        if system_config.default_llm:
            settings["default_model"] = system_config.default_llm
        self.save_system_settings(settings)
        return True
    except Exception as e:
        print(f"同步配置到传统格式失败: {e}")
        return False

# 创建全局实例
unified_config = UnifiedConfigManager()
```

FILE: app/main.py

```
"""
TradingAgents-CN v1.0.0-preview FastAPI Backend
主应用程序入口
Copyright (c) 2025 hsliuping. All rights reserved.
版权所有 (c) 2025 hsliuping。保留所有权利。
This software is proprietary and confidential. Unauthorized copying, distribution,
or use of this software, via any medium, is strictly prohibited.
本软件为专有和机密软件。严禁通过任何媒介未经授权复制、分发或使用本软件。
For commercial licensing, please contact: hsliup@163.com
商业许可咨询，请联系：hsliup@163.com
"""
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware
from fastapi.responses import JSONResponse
import uvicorn
import logging
import time
from datetime import datetime
from contextlib import asynccontextmanager
import asyncio
from pathlib import Path
from app.core.config import settings
from app.core.database import init_db, close_db
from app.core.logging_config import setup_logging
from app.routers import auth_db as auth, analysis, screening, queue, sse, health, favorites, config, reports, database, operation_logs, tags, tushare_init, akshare_init, baostock_init, historical_data, multi_period_sync, financial_data, news_data, social_r
from app.routers import sync as sync_router, multi_source_sync
from app.routers import stocks as stocks_router
from app.routers import stock_data as stock_data_router
from app.routers import stock_sync as stock_sync_router
from app.routers import multi_market_stocks as multi_market_stocks_router
from app.routers import notifications as notifications_router
from app.routers import websocket_notifications as websocket_notifications_router
from app.routers import scheduler as scheduler_router
from app.services.basics_sync_service import get_basics_sync_service
from app.services.multi_source_basics_sync_service import MultiSourceBasicsSyncService
from app.services.scheduler_service import set_scheduler_instance
from app.worker.tushare_sync_service import (
    run_tushare_basic_info_sync,
    run_tushare_quotes_sync,
    run_tushare_historical_sync,
    run_tushare_financial_sync,
    run_tushare_status_check
)
from app.worker.akshare_sync_service import (
    run_akshare_basic_info_sync,
    run_akshare_quotes_sync,
    run_akshare_historical_sync,
    run_akshare_financial_sync,
    run_akshare_status_check
)
from app.worker.baostock_sync_service import (
    run_baostock_basic_info_sync,
    run_baostock_daily_quotes_sync,
    run_baostock_historical_sync,
    run_baostock_status_check
)
# 港股和美股改为按需获取+缓存模式，不再需要定时同步任务
# from app.worker.hk_sync_service import ...
# from app.worker.us_sync_service import ...
from app.middleware.operation_log_middleware import OperationLogMiddleware
from apscheduler.schedulers.asyncio import AsyncIOScheduler
from apscheduler.triggers.cron import CronTrigger
from apscheduler.triggers.interval import IntervalTrigger
from app.services.quotes_ingestion_service import QuotesIngestionService
from app.routers import paper as paper_router

def get_version() -> str:
    """从 VERSION 文件读取版本号"""
    try:
        version_file = Path(__file__).parent.parent / "VERSION"
        if version_file.exists():
            return version_file.read_text(encoding='utf-8').strip()
    except Exception:
        pass
    return "1.0.0"  # 默认版本号

async def _print_config_summary(logger):
    """显示配置摘要"""
    try:
        logger.info("=" * 70)
        logger.info("📋 TradingAgents-CN Configuration Summary")
        logger.info("=" * 70)
        # .env 文件路径信息
        import os
        from pathlib import Path
```

```
current_dir = Path.cwd()
logger.info(f"📂 Current working directory: {current_dir}")

# 检查可能的 .env 文件位置
env_files_to_check = [
    current_dir / ".env",
    current_dir / "app" / ".env",
    Path(__file__).parent.parent / ".env",  # 项目根目录
]

logger.info("🔍 Checking .env file locations:")
env_file_found = False
for env_file in env_files_to_check:
    if env_file.exists():
        logger.info(f"  ✅ Found: {env_file} (size: {env_file.stat().st_size} bytes)")
        env_file_found = True
        # 显示文件的前几行（隐藏敏感信息）
        try:
            with open(env_file, 'r', encoding='utf-8') as f:
                lines = f.readlines()[:5]  # 只读前5行
                logger.info(f"    Preview (first 5 lines):")
                for i, line in enumerate(lines, 1):
                    # 隐藏包含密码、密钥等敏感信息的行
                    if any(keyword in line.upper() for keyword in ['PASSWORD', 'SECRET', 'KEY', 'TOKEN']):
                        logger.info(f"      {i}: {line.split('=')[0]}=***")
                    else:
                        logger.info(f"      {i}: {line.strip()}")
        except Exception as e:
            logger.warning(f"    Could not preview file: {e}")
    else:
        logger.info(f"  ❌ Not found: {env_file}")

if not env_file_found:
    logger.warning("⚠️  No .env file found in checked locations")

# Pydantic Settings 配置加载状态
logger.info("⚙️  Pydantic Settings Configuration:")
logger.info(f"  • Settings class: {settings.__class__.__name__}")
logger.info(f"  • Config source: {getattr(settings.model_config, 'env_file', 'Not specified')}")
logger.info(f"  • Encoding: {getattr(settings.model_config, 'env_file_encoding', 'Not specified')}")

# 显示一些关键配置值的来源（环境变量 vs 默认值）
key_settings = ['HOST', 'PORT', 'DEBUG', 'MONGODB_HOST', 'REDIS_HOST']
logger.info("  • Key settings sources:")
for setting_name in key_settings:
    env_var_name = setting_name
    env_value = os.getenv(env_var_name)
    config_value = getattr(settings, setting_name, None)
    if env_value is not None:
        logger.info(f"    - {setting_name}: from environment variable ({config_value})")
    else:
        logger.info(f"    - {setting_name}: using default value ({config_value})")

# 环境信息
env = "Production" if settings.is_production else "Development"
logger.info(f"Environment: {env}")
# 数据库连接
logger.info(f"MongoDB: {settings.MONGODB_HOST}:{settings.MONGODB_PORT}/{settings.MONGODB_DATABASE}")
logger.info(f"Redis: {settings.REDIS_HOST}:{settings.REDIS_PORT}/{settings.REDIS_DB}")
# 代理配置
import os
if settings.HTTP_PROXY or settings.HTTPS_PROXY:
    logger.info("Proxy Configuration:")
    if settings.HTTP_PROXY:
        logger.info(f"  HTTP_PROXY: {settings.HTTP_PROXY}")
    if settings.HTTPS_PROXY:
        logger.info(f"  HTTPS_PROXY: {settings.HTTPS_PROXY}")
    if settings.NO_PROXY:
        # 只显示前3个域名
        no_proxy_list = settings.NO_PROXY.split(',')
        if len(no_proxy_list) <= 3:
            logger.info(f"  NO_PROXY: {settings.NO_PROXY}")
        else:
            logger.info(f"  NO_PROXY: {','.join(no_proxy_list[:3])}... ({len(no_proxy_list)} domains)")
    logger.info(f"  ✅ Proxy environment variables set successfully")
else:
    logger.info("Proxy: Not configured (direct connection)")
# 检查大模型配置
try:
    from app.services.config_service import config_service
    config = await config_service.get_system_config()
    if config and config.llm_configs:
        enabled_llms = [llm for llm in config.llm_configs if llm.enabled]
        logger.info(f"Enabled LLMs: {len(enabled_llms)}")
        if enabled_llms:
            for llm in enabled_llms[:3]:  # 只显示前3个
                logger.info(f"  • {llm.provider}: {llm.model_name}")
            if len(enabled_llms) > 3:
                logger.info(f"  • ... and {len(enabled_llms) - 3} more")
        else:
            logger.warning("⚠️  No LLM enabled. Please configure at least one LLM in Web UI.")
    else:
        logger.warning("⚠️  No LLM configured. Please configure at least one LLM in Web UI.")
except Exception as e:
    logger.warning(f"⚠️  Failed to check LLM configs: {e}")
# 检查数据源配置
try:
    if config and config.data_source_configs:
        enabled_sources = [ds for ds in config.data_source_configs if ds.enabled]
        logger.info(f"Enabled Data Sources: {len(enabled_sources)}")
        if enabled_sources:
            for ds in enabled_sources[:3]:  # 只显示前3个
                logger.info(f"  • {ds.type.value}: {ds.name}")
            if len(enabled_sources) > 3:
```

```
                logger.info(f"  • ... and {len(enabled_sources) - 3} more")
        else:
            logger.info("Data Sources: Using default (AKShare)")
    except Exception as e:
        logger.warning(f"⚠️  Failed to check data source configs: {e}")
    logger.info("=" * 70)
except Exception as e:
    logger.error(f"Failed to print config summary: {e}")

@asynccontextmanager
async def lifespan(app: FastAPI):
    """应用生命周期管理"""
    # 启动时初始化
    setup_logging()
    logger = logging.getLogger("app.main")
    # 验证启动配置
    try:
        from app.core.startup_validator import validate_startup_config
        validate_startup_config()
    except Exception as e:
        logger.error(f"配置验证失败: {e}")
        raise
    await init_db()
    # 配置桥接：将统一配置写入环境变量，供 TradingAgents 核心库使用
    try:
        from app.core.config_bridge import bridge_config_to_env
        bridge_config_to_env()
    except Exception as e:
        logger.warning(f"⚠️  配置桥接失败: {e}")
        logger.warning("⚠️  TradingAgents 将使用 .env 文件中的配置")
    # Apply dynamic settings (log_level, enable_monitoring) from ConfigProvider
    try:
        from app.services.config_provider import provider as config_provider  # local import to avoid early DB init issues
        eff = await config_provider.get_effective_system_settings()
        desired_level = str(eff.get("log_level", "INFO")).upper()
        setup_logging(log_level=desired_level)
        for name in ("webapi", "worker", "uvicorn", "fastapi"):
            logging.getLogger(name).setLevel(desired_level)
        try:
            from app.middleware.operation_log_middleware import set_operation_log_enabled
            set_operation_log_enabled(bool(eff.get("enable_monitoring", True)))
        except Exception:
            pass
    except Exception as e:
        logging.getLogger("webapi").warning(f"Failed to apply dynamic settings: {e}")
    # 显示配置摘要
    await _print_config_summary(logger)
    logger.info("TradingAgents FastAPI backend started")
    # 启动期：若需要在休市时补充上一交易日收盘快照
    if settings.QUOTES_BACKFILL_ON_STARTUP:
        try:
            qi = QuotesIngestionService()
            await qi.ensure_indexes()
            await qi.backfill_last_close_snapshot_if_needed()
        except Exception as e:
            logger.warning(f"Startup backfill failed (ignored): {e}")
    # 启动每日定时任务：可配置
    scheduler: AsyncIOScheduler | None = None
    try:
        from croniter import croniter
    except Exception:
        croniter = None  # 可选依赖
    try:
        scheduler = AsyncIOScheduler(timezone=settings.TIMEZONE)
        # 使用多数据源同步服务（支持自动切换）
        multi_source_service = MultiSourceBasicsSyncService()
        # 根据 TUSHARE_ENABLED 配置决定优先数据源
        # 如果 Tushare 被禁用，系统会自动使用其他可用数据源（AKShare/BaoStock）
        preferred_sources = None  # None 表示使用默认优先级顺序
        if settings.TUSHARE_ENABLED:
            # Tushare 启用时，优先使用 Tushare
            preferred_sources = ["tushare", "akshare", "baostock"]
            logger.info(f"📊 股票基础信息同步优先数据源: Tushare > AKShare > BaoStock")
        else:
            # Tushare 禁用时，使用 AKShare 和 BaoStock
            preferred_sources = ["akshare", "baostock"]
            logger.info(f"📊 股票基础信息同步优先数据源: AKShare > BaoStock (Tushare已禁用)")
        # 立即在启动后尝试一次（不阻塞）
        async def run_sync_with_sources():
            await multi_source_service.run_full_sync(force=False, preferred_sources=preferred_sources)
        asyncio.create_task(run_sync_with_sources())
        # 配置调度：优先使用 CRON，其次使用 HH:MM
        if settings.SYNC_STOCK_BASICS_ENABLED:
            if settings.SYNC_STOCK_BASICS_CRON:
                # 如果提供了cron表达式
                scheduler.add_job(
                    lambda: multi_source_service.run_full_sync(force=False, preferred_sources=preferred_sources),
                    CronTrigger.from_crontab(settings.SYNC_STOCK_BASICS_CRON, timezone=settings.TIMEZONE),
                    id="basics_sync_service",
                    name="股票基础信息同步（多数据源）"
                )
                logger.info(f"📅 Stock basics sync scheduled by CRON: {settings.SYNC_STOCK_BASICS_CRON} ({settings.TIMEZONE})")
            else:
                hh, mm = (settings.SYNC_STOCK_BASICS_TIME or "06:30").split(":")
                scheduler.add_job(
                    lambda: multi_source_service.run_full_sync(force=False, preferred_sources=preferred_sources),
                    CronTrigger(hour=int(hh), minute=int(mm), timezone=settings.TIMEZONE),
                    id="basics_sync_service",
                    name="股票基础信息同步（多数据源）"
                )
                logger.info(f"📅 Stock basics sync scheduled daily at {settings.SYNC_STOCK_BASICS_TIME} ({settings.TIMEZONE})")
        # 实时行情入库任务（每N秒），内部自判交易时段
        if settings.QUOTES_INGEST_ENABLED:
            quotes_ingestion = QuotesIngestionService()
```

```
    await quotes_ingestion.ensure_indexes()
    scheduler.add_job(
        quotes_ingestion.run_once,  # coroutine function; AsyncIOScheduler will await it
        IntervalTrigger(seconds=settings.QUOTES_INGEST_INTERVAL_SECONDS, timezone=settings.TIMEZONE),
        id="quotes_ingestion_service",
        name="实时行情入库服务"
    )
    logger.info(f"⏱ 实时行情入库任务已启动: 每 {settings.QUOTES_INGEST_INTERVAL_SECONDS}s")
# Tushare统一数据同步任务配置
logger.info("🔄 配置Tushare统一数据同步任务...")
# 基础信息同步任务
scheduler.add_job(
    run_tushare_basic_info_sync,
    CronTrigger.from_crontab(settings.TUSHARE_BASIC_INFO_SYNC_CRON, timezone=settings.TIMEZONE),
    id="tushare_basic_info_sync",
    name="股票基础信息同步（Tushare）",
    kwargs={"force_update": False}
)
if not (settings.TUSHARE_UNIFIED_ENABLED and settings.TUSHARE_BASIC_INFO_SYNC_ENABLED):
    scheduler.pause_job("tushare_basic_info_sync")
    logger.info(f"⏸ Tushare基础信息同步已添加但暂停: {settings.TUSHARE_BASIC_INFO_SYNC_CRON}")
else:
    logger.info(f"📅 Tushare基础信息同步已配置: {settings.TUSHARE_BASIC_INFO_SYNC_CRON}")
# 实时行情同步任务
scheduler.add_job(
    run_tushare_quotes_sync,
    CronTrigger.from_crontab(settings.TUSHARE_QUOTES_SYNC_CRON, timezone=settings.TIMEZONE),
    id="tushare_quotes_sync",
    name="实时行情同步（Tushare）"
)
if not (settings.TUSHARE_UNIFIED_ENABLED and settings.TUSHARE_QUOTES_SYNC_ENABLED):
    scheduler.pause_job("tushare_quotes_sync")
    logger.info(f"⏸ Tushare行情同步已添加但暂停: {settings.TUSHARE_QUOTES_SYNC_CRON}")
else:
    logger.info(f"📊 Tushare行情同步已配置: {settings.TUSHARE_QUOTES_SYNC_CRON}")
# 历史数据同步任务
scheduler.add_job(
    run_tushare_historical_sync,
    CronTrigger.from_crontab(settings.TUSHARE_HISTORICAL_SYNC_CRON, timezone=settings.TIMEZONE),
    id="tushare_historical_sync",
    name="历史数据同步（Tushare）",
    kwargs={"incremental": True}
)
if not (settings.TUSHARE_UNIFIED_ENABLED and settings.TUSHARE_HISTORICAL_SYNC_ENABLED):
    scheduler.pause_job("tushare_historical_sync")
    logger.info(f"⏸ Tushare历史数据同步已添加但暂停: {settings.TUSHARE_HISTORICAL_SYNC_CRON}")
else:
    logger.info(f"📈 Tushare历史数据同步已配置: {settings.TUSHARE_HISTORICAL_SYNC_CRON}")
# 财务数据同步任务
scheduler.add_job(
    run_tushare_financial_sync,
    CronTrigger.from_crontab(settings.TUSHARE_FINANCIAL_SYNC_CRON, timezone=settings.TIMEZONE),
    id="tushare_financial_sync",
    name="财务数据同步（Tushare）"
)
if not (settings.TUSHARE_UNIFIED_ENABLED and settings.TUSHARE_FINANCIAL_SYNC_ENABLED):
    scheduler.pause_job("tushare_financial_sync")
    logger.info(f"⏸ Tushare财务数据同步已添加但暂停: {settings.TUSHARE_FINANCIAL_SYNC_CRON}")
else:
    logger.info(f"⏱ Tushare财务数据同步已配置: {settings.TUSHARE_FINANCIAL_SYNC_CRON}")
# 状态检查任务
scheduler.add_job(
    run_tushare_status_check,
    CronTrigger.from_crontab(settings.TUSHARE_STATUS_CHECK_CRON, timezone=settings.TIMEZONE),
    id="tushare_status_check",
    name="数据源状态检查（Tushare）"
)
if not (settings.TUSHARE_UNIFIED_ENABLED and settings.TUSHARE_STATUS_CHECK_ENABLED):
    scheduler.pause_job("tushare_status_check")
    logger.info(f"⏸ Tushare状态检查已添加但暂停: {settings.TUSHARE_STATUS_CHECK_CRON}")
else:
    logger.info(f"🔍 Tushare状态检查已配置: {settings.TUSHARE_STATUS_CHECK_CRON}")
# AKShare统一数据同步任务配置
logger.info("🔄 配置AKShare统一数据同步任务...")
# 基础信息同步任务
scheduler.add_job(
    run_akshare_basic_info_sync,
    CronTrigger.from_crontab(settings.AKSHARE_BASIC_INFO_SYNC_CRON, timezone=settings.TIMEZONE),
    id="akshare_basic_info_sync",
    name="股票基础信息同步（AKShare）",
    kwargs={"force_update": False}
)
if not (settings.AKSHARE_UNIFIED_ENABLED and settings.AKSHARE_BASIC_INFO_SYNC_ENABLED):
    scheduler.pause_job("akshare_basic_info_sync")
    logger.info(f"⏸ AKShare基础信息同步已添加但暂停: {settings.AKSHARE_BASIC_INFO_SYNC_CRON}")
else:
    logger.info(f"📅 AKShare基础信息同步已配置: {settings.AKSHARE_BASIC_INFO_SYNC_CRON}")
# 实时行情同步任务
scheduler.add_job(
    run_akshare_quotes_sync,
    CronTrigger.from_crontab(settings.AKSHARE_QUOTES_SYNC_CRON, timezone=settings.TIMEZONE),
    id="akshare_quotes_sync",
    name="实时行情同步（AKShare）"
)
if not (settings.AKSHARE_UNIFIED_ENABLED and settings.AKSHARE_QUOTES_SYNC_ENABLED):
    scheduler.pause_job("akshare_quotes_sync")
    logger.info(f"⏸ AKShare行情同步已添加但暂停: {settings.AKSHARE_QUOTES_SYNC_CRON}")
else:
    logger.info(f"📊 AKShare行情同步已配置: {settings.AKSHARE_QUOTES_SYNC_CRON}")
# 历史数据同步任务
scheduler.add_job(
    run_akshare_historical_sync,
    CronTrigger.from_crontab(settings.AKSHARE_HISTORICAL_SYNC_CRON, timezone=settings.TIMEZONE),
    id="akshare_historical_sync",
```

```
    name="历史数据同步（AKShare）",
    kwargs={"incremental": True}
)
if not (settings.AKSHARE_UNIFIED_ENABLED and settings.AKSHARE_HISTORICAL_SYNC_ENABLED):
    scheduler.pause_job("akshare_historical_sync")
    logger.info(f"⏸️ AKShare历史数据同步已添加但暂停: {settings.AKSHARE_HISTORICAL_SYNC_CRON}")
else:
    logger.info(f"📊 AKShare历史数据同步已配置: {settings.AKSHARE_HISTORICAL_SYNC_CRON}")
# 财务数据同步任务
scheduler.add_job(
    run_akshare_financial_sync,
    CronTrigger.from_crontab(settings.AKSHARE_FINANCIAL_SYNC_CRON, timezone=settings.TIMEZONE),
    id="akshare_financial_sync",
    name="财务数据同步（AKShare）"
)
if not (settings.AKSHARE_UNIFIED_ENABLED and settings.AKSHARE_FINANCIAL_SYNC_ENABLED):
    scheduler.pause_job("akshare_financial_sync")
    logger.info(f"⏸️ AKShare财务数据同步已添加但暂停: {settings.AKSHARE_FINANCIAL_SYNC_CRON}")
else:
    logger.info(f"⏱️ AKShare财务数据同步已配置: {settings.AKSHARE_FINANCIAL_SYNC_CRON}")
# 状态检查任务
scheduler.add_job(
    run_akshare_status_check,
    CronTrigger.from_crontab(settings.AKSHARE_STATUS_CHECK_CRON, timezone=settings.TIMEZONE),
    id="akshare_status_check",
    name="数据源状态检查（AKShare）"
)
if not (settings.AKSHARE_UNIFIED_ENABLED and settings.AKSHARE_STATUS_CHECK_ENABLED):
    scheduler.pause_job("akshare_status_check")
    logger.info(f"⏸️ AKShare状态检查已添加但暂停: {settings.AKSHARE_STATUS_CHECK_CRON}")
else:
    logger.info(f"🔍 AKShare状态检查已配置: {settings.AKSHARE_STATUS_CHECK_CRON}")
# BaoStock统一数据同步任务配置
logger.info("🔄 配置BaoStock统一数据同步任务...")
# 基础信息同步任务
scheduler.add_job(
    run_baostock_basic_info_sync,
    CronTrigger.from_crontab(settings.BAOSTOCK_BASIC_INFO_SYNC_CRON, timezone=settings.TIMEZONE),
    id="baostock_basic_info_sync",
    name="股票基础信息同步（BaoStock）"
)
if not (settings.BAOSTOCK_UNIFIED_ENABLED and settings.BAOSTOCK_BASIC_INFO_SYNC_ENABLED):
    scheduler.pause_job("baostock_basic_info_sync")
    logger.info(f"⏸️ BaoStock基础信息同步已添加但暂停: {settings.BAOSTOCK_BASIC_INFO_SYNC_CRON}")
else:
    logger.info(f"📋 BaoStock基础信息同步已配置: {settings.BAOSTOCK_BASIC_INFO_SYNC_CRON}")
# 日K线同步任务（注意：BaoStock不支持实时行情）
scheduler.add_job(
    run_baostock_daily_quotes_sync,
    CronTrigger.from_crontab(settings.BAOSTOCK_DAILY_QUOTES_SYNC_CRON, timezone=settings.TIMEZONE),
    id="baostock_daily_quotes_sync",
    name="日K线数据同步（BaoStock）"
)
if not (settings.BAOSTOCK_UNIFIED_ENABLED and settings.BAOSTOCK_DAILY_QUOTES_SYNC_ENABLED):
    scheduler.pause_job("baostock_daily_quotes_sync")
    logger.info(f"⏸️ BaoStock日K线同步已添加但暂停: {settings.BAOSTOCK_DAILY_QUOTES_SYNC_CRON}")
else:
    logger.info(f"📈 BaoStock日K线同步已配置: {settings.BAOSTOCK_DAILY_QUOTES_SYNC_CRON} (注意：BaoStock不支持实时行情)")
# 历史数据同步任务
scheduler.add_job(
    run_baostock_historical_sync,
    CronTrigger.from_crontab(settings.BAOSTOCK_HISTORICAL_SYNC_CRON, timezone=settings.TIMEZONE),
    id="baostock_historical_sync",
    name="历史数据同步（BaoStock）"
)
if not (settings.BAOSTOCK_UNIFIED_ENABLED and settings.BAOSTOCK_HISTORICAL_SYNC_ENABLED):
    scheduler.pause_job("baostock_historical_sync")
    logger.info(f"⏸️ BaoStock历史数据同步已添加但暂停: {settings.BAOSTOCK_HISTORICAL_SYNC_CRON}")
else:
    logger.info(f"📊 BaoStock历史数据同步已配置: {settings.BAOSTOCK_HISTORICAL_SYNC_CRON}")
# 状态检查任务
scheduler.add_job(
    run_baostock_status_check,
    CronTrigger.from_crontab(settings.BAOSTOCK_STATUS_CHECK_CRON, timezone=settings.TIMEZONE),
    id="baostock_status_check",
    name="数据源状态检查（BaoStock）"
)
if not (settings.BAOSTOCK_UNIFIED_ENABLED and settings.BAOSTOCK_STATUS_CHECK_ENABLED):
    scheduler.pause_job("baostock_status_check")
    logger.info(f"⏸️ BaoStock状态检查已添加但暂停: {settings.BAOSTOCK_STATUS_CHECK_CRON}")
else:
    logger.info(f"🔍 BaoStock状态检查已配置: {settings.BAOSTOCK_STATUS_CHECK_CRON}")
# 新闻数据同步任务配置（使用AKShare同步所有股票新闻）
logger.info("🔄 配置新闻数据同步任务...")
from app.worker.akshare_sync_service import get_akshare_sync_service
async def run_news_sync():
    """运行新闻同步任务 - 使用AKShare同步自选股新闻"""
    try:
        logger.info("📰 开始新闻数据同步（AKShare - 仅自选股）...")
        service = await get_akshare_sync_service()
        result = await service.sync_news_data(
            symbols=None,  # None + favorites_only=True 表示只同步自选股
            max_news_per_stock=settings.NEWS_SYNC_MAX_PER_SOURCE,
            favorites_only=True  # 只同步自选股
        )
        logger.info(
            f"✅ 新闻同步完成: "
            f"处理{result['total_processed']}只自选股, "
            f"成功{result['success_count']}只, "
            f"失败{result['error_count']}只, "
            f"新闻总数{result['news_count']}条, "
            f"耗时{(datetime.utcnow() - result['start_time']).total_seconds():.2f}秒"
        )
    except Exception as e:
```

```
            logger.error(f"❌ 新闻同步失败: {e}", exc_info=True)
        # ==================== 港股/美股数据配置 ====================
        # 港股和美股采用按需获取+缓存模式，不再配置定时同步任务
        logger.info("HK 港股数据采用按需获取+缓存模式")
        logger.info("US 美股数据采用按需获取+缓存模式")
        scheduler.add_job(
            run_news_sync,
            CronTrigger.from_crontab(settings.NEWS_SYNC_CRON, timezone=settings.TIMEZONE),
            id="news_sync",
            name="新闻数据同步（AKShare - 仅自选股）"
        )
        if not settings.NEWS_SYNC_ENABLED:
            scheduler.pause_job("news_sync")
            logger.info(f"⏸️ 新闻数据同步已添加但暂停: {settings.NEWS_SYNC_CRON}")
        else:
            logger.info(f"📰 新闻数据同步已配置（仅自选股）: {settings.NEWS_SYNC_CRON}")
        scheduler.start()
        # 设置调度器实例到服务中，以便API可以管理任务
        set_scheduler_instance(scheduler)
        logger.info("✅ 调度器服务已初始化")
    except Exception as e:
        logger.error(f"❌ 调度器启动失败: {e}", exc_info=True)
        raise  # 抛出异常，阻止应用启动
    try:
        yield
    finally:
        # 关闭时清理
        if scheduler:
            try:
                scheduler.shutdown(wait=False)
                logger.info("🔴 Scheduler stopped")
            except Exception as e:
                logger.warning(f"Scheduler shutdown error: {e}")
        # 关闭 UserService MongoDB 连接
        try:
            from app.services.user_service import user_service
            user_service.close()
        except Exception as e:
            logger.warning(f"UserService cleanup error: {e}")
        await close_db()
        logger.info("TradingAgents FastAPI backend stopped")
# 创建FastAPI应用
app = FastAPI(
    title="TradingAgents-CN API",
    description="股票分析与批量队列系统 API",
    version=get_version(),
    docs_url="/docs" if settings.DEBUG else None,
    redoc_url="/redoc" if settings.DEBUG else None,
    lifespan=lifespan
)
# 安全中间件
if not settings.DEBUG:
    app.add_middleware(
        TrustedHostMiddleware,
        allowed_hosts=settings.ALLOWED_HOSTS
    )
# CORS中间件
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allow_headers=["*"],
)
# 操作日志中间件
app.add_middleware(OperationLogMiddleware)
# 请求日志中间件
@app.middleware("http")
async def log_requests(request: Request, call_next):
    start_time = time.time()
    # 跳过健康检查和静态文件请求的日志
    if request.url.path in ["/health", "/favicon.ico"] or request.url.path.startswith("/static"):
        response = await call_next(request)
        return response
    # 使用webapi logger记录请求
    logger = logging.getLogger("webapi")
    logger.info(f"🔄 {request.method} {request.url.path} - 开始处理")
    response = await call_next(request)
    process_time = time.time() - start_time
    # 记录请求完成
    status_emoji = "✅" if response.status_code < 400 else "❌"
    logger.info(f"{status_emoji} {request.method} {request.url.path} - 状态: {response.status_code} - 耗时: {process_time:.3f}s")
    return response
# 全局异常处理
# 请求ID/Trace-ID 中间件（需作为最外层，放在函数式中间件之后）
from app.middleware.request_id import RequestIDMiddleware
app.add_middleware(RequestIDMiddleware)
@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    logging.error(f"Unhandled exception: {exc}", exc_info=True)
    return JSONResponse(
        status_code=500,
        content={
            "error": {
                "code": "INTERNAL_SERVER_ERROR",
                "message": "Internal server error occurred",
                "request_id": getattr(request.state, "request_id", None)
            }
        }
    )
# 测试端点 - 验证中间件是否工作
@app.get("/api/test-log")
```

```
async def test_log():
    """测试日志中间件是否工作"""
    print("✏️ 测试端点被调用 - 这条消息应该出现在控制台")
    return {"message": "测试成功", "timestamp": time.time()}
# 注册路由
app.include_router(health.router, prefix="/api", tags=["health"])
app.include_router(auth.router, prefix="/api/auth", tags=["authentication"])
app.include_router(analysis.router, prefix="/api/analysis", tags=["analysis"])
app.include_router(reports.router, tags=["reports"])
app.include_router(screening.router, prefix="/api/screening", tags=["screening"])
app.include_router(queue.router, prefix="/api/queue", tags=["queue"])
app.include_router(favorites.router, prefix="/api", tags=["favorites"])
app.include_router(stocks_router.router, prefix="/api", tags=["stocks"])
app.include_router(multi_market_stocks_router.router, prefix="/api", tags=["multi-market"])
app.include_router(stock_data_router.router, tags=["stock-data"])
app.include_router(stock_sync_router.router, tags=["stock-sync"])
app.include_router(tags.router, prefix="/api", tags=["tags"])
app.include_router(config.router, prefix="/api", tags=["config"])
app.include_router(model_capabilities.router, tags=["model-capabilities"])
app.include_router(usage_statistics.router, tags=["usage-statistics"])
app.include_router(database.router, prefix="/api/system", tags=["database"])
app.include_router(cache.router, tags=["cache"])
app.include_router(operation_logs.router, prefix="/api/system", tags=["operation_logs"])
app.include_router(logs.router, prefix="/api/system", tags=["logs"])
# 新增：系统配置只读摘要
from app.routers import system_config as system_config_router
app.include_router(system_config_router.router, prefix="/api/system", tags=["system"])
# 通知模块（REST + SSE）
app.include_router(notifications_router.router, prefix="/api", tags=["notifications"])
# 🔥 WebSocket 通知模块（替代 SSE + Redis PubSub）
app.include_router(websocket_notifications_router.router, prefix="/api", tags=["websocket"])
# 定时任务管理
app.include_router(scheduler_router.router, tags=["scheduler"])
app.include_router(sse.router, prefix="/api/stream", tags=["streaming"])
app.include_router(sync_router.router)
app.include_router(multi_source_sync.router)
app.include_router(paper_router.router, prefix="/api", tags=["paper"])
app.include_router(tushare_init.router, prefix="/api", tags=["tushare-init"])
app.include_router(akshare_init.router, prefix="/api", tags=["akshare-init"])
app.include_router(baostock_init.router, prefix="/api", tags=["baostock-init"])
app.include_router(historical_data.router, tags=["historical-data"])
app.include_router(multi_period_sync.router, tags=["multi-period-sync"])
app.include_router(financial_data.router, tags=["financial-data"])
app.include_router(news_data.router, tags=["news-data"])
app.include_router(social_media.router, tags=["social-media"])
app.include_router(internal_messages.router, tags=["internal-messages"])
@app.get("/")
async def root():
    """根路径，返回API信息"""
    print("🏠 根路径被访问")
    return {
        "name": "TradingAgents-CN API",
        "version": get_version(),
        "status": "running",
        "docs_url": "/docs" if settings.DEBUG else None
    }
if __name__ == "__main__":
    uvicorn.run(
        "app.main:app",
        host=settings.HOST,
        port=settings.PORT,
        reload=settings.DEBUG,
        log_level="info",
        reload_dirs=["app"] if settings.DEBUG else None,
        reload_excludes=[
            "__pycache__",
            "*.pyc",
            "*.pyo",
            "*.pyd",
            ".git",
            ".pytest_cache",
            "*.log",
            "*.tmp"
        ] if settings.DEBUG else None,
        reload_includes=["*.py"] if settings.DEBUG else None
    )
```

FILE: app/middleware/__init__.py

```
"""
中间件模块
"""
```

FILE: app/middleware/error_handler.py

```
"""
错误处理中间件
"""
from fastapi import Request, Response
from fastapi.responses import JSONResponse
from starlette.middleware.base import BaseHTTPMiddleware
import logging
import traceback
from typing import Callable
logger = logging.getLogger(__name__)
class ErrorHandlerMiddleware(BaseHTTPMiddleware):
    """全局错误处理中间件"""

    async def dispatch(self, request: Request, call_next: Callable) -> Response:
        try:
            response = await call_next(request)
```

```
        return response
    except Exception as exc:
        return await self.handle_error(request, exc)

async def handle_error(self, request: Request, exc: Exception) -> JSONResponse:
    """处理异常并返回标准化错误响应"""

    # 获取请求ID
    request_id = getattr(request.state, "request_id", "unknown")

    # 记录错误日志
    logger.error(
        f"请求异常 - ID: {request_id}, "
        f"路径: {request.url.path}, "
        f"方法: {request.method}, "
        f"异常: {str(exc)}",
        exc_info=True
    )

    # 根据异常类型返回不同的错误响应
    if isinstance(exc, ValueError):
        return JSONResponse(
            status_code=400,
            content={
                "error": {
                    "code": "VALIDATION_ERROR",
                    "message": str(exc),
                    "request_id": request_id
                }
            }
        )

    elif isinstance(exc, PermissionError):
        return JSONResponse(
            status_code=403,
            content={
                "error": {
                    "code": "PERMISSION_DENIED",
                    "message": "权限不足",
                    "request_id": request_id
                }
            }
        )

    elif isinstance(exc, FileNotFoundError):
        return JSONResponse(
            status_code=404,
            content={
                "error": {
                    "code": "RESOURCE_NOT_FOUND",
                    "message": "请求的资源不存在",
                    "request_id": request_id
                }
            }
        )

    else:
        # 未知异常
        return JSONResponse(
            status_code=500,
            content={
                "error": {
                    "code": "INTERNAL_SERVER_ERROR",
                    "message": "服务器内部错误，请稍后重试",
                    "request_id": request_id
                }
            }
        )
```

FILE: app/middleware/operation_log_middleware.py

```
"""
操作日志记录中间件
自动记录用户的API操作日志
"""
import time
import json
import logging
from typing import Optional, Dict, Any
from fastapi import Request, Response
from starlette.middleware.base import BaseHTTPMiddleware
from app.services.operation_log_service import log_operation
from app.models.operation_log import ActionType
logger = logging.getLogger("webapi")
# 全局开关：是否启用操作日志记录（可由系统设置动态控制）
OPLOG_ENABLED: bool = True
def set_operation_log_enabled(flag: bool) -> None:
    global OPLOG_ENABLED
    OPLOG_ENABLED = bool(flag)

class OperationLogMiddleware(BaseHTTPMiddleware):
    """操作日志记录中间件"""
    def __init__(self, app, skip_paths: Optional[list] = None):
        super().__init__(app)
        # 跳过记录日志的路径
        self.skip_paths = skip_paths or [
            "/health",
            "/healthz",
            "/readyz",
            "/favicon.ico",
            "/docs",
            "/redoc",
            "/openapi.json",
            "/api/stream/",  # SSE流不记录
```

```
        "/api/system/logs/",  # 操作日志API本身不记录
    ]
    # 路径到操作类型的映射
    self.path_action_mapping = {
        "/api/analysis/": ActionType.STOCK_ANALYSIS,
        "/api/screening/": ActionType.SCREENING,
        "/api/config/": ActionType.CONFIG_MANAGEMENT,
        "/api/system/database/": ActionType.DATABASE_OPERATION,
        "/api/auth/login": ActionType.USER_LOGIN,
        "/api/auth/logout": ActionType.USER_LOGOUT,
        "/api/auth/change-password": ActionType.USER_MANAGEMENT,  # 🔧 添加修改密码操作类型
        "/api/reports/": ActionType.REPORT_GENERATION,
    }
async def dispatch(self, request: Request, call_next):
    # 检查是否需要跳过记录
    if self._should_skip_logging(request):
        return await call_next(request)
    # 记录开始时间
    start_time = time.time()
    # 获取请求信息
    method = request.method
    path = request.url.path
    ip_address = self._get_client_ip(request)
    user_agent = request.headers.get("user-agent", "")
    # 获取用户信息（如果已认证）
    user_info = await self._get_user_info(request)
    # 处理请求
    response = await call_next(request)
    # 计算耗时
    duration_ms = int((time.time() - start_time) * 1000)
    # 异步记录操作日志
    if user_info:
        try:
            await self._log_operation(
                user_info=user_info,
                method=method,
                path=path,
                response=response,
                duration_ms=duration_ms,
                ip_address=ip_address,
                user_agent=user_agent,
                request=request
            )
        except Exception as e:
            logger.error(f"记录操作日志失败: {e}")
    return response
def _should_skip_logging(self, request: Request) -> bool:
    """判断是否应该跳过日志记录"""
    # 全局关闭时直接跳过
    if not OPLOG_ENABLED:
        return True
    path = request.url.path
    # 检查跳过路径
    for skip_path in self.skip_paths:
        if path.startswith(skip_path):
            return True
    # 只记录API请求
    if not path.startswith("/api/"):
        return True
    # 只记录特定HTTP方法
    if request.method not in ["POST", "PUT", "DELETE", "PATCH"]:
        return True
    return False
def _get_client_ip(self, request: Request) -> str:
    """获取客户端IP地址"""
    # 检查代理头
    forwarded_for = request.headers.get("x-forwarded-for")
    if forwarded_for:
        return forwarded_for.split(",")[0].strip()
    real_ip = request.headers.get("x-real-ip")
    if real_ip:
        return real_ip
    # 使用直接连接IP
    if request.client:
        return request.client.host
    return "unknown"
async def _get_user_info(self, request: Request) -> Optional[Dict[str, Any]]:
    """获取用户信息"""
    try:
        # 从请求状态中获取用户信息（由认证中间件设置）
        if hasattr(request.state, "user"):
            return request.state.user
        # 尝试从Authorization头解析用户信息
        auth_header = request.headers.get("authorization")
        if auth_header and auth_header.startswith("Bearer "):
            token = auth_header.split(" ", 1)[1]
            # 使用AuthService验证token
            from app.services.auth_service import AuthService
            token_data = AuthService.verify_token(token)
            if token_data:
                # 返回用户信息（开源版只有admin用户）
                return {
                    "id": "admin",
                    "username": "admin",
                    "name": "管理员",
                    "is_admin": True,
                    "roles": ["admin"]
                }
        return None
    except Exception as e:
        logger.debug(f"获取用户信息失败: {e}")
        return None
```

```
    def _get_action_type(self, path: str) -> str:
        """根据路径获取操作类型"""
        for path_prefix, action_type in self.path_action_mapping.items():
            if path.startswith(path_prefix):
                return action_type
        return ActionType.SYSTEM_SETTINGS  # 默认类型
    def _get_action_description(self, method: str, path: str, request: Request) -> str:
        """生成操作描述"""
        # 基础描述
        action_map = {
            "POST": "创建",
            "PUT": "更新",
            "PATCH": "修改",
            "DELETE": "删除"
        }
        action_verb = action_map.get(method, method)
        # 根据路径生成更具体的描述
        if "/analysis/" in path:
            if "single" in path:
                return f"{action_verb}单股分析任务"
            elif "batch" in path:
                return f"{action_verb}批量分析任务"
            else:
                return f"{action_verb}分析任务"
        elif "/screening/" in path:
            return f"{action_verb}股票筛选"
        elif "/config/" in path:
            if "llm" in path:
                return f"{action_verb}大模型配置"
            elif "datasource" in path:
                return f"{action_verb}数据源配置"
            else:
                return f"{action_verb}系统配置"
        elif "/database/" in path:
            if "backup" in path:
                return f"{action_verb}数据库备份"
            elif "cleanup" in path:
                return f"{action_verb}数据库清理"
            else:
                return f"{action_verb}数据库操作"
        elif "/auth/" in path:
            if "login" in path:
                return "用户登录"
            elif "logout" in path:
                return "用户登出"
            elif "change-password" in path:
                return "修改密码"
            else:
                return f"{action_verb}认证操作"
        else:
            return f"{action_verb} {path}"
    async def _log_operation(
        self,
        user_info: Dict[str, Any],
        method: str,
        path: str,
        response: Response,
        duration_ms: int,
        ip_address: str,
        user_agent: str,
        request: Request
    ):
        """记录操作日志"""
        try:
            # 判断操作是否成功
            success = 200 <= response.status_code < 400
            # 获取操作类型和描述
            action_type = self._get_action_type(path)
            action = self._get_action_description(method, path, request)
            # 构建详细信息
            details = {
                "method": method,
                "path": path,
                "status_code": response.status_code,
                "query_params": dict(request.query_params) if request.query_params else None,
            }
            # 获取错误信息（如果有）
            error_message = None
            if not success:
                error_message = f"HTTP {response.status_code}"
            # 记录操作日志
            await log_operation(
                user_id=user_info.get("id", ""),
                username=user_info.get("username", "unknown"),
                action_type=action_type,
                action=action,
                details=details,
                success=success,
                error_message=error_message,
                duration_ms=duration_ms,
                ip_address=ip_address,
                user_agent=user_agent,
                session_id=user_info.get("session_id")
            )
        except Exception as e:
            logger.error(f"记录操作日志失败: {e}")
# 便捷函数：手动记录操作日志
async def manual_log_operation(
    request: Request,
    user_info: Dict[str, Any],
    action_type: str,
    action: str,
    details: Optional[Dict[str, Any]] = None,
```

```
    success: bool = True,
    error_message: Optional[str] = None,
    duration_ms: Optional[int] = None
):
    """手动记录操作日志"""
    try:
        ip_address = request.client.host if request.client else "unknown"
        user_agent = request.headers.get("user-agent", "")
        await log_operation(
            user_id=user_info.get("id", ""),
            username=user_info.get("username", "unknown"),
            action_type=action_type,
            action=action,
            details=details,
            success=success,
            error_message=error_message,
            duration_ms=duration_ms,
            ip_address=ip_address,
            user_agent=user_agent,
            session_id=user_info.get("session_id")
        )
    except Exception as e:
        logger.error(f"手动记录操作日志失败: {e}")
```

FILE: app/middleware/rate_limit.py

```
"""
速率限制中间件
防止API滥用，实现用户级和端点级速率限制
"""
from fastapi import Request, Response, HTTPException
from starlette.middleware.base import BaseHTTPMiddleware
import logging
from typing import Callable, Dict, Optional
from core.redis_client import get_redis_service, RedisKeys
logger = logging.getLogger(__name__)
class RateLimitMiddleware(BaseHTTPMiddleware):
    """速率限制中间件"""

    def __init__(self, app, default_rate_limit: int = 100):
        super().__init__(app)
        self.default_rate_limit = default_rate_limit

        # 不同端点的速率限制配置
        self.endpoint_limits = {
            "/api/analysis/single": 10,     # 单股分析：每分钟10次
            "/api/analysis/batch": 5,       # 批量分析：每分钟5次
            "/api/screening/filter": 20,    # 股票筛选：每分钟20次
            "/api/auth/login": 5,           # 登录：每分钟5次
            "/api/auth/register": 3,        # 注册：每分钟3次
        }

    async def dispatch(self, request: Request, call_next: Callable) -> Response:
        # 跳过健康检查和静态资源
        if request.url.path.startswith(("/api/health", "/docs", "/redoc", "/openapi.json")):
            return await call_next(request)

        # 获取用户ID（如果已认证）
        user_id = getattr(request.state, "user_id", None)
        if not user_id:
            # 对于未认证用户，使用IP地址
            user_id = f"ip:{request.client.host}" if request.client else "unknown"

        # 检查速率限制
        try:
            await self.check_rate_limit(user_id, request.url.path)
        except HTTPException:
            raise
        except Exception as exc:
            logger.error(f"速率限制检查失败: {exc}")
            # 如果Redis不可用，允许请求通过

        return await call_next(request)

    async def check_rate_limit(self, user_id: str, endpoint: str):
        """检查速率限制"""
        redis_service = get_redis_service()

        # 获取端点的速率限制
        rate_limit = self.endpoint_limits.get(endpoint, self.default_rate_limit)

        # 构建Redis键
        rate_key = RedisKeys.USER_RATE_LIMIT.format(
            user_id=user_id,
            endpoint=endpoint.replace("/", "_")
        )

        # 获取当前计数
        current_count = await redis_service.increment_with_ttl(rate_key, ttl=60)

        # 检查是否超过限制
        if current_count > rate_limit:
            logger.warning(
                f"速率限制触发 - 用户: {user_id}, "
                f"端点: {endpoint}, "
                f"当前计数: {current_count}, "
                f"限制: {rate_limit}"
            )

            raise HTTPException(
                status_code=429,
                detail={
                    "error": {
                        "code": "RATE_LIMIT_EXCEEDED",
```

```
                "message": f"请求过于频繁，请稍后重试",
                "rate_limit": rate_limit,
                "current_count": current_count,
                "reset_time": 60
            }
        }
    )

    logger.debug(
        f"速率限制检查通过 - 用户: {user_id}, "
        f"端点: {endpoint}, "
        f"当前计数: {current_count}/{rate_limit}"
    )
class QuotaMiddleware(BaseHTTPMiddleware):
    """每日配额中间件"""

    def __init__(self, app, daily_quota: int = 1000):
        super().__init__(app)
        self.daily_quota = daily_quota

        # 需要计入配额的端点
        self.quota_endpoints = {
            "/api/analysis/single",
            "/api/analysis/batch",
            "/api/screening/filter"
        }

    async def dispatch(self, request: Request, call_next: Callable) -> Response:
        # 只对需要配额的端点进行检查
        if request.url.path not in self.quota_endpoints:
            return await call_next(request)

        # 获取用户ID
        user_id = getattr(request.state, "user_id", None)
        if not user_id:
            # 未认证用户不受配额限制
            return await call_next(request)

        # 检查每日配额
        try:
            await self.check_daily_quota(user_id)
        except HTTPException:
            raise
        except Exception as exc:
            logger.error(f"配额检查失败: {exc}")
            # 如果Redis不可用，允许请求通过

        return await call_next(request)

    async def check_daily_quota(self, user_id: str):
        """检查每日配额"""
        import datetime

        redis_service = get_redis_service()

        # 获取今天的日期
        today = datetime.date.today().isoformat()

        # 构建Redis键
        quota_key = RedisKeys.USER_DAILY_QUOTA.format(
            user_id=user_id,
            date=today
        )

        # 获取今日使用量
        current_usage = await redis_service.increment_with_ttl(quota_key, ttl=86400)  # 24小时TTL

        # 检查是否超过配额
        if current_usage > self.daily_quota:
            logger.warning(
                f"每日配额超限 - 用户: {user_id}, "
                f"今日使用: {current_usage}, "
                f"配额: {self.daily_quota}"
            )

            raise HTTPException(
                status_code=429,
                detail={
                    "error": {
                        "code": "DAILY_QUOTA_EXCEEDED",
                        "message": "今日配额已用完，请明天再试",
                        "daily_quota": self.daily_quota,
                        "current_usage": current_usage,
                        "reset_date": today
                    }
                }
            )

        logger.debug(
            f"配额检查通过 - 用户: {user_id}, "
            f"今日使用: {current_usage}/{self.daily_quota}"
        )
```

FILE: app/middleware/request_id.py

```
"""
请求ID/Trace-ID 中间件
- 为每个请求生成唯一 ID（trace_id），写入 request.state 与响应头
- 将 trace_id 写入 logging 的 contextvars，使所有日志自动带出
"""
from fastapi import Request, Response
from starlette.middleware.base import BaseHTTPMiddleware
import uuid
import time
```

```
import logging
from typing import Callable
from app.core.logging_context import trace_id_var
logger = logging.getLogger(__name__)
class RequestIDMiddleware(BaseHTTPMiddleware):
    """请求ID和日志中间件（trace_id）"""
    async def dispatch(self, request: Request, call_next: Callable) -> Response:
        # 生成请求ID/trace_id
        trace_id = str(uuid.uuid4())
        request.state.request_id = trace_id  # 兼容现有字段名
        request.state.trace_id = trace_id
        # 将 trace_id 写入 contextvars
        token = trace_id_var.set(trace_id)
        # 记录请求开始时间
        start_time = time.time()
        # 记录请求信息
        logger.info(
            f"请求开始 - trace_id: {trace_id}, "
            f"方法: {request.method}, 路径: {request.url.path}, "
            f"客户端: {request.client.host if request.client else 'unknown'}"
        )
        try:
            # 处理请求
            response = await call_next(request)
            # 计算处理时间
            process_time = time.time() - start_time
            # 添加响应头
            response.headers["X-Trace-ID"] = trace_id
            response.headers["X-Request-ID"] = trace_id  # 兼容
            response.headers["X-Process-Time"] = f"{process_time:.3f}"
            # 记录请求完成信息
            logger.info(
                f"请求完成 - trace_id: {trace_id}, 状态码: {response.status_code}, 处理时间: {process_time:.3f}s"
            )
            return response
        except Exception as exc:
            # 计算处理时间
            process_time = time.time() - start_time
            # 记录请求异常信息
            logger.error(
                f"请求异常 - trace_id: {trace_id}, 处理时间: {process_time:.3f}s, 异常: {str(exc)}"
            )
            raise
        finally:
            # 清理 contextvar，避免泄露到后续请求
            try:
                trace_id_var.reset(token)
            except Exception:
                pass
```

FILE: app/models/__init__.py

```
"""
数据模型模块
"""
# 导入股票数据模型
from .stock_models import (
    StockBasicInfoExtended,
    MarketQuotesExtended,
    MarketInfo,
    TechnicalIndicators,
    StockBasicInfoResponse,
    MarketQuotesResponse,
    StockListResponse,
    MarketType,
    ExchangeType,
    CurrencyType,
    StockStatus
)
__all__ = [
    "StockBasicInfoExtended",
    "MarketQuotesExtended",
    "MarketInfo",
    "TechnicalIndicators",
    "StockBasicInfoResponse",
    "MarketQuotesResponse",
    "StockListResponse",
    "MarketType",
    "ExchangeType",
    "CurrencyType",
    "StockStatus"
]
```

FILE: app/models/analysis.py

```
"""
分析相关数据模型
"""
from datetime import datetime
from typing import Optional, List, Dict, Any
from pydantic import BaseModel, Field, ConfigDict, field_serializer
from enum import Enum
from bson import ObjectId
from .user import PyObjectId
from app.utils.timezone import now_tz
class AnalysisStatus(str, Enum):
    """分析状态枚举"""
    PENDING = "pending"
    PROCESSING = "processing"
    COMPLETED = "completed"
```

```
    FAILED = "failed"
    CANCELLED = "cancelled"

class BatchStatus(str, Enum):
    """批次状态枚举"""
    PENDING = "pending"
    PROCESSING = "processing"
    COMPLETED = "completed"
    PARTIAL_SUCCESS = "partial_success"
    FAILED = "failed"
    CANCELLED = "cancelled"

class AnalysisParameters(BaseModel):
    """分析参数模型
    研究深度说明：
    - 快速: 1级 - 快速分析 (2-4分钟)
    - 基础: 2级 - 基础分析 (4-6分钟)
    - 标准: 3级 - 标准分析 (6-10分钟，推荐)
    - 深度: 4级 - 深度分析 (10-15分钟)
    - 全面: 5级 - 全面分析 (15-25分钟)
    """
    market_type: str = "A股"
    analysis_date: Optional[datetime] = None
    research_depth: str = "标准"  # 默认使用3级标准分析（推荐）
    selected_analysts: List[str] = Field(default_factory=lambda: ["market", "fundamentals", "news", "social"])
    custom_prompt: Optional[str] = None
    include_sentiment: bool = True
    include_risk: bool = True
    language: str = "zh-CN"
    # 模型配置
    quick_analysis_model: Optional[str] = "qwen-turbo"
    deep_analysis_model: Optional[str] = "qwen-max"

class AnalysisResult(BaseModel):
    """分析结果模型"""
    analysis_id: Optional[str] = None
    summary: Optional[str] = None
    recommendation: Optional[str] = None
    confidence_score: Optional[float] = None
    risk_level: Optional[str] = None
    key_points: List[str] = Field(default_factory=list)
    detailed_analysis: Optional[Dict[str, Any]] = None
    charts: List[str] = Field(default_factory=list)
    tokens_used: int = 0
    execution_time: float = 0.0
    error_message: Optional[str] = None
    model_info: Optional[str] = None  # 🔥 添加模型信息字段

class AnalysisTask(BaseModel):
    """分析任务模型"""
    id: Optional[PyObjectId] = Field(default_factory=PyObjectId, alias="_id")
    task_id: str = Field(..., description="任务唯一标识")
    batch_id: Optional[str] = None
    user_id: PyObjectId
    symbol: str = Field(..., description="6位股票代码")
    stock_code: Optional[str] = Field(None, description="股票代码(已废弃,使用symbol)")
    stock_name: Optional[str] = None
    status: AnalysisStatus = AnalysisStatus.PENDING
    progress: int = Field(default=0, ge=0, le=100, description="任务进度 0-100")
    # 时间戳
    created_at: datetime = Field(default_factory=now_tz)
    started_at: Optional[datetime] = None
    completed_at: Optional[datetime] = None

    # 执行信息
    worker_id: Optional[str] = None
    parameters: AnalysisParameters = Field(default_factory=AnalysisParameters)
    result: Optional[AnalysisResult] = None

    # 重试机制
    retry_count: int = 0
    max_retries: int = 3
    last_error: Optional[str] = None

    model_config = ConfigDict(
        populate_by_name=True,
        arbitrary_types_allowed=True
    )
class AnalysisBatch(BaseModel):
    """分析批次模型"""
    id: Optional[PyObjectId] = Field(default_factory=PyObjectId, alias="_id")
    batch_id: str = Field(..., description="批次唯一标识")
    user_id: PyObjectId
    title: str = Field(..., description="批次标题")
    description: Optional[str] = None
    status: BatchStatus = BatchStatus.PENDING

    # 任务统计
    total_tasks: int = 0
    completed_tasks: int = 0
    failed_tasks: int = 0
    cancelled_tasks: int = 0
    progress: int = Field(default=0, ge=0, le=100, description="整体进度 0-100")

    # 时间戳
    created_at: datetime = Field(default_factory=datetime.utcnow)
    started_at: Optional[datetime] = None
    completed_at: Optional[datetime] = None

    # 配置参数
    parameters: AnalysisParameters = Field(default_factory=AnalysisParameters)

    # 结果摘要
    results_summary: Optional[Dict[str, Any]] = None

    model_config = ConfigDict(
        populate_by_name=True,
        arbitrary_types_allowed=True
```

```
    )
class StockInfo(BaseModel):
    """股票信息模型"""
    symbol: str = Field(..., description="6位股票代码")
    code: Optional[str] = Field(None, description="股票代码(已废弃,使用symbol)")
    name: str = Field(..., description="股票名称")
    market: str = Field(..., description="市场类型")
    industry: Optional[str] = None
    sector: Optional[str] = None
    market_cap: Optional[float] = None
    price: Optional[float] = None
    change_percent: Optional[float] = None

# API请求/响应模型
class SingleAnalysisRequest(BaseModel):
    """单股分析请求"""
    symbol: Optional[str] = Field(None, description="6位股票代码")
    stock_code: Optional[str] = Field(None, description="股票代码(已废弃,使用symbol)")
    parameters: Optional[AnalysisParameters] = None
    def get_symbol(self) -> str:
        """获取股票代码(兼容旧字段)"""
        return self.symbol or self.stock_code or ""
class BatchAnalysisRequest(BaseModel):
    """批量分析请求"""
    title: str = Field(..., description="批次标题")
    description: Optional[str] = None
    symbols: Optional[List[str]] = Field(None, min_items=1, max_items=10, description="股票代码列表（最多10个）")
    stock_codes: Optional[List[str]] = Field(None, min_items=1, max_items=10, description="股票代码列表(已废弃,使用symbols，最多10个)")
    parameters: Optional[AnalysisParameters] = None
    def get_symbols(self) -> List[str]:
        """获取股票代码列表(兼容旧字段)"""
        return self.symbols or self.stock_codes or []
class AnalysisTaskResponse(BaseModel):
    """分析任务响应"""
    task_id: str
    batch_id: Optional[str]
    symbol: str
    stock_code: Optional[str] = None  # 兼容字段
    stock_name: Optional[str]
    status: AnalysisStatus
    progress: int
    created_at: datetime
    started_at: Optional[datetime]
    completed_at: Optional[datetime]
    result: Optional[AnalysisResult]
    @field_serializer('created_at', 'started_at', 'completed_at')
    def serialize_datetime(self, dt: Optional[datetime], _info) -> Optional[str]:
        """序列化 datetime 为 ISO 8601 格式，保留时区信息"""
        if dt:
            return dt.isoformat()
        return None
class AnalysisBatchResponse(BaseModel):
    """分析批次响应"""
    batch_id: str
    title: str
    description: Optional[str]
    status: BatchStatus
    total_tasks: int
    completed_tasks: int
    failed_tasks: int
    progress: int
    created_at: datetime
    started_at: Optional[datetime]
    completed_at: Optional[datetime]
    parameters: AnalysisParameters
    @field_serializer('created_at', 'started_at', 'completed_at')
    def serialize_datetime(self, dt: Optional[datetime], _info) -> Optional[str]:
        """序列化 datetime 为 ISO 8601 格式，保留时区信息"""
        if dt:
            return dt.isoformat()
        return None
class AnalysisHistoryQuery(BaseModel):
    """分析历史查询参数"""
    status: Optional[AnalysisStatus] = None
    start_date: Optional[datetime] = None
    end_date: Optional[datetime] = None
    symbol: Optional[str] = None
    stock_code: Optional[str] = None  # 兼容字段
    batch_id: Optional[str] = None
    page: int = Field(default=1, ge=1)
    page_size: int = Field(default=20, ge=1, le=100)
    def get_symbol(self) -> Optional[str]:
        """获取股票代码(兼容旧字段)"""
        return self.symbol or self.stock_code
```

FILE: app/models/config.py

```
"""
系统配置相关数据模型
"""
from datetime import datetime, timezone
from app.utils.timezone import now_tz
from typing import Optional, Dict, Any, List
from pydantic import BaseModel, Field, ConfigDict, field_serializer
from enum import Enum
from bson import ObjectId
from .user import PyObjectId
class ModelProvider(str, Enum):
    """大模型提供商枚举"""
    OPENAI = "openai"
    ANTHROPIC = "anthropic"
    ZHIPU = "zhipu"
```

```
    QWEN = "qwen"
    BAIDU = "baidu"
    TENCENT = "tencent"
    GEMINI = "gemini"
    GLM = "glm"
    CLAUDE = "claude"
    DEEPSEEK = "deepseek"
    DASHSCOPE = "dashscope"
    GOOGLE = "google"
    SILICONFLOW = "siliconflow"
    OPENROUTER = "openrouter"
    CUSTOM_OPENAI = "custom_openai"
    QIANFAN = "qianfan"
    LOCAL = "local"
    # 🆕 聚合渠道
    AI302 = "302ai"            # 302.AI
    ONEAPI = "oneapi"          # One API
    NEWAPI = "newapi"          # New API
    FASTGPT = "fastgpt"        # FastGPT
    CUSTOM_AGGREGATOR = "custom_aggregator"  # 自定义聚合渠道

class LLMProvider(BaseModel):
    """大模型厂家配置"""
    id: Optional[PyObjectId] = Field(default_factory=PyObjectId, alias="_id")
    name: str = Field(..., description="厂家唯一标识")
    display_name: str = Field(..., description="显示名称")
    description: Optional[str] = Field(None, description="厂家描述")
    website: Optional[str] = Field(None, description="官网地址")
    api_doc_url: Optional[str] = Field(None, description="API文档地址")
    logo_url: Optional[str] = Field(None, description="Logo地址")
    is_active: bool = Field(True, description="是否启用")
    supported_features: List[str] = Field(default_factory=list, description="支持的功能")
    default_base_url: Optional[str] = Field(None, description="默认API地址")
    api_key: Optional[str] = Field(None, description="API密钥")
    api_secret: Optional[str] = Field(None, description="API密钥（某些厂家需要）")
    extra_config: Dict[str, Any] = Field(default_factory=dict, description="额外配置参数")
    # 🆕 聚合渠道支持
    is_aggregator: bool = Field(default=False, description="是否为聚合渠道（如302.AI、OpenRouter）")
    aggregator_type: Optional[str] = Field(None, description="聚合渠道类型（openai_compatible/custom）")
    model_name_format: Optional[str] = Field(None, description="模型名称格式（如：{provider}/{model}）")
    created_at: Optional[datetime] = Field(default_factory=now_tz)
    updated_at: Optional[datetime] = Field(default_factory=now_tz)
    model_config = ConfigDict(populate_by_name=True, arbitrary_types_allowed=True)

class ModelInfo(BaseModel):
    """模型信息"""
    name: str = Field(..., description="模型标识名称")
    display_name: str = Field(..., description="模型显示名称")
    description: Optional[str] = Field(None, description="模型描述")
    context_length: Optional[int] = Field(None, description="上下文长度")
    max_tokens: Optional[int] = Field(None, description="最大输出token数")
    input_price_per_1k: Optional[float] = Field(None, description="输入价格(每1K tokens)")
    output_price_per_1k: Optional[float] = Field(None, description="输出价格(每1K tokens)")
    currency: str = Field(default="CNY", description="货币单位")
    is_deprecated: bool = Field(default=False, description="是否已废弃")
    release_date: Optional[str] = Field(None, description="发布日期")
    capabilities: List[str] = Field(default_factory=list, description="能力标签(如: vision, function_calling)")
    # 🆕 聚合渠道模型映射支持
    original_provider: Optional[str] = Field(None, description="原厂商标识（用于聚合渠道）")
    original_model: Optional[str] = Field(None, description="原厂商模型名（用于能力映射）")

class ModelCatalog(BaseModel):
    """模型目录"""
    id: Optional[PyObjectId] = Field(default_factory=PyObjectId, alias="_id")
    provider: str = Field(..., description="厂家标识")
    provider_name: str = Field(..., description="厂家显示名称")
    models: List[ModelInfo] = Field(default_factory=list, description="模型列表")
    created_at: Optional[datetime] = Field(default_factory=now_tz)
    updated_at: Optional[datetime] = Field(default_factory=now_tz)
    model_config = ConfigDict(populate_by_name=True, arbitrary_types_allowed=True)

class LLMProviderRequest(BaseModel):
    """大模型厂家请求"""
    name: str = Field(..., description="厂家唯一标识")
    display_name: str = Field(..., description="显示名称")
    description: Optional[str] = Field(None, description="厂家描述")
    website: Optional[str] = Field(None, description="官网地址")
    api_doc_url: Optional[str] = Field(None, description="API文档地址")
    logo_url: Optional[str] = Field(None, description="Logo地址")
    is_active: bool = Field(True, description="是否启用")
    supported_features: List[str] = Field(default_factory=list, description="支持的功能")
    default_base_url: Optional[str] = Field(None, description="默认API地址")
    api_key: Optional[str] = Field(None, description="API密钥")
    api_secret: Optional[str] = Field(None, description="API密钥（某些厂家需要）")
    extra_config: Dict[str, Any] = Field(default_factory=dict, description="额外配置参数")
    # 🆕 聚合渠道支持
    is_aggregator: bool = Field(default=False, description="是否为聚合渠道")
    aggregator_type: Optional[str] = Field(None, description="聚合渠道类型")
    model_name_format: Optional[str] = Field(None, description="模型名称格式")

class LLMProviderResponse(BaseModel):
    """大模型厂家响应"""
    id: str
    name: str
    display_name: str
    description: Optional[str] = None
    website: Optional[str] = None
    api_doc_url: Optional[str] = None
    logo_url: Optional[str] = None
    is_active: bool
    supported_features: List[str]
    default_base_url: Optional[str] = None
    api_key: Optional[str] = None
    api_secret: Optional[str] = None
    extra_config: Dict[str, Any] = Field(default_factory=dict)
    # 🆕 聚合渠道支持
    is_aggregator: bool = False
    aggregator_type: Optional[str] = None
```

```
    model_name_format: Optional[str] = None
    created_at: Optional[datetime] = None
    updated_at: Optional[datetime] = None

class DataSourceType(str, Enum):
    """
    数据源类型枚举
    注意：这个枚举与 tradingagents.constants.DataSourceCode 保持同步
    添加新数据源时，请先在 tradingagents/constants/data_sources.py 中注册
    """
    # 缓存数据源
    MONGODB = "mongodb"
    # 中国市场数据源
    TUSHARE = "tushare"
    AKSHARE = "akshare"
    BAOSTOCK = "baostock"
    # 美股数据源
    FINNHUB = "finnhub"
    YAHOO_FINANCE = "yahoo_finance"
    ALPHA_VANTAGE = "alpha_vantage"
    IEX_CLOUD = "iex_cloud"
    # 专业数据源
    WIND = "wind"
    CHOICE = "choice"
    # 其他数据源
    QUANDL = "quandl"
    LOCAL_FILE = "local_file"
    CUSTOM = "custom"

class DatabaseType(str, Enum):
    """数据库类型枚举"""
    MONGODB = "mongodb"
    MYSQL = "mysql"
    POSTGRESQL = "postgresql"
    REDIS = "redis"
    SQLITE = "sqlite"

class LLMConfig(BaseModel):
    """大模型配置"""
    provider: str = Field(default="openai", description="供应商标识（支持动态添加）")
    model_name: str = Field(..., description="模型名称/代码")
    model_display_name: Optional[str] = Field(None, description="模型显示名称")
    api_key: Optional[str] = Field(None, description="API密钥(可选，优先从厂家配置获取)")
    api_base: Optional[str] = Field(None, description="API基础URL")
    max_tokens: int = Field(default=4000, description="最大token数")
    temperature: float = Field(default=0.7, ge=0.0, le=2.0, description="温度参数")
    timeout: int = Field(default=180, description="请求超时时间(秒)")
    retry_times: int = Field(default=3, description="重试次数")
    enabled: bool = Field(default=True, description="是否启用")
    description: Optional[str] = Field(None, description="配置描述")
    # 新增字段 - 来自sidebar.py的配置项
    model_category: Optional[str] = Field(None, description="模型类别(用于OpenRouter等)")
    custom_endpoint: Optional[str] = Field(None, description="自定义端点URL")
    enable_memory: bool = Field(default=False, description="启用记忆功能")
    enable_debug: bool = Field(default=False, description="启用调试模式")
    priority: int = Field(default=0, description="优先级")
    # 定价配置
    input_price_per_1k: Optional[float] = Field(None, description="输入token价格(每1000个token)")
    output_price_per_1k: Optional[float] = Field(None, description="输出token价格(每1000个token)")
    currency: str = Field(default="CNY", description="货币单位(CNY/USD/EUR)")
    # 🆕 模型能力分级系统
    capability_level: int = Field(
        default=2,
        ge=1,
        le=5,
        description="模型能力等级(1-5): 1=基础, 2=标准, 3=高级, 4=专业, 5=旗舰"
    )
    suitable_roles: List[str] = Field(
        default_factory=lambda: ["both"],
        description="适用角色: quick_analysis(快速分析), deep_analysis(深度分析), both(两者都适合)"
    )
    features: List[str] = Field(
        default_factory=list,
        description="模型特性: tool_calling(工具调用), long_context(长上下文), reasoning(推理), vision(视觉), fast_response(快速), cost_effective(经济)"
    )
    recommended_depths: List[str] = Field(
        default_factory=lambda: ["快速", "基础", "标准"],
        description="推荐的分析深度级别"
    )
    performance_metrics: Optional[Dict[str, Any]] = Field(
        default=None,
        description="性能指标: speed(速度1-5), cost(成本1-5), quality(质量1-5)"
    )

class DataSourceConfig(BaseModel):
    """数据源配置"""
    name: str = Field(..., description="数据源名称")
    type: DataSourceType = Field(..., description="数据源类型")
    api_key: Optional[str] = Field(None, description="API密钥")
    api_secret: Optional[str] = Field(None, description="API密钥")
    endpoint: Optional[str] = Field(None, description="API端点")
    timeout: int = Field(default=30, description="请求超时时间(秒)")
    rate_limit: int = Field(default=100, description="每分钟请求限制")
    enabled: bool = Field(default=True, description="是否启用")
    priority: int = Field(default=0, description="优先级，数字越大优先级越高")
    config_params: Dict[str, Any] = Field(default_factory=dict, description="额外配置参数")
    description: Optional[str] = Field(None, description="配置描述")
    # 新增字段：支持市场分类
    market_categories: Optional[List[str]] = Field(default_factory=list, description="所属市场分类列表")
    display_name: Optional[str] = Field(None, description="显示名称")
    provider: Optional[str] = Field(None, description="数据提供商")
    created_at: Optional[datetime] = Field(default_factory=now_tz, description="创建时间")
    updated_at: Optional[datetime] = Field(default_factory=now_tz, description="更新时间")

class DatabaseConfig(BaseModel):
    """数据库配置"""
    name: str = Field(..., description="数据库名称")
    type: DatabaseType = Field(..., description="数据库类型")
```

```
    host: str = Field(..., description="主机地址")
    port: int = Field(..., description="端口号")
    username: Optional[str] = Field(None, description="用户名")
    password: Optional[str] = Field(None, description="密码")
    database: Optional[str] = Field(None, description="数据库名")
    connection_params: Dict[str, Any] = Field(default_factory=dict, description="连接参数")
    pool_size: int = Field(default=10, description="连接池大小")
    max_overflow: int = Field(default=20, description="最大溢出连接数")
    enabled: bool = Field(default=True, description="是否启用")
    description: Optional[str] = Field(None, description="配置描述")

class MarketCategory(BaseModel):
    """市场分类配置"""
    id: str = Field(..., description="分类ID")
    name: str = Field(..., description="分类名称")
    display_name: str = Field(..., description="显示名称")
    description: Optional[str] = Field(None, description="分类描述")
    enabled: bool = Field(default=True, description="是否启用")
    sort_order: int = Field(default=1, description="排序顺序")
    created_at: Optional[datetime] = Field(default_factory=now_tz, description="创建时间")
    updated_at: Optional[datetime] = Field(default_factory=now_tz, description="更新时间")

class DataSourceGrouping(BaseModel):
    """数据源分组关系"""
    data_source_name: str = Field(..., description="数据源名称")
    market_category_id: str = Field(..., description="市场分类ID")
    priority: int = Field(default=0, description="在该分类中的优先级")
    enabled: bool = Field(default=True, description="是否启用")
    created_at: Optional[datetime] = Field(default_factory=now_tz, description="创建时间")
    updated_at: Optional[datetime] = Field(default_factory=now_tz, description="更新时间")

class UsageRecord(BaseModel):
    """使用记录"""
    id: Optional[str] = Field(None, description="记录ID")
    timestamp: str = Field(..., description="时间戳")
    provider: str = Field(..., description="供应商")
    model_name: str = Field(..., description="模型名称")
    input_tokens: int = Field(..., description="输入token数")
    output_tokens: int = Field(..., description="输出token数")
    cost: float = Field(..., description="成本")
    currency: str = Field(default="CNY", description="货币单位")
    session_id: str = Field(..., description="会话ID")
    analysis_type: str = Field(default="stock_analysis", description="分析类型")
    stock_code: Optional[str] = Field(None, description="股票代码")

class UsageStatistics(BaseModel):
    """使用统计"""
    total_requests: int = Field(default=0, description="总请求数")
    total_input_tokens: int = Field(default=0, description="总输入token数")
    total_output_tokens: int = Field(default=0, description="总输出token数")
    total_cost: float = Field(default=0.0, description="总成本（已废弃，使用 cost_by_currency）")
    cost_by_currency: Dict[str, float] = Field(default_factory=dict, description="按货币统计的成本")
    by_provider: Dict[str, Any] = Field(default_factory=dict, description="按供应商统计")
    by_model: Dict[str, Any] = Field(default_factory=dict, description="按模型统计")
    by_date: Dict[str, Any] = Field(default_factory=dict, description="按日期统计")

class SystemConfig(BaseModel):
    """系统配置模型"""
    id: Optional[PyObjectId] = Field(default_factory=PyObjectId, alias="_id")
    config_name: str = Field(..., description="配置名称")
    config_type: str = Field(..., description="配置类型")

    # 大模型配置
    llm_configs: List[LLMConfig] = Field(default_factory=list, description="大模型配置列表")
    default_llm: Optional[str] = Field(None, description="默认大模型")

    # 数据源配置
    data_source_configs: List[DataSourceConfig] = Field(default_factory=list, description="数据源配置列表")
    default_data_source: Optional[str] = Field(None, description="默认数据源")

    # 数据库配置
    database_configs: List[DatabaseConfig] = Field(default_factory=list, description="数据库配置列表")

    # 系统设置
    system_settings: Dict[str, Any] = Field(default_factory=dict, description="系统设置")

    # 元数据
    created_at: datetime = Field(default_factory=now_tz)
    updated_at: datetime = Field(default_factory=now_tz)
    created_by: Optional[PyObjectId] = Field(None, description="创建者")
    updated_by: Optional[PyObjectId] = Field(None, description="更新者")
    version: int = Field(default=1, description="配置版本")
    is_active: bool = Field(default=True, description="是否激活")

    model_config = ConfigDict(populate_by_name=True, arbitrary_types_allowed=True)

# API请求/响应模型
class LLMConfigRequest(BaseModel):
    """大模型配置请求"""
    provider: str = Field(..., description="供应商标识（支持动态添加）")
    model_name: str
    model_display_name: Optional[str] = None # 新增：模型显示名称
    api_key: Optional[str] = None # 可选，优先从厂家配置获取
    api_base: Optional[str] = None
    max_tokens: int = 4000
    temperature: float = 0.7
    timeout: int = 180 # 默认超时时间改为180秒
    retry_times: int = 3
    enabled: bool = True
    description: Optional[str] = None
    # 新增字段以匹配前端
    enable_memory: bool = False
    enable_debug: bool = False
    priority: int = 0
    model_category: Optional[str] = None
    # 定价配置
    input_price_per_1k: Optional[float] = None
    output_price_per_1k: Optional[float] = None
    currency: str = "CNY"
```

```
    # 🆕 模型能力分级系统
    capability_level: int = Field(default=2, ge=1, le=5)
    suitable_roles: List[str] = Field(default_factory=lambda: ["both"])
    features: List[str] = Field(default_factory=list)
    recommended_depths: List[str] = Field(default_factory=lambda: ["快速", "基础", "标准"])
    performance_metrics: Optional[Dict[str, Any]] = None

class DataSourceConfigRequest(BaseModel):
    """数据源配置请求"""
    name: str
    type: DataSourceType
    api_key: Optional[str] = None
    api_secret: Optional[str] = None
    endpoint: Optional[str] = None
    timeout: int = 30
    rate_limit: int = 100
    enabled: bool = True
    priority: int = 0
    config_params: Dict[str, Any] = Field(default_factory=dict)
    description: Optional[str] = None
    # 新增字段
    market_categories: Optional[List[str]] = Field(default_factory=list)
    display_name: Optional[str] = None
    provider: Optional[str] = None

class MarketCategoryRequest(BaseModel):
    """市场分类请求"""
    id: str
    name: str
    display_name: str
    description: Optional[str] = None
    enabled: bool = True
    sort_order: int = 1

class DataSourceGroupingRequest(BaseModel):
    """数据源分组请求"""
    data_source_name: str
    market_category_id: str
    priority: int = 0
    enabled: bool = True

class DataSourceOrderRequest(BaseModel):
    """数据源排序请求"""
    data_sources: List[Dict[str, Any]] = Field(..., description="排序后的数据源列表")

class DatabaseConfigRequest(BaseModel):
    """数据库配置请求"""
    name: str
    type: DatabaseType
    host: str
    port: int
    username: Optional[str] = None
    password: Optional[str] = None
    database: Optional[str] = None
    connection_params: Dict[str, Any] = Field(default_factory=dict)
    pool_size: int = 10
    max_overflow: int = 20
    enabled: bool = True
    description: Optional[str] = None

class SystemConfigResponse(BaseModel):
    """系统配置响应"""
    config_name: str
    config_type: str
    llm_configs: List[LLMConfig]
    default_llm: Optional[str]
    data_source_configs: List[DataSourceConfig]
    default_data_source: Optional[str]
    database_configs: List[DatabaseConfig]
    system_settings: Dict[str, Any]
    created_at: datetime
    updated_at: datetime
    version: int
    is_active: bool
    @field_serializer('created_at', 'updated_at')
    def serialize_datetime(self, dt: Optional[datetime], _info) -> Optional[str]:
        """序列化 datetime 为 ISO 8601 格式，保留时区信息"""
        if dt:
            return dt.isoformat()
        return None

class ConfigTestRequest(BaseModel):
    """配置测试请求"""
    config_type: str = Field(..., description="配置类型: llm/datasource/database")
    config_data: Dict[str, Any] = Field(..., description="配置数据")

class ConfigTestResponse(BaseModel):
    """配置测试响应"""
    success: bool
    message: str
    details: Optional[Dict[str, Any]] = None
    response_time: Optional[float] = None
```

FILE: app/models/notification.py

```
"""
通知数据模型（MongoDB + Pydantic）
"""
from datetime import datetime
from typing import Optional, Literal, List, Dict, Any
from pydantic import BaseModel, Field, field_serializer
from bson import ObjectId
from app.utils.timezone import now_tz
# 简单工具：ObjectId -> str
def to_str_id(v: Any) -> str:
    try:
        if isinstance(v, ObjectId):
            return str(v)
        return str(v)
```

```
    except Exception:
        return ""

NotificationType = Literal['analysis', 'alert', 'system']
NotificationStatus = Literal['unread', 'read']

class NotificationCreate(BaseModel):
    user_id: str
    type: NotificationType
    title: str
    content: Optional[str] = None
    link: Optional[str] = None
    source: Optional[str] = None
    severity: Optional[Literal['info','success','warning','error']] = None
    metadata: Optional[Dict[str, Any]] = None

class NotificationDB(BaseModel):
    id: Optional[str] = Field(default=None)
    user_id: str
    type: NotificationType
    title: str
    content: Optional[str] = None
    link: Optional[str] = None
    source: Optional[str] = None
    severity: Optional[Literal['info','success','warning','error']] = 'info'
    status: NotificationStatus = 'unread'
    created_at: datetime = Field(default_factory=now_tz)
    metadata: Optional[Dict[str, Any]] = None

class NotificationOut(BaseModel):
    id: str
    type: NotificationType
    title: str
    content: Optional[str] = None
    link: Optional[str] = None
    source: Optional[str] = None
    status: NotificationStatus
    created_at: datetime

    @field_serializer('created_at')
    def serialize_datetime(self, dt: Optional[datetime], _info) -> Optional[str]:
        """序列化 datetime 为 ISO 8601 格式，保留时区信息"""
        if dt:
            return dt.isoformat()
        return None

class NotificationList(BaseModel):
    items: List[NotificationOut]
    total: int = 0
    page: int = 1
    page_size: int = 20
```

FILE: app/models/operation_log.py

```
"""
操作日志数据模型
"""
from datetime import datetime
from typing import Dict, Any, Optional, List
from pydantic import BaseModel, Field, field_serializer
from bson import ObjectId

class OperationLogCreate(BaseModel):
    """创建操作日志请求"""
    action_type: str = Field(..., description="操作类型")
    action: str = Field(..., description="操作描述")
    details: Optional[Dict[str, Any]] = Field(None, description="详细信息")
    success: bool = Field(True, description="是否成功")
    error_message: Optional[str] = Field(None, description="错误信息")
    duration_ms: Optional[int] = Field(None, description="操作耗时(毫秒)")
    ip_address: Optional[str] = Field(None, description="IP地址")
    user_agent: Optional[str] = Field(None, description="用户代理")
    session_id: Optional[str] = Field(None, description="会话ID")

class OperationLogResponse(BaseModel):
    """操作日志响应"""
    id: str = Field(..., description="日志ID")
    user_id: str = Field(..., description="用户ID")
    username: str = Field(..., description="用户名")
    action_type: str = Field(..., description="操作类型")
    action: str = Field(..., description="操作描述")
    details: Optional[Dict[str, Any]] = Field(None, description="详细信息")
    success: bool = Field(..., description="是否成功")
    error_message: Optional[str] = Field(None, description="错误信息")
    duration_ms: Optional[int] = Field(None, description="操作耗时(毫秒)")
    ip_address: Optional[str] = Field(None, description="IP地址")
    user_agent: Optional[str] = Field(None, description="用户代理")
    session_id: Optional[str] = Field(None, description="会话ID")
    timestamp: datetime = Field(..., description="操作时间")
    created_at: datetime = Field(..., description="创建时间")

    @field_serializer('timestamp', 'created_at')
    def serialize_datetime(self, dt: datetime, _info) -> Optional[str]:
        """序列化 datetime 为 ISO 8601 格式，保留时区信息"""
        if dt:
            return dt.isoformat()
        return None

class OperationLogQuery(BaseModel):
    """操作日志查询参数"""
    page: int = Field(1, ge=1, description="页码")
    page_size: int = Field(20, ge=1, le=100, description="每页数量")
    start_date: Optional[str] = Field(None, description="开始日期")
    end_date: Optional[str] = Field(None, description="结束日期")
    action_type: Optional[str] = Field(None, description="操作类型")
    success: Optional[bool] = Field(None, description="是否成功")
    keyword: Optional[str] = Field(None, description="关键词搜索")
    user_id: Optional[str] = Field(None, description="用户ID")

class OperationLogListResponse(BaseModel):
    """操作日志列表响应"""
    success: bool = Field(True, description="是否成功")
```

```
    data: Dict[str, Any] = Field(..., description="响应数据")
    message: str = Field("操作成功", description="响应消息")

class OperationLogStats(BaseModel):
    """操作日志统计"""
    total_logs: int = Field(..., description="总日志数")
    success_logs: int = Field(..., description="成功日志数")
    failed_logs: int = Field(..., description="失败日志数")
    success_rate: float = Field(..., description="成功率")
    action_type_distribution: Dict[str, int] = Field(..., description="操作类型分布")
    hourly_distribution: List[Dict[str, Any]] = Field(..., description="小时分布")

class OperationLogStatsResponse(BaseModel):
    """操作日志统计响应"""
    success: bool = Field(True, description="是否成功")
    data: OperationLogStats = Field(..., description="统计数据")
    message: str = Field("获取统计信息成功", description="响应消息")

class ClearLogsRequest(BaseModel):
    """清空日志请求"""
    days: Optional[int] = Field(None, description="保留最近N天的日志，不传则清空所有")
    action_type: Optional[str] = Field(None, description="只清空指定类型的日志")

class ClearLogsResponse(BaseModel):
    """清空日志响应"""
    success: bool = Field(True, description="是否成功")
    data: Dict[str, Any] = Field(..., description="清空结果")
    message: str = Field("清空日志成功", description="响应消息")

# 操作类型常量
class ActionType:
    """操作类型常量"""
    STOCK_ANALYSIS = "stock_analysis"
    CONFIG_MANAGEMENT = "config_management"
    CACHE_OPERATION = "cache_operation"
    DATA_IMPORT = "data_import"
    DATA_EXPORT = "data_export"
    SYSTEM_SETTINGS = "system_settings"
    USER_LOGIN = "user_login"
    USER_LOGOUT = "user_logout"
    USER_MANAGEMENT = "user_management"  # 🔧 添加用户管理操作类型
    DATABASE_OPERATION = "database_operation"
    SCREENING = "screening"
    REPORT_GENERATION = "report_generation"

# 操作类型映射
ACTION_TYPE_NAMES = {
    ActionType.STOCK_ANALYSIS: "股票分析",
    ActionType.CONFIG_MANAGEMENT: "配置管理",
    ActionType.CACHE_OPERATION: "缓存操作",
    ActionType.DATA_IMPORT: "数据导入",
    ActionType.DATA_EXPORT: "数据导出",
    ActionType.SYSTEM_SETTINGS: "系统设置",
    ActionType.USER_LOGIN: "用户登录",
    ActionType.USER_LOGOUT: "用户登出",
    ActionType.USER_MANAGEMENT: "用户管理",  # 🔧 添加用户管理操作类型名称
    ActionType.DATABASE_OPERATION: "数据库操作",
    ActionType.SCREENING: "股票筛选",
    ActionType.REPORT_GENERATION: "报告生成",
}

def convert_objectid_to_str(doc: Dict[str, Any]) -> Dict[str, Any]:
    """将MongoDB文档中的ObjectId转换为字符串"""
    if doc and "_id" in doc:
        doc["id"] = str(doc["_id"])
        del doc["_id"]
    return doc
```

FILE: app/models/screening.py

```
"""
股票筛选相关的数据模型
"""
from pydantic import BaseModel, Field
from typing import Any, Dict, List, Optional, Union
from enum import Enum

class OperatorType(str, Enum):
    """筛选操作符类型"""
    GT = ">"          # 大于
    LT = "<"          # 小于
    GTE = ">="        # 大于等于
    LTE = "<="        # 小于等于
    EQ = "=="         # 等于
    NE = "!="         # 不等于
    BETWEEN = "between"  # 区间
    IN = "in"         # 包含于
    NOT_IN = "not_in"  # 不包含于
    CONTAINS = "contains"  # 字符串包含
    CROSS_UP = "cross_up"   # 技术指标：向上穿越
    CROSS_DOWN = "cross_down"  # 技术指标：向下穿越

class FieldType(str, Enum):
    """字段类型"""
    BASIC = "basic"       # 基础信息字段
    TECHNICAL = "technical"  # 技术指标字段
    FUNDAMENTAL = "fundamental"  # 基本面字段

class ScreeningCondition(BaseModel):
    """单个筛选条件"""
    field: str = Field(..., description="字段名")
    operator: OperatorType = Field(..., description="操作符")
    value: Union[float, int, str, List[Union[float, int, str]]] = Field(..., description="筛选值")
    field_type: Optional[FieldType] = Field(None, description="字段类型")

    class Config:
        use_enum_values = True

class ScreeningRequest(BaseModel):
    """筛选请求"""
    market: str = Field("CN", description="市场：CN/HK/US")
    date: Optional[str] = Field(None, description="交易日YYYY-MM-DD，缺省为最新")
```

```
    adj: str = Field("qfq", description="复权口径：qfq/hfq/none")

    # 筛选条件
    conditions: List[ScreeningCondition] = Field(default_factory=list, description="筛选条件列表")

    # 排序和分页
    order_by: Optional[List[Dict[str, str]]] = Field(None, description="排序条件")
    limit: int = Field(50, ge=1, le=500, description="返回数量限制")
    offset: int = Field(0, ge=0, description="偏移量")

    # 优化选项
    use_database_optimization: bool = Field(True, description="是否使用数据库优化")
class ScreeningResponse(BaseModel):
    """筛选响应"""
    total: int = Field(..., description="总数量")
    items: List[Dict[str, Any]] = Field(..., description="筛选结果")
    took_ms: Optional[int] = Field(None, description="耗时(毫秒)")
    optimization_used: Optional[str] = Field(None, description="使用的优化方式")
    source: Optional[str] = Field(None, description="数据源")
class FieldInfo(BaseModel):
    """字段信息"""
    name: str = Field(..., description="字段名")
    display_name: str = Field(..., description="显示名称")
    field_type: FieldType = Field(..., description="字段类型")
    data_type: str = Field(..., description="数据类型: number/string/date")
    description: str = Field("", description="字段描述")
    unit: Optional[str] = Field(None, description="单位")

    # 数值字段的统计信息
    min_value: Optional[float] = Field(None, description="最小值")
    max_value: Optional[float] = Field(None, description="最大值")
    avg_value: Optional[float] = Field(None, description="平均值")

    # 枚举字段的可选值
    available_values: Optional[List[str]] = Field(None, description="可选值列表")

    # 支持的操作符
    supported_operators: List[OperatorType] = Field(default_factory=list, description="支持的操作符")
class FieldStatistics(BaseModel):
    """字段统计信息"""
    field: str = Field(..., description="字段名")
    count: int = Field(..., description="有效数据数量")
    min_value: Optional[float] = Field(None, description="最小值")
    max_value: Optional[float] = Field(None, description="最大值")
    avg_value: Optional[float] = Field(None, description="平均值")
    median_value: Optional[float] = Field(None, description="中位数")
    std_value: Optional[float] = Field(None, description="标准差")
# 预定义的字段信息
BASIC_FIELDS_INFO = {
    "symbol": FieldInfo(
        name="symbol",
        display_name="股票代码",
        field_type=FieldType.BASIC,
        data_type="string",
        description="6位股票代码",
        supported_operators=[OperatorType.EQ, OperatorType.NE, OperatorType.IN, OperatorType.NOT_IN, OperatorType.CONTAINS]
    ),
    "code": FieldInfo(  # 兼容旧字段
        name="code",
        display_name="股票代码(已废弃)",
        field_type=FieldType.BASIC,
        data_type="string",
        description="6位股票代码(已废弃,使用symbol)",
        supported_operators=[OperatorType.EQ, OperatorType.NE, OperatorType.IN, OperatorType.NOT_IN, OperatorType.CONTAINS]
    ),
    "name": FieldInfo(
        name="name",
        display_name="股票名称",
        field_type=FieldType.BASIC,
        data_type="string",
        description="股票简称",
        supported_operators=[OperatorType.CONTAINS, OperatorType.EQ, OperatorType.NE]
    ),
    "industry": FieldInfo(
        name="industry",
        display_name="所属行业",
        field_type=FieldType.BASIC,
        data_type="string",
        description="申万行业分类",
        supported_operators=[OperatorType.EQ, OperatorType.NE, OperatorType.IN, OperatorType.NOT_IN, OperatorType.CONTAINS]
    ),
    "area": FieldInfo(
        name="area",
        display_name="所属地区",
        field_type=FieldType.BASIC,
        data_type="string",
        description="公司注册地区",
        supported_operators=[OperatorType.EQ, OperatorType.NE, OperatorType.IN, OperatorType.NOT_IN]
    ),
    "market": FieldInfo(
        name="market",
        display_name="所属市场",
        field_type=FieldType.BASIC,
        data_type="string",
        description="交易市场",
        supported_operators=[OperatorType.EQ, OperatorType.NE, OperatorType.IN, OperatorType.NOT_IN]
    ),
    "total_mv": FieldInfo(
        name="total_mv",
        display_name="总市值",
        field_type=FieldType.FUNDAMENTAL,
        data_type="number",
        description="总市值",
        unit="亿元",
        supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
```

```
),
"circ_mv": FieldInfo(
    name="circ_mv",
    display_name="流通市值",
    field_type=FieldType.FUNDAMENTAL,
    data_type="number",
    description="流通市值",
    unit="亿元",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
"pe": FieldInfo(
    name="pe",
    display_name="市盈率",
    field_type=FieldType.FUNDAMENTAL,
    data_type="number",
    description="市盈率(PE)",
    unit="倍",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
"pb": FieldInfo(
    name="pb",
    display_name="市净率",
    field_type=FieldType.FUNDAMENTAL,
    data_type="number",
    description="市净率(PB)",
    unit="倍",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
"pe_ttm": FieldInfo(
    name="pe_ttm",
    display_name="滚动市盈率",
    field_type=FieldType.FUNDAMENTAL,
    data_type="number",
    description="滚动市盈率(PE TTM)",
    unit="倍",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
"pb_mrq": FieldInfo(
    name="pb_mrq",
    display_name="最新市净率",
    field_type=FieldType.FUNDAMENTAL,
    data_type="number",
    description="最新市净率(PB MRQ)",
    unit="倍",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
"roe": FieldInfo(
    name="roe",
    display_name="净资产收益率",
    field_type=FieldType.FUNDAMENTAL,
    data_type="number",
    description="净资产收益率(最近一期，%)",
    unit="%",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
"turnover_rate": FieldInfo(
    name="turnover_rate",
    display_name="换手率",
    field_type=FieldType.TECHNICAL,
    data_type="number",
    description="换手率",
    unit="%",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
"volume_ratio": FieldInfo(
    name="volume_ratio",
    display_name="量比",
    field_type=FieldType.TECHNICAL,
    data_type="number",
    description="量比",
    unit="倍",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
# 价格数据字段（现在在视图中，可以直接从数据库查询）
"close": FieldInfo(
    name="close",
    display_name="收盘价",
    field_type=FieldType.FUNDAMENTAL, # 改为 FUNDAMENTAL，因为现在在视图中可以直接查询
    data_type="number",
    description="最新收盘价",
    unit="元",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
"pct_chg": FieldInfo(
    name="pct_chg",
    display_name="涨跌幅",
    field_type=FieldType.FUNDAMENTAL, # 改为 FUNDAMENTAL，因为现在在视图中可以直接查询
    data_type="number",
    description="涨跌幅",
    unit="%",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
"amount": FieldInfo(
    name="amount",
    display_name="成交额",
    field_type=FieldType.FUNDAMENTAL, # 改为 FUNDAMENTAL，因为现在在视图中可以直接查询
    data_type="number",
    description="成交额",
    unit="元",
    supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
),
"volume": FieldInfo(
    name="volume",
    display_name="成交量",
    field_type=FieldType.FUNDAMENTAL, # 改为 FUNDAMENTAL，因为现在在视图中可以直接查询
    data_type="number",
```

```
        description="成交量",
        unit="手",
        supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
    ),
    # 技术指标字段
    "ma20": FieldInfo(
        name="ma20",
        display_name="20日均线",
        field_type=FieldType.TECHNICAL,
        data_type="number",
        description="20日移动平均线",
        unit="元",
        supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
    ),
    "rsi14": FieldInfo(
        name="rsi14",
        display_name="RSI指标",
        field_type=FieldType.TECHNICAL,
        data_type="number",
        description="14日相对强弱指标",
        unit="",
        supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
    ),
    "kdj_k": FieldInfo(
        name="kdj_k",
        display_name="KDJ-K",
        field_type=FieldType.TECHNICAL,
        data_type="number",
        description="KDJ指标K值",
        unit="",
        supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
    ),
    "kdj_d": FieldInfo(
        name="kdj_d",
        display_name="KDJ-D",
        field_type=FieldType.TECHNICAL,
        data_type="number",
        description="KDJ指标D值",
        unit="",
        supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
    ),
    "kdj_j": FieldInfo(
        name="kdj_j",
        display_name="KDJ-J",
        field_type=FieldType.TECHNICAL,
        data_type="number",
        description="KDJ指标J值",
        unit="",
        supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
    ),
    "dif": FieldInfo(
        name="dif",
        display_name="MACD-DIF",
        field_type=FieldType.TECHNICAL,
        data_type="number",
        description="MACD指标DIF值",
        unit="",
        supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
    ),
    "dea": FieldInfo(
        name="dea",
        display_name="MACD-DEA",
        field_type=FieldType.TECHNICAL,
        data_type="number",
        description="MACD指标DEA值",
        unit="",
        supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
    ),
    "macd_hist": FieldInfo(
        name="macd_hist",
        display_name="MACD柱状图",
        field_type=FieldType.TECHNICAL,
        data_type="number",
        description="MACD柱状图值",
        unit="",
        supported_operators=[OperatorType.GT, OperatorType.LT, OperatorType.GTE, OperatorType.LTE, OperatorType.BETWEEN]
    ),
}
```

FILE: app/models/stock_models.py

```
"""
股票数据模型 - 基于现有集合扩展
采用方案B: 在现有集合基础上扩展字段，保持向后兼容
"""
from datetime import datetime, date
from typing import Optional, Dict, Any, List, Literal
from pydantic import BaseModel, Field
from bson import ObjectId

def to_str_id(v: Any) -> str:
    """ObjectId转字符串工具函数"""
    try:
        if isinstance(v, ObjectId):
            return str(v)
        return str(v)
    except Exception:
        return ""

# 枚举类型定义
MarketType = Literal["CN", "HK", "US"]  # 市场类型
ExchangeType = Literal["SZSE", "SSE", "SEHK", "NYSE", "NASDAQ"]  # 交易所
StockStatus = Literal["L", "D", "P"]  # 上市状态: L-上市 D-退市 P-暂停
CurrencyType = Literal["CNY", "HKD", "USD"]  # 货币类型

class MarketInfo(BaseModel):
```

```
    """市场信息结构 - 新增字段"""
    market: MarketType = Field(..., description="市场标识")
    exchange: ExchangeType = Field(..., description="交易所代码")
    exchange_name: str = Field(..., description="交易所名称")
    currency: CurrencyType = Field(..., description="交易货币")
    timezone: str = Field(..., description="时区")
    trading_hours: Optional[Dict[str, Any]] = Field(None, description="交易时间")

class TechnicalIndicators(BaseModel):
    """技术指标结构 - 分类扩展设计"""
    # 趋势指标
    trend: Optional[Dict[str, float]] = Field(None, description="趋势指标")
    # 震荡指标
    oscillator: Optional[Dict[str, float]] = Field(None, description="震荡指标")
    # 通道指标
    channel: Optional[Dict[str, float]] = Field(None, description="通道指标")
    # 成交量指标
    volume: Optional[Dict[str, float]] = Field(None, description="成交量指标")
    # 波动率指标
    volatility: Optional[Dict[str, float]] = Field(None, description="波动率指标")
    # 自定义指标
    custom: Optional[Dict[str, Any]] = Field(None, description="自定义指标")

class StockBasicInfoExtended(BaseModel):
    """
    股票基础信息扩展模型 - 基于现有 stock_basic_info 集合
    统一使用 symbol 作为主要股票代码字段
    """
    # === 标准化字段 (主要字段) ===
    symbol: str = Field(..., description="6位股票代码", pattern=r"^\d{6}$")
    full_symbol: str = Field(..., description="完整标准化代码(如 000001.SZ)")
    name: str = Field(..., description="股票名称")
    # === 兼容字段 (保持向后兼容) ===
    code: Optional[str] = Field(None, description="6位股票代码(已废弃,使用symbol)")
    # === 基础信息字段 ===
    area: Optional[str] = Field(None, description="所在地区")
    industry: Optional[str] = Field(None, description="行业")
    market: Optional[str] = Field(None, description="交易所名称")
    list_date: Optional[str] = Field(None, description="上市日期")
    sse: Optional[str] = Field(None, description="板块")
    sec: Optional[str] = Field(None, description="所属板块")
    source: Optional[str] = Field(None, description="数据来源")
    updated_at: Optional[datetime] = Field(None, description="更新时间")
    # 市值字段
    total_mv: Optional[float] = Field(None, description="总市值(亿元)")
    circ_mv: Optional[float] = Field(None, description="流通市值(亿元)")
    # 财务指标
    pe: Optional[float] = Field(None, description="市盈率")
    pb: Optional[float] = Field(None, description="市净率")
    pe_ttm: Optional[float] = Field(None, description="滚动市盈率")
    pb_mrq: Optional[float] = Field(None, description="最新市净率")
    roe: Optional[float] = Field(None, description="净资产收益率")
    # 交易指标
    turnover_rate: Optional[float] = Field(None, description="换手率%")
    volume_ratio: Optional[float] = Field(None, description="量比")
    # === 扩展字段 ===
    name_en: Optional[str] = Field(None, description="英文名称")

    # 新增市场信息
    market_info: Optional[MarketInfo] = Field(None, description="市场信息")

    # 新增标准化字段
    board: Optional[str] = Field(None, description="板块标准化")
    industry_code: Optional[str] = Field(None, description="行业代码")
    sector: Optional[str] = Field(None, description="所属板块标准化（GICS行业）")
    delist_date: Optional[str] = Field(None, description="退市日期")
    status: Optional[StockStatus] = Field(None, description="上市状态")
    is_hs: Optional[bool] = Field(None, description="是否沪深港通标的")
    # 新增股本信息
    total_shares: Optional[float] = Field(None, description="总股本")
    float_shares: Optional[float] = Field(None, description="流通股本")
    # 港股特有字段
    lot_size: Optional[int] = Field(None, description="每手股数（港股特有）")
    # 货币字段
    currency: Optional[CurrencyType] = Field(None, description="交易货币")

    # 版本控制
    data_version: Optional[int] = Field(None, description="数据版本")

    class Config:
        # 允许额外字段，保持向后兼容
        extra = "allow"
        # 示例数据
        json_schema_extra = {
            "example": {
                # 标准化字段
                "symbol": "000001",
                "full_symbol": "000001.SZ",
                "name": "平安银行",
                # 基础信息
                "area": "深圳",
                "industry": "银行",
                "market": "深圳证券交易所",
                "sse": "主板",
                "total_mv": 2500.0,
                "pe": 5.2,
                "pb": 0.8,
                # 扩展字段
                "market_info": {
                    "market": "CN",
                    "exchange": "SZSE",
                    "exchange_name": "深圳证券交易所",
                    "currency": "CNY",
                    "timezone": "Asia/Shanghai"
                },
```

```
            "status": "L",
            "data_version": 1
        }
    }
class MarketQuotesExtended(BaseModel):
    """
    实时行情扩展模型 - 基于现有 market_quotes 集合
    统一使用 symbol 作为主要股票代码字段
    """
    # === 标准化字段 (主要字段) ===
    symbol: str = Field(..., description="6位股票代码", pattern=r"^\d{6}$")
    full_symbol: Optional[str] = Field(None, description="完整标准化代码")
    market: Optional[MarketType] = Field(None, description="市场标识")
    # === 兼容字段 (保持向后兼容) ===
    code: Optional[str] = Field(None, description="6位股票代码(已废弃,使用symbol)")
    # === 行情字段 ===
    close: Optional[float] = Field(None, description="收盘价")
    pct_chg: Optional[float] = Field(None, description="涨跌幅%")
    amount: Optional[float] = Field(None, description="成交额")
    open: Optional[float] = Field(None, description="开盘价")
    high: Optional[float] = Field(None, description="最高价")
    low: Optional[float] = Field(None, description="最低价")
    pre_close: Optional[float] = Field(None, description="前收盘价")
    trade_date: Optional[str] = Field(None, description="交易日期")
    updated_at: Optional[datetime] = Field(None, description="更新时间")

    # 新增行情字段
    current_price: Optional[float] = Field(None, description="当前价格(与close相同)")
    change: Optional[float] = Field(None, description="涨跌额")
    volume: Optional[float] = Field(None, description="成交量")
    turnover_rate: Optional[float] = Field(None, description="换手率")
    volume_ratio: Optional[float] = Field(None, description="量比")

    # 五档行情
    bid_prices: Optional[List[float]] = Field(None, description="买1-5价")
    bid_volumes: Optional[List[float]] = Field(None, description="买1-5量")
    ask_prices: Optional[List[float]] = Field(None, description="卖1-5价")
    ask_volumes: Optional[List[float]] = Field(None, description="卖1-5量")

    # 时间戳
    timestamp: Optional[datetime] = Field(None, description="行情时间戳")

    # 数据源和版本
    data_source: Optional[str] = Field(None, description="数据来源")
    data_version: Optional[int] = Field(None, description="数据版本")

    class Config:
        extra = "allow"
        json_schema_extra = {
            "example": {
                # 标准化字段
                "symbol": "000001",
                "full_symbol": "000001.SZ",
                "market": "CN",
                # 行情字段
                "close": 12.65,
                "pct_chg": 1.61,
                "amount": 1580000000,
                "open": 12.50,
                "high": 12.80,
                "low": 12.30,
                "trade_date": "2024-01-15",
                # 扩展字段
                "current_price": 12.65,
                "change": 0.20,
                "volume": 125000000
            }
        }
# 数据库操作相关的响应模型
class StockBasicInfoResponse(BaseModel):
    """股票基础信息API响应模型"""
    success: bool = True
    data: Optional[StockBasicInfoExtended] = None
    message: str = ""
class MarketQuotesResponse(BaseModel):
    """实时行情API响应模型"""
    success: bool = True
    data: Optional[MarketQuotesExtended] = None
    message: str = ""
class StockListResponse(BaseModel):
    """股票列表API响应模型"""
    success: bool = True
    data: Optional[List[StockBasicInfoExtended]] = None
    total: int = 0
    page: int = 1
    page_size: int = 20
    message: str = ""
```

FILE: app/models/user.py

```
"""
用户数据模型
"""
from datetime import datetime, timezone
from app.utils.timezone import now_tz
from typing import Optional, Dict, Any, Annotated, List
from pydantic import BaseModel, Field, BeforeValidator, PlainSerializer, ConfigDict, field_serializer
from pydantic.json_schema import JsonSchemaValue
from pydantic_core import core_schema
from bson import ObjectId
def validate_object_id(v: Any) -> ObjectId:
    """验证ObjectId"""
```

```
    if isinstance(v, ObjectId):
        return v
    if isinstance(v, str):
        if ObjectId.is_valid(v):
            return ObjectId(v)
    raise ValueError("Invalid ObjectId")

def serialize_object_id(v: ObjectId) -> str:
    """序列化ObjectId为字符串"""
    return str(v)

# 创建自定义ObjectId类型
PyObjectId = Annotated[
    ObjectId,
    BeforeValidator(validate_object_id),
    PlainSerializer(serialize_object_id, return_type=str),
]

class UserPreferences(BaseModel):
    """用户偏好设置"""
    # 分析偏好
    default_market: str = "A股"
    default_depth: str = "3"  # 1-5级，3级为标准分析（推荐）
    default_analysts: List[str] = Field(default_factory=lambda: ["市场分析师", "基本面分析师"])
    auto_refresh: bool = True
    refresh_interval: int = 30  # 秒
    # 外观设置
    ui_theme: str = "light"
    sidebar_width: int = 240
    # 语言和地区
    language: str = "zh-CN"
    # 通知设置
    notifications_enabled: bool = True
    email_notifications: bool = False
    desktop_notifications: bool = True
    analysis_complete_notification: bool = True
    system_maintenance_notification: bool = True

class FavoriteStock(BaseModel):
    """自选股信息"""
    stock_code: str = Field(..., description="股票代码")
    stock_name: str = Field(..., description="股票名称")
    market: str = Field(..., description="市场类型")
    added_at: datetime = Field(default_factory=now_tz, description="添加时间")
    tags: List[str] = Field(default_factory=list, description="用户标签")
    notes: str = Field(default="", description="用户备注")
    alert_price_high: Optional[float] = Field(None, description="价格上限提醒")
    alert_price_low: Optional[float] = Field(None, description="价格下限提醒")

class User(BaseModel):
    """用户模型"""
    id: Optional[PyObjectId] = Field(default_factory=PyObjectId, alias="_id")
    username: str = Field(..., min_length=3, max_length=50)
    email: str = Field(..., pattern=r'^[^@]+@[^@]+\.[^@]+$')
    hashed_password: str
    is_active: bool = True
    is_verified: bool = False
    is_admin: bool = False
    created_at: datetime = Field(default_factory=now_tz)
    updated_at: datetime = Field(default_factory=now_tz)
    last_login: Optional[datetime] = None
    preferences: UserPreferences = Field(default_factory=UserPreferences)

    # 配额和限制
    daily_quota: int = 1000
    concurrent_limit: int = 3

    # 统计信息
    total_analyses: int = 0
    successful_analyses: int = 0
    failed_analyses: int = 0
    # 自选股
    favorite_stocks: List[FavoriteStock] = Field(default_factory=list, description="用户自选股列表")

    model_config = ConfigDict(populate_by_name=True, arbitrary_types_allowed=True)

class UserCreate(BaseModel):
    """创建用户请求模型"""
    username: str = Field(..., min_length=3, max_length=50)
    email: str = Field(..., pattern=r'^[^@]+@[^@]+\.[^@]+$')
    password: str = Field(..., min_length=6, max_length=100)

class UserUpdate(BaseModel):
    """更新用户请求模型"""
    email: Optional[str] = Field(None, pattern=r'^[^@]+@[^@]+\.[^@]+$')
    preferences: Optional[UserPreferences] = None
    daily_quota: Optional[int] = None
    concurrent_limit: Optional[int] = None

class UserResponse(BaseModel):
    """用户响应模型"""
    id: str
    username: str
    email: str
    is_active: bool
    is_verified: bool
    created_at: datetime
    last_login: Optional[datetime]
    preferences: UserPreferences
    daily_quota: int
    concurrent_limit: int
    total_analyses: int
    successful_analyses: int
    failed_analyses: int
    @field_serializer('created_at', 'last_login')
    def serialize_datetime(self, dt: Optional[datetime], _info) -> Optional[str]:
        """序列化 datetime 为 ISO 8601 格式，保留时区信息"""
        if dt:
            return dt.isoformat()
        return None
```

```
class UserLogin(BaseModel):
    """用户登录请求模型"""
    username: str
    password: str

class UserSession(BaseModel):
    """用户会话模型"""
    session_id: str
    user_id: str
    created_at: datetime
    expires_at: datetime
    last_activity: datetime
    ip_address: Optional[str] = None
    user_agent: Optional[str] = None

    @field_serializer('created_at', 'expires_at', 'last_activity')
    def serialize_datetime(self, dt: Optional[datetime], _info) -> Optional[str]:
        """序列化 datetime 为 ISO 8601 格式，保留时区信息"""
        if dt:
            return dt.isoformat()
        return None

class TokenResponse(BaseModel):
    """Token响应模型"""
    access_token: str
    token_type: str = "bearer"
    expires_in: int
    refresh_token: Optional[str] = None
    user: UserResponse
```

FILE: app/routers/__init__.py

```
"""
Routers package: expose API routers
"""
```

FILE: app/routers/akshare_init.py

```
"""
AKShare数据初始化API路由
提供Web接口进行AKShare数据初始化和管理
"""
import asyncio
import logging
from datetime import datetime
from typing import Dict, Any, Optional
from fastapi import APIRouter, HTTPException, BackgroundTasks, Depends
from pydantic import BaseModel, Field
from app.core.database import get_mongo_db
from app.worker.akshare_init_service import get_akshare_init_service
from app.worker.akshare_sync_service import get_akshare_sync_service
from app.routers.auth_db import get_current_user
from app.utils.timezone import now_tz

logger = logging.getLogger(__name__)
router = APIRouter(prefix="/api/akshare-init", tags=["AKShare初始化"])
# 全局任务状态存储
_initialization_status = {
    "is_running": False,
    "current_task": None,
    "start_time": None,
    "progress": None,
    "result": None
}
class InitializationRequest(BaseModel):
    """初始化请求模型"""
    historical_days: int = Field(default=365, ge=1, le=3650, description="历史数据天数")
    force: bool = Field(default=False, description="是否强制重新初始化")
    skip_if_exists: bool = Field(default=True, description="如果数据存在是否跳过")
class SyncRequest(BaseModel):
    """同步请求模型"""
    force_update: bool = Field(default=False, description="是否强制更新")
    symbols: Optional[list] = Field(default=None, description="指定股票代码列表")
@router.get("/status")
async def get_database_status():
    """
    获取数据库状态

    Returns:
        数据库状态信息
    """
    try:
        db = get_mongo_db()

        # 检查基础信息
        basic_count = await db.stock_basic_info.count_documents({})
        extended_count = await db.stock_basic_info.count_documents({
            "full_symbol": {"$exists": True},
            "market_info": {"$exists": True}
        })

        # 获取最新更新时间
        latest_basic = await db.stock_basic_info.find_one(
            {}, sort=[("updated_at", -1)]
        )

        # 检查行情数据
        quotes_count = await db.market_quotes.count_documents({})
        latest_quotes = await db.market_quotes.find_one(
            {}, sort=[("updated_at", -1)]
        )

        # 数据质量评估
        data_quality = "excellent"
```

```
        if basic_count == 0:
            data_quality = "empty"
        elif extended_count / basic_count < 0.5:
            data_quality = "poor"
        elif extended_count / basic_count < 0.9:
            data_quality = "good"

        return {
            "success": True,
            "data": {
                "basic_info": {
                    "total_count": basic_count,
                    "extended_count": extended_count,
                    "coverage_rate": round(extended_count / basic_count * 100, 2) if basic_count > 0 else 0,
                    "latest_update": latest_basic.get("updated_at") if latest_basic else None
                },
                "market_quotes": {
                    "total_count": quotes_count,
                    "latest_update": latest_quotes.get("updated_at") if latest_quotes else None
                },
                "data_quality": data_quality,
                "check_time": now_tz()
            },
            "message": "数据库状态检查完成"
        }

    except Exception as e:
        logger.error(f"获取数据库状态失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取数据库状态失败: {str(e)}")
@router.get("/connection-test")
async def test_akshare_connection():
    """
    测试AKShare连接状态

    Returns:
        连接测试结果
    """
    try:
        service = await get_akshare_sync_service()
        connected = await service.provider.test_connection()

        result = {
            "connected": connected,
            "test_time": now_tz()
        }

        if connected:
            # 测试获取股票列表
            try:
                stock_list = await service.provider.get_stock_list()
                result["stock_count"] = len(stock_list) if stock_list else 0
                result["sample_stocks"] = stock_list[:5] if stock_list else []
            except Exception as e:
                result["stock_list_error"] = str(e)

        return {
            "success": True,
            "data": result,
            "message": "AKShare连接测试完成"
        }

    except Exception as e:
        logger.error(f"AKShare连接测试失败: {e}")
        raise HTTPException(status_code=500, detail=f"连接测试失败: {str(e)}")
@router.post("/start-full")
async def start_full_initialization(
    request: InitializationRequest,
    background_tasks: BackgroundTasks,
    current_user: dict = Depends(get_current_user)
):
    """
    启动完整的数据初始化

    Args:
        request: 初始化请求参数
        background_tasks: 后台任务管理器
        current_user: 当前用户信息

    Returns:
        初始化启动结果
    """
    global _initialization_status

    if _initialization_status["is_running"]:
        raise HTTPException(status_code=400, detail="初始化任务正在运行中")

    try:
        # 设置任务状态
        _initialization_status.update({
            "is_running": True,
            "current_task": "full_initialization",
            "start_time": now_tz(),
            "progress": {"current_step": "准备中", "completed_steps": 0, "total_steps": 6},
            "result": None
        })

        # 启动后台任务
        background_tasks.add_task(
            _run_full_initialization_background,
            request.historical_days,
            not request.skip_if_exists
        )

        return {
            "success": True,
```

```
            "data": {
                "task_id": "full_initialization",
                "start_time": _initialization_status["start_time"],
                "parameters": {
                    "historical_days": request.historical_days,
                    "force": not request.skip_if_exists
                }
            },
            "message": "完整初始化任务已启动，请使用 /initialization-status 查看进度"
        }

    except Exception as e:
        _initialization_status["is_running"] = False
        logger.error(f"启动完整初始化失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动初始化失败: {str(e)}")
@router.post("/start-basic-sync")
async def start_basic_sync(
    request: SyncRequest,
    background_tasks: BackgroundTasks,
    current_user: dict = Depends(get_current_user)
):
    """
    启动基础信息同步

    Args:
        request: 同步请求参数
        background_tasks: 后台任务管理器
        current_user: 当前用户信息

    Returns:
        同步启动结果
    """
    global _initialization_status

    if _initialization_status["is_running"]:
        raise HTTPException(status_code=400, detail="同步任务正在运行中")

    try:
        # 设置任务状态
        _initialization_status.update({
            "is_running": True,
            "current_task": "basic_sync",
            "start_time": now_tz(),
            "progress": {"current_step": "同步基础信息", "completed_steps": 0, "total_steps": 1},
            "result": None
        })

        # 启动后台任务
        background_tasks.add_task(
            _run_basic_sync_background,
            request.force_update
        )

        return {
            "success": True,
            "data": {
                "task_id": "basic_sync",
                "start_time": _initialization_status["start_time"],
                "parameters": {
                    "force_update": request.force_update
                }
            },
            "message": "基础信息同步任务已启动"
        }

    except Exception as e:
        _initialization_status["is_running"] = False
        logger.error(f"启动基础信息同步失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动同步失败: {str(e)}")
@router.get("/initialization-status")
async def get_initialization_status():
    """
    获取初始化任务状态

    Returns:
        当前任务状态
    """
    global _initialization_status

    return {
        "success": True,
        "data": {
            "is_running": _initialization_status["is_running"],
            "current_task": _initialization_status["current_task"],
            "start_time": _initialization_status["start_time"],
            "progress": _initialization_status["progress"],
            "result": _initialization_status["result"],
            "duration": (
                (now_tz() - _initialization_status["start_time"]).total_seconds()
                if _initialization_status["start_time"] else 0
            )
        },
        "message": "任务状态获取成功"
    }
@router.post("/stop")
async def stop_initialization(current_user: dict = Depends(get_current_user)):
    """
    停止当前初始化任务

    Args:
        current_user: 当前用户信息

    Returns:
        停止结果
    """
```

```
    global _initialization_status

    if not _initialization_status["is_running"]:
        raise HTTPException(status_code=400, detail="没有正在运行的任务")

    try:
        # 重置任务状态
        _initialization_status.update({
            "is_running": False,
            "current_task": None,
            "start_time": None,
            "progress": None,
            "result": {"stopped": True, "stop_time": datetime.utcnow()}
        })

        return {
            "success": True,
            "data": {
                "stopped": True,
                "stop_time": datetime.utcnow()
            },
            "message": "初始化任务已停止"
        }

    except Exception as e:
        logger.error(f"停止初始化任务失败: {e}")
        raise HTTPException(status_code=500, detail=f"停止任务失败: {str(e)}")
async def _run_full_initialization_background(historical_days: int, force: bool):
    """后台运行完整初始化"""
    global _initialization_status

    try:
        service = await get_akshare_init_service()
        result = await service.run_full_initialization(
            historical_days=historical_days,
            skip_if_exists=not force
        )

        _initialization_status.update({
            "is_running": False,
            "result": result
        })

        logger.info(f"完整初始化后台任务完成: {result}")

    except Exception as e:
        _initialization_status.update({
            "is_running": False,
            "result": {"success": False, "error": str(e)}
        })
        logger.error(f"完整初始化后台任务失败: {e}")
async def _run_basic_sync_background(force_update: bool):
    """后台运行基础信息同步"""
    global _initialization_status

    try:
        service = await get_akshare_sync_service()
        result = await service.sync_stock_basic_info(force_update=force_update)

        _initialization_status.update({
            "is_running": False,
            "result": result
        })

        logger.info(f"基础信息同步后台任务完成: {result}")

    except Exception as e:
        _initialization_status.update({
            "is_running": False,
            "result": {"success": False, "error": str(e)}
        })
        logger.error(f"基础信息同步后台任务失败: {e}")
```

FILE: app/routers/analysis.py

```
"""
股票分析API路由
增强版本，支持优先级、进度跟踪、任务管理等功能
"""
from fastapi import APIRouter, HTTPException, Depends, Query, BackgroundTasks, WebSocket, WebSocketDisconnect
from pydantic import BaseModel, Field
from typing import List, Optional, Dict, Any
from datetime import datetime
import logging
import time
import uuid
import asyncio
from app.routers.auth_db import get_current_user
from app.services.queue_service import get_queue_service, QueueService
from app.services.analysis_service import get_analysis_service
from app.services.simple_analysis_service import get_simple_analysis_service
from app.services.websocket_manager import get_websocket_manager
from app.models.analysis import (
    SingleAnalysisRequest, BatchAnalysisRequest, AnalysisParameters,
    AnalysisTaskResponse, AnalysisBatchResponse, AnalysisHistoryQuery
)
router = APIRouter()
logger = logging.getLogger("webapi")
# 兼容性：保留原有的请求模型
class SingleAnalyzeRequest(BaseModel):
    symbol: str
    parameters: dict = Field(default_factory=dict)
class BatchAnalyzeRequest(BaseModel):
```

```
    symbols: List[str]
    parameters: dict = Field(default_factory=dict)
    title: str = Field(default="批量分析", description="批次标题")
    description: Optional[str] = Field(None, description="批次描述")
# 新版API端点
@router.post("/single", response_model=Dict[str, Any])
async def submit_single_analysis(
    request: SingleAnalysisRequest,
    background_tasks: BackgroundTasks,
    user: dict = Depends(get_current_user)
):
    """提交单股分析任务 - 使用 BackgroundTasks 异步执行"""
    try:
        logger.info(f"🎯 收到单股分析请求")
        logger.info(f"👤 用户信息: {user}")
        logger.info(f"📊 请求数据: {request}")
        # 立即创建任务记录并返回，不等待执行完成
        analysis_service = get_simple_analysis_service()
        result = await analysis_service.create_analysis_task(user["id"], request)
        # 提取变量，避免闭包问题
        task_id = result["task_id"]
        user_id = user["id"]
        # 定义一个包装函数来运行异步任务
        async def run_analysis_task():
            """包装函数：在后台运行分析任务"""
            try:
                logger.info(f"🚀 [BackgroundTask] 开始执行分析任务: {task_id}")
                logger.info(f"📝 [BackgroundTask] task_id={task_id}, user_id={user_id}")
                logger.info(f"📝 [BackgroundTask] request={request}")
                # 重新获取服务实例，确保在正确的上下文中
                logger.info(f"🔧 [BackgroundTask] 正在获取服务实例...")
                service = get_simple_analysis_service()
                logger.info(f"✅ [BackgroundTask] 服务实例获取成功: {id(service)}")
                logger.info(f"🚀 [BackgroundTask] 准备调用 execute_analysis_background...")
                await service.execute_analysis_background(
                    task_id,
                    user_id,
                    request
                )
                logger.info(f"✅ [BackgroundTask] 分析任务完成: {task_id}")
            except Exception as e:
                logger.error(f"❌ [BackgroundTask] 分析任务失败: {task_id}, 错误: {e}", exc_info=True)
        # 使用 BackgroundTasks 执行异步任务
        background_tasks.add_task(run_analysis_task)
        logger.info(f"✅ 分析任务已在后台启动: {result}")
        return {
            "success": True,
            "data": result,
            "message": "分析任务已在后台启动"
        }
    except Exception as e:
        logger.error(f"❌ 提交单股分析任务失败: {e}")
        raise HTTPException(status_code=400, detail=str(e))
# 测试路由 - 验证路由是否被正确注册
@router.get("/test-route")
async def test_route():
    """测试路由是否工作"""
    logger.info("🔥 测试路由被调用了！ ")
    return {"message": "测试路由工作正常", "timestamp": time.time()}
@router.get("/tasks/{task_id}/status", response_model=Dict[str, Any])
async def get_task_status_new(
    task_id: str,
    user: dict = Depends(get_current_user)
):
    """获取分析任务状态（新版异步实现）"""
    try:
        logger.info(f"🔍 [NEW ROUTE] 进入新版状态查询路由: {task_id}")
        logger.info(f"👤 [NEW ROUTE] 用户: {user}")
        analysis_service = get_simple_analysis_service()
        logger.info(f"🔧 [NEW ROUTE] 获取分析服务实例: {id(analysis_service)}")
        result = await analysis_service.get_task_status(task_id)
        logger.info(f"📊 [NEW ROUTE] 查询结果: {result is not None}")
        if result:
            return {
                "success": True,
                "data": result,
                "message": "任务状态获取成功"
            }
        else:
            # 内存中没有找到，尝试从MongoDB中查找
            logger.info(f"📊 [STATUS] 内存中未找到，尝试从MongoDB查找: {task_id}")
            from app.core.database import get_mongo_db
            db = get_mongo_db()
            # 首先从analysis_tasks集合中查找（正在进行的任务）
            task_result = await db.analysis_tasks.find_one({"task_id": task_id})
            if task_result:
                logger.info(f"✅ [STATUS] 从analysis_tasks找到任务: {task_id}")
                # 构造状态响应（正在进行的任务）
                status = task_result.get("status", "pending")
                progress = task_result.get("progress", 0)
                # 计算时间信息
                start_time = task_result.get("started_at") or task_result.get("created_at")
                current_time = datetime.utcnow()
                elapsed_time = 0
                if start_time:
                    elapsed_time = (current_time - start_time).total_seconds()
                status_data = {
                    "task_id": task_id,
                    "status": status,
                    "progress": progress,
                    "message": f"任务{status}中...",
                    "current_step": status,
                    "start_time": start_time,
```

```
            "end_time": task_result.get("completed_at"),
            "elapsed_time": elapsed_time,
            "remaining_time": 0,  # 无法准确估算
            "estimated_total_time": 0,
            "symbol": task_result.get("symbol") or task_result.get("stock_code"),
            "stock_code": task_result.get("symbol") or task_result.get("stock_code"),  # 兼容字段
            "stock_symbol": task_result.get("symbol") or task_result.get("stock_code"),
            "source": "mongodb_tasks"  # 标记数据来源
        }
        return {
            "success": True,
            "data": status_data,
            "message": "任务状态获取成功（从任务记录恢复）"
        }
    # 如果analysis_tasks中没有找到，再从analysis_reports集合中查找（已完成的任务）
    mongo_result = await db.analysis_reports.find_one({"task_id": task_id})
    if mongo_result:
        logger.info(f"✅ [STATUS] 从analysis_reports找到任务: {task_id}")
        # 构造状态响应（模拟已完成的任务）
        # 计算已完成任务的时间信息
        start_time = mongo_result.get("created_at")
        end_time = mongo_result.get("updated_at")
        elapsed_time = 0
        if start_time and end_time:
            elapsed_time = (end_time - start_time).total_seconds()
        status_data = {
            "task_id": task_id,
            "status": "completed",
            "progress": 100,
            "message": "分析完成（从历史记录恢复）",
            "current_step": "completed",
            "start_time": start_time,
            "end_time": end_time,
            "elapsed_time": elapsed_time,
            "remaining_time": 0,
            "estimated_total_time": elapsed_time,  # 已完成任务的总时长就是已用时间
            "stock_code": mongo_result.get("stock_symbol"),
            "stock_symbol": mongo_result.get("stock_symbol"),
            "analysts": mongo_result.get("analysts", []),
            "research_depth": mongo_result.get("research_depth", "快速"),
            "source": "mongodb_reports"  # 标记数据来源
        }
        return {
            "success": True,
            "data": status_data,
            "message": "任务状态获取成功（从历史记录恢复）"
        }
    else:
        logger.warning(f"❌ [STATUS] MongoDB中也未找到: {task_id} trace={task_id}")
        raise HTTPException(status_code=404, detail="任务不存在")
except HTTPException:
    raise
except Exception as e:
    logger.error(f"❌ 获取任务状态失败: {e}")
    raise HTTPException(status_code=500, detail=str(e))
@router.get("/tasks/{task_id}/result", response_model=Dict[str, Any])
async def get_task_result(
    task_id: str,
    user: dict = Depends(get_current_user)
):
    """获取分析任务结果"""
    try:
        logger.info(f"🔍 [RESULT] 获取任务结果: {task_id}")
        logger.info(f"👤 [RESULT] 用户: {user}")
        analysis_service = get_simple_analysis_service()
        task_status = await analysis_service.get_task_status(task_id)
        result_data = None
        if task_status and task_status.get('status') == 'completed':
            # 从内存中获取结果数据
            result_data = task_status.get('result_data')
            logger.info(f"📊 [RESULT] 从内存中获取到结果数据")
            # 🔍 调试：检查内存中的数据结构
            if result_data:
                logger.info(f"📊 [RESULT] 内存数据键: {list(result_data.keys())}")
                logger.info(f"📊 [RESULT] 内存中有decision字段: {bool(result_data.get('decision'))}")
                logger.info(f"📊 [RESULT] 内存中summary长度: {len(result_data.get('summary', ''))}")
                logger.info(f"📊 [RESULT] 内存中recommendation长度: {len(result_data.get('recommendation', ''))}")
                if result_data.get('decision'):
                    decision = result_data['decision']
                    logger.info(f"📊 [RESULT] 内存decision内容: action={decision.get('action')}, target_price={decision.get('target_price')}")
            else:
                logger.warning(f"⚠️ [RESULT] 内存中result_data为空")
        if not result_data:
            # 内存中没有找到，尝试从MongoDB中查找
            logger.info(f"📊 [RESULT] 内存中未找到，尝试从MongoDB查找: {task_id}")
            from app.core.database import get_mongo_db
            db = get_mongo_db()
            # 从analysis_reports集合中查找（优先使用 task_id 匹配）
            mongo_result = await db.analysis_reports.find_one({"task_id": task_id})
            if not mongo_result:
                # 兼容旧数据：旧记录可能没有 task_id，但 analysis_id 存在于 analysis_tasks.result
                tasks_doc_for_id = await db.analysis_tasks.find_one({"task_id": task_id}, {"result.analysis_id": 1})
                analysis_id = tasks_doc_for_id.get("result", {}).get("analysis_id") if tasks_doc_for_id else None
                if analysis_id:
                    logger.info(f"🔑 [RESULT] 按analysis_id兜底查询 analysis_reports: {analysis_id}")
                    mongo_result = await db.analysis_reports.find_one({"analysis_id": analysis_id})
            if mongo_result:
                logger.info(f"✅ [RESULT] 从MongoDB找到结果: {task_id}")
                # 直接使用MongoDB中的数据结构（与web目录保持一致）
                result_data = {
                    "analysis_id": mongo_result.get("analysis_id"),
                    "stock_symbol": mongo_result.get("stock_symbol"),
                    "stock_code": mongo_result.get("stock_symbol"),  # 兼容性
```

```
            "analysis_date": mongo_result.get("analysis_date"),
            "summary": mongo_result.get("summary", ""),
            "recommendation": mongo_result.get("recommendation", ""),
            "confidence_score": mongo_result.get("confidence_score", 0.0),
            "risk_level": mongo_result.get("risk_level", "中等"),
            "key_points": mongo_result.get("key_points", []),
            "execution_time": mongo_result.get("execution_time", 0),
            "tokens_used": mongo_result.get("tokens_used", 0),
            "analysts": mongo_result.get("analysts", []),
            "research_depth": mongo_result.get("research_depth", "快速"),
            "reports": mongo_result.get("reports", {}),
            "created_at": mongo_result.get("created_at"),
            "updated_at": mongo_result.get("updated_at"),
            "status": mongo_result.get("status", "completed"),
            "decision": mongo_result.get("decision", {}),
            "source": "mongodb"  # 标记数据来源
        }
        # 添加调试信息
        logger.info(f"📊 [RESULT] MongoDB数据结构: {list(result_data.keys())}")
        logger.info(f"📊 [RESULT] MongoDB summary长度: {len(result_data['summary'])}")
        logger.info(f"📊 [RESULT] MongoDB recommendation长度: {len(result_data['recommendation'])}")
        logger.info(f"📊 [RESULT] MongoDB decision字段: {bool(result_data.get('decision'))}")
        if result_data.get('decision'):
            decision = result_data['decision']
            logger.info(f"📊 [RESULT] MongoDB decision内容: action={decision.get('action')}, target_price={decision.get('target_price')}, confidence={decision.get('confidence')}")
    else:
        # 兜底：analysis_tasks 集合中的 result 字段
        tasks_doc = await db.analysis_tasks.find_one(
            {"task_id": task_id},
            {"result": 1, "symbol": 1, "stock_code": 1, "created_at": 1, "completed_at": 1}
        )
        if tasks_doc and tasks_doc.get("result"):
            r = tasks_doc["result"] or {}
            logger.info("✅ [RESULT] 从analysis_tasks.result 找到结果")
            # 获取股票代码 (优先使用symbol)
            symbol = (tasks_doc.get("symbol") or tasks_doc.get("stock_code") or
                      r.get("stock_symbol") or r.get("stock_code"))
            result_data = {
                "analysis_id": r.get("analysis_id"),
                "stock_symbol": symbol,
                "stock_code": symbol,  # 兼容字段
                "analysis_date": r.get("analysis_date"),
                "summary": r.get("summary", ""),
                "recommendation": r.get("recommendation", ""),
                "confidence_score": r.get("confidence_score", 0.0),
                "risk_level": r.get("risk_level", "中等"),
                "key_points": r.get("key_points", []),
                "execution_time": r.get("execution_time", 0),
                "tokens_used": r.get("tokens_used", 0),
                "analysts": r.get("analysts", []),
                "research_depth": r.get("research_depth", "快速"),
                "reports": r.get("reports", {}),
                "state": r.get("state", {}),
                "detailed_analysis": r.get("detailed_analysis", {}),
                "created_at": tasks_doc.get("created_at"),
                "updated_at": tasks_doc.get("completed_at"),
                "status": r.get("status", "completed"),
                "decision": r.get("decision", {}),
                "source": "analysis_tasks"  # 数据来源标记
            }
if not result_data:
    logger.warning(f"❌ [RESULT] 所有数据源都未找到结果: {task_id}")
    raise HTTPException(status_code=404, detail="分析结果不存在")
if not result_data:
    raise HTTPException(status_code=404, detail="分析结果不存在")
# 处理reports字段 - 如果没有reports字段，优先尝试从文件系统加载，其次从state中提取
if 'reports' not in result_data or not result_data['reports']:
    import os
    from pathlib import Path
    stock_symbol = result_data.get('stock_symbol') or result_data.get('stock_code')
    # analysis_date 可能是日期或时间戳字符串，这里只取日期部分
    analysis_date_raw = result_data.get('analysis_date')
    analysis_date = str(analysis_date_raw)[:10] if analysis_date_raw else None
    loaded_reports = {}
    try:
        # 1) 尝试从环境变量 TRADINGAGENTS_RESULTS_DIR 指定的位置读取
        base_env = os.getenv('TRADINGAGENTS_RESULTS_DIR')
        project_root = Path.cwd()
        if base_env:
            base_path = Path(base_env)
            if not base_path.is_absolute():
                base_path = project_root / base_env
        else:
            base_path = project_root / 'results'
        candidate_dirs = []
        if stock_symbol and analysis_date:
            candidate_dirs.append(base_path / stock_symbol / analysis_date / 'reports')
        # 2) 兼容其他保存路径
        if stock_symbol and analysis_date:
            candidate_dirs.append(project_root / 'data' / 'analysis_results' / stock_symbol / analysis_date / 'reports')
            candidate_dirs.append(project_root / 'data' / 'analysis_results' / 'detailed' / stock_symbol / analysis_date / 'reports')
        for d in candidate_dirs:
            if d.exists() and d.is_dir():
                for f in d.glob('*.md'):
                    try:
                        content = f.read_text(encoding='utf-8')
                        if content and content.strip():
                            loaded_reports[f.stem] = content.strip()
                    except Exception:
                        pass
        if loaded_reports:
            result_data['reports'] = loaded_reports
            # 若 summary / recommendation 缺失，尝试从同名报告补全
            if not result_data.get('summary') and loaded_reports.get('summary'):
                result_data['summary'] = loaded_reports.get('summary')
```

```
            if not result_data.get('recommendation') and loaded_reports.get('recommendation'):
                result_data['recommendation'] = loaded_reports.get('recommendation')
            logger.info(f"📂 [RESULT] 从文件系统加载到 {len(loaded_reports)} 个报告: {list(loaded_reports.keys())}")
        except Exception as fs_err:
            logger.warning(f"⚠️ [RESULT] 从文件系统加载报告失败: {fs_err}")
    if 'reports' not in result_data or not result_data['reports']:
        logger.info(f"📊 [RESULT] reports字段缺失，尝试从state中提取")
        # 从state中提取报告内容
        reports = {}
        state = result_data.get('state', {})
        if isinstance(state, dict):
            # 定义所有可能的报告字段
            report_fields = [
                'market_report',
                'sentiment_report',
                'news_report',
                'fundamentals_report',
                'investment_plan',
                'trader_investment_plan',
                'final_trade_decision'
            ]
            # 从state中提取报告内容
            for field in report_fields:
                value = state.get(field, "")
                if isinstance(value, str) and len(value.strip()) > 10:
                    reports[field] = value.strip()
            # 处理研究团队辩论状态报告
            investment_debate_state = state.get('investment_debate_state', {})
            if isinstance(investment_debate_state, dict):
                # 提取多头研究员历史
                bull_content = investment_debate_state.get('bull_history', "")
                if isinstance(bull_content, str) and len(bull_content.strip()) > 10:
                    reports['bull_researcher'] = bull_content.strip()
                # 提取空头研究员历史
                bear_content = investment_debate_state.get('bear_history', "")
                if isinstance(bear_content, str) and len(bear_content.strip()) > 10:
                    reports['bear_researcher'] = bear_content.strip()
                # 提取研究经理决策
                judge_decision = investment_debate_state.get('judge_decision', "")
                if isinstance(judge_decision, str) and len(judge_decision.strip()) > 10:
                    reports['research_team_decision'] = judge_decision.strip()
            # 处理风险管理团队辩论状态报告
            risk_debate_state = state.get('risk_debate_state', {})
            if isinstance(risk_debate_state, dict):
                # 提取激进分析师历史
                risky_content = risk_debate_state.get('risky_history', "")
                if isinstance(risky_content, str) and len(risky_content.strip()) > 10:
                    reports['risky_analyst'] = risky_content.strip()
                # 提取保守分析师历史
                safe_content = risk_debate_state.get('safe_history', "")
                if isinstance(safe_content, str) and len(safe_content.strip()) > 10:
                    reports['safe_analyst'] = safe_content.strip()
                # 提取中性分析师历史
                neutral_content = risk_debate_state.get('neutral_history', "")
                if isinstance(neutral_content, str) and len(neutral_content.strip()) > 10:
                    reports['neutral_analyst'] = neutral_content.strip()
                # 提取投资组合经理决策
                risk_decision = risk_debate_state.get('judge_decision', "")
                if isinstance(risk_decision, str) and len(risk_decision.strip()) > 10:
                    reports['risk_management_decision'] = risk_decision.strip()
            logger.info(f"📊 [RESULT] 从state中提取到 {len(reports)} 个报告: {list(reports.keys())}")
            result_data['reports'] = reports
        else:
            logger.warning(f"⚠️ [RESULT] state字段不是字典类型: {type(state)}")
# 确保reports字段中的所有内容都是字符串类型
if 'reports' in result_data and result_data['reports']:
    reports = result_data['reports']
    if isinstance(reports, dict):
        # 确保每个报告内容都是字符串且不为空
        cleaned_reports = {}
        for key, value in reports.items():
            if isinstance(value, str) and value.strip():
                # 确保字符串不为空
                cleaned_reports[key] = value.strip()
            elif value is not None:
                # 如果不是字符串，转换为字符串
                str_value = str(value).strip()
                if str_value:  # 只保存非空字符串
                    cleaned_reports[key] = str_value
            # 如果value为None或空字符串，则跳过该报告
        result_data['reports'] = cleaned_reports
        logger.info(f"📊 [RESULT] 清理reports字段，包含 {len(cleaned_reports)} 个有效报告")
        # 如果清理后没有有效报告，设置为空字典
        if not cleaned_reports:
            logger.warning(f"⚠️ [RESULT] 清理后没有有效报告")
            result_data['reports'] = {}
    else:
        logger.warning(f"⚠️ [RESULT] reports字段不是字典类型: {type(reports)}")
        result_data['reports'] = {}
# 补全关键字段：recommendation/summary/key_points
try:
    reports = result_data.get('reports', {}) or {}
    decision = result_data.get('decision', {}) or {}
    # recommendation 优先使用决策摘要或报告中的决策
    if not result_data.get('recommendation'):
        rec_candidates = []
        if isinstance(decision, dict) and decision.get('action'):
            parts = [
                f"操作: {decision.get('action')}",
                f"目标价: {decision.get('target_price')}" if decision.get('target_price') else None,
                f"置信度: {decision.get('confidence')}" if decision.get('confidence') is not None else None
            ]
            rec_candidates.append("；".join([p for p in parts if p]))
```

```
        # 从报告中兜底
        for k in ['final_trade_decision', 'investment_plan']:
            v = reports.get(k)
            if isinstance(v, str) and len(v.strip()) > 10:
                rec_candidates.append(v.strip())
        if rec_candidates:
            # 取最有信息量的一条（最长）
            result_data['recommendation'] = max(rec_candidates, key=len)[:2000]
    # summary 从若干报告拼接生成
    if not result_data.get('summary'):
        sum_candidates = []
        for k in ['market_report', 'fundamentals_report', 'sentiment_report', 'news_report']:
            v = reports.get(k)
            if isinstance(v, str) and len(v.strip()) > 50:
                sum_candidates.append(v.strip())
        if sum_candidates:
            result_data['summary'] = ("\n\n".join(sum_candidates))[:3000]
    # key_points 兜底
    if not result_data.get('key_points'):
        kp = []
        if isinstance(decision, dict):
            if decision.get('action'):
                kp.append(f"操作建议: {decision.get('action')}")
            if decision.get('target_price'):
                kp.append(f"目标价: {decision.get('target_price')}")
            if decision.get('confidence') is not None:
                kp.append(f"置信度: {decision.get('confidence')}")
        # 从reports中截取前几句作为要点
        for k in ['investment_plan', 'final_trade_decision']:
            v = reports.get(k)
            if isinstance(v, str) and len(v.strip()) > 10:
                kp.append(v.strip()[:120])
        if kp:
            result_data['key_points'] = kp[:5]
except Exception as fill_err:
    logger.warning(f"⚠️ [RESULT] 补全关键字段时出错: {fill_err}")

# 进一步兜底：从 detailed_analysis 推断并补全
try:
    if not result_data.get('summary') or not result_data.get('recommendation') or not result_data.get('reports'):
        da = result_data.get('detailed_analysis')
        # 若reports仍为空，放入一份原始详细分析，便于前端"查看报告详情"
        if (not result_data.get('reports')) and isinstance(da, str) and len(da.strip()) > 20:
            result_data['reports'] = {'detailed_analysis': da.strip()}
        elif (not result_data.get('reports')) and isinstance(da, dict) and da:
            # 将字典的长文本项放入reports
            extracted = {}
            for k, v in da.items():
                if isinstance(v, str) and len(v.strip()) > 20:
                    extracted[k] = v.strip()
            if extracted:
                result_data['reports'] = extracted
        # 补 summary
        if not result_data.get('summary'):
            if isinstance(da, str) and da.strip():
                result_data['summary'] = da.strip()[:3000]
            elif isinstance(da, dict) and da:
                # 取最长的文本作为摘要
                texts = [v.strip() for v in da.values() if isinstance(v, str) and v.strip()]
                if texts:
                    result_data['summary'] = max(texts, key=len)[:3000]
        # 补 recommendation
        if not result_data.get('recommendation'):
            rec = None
            if isinstance(da, str):
                # 简单基于关键字提取包含"建议"的段落
                import re
                m = re.search(r'(投资建议|建议|结论)[：:]?\s*(.+)', da)
                if m:
                    rec = m.group(0)
            elif isinstance(da, dict):
                for key in ['final_trade_decision', 'investment_plan', '结论', '建议']:
                    v = da.get(key)
                    if isinstance(v, str) and len(v.strip()) > 10:
                        rec = v.strip()
                        break
            if rec:
                result_data['recommendation'] = rec[:2000]
except Exception as da_err:
    logger.warning(f"⚠️ [RESULT] 从detailed_analysis补全失败: {da_err}")
# 严格的数据格式化和验证
def safe_string(value, default=""):
    """安全地转换为字符串"""
    if value is None:
        return default
    if isinstance(value, str):
        return value
    return str(value)
def safe_number(value, default=0):
    """安全地转换为数字"""
    if value is None:
        return default
    if isinstance(value, (int, float)):
        return value
    try:
        return float(value)
    except (ValueError, TypeError):
        return default
def safe_list(value, default=None):
    """安全地转换为列表"""
    if default is None:
        default = []
    if value is None:
        return default
    if isinstance(value, list):
```

```
            return value
        return default
    def safe_dict(value, default=None):
        """安全地转换为字典"""
        if default is None:
            default = {}
        if value is None:
            return default
        if isinstance(value, dict):
            return value
        return default
    # 🔍 调试：检查最终构建前的result_data
    logger.info(f"🔍 [FINAL] 构建最终结果前，result_data键: {list(result_data.keys())}")
    logger.info(f"🔍 [FINAL] result_data中有decision: {bool(result_data.get('decision'))}")
    if result_data.get('decision'):
        logger.info(f"🔍 [FINAL] decision内容: {result_data['decision']}")
    # 构建严格验证的结果数据
    final_result_data = {
        "analysis_id": safe_string(result_data.get("analysis_id"), "unknown"),
        "stock_symbol": safe_string(result_data.get("stock_symbol"), "UNKNOWN"),
        "stock_code": safe_string(result_data.get("stock_code"), "UNKNOWN"),
        "analysis_date": safe_string(result_data.get("analysis_date"), "2025-08-20"),
        "summary": safe_string(result_data.get("summary"), "分析摘要暂无"),
        "recommendation": safe_string(result_data.get("recommendation"), "投资建议暂无"),
        "confidence_score": safe_number(result_data.get("confidence_score"), 0.0),
        "risk_level": safe_string(result_data.get("risk_level"), "中等"),
        "key_points": safe_list(result_data.get("key_points")),
        "execution_time": safe_number(result_data.get("execution_time"), 0),
        "tokens_used": safe_number(result_data.get("tokens_used"), 0),
        "analysts": safe_list(result_data.get("analysts")),
        "research_depth": safe_string(result_data.get("research_depth"), "快速"),
        "detailed_analysis": safe_dict(result_data.get("detailed_analysis")),
        "state": safe_dict(result_data.get("state")),
        # 🔥 关键修复：添加decision字段！
        "decision": safe_dict(result_data.get("decision"))
    }
    # 特别处理reports字段 - 确保每个报告都是有效字符串
    reports_data = safe_dict(result_data.get("reports"))
    validated_reports = {}
    for report_key, report_content in reports_data.items():
        # 确保报告键是字符串
        safe_key = safe_string(report_key, "unknown_report")
        # 确保报告内容是非空字符串
        if report_content is None:
            validated_content = "报告内容暂无"
        elif isinstance(report_content, str):
            validated_content = report_content.strip() if report_content.strip() else "报告内容为空"
        else:
            validated_content = str(report_content).strip() if str(report_content).strip() else "报告内容格式错误"
        validated_reports[safe_key] = validated_content
    final_result_data["reports"] = validated_reports
    logger.info(f"✅ [RESULT] 成功获取任务结果: {task_id}")
    logger.info(f"📊 [RESULT] 最终返回 {len(final_result_data.get('reports', {}))} 个报告")
    # 🔍 调试：检查最终返回的数据
    logger.info(f"🔍 [FINAL] 最终返回数据键: {list(final_result_data.keys())}")
    logger.info(f"🔍 [FINAL] 最终返回中有decision: {bool(final_result_data.get('decision'))}")
    if final_result_data.get('decision'):
        logger.info(f"🔍 [FINAL] 最终decision内容: {final_result_data['decision']}")
    return {
        "success": True,
        "data": final_result_data,
        "message": "分析结果获取成功"
    }
except HTTPException:
    raise
except Exception as e:
    logger.error(f"❌ [RESULT] 获取任务结果失败: {e}")
    raise HTTPException(status_code=400, detail=str(e))
@router.get("/tasks/all", response_model=Dict[str, Any])
async def list_all_tasks(
    user: dict = Depends(get_current_user),
    status: Optional[str] = Query(None, description="任务状态过滤"),
    limit: int = Query(20, ge=1, le=100, description="返回数量限制"),
    offset: int = Query(0, ge=0, description="偏移量")
):
    """获取所有任务列表（不限用户）"""
    try:
        logger.info(f"📋 查询所有任务列表")
        tasks = await get_simple_analysis_service().list_all_tasks(
            status=status,
            limit=limit,
            offset=offset
        )
        return {
            "success": True,
            "data": {
                "tasks": tasks,
                "total": len(tasks),
                "limit": limit,
                "offset": offset
            },
            "message": "任务列表获取成功"
        }
    except Exception as e:
        logger.error(f"❌ 获取任务列表失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
@router.get("/tasks", response_model=Dict[str, Any])
async def list_user_tasks(
    user: dict = Depends(get_current_user),
    status: Optional[str] = Query(None, description="任务状态过滤"),
    limit: int = Query(20, ge=1, le=100, description="返回数量限制"),
    offset: int = Query(0, ge=0, description="偏移量")
):
```

```
    """获取用户的任务列表"""
    try:
        logger.info(f"📋 查询用户任务列表: {user['id']}")
        tasks = await get_simple_analysis_service().list_user_tasks(
            user_id=user["id"],
            status=status,
            limit=limit,
            offset=offset
        )
        return {
            "success": True,
            "data": {
                "tasks": tasks,
                "total": len(tasks),
                "limit": limit,
                "offset": offset
            },
            "message": "任务列表获取成功"
        }
    except Exception as e:
        logger.error(f"❌ 获取任务列表失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
@router.post("/batch", response_model=Dict[str, Any])
async def submit_batch_analysis(
    request: BatchAnalysisRequest,
    user: dict = Depends(get_current_user)
):
    """提交批量分析任务（真正的并发执行）
    ⚠️ 注意：不使用 BackgroundTasks，因为它是串行执行的！
    改用 asyncio.create_task 实现真正的并发执行。
    """
    try:
        logger.info(f"🎯 [批量分析] 收到批量分析请求: title={request.title}")
        simple_service = get_simple_analysis_service()
        batch_id = str(uuid.uuid4())
        task_ids: List[str] = []
        mapping: List[Dict[str, str]] = []
        # 获取股票代码列表 (兼容旧字段)
        stock_symbols = request.get_symbols()
        logger.info(f"📊 [批量分析] 股票代码列表: {stock_symbols}")
        # 验证股票代码列表
        if not stock_symbols:
            raise ValueError("股票代码列表不能为空")
        # 🔧 限制批量分析的股票数量（最多10个）
        MAX_BATCH_SIZE = 10
        if len(stock_symbols) > MAX_BATCH_SIZE:
            raise ValueError(f"批量分析最多支持 {MAX_BATCH_SIZE} 个股票，当前提交了 {len(stock_symbols)} 个")
        # 为每只股票创建单股分析任务
        for i, symbol in enumerate(stock_symbols):
            logger.info(f"📝 [批量分析] 正在创建第 {i+1}/{len(stock_symbols)} 个任务: {symbol}")
            single_req = SingleAnalysisRequest(
                symbol=symbol,
                stock_code=symbol,  # 兼容字段
                parameters=request.parameters
            )
            try:
                create_res = await simple_service.create_analysis_task(user["id"], single_req)
                task_id = create_res.get("task_id")
                if not task_id:
                    raise RuntimeError(f"创建任务失败：未返回task_id (symbol={symbol})")
                task_ids.append(task_id)
                mapping.append({"symbol": symbol, "stock_code": symbol, "task_id": task_id})
                logger.info(f"☑️ [批量分析] 已创建任务: {task_id} - {symbol}")
            except Exception as create_error:
                logger.error(f"❌ [批量分析] 创建任务失败: {symbol}, 错误: {create_error}", exc_info=True)
                raise
        # 🔧 使用 asyncio.create_task 实现真正的并发执行
        # 不使用 BackgroundTasks，因为它是串行执行的
        async def run_concurrent_analysis():
            """并发执行所有分析任务"""
            tasks = []
            for i, symbol in enumerate(stock_symbols):
                task_id = task_ids[i]
                single_req = SingleAnalysisRequest(
                    symbol=symbol,
                    stock_code=symbol,
                    parameters=request.parameters
                )
                # 创建异步任务
                async def run_single_analysis(tid: str, req: SingleAnalysisRequest, uid: str):
                    try:
                        logger.info(f"🚀 [并发任务] 开始执行: {tid} - {req.stock_code}")
                        await simple_service.execute_analysis_background(tid, uid, req)
                        logger.info(f"☑️ [并发任务] 执行完成: {tid}")
                    except Exception as e:
                        logger.error(f"❌ [并发任务] 执行失败: {tid}, 错误: {e}", exc_info=True)
                # 添加到任务列表
                task = asyncio.create_task(run_single_analysis(task_id, single_req, user["id"]))
                tasks.append(task)
                logger.info(f"☑️ [批量分析] 已创建并发任务: {task_id} - {symbol}")
            # 等待所有任务完成（不阻塞响应）
            await asyncio.gather(*tasks, return_exceptions=True)
            logger.info(f"🎉 [批量分析] 所有任务执行完成: batch_id={batch_id}")
        # 在后台启动并发任务（不等待完成）
        asyncio.create_task(run_concurrent_analysis())
        logger.info(f"🚀 [批量分析] 已启动 {len(task_ids)} 个并发任务")
        return {
            "success": True,
            "data": {
                "batch_id": batch_id,
                "total_tasks": len(task_ids),
                "task_ids": task_ids,
                "mapping": mapping,
```

```
            "status": "submitted"
        },
        "message": f"批量分析任务已提交，共{len(task_ids)}个股票，正在并发执行"
    }
    except Exception as e:
        logger.error(f"❌ [批量分析] 提交失败: {e}", exc_info=True)
        raise HTTPException(status_code=400, detail=str(e))
# 兼容性：保留原有端点
@router.post("/analyze")
async def analyze_single(
    req: SingleAnalyzeRequest,
    user: dict = Depends(get_current_user),
    svc: QueueService = Depends(get_queue_service)
):
    """单股分析（兼容性端点）"""
    try:
        task_id = await svc.enqueue_task(
            user_id=user["id"],
            symbol=req.symbol,
            params=req.parameters
        )
        return {"task_id": task_id, "status": "queued"}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
@router.post("/analyze/batch")
async def analyze_batch(
    req: BatchAnalyzeRequest,
    user: dict = Depends(get_current_user),
    svc: QueueService = Depends(get_queue_service)
):
    """批量分析（兼容性端点）"""
    try:
        batch_id, submitted = await svc.create_batch(
            user_id=user["id"],
            symbols=req.symbols,
            params=req.parameters
        )
        return {"batch_id": batch_id, "submitted": submitted}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
@router.get("/batches/{batch_id}")
async def get_batch(batch_id: str, user: dict = Depends(get_current_user), svc: QueueService = Depends(get_queue_service)):
    b = await svc.get_batch(batch_id)
    if not b or b.get("user") != user["id"]:
        raise HTTPException(status_code=404, detail="batch not found")
    return b
# 任务和批次查询端点
# 注意：这个路由被移到了 /tasks/{task_id}/status 之后，避免路由冲突
# @router.get("/tasks/{task_id}")
# async def get_task(
#     task_id: str,
#     user: dict = Depends(get_current_user),
#     svc: QueueService = Depends(get_queue_service)
# ):
#     """获取任务详情"""
#     t = await svc.get_task(task_id)
#     if not t or t.get("user") != user["id"]:
#         raise HTTPException(status_code=404, detail="任务不存在")
#     return t
# 原有的路由已被新的异步实现替代
# @router.get("/tasks/{task_id}/status")
# async def get_task_status_old(
#     task_id: str,
#     user: dict = Depends(get_current_user)
# ):
#     """获取任务状态和进度（旧版实现）"""
#     try:
#         status = await get_analysis_service().get_task_status(task_id)
#         if not status:
#             raise HTTPException(status_code=404, detail="任务不存在")
#         return {
#             "success": True,
#             "data": status
#         }
#     except Exception as e:
#         raise HTTPException(status_code=400, detail=str(e))
@router.post("/tasks/{task_id}/cancel")
async def cancel_task(
    task_id: str,
    user: dict = Depends(get_current_user),
    svc: QueueService = Depends(get_queue_service)
):
    """取消任务"""
    try:
        # 验证任务所有权
        task = await svc.get_task(task_id)
        if not task or task.get("user") != user["id"]:
            raise HTTPException(status_code=404, detail="任务不存在")
        success = await svc.cancel_task(task_id)
        if success:
            return {"success": True, "message": "任务已取消"}
        else:
            raise HTTPException(status_code=400, detail="取消任务失败")
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
@router.get("/user/queue-status")
async def get_user_queue_status(
    user: dict = Depends(get_current_user),
    svc: QueueService = Depends(get_queue_service)
):
    """获取用户队列状态"""
    try:
        status = await svc.get_user_queue_status(user["id"])
        return {
            "success": True,
```

```
            "data": status
        }
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
@router.get("/user/history")
async def get_user_analysis_history(
    user: dict = Depends(get_current_user),
    status: Optional[str] = Query(None, description="任务状态过滤"),
    start_date: Optional[str] = Query(None, description="开始日期，YYYY-MM-DD"),
    end_date: Optional[str] = Query(None, description="结束日期，YYYY-MM-DD"),
    symbol: Optional[str] = Query(None, description="股票代码"),
    stock_code: Optional[str] = Query(None, description="股票代码(已废弃,使用symbol)"),
    market_type: Optional[str] = Query(None, description="市场类型"),
    page: int = Query(1, ge=1, description="页码"),
    page_size: int = Query(20, ge=1, le=100, description="每页大小")
):
    """获取用户分析历史（支持基础筛选与分页）"""
    try:
        # 先获取用户任务列表（内存优先，MongoDB兜底）
        raw_tasks = await get_simple_analysis_service().list_user_tasks(
            user_id=user["id"],
            status=status,
            limit=page_size,
            offset=(page - 1) * page_size
        )
        # 进行基础筛选
        from datetime import datetime
        def in_date_range(t: Optional[str]) -> bool:
            if not t:
                return True
            try:
                dt = datetime.fromisoformat(t.replace('Z', '+00:00')) if 'Z' in t else datetime.fromisoformat(t)
            except Exception:
                return True
            ok = True
            if start_date:
                try:
                    ok = ok and (dt.date() >= datetime.fromisoformat(start_date).date())
                except Exception:
                    pass
            if end_date:
                try:
                    ok = ok and (dt.date() <= datetime.fromisoformat(end_date).date())
                except Exception:
                    pass
            return ok
        # 获取查询的股票代码 (兼容旧字段)
        query_symbol = symbol or stock_code
        filtered = []
        for x in raw_tasks:
            if query_symbol:
                task_symbol = x.get("symbol") or x.get("stock_code") or x.get("stock_symbol")
                if task_symbol not in [query_symbol]:
                    continue
            # 市场类型暂时从参数内判断（如有）
            if market_type:
                params = x.get("parameters") or {}
                if params.get("market_type") != market_type:
                    continue
            # 时间范围（使用 start_time 或 created_at）
            t = x.get("start_time") or x.get("created_at")
            if not in_date_range(t):
                continue
            filtered.append(x)
        return {
            "success": True,
            "data": {
                "tasks": filtered,
                "total": len(filtered),
                "page": page,
                "page_size": page_size
            },
            "message": "历史查询成功"
        }
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
# WebSocket 端点
@router.websocket("/ws/task/{task_id}")
async def websocket_task_progress(websocket: WebSocket, task_id: str):
    """WebSocket 端点：实时获取任务进度"""
    import json
    websocket_manager = get_websocket_manager()
    try:
        await websocket_manager.connect(websocket, task_id)
        # 发送连接确认消息
        await websocket.send_text(json.dumps({
            "type": "connection_established",
            "task_id": task_id,
            "message": "WebSocket 连接已建立"
        }))
        # 保持连接活跃
        while True:
            try:
                # 接收客户端的心跳消息
                data = await websocket.receive_text()
                # 可以处理客户端发送的消息
                logger.debug(f"📨 收到 WebSocket 消息: {data}")
            except WebSocketDisconnect:
                break
            except Exception as e:
                logger.warning(f"⚠️ WebSocket 消息处理错误: {e}")
                break
    except WebSocketDisconnect:
        logger.info(f"🔌 WebSocket 客户端断开连接: {task_id}")
```

```
    except Exception as e:
        logger.error(f"❌ WebSocket 连接错误: {e}")
    finally:
        await websocket_manager.disconnect(websocket, task_id)
# 任务详情查询路由（放在最后避免与 /tasks/{task_id}/status 冲突）
@router.get("/tasks/{task_id}/details")
async def get_task_details(
    task_id: str,
    user: dict = Depends(get_current_user),
    svc: QueueService = Depends(get_queue_service)
):
    """获取任务详情（使用不同的路径避免冲突）"""
    t = await svc.get_task(task_id)
    if not t or t.get("user") != user["id"]:
        raise HTTPException(status_code=404, detail="任务不存在")
    return t

# ==================== 僵尸任务管理 ====================
@router.get("/admin/zombie-tasks")
async def get_zombie_tasks(
    max_running_hours: int = Query(default=2, ge=1, le=72, description="最大运行时长（小时）"),
    user: dict = Depends(get_current_user)
):
    """获取僵尸任务列表（仅管理员）
    僵尸任务：长时间处于 processing/running/pending 状态的任务
    """
    # 检查管理员权限
    if user.get("username") != "admin":
        raise HTTPException(status_code=403, detail="仅管理员可访问")
    try:
        svc = get_simple_analysis_service()
        zombie_tasks = await svc.get_zombie_tasks(max_running_hours)
        return {
            "success": True,
            "data": zombie_tasks,
            "total": len(zombie_tasks),
            "max_running_hours": max_running_hours
        }
    except Exception as e:
        logger.error(f"❌ 获取僵尸任务失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取僵尸任务失败: {str(e)}")
@router.post("/admin/cleanup-zombie-tasks")
async def cleanup_zombie_tasks(
    max_running_hours: int = Query(default=2, ge=1, le=72, description="最大运行时长（小时）"),
    user: dict = Depends(get_current_user)
):
    """清理僵尸任务（仅管理员）
    将长时间处于 processing/running/pending 状态的任务标记为失败
    """
    # 检查管理员权限
    if user.get("username") != "admin":
        raise HTTPException(status_code=403, detail="仅管理员可访问")
    try:
        svc = get_simple_analysis_service()
        result = await svc.cleanup_zombie_tasks(max_running_hours)
        return {
            "success": True,
            "data": result,
            "message": f"已清理 {result.get('total_cleaned', 0)} 个僵尸任务"
        }
    except Exception as e:
        logger.error(f"❌ 清理僵尸任务失败: {e}")
        raise HTTPException(status_code=500, detail=f"清理僵尸任务失败: {str(e)}")
@router.post("/tasks/{task_id}/mark-failed")
async def mark_task_as_failed(
    task_id: str,
    user: dict = Depends(get_current_user)
):
    """将指定任务标记为失败
    用于手动清理卡住的任务
    """
    try:
        svc = get_simple_analysis_service()
        # 更新内存中的任务状态
        from app.services.memory_state_manager import TaskStatus
        await svc.memory_manager.update_task_status(
            task_id=task_id,
            status=TaskStatus.FAILED,
            message="手动标记为失败",
            error_message="用户手动标记为失败"
        )
        # 更新 MongoDB 中的任务状态
        from app.core.database import get_mongo_db
        from datetime import datetime
        db = get_mongo_db()
        result = await db.analysis_tasks.update_one(
            {"task_id": task_id},
            {
                "$set": {
                    "status": "failed",
                    "last_error": "用户手动标记为失败",
                    "completed_at": datetime.utcnow(),
                    "updated_at": datetime.utcnow()
                }
            }
        )
        if result.modified_count > 0:
            logger.info(f"✅ 任务 {task_id} 已标记为失败")
            return {
                "success": True,
                "message": "任务已标记为失败"
            }
        else:
            logger.warning(f"⚠️ 任务 {task_id} 未找到或已是失败状态")
```

```
        return {
            "success": True,
            "message": "任务未找到或已是失败状态"
        }
    except Exception as e:
        logger.error(f"❌ 标记任务失败: {e}")
        raise HTTPException(status_code=500, detail=f"标记任务失败: {str(e)}")

@router.delete("/tasks/{task_id}")
async def delete_task(
    task_id: str,
    user: dict = Depends(get_current_user)
):
    """删除指定任务
    从内存和数据库中删除任务记录
    """
    try:
        svc = get_simple_analysis_service()
        # 从内存中删除任务
        await svc.memory_manager.remove_task(task_id)
        # 从 MongoDB 中删除任务
        from app.core.database import get_mongo_db
        db = get_mongo_db()
        result = await db.analysis_tasks.delete_one({"task_id": task_id})
        if result.deleted_count > 0:
            logger.info(f"☑ 任务 {task_id} 已删除")
            return {
                "success": True,
                "message": "任务已删除"
            }
        else:
            logger.warning(f"⚠️ 任务 {task_id} 未找到")
            return {
                "success": True,
                "message": "任务未找到"
            }
    except Exception as e:
        logger.error(f"❌ 删除任务失败: {e}")
        raise HTTPException(status_code=500, detail=f"删除任务失败: {str(e)}")
```

FILE: app/routers/auth_db.py

```
"""
基于数据库的认证路由 - 改进版
替代原有的基于配置文件的认证机制
"""
import time
from typing import Optional
from fastapi import APIRouter, Depends, HTTPException, Header, Request
from pydantic import BaseModel
from app.services.auth_service import AuthService
from app.services.user_service import user_service
from app.models.user import UserCreate, UserUpdate
from app.services.operation_log_service import log_operation
from app.models.operation_log import ActionType
# 尝试导入日志管理器
try:
    from tradingagents.utils.logging_manager import get_logger
except ImportError:
    # 如果导入失败，使用标准日志
    import logging
    def get_logger(name: str) -> logging.Logger:
        return logging.getLogger(name)
logger = get_logger('auth_db')
# 统一响应格式
class ApiResponse(BaseModel):
    success: bool = True
    data: dict = {}
    message: str = ""
router = APIRouter()
class LoginRequest(BaseModel):
    username: str
    password: str
class LoginResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    expires_in: int
    user: dict
class RefreshTokenRequest(BaseModel):
    refresh_token: str
class RefreshTokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    expires_in: int
class ChangePasswordRequest(BaseModel):
    old_password: str
    new_password: str
class ResetPasswordRequest(BaseModel):
    username: str
    new_password: str
class CreateUserRequest(BaseModel):
    username: str
    email: str
    password: str
    is_admin: bool = False
async def get_current_user(authorization: Optional[str] = Header(default=None)) -> dict:
    """获取当前用户信息"""
    logger.debug(f"🔐 认证检查开始")
    logger.debug(f"📋 Authorization header: {authorization[:50] if authorization else 'None'}...")
    if not authorization:
        logger.warning("❌ 没有Authorization header")
        raise HTTPException(status_code=401, detail="No authorization header")
```

```
    if not authorization.lower().startswith("bearer "):
        logger.warning(f"❌ Authorization header格式错误: {authorization[:20]}...")
        raise HTTPException(status_code=401, detail="Invalid authorization format")
    token = authorization.split(" ", 1)[1]
    logger.debug(f"⌨️ 提取的token长度: {len(token)}")
    logger.debug(f"⌨️ Token前20位: {token[:20]}...")
    token_data = AuthService.verify_token(token)
    logger.debug(f"🔍 Token验证结果: {token_data is not None}")
    if not token_data:
        logger.warning("❌ Token验证失败")
        raise HTTPException(status_code=401, detail="Invalid token")
    # 从数据库获取用户信息
    user = await user_service.get_user_by_username(token_data.sub)
    if not user:
        logger.warning(f"❌ 用户不存在: {token_data.sub}")
        raise HTTPException(status_code=401, detail="User not found")
    if not user.is_active:
        logger.warning(f"❌ 用户已禁用: {token_data.sub}")
        raise HTTPException(status_code=401, detail="User is inactive")
    logger.debug(f"☑️ 认证成功，用户: {token_data.sub}")
    # 返回完整的用户信息，包括偏好设置
    return {
        "id": str(user.id),
        "username": user.username,
        "email": user.email,
        "name": user.username,
        "is_admin": user.is_admin,
        "roles": ["admin"] if user.is_admin else ["user"],
        "preferences": user.preferences.model_dump() if user.preferences else {}
    }
@router.post("/login")
async def login(payload: LoginRequest, request: Request):
    """用户登录"""
    start_time = time.time()
    # 获取客户端信息
    ip_address = request.client.host if request.client else "unknown"
    user_agent = request.headers.get("user-agent", "")
    logger.info(f"🔐 登录请求 - 用户名: {payload.username}, IP: {ip_address}")
    try:
        # 验证输入
        if not payload.username or not payload.password:
            logger.warning(f"❌ 登录失败 - 用户名或密码为空")
            await log_operation(
                user_id="unknown",
                username=payload.username or "unknown",
                action_type=ActionType.USER_LOGIN,
                action="用户登录",
                details={"reason": "用户名和密码不能为空"},
                success=False,
                error_message="用户名和密码不能为空",
                duration_ms=int((time.time() - start_time) * 1000),
                ip_address=ip_address,
                user_agent=user_agent
            )
            raise HTTPException(status_code=400, detail="用户名和密码不能为空")
        logger.info(f"🔍 开始认证用户: {payload.username}")
        # 使用数据库认证
        user = await user_service.authenticate_user(payload.username, payload.password)
        logger.info(f"🔍 认证结果: user={'存在' if user else '不存在'}")
        if not user:
            logger.warning(f"❌ 登录失败 - 用户名或密码错误: {payload.username}")
            await log_operation(
                user_id="unknown",
                username=payload.username,
                action_type=ActionType.USER_LOGIN,
                action="用户登录",
                details={"reason": "用户名或密码错误"},
                success=False,
                error_message="用户名或密码错误",
                duration_ms=int((time.time() - start_time) * 1000),
                ip_address=ip_address,
                user_agent=user_agent
            )
            raise HTTPException(status_code=401, detail="用户名或密码错误")
        # 生成 token
        token = AuthService.create_access_token(sub=user.username)
        refresh_token = AuthService.create_access_token(sub=user.username, expires_delta=60*60*24*7)  # 7天有效期
        # 记录登录成功日志
        await log_operation(
            user_id=str(user.id),
            username=user.username,
            action_type=ActionType.USER_LOGIN,
            action="用户登录",
            details={"login_method": "password"},
            success=True,
            duration_ms=int((time.time() - start_time) * 1000),
            ip_address=ip_address,
            user_agent=user_agent
        )
        return {
            "success": True,
            "data": {
                "access_token": token,
                "refresh_token": refresh_token,
                "expires_in": 60 * 60,
                "user": {
                    "id": str(user.id),
                    "username": user.username,
                    "email": user.email,
                    "name": user.username,
                    "is_admin": user.is_admin
                }
            },
```

```
            "message": "登录成功"
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 登录异常: {e}")
        await log_operation(
            user_id="unknown",
            username=payload.username or "unknown",
            action_type=ActionType.USER_LOGIN,
            action="用户登录",
            details={"error": str(e)},
            success=False,
            error_message=f"系统错误: {str(e)}",
            duration_ms=int((time.time() - start_time) * 1000),
            ip_address=ip_address,
            user_agent=user_agent
        )
        raise HTTPException(status_code=500, detail="登录过程中发生系统错误")
@router.post("/refresh")
async def refresh_token(payload: RefreshTokenRequest):
    """刷新访问令牌"""
    try:
        logger.debug(f"🔄 收到refresh token请求")
        logger.debug(f"📝 Refresh token长度: {len(payload.refresh_token) if payload.refresh_token else 0}")
        if not payload.refresh_token:
            logger.warning("❌ Refresh token为空")
            raise HTTPException(status_code=401, detail="Refresh token is required")
        # 验证refresh token
        token_data = AuthService.verify_token(payload.refresh_token)
        logger.debug(f"🔍 Token验证结果: {token_data is not None}")
        if not token_data:
            logger.warning("❌ Refresh token验证失败")
            raise HTTPException(status_code=401, detail="Invalid refresh token")
        # 验证用户是否仍然存在且激活
        user = await user_service.get_user_by_username(token_data.sub)
        if not user or not user.is_active:
            logger.warning(f"❌ 用户不存在或已禁用: {token_data.sub}")
            raise HTTPException(status_code=401, detail="User not found or inactive")
        logger.debug(f"✅ Token验证成功，用户: {token_data.sub}")
        # 生成新的tokens
        new_token = AuthService.create_access_token(sub=token_data.sub)
        new_refresh_token = AuthService.create_access_token(sub=token_data.sub, expires_delta=60*60*24*7)
        logger.debug(f"🎉 新token生成成功")
        return {
            "success": True,
            "data": {
                "access_token": new_token,
                "refresh_token": new_refresh_token,
                "expires_in": 60 * 60
            },
            "message": "Token刷新成功"
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ Refresh token处理异常: {str(e)}")
        raise HTTPException(status_code=401, detail=f"Token refresh failed: {str(e)}")
@router.post("/logout")
async def logout(request: Request, user: dict = Depends(get_current_user)):
    """用户登出"""
    start_time = time.time()
    # 获取客户端信息
    ip_address = request.client.host if request.client else "unknown"
    user_agent = request.headers.get("user-agent", "")
    try:
        # 记录登出日志
        await log_operation(
            user_id=user["id"],
            username=user["username"],
            action_type=ActionType.USER_LOGOUT,
            action="用户登出",
            details={"logout_method": "manual"},
            success=True,
            duration_ms=int((time.time() - start_time) * 1000),
            ip_address=ip_address,
            user_agent=user_agent
        )
        return {
            "success": True,
            "data": {},
            "message": "登出成功"
        }
    except Exception as e:
        logger.error(f"记录登出日志失败: {e}")
        return {
            "success": True,
            "data": {},
            "message": "登出成功"
        }
@router.get("/me")
async def me(user: dict = Depends(get_current_user)):
    """获取当前用户信息"""
    return {
        "success": True,
        "data": user,
        "message": "获取用户信息成功"
    }
@router.put("/me")
async def update_me(
    payload: dict,
    user: dict = Depends(get_current_user)
):
```

```
    """更新当前用户信息"""
    try:
        from app.models.user import UserUpdate, UserPreferences
        # 构建更新数据
        update_data = {}
        # 更新邮箱
        if "email" in payload:
            update_data["email"] = payload["email"]
        # 更新偏好设置（支持部分更新）
        if "preferences" in payload:
            # 获取当前偏好
            current_prefs = user.get("preferences", {})
            # 合并新的偏好设置
            merged_prefs = {**current_prefs, **payload["preferences"]}
            # 创建 UserPreferences 对象
            update_data["preferences"] = UserPreferences(**merged_prefs)
        # 如果有语言设置，更新到偏好中
        if "language" in payload:
            if "preferences" not in update_data:
                # 获取当前偏好
                current_prefs = user.get("preferences", {})
                update_data["preferences"] = UserPreferences(**current_prefs)
            update_data["preferences"].language = payload["language"]
        # 如果有时区设置，更新到偏好中（如果需要）
        # 注意：时区通常是系统级设置，不是用户级设置
        # 调用服务更新用户
        user_update = UserUpdate(**update_data)
        updated_user = await user_service.update_user(user["username"], user_update)
        if not updated_user:
            raise HTTPException(status_code=400, detail="更新失败，邮箱可能已被使用")
        # 返回更新后的用户信息
        return {
            "success": True,
            "data": updated_user.model_dump(by_alias=True),
            "message": "用户信息更新成功"
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"更新用户信息失败: {e}", exc_info=True)
        raise HTTPException(status_code=500, detail=f"更新用户信息失败: {str(e)}")
@router.post("/change-password")
async def change_password(
    payload: ChangePasswordRequest,
    request: Request,
    user: dict = Depends(get_current_user)
):
    """修改密码"""
    try:
        # 使用数据库服务修改密码
        success = await user_service.change_password(
            user["username"],
            payload.old_password,
            payload.new_password
        )

        if not success:
            raise HTTPException(status_code=400, detail="旧密码错误")
        return {
            "success": True,
            "data": {},
            "message": "密码修改成功"
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"修改密码失败: {e}")
        raise HTTPException(status_code=500, detail=f"修改密码失败: {str(e)}")
@router.post("/reset-password")
async def reset_password(
    payload: ResetPasswordRequest,
    request: Request,
    user: dict = Depends(get_current_user)
):
    """重置密码（管理员操作）"""
    try:
        # 检查权限
        if not user.get("is_admin", False):
            raise HTTPException(status_code=403, detail="权限不足")
        # 重置密码
        success = await user_service.reset_password(payload.username, payload.new_password)

        if not success:
            raise HTTPException(status_code=404, detail="用户不存在")
        return {
            "success": True,
            "data": {},
            "message": f"用户 {payload.username} 的密码已重置"
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"重置密码失败: {e}")
        raise HTTPException(status_code=500, detail=f"重置密码失败: {str(e)}")
@router.post("/create-user")
async def create_user(
    payload: CreateUserRequest,
    request: Request,
    user: dict = Depends(get_current_user)
):
    """创建用户（管理员操作）"""
    try:
        # 检查权限
```

```
        if not user.get("is_admin", False):
            raise HTTPException(status_code=403, detail="权限不足")
        # 创建用户
        user_create = UserCreate(
            username=payload.username,
            email=payload.email,
            password=payload.password
        )

        new_user = await user_service.create_user(user_create)

        if not new_user:
            raise HTTPException(status_code=400, detail="用户名或邮箱已存在")
        # 如果需要设置为管理员
        if payload.is_admin:
            from pymongo import MongoClient
            from app.core.config import settings
            client = MongoClient(settings.MONGO_URI)
            db = client[settings.MONGO_DB]
            db.users.update_one(
                {"username": payload.username},
                {"$set": {"is_admin": True}}
            )
        return {
            "success": True,
            "data": {
                "id": str(new_user.id),
                "username": new_user.username,
                "email": new_user.email,
                "is_admin": payload.is_admin
            },
            "message": f"用户 {payload.username} 创建成功"
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"创建用户失败: {e}")
        raise HTTPException(status_code=500, detail=f"创建用户失败: {str(e)}")
@router.get("/users")
async def list_users(
    skip: int = 0,
    limit: int = 100,
    user: dict = Depends(get_current_user)
):
    """获取用户列表（管理员操作）"""
    try:
        # 检查权限
        if not user.get("is_admin", False):
            raise HTTPException(status_code=403, detail="权限不足")
        users = await user_service.list_users(skip=skip, limit=limit)

        return {
            "success": True,
            "data": {
                "users": [user.model_dump() for user in users],
                "total": len(users)
            },
            "message": "获取用户列表成功"
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"获取用户列表失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取用户列表失败: {str(e)}")
```

FILE: app/routers/baostock_init.py

```
#!/usr/bin/env python3
"""
BaoStock初始化API路由
提供BaoStock数据初始化的RESTful API接口
"""
import asyncio
import logging
from datetime import datetime
from typing import Dict, Any, Optional
from fastapi import APIRouter, BackgroundTasks, HTTPException
from pydantic import BaseModel, Field
from app.worker.baostock_init_service import BaoStockInitService
from app.worker.baostock_sync_service import BaoStockSyncService
logger = logging.getLogger(__name__)
router = APIRouter(prefix="/api/baostock-init", tags=["BaoStock初始化"])
# 全局状态管理
_initialization_status = {
    "is_running": False,
    "current_task": None,
    "stats": None,
    "start_time": None,
    "last_update": None
}
class InitializationRequest(BaseModel):
    """初始化请求模型"""
    historical_days: int = Field(default=365, ge=1, le=3650, description="历史数据天数")
    force: bool = Field(default=False, description="是否强制重新初始化")
class InitializationResponse(BaseModel):
    """初始化响应模型"""
    success: bool
    message: str
    task_id: Optional[str] = None
    data: Optional[Dict[str, Any]] = None
@router.get("/status", response_model=Dict[str, Any])
async def get_database_status():
```

```
    """获取数据库状态"""
    try:
        service = BaoStockInitService()
        status = await service.check_database_status()

        return {
            "success": True,
            "data": status,
            "message": "数据库状态获取成功"
        }

    except Exception as e:
        logger.error(f"获取数据库状态失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取数据库状态失败: {e}")
@router.get("/connection-test", response_model=Dict[str, Any])
async def test_baostock_connection():
    """测试BaoStock连接"""
    try:
        service = BaoStockSyncService()
        connected = await service.provider.test_connection()

        return {
            "success": connected,
            "data": {
                "connected": connected,
                "test_time": datetime.now().isoformat()
            },
            "message": "BaoStock连接正常" if connected else "BaoStock连接失败"
        }

    except Exception as e:
        logger.error(f"BaoStock连接测试失败: {e}")
        raise HTTPException(status_code=500, detail=f"连接测试失败: {e}")
@router.post("/start-full", response_model=InitializationResponse)
async def start_full_initialization(
    request: InitializationRequest,
    background_tasks: BackgroundTasks
):
    """启动完整初始化"""
    global _initialization_status

    if _initialization_status["is_running"]:
        raise HTTPException(
            status_code=409,
            detail="初始化任务正在运行中，请等待完成后再试"
        )

    try:
        # 生成任务ID
        task_id = f"baostock_full_init_{datetime.now().strftime('%Y%m%d_%H%M%S')}"

        # 更新状态
        _initialization_status.update({
            "is_running": True,
            "current_task": "full_initialization",
            "stats": None,
            "start_time": datetime.now(),
            "last_update": datetime.now()
        })

        # 启动后台任务
        background_tasks.add_task(
            _run_full_initialization_task,
            request.historical_days,
            request.force,
            task_id
        )

        return InitializationResponse(
            success=True,
            message="完整初始化任务已启动",
            task_id=task_id,
            data={
                "historical_days": request.historical_days,
                "force": request.force,
                "estimated_duration": "30-60分钟"
            }
        )

    except Exception as e:
        _initialization_status["is_running"] = False
        logger.error(f"启动完整初始化失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动初始化失败: {e}")
@router.post("/start-basic", response_model=InitializationResponse)
async def start_basic_initialization(background_tasks: BackgroundTasks):
    """启动基础初始化"""
    global _initialization_status

    if _initialization_status["is_running"]:
        raise HTTPException(
            status_code=409,
            detail="初始化任务正在运行中，请等待完成后再试"
        )

    try:
        # 生成任务ID
        task_id = f"baostock_basic_init_{datetime.now().strftime('%Y%m%d_%H%M%S')}"

        # 更新状态
        _initialization_status.update({
            "is_running": True,
            "current_task": "basic_initialization",
            "stats": None,
            "start_time": datetime.now(),
            "last_update": datetime.now()
```

```
        })

        # 启动后台任务
        background_tasks.add_task(_run_basic_initialization_task, task_id)

        return InitializationResponse(
            success=True,
            message="基础初始化任务已启动",
            task_id=task_id,
            data={
                "estimated_duration": "10-20分钟"
            }
        )

    except Exception as e:
        _initialization_status["is_running"] = False
        logger.error(f"启动基础初始化失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动初始化失败: {e}")

@router.get("/initialization-status", response_model=Dict[str, Any])
async def get_initialization_status():
    """获取初始化状态"""
    global _initialization_status

    try:
        status = _initialization_status.copy()

        # 计算运行时间
        if status["start_time"]:
            if status["is_running"]:
                duration = (datetime.now() - status["start_time"]).total_seconds()
            else:
                duration = (status["last_update"] - status["start_time"]).total_seconds() if status["last_update"] else 0
            status["duration"] = duration

        # 格式化统计信息
        if status["stats"]:
            stats = status["stats"]
            status["progress"] = {
                "completed_steps": stats.completed_steps,
                "total_steps": stats.total_steps,
                "current_step": stats.current_step,
                "progress_percent": (stats.completed_steps / stats.total_steps) * 100
            }
            status["data_summary"] = {
                "basic_info_count": stats.basic_info_count,
                "quotes_count": stats.quotes_count,
                "historical_records": stats.historical_records,
                "financial_records": stats.financial_records,
                "error_count": len(stats.errors)
            }

        return {
            "success": True,
            "data": status,
            "message": "状态获取成功"
        }

    except Exception as e:
        logger.error(f"获取初始化状态失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取状态失败: {e}")

@router.post("/stop", response_model=Dict[str, Any])
async def stop_initialization():
    """停止初始化任务"""
    global _initialization_status

    if not _initialization_status["is_running"]:
        return {
            "success": True,
            "message": "没有正在运行的初始化任务",
            "data": {"was_running": False}
        }

    try:
        # 更新状态
        _initialization_status.update({
            "is_running": False,
            "current_task": None,
            "last_update": datetime.now()
        })

        return {
            "success": True,
            "message": "初始化任务已停止",
            "data": {"was_running": True}
        }

    except Exception as e:
        logger.error(f"停止初始化任务失败: {e}")
        raise HTTPException(status_code=500, detail=f"停止任务失败: {e}")

async def _run_full_initialization_task(historical_days: int, force: bool, task_id: str):
    """运行完整初始化任务"""
    global _initialization_status

    try:
        logger.info(f"🚀 开始BaoStock完整初始化任务: {task_id}")

        service = BaoStockInitService()
        stats = await service.full_initialization(
            historical_days=historical_days,
            force=force
        )

        # 更新状态
        _initialization_status.update({
            "is_running": False,
```

```
            "stats": stats,
            "last_update": datetime.now()
        })

        if stats.completed_steps == stats.total_steps:
            logger.info(f"✅ BaoStock完整初始化任务完成: {task_id}")
        else:
            logger.warning(f"⚠️ BaoStock完整初始化任务部分完成: {task_id}")

    except Exception as e:
        logger.error(f"❌ BaoStock完整初始化任务失败: {task_id}, 错误: {e}")
        _initialization_status.update({
            "is_running": False,
            "last_update": datetime.now()
        })
async def _run_basic_initialization_task(task_id: str):
    """运行基础初始化任务"""
    global _initialization_status

    try:
        logger.info(f"🚀 开始BaoStock基础初始化任务: {task_id}")

        service = BaoStockInitService()
        stats = await service.basic_initialization()

        # 更新状态
        _initialization_status.update({
            "is_running": False,
            "stats": stats,
            "last_update": datetime.now()
        })

        if stats.completed_steps == stats.total_steps:
            logger.info(f"✅ BaoStock基础初始化任务完成: {task_id}")
        else:
            logger.warning(f"⚠️ BaoStock基础初始化任务部分完成: {task_id}")

    except Exception as e:
        logger.error(f"❌ BaoStock基础初始化任务失败: {task_id}, 错误: {e}")
        _initialization_status.update({
            "is_running": False,
            "last_update": datetime.now()
        })
@router.get("/service-status", response_model=Dict[str, Any])
async def get_service_status():
    """获取BaoStock服务状态"""
    try:
        service = BaoStockSyncService()
        status = await service.check_service_status()

        return {
            "success": True,
            "data": status,
            "message": "服务状态获取成功"
        }

    except Exception as e:
        logger.error(f"获取服务状态失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取服务状态失败: {e}")
```

FILE: app/routers/cache.py

```
"""
缓存管理路由
提供缓存统计、清理等功能
"""
from fastapi import APIRouter, HTTPException, Depends, Query
from typing import Optional
from datetime import datetime, timedelta
from app.routers.auth_db import get_current_user
from app.core.response import ok
from tradingagents.utils.logging_manager import get_logger
logger = get_logger(__name__)
router = APIRouter(prefix="/api/cache", tags=["cache"])
@router.get("/stats")
async def get_cache_stats(current_user: dict = Depends(get_current_user)):
    """
    获取缓存统计信息

    Returns:
        dict: 缓存统计数据
    """
    try:
        from tradingagents.dataflows.cache import get_cache

        cache = get_cache()

        # 获取缓存统计
        stats = cache.get_cache_stats()

        logger.info(f"用户 {current_user['username']} 获取缓存统计")

        return ok(
            data={
                "totalFiles": stats.get('total_files', 0),
                "totalSize": stats.get('total_size', 0),  # 字节
                "maxSize": 1024 * 1024 * 1024,  # 1GB
                "stockDataCount": stats.get('stock_data_count', 0),
                "newsDataCount": stats.get('news_count', 0),
                "analysisDataCount": stats.get('fundamentals_count', 0)
            },
            message="获取缓存统计成功"
        )
```

```
    except Exception as e:
        logger.error(f"获取缓存统计失败: {e}")
        raise HTTPException(
            status_code=500,
            detail=f"获取缓存统计失败: {str(e)}"
        )
@router.delete("/cleanup")
async def cleanup_old_cache(
    days: int = Query(7, ge=1, le=30, description="清理多少天前的缓存"),
    current_user: dict = Depends(get_current_user)
):
    """
    清理过期缓存

    Args:
        days: 清理多少天前的缓存

    Returns:
        dict: 清理结果
    """
    try:
        from tradingagents.dataflows.cache import get_cache

        cache = get_cache()

        # 清理过期缓存
        cache.clear_old_cache(days)

        logger.info(f"用户 {current_user['username']} 清理了 {days} 天前的缓存")

        return ok(
            data={"days": days},
            message=f"已清理 {days} 天前的缓存"
        )

    except Exception as e:
        logger.error(f"清理缓存失败: {e}")
        raise HTTPException(
            status_code=500,
            detail=f"清理缓存失败: {str(e)}"
        )
@router.delete("/clear")
async def clear_all_cache(current_user: dict = Depends(get_current_user)):
    """
    清空所有缓存
    Returns:
        dict: 清理结果
    """
    try:
        from tradingagents.dataflows.cache import get_cache
        cache = get_cache()
        # 清空所有缓存（清理所有过期和未过期的缓存）
        # 使用 clear_old_cache(0) 来清理所有缓存
        cache.clear_old_cache(0)
        logger.warning(f"用户 {current_user['username']} 清空了所有缓存")
        return ok(
            data={},
            message="所有缓存已清空"
        )
    except Exception as e:
        logger.error(f"清空缓存失败: {e}")
        raise HTTPException(
            status_code=500,
            detail=f"清空缓存失败: {str(e)}"
        )
@router.get("/details")
async def get_cache_details(
    page: int = Query(1, ge=1, description="页码"),
    page_size: int = Query(20, ge=1, le=100, description="每页数量"),
    current_user: dict = Depends(get_current_user)
):
    """
    获取缓存详情列表

    Args:
        page: 页码
        page_size: 每页数量

    Returns:
        dict: 缓存详情列表
    """
    try:
        from tradingagents.dataflows.cache import get_cache

        cache = get_cache()

        # 获取缓存详情
        # 注意：这个方法可能需要在缓存类中实现
        try:
            details = cache.get_cache_details(page=page, page_size=page_size)
        except AttributeError:
            # 如果缓存类没有实现这个方法，返回空列表
            details = {
                "items": [],
                "total": 0,
                "page": page,
                "page_size": page_size
            }

        logger.info(f"用户 {current_user['username']} 获取缓存详情 (页码: {page})")

        return ok(
            data=details,
```

```
            message="获取缓存详情成功"
        )

    except Exception as e:
        logger.error(f"获取缓存详情失败: {e}")
        raise HTTPException(
            status_code=500,
            detail=f"获取缓存详情失败: {str(e)}"
        )
@router.get("/backend-info")
async def get_cache_backend_info(current_user: dict = Depends(get_current_user)):
    """
    获取缓存后端信息

    Returns:
        dict: 缓存后端配置信息
    """
    try:
        from tradingagents.dataflows.cache import get_cache

        cache = get_cache()

        # 获取后端信息
        try:
            backend_info = cache.get_cache_backend_info()
        except AttributeError:
            # 如果缓存类没有实现这个方法，返回基本信息
            backend_info = {
                "system": "file",
                "primary_backend": "file",
                "fallback_enabled": False
            }

        logger.info(f"用户 {current_user['username']} 获取缓存后端信息")

        return ok(
            data=backend_info,
            message="获取缓存后端信息成功"
        )

    except Exception as e:
        logger.error(f"获取缓存后端信息失败: {e}")
        raise HTTPException(
            status_code=500,
            detail=f"获取缓存后端信息失败: {str(e)}"
        )
```

FILE: app/routers/config.py

```
"""
配置管理API路由
"""
import logging
from typing import List, Dict, Any
from fastapi import APIRouter, Depends, HTTPException, status
from pydantic import BaseModel
from app.routers.auth_db import get_current_user
from app.models.user import User
from app.models.config import (
    SystemConfigResponse, LLMConfigRequest, DataSourceConfigRequest,
    DatabaseConfigRequest, ConfigTestRequest, ConfigTestResponse,
    LLMConfig, DataSourceConfig, DatabaseConfig,
    LLMProvider, LLMProviderRequest, LLMProviderResponse,
    MarketCategory, MarketCategoryRequest, DataSourceGrouping,
    DataSourceGroupingRequest, DataSourceOrderRequest,
    ModelCatalog, ModelInfo
)
from app.services.config_service import config_service
from datetime import datetime
from app.utils.timezone import now_tz
from app.services.operation_log_service import log_operation
from app.models.operation_log import ActionType
from app.services.config_provider import provider as config_provider

router = APIRouter(prefix="/config", tags=["配置管理"])
logger = logging.getLogger("webapi")

# ===== 配置重载端点 =====
@router.post("/reload", summary="重新加载配置")
async def reload_config(current_user: dict = Depends(get_current_user)):
    """
    重新加载配置并桥接到环境变量
    用于配置更新后立即生效，无需重启服务
    """
    try:
        from app.core.config_bridge import reload_bridged_config
        success = reload_bridged_config()
        if success:
            await log_operation(
                user_id=str(current_user.get("user_id", "")),
                username=current_user.get("username", "unknown"),
                action_type=ActionType.CONFIG_MANAGEMENT,
                action="重载配置",
                details={"action": "reload_config"},
                ip_address="",
                user_agent=""
            )
            return {
                "success": True,
                "message": "配置重载成功",
                "data": {
                    "reloaded_at": now_tz().isoformat()
                }
            }
```

```
        else:
            return {
                "success": False,
                "message": "配置重载失败，请查看日志"
            }
    except Exception as e:
        logger.error(f"配置重载失败: {e}", exc_info=True)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"配置重载失败: {str(e)}"
        )

# ===== 方案A：敏感字段响应脱敏 & 请求清洗 =====
from copy import deepcopy

def _sanitize_llm_configs(items):
    try:
        return [LLMConfig(**{**i.model_dump(), "api_key": None}) for i in items]
    except Exception:
        return items

def _sanitize_datasource_configs(items):
    """
    脱敏数据源配置，返回缩略的 API Key
    逻辑：
    1. 如果数据库中有有效的 API Key，返回缩略版本
    2. 如果数据库中没有，尝试从环境变量读取并返回缩略版本
    3. 如果都没有，返回 None
    """
    try:
        from app.utils.api_key_utils import (
            is_valid_api_key,
            truncate_api_key,
            get_env_api_key_for_datasource
        )
        result = []
        for item in items:
            data = item.model_dump()

            # 处理 API Key
            db_key = data.get("api_key")
            if is_valid_api_key(db_key):
                # 数据库中有有效的 API Key，返回缩略版本
                data["api_key"] = truncate_api_key(db_key)
            else:
                # 数据库中没有有效的 API Key，尝试从环境变量读取
                ds_type = data.get("type")
                if isinstance(ds_type, str):
                    env_key = get_env_api_key_for_datasource(ds_type)
                    if env_key:
                        # 环境变量中有有效的 API Key，返回缩略版本
                        data["api_key"] = truncate_api_key(env_key)
                    else:
                        data["api_key"] = None
                else:
                    data["api_key"] = None

            # 处理 API Secret（同样的逻辑）
            db_secret = data.get("api_secret")
            if is_valid_api_key(db_secret):
                data["api_secret"] = truncate_api_key(db_secret)
            else:
                data["api_secret"] = None

            result.append(DataSourceConfig(**data))

        return result
    except Exception as e:
        print(f"⚠️ 脱敏数据源配置失败: {e}")
        return items

def _sanitize_database_configs(items):
    try:
        return [DatabaseConfig(**{**i.model_dump(), "password": None}) for i in items]
    except Exception:
        return items

def _sanitize_kv(d: Dict[str, Any]) -> Dict[str, Any]:
    """对字典中的可能敏感键进行脱敏（仅用于响应）。"""
    try:
        if not isinstance(d, dict):
            return d
        sens_patterns = ("key", "secret", "password", "token", "client_secret")
        redacted = {}
        for k, v in d.items():
            if isinstance(k, str) and any(p in k.lower() for p in sens_patterns):
                redacted[k] = None
            else:
                redacted[k] = v
        return redacted
    except Exception:
        return d

class SetDefaultRequest(BaseModel):
    """设置默认配置请求"""
    name: str

@router.get("/system", response_model=SystemConfigResponse)
async def get_system_config(
    current_user: User = Depends(get_current_user)
):
    """获取系统配置"""
    try:
        config = await config_service.get_system_config()
        if not config:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="系统配置不存在"
            )
        return SystemConfigResponse(
            config_name=config.config_name,
            config_type=config.config_type,
            llm_configs=_sanitize_llm_configs(config.llm_configs),
            default_llm=config.default_llm,
```

```
            data_source_configs=_sanitize_datasource_configs(config.data_source_configs),
            default_data_source=config.default_data_source,
            database_configs=_sanitize_database_configs(config.database_configs),
            system_settings=_sanitize_kv(config.system_settings),
            created_at=config.created_at,
            updated_at=config.updated_at,
            version=config.version,
            is_active=config.is_active
        )
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取系统配置失败: {str(e)}"
        )

# ========== 大模型厂家管理 ==========
@router.get("/llm/providers", response_model=List[LLMProviderResponse])
async def get_llm_providers(
    current_user: User = Depends(get_current_user)
):
    """获取所有大模型厂家"""
    try:
        from app.utils.api_key_utils import (
            is_valid_api_key,
            truncate_api_key,
            get_env_api_key_for_provider
        )
        providers = await config_service.get_llm_providers()
        result = []
        for provider in providers:
            # 处理 API Key：优先使用数据库配置，如果数据库没有则检查环境变量
            db_key_valid = is_valid_api_key(provider.api_key)
            if db_key_valid:
                # 数据库中有有效的 API Key，返回缩略版本
                api_key_display = truncate_api_key(provider.api_key)
            else:
                # 数据库中没有有效的 API Key，尝试从环境变量读取
                env_key = get_env_api_key_for_provider(provider.name)
                if env_key:
                    # 环境变量中有有效的 API Key，返回缩略版本
                    api_key_display = truncate_api_key(env_key)
                else:
                    api_key_display = None
            # 处理 API Secret（同样的逻辑）
            db_secret_valid = is_valid_api_key(provider.api_secret)
            if db_secret_valid:
                api_secret_display = truncate_api_key(provider.api_secret)
            else:
                # 注意：API Secret 通常不在环境变量中，所以这里只检查数据库
                api_secret_display = None
            result.append(
                LLMProviderResponse(
                    id=str(provider.id),
                    name=provider.name,
                    display_name=provider.display_name,
                    description=provider.description,
                    website=provider.website,
                    api_doc_url=provider.api_doc_url,
                    logo_url=provider.logo_url,
                    is_active=provider.is_active,
                    supported_features=provider.supported_features,
                    default_base_url=provider.default_base_url,
                    # 返回缩略的 API Key（前6位 + "..." + 后6位）
                    api_key=api_key_display,
                    api_secret=api_secret_display,
                    extra_config={
                        **provider.extra_config,
                        "has_api_key": bool(api_key_display),
                        "has_api_secret": bool(api_secret_display)
                    },
                    created_at=provider.created_at,
                    updated_at=provider.updated_at
                )
            )
        return result
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取厂家列表失败: {str(e)}"
        )

@router.post("/llm/providers", response_model=dict)
async def add_llm_provider(
    request: LLMProviderRequest,
    current_user: User = Depends(get_current_user)
):
    """添加大模型厂家（方案A：REST不接受密钥，强制清洗）"""
    try:
        sanitized = request.model_dump()
        if 'api_key' in sanitized:
            sanitized['api_key'] = ""
        provider = LLMProvider(**sanitized)
        provider_id = await config_service.add_llm_provider(provider)
        # 审计日志（忽略异常）
        try:
            await log_operation(
                user_id=str(getattr(current_user, "id", "")),
                username=getattr(current_user, "username", "unknown"),
                action_type=ActionType.CONFIG_MANAGEMENT,
                action="add_llm_provider",
                details={"provider_id": str(provider_id), "name": request.name},
                success=True,
            )
        except Exception:
            pass
        return {
```

```
            "success": True,
            "message": "厂家添加成功",
            "data": {"id": str(provider_id)}
        }
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"添加厂家失败: {str(e)}"
        )
@router.put("/llm/providers/{provider_id}", response_model=dict)
async def update_llm_provider(
    provider_id: str,
    request: LLMProviderRequest,
    current_user: User = Depends(get_current_user)
):
    """更新大模型厂家"""
    try:
        from app.utils.api_key_utils import should_skip_api_key_update
        update_data = request.model_dump(exclude_unset=True)
        # 🔥 修改：处理 API Key 的更新逻辑
        # 1. 如果 API Key 是空字符串，表示用户想清空密钥 → 保存空字符串
        # 2. 如果 API Key 是占位符或截断的密钥（如 "sk-99054..."），则删除该字段（不更新）
        # 3. 如果 API Key 是有效的完整密钥，则更新
        if 'api_key' in update_data:
            api_key = update_data.get('api_key', '')
            # 如果应该跳过更新（占位符或截断的密钥），则删除该字段
            if should_skip_api_key_update(api_key):
                del update_data['api_key']
            # 如果是空字符串，保留（表示清空）
            # 如果是有效的完整密钥，保留（表示更新）
        if 'api_secret' in update_data:
            api_secret = update_data.get('api_secret', '')
            # 同样的逻辑处理 API Secret
            if should_skip_api_key_update(api_secret):
                del update_data['api_secret']
        success = await config_service.update_llm_provider(provider_id, update_data)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="update_llm_provider",
                    details={"provider_id": provider_id, "changed_keys": list(request.model_dump().keys())},
                    success=True,
                )
            except Exception:
                pass
            return {
                "success": True,
                "message": "厂家更新成功",
                "data": {}
            }
        else:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="厂家不存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"更新厂家失败: {str(e)}"
        )
@router.delete("/llm/providers/{provider_id}", response_model=dict)
async def delete_llm_provider(
    provider_id: str,
    current_user: User = Depends(get_current_user)
):
    """删除大模型厂家"""
    try:
        success = await config_service.delete_llm_provider(provider_id)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="delete_llm_provider",
                    details={"provider_id": provider_id},
                    success=True,
                )
            except Exception:
                pass
            return {
                "success": True,
                "message": "厂家删除成功",
                "data": {}
            }
        else:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="厂家不存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"删除厂家失败: {str(e)}"
        )
```

```
@router.patch("/llm/providers/{provider_id}/toggle", response_model=dict)
async def toggle_llm_provider(
    provider_id: str,
    request: dict,
    current_user: User = Depends(get_current_user)
):
    """切换大模型厂家状态"""
    try:
        is_active = request.get("is_active", True)
        success = await config_service.toggle_llm_provider(provider_id, is_active)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="toggle_llm_provider",
                    details={"provider_id": provider_id, "is_active": bool(is_active)},
                    success=True,
                )
            except Exception:
                pass
            return {
                "success": True,
                "message": f"厂家已{'启用' if is_active else '禁用'}",
                "data": {}
            }
        else:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="厂家不存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"切换厂家状态失败: {str(e)}"
        )
@router.post("/llm/providers/{provider_id}/fetch-models", response_model=dict)
async def fetch_provider_models(
    provider_id: str,
    current_user: User = Depends(get_current_user)
):
    """从厂家 API 获取模型列表"""
    try:
        result = await config_service.fetch_provider_models(provider_id)
        return result
    except HTTPException:
        raise
    except Exception as e:
        print(f"获取模型列表失败: {e}")
        import traceback
        traceback.print_exc()
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取模型列表失败: {str(e)}"
        )
@router.post("/llm/providers/migrate-env", response_model=dict)
async def migrate_env_to_providers(
    current_user: User = Depends(get_current_user)
):
    """将环境变量配置迁移到厂家管理"""
    try:
        result = await config_service.migrate_env_to_providers()
        # 审计日志（忽略异常）
        try:
            await log_operation(
                user_id=str(getattr(current_user, "id", "")),
                username=getattr(current_user, "username", "unknown"),
                action_type=ActionType.CONFIG_MANAGEMENT,
                action="migrate_env_to_providers",
                details={
                    "migrated_count": result.get("migrated_count", 0),
                    "skipped_count": result.get("skipped_count", 0)
                },
                success=bool(result.get("success", False)),
            )
        except Exception:
            pass
        return {
            "success": result["success"],
            "message": result["message"],
            "data": {
                "migrated_count": result.get("migrated_count", 0),
                "skipped_count": result.get("skipped_count", 0)
            }
        }
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"环境变量迁移失败: {str(e)}"
        )
@router.post("/llm/providers/init-aggregators", response_model=dict)
async def init_aggregator_providers(
    current_user: User = Depends(get_current_user)
):
    """初始化聚合渠道厂家配置（302.AI、OpenRouter等）"""
    try:
        result = await config_service.init_aggregator_providers()
        # 审计日志（忽略异常）
        try:
            await log_operation(
```

```
            user_id=str(getattr(current_user, "id", "")),
            username=getattr(current_user, "username", "unknown"),
            action_type=ActionType.CONFIG_MANAGEMENT,
            action="init_aggregator_providers",
            details={
                "added_count": result.get("added", 0),
                "skipped_count": result.get("skipped", 0)
            },
            success=bool(result.get("success", False)),
        )
    except Exception:
        pass
    return {
        "success": result["success"],
        "message": result["message"],
        "data": {
            "added_count": result.get("added", 0),
            "skipped_count": result.get("skipped", 0)
        }
    }
  except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"初始化聚合渠道失败: {str(e)}"
    )
@router.post("/llm/providers/{provider_id}/test", response_model=dict)
async def test_provider_api(
  provider_id: str,
  current_user: User = Depends(get_current_user)
):
  """测试厂家API密钥"""
  try:
    logger.info(f"🔧 收到API测试请求 - provider_id: {provider_id}")
    result = await config_service.test_provider_api(provider_id)
    logger.info(f"🔧 API测试结果: {result}")
    return result
  except Exception as e:
    logger.error(f"测试厂家API失败: {e}")
    raise HTTPException(
        status_code=500,
        detail=f"测试厂家API失败: {str(e)}"
    )
# ========== 大模型配置管理 ==========
@router.post("/llm", response_model=dict)
async def add_llm_config(
  request: LLMConfigRequest,
  current_user: User = Depends(get_current_user)
):
  """添加或更新大模型配置"""
  try:
    logger.info(f"🔧 添加/更新大模型配置开始")
    logger.info(f"📊 请求数据: {request.model_dump()}")
    logger.info(f"🏷️ 厂家: {request.provider}, 模型: {request.model_name}")
    # 创建LLM配置
    llm_config_data = request.model_dump()
    logger.info(f"📋 原始配置数据: {llm_config_data}")
    # 如果没有提供API密钥，从厂家配置中获取
    if not llm_config_data.get('api_key'):
        logger.info(f"🔑 API密钥为空，从厂家配置获取: {request.provider}")
        # 获取厂家配置
        providers = await config_service.get_llm_providers()
        logger.info(f"📊 找到 {len(providers)} 个厂家配置")
        for p in providers:
            logger.info(f"  - 厂家: {p.name}, 有API密钥: {bool(p.api_key)}")
        provider_config = next((p for p in providers if p.name == request.provider), None)
        if provider_config:
            logger.info(f"✅ 找到厂家配置: {provider_config.name}")
            if provider_config.api_key:
                llm_config_data['api_key'] = provider_config.api_key
                logger.info(f"✅ 成功获取厂家API密钥 (长度: {len(provider_config.api_key)})")
            else:
                logger.warning(f"⚠️ 厂家 {request.provider} 没有配置API密钥")
                llm_config_data['api_key'] = ""
        else:
            logger.warning(f"⚠️ 未找到厂家 {request.provider} 的配置")
            llm_config_data['api_key'] = ""
    else:
        logger.info(f"🔑 使用提供的API密钥 (长度: {len(llm_config_data.get('api_key', ''))})")
    logger.info(f"📋 最终配置数据: {llm_config_data}")
    # 🔥 修改：允许通过 REST 写入密钥，但如果是无效的密钥则清空
    # 无效的密钥：空字符串、占位符（your_xxx）、长度不够
    if 'api_key' in llm_config_data:
        api_key = llm_config_data.get('api_key', '')
        # 如果是无效的 Key，则清空（让系统使用环境变量）
        if not api_key or api_key.startswith('your_') or api_key.startswith('your-') or len(api_key) <= 10:
            llm_config_data['api_key'] = ""
    # 尝试创建LLMConfig对象
    try:
        llm_config = LLMConfig(**llm_config_data)
        logger.info(f"✅ LLMConfig对象创建成功")
    except Exception as e:
        logger.error(f"❌ LLMConfig对象创建失败: {e}")
        logger.error(f"📋 失败的数据: {llm_config_data}")
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=f"配置数据验证失败: {str(e)}"
        )
    # 保存配置
    success = await config_service.update_llm_config(llm_config)
    if success:
        logger.info(f"✅ 大模型配置更新成功: {llm_config.provider}/{llm_config.model_name}")
        # 同步定价配置到 tradingagents
        try:
```

```
            from app.core.config_bridge import sync_pricing_config_now
            sync_pricing_config_now()
            logger.info(f"☑ 定价配置已同步到 tradingagents")
        except Exception as e:
            logger.warning(f"⚠ 同步定价配置失败: {e}")
        # 审计日志（忽略异常）
        try:
            await log_operation(
                user_id=str(getattr(current_user, "id", "")),
                username=getattr(current_user, "username", "unknown"),
                action_type=ActionType.CONFIG_MANAGEMENT,
                action="update_llm_config",
                details={"provider": llm_config.provider, "model_name": llm_config.model_name},
                success=True,
            )
        except Exception:
            pass
        return {"message": "大模型配置更新成功", "model_name": llm_config.model_name}
    else:
        logger.error(f"❌ 大模型配置保存失败")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail="大模型配置更新失败"
        )
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 添加大模型配置异常: {e}")
        import traceback
        logger.error(f"📋 异常堆栈: {traceback.format_exc()}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"添加大模型配置失败: {str(e)}"
        )
@router.post("/datasource", response_model=dict)
async def add_data_source_config(
    request: DataSourceConfigRequest,
    current_user: User = Depends(get_current_user)
):
    """添加数据源配置"""
    try:
        # 开源版本：所有用户都可以修改配置
        # 获取当前配置
        config = await config_service.get_system_config()
        if not config:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="系统配置不存在"
            )
        # 添加新的数据源配置
        # 🔥 修改：支持保存 API Key（与大模型厂家管理逻辑一致）
        from app.utils.api_key_utils import should_skip_api_key_update, is_valid_api_key
        _req = request.model_dump()
        # 处理 API Key
        if 'api_key' in _req:
            api_key = _req.get('api_key', '')
            # 如果是占位符或截断的密钥，清空该字段
            if should_skip_api_key_update(api_key):
                _req['api_key'] = ""
            # 如果是空字符串，保留（表示使用环境变量）
            elif api_key == '':
                _req['api_key'] = ''
            # 如果是新输入的密钥，必须验证有效性
            elif not is_valid_api_key(api_key):
                raise HTTPException(
                    status_code=status.HTTP_400_BAD_REQUEST,
                    detail="API Key 无效：长度必须大于 10 个字符，且不能是占位符"
                )
            # 有效的完整密钥，保留
        # 处理 API Secret
        if 'api_secret' in _req:
            api_secret = _req.get('api_secret', '')
            if should_skip_api_key_update(api_secret):
                _req['api_secret'] = ""
            # 如果是空字符串，保留
            elif api_secret == '':
                _req['api_secret'] = ''
            # 如果是新输入的密钥，必须验证有效性
            elif not is_valid_api_key(api_secret):
                raise HTTPException(
                    status_code=status.HTTP_400_BAD_REQUEST,
                    detail="API Secret 无效：长度必须大于 10 个字符，且不能是占位符"
                )
        ds_config = DataSourceConfig(**_req)
        config.data_source_configs.append(ds_config)
        success = await config_service.save_system_config(config)
        if success:
            # 🆕 自动创建数据源分组关系
            market_categories = _req.get('market_categories', [])
            if market_categories:
                for category_id in market_categories:
                    try:
                        grouping = DataSourceGrouping(
                            data_source_name=ds_config.name,
                            market_category_id=category_id,
                            priority=ds_config.priority,
                            enabled=ds_config.enabled
                        )
                        await config_service.add_datasource_to_category(grouping)
                    except Exception as e:
                        # 如果分组已存在或其他错误，记录但不影响主流程
                        logger.warning(f"自动创建数据源分组失败: {str(e)}")
            # 审计日志（忽略异常）
            try:
```

```
            await log_operation(
                user_id=str(getattr(current_user, "id", "")),
                username=getattr(current_user, "username", "unknown"),
                action_type=ActionType.CONFIG_MANAGEMENT,
                action="add_data_source_config",
                details={"name": ds_config.name, "market_categories": market_categories},
                success=True,
            )
        except Exception:
            pass
        return {"message": "数据源配置添加成功", "name": ds_config.name}
    else:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail="数据源配置添加失败"
        )
  except HTTPException:
    raise
  except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"添加数据源配置失败: {str(e)}"
    )
@router.post("/database", response_model=dict)
async def add_database_config(
  request: DatabaseConfigRequest,
  current_user: User = Depends(get_current_user)
):
  """添加数据库配置"""
  try:
    # 开源版本：所有用户都可以修改配置
    # 获取当前配置
    config = await config_service.get_system_config()
    if not config:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="系统配置不存在"
        )
    # 添加新的数据库配置（方案A：清洗敏感字段）
    _req = request.model_dump()
    _req['password'] = ""
    db_config = DatabaseConfig(**_req)
    config.database_configs.append(db_config)
    success = await config_service.save_system_config(config)
    if success:
        # 审计日志（忽略异常）
        try:
            await log_operation(
                user_id=str(getattr(current_user, "id", "")),
                username=getattr(current_user, "username", "unknown"),
                action_type=ActionType.CONFIG_MANAGEMENT,
                action="add_database_config",
                details={"name": db_config.name},
                success=True,
            )
        except Exception:
            pass
        return {"message": "数据库配置添加成功", "name": db_config.name}
    else:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail="数据库配置添加失败"
        )
  except HTTPException:
    raise
  except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"添加数据库配置失败: {str(e)}"
    )
@router.post("/test", response_model=ConfigTestResponse)
async def test_config(
  request: ConfigTestRequest,
  current_user: User = Depends(get_current_user)
):
  """测试配置连接"""
  try:
    if request.config_type == "llm":
        llm_config = LLMConfig(**request.config_data)
        result = await config_service.test_llm_config(llm_config)
    elif request.config_type == "datasource":
        ds_config = DataSourceConfig(**request.config_data)
        result = await config_service.test_data_source_config(ds_config)
    elif request.config_type == "database":
        db_config = DatabaseConfig(**request.config_data)
        result = await config_service.test_database_config(db_config)
    else:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="不支持的配置类型"
        )
    return ConfigTestResponse(**result)
  except HTTPException:
    raise
  except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"测试配置失败: {str(e)}"
    )
@router.post("/database/{db_name}/test", response_model=ConfigTestResponse)
async def test_saved_database_config(
  db_name: str,
  current_user: dict = Depends(get_current_user)
):
```

```
    """测试已保存的数据库配置（从数据库中获取完整配置包括密码）"""
    try:
        logger.info(f"🔧 测试已保存的数据库配置: {db_name}")
        # 从数据库获取完整的系统配置
        config = await config_service.get_system_config()
        if not config:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="系统配置不存在"
            )
        # 查找指定的数据库配置
        db_config = None
        for db in config.database_configs:
            if db.name == db_name:
                db_config = db
                break
        if not db_config:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"数据库配置 '{db_name}' 不存在"
            )
        logger.info(f"✅ 找到数据库配置: {db_config.name} ({db_config.type})")
        logger.info(f"📍 连接信息: {db_config.host}:{db_config.port}")
        logger.info(f"👤 用户名: {db_config.username or '(无)'}")
        logger.info(f"🔑 密码: {'***' if db_config.password else '(无)'}")
        # 使用完整配置进行测试
        result = await config_service.test_database_config(db_config)
        return ConfigTestResponse(**result)
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 测试数据库配置失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"测试数据库配置失败: {str(e)}"
        )
@router.get("/llm", response_model=List[LLMConfig])
async def get_llm_configs(
    current_user: User = Depends(get_current_user)
):
    """获取所有大模型配置"""
    try:
        logger.info("🔄 开始获取大模型配置...")
        config = await config_service.get_system_config()
        if not config:
            logger.warning("⚠️ 系统配置为空，返回空列表")
            return []
        logger.info(f"📊 系统配置存在，大模型配置数量: {len(config.llm_configs)}")
        # 如果没有大模型配置，创建一些示例配置
        if not config.llm_configs:
            logger.info("🔧 没有大模型配置，创建示例配置...")
            # 这里可以根据已有的厂家创建示例配置
            # 暂时返回空列表，让前端显示"暂无配置"
        # 获取所有供应商信息，用于过滤被禁用供应商的模型
        providers = await config_service.get_llm_providers()
        active_provider_names = {p.name for p in providers if p.is_active}
        # 过滤：只返回启用的模型 且 供应商也启用的模型
        filtered_configs = [
            llm_config for llm_config in config.llm_configs
            if llm_config.enabled and llm_config.provider in active_provider_names
        ]
        logger.info(f"✅ 过滤后的大模型配置数量: {len(filtered_configs)} (原始: {len(config.llm_configs)})")
        return _sanitize_llm_configs(filtered_configs)
    except Exception as e:
        logger.error(f"❌ 获取大模型配置失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取大模型配置失败: {str(e)}"
        )
@router.delete("/llm/{provider}/{model_name}")
async def delete_llm_config(
    provider: str,
    model_name: str,
    current_user: User = Depends(get_current_user)
):
    """删除大模型配置"""
    try:
        logger.info(f"🗑️ 删除大模型配置请求 - provider: {provider}, model_name: {model_name}")
        success = await config_service.delete_llm_config(provider, model_name)
        if success:
            logger.info(f"✅ 大模型配置删除成功 - {provider}/{model_name}")
            # 同步定价配置到 tradingagents
            try:
                from app.core.config_bridge import sync_pricing_config_now
                sync_pricing_config_now()
                logger.info(f"✅ 定价配置已同步到 tradingagents")
            except Exception as e:
                logger.warning(f"⚠️ 同步定价配置失败: {e}")
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="delete_llm_config",
                    details={"provider": provider, "model_name": model_name},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "大模型配置删除成功"}
        else:
            logger.warning(f"⚠️ 未找到大模型配置 - {provider}/{model_name}")
```

```
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="大模型配置不存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 删除大模型配置异常 - {provider}/{model_name}: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"删除大模型配置失败: {str(e)}"
        )
@router.post("/llm/set-default")
async def set_default_llm(
    request: SetDefaultRequest,
    current_user: User = Depends(get_current_user)
):
    """设置默认大模型"""
    try:
        success = await config_service.set_default_llm(request.name)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="set_default_llm",
                    details={"name": request.name},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "默认大模型设置成功", "default_llm": request.name}
        else:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="指定的大模型不存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"设置默认大模型失败: {str(e)}"
        )
@router.get("/datasource", response_model=List[DataSourceConfig])
async def get_data_source_configs(
    current_user: User = Depends(get_current_user)
):
    """获取所有数据源配置"""
    try:
        config = await config_service.get_system_config()
        if not config:
            return []
        return _sanitize_datasource_configs(config.data_source_configs)
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取数据源配置失败: {str(e)}"
        )
@router.put("/datasource/{name}", response_model=dict)
async def update_data_source_config(
    name: str,
    request: DataSourceConfigRequest,
    current_user: User = Depends(get_current_user)
):
    """更新数据源配置"""
    try:
        # 获取当前配置
        config = await config_service.get_system_config()
        if not config:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="系统配置不存在"
            )
        # 查找并更新数据源配置
        from app.utils.api_key_utils import should_skip_api_key_update, is_valid_api_key
        def _truncate_api_key(api_key: str, prefix_len: int = 6, suffix_len: int = 6) -> str:
            """截断 API Key 用于显示"""
            if not api_key or len(api_key) <= prefix_len + suffix_len:
                return api_key
            return f"{api_key[:prefix_len]}...{api_key[-suffix_len:]}"
        for i, ds_config in enumerate(config.data_source_configs):
            if ds_config.name == name:
                # 更新配置
                # 🔥 修改：处理 API Key 的更新逻辑（与大模型厂家管理逻辑一致）
                _req = request.model_dump()
                # 处理 API Key
                if 'api_key' in _req:
                    api_key = _req.get('api_key')
                    logger.info(f"🔍 [API Key 验证] 收到的 API Key: {repr(api_key)} (类型: {type(api_key).__name__}, 长度: {len(api_key) if api_key else 0})")
                    # 如果是 None 或空字符串，保留原值（不更新）
                    if api_key is None or api_key == '':
                        logger.info(f"⏭️ [API Key 验证] None 或空字符串，保留原值")
                        _req['api_key'] = ds_config.api_key or ""
                    # 🔥 如果包含 "..."（截断标记），需要验证是否是未修改的原值
                    elif api_key and "..." in api_key:
                        logger.info(f"🔍 [API Key 验证] 检测到截断标记，验证是否与数据库原值匹配")
                        # 对数据库中的完整 API Key 进行相同的截断处理
                        if ds_config.api_key:
                            truncated_db_key = _truncate_api_key(ds_config.api_key)
                            logger.info(f"🔍 [API Key 验证] 数据库原值截断后: {truncated_db_key}")
                            logger.info(f"🔍 [API Key 验证] 收到的值: {api_key}")
```

```
            # 比较截断后的值
            if api_key == truncated_db_key:
                # 相同，说明用户没有修改，保留数据库中的完整值
                logger.info(f"✅ [API Key 验证] 截断值匹配，保留数据库原值")
                _req['api_key'] = ds_config.api_key
            else:
                # 不同，说明用户修改了但修改得不完整
                logger.error(f"❌ [API Key 验证] 截断值不匹配，用户可能修改了不完整的密钥")
                raise HTTPException(
                    status_code=status.HTTP_400_BAD_REQUEST,
                    detail=f"API Key 格式错误：检测到截断标记但与数据库中的值不匹配，请输入完整的 API Key"
                )
        else:
            # 数据库中没有原值，但前端发送了截断值，这是不合理的
            logger.error(f"❌ [API Key 验证] 数据库中没有原值，但收到了截断值")
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"API Key 格式错误：请输入完整的 API Key"
            )
    # 如果是占位符，则不更新（保留原值）
    elif should_skip_api_key_update(api_key):
        logger.info(f"⏭️ [API Key 验证] 跳过更新（占位符），保留原值")
        _req['api_key'] = ds_config.api_key or ""
    # 如果是新输入的密钥，必须验证有效性
    elif not is_valid_api_key(api_key):
        logger.error(f"❌ [API Key 验证] 验证失败: '{api_key}' (长度: {len(api_key)})")
        logger.error(f"   - 长度检查: {len(api_key)} > 10? {len(api_key) > 10}")
        logger.error(f"   - 占位符前缀检查: startswith('your_')? {api_key.startswith('your_')}, startswith('your-')? {api_key.startswith('your-')}")
        logger.error(f"   - 占位符后缀检查: endswith('_here')? {api_key.endswith('_here')}, endswith('-here')? {api_key.endswith('-here')}")
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"API Key 无效：长度必须大于 10 个字符，且不能是占位符（当前长度: {len(api_key)}）"
        )
    else:
        logger.info(f"✅ [API Key 验证] 验证通过，将更新密钥 (长度: {len(api_key)})")
    # 有效的完整密钥，保留（表示更新）
# 处理 API Secret
if 'api_secret' in _req:
    api_secret = _req.get('api_secret')
    logger.info(f"🔍 [API Secret 验证] 收到的 API Secret: {repr(api_secret)} (类型: {type(api_secret).__name__}, 长度: {len(api_secret) if api_secret else 0})")
    # 如果是 None 或空字符串，保留原值（不更新）
    if api_secret is None or api_secret == '':
        logger.info(f"⏭️ [API Secret 验证] None 或空字符串，保留原值")
        _req['api_secret'] = ds_config.api_secret or ""
    # 🔒 如果包含 "..."（截断标记），需要验证是否是未修改的原值
    elif api_secret and "..." in api_secret:
        logger.info(f"🔍 [API Secret 验证] 检测到截断标记，验证是否与数据库原值匹配")
        # 对数据库中的完整 API Secret 进行相同的截断处理
        if ds_config.api_secret:
            truncated_db_secret = _truncate_api_key(ds_config.api_secret)
            logger.info(f"🔍 [API Secret 验证] 数据库原值截断后: {truncated_db_secret}")
            logger.info(f"🔍 [API Secret 验证] 收到的值: {api_secret}")
            # 比较截断后的值
            if api_secret == truncated_db_secret:
                # 相同，说明用户没有修改，保留数据库中的完整值
                logger.info(f"✅ [API Secret 验证] 截断值匹配，保留数据库原值")
                _req['api_secret'] = ds_config.api_secret
            else:
                # 不同，说明用户修改了但修改得不完整
                logger.error(f"❌ [API Secret 验证] 截断值不匹配，用户可能修改了不完整的密钥")
                raise HTTPException(
                    status_code=status.HTTP_400_BAD_REQUEST,
                    detail=f"API Secret 格式错误：检测到截断标记但与数据库中的值不匹配，请输入完整的 API Secret"
                )
        else:
            # 数据库中没有原值，但前端发送了截断值，这是不合理的
            logger.error(f"❌ [API Secret 验证] 数据库中没有原值，但收到了截断值")
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"API Secret 格式错误：请输入完整的 API Secret"
            )
    # 如果是占位符，则不更新（保留原值）
    elif should_skip_api_key_update(api_secret):
        logger.info(f"⏭️ [API Secret 验证] 跳过更新（占位符），保留原值")
        _req['api_secret'] = ds_config.api_secret or ""
    # 如果是新输入的密钥，必须验证有效性
    elif not is_valid_api_key(api_secret):
        logger.error(f"❌ [API Secret 验证] 验证失败: '{api_secret}' (长度: {len(api_secret)})")
        logger.error(f"   - 长度检查: {len(api_secret)} > 10? {len(api_secret) > 10}")
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"API Secret 无效：长度必须大于 10 个字符，且不能是占位符（当前长度: {len(api_secret)}）"
        )
    else:
        logger.info(f"✅ [API Secret 验证] 验证通过，将更新密钥 (长度: {len(api_secret)})")
updated_config = DataSourceConfig(**_req)
config.data_source_configs[i] = updated_config
success = await config_service.save_system_config(config)
if success:
    # 🔄 同步市场分类关系
    new_categories = set(_req.get('market_categories', []))
    # 获取当前的分组关系
    current_groupings = await config_service.get_datasource_groupings()
    current_categories = set(
        g.market_category_id
        for g in current_groupings
        if g.data_source_name == name
    )
    # 需要添加的分类
    to_add = new_categories - current_categories
    for category_id in to_add:
        try:
            grouping = DataSourceGrouping(
                data_source_name=name,
                market_category_id=category_id,
```

```
                    priority=updated_config.priority,
                    enabled=updated_config.enabled
                )
                await config_service.add_datasource_to_category(grouping)
            except Exception as e:
                logger.warning(f"添加数据源分组失败: {str(e)}")
        # 需要删除的分类
        to_remove = current_categories - new_categories
        for category_id in to_remove:
            try:
                await config_service.remove_datasource_from_category(name, category_id)
            except Exception as e:
                logger.warning(f"删除数据源分组失败: {str(e)}")
        # 审计日志（忽略异常）
        try:
            await log_operation(
                user_id=str(getattr(current_user, "id", "")),
                username=getattr(current_user, "username", "unknown"),
                action_type=ActionType.CONFIG_MANAGEMENT,
                action="update_data_source_config",
                details={"name": name, "market_categories": list(new_categories)},
                success=True,
            )
        except Exception:
            pass
        return {"message": "数据源配置更新成功"}
    else:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail="数据源配置更新失败"
        )
raise HTTPException(
    status_code=status.HTTP_404_NOT_FOUND,
    detail="数据源配置不存在"
)
except HTTPException:
    raise
except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"更新数据源配置失败: {str(e)}"
    )
@router.delete("/datasource/{name}", response_model=dict)
async def delete_data_source_config(
    name: str,
    current_user: User = Depends(get_current_user)
):
    """删除数据源配置"""
    try:
        # 获取当前配置
        config = await config_service.get_system_config()
        if not config:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="系统配置不存在"
            )
        # 查找并删除数据源配置
        for i, ds_config in enumerate(config.data_source_configs):
            if ds_config.name == name:
                config.data_source_configs.pop(i)
                success = await config_service.save_system_config(config)
                if success:
                    # 审计日志（忽略异常）
                    try:
                        await log_operation(
                            user_id=str(getattr(current_user, "id", "")),
                            username=getattr(current_user, "username", "unknown"),
                            action_type=ActionType.CONFIG_MANAGEMENT,
                            action="delete_data_source_config",
                            details={"name": name},
                            success=True,
                        )
                    except Exception:
                        pass
                    return {"message": "数据源配置删除成功"}
                else:
                    raise HTTPException(
                        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                        detail="数据源配置删除失败"
                    )
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="数据源配置不存在"
        )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"删除数据源配置失败: {str(e)}"
        )
# ==================== 市场分类管理 ====================
@router.get("/market-categories", response_model=List[MarketCategory])
async def get_market_categories(
    current_user: User = Depends(get_current_user)
):
    """获取所有市场分类"""
    try:
        categories = await config_service.get_market_categories()
        return categories
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取市场分类失败: {str(e)}"
```

```
        )
@router.post("/market-categories", response_model=dict)
async def add_market_category(
    request: MarketCategoryRequest,
    current_user: User = Depends(get_current_user)
):
    """添加市场分类"""
    try:
        category = MarketCategory(**request.model_dump())
        success = await config_service.add_market_category(category)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="add_market_category",
                    details={"id": str(getattr(category, 'id', ''))},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "市场分类添加成功", "id": category.id}
        else:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="市场分类ID已存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"添加市场分类失败: {str(e)}"
        )
@router.put("/market-categories/{category_id}", response_model=dict)
async def update_market_category(
    category_id: str,
    request: Dict[str, Any],
    current_user: User = Depends(get_current_user)
):
    """更新市场分类"""
    try:
        success = await config_service.update_market_category(category_id, request)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="update_market_category",
                    details={"category_id": category_id, "changed_keys": list(request.keys())},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "市场分类更新成功"}
        else:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="市场分类不存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"更新市场分类失败: {str(e)}"
        )
@router.delete("/market-categories/{category_id}", response_model=dict)
async def delete_market_category(
    category_id: str,
    current_user: User = Depends(get_current_user)
):
    """删除市场分类"""
    try:
        success = await config_service.delete_market_category(category_id)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="delete_market_category",
                    details={"category_id": category_id},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "市场分类删除成功"}
        else:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="无法删除分类，可能还有数据源使用此分类"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"删除市场分类失败: {str(e)}"
```

```
        )
# ==================== 数据源分组管理 ====================
@router.get("/datasource-groupings", response_model=List[DataSourceGrouping])
async def get_datasource_groupings(
    current_user: User = Depends(get_current_user)
):
    """获取所有数据源分组关系"""
    try:
        groupings = await config_service.get_datasource_groupings()
        return groupings
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取数据源分组关系失败: {str(e)}"
        )
@router.post("/datasource-groupings", response_model=dict)
async def add_datasource_to_category(
    request: DataSourceGroupingRequest,
    current_user: User = Depends(get_current_user)
):
    """将数据源添加到分类"""
    try:
        grouping = DataSourceGrouping(**request.model_dump())
        success = await config_service.add_datasource_to_category(grouping)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="add_datasource_to_category",
                    details={"data_source_name": request.data_source_name, "category_id": request.category_id},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "数据源添加到分类成功"}
        else:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="数据源已在该分类中"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"添加数据源到分类失败: {str(e)}"
        )
@router.delete("/datasource-groupings/{data_source_name}/{category_id}", response_model=dict)
async def remove_datasource_from_category(
    data_source_name: str,
    category_id: str,
    current_user: User = Depends(get_current_user)
):
    """从分类中移除数据源"""
    try:
        success = await config_service.remove_datasource_from_category(data_source_name, category_id)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="remove_datasource_from_category",
                    details={"data_source_name": data_source_name, "category_id": category_id},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "数据源从分类中移除成功"}
        else:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="数据源分组关系不存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"从分类中移除数据源失败: {str(e)}"
        )
@router.put("/datasource-groupings/{data_source_name}/{category_id}", response_model=dict)
async def update_datasource_grouping(
    data_source_name: str,
    category_id: str,
    request: Dict[str, Any],
    current_user: User = Depends(get_current_user)
):
    """更新数据源分组关系"""
    try:
        success = await config_service.update_datasource_grouping(data_source_name, category_id, request)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="update_datasource_grouping",
                    details={"data_source_name": data_source_name, "category_id": category_id, "changed_keys": list(request.keys())},
```

```
                success=True,
            )
        except Exception:
            pass
        return {"message": "数据源分组关系更新成功"}
    else:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="数据源分组关系不存在"
        )
except HTTPException:
    raise
except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"更新数据源分组关系失败: {str(e)}"
    )

@router.put("/market-categories/{category_id}/datasource-order", response_model=dict)
async def update_category_datasource_order(
    category_id: str,
    request: DataSourceOrderRequest,
    current_user: User = Depends(get_current_user)
):
    """更新分类中数据源的排序"""
    try:
        success = await config_service.update_category_datasource_order(category_id, request.data_sources)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="update_category_datasource_order",
                    details={"category_id": category_id, "data_sources": request.data_sources},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "数据源排序更新成功"}
        else:
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail="数据源排序更新失败"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"更新数据源排序失败: {str(e)}"
        )

@router.post("/datasource/set-default")
async def set_default_data_source(
    request: SetDefaultRequest,
    current_user: User = Depends(get_current_user)
):
    """设置默认数据源"""
    try:
        success = await config_service.set_default_data_source(request.name)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="set_default_datasource",
                    details={"name": request.name},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "默认数据源设置成功", "default_data_source": request.name}
        else:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="指定的数据源不存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"设置默认数据源失败: {str(e)}"
        )

@router.get("/settings", response_model=Dict[str, Any])
async def get_system_settings(
    current_user: User = Depends(get_current_user)
):
    """获取系统设置"""
    try:
        effective = await config_provider.get_effective_system_settings()
        return _sanitize_kv(effective)
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取系统设置失败: {str(e)}"
        )

@router.get("/settings/meta", response_model=dict)
async def get_system_settings_meta(
    current_user: User = Depends(get_current_user)
):
    """获取系统设置的元数据（敏感性、可编辑性、来源、是否有值）。
```

```
    返回结构：{success, data: {items: [{key,sensitive,editable,source,has_value}]}, message}
    """
    try:
        meta_map = await config_provider.get_system_settings_meta()
        items = [
            {"key": k, **v} for k, v in meta_map.items()
        ]
        return {"success": True, "data": {"items": items}, "message": ""}
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取系统设置元数据失败: {str(e)}"
        )
@router.put("/settings", response_model=dict)
async def update_system_settings(
    settings: Dict[str, Any],
    current_user: User = Depends(get_current_user)
):
    """更新系统设置"""
    try:
        # 打印接收到的设置（用于调试）
        logger.info(f"📝 接收到的系统设置更新请求，包含 {len(settings)} 项")
        if 'quick_analysis_model' in settings:
            logger.info(f"  ✓ quick_analysis_model: {settings['quick_analysis_model']}")
        else:
            logger.warning(f"  ⚠️  未包含 quick_analysis_model")
        if 'deep_analysis_model' in settings:
            logger.info(f"  ✓ deep_analysis_model: {settings['deep_analysis_model']}")
        else:
            logger.warning(f"  ⚠️  未包含 deep_analysis_model")
        success = await config_service.update_system_settings(settings)
        if success:
            # 审计日志（忽略日志异常，不影响主流程）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="update_system_settings",
                    details={"changed_keys": list(settings.keys())},
                    success=True,
                )
            except Exception:
                pass
            # 失效缓存
            try:
                config_provider.invalidate()
            except Exception:
                pass
            return {"message": "系统设置更新成功"}
        else:
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail="系统设置更新失败"
            )
    except HTTPException:
        raise
    except Exception as e:
        # 审计失败记录（忽略日志异常）
        try:
            await log_operation(
                user_id=str(getattr(current_user, "id", "")),
                username=getattr(current_user, "username", "unknown"),
                action_type=ActionType.CONFIG_MANAGEMENT,
                action="update_system_settings",
                details={"changed_keys": list(settings.keys())},
                success=False,
                error_message=str(e),
            )
        except Exception:
            pass
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"更新系统设置失败: {str(e)}"
        )
@router.post("/export", response_model=dict)
async def export_config(
    current_user: User = Depends(get_current_user)
):
    """导出配置"""
    try:
        config_data = await config_service.export_config()
        # 审计日志（忽略异常）
        try:
            await log_operation(
                user_id=str(getattr(current_user, "id", "")),
                username=getattr(current_user, "username", "unknown"),
                action_type=ActionType.DATA_EXPORT,
                action="export_config",
                details={"size": len(str(config_data))},
                success=True,
            )
        except Exception:
            pass
        return {
            "message": "配置导出成功",
            "data": config_data,
            "exported_at": now_tz().isoformat()
        }
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"导出配置失败: {str(e)}"
        )
@router.post("/import", response_model=dict)
```

```
async def import_config(
    config_data: Dict[str, Any],
    current_user: User = Depends(get_current_user)
):
    """导入配置"""
    try:
        success = await config_service.import_config(config_data)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.DATA_IMPORT,
                    action="import_config",
                    details={"keys": list(config_data.keys())[:10]},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "配置导入成功"}
        else:
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail="配置导入失败"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"导入配置失败: {str(e)}"
        )

@router.post("/migrate-legacy", response_model=dict)
async def migrate_legacy_config(
    current_user: User = Depends(get_current_user)
):
    """迁移传统配置"""
    try:
        success = await config_service.migrate_legacy_config()
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="migrate_legacy_config",
                    details={},
                    success=True,
                )
            except Exception:
                pass
            return {"message": "传统配置迁移成功"}
        else:
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail="传统配置迁移失败"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"迁移传统配置失败: {str(e)}"
        )

@router.post("/default/llm", response_model=dict)
async def set_default_llm(
    request: SetDefaultRequest,
    current_user: User = Depends(get_current_user)
):
    """设置默认大模型"""
    try:
        # 开源版本：所有用户都可以修改配置
        success = await config_service.set_default_llm(request.name)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="set_default_llm",
                    details={"name": request.name},
                    success=True,
                )
            except Exception:
                pass
            return {"message": f"默认大模型已设置为: {request.name}"}
        else:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="设置默认大模型失败，请检查模型名称是否正确"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"设置默认大模型失败: {str(e)}"
        )

@router.post("/default/datasource", response_model=dict)
async def set_default_data_source(
    request: SetDefaultRequest,
    current_user: User = Depends(get_current_user)
```

```
):
    """设置默认数据源"""
    try:
        # 开源版本：所有用户都可以修改配置
        success = await config_service.set_default_data_source(request.name)
        if success:
            # 审计日志（忽略异常）
            try:
                await log_operation(
                    user_id=str(getattr(current_user, "id", "")),
                    username=getattr(current_user, "username", "unknown"),
                    action_type=ActionType.CONFIG_MANAGEMENT,
                    action="set_default_datasource",
                    details={"name": request.name},
                    success=True,
                )
            except Exception:
                pass
            return {"message": f"默认数据源已设置为: {request.name}"}
        else:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="设置默认数据源失败，请检查数据源名称是否正确"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"设置默认数据源失败: {str(e)}"
        )

@router.get("/models", response_model=List[Dict[str, Any]])
async def get_available_models(
    current_user: User = Depends(get_current_user)
):
    """获取可用的模型列表"""
    try:
        models = await config_service.get_available_models()
        return models
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取模型列表失败: {str(e)}"
        )

# ========== 模型目录管理 ==========

@router.get("/model-catalog", response_model=List[Dict[str, Any]])
async def get_model_catalog(
    current_user: User = Depends(get_current_user)
):
    """获取所有模型目录"""
    try:
        catalogs = await config_service.get_model_catalog()
        return [catalog.model_dump(by_alias=False) for catalog in catalogs]
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取模型目录失败: {str(e)}"
        )

@router.get("/model-catalog/{provider}", response_model=Dict[str, Any])
async def get_provider_model_catalog(
    provider: str,
    current_user: User = Depends(get_current_user)
):
    """获取指定厂家的模型目录"""
    try:
        catalog = await config_service.get_provider_models(provider)
        if not catalog:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"未找到厂家 {provider} 的模型目录"
            )
        return catalog.model_dump(by_alias=False)
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取模型目录失败: {str(e)}"
        )

class ModelCatalogRequest(BaseModel):
    """模型目录请求"""
    provider: str
    provider_name: str
    models: List[Dict[str, Any]]

@router.post("/model-catalog", response_model=dict)
async def save_model_catalog(
    request: ModelCatalogRequest,
    current_user: User = Depends(get_current_user)
):
    """保存或更新模型目录"""
    try:
        logger.info(f"📝 收到保存模型目录请求: provider={request.provider}, models数量={len(request.models)}")
        logger.info(f"📝 请求数据: {request.model_dump()}")
        # 转换为 ModelInfo 列表
        models = [ModelInfo(**m) for m in request.models]
        logger.info(f"✅ 成功转换 {len(models)} 个模型")
        catalog = ModelCatalog(
            provider=request.provider,
            provider_name=request.provider_name,
            models=models
        )
        logger.info(f"✅ 创建 ModelCatalog 对象成功")
        success = await config_service.save_model_catalog(catalog)
```

```
        logger.info(f"💾 保存结果: {success}")
        if not success:
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail="保存模型目录失败"
            )
        # 记录操作日志
        await log_operation(
            user_id=str(current_user["id"]),
            username=current_user.get("username", "unknown"),
            action_type=ActionType.CONFIG_MANAGEMENT,
            action="update_model_catalog",
            details={"provider": request.provider, "provider_name": request.provider_name, "models_count": len(request.models)}
        )
        return {"success": True, "message": "模型目录保存成功"}
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 保存模型目录失败: {str(e)}", exc_info=True)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"保存模型目录失败: {str(e)}"
        )
@router.delete("/model-catalog/{provider}", response_model=dict)
async def delete_model_catalog(
    provider: str,
    current_user: User = Depends(get_current_user)
):
    """删除模型目录"""
    try:
        success = await config_service.delete_model_catalog(provider)
        if not success:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"未找到厂家 {provider} 的模型目录"
            )
        # 记录操作日志
        await log_operation(
            user_id=str(current_user["id"]),
            username=current_user.get("username", "unknown"),
            action_type=ActionType.CONFIG_MANAGEMENT,
            action="delete_model_catalog",
            details={"provider": provider}
        )
        return {"success": True, "message": "模型目录删除成功"}
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"删除模型目录失败: {str(e)}"
        )
@router.post("/model-catalog/init", response_model=dict)
async def init_model_catalog(
    current_user: User = Depends(get_current_user)
):
    """初始化默认模型目录"""
    try:
        success = await config_service.init_default_model_catalog()
        if not success:
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail="初始化模型目录失败"
            )
        return {"success": True, "message": "模型目录初始化成功"}
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"初始化模型目录失败: {str(e)}"
        )
# ===== 数据库配置管理端点 =====
@router.get("/database", response_model=List[DatabaseConfig])
async def get_database_configs(
    current_user: dict = Depends(get_current_user)
):
    """获取所有数据库配置"""
    try:
        logger.info("🗄️ 获取数据库配置列表...")
        configs = await config_service.get_database_configs()
        logger.info(f"✅ 获取到 {len(configs)} 个数据库配置")
        return configs
    except Exception as e:
        logger.error(f"❌ 获取数据库配置失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取数据库配置失败: {str(e)}"
        )
@router.get("/database/{db_name}", response_model=DatabaseConfig)
async def get_database_config(
    db_name: str,
    current_user: dict = Depends(get_current_user)
):
    """获取指定的数据库配置"""
    try:
        logger.info(f"🗄️ 获取数据库配置: {db_name}")
        config = await config_service.get_database_config(db_name)
        if not config:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"数据库配置 '{db_name}' 不存在"
            )
```

```
        return config
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 获取数据库配置失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取数据库配置失败: {str(e)}"
        )
@router.post("/database", response_model=dict)
async def add_database_config(
    request: DatabaseConfigRequest,
    current_user: dict = Depends(get_current_user)
):
    """添加数据库配置"""
    try:
        logger.info(f"➕ 添加数据库配置: {request.name}")
        # 转换为 DatabaseConfig 对象
        db_config = DatabaseConfig(**request.model_dump())
        # 添加配置
        success = await config_service.add_database_config(db_config)
        if not success:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="添加数据库配置失败，可能已存在同名配置"
            )
        # 记录操作日志
        await log_operation(
            user_id=current_user["id"],
            username=current_user.get("username", "unknown"),
            action_type=ActionType.CONFIG_MANAGEMENT,
            action=f"添加数据库配置: {request.name}",
            details={"name": request.name, "type": request.type, "host": request.host, "port": request.port}
        )
        return {"success": True, "message": "数据库配置添加成功"}
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 添加数据库配置失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"添加数据库配置失败: {str(e)}"
        )
@router.put("/database/{db_name}", response_model=dict)
async def update_database_config(
    db_name: str,
    request: DatabaseConfigRequest,
    current_user: dict = Depends(get_current_user)
):
    """更新数据库配置"""
    try:
        logger.info(f"🔄 更新数据库配置: {db_name}")
        # 检查名称是否匹配
        if db_name != request.name:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="URL中的名称与请求体中的名称不匹配"
            )
        # 转换为 DatabaseConfig 对象
        db_config = DatabaseConfig(**request.model_dump())
        # 更新配置
        success = await config_service.update_database_config(db_config)
        if not success:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"数据库配置 '{db_name}' 不存在"
            )
        # 记录操作日志
        await log_operation(
            user_id=current_user["id"],
            username=current_user.get("username", "unknown"),
            action_type=ActionType.CONFIG_MANAGEMENT,
            action=f"更新数据库配置: {db_name}",
            details={"name": request.name, "type": request.type, "host": request.host, "port": request.port}
        )
        return {"success": True, "message": "数据库配置更新成功"}
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 更新数据库配置失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"更新数据库配置失败: {str(e)}"
        )
@router.delete("/database/{db_name}", response_model=dict)
async def delete_database_config(
    db_name: str,
    current_user: dict = Depends(get_current_user)
):
    """删除数据库配置"""
    try:
        logger.info(f"🗑️ 删除数据库配置: {db_name}")
        # 删除配置
        success = await config_service.delete_database_config(db_name)
        if not success:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"数据库配置 '{db_name}' 不存在"
            )
        # 记录操作日志
        await log_operation(
            user_id=current_user["id"],
```

```
        username=current_user.get("username", "unknown"),
        action_type=ActionType.CONFIG_MANAGEMENT,
        action=f"删除数据库配置: {db_name}",
        details={"name": db_name}
    )
    return {"success": True, "message": "数据库配置删除成功"}
except HTTPException:
    raise
except Exception as e:
    logger.error(f"❌ 删除数据库配置失败: {e}")
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"删除数据库配置失败: {str(e)}"
    )
```

FILE: app/routers/database.py

```
"""
数据库管理API路由
"""
import logging
import json
import os
from datetime import datetime
from typing import Dict, Any, List
from fastapi import APIRouter, Depends, HTTPException, status, UploadFile, File
from fastapi.responses import FileResponse
from pydantic import BaseModel
from app.routers.auth_db import get_current_user
from app.core.database import get_mongo_db, get_redis_client
from app.services.database_service import DatabaseService
router = APIRouter(prefix="/database", tags=["数据库管理"])
logger = logging.getLogger("webapi")
# 请求模型
class BackupRequest(BaseModel):
    """备份请求"""
    name: str
    collections: List[str] = []  # 空列表表示备份所有集合
class ImportRequest(BaseModel):
    """导入请求"""
    collection: str
    format: str = "json"  # json, csv
    overwrite: bool = False
class ExportRequest(BaseModel):
    """导出请求"""
    collections: List[str] = []  # 空列表表示导出所有集合
    format: str = "json"  # json, csv
    sanitize: bool = False  # 是否脱敏（清空敏感字段，用于演示系统）
# 响应模型
class DatabaseStatusResponse(BaseModel):
    """数据库状态响应"""
    mongodb: Dict[str, Any]
    redis: Dict[str, Any]
class DatabaseStatsResponse(BaseModel):
    """数据库统计响应"""
    total_collections: int
    total_documents: int
    total_size: int
    collections: List[Dict[str, Any]]
class BackupResponse(BaseModel):
    """备份响应"""
    id: str
    name: str
    size: int
    created_at: str
    collections: List[str]
# 数据库服务实例
database_service = DatabaseService()
@router.get("/status")
async def get_database_status(
    current_user: dict = Depends(get_current_user)
):
    """获取数据库连接状态"""
    try:
        logger.info(f"🔍 用户 {current_user['username']} 请求数据库状态")
        status_info = await database_service.get_database_status()
        return {
            "success": True,
            "message": "获取数据库状态成功",
            "data": status_info
        }
    except Exception as e:
        logger.error(f"获取数据库状态失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取数据库状态失败: {str(e)}"
        )
@router.get("/stats")
async def get_database_stats(
    current_user: dict = Depends(get_current_user)
):
    """获取数据库统计信息"""
    try:
        logger.info(f"📊 用户 {current_user['username']} 请求数据库统计")
        stats = await database_service.get_database_stats()
        return {
            "success": True,
            "message": "获取数据库统计成功",
            "data": stats
        }
    except Exception as e:
        logger.error(f"获取数据库统计失败: {e}")
```

```
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取数据库统计失败: {str(e)}"
        )
@router.post("/test")
async def test_database_connections(
    current_user: dict = Depends(get_current_user)
):
    """测试数据库连接"""
    try:
        logger.info(f"🔧 用户 {current_user['username']} 测试数据库连接")
        results = await database_service.test_connections()
        return {
            "success": True,
            "message": "数据库连接测试完成",
            "data": results
        }
    except Exception as e:
        logger.error(f"测试数据库连接失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"测试数据库连接失败: {str(e)}"
        )
@router.post("/backup")
async def create_backup(
    request: BackupRequest,
    current_user: dict = Depends(get_current_user)
):
    """创建数据库备份"""
    try:
        logger.info(f"💾 用户 {current_user['username']} 创建备份: {request.name}")
        backup_info = await database_service.create_backup(
            name=request.name,
            collections=request.collections,
            user_id=current_user['id']
        )
        return {
            "success": True,
            "message": "备份创建成功",
            "data": backup_info
        }
    except Exception as e:
        logger.error(f"创建备份失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"创建备份失败: {str(e)}"
        )
@router.get("/backups")
async def list_backups(
    current_user: dict = Depends(get_current_user)
):
    """获取备份列表"""
    try:
        logger.info(f"📋 用户 {current_user['username']} 获取备份列表")
        backups = await database_service.list_backups()
        return {
            "success": True,
            "data": backups
        }
    except Exception as e:
        logger.error(f"获取备份列表失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取备份列表失败: {str(e)}"
        )
@router.post("/import")
async def import_data(
    file: UploadFile = File(...),
    collection: str = "imported_data",
    format: str = "json",
    overwrite: bool = False,
    current_user: dict = Depends(get_current_user)
):
    """导入数据"""
    try:
        logger.info(f"📥 用户 {current_user['username']} 导入数据到集合: {collection}")
        logger.info(f"   文件名: {file.filename}")
        logger.info(f"   格式: {format}")
        logger.info(f"   覆盖模式: {overwrite}")
        # 读取文件内容
        content = await file.read()
        logger.info(f"   文件大小: {len(content)} 字节")
        result = await database_service.import_data(
            content=content,
            collection=collection,
            format=format,
            overwrite=overwrite,
            filename=file.filename
        )
        logger.info(f"✅ 导入成功: {result}")
        return {
            "success": True,
            "message": "数据导入成功",
            "data": result
        }
    except Exception as e:
        logger.error(f"❌ 导入数据失败: {e}")
        import traceback
        logger.error(traceback.format_exc())
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"导入数据失败: {str(e)}"
        )
@router.post("/export")
```

```
async def export_data(
    request: ExportRequest,
    current_user: dict = Depends(get_current_user)
):
    """导出数据"""
    try:
        sanitize_info = "（脱敏模式）" if request.sanitize else ""
        logger.info(f"📤 用户 {current_user['username']} 导出数据{sanitize_info}")
        file_path = await database_service.export_data(
            collections=request.collections,
            format=request.format,
            sanitize=request.sanitize
        )
        return FileResponse(
            path=file_path,
            filename=os.path.basename(file_path),
            media_type='application/octet-stream'
        )
    except Exception as e:
        logger.error(f"导出数据失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"导出数据失败: {str(e)}"
        )
@router.delete("/backups/{backup_id}")
async def delete_backup(
    backup_id: str,
    current_user: dict = Depends(get_current_user)
):
    """删除备份"""
    try:
        logger.info(f"🗑️ 用户 {current_user['username']} 删除备份: {backup_id}")
        await database_service.delete_backup(backup_id)
        return {
            "success": True,
            "message": "备份删除成功"
        }
    except Exception as e:
        logger.error(f"删除备份失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"删除备份失败: {str(e)}"
        )
@router.post("/cleanup")
async def cleanup_old_data(
    days: int = 30,
    current_user: dict = Depends(get_current_user)
):
    """清理旧数据"""
    try:
        logger.info(f"🧹 用户 {current_user['username']} 清理 {days} 天前的数据")
        result = await database_service.cleanup_old_data(days)
        return {
            "success": True,
            "message": f"清理完成，删除了 {result['deleted_count']} 条记录",
            "data": result
        }
    except Exception as e:
        logger.error(f"清理数据失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"清理数据失败: {str(e)}"
        )
@router.post("/cleanup/analysis")
async def cleanup_analysis_results(
    days: int = 30,
    current_user: dict = Depends(get_current_user)
):
    """清理过期分析结果"""
    try:
        logger.info(f"🧹 用户 {current_user['username']} 清理 {days} 天前的分析结果")
        result = await database_service.cleanup_analysis_results(days)
        return {
            "success": True,
            "message": f"分析结果清理完成，删除了 {result['deleted_count']} 条记录",
            "data": result
        }
    except Exception as e:
        logger.error(f"清理分析结果失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"清理分析结果失败: {str(e)}"
        )
@router.post("/cleanup/logs")
async def cleanup_operation_logs(
    days: int = 90,
    current_user: dict = Depends(get_current_user)
):
    """清理操作日志"""
    try:
        logger.info(f"🧹 用户 {current_user['username']} 清理 {days} 天前的操作日志")
        result = await database_service.cleanup_operation_logs(days)
        return {
            "success": True,
            "message": f"操作日志清理完成，删除了 {result['deleted_count']} 条记录",
            "data": result
        }
    except Exception as e:
        logger.error(f"清理操作日志失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"清理操作日志失败: {str(e)}"
        )
```

FILE: app/routers/favorites.py

```
"""
自选股管理API路由
"""
from typing import List, Optional
from fastapi import APIRouter, Depends, HTTPException, status
from pydantic import BaseModel
import logging
from app.routers.auth_db import get_current_user
from app.models.user import User, FavoriteStock
from app.services.favorites_service import favorites_service
from app.core.response import ok
logger = logging.getLogger("webapi")
router = APIRouter(prefix="/favorites", tags=["自选股管理"])
class AddFavoriteRequest(BaseModel):
    """添加自选股请求"""
    stock_code: str
    stock_name: str
    market: str = "A股"
    tags: List[str] = []
    notes: str = ""
    alert_price_high: Optional[float] = None
    alert_price_low: Optional[float] = None
class UpdateFavoriteRequest(BaseModel):
    """更新自选股请求"""
    tags: Optional[List[str]] = None
    notes: Optional[str] = None
    alert_price_high: Optional[float] = None
    alert_price_low: Optional[float] = None
class FavoriteStockResponse(BaseModel):
    """自选股响应"""
    stock_code: str
    stock_name: str
    market: str
    added_at: str
    tags: List[str]
    notes: str
    alert_price_high: Optional[float]
    alert_price_low: Optional[float]
    # 实时数据
    current_price: Optional[float] = None
    change_percent: Optional[float] = None
    volume: Optional[int] = None
@router.get("/", response_model=dict)
async def get_favorites(
    current_user: dict = Depends(get_current_user)
):
    """获取用户自选股列表"""
    try:
        favorites = await favorites_service.get_user_favorites(current_user["id"])
        return ok(favorites)
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取自选股失败: {str(e)}"
        )
@router.post("/", response_model=dict)
async def add_favorite(
    request: AddFavoriteRequest,
    current_user: dict = Depends(get_current_user)
):
    """添加股票到自选股"""
    import logging
    logger = logging.getLogger("webapi")
    try:
        logger.info(f"📝 添加自选股请求: user_id={current_user['id']}, stock_code={request.stock_code}, stock_name={request.stock_name}")
        # 检查是否已存在
        is_fav = await favorites_service.is_favorite(current_user["id"], request.stock_code)
        logger.info(f"🔍 检查是否已存在: {is_fav}")
        if is_fav:
            logger.warning(f"⚠️ 股票已在自选股中: {request.stock_code}")
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="该股票已在自选股中"
            )
        # 添加到自选股
        logger.info(f"➕ 开始添加自选股...")
        success = await favorites_service.add_favorite(
            user_id=current_user["id"],
            stock_code=request.stock_code,
            stock_name=request.stock_name,
            market=request.market,
            tags=request.tags,
            notes=request.notes,
            alert_price_high=request.alert_price_high,
            alert_price_low=request.alert_price_low
        )
        logger.info(f"✅ 添加结果: success={success}")
        if success:
            return ok({"stock_code": request.stock_code}, "添加成功")
        else:
            logger.error(f"❌ 添加失败: success=False")
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail="添加失败"
            )
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 添加自选股异常: {type(e).__name__}: {str(e)}", exc_info=True)
        raise HTTPException(
```

```
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"添加自选股失败: {str(e)}"
        )

@router.put("/{stock_code}", response_model=dict)
async def update_favorite(
    stock_code: str,
    request: UpdateFavoriteRequest,
    current_user: dict = Depends(get_current_user)
):
    """更新自选股信息"""
    try:
        success = await favorites_service.update_favorite(
            user_id=current_user["id"],
            stock_code=stock_code,
            tags=request.tags,
            notes=request.notes,
            alert_price_high=request.alert_price_high,
            alert_price_low=request.alert_price_low
        )
        if success:
            return ok({"stock_code": stock_code}, "更新成功")
        else:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="自选股不存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"更新自选股失败: {str(e)}"
        )

@router.delete("/{stock_code}", response_model=dict)
async def remove_favorite(
    stock_code: str,
    current_user: dict = Depends(get_current_user)
):
    """从自选股中移除股票"""
    try:
        success = await favorites_service.remove_favorite(current_user["id"], stock_code)
        if success:
            return ok({"stock_code": stock_code}, "移除成功")
        else:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="自选股不存在"
            )
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"移除自选股失败: {str(e)}"
        )

@router.get("/check/{stock_code}", response_model=dict)
async def check_favorite(
    stock_code: str,
    current_user: dict = Depends(get_current_user)
):
    """检查股票是否在自选股中"""
    try:
        is_favorite = await favorites_service.is_favorite(current_user["id"], stock_code)
        return ok({"stock_code": stock_code, "is_favorite": is_favorite})
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"检查自选股状态失败: {str(e)}"
        )

@router.get("/tags", response_model=dict)
async def get_user_tags(
    current_user: dict = Depends(get_current_user)
):
    """获取用户使用的所有标签"""
    try:
        tags = await favorites_service.get_user_tags(current_user["id"])
        return ok(tags)
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取标签失败: {str(e)}"
        )

class SyncFavoritesRequest(BaseModel):
    """同步自选股实时行情请求"""
    data_source: str = "tushare"  # tushare/akshare

@router.post("/sync-realtime", response_model=dict)
async def sync_favorites_realtime(
    request: SyncFavoritesRequest,
    current_user: dict = Depends(get_current_user)
):
    """
    同步自选股实时行情
    - **data_source**: 数据源（tushare/akshare）
    """
    try:
        logger.info(f"📊 开始同步自选股实时行情: user_id={current_user['id']}, data_source={request.data_source}")
        # 获取用户自选股列表
        favorites = await favorites_service.get_user_favorites(current_user["id"])
        if not favorites:
            logger.info("⚠️ 用户没有自选股")
            return ok({
                "total": 0,
                "success_count": 0,
```

```
            "failed_count": 0,
            "message": "没有自选股需要同步"
        })
    # 提取股票代码列表
    symbols = [fav.get("stock_code") or fav.get("symbol") for fav in favorites]
    symbols = [s for s in symbols if s]  # 过滤空值
    logger.info(f"🎯 需要同步的股票: {len(symbols)} 只 - {symbols}")
    # 根据数据源选择同步服务
    if request.data_source == "tushare":
        from app.worker.tushare_sync_service import get_tushare_sync_service
        service = await get_tushare_sync_service()
    elif request.data_source == "akshare":
        from app.worker.akshare_sync_service import get_akshare_sync_service
        service = await get_akshare_sync_service()
    else:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"不支持的数据源: {request.data_source}"
        )
    if not service:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail=f"{request.data_source} 服务不可用"
        )
    # 同步实时行情
    logger.info(f"🔄 调用 {request.data_source} 同步服务...")
    sync_result = await service.sync_realtime_quotes(
        symbols=symbols,
        force=True  # 强制执行，跳过交易时间检查
    )
    success_count = sync_result.get("success_count", 0)
    failed_count = sync_result.get("failed_count", 0)
    logger.info(f"✅ 自选股实时行情同步完成: 成功 {success_count}/{len(symbols)} 只")
    return ok({
        "total": len(symbols),
        "success_count": success_count,
        "failed_count": failed_count,
        "symbols": symbols,
        "data_source": request.data_source,
        "message": f"同步完成: 成功 {success_count} 只，失败 {failed_count} 只"
    })
except HTTPException:
    raise
except Exception as e:
    logger.error(f"❌ 同步自选股实时行情失败: {e}", exc_info=True)
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"同步失败: {str(e)}"
    )
```

FILE: app/routers/financial_data.py

```
#!/usr/bin/env python3
"""
财务数据API路由
提供财务数据查询和同步管理接口
"""
import logging
from typing import Dict, Any, List, Optional
from fastapi import APIRouter, HTTPException, Query, BackgroundTasks
from pydantic import BaseModel, Field
from app.worker.financial_data_sync_service import get_financial_sync_service
from app.services.financial_data_service import get_financial_data_service
from app.core.response import ok
logger = logging.getLogger(__name__)
router = APIRouter(prefix="/api/financial-data", tags=["财务数据"])
# ==================== 请求模型 ====================
class FinancialSyncRequest(BaseModel):
    """财务数据同步请求"""
    symbols: Optional[List[str]] = Field(None, description="股票代码列表，为空则同步所有股票")
    data_sources: Optional[List[str]] = Field(
        ["tushare", "akshare", "baostock"],
        description="数据源列表"
    )
    report_types: Optional[List[str]] = Field(
        ["quarterly"],
        description="报告类型列表 (quarterly/annual)"
    )
    batch_size: int = Field(50, description="批处理大小", ge=1, le=200)
    delay_seconds: float = Field(1.0, description="API调用延迟秒数", ge=0.1, le=10.0)
class SingleStockSyncRequest(BaseModel):
    """单股票财务数据同步请求"""
    symbol: str = Field(..., description="股票代码")
    data_sources: Optional[List[str]] = Field(
        ["tushare", "akshare", "baostock"],
        description="数据源列表"
    )
# ==================== API端点 ====================
@router.get("/query/{symbol}", summary="查询股票财务数据")
async def query_financial_data(
    symbol: str,
    report_period: Optional[str] = Query(None, description="报告期筛选 (YYYYMMDD)"),
    data_source: Optional[str] = Query(None, description="数据源筛选"),
    report_type: Optional[str] = Query(None, description="报告类型筛选"),
    limit: Optional[int] = Query(10, description="限制返回数量", ge=1, le=100)
) -> dict:
    """
    查询股票财务数据

    - **symbol**: 股票代码 (必填)
    - **report_period**: 报告期筛选，格式YYYYMMDD
```

```
    - **data_source**: 数据源筛选 (tushare/akshare/baostock)
    - **report_type**: 报告类型筛选 (quarterly/annual)
    - **limit**: 限制返回数量，默认10条
    """
    try:
        service = await get_financial_data_service()

        results = await service.get_financial_data(
            symbol=symbol,
            report_period=report_period,
            data_source=data_source,
            report_type=report_type,
            limit=limit
        )

        return ok(data={
                "symbol": symbol,
                "count": len(results),
                "financial_data": results
            },
            message=f"查询到 {len(results)} 条财务数据"
        )

    except Exception as e:
        logger.error(f"❌ 查询财务数据失败 {symbol}: {e}")
        raise HTTPException(status_code=500, detail=f"查询财务数据失败: {str(e)}")
@router.get("/latest/{symbol}", summary="获取最新财务数据")
async def get_latest_financial_data(
    symbol: str,
    data_source: Optional[str] = Query(None, description="数据源筛选")
) -> dict:
    """
    获取股票最新财务数据

    - **symbol**: 股票代码 (必填)
    - **data_source**: 数据源筛选 (tushare/akshare/baostock)
    """
    try:
        service = await get_financial_data_service()

        result = await service.get_latest_financial_data(
            symbol=symbol,
            data_source=data_source
        )

        if result:
            return ok(data=result,
                message="获取最新财务数据成功"
            )
        else:
            return ok(success=False, data=None,
                message="未找到财务数据"
            )

    except Exception as e:
        logger.error(f"❌ 获取最新财务数据失败 {symbol}: {e}")
        raise HTTPException(status_code=500, detail=f"获取最新财务数据失败: {str(e)}")
@router.get("/statistics", summary="获取财务数据统计")
async def get_financial_statistics() -> dict:
    """
    获取财务数据统计信息

    返回各数据源的财务数据统计，包括：
    - 总记录数
    - 总股票数
    - 按数据源和报告类型分组的统计
    """
    try:
        service = await get_financial_data_service()

        stats = await service.get_financial_statistics()

        return ok(data=stats,
            message="获取财务数据统计成功"
        )

    except Exception as e:
        logger.error(f"❌ 获取财务数据统计失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取财务数据统计失败: {str(e)}")
@router.post("/sync/start", summary="启动财务数据同步")
async def start_financial_sync(
    request: FinancialSyncRequest,
    background_tasks: BackgroundTasks
) -> dict:
    """
    启动财务数据同步任务

    支持配置：
    - 股票代码列表（为空则同步所有股票）
    - 数据源选择
    - 报告类型选择
    - 批处理大小和延迟设置
    """
    try:
        service = await get_financial_sync_service()

        # 在后台执行同步任务
        background_tasks.add_task(
            _execute_financial_sync,
            service,
            request
        )

        return ok(data={
                "task_started": True,
```

```
            "config": request.dict()
        },
        message="财务数据同步任务已启动"
    )

    except Exception as e:
        logger.error(f"❌ 启动财务数据同步失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动财务数据同步失败: {str(e)}")
@router.post("/sync/single", summary="同步单只股票财务数据")
async def sync_single_stock_financial(
    request: SingleStockSyncRequest
) -> dict:
    """
    同步单只股票的财务数据

    - **symbol**: 股票代码 (必填)
    - **data_sources**: 数据源列表，默认使用所有数据源
    """
    try:
        service = await get_financial_sync_service()

        results = await service.sync_single_stock(
            symbol=request.symbol,
            data_sources=request.data_sources
        )

        success_count = sum(1 for success in results.values() if success)
        total_count = len(results)

        return ok(
            success=success_count > 0,
            data={
                "symbol": request.symbol,
                "results": results,
                "success_count": success_count,
                "total_count": total_count
            },
            message=f"单股票财务数据同步完成: {success_count}/{total_count} 成功"
        )

    except Exception as e:
        logger.error(f"❌ 单股票财务数据同步失败 {request.symbol}: {e}")
        raise HTTPException(status_code=500, detail=f"单股票财务数据同步失败: {str(e)}")
@router.get("/sync/statistics", summary="获取同步统计信息")
async def get_sync_statistics() -> dict:
    """
    获取财务数据同步统计信息

    返回各数据源的同步统计，包括记录数、股票数等
    """
    try:
        service = await get_financial_sync_service()

        stats = await service.get_sync_statistics()

        return ok(data=stats,
            message="获取同步统计信息成功"
        )

    except Exception as e:
        logger.error(f"❌ 获取同步统计信息失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取同步统计信息失败: {str(e)}")
@router.get("/health", summary="财务数据服务健康检查")
async def health_check() -> dict:
    """
    财务数据服务健康检查

    检查服务状态和数据库连接
    """
    try:
        # 检查服务初始化状态
        service = await get_financial_data_service()
        sync_service = await get_financial_sync_service()

        # 简单的数据库连接测试
        stats = await service.get_financial_statistics()

        return ok(data={
            "service_status": "healthy",
            "database_connected": True,
            "total_records": stats.get("total_records", 0),
            "total_symbols": stats.get("total_symbols", 0)
        },
        message="财务数据服务运行正常"
        )

    except Exception as e:
        logger.error(f"❌ 财务数据服务健康检查失败: {e}")
        return ok(success=False, data={
            "service_status": "unhealthy",
            "error": str(e)
        },
        message="财务数据服务异常"
        )
# ==================== 后台任务 ====================
async def _execute_financial_sync(
    service: Any,
    request: FinancialSyncRequest
):
    """执行财务数据同步后台任务"""
    try:
        logger.info(f"🚀 开始执行财务数据同步任务: {request.dict()}")

        results = await service.sync_financial_data(
```

```
        symbols=request.symbols,
        data_sources=request.data_sources,
        report_types=request.report_types,
        batch_size=request.batch_size,
        delay_seconds=request.delay_seconds
    )

    # 统计总体结果
    total_success = sum(stats.success_count for stats in results.values())
    total_symbols = sum(stats.total_symbols for stats in results.values())

    logger.info(f"✅ 财务数据同步任务完成: {total_success}/{total_symbols} 成功")

    # 这里可以添加通知逻辑，比如发送邮件或消息

  except Exception as e:
    logger.error(f"❌ 财务数据同步任务执行失败: {e}")
# 导入datetime用于时间戳
from datetime import datetime
```

FILE: app/routers/health.py

```
from fastapi import APIRouter
import time
from pathlib import Path
router = APIRouter()
def get_version() -> str:
  """从 VERSION 文件读取版本号"""
  try:
    version_file = Path(__file__).parent.parent.parent / "VERSION"
    if version_file.exists():
      return version_file.read_text(encoding='utf-8').strip()
  except Exception:
    pass
  return "0.1.16"  # 默认版本号
@router.get("/health")
async def health():
  """健康检查接口 - 前端使用"""
  return {
    "success": True,
    "data": {
      "status": "ok",
      "version": get_version(),
      "timestamp": int(time.time()),
      "service": "TradingAgents-CN API"
    },
    "message": "服务运行正常"
  }
@router.get("/healthz")
async def healthz():
  """Kubernetes健康检查"""
  return {"status": "ok"}
@router.get("/readyz")
async def readyz():
  """Kubernetes就绪检查"""
  return {"ready": True}
```

FILE: app/routers/historical_data.py

```
#!/usr/bin/env python3
"""
历史数据查询API
提供统一的历史K线数据查询接口
"""
import logging
from datetime import datetime, date
from typing import Dict, Any, List, Optional
from fastapi import APIRouter, HTTPException, Query
from pydantic import BaseModel, Field
from app.services.historical_data_service import get_historical_data_service
logger = logging.getLogger(__name__)
router = APIRouter(prefix="/api/historical-data", tags=["历史数据"])
class HistoricalDataQuery(BaseModel):
  """历史数据查询请求"""
  symbol: str = Field(..., description="股票代码")
  start_date: Optional[str] = Field(None, description="开始日期 (YYYY-MM-DD)")
  end_date: Optional[str] = Field(None, description="结束日期 (YYYY-MM-DD)")
  data_source: Optional[str] = Field(None, description="数据源 (tushare/akshare/baostock)")
  period: Optional[str] = Field(None, description="数据周期 (daily/weekly/monthly)")
  limit: Optional[int] = Field(None, ge=1, le=1000, description="限制返回数量")
class HistoricalDataResponse(BaseModel):
  """历史数据响应"""
  success: bool
  message: str
  data: Optional[Dict[str, Any]] = None
@router.get("/query/{symbol}", response_model=HistoricalDataResponse)
async def get_historical_data(
  symbol: str,
  start_date: Optional[str] = Query(None, description="开始日期 (YYYY-MM-DD)"),
  end_date: Optional[str] = Query(None, description="结束日期 (YYYY-MM-DD)"),
  data_source: Optional[str] = Query(None, description="数据源 (tushare/akshare/baostock)"),
  period: Optional[str] = Query(None, description="数据周期 (daily/weekly/monthly)"),
  limit: Optional[int] = Query(None, ge=1, le=1000, description="限制返回数量")
):
  """
  查询股票历史数据

  Args:
    symbol: 股票代码
    start_date: 开始日期
```

```
        end_date: 结束日期
        data_source: 数据源筛选
        period: 数据周期筛选
        limit: 限制返回数量
    """
    try:
        service = await get_historical_data_service()

        # 查询历史数据
        results = await service.get_historical_data(
            symbol=symbol,
            start_date=start_date,
            end_date=end_date,
            data_source=data_source,
            period=period,
            limit=limit
        )

        # 格式化响应
        response_data = {
            "symbol": symbol,
            "count": len(results),
            "query_params": {
                "start_date": start_date,
                "end_date": end_date,
                "data_source": data_source,
                "period": period,
                "limit": limit
            },
            "records": results
        }

        return HistoricalDataResponse(
            success=True,
            message=f"查询成功，返回 {len(results)} 条记录",
            data=response_data
        )

    except Exception as e:
        logger.error(f"查询历史数据失败 {symbol}: {e}")
        raise HTTPException(status_code=500, detail=f"查询失败: {e}")

@router.post("/query", response_model=HistoricalDataResponse)
async def query_historical_data(request: HistoricalDataQuery):
    """
    POST方式查询历史数据
    """
    try:
        service = await get_historical_data_service()

        # 查询历史数据
        results = await service.get_historical_data(
            symbol=request.symbol,
            start_date=request.start_date,
            end_date=request.end_date,
            data_source=request.data_source,
            period=request.period,
            limit=request.limit
        )

        # 格式化响应
        response_data = {
            "symbol": request.symbol,
            "count": len(results),
            "query_params": request.dict(),
            "records": results
        }

        return HistoricalDataResponse(
            success=True,
            message=f"查询成功，返回 {len(results)} 条记录",
            data=response_data
        )

    except Exception as e:
        logger.error(f"查询历史数据失败 {request.symbol}: {e}")
        raise HTTPException(status_code=500, detail=f"查询失败: {e}")

@router.get("/latest-date/{symbol}")
async def get_latest_date(
    symbol: str,
    data_source: str = Query(..., description="数据源 (tushare/akshare/baostock)")
):
    """获取股票最新数据日期"""
    try:
        service = await get_historical_data_service()
        latest_date = await service.get_latest_date(symbol, data_source)

        return {
            "success": True,
            "data": {
                "symbol": symbol,
                "data_source": data_source,
                "latest_date": latest_date
            },
            "message": "查询成功"
        }

    except Exception as e:
        logger.error(f"获取最新日期失败 {symbol}: {e}")
        raise HTTPException(status_code=500, detail=f"查询失败: {e}")

@router.get("/statistics")
async def get_data_statistics():
    """获取历史数据统计信息"""
    try:
        service = await get_historical_data_service()
        stats = await service.get_data_statistics()
```

```
        return {
            "success": True,
            "data": stats,
            "message": "统计信息获取成功"
        }

    except Exception as e:
        logger.error(f"获取统计信息失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取统计信息失败: {e}")
@router.get("/compare/{symbol}")
async def compare_data_sources(
    symbol: str,
    trade_date: str = Query(..., description="交易日期 (YYYY-MM-DD)")
):
    """
    对比不同数据源的同一股票同一日期的数据
    """
    try:
        service = await get_historical_data_service()

        # 查询三个数据源的数据
        sources = ["tushare", "akshare", "baostock"]
        comparison = {}

        for source in sources:
            results = await service.get_historical_data(
                symbol=symbol,
                start_date=trade_date,
                end_date=trade_date,
                data_source=source,
                limit=1
            )

            if results:
                comparison[source] = results[0]
            else:
                comparison[source] = None

        return {
            "success": True,
            "data": {
                "symbol": symbol,
                "trade_date": trade_date,
                "comparison": comparison,
                "available_sources": [k for k, v in comparison.items() if v is not None]
            },
            "message": "数据对比完成"
        }

    except Exception as e:
        logger.error(f"数据对比失败 {symbol}: {e}")
        raise HTTPException(status_code=500, detail=f"数据对比失败: {e}")
@router.get("/health")
async def health_check():
    """健康检查"""
    try:
        service = await get_historical_data_service()
        stats = await service.get_data_statistics()

        return {
            "success": True,
            "data": {
                "service": "历史数据服务",
                "status": "healthy",
                "total_records": stats.get("total_records", 0),
                "total_symbols": stats.get("total_symbols", 0),
                "last_check": datetime.utcnow().isoformat()
            },
            "message": "服务正常"
        }

    except Exception as e:
        logger.error(f"健康检查失败: {e}")
        return {
            "success": False,
            "data": {
                "service": "历史数据服务",
                "status": "unhealthy",
                "error": str(e),
                "last_check": datetime.utcnow().isoformat()
            },
            "message": "服务异常"
        }
```

FILE: app/routers/internal_messages.py

```
"""
内部消息数据API路由
提供内部消息的查询、搜索和管理接口
"""
from typing import Optional, List, Dict, Any
from datetime import datetime, timedelta
from fastapi import APIRouter, HTTPException, BackgroundTasks, Query
from pydantic import BaseModel, Field
from app.services.internal_message_service import (
    get_internal_message_service,
    InternalMessageQueryParams,
    InternalMessageStats
)
from app.core.response import ok
router = APIRouter(prefix="/api/internal-messages", tags=["internal-messages"])
class InternalMessage(BaseModel):
```

```
    """内部消息模型"""
    message_id: str
    message_type: str  # research_report/insider_info/analyst_note/meeting_minutes/internal_analysis
    title: str
    content: str
    summary: Optional[str] = ""
    source: Dict[str, Any]
    category: str
    subcategory: Optional[str] = ""
    tags: Optional[List[str]] = []
    importance: str = "medium"
    impact_scope: str = "stock_specific"
    time_sensitivity: str = "medium_term"
    confidence_level: float = Field(0.5, ge=0.0, le=1.0)
    sentiment: Optional[str] = "neutral"
    sentiment_score: Optional[float] = 0.0
    keywords: Optional[List[str]] = []
    risk_factors: Optional[List[str]] = []
    opportunities: Optional[List[str]] = []
    related_data: Optional[Dict[str, Any]] = {}
    access_level: str = "internal"
    permissions: Optional[List[str]] = []
    created_time: datetime
    effective_time: Optional[datetime] = None
    expiry_time: Optional[datetime] = None
    language: str = "zh-CN"
    data_source: str = "internal_system"

class InternalMessageBatchRequest(BaseModel):
    """批量保存内部消息请求"""
    symbol: str = Field(..., description="股票代码")
    messages: List[InternalMessage] = Field(..., description="内部消息列表")

class InternalMessageQueryRequest(BaseModel):
    """内部消息查询请求"""
    symbol: Optional[str] = None
    symbols: Optional[List[str]] = None
    message_type: Optional[str] = None
    category: Optional[str] = None
    source_type: Optional[str] = None
    department: Optional[str] = None
    author: Optional[str] = None
    start_time: Optional[datetime] = None
    end_time: Optional[datetime] = None
    importance: Optional[str] = None
    access_level: Optional[str] = None
    min_confidence: Optional[float] = None
    rating: Optional[str] = None
    keywords: Optional[List[str]] = None
    tags: Optional[List[str]] = None
    limit: int = Field(50, ge=1, le=1000)
    skip: int = Field(0, ge=0)

@router.post("/save", response_model=dict)
async def save_internal_messages(request: InternalMessageBatchRequest):
    """批量保存内部消息"""
    try:
        service = await get_internal_message_service()

        # 转换消息格式并添加股票代码
        messages = []
        for msg in request.messages:
            message_dict = msg.dict()
            message_dict["symbol"] = request.symbol
            messages.append(message_dict)

        # 保存消息
        result = await service.save_internal_messages(messages)

        return ok(data=result,
            message=f"成功保存 {result['saved']} 条内部消息"
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"保存内部消息失败: {str(e)}")

@router.post("/query", response_model=dict)
async def query_internal_messages(request: InternalMessageQueryRequest):
    """查询内部消息"""
    try:
        service = await get_internal_message_service()

        # 构建查询参数
        params = InternalMessageQueryParams(
            symbol=request.symbol,
            symbols=request.symbols,
            message_type=request.message_type,
            category=request.category,
            source_type=request.source_type,
            department=request.department,
            author=request.author,
            start_time=request.start_time,
            end_time=request.end_time,
            importance=request.importance,
            access_level=request.access_level,
            min_confidence=request.min_confidence,
            rating=request.rating,
            keywords=request.keywords,
            tags=request.tags,
            limit=request.limit,
            skip=request.skip
        )

        # 执行查询
        messages = await service.query_internal_messages(params)

        return ok(data={
                "messages": messages,
                "count": len(messages),
```

```
            "params": request.dict()
        },
        message=f"查询到 {len(messages)} 条内部消息"
    )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"查询内部消息失败: {str(e)}")

@router.get("/latest/{symbol}", response_model=dict)
async def get_latest_messages(
    symbol: str,
    message_type: Optional[str] = Query(None, description="消息类型"),
    access_level: Optional[str] = Query(None, description="访问级别"),
    limit: int = Query(20, ge=1, le=100, description="返回数量")
):
    """获取最新内部消息"""
    try:
        service = await get_internal_message_service()
        messages = await service.get_latest_messages(symbol, message_type, access_level, limit)

        return ok(data={
            "messages": messages,
            "count": len(messages),
            "symbol": symbol,
            "message_type": message_type,
            "access_level": access_level
        },
        message=f"获取到 {len(messages)} 条最新消息"
    )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取最新消息失败: {str(e)}")

@router.get("/search", response_model=dict)
async def search_messages(
    query: str = Query(..., description="搜索关键词"),
    symbol: Optional[str] = Query(None, description="股票代码"),
    access_level: Optional[str] = Query(None, description="访问级别"),
    limit: int = Query(50, ge=1, le=200, description="返回数量")
):
    """全文搜索内部消息"""
    try:
        service = await get_internal_message_service()
        messages = await service.search_messages(query, symbol, access_level, limit)

        return ok(data={
            "messages": messages,
            "count": len(messages),
            "query": query,
            "symbol": symbol,
            "access_level": access_level
        },
        message=f"搜索到 {len(messages)} 条相关消息"
    )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"搜索消息失败: {str(e)}")

@router.get("/research-reports/{symbol}", response_model=dict)
async def get_research_reports(
    symbol: str,
    department: Optional[str] = Query(None, description="部门"),
    limit: int = Query(20, ge=1, le=100, description="返回数量")
):
    """获取研究报告"""
    try:
        service = await get_internal_message_service()
        reports = await service.get_research_reports(symbol, department, limit)

        return ok(data={
            "reports": reports,
            "count": len(reports),
            "symbol": symbol,
            "department": department
        },
        message=f"获取到 {len(reports)} 份研究报告"
    )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取研究报告失败: {str(e)}")

@router.get("/analyst-notes/{symbol}", response_model=dict)
async def get_analyst_notes(
    symbol: str,
    author: Optional[str] = Query(None, description="分析师"),
    limit: int = Query(20, ge=1, le=100, description="返回数量")
):
    """获取分析师笔记"""
    try:
        service = await get_internal_message_service()
        notes = await service.get_analyst_notes(symbol, author, limit)

        return ok(data={
            "notes": notes,
            "count": len(notes),
            "symbol": symbol,
            "author": author
        },
        message=f"获取到 {len(notes)} 条分析师笔记"
    )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取分析师笔记失败: {str(e)}")

@router.get("/statistics", response_model=dict)
async def get_statistics(
    symbol: Optional[str] = Query(None, description="股票代码"),
    hours_back: int = Query(24, ge=1, le=168, description="回溯小时数")
):
```

```
    """获取内部消息统计信息"""
    try:
        service = await get_internal_message_service()

        # 计算时间范围
        end_time = datetime.utcnow()
        start_time = end_time - timedelta(hours=hours_back)

        stats = await service.get_internal_statistics(symbol, start_time, end_time)

        return ok(data={
                "statistics": stats.__dict__,
                "symbol": symbol,
                "time_range": {
                    "start_time": start_time,
                    "end_time": end_time,
                    "hours_back": hours_back
                }
            },
            message="统计信息获取成功"
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取统计信息失败: {str(e)}")
@router.get("/message-types", response_model=dict)
async def get_message_types():
    """获取支持的消息类型列表"""
    message_types = [
        {
            "code": "research_report",
            "name": "研究报告",
            "description": "深度研究分析报告"
        },
        {
            "code": "insider_info",
            "name": "内幕信息",
            "description": "内部获得的重要信息"
        },
        {
            "code": "analyst_note",
            "name": "分析师笔记",
            "description": "分析师的观点和笔记"
        },
        {
            "code": "meeting_minutes",
            "name": "会议纪要",
            "description": "重要会议的记录"
        },
        {
            "code": "internal_analysis",
            "name": "内部分析",
            "description": "内部团队的分析结果"
        }
    ]

    return ok(data={
            "message_types": message_types,
            "count": len(message_types)
        },
        message="消息类型列表获取成功"
    )
@router.get("/categories", response_model=dict)
async def get_categories():
    """获取支持的分类列表"""
    categories = [
        {
            "code": "fundamental_analysis",
            "name": "基本面分析",
            "description": "公司基本面相关分析"
        },
        {
            "code": "technical_analysis",
            "name": "技术分析",
            "description": "技术指标和图表分析"
        },
        {
            "code": "market_sentiment",
            "name": "市场情绪",
            "description": "市场情绪和投资者行为分析"
        },
        {
            "code": "risk_assessment",
            "name": "风险评估",
            "description": "投资风险评估和管理"
        }
    ]

    return ok(data={
            "categories": categories,
            "count": len(categories)
        },
        message="分类列表获取成功"
    )
@router.get("/health", response_model=dict)
async def health_check():
    """健康检查"""
    try:
        service = await get_internal_message_service()

        # 简单的连接测试
        collection = await service._get_collection()
        count = await collection.estimated_document_count()

        return ok(data={
                "status": "healthy",
```

```
            "total_messages": count,
            "service": "internal_message_service"
        },
        message="内部消息服务运行正常"
    )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"健康检查失败: {str(e)}")
```

FILE: app/routers/logs.py

```
"""
日志管理API路由
提供日志查询、过滤和导出功能
"""
import logging
from typing import List, Optional
from fastapi import APIRouter, Depends, HTTPException, Query
from fastapi.responses import FileResponse
from pydantic import BaseModel, Field
from app.routers.auth_db import get_current_user
from app.services.log_export_service import get_log_export_service
router = APIRouter(prefix="/system-logs", tags=["系统日志"])
logger = logging.getLogger("webapi")
# 请求模型
class LogReadRequest(BaseModel):
    """日志读取请求"""
    filename: str = Field(..., description="日志文件名")
    lines: int = Field(default=1000, ge=1, le=10000, description="读取行数")
    level: Optional[str] = Field(default=None, description="日志级别过滤")
    keyword: Optional[str] = Field(default=None, description="关键词过滤")
    start_time: Optional[str] = Field(default=None, description="开始时间（ISO格式）")
    end_time: Optional[str] = Field(default=None, description="结束时间（ISO格式）")
class LogExportRequest(BaseModel):
    """日志导出请求"""
    filenames: Optional[List[str]] = Field(default=None, description="要导出的文件名列表（空表示全部）")
    level: Optional[str] = Field(default=None, description="日志级别过滤")
    start_time: Optional[str] = Field(default=None, description="开始时间（ISO格式）")
    end_time: Optional[str] = Field(default=None, description="结束时间（ISO格式）")
    format: str = Field(default="zip", description="导出格式：zip, txt")
# 响应模型
class LogFileInfo(BaseModel):
    """日志文件信息"""
    name: str
    path: str
    size: int
    size_mb: float
    modified_at: str
    type: str
class LogContentResponse(BaseModel):
    """日志内容响应"""
    filename: str
    lines: List[str]
    stats: dict
class LogStatisticsResponse(BaseModel):
    """日志统计响应"""
    total_files: int
    total_size_mb: float
    error_files: int
    recent_errors: List[str]
    log_types: dict
@router.get("/files", response_model=List[LogFileInfo])
async def list_log_files(
    current_user: dict = Depends(get_current_user)
):
    """
    获取所有日志文件列表

    返回日志文件的基本信息，包括文件名、大小、修改时间等
    """
    try:
        logger.info(f"📋 用户 {current_user['username']} 查询日志文件列表")

        service = get_log_export_service()
        files = service.list_log_files()

        return files

    except Exception as e:
        logger.error(f"❌ 获取日志文件列表失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取日志文件列表失败: {str(e)}")
@router.post("/read", response_model=LogContentResponse)
async def read_log_file(
    request: LogReadRequest,
    current_user: dict = Depends(get_current_user)
):
    """
    读取日志文件内容

    支持过滤条件：
    - lines: 读取的行数（从末尾开始）
    - level: 日志级别（ERROR, WARNING, INFO, DEBUG）
    - keyword: 关键词搜索
    - start_time/end_time: 时间范围
    """
    try:
        logger.info(f"📖 用户 {current_user['username']} 读取日志文件: {request.filename}")

        service = get_log_export_service()
        content = service.read_log_file(
            filename=request.filename,
            lines=request.lines,
```

```
            level=request.level,
            keyword=request.keyword,
            start_time=request.start_time,
            end_time=request.end_time
        )

        return content

    except FileNotFoundError as e:
        raise HTTPException(status_code=404, detail=str(e))
    except Exception as e:
        logger.error(f"❌ 读取日志文件失败: {e}")
        raise HTTPException(status_code=500, detail=f"读取日志文件失败: {str(e)}")
@router.post("/export")
async def export_logs(
    request: LogExportRequest,
    current_user: dict = Depends(get_current_user)
):
    """
    导出日志文件

    支持导出格式：
    - zip: 压缩包（推荐）
    - txt: 合并的文本文件

    支持过滤条件：
    - filenames: 指定要导出的文件
    - level: 日志级别过滤
    - start_time/end_time: 时间范围过滤
    """
    try:
        logger.info(f"📤 用户 {current_user['username']} 导出日志文件")

        service = get_log_export_service()
        export_path = service.export_logs(
            filenames=request.filenames,
            level=request.level,
            start_time=request.start_time,
            end_time=request.end_time,
            format=request.format
        )

        # 返回文件下载
        import os
        filename = os.path.basename(export_path)
        media_type = "application/zip" if request.format == "zip" else "text/plain"

        return FileResponse(
            path=export_path,
            filename=filename,
            media_type=media_type,
            headers={"Content-Disposition": f"attachment; filename={filename}"}
        )

    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
    except Exception as e:
        logger.error(f"❌ 导出日志文件失败: {e}")
        raise HTTPException(status_code=500, detail=f"导出日志文件失败: {str(e)}")
@router.get("/statistics", response_model=LogStatisticsResponse)
async def get_log_statistics(
    days: int = Query(default=7, ge=1, le=30, description="统计最近几天的日志"),
    current_user: dict = Depends(get_current_user)
):
    """
    获取日志统计信息

    返回最近N天的日志统计，包括：
    - 文件数量和总大小
    - 错误日志数量
    - 最近的错误信息
    - 日志类型分布
    """
    try:
        logger.info(f"📊 用户 {current_user['username']} 查询日志统计信息")

        service = get_log_export_service()
        stats = service.get_log_statistics(days=days)

        return stats

    except Exception as e:
        logger.error(f"❌ 获取日志统计失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取日志统计失败: {str(e)}")
@router.delete("/files/{filename}")
async def delete_log_file(
    filename: str,
    current_user: dict = Depends(get_current_user)
):
    """
    删除日志文件

    注意：此操作不可恢复，请谨慎使用
    """
    try:
        logger.warning(f"🗑️ 用户 {current_user['username']} 删除日志文件: {filename}")

        service = get_log_export_service()
        file_path = service.log_dir / filename

        if not file_path.exists():
            raise HTTPException(status_code=404, detail="日志文件不存在")

        # 安全检查：只允许删除 .log 文件
        if not filename.endswith('.log') and not '.log.' in filename:
```

```
            raise HTTPException(status_code=400, detail="只能删除日志文件")

        file_path.unlink()

        return {
            "success": True,
            "message": f"日志文件已删除: {filename}"
        }

    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 删除日志文件失败: {e}")
        raise HTTPException(status_code=500, detail=f"删除日志文件失败: {str(e)}")
```

FILE: app/routers/model_capabilities.py

```
"""
模型能力管理API路由
"""
from fastapi import APIRouter, HTTPException, Depends
from typing import List, Dict, Any, Optional
from pydantic import BaseModel, Field
from app.services.model_capability_service import get_model_capability_service
from app.constants.model_capabilities import (
    DEFAULT_MODEL_CAPABILITIES,
    ANALYSIS_DEPTH_REQUIREMENTS,
    CAPABILITY_DESCRIPTIONS,
    ModelRole,
    ModelFeature,
    get_model_capability_badge,
    get_role_badge,
    get_feature_badge
)
from app.core.unified_config import unified_config
from app.core.response import ok, fail
import logging

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/api/model-capabilities", tags=["模型能力管理"])

# ==================== 请求/响应模型 ====================
class ModelCapabilityInfo(BaseModel):
    """模型能力信息"""
    model_name: str
    capability_level: int
    suitable_roles: List[str]
    features: List[str]
    recommended_depths: List[str]
    performance_metrics: Optional[Dict[str, Any]] = None
    description: Optional[str] = None

class ModelRecommendationRequest(BaseModel):
    """模型推荐请求"""
    research_depth: str = Field(..., description="研究深度：快速/基础/标准/深度/全面")

class ModelRecommendationResponse(BaseModel):
    """模型推荐响应"""
    quick_model: str
    deep_model: str
    quick_model_info: ModelCapabilityInfo
    deep_model_info: ModelCapabilityInfo
    reason: str

class ModelValidationRequest(BaseModel):
    """模型验证请求"""
    quick_model: str
    deep_model: str
    research_depth: str

class ModelValidationResponse(BaseModel):
    """模型验证响应"""
    valid: bool
    warnings: List[str]
    recommendations: List[str]

class BatchInitRequest(BaseModel):
    """批量初始化请求"""
    overwrite: bool = Field(default=False, description="是否覆盖已有配置")

# ==================== API路由 ====================
@router.get("/default-configs")
async def get_default_model_configs():
    """
    获取所有默认模型能力配置
    返回预定义的常见模型能力配置，用于参考和初始化。
    """
    try:
        # 转换为可序列化的格式
        configs = {}
        for model_name, config in DEFAULT_MODEL_CAPABILITIES.items():
            configs[model_name] = {
                "model_name": model_name,
                "capability_level": config["capability_level"],
                "suitable_roles": [str(role) for role in config["suitable_roles"]],
                "features": [str(feature) for feature in config["features"]],
                "recommended_depths": config["recommended_depths"],
                "performance_metrics": config.get("performance_metrics"),
                "description": config.get("description")
            }
        return {
            "success": True,
            "data": configs,
            "message": "获取默认模型配置成功"
        }
    except Exception as e:
        logger.error(f"获取默认模型配置失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
```

```
@router.get("/depth-requirements", response_model=dict)
async def get_depth_requirements():
    """
    获取分析深度要求
    返回各个分析深度对模型的最低要求。
    """
    try:
        # 转换为可序列化的格式
        requirements = {}
        for depth, req in ANALYSIS_DEPTH_REQUIREMENTS.items():
            requirements[depth] = {
                "min_capability": req["min_capability"],
                "quick_model_min": req["quick_model_min"],
                "deep_model_min": req["deep_model_min"],
                "required_features": [str(f) for f in req["required_features"]],
                "description": req["description"]
            }
        return ok(requirements, "获取分析深度要求成功")
    except Exception as e:
        logger.error(f"获取分析深度要求失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/capability-descriptions", response_model=dict)
async def get_capability_descriptions():
    """获取能力等级描述"""
    try:
        return ok(CAPABILITY_DESCRIPTIONS, "获取能力等级描述成功")
    except Exception as e:
        logger.error(f"获取能力等级描述失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/badges", response_model=dict)
async def get_all_badges():
    """
    获取所有徽章样式
    返回能力等级、角色、特性的徽章样式配置。
    """
    try:
        badges = {
            "capability_levels": {
                str(level): get_model_capability_badge(level)
                for level in range(1, 6)
            },
            "roles": {
                str(role): get_role_badge(role)
                for role in ModelRole
            },
            "features": {
                str(feature): get_feature_badge(feature)
                for feature in ModelFeature
            }
        }
        return ok(badges, "获取徽章样式成功")
    except Exception as e:
        logger.error(f"获取徽章样式失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/recommend", response_model=dict)
async def recommend_models(request: ModelRecommendationRequest):
    """
    推荐模型
    根据分析深度推荐最合适的模型对。
    """
    try:
        capability_service = get_model_capability_service()
        # 获取推荐模型
        quick_model, deep_model = capability_service.recommend_models_for_depth(
            request.research_depth
        )
        logger.info(f"🔍 推荐模型: quick={quick_model}, deep={deep_model}")
        # 获取模型详细信息
        quick_info = capability_service.get_model_config(quick_model)
        deep_info = capability_service.get_model_config(deep_model)
        logger.info(f"🔍 模型详细信息: quick_info={quick_info}, deep_info={deep_info}")
        # 生成推荐理由
        depth_req = ANALYSIS_DEPTH_REQUIREMENTS.get(
            request.research_depth,
            ANALYSIS_DEPTH_REQUIREMENTS["标准"]
        )
        # 获取能力等级描述
        capability_desc = {
            1: "基础级",
            2: "标准级",
            3: "高级",
            4: "专业级",
            5: "旗舰级"
        }
        quick_level_desc = capability_desc.get(quick_info['capability_level'], "标准级")
        deep_level_desc = capability_desc.get(deep_info['capability_level'], "标准级")
        reason = (
            f"• 快速模型：{quick_level_desc}，注重速度和成本，适合数据收集\n"
            f"• 深度模型：{deep_level_desc}，注重质量和推理，适合分析决策"
        )
        response_data = {
            "quick_model": quick_model,
            "deep_model": deep_model,
            "quick_model_info": quick_info,
            "deep_model_info": deep_info,
            "reason": reason
        }
        logger.info(f"🔍 返回的响应数据: {response_data}")
        return ok(response_data, "模型推荐成功")
    except Exception as e:
        logger.error(f"模型推荐失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
```

```
@router.post("/validate", response_model=dict)
async def validate_models(request: ModelValidationRequest):
    """
    验证模型对
    验证选择的模型对是否适合指定的分析深度。
    """
    try:
        capability_service = get_model_capability_service()
        # 验证模型对
        validation = capability_service.validate_model_pair(
            request.quick_model,
            request.deep_model,
            request.research_depth
        )
        return ok(validation, "模型验证完成")
    except Exception as e:
        logger.error(f"模型验证失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/batch-init", response_model=dict)
async def batch_init_capabilities(request: BatchInitRequest):
    """
    批量初始化模型能力
    为数据库中的模型配置自动填充能力参数。
    """
    try:
        # 获取所有LLM配置
        llm_configs = unified_config.get_llm_configs()
        updated_count = 0
        skipped_count = 0
        for config in llm_configs:
            model_name = config.model_name
            # 检查是否已有能力配置
            has_capability = hasattr(config, 'capability_level') and config.capability_level is not None
            if has_capability and not request.overwrite:
                skipped_count += 1
                continue
            # 从默认配置获取能力参数
            if model_name in DEFAULT_MODEL_CAPABILITIES:
                default_config = DEFAULT_MODEL_CAPABILITIES[model_name]
                # 更新配置
                config.capability_level = default_config["capability_level"]
                config.suitable_roles = [str(role) for role in default_config["suitable_roles"]]
                config.features = [str(feature) for feature in default_config["features"]]
                config.recommended_depths = default_config["recommended_depths"]
                config.performance_metrics = default_config.get("performance_metrics")
                # 保存到数据库
                # TODO: 实现保存逻辑
                updated_count += 1
                logger.info(f"已初始化模型 {model_name} 的能力参数")
            else:
                logger.warning(f"模型 {model_name} 没有默认配置，跳过")
                skipped_count += 1
        return ok(
            {
                "updated_count": updated_count,
                "skipped_count": skipped_count,
                "total_count": len(llm_configs)
            },
            f"批量初始化完成：更新{updated_count}个，跳过{skipped_count}个"
        )
    except Exception as e:
        logger.error(f"批量初始化失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/model/{model_name}", response_model=dict)
async def get_model_capability(model_name: str):
    """
    获取指定模型的能力信息
    Args:
        model_name: 模型名称
    """
    try:
        capability_service = get_model_capability_service()
        config = capability_service.get_model_config(model_name)
        return ok(config, f"获取模型 {model_name} 能力信息成功")
    except Exception as e:
        logger.error(f"获取模型能力信息失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
```

FILE: app/routers/multi_market_stocks.py

```
"""
多市场股票API路由
支持A股、港股、美股的统一查询接口
功能：
1. 跨市场股票信息查询
2. 多数据源优先级查询
3. 统一的响应格式
路径前缀: /api/markets
"""
from typing import Optional, Dict, Any, List
from fastapi import APIRouter, Depends, HTTPException, status, Query
import logging
from app.routers.auth_db import get_current_user
from app.core.database import get_mongo_db
from app.core.response import ok
from app.services.unified_stock_service import UnifiedStockService
logger = logging.getLogger("webapi")
router = APIRouter(prefix="/markets", tags=["multi-market"])
@router.get("", response_model=dict)
async def get_supported_markets(current_user: dict = Depends(get_current_user)):
```

```
    """
    获取支持的市场列表

    Returns:
        {
            "success": true,
            "data": {
                "markets": [
                    {
                        "code": "CN",
                        "name": "A股",
                        "name_en": "China A-Shares",
                        "currency": "CNY",
                        "timezone": "Asia/Shanghai"
                    },
                    ...
                ]
            }
        }
    """
    markets = [
        {
            "code": "CN",
            "name": "A股",
            "name_en": "China A-Shares",
            "currency": "CNY",
            "timezone": "Asia/Shanghai",
            "trading_hours": "09:30-15:00"
        },
        {
            "code": "HK",
            "name": "港股",
            "name_en": "Hong Kong Stocks",
            "currency": "HKD",
            "timezone": "Asia/Hong_Kong",
            "trading_hours": "09:30-16:00"
        },
        {
            "code": "US",
            "name": "美股",
            "name_en": "US Stocks",
            "currency": "USD",
            "timezone": "America/New_York",
            "trading_hours": "09:30-16:00 EST"
        }
    ]

    return ok(data={"markets": markets})
@router.get("/{market}/stocks/search", response_model=dict)
async def search_stocks(
    market: str,
    q: str = Query(..., description="搜索关键词（代码或名称）"),
    limit: int = Query(20, ge=1, le=100, description="返回结果数量"),
    current_user: dict = Depends(get_current_user)
):
    """
    搜索股票（支持多市场）

    Args:
        market: 市场类型 (CN/HK/US)
        q: 搜索关键词
        limit: 返回结果数量

    Returns:
        {
            "success": true,
            "data": {
                "stocks": [
                    {
                        "code": "00700",
                        "name": "腾讯控股",
                        "name_en": "Tencent Holdings",
                        "market": "HK",
                        "source": "yfinance",
                        ...
                    }
                ],
                "total": 1
            }
        }
    """
    market = market.upper()
    if market not in ["CN", "HK", "US"]:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"不支持的市场类型: {market}"
        )

    db = get_mongo_db()
    service = UnifiedStockService(db)

    try:
        results = await service.search_stocks(market, q, limit)
        return ok(data={
            "stocks": results,
            "total": len(results)
        })
    except Exception as e:
        logger.error(f"❌ 搜索股票失败: market={market}, q={q}, error={e}", exc_info=True)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"搜索失败: {str(e)}"
        )
@router.get("/{market}/stocks/{code}/info", response_model=dict)
async def get_stock_info(
```

```
    market: str,
    code: str,
    source: Optional[str] = Query(None, description="指定数据源（可选）"),
    current_user: dict = Depends(get_current_user)
):
    """
    获取股票基础信息（支持多市场、多数据源）

    Args:
        market: 市场类型 (CN/HK/US)
        code: 股票代码
        source: 指定数据源（可选，不指定则按优先级自动选择）

    Returns:
        {
            "success": true,
            "data": {
                "code": "00700",
                "name": "腾讯控股",
                "name_en": "Tencent Holdings",
                "market": "HK",
                "source": "yfinance",
                "total_mv": 32000.0,
                "pe": 25.5,
                "pb": 4.2,
                "lot_size": 100,
                "currency": "HKD",
                ...
            }
        }
    """
    market = market.upper()
    if market not in ["CN", "HK", "US"]:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"不支持的市场类型: {market}"
        )

    db = get_mongo_db()
    service = UnifiedStockService(db)

    try:
        stock_info = await service.get_stock_info(market, code, source)

        if not stock_info:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"未找到股票: {market}:{code}"
            )

        return ok(data=stock_info)
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 获取股票信息失败: market={market}, code={code}, error={e}", exc_info=True)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取股票信息失败: {str(e)}"
        )
@router.get("/{market}/stocks/{code}/quote", response_model=dict)
async def get_stock_quote(
    market: str,
    code: str,
    current_user: dict = Depends(get_current_user)
):
    """
    获取股票实时行情（支持多市场）

    Args:
        market: 市场类型 (CN/HK/US)
        code: 股票代码

    Returns:
        {
            "success": true,
            "data": {
                "code": "00700",
                "close": 320.50,
                "pct_chg": 2.15,
                "open": 315.00,
                "high": 325.00,
                "low": 312.00,
                "volume": 48500000,
                "amount": 15800000000,
                "trade_date": "2024-01-15",
                "currency": "HKD",
                ...
            }
        }
    """
    market = market.upper()
    if market not in ["CN", "HK", "US"]:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"不支持的市场类型: {market}"
        )

    db = get_mongo_db()
    service = UnifiedStockService(db)

    try:
        quote = await service.get_stock_quote(market, code)

        if not quote:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
```

```
            detail=f"未找到股票行情: {market}:{code}"
        )

    return ok(data=quote)
  except HTTPException:
    raise
  except Exception as e:
    logger.error(f"❌ 获取股票行情失败: market={market}, code={code}, error={e}", exc_info=True)
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"获取股票行情失败: {str(e)}"
    )
@router.get("/{market}/stocks/{code}/daily", response_model=dict)
async def get_stock_daily_quotes(
  market: str,
  code: str,
  start_date: Optional[str] = Query(None, description="开始日期 (YYYY-MM-DD)"),
  end_date: Optional[str] = Query(None, description="结束日期 (YYYY-MM-DD)"),
  limit: int = Query(100, ge=1, le=1000, description="返回记录数"),
  current_user: dict = Depends(get_current_user)
):
  """
  获取股票历史K线数据（支持多市场）

  Args:
    market: 市场类型 (CN/HK/US)
    code: 股票代码
    start_date: 开始日期
    end_date: 结束日期
    limit: 返回记录数

  Returns:
    {
      "success": true,
      "data": {
        "code": "00700",
        "market": "HK",
        "quotes": [
          {
            "trade_date": "2024-01-15",
            "open": 315.00,
            "high": 325.00,
            "low": 312.00,
            "close": 320.50,
            "volume": 48500000,
            "amount": 15800000000
          },
          ...
        ],
        "total": 100
      }
    }
  """
  market = market.upper()
  if market not in ["CN", "HK", "US"]:
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail=f"不支持的市场类型: {market}"
    )

  db = get_mongo_db()
  service = UnifiedStockService(db)

  try:
    quotes = await service.get_daily_quotes(
        market, code, start_date, end_date, limit
    )

    return ok(data={
      "code": code,
      "market": market,
      "quotes": quotes,
      "total": len(quotes)
    })
  except Exception as e:
    logger.error(f"❌ 获取历史K线失败: market={market}, code={code}, error={e}", exc_info=True)
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"获取历史K线失败: {str(e)}"
    )
```

FILE: app/routers/multi_period_sync.py

```
#!/usr/bin/env python3
"""
多周期数据同步API
提供日线、周线、月线数据的同步管理接口
"""
import logging
from datetime import datetime
from typing import Dict, Any, List, Optional
from fastapi import APIRouter, HTTPException, BackgroundTasks
from pydantic import BaseModel, Field
from app.worker.multi_period_sync_service import get_multi_period_sync_service
logger = logging.getLogger(__name__)
router = APIRouter(prefix="/api/multi-period-sync", tags=["多周期同步"])
class MultiPeriodSyncRequest(BaseModel):
  """多周期同步请求"""
  symbols: Optional[List[str]] = Field(None, description="股票代码列表，None表示所有股票")
  periods: Optional[List[str]] = Field(["daily"], description="周期列表 (daily/weekly/monthly)")
  data_sources: Optional[List[str]] = Field(["tushare", "akshare", "baostock"], description="数据源列表")
  start_date: Optional[str] = Field(None, description="开始日期 (YYYY-MM-DD)")
  end_date: Optional[str] = Field(None, description="结束日期 (YYYY-MM-DD)")
```

```
    all_history: Optional[bool] = Field(False, description="是否同步所有历史数据（忽略时间范围）")

class MultiPeriodSyncResponse(BaseModel):
    """多周期同步响应"""
    success: bool
    message: str
    data: Optional[Dict[str, Any]] = None

@router.post("/start", response_model=MultiPeriodSyncResponse)
async def start_multi_period_sync(
    request: MultiPeriodSyncRequest,
    background_tasks: BackgroundTasks
):
    """
    启动多周期数据同步
    """
    try:
        service = await get_multi_period_sync_service()

        # 后台任务执行同步
        background_tasks.add_task(
            service.sync_multi_period_data,
            symbols=request.symbols,
            periods=request.periods,
            data_sources=request.data_sources,
            start_date=request.start_date,
            end_date=request.end_date,
            all_history=request.all_history
        )

        return MultiPeriodSyncResponse(
            success=True,
            message="多周期数据同步已启动",
            data={
                "request_params": request.dict(),
                "start_time": datetime.utcnow().isoformat()
            }
        )

    except Exception as e:
        logger.error(f"启动多周期同步失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动同步失败: {e}")

@router.post("/start-daily", response_model=MultiPeriodSyncResponse)
async def start_daily_sync(
    background_tasks: BackgroundTasks,
    symbols: Optional[List[str]] = None,
    data_sources: Optional[List[str]] = None
):
    """启动日线数据同步"""
    try:
        service = await get_multi_period_sync_service()

        background_tasks.add_task(
            service.sync_multi_period_data,
            symbols=symbols,
            periods=["daily"],
            data_sources=data_sources or ["tushare", "akshare", "baostock"]
        )

        return MultiPeriodSyncResponse(
            success=True,
            message="日线数据同步已启动",
            data={
                "period": "daily",
                "start_time": datetime.utcnow().isoformat()
            }
        )

    except Exception as e:
        logger.error(f"启动日线同步失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动日线同步失败: {e}")

@router.post("/start-weekly", response_model=MultiPeriodSyncResponse)
async def start_weekly_sync(
    background_tasks: BackgroundTasks,
    symbols: Optional[List[str]] = None,
    data_sources: Optional[List[str]] = None
):
    """启动周线数据同步"""
    try:
        service = await get_multi_period_sync_service()

        background_tasks.add_task(
            service.sync_multi_period_data,
            symbols=symbols,
            periods=["weekly"],
            data_sources=data_sources or ["tushare", "akshare", "baostock"]
        )

        return MultiPeriodSyncResponse(
            success=True,
            message="周线数据同步已启动",
            data={
                "period": "weekly",
                "start_time": datetime.utcnow().isoformat()
            }
        )

    except Exception as e:
        logger.error(f"启动周线同步失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动周线同步失败: {e}")

@router.post("/start-monthly", response_model=MultiPeriodSyncResponse)
async def start_monthly_sync(
    background_tasks: BackgroundTasks,
    symbols: Optional[List[str]] = None,
    data_sources: Optional[List[str]] = None
):
```

```
    """启动月线数据同步"""
    try:
        service = await get_multi_period_sync_service()
        background_tasks.add_task(
            service.sync_multi_period_data,
            symbols=symbols,
            periods=["monthly"],
            data_sources=data_sources or ["tushare", "akshare", "baostock"]
        )
        return MultiPeriodSyncResponse(
            success=True,
            message="月线数据同步已启动",
            data={
                "period": "monthly",
                "start_time": datetime.utcnow().isoformat()
            }
        )
    except Exception as e:
        logger.error(f"启动月线同步失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动月线同步失败: {e}")

@router.post("/start-all-history", response_model=MultiPeriodSyncResponse)
async def start_all_history_sync(
    background_tasks: BackgroundTasks,
    symbols: Optional[List[str]] = None,
    periods: Optional[List[str]] = None,
    data_sources: Optional[List[str]] = None
):
    """启动全历史数据同步（从1990年开始）"""
    try:
        service = await get_multi_period_sync_service()
        background_tasks.add_task(
            service.sync_multi_period_data,
            symbols=symbols,
            periods=periods or ["daily", "weekly", "monthly"],
            data_sources=data_sources or ["tushare", "akshare", "baostock"],
            all_history=True
        )
        return MultiPeriodSyncResponse(
            success=True,
            message="全历史数据同步已启动（从1990年开始）",
            data={
                "sync_type": "all_history",
                "periods": periods or ["daily", "weekly", "monthly"],
                "data_sources": data_sources or ["tushare", "akshare", "baostock"],
                "date_range": "1990-01-01 到 今天",
                "start_time": datetime.utcnow().isoformat(),
                "warning": "全历史数据同步可能需要很长时间，请耐心等待"
            }
        )
    except Exception as e:
        logger.error(f"启动全历史同步失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动全历史同步失败: {e}")

@router.post("/start-incremental", response_model=MultiPeriodSyncResponse)
async def start_incremental_sync(
    background_tasks: BackgroundTasks,
    symbols: Optional[List[str]] = None,
    periods: Optional[List[str]] = None,
    data_sources: Optional[List[str]] = None,
    days_back: Optional[int] = 30
):
    """启动增量数据同步（最近N天）"""
    try:
        from datetime import datetime, timedelta
        service = await get_multi_period_sync_service()
        # 计算增量同步的日期范围
        end_date = datetime.now().strftime('%Y-%m-%d')
        start_date = (datetime.now() - timedelta(days=days_back)).strftime('%Y-%m-%d')
        background_tasks.add_task(
            service.sync_multi_period_data,
            symbols=symbols,
            periods=periods or ["daily"],
            data_sources=data_sources or ["tushare", "akshare", "baostock"],
            start_date=start_date,
            end_date=end_date
        )
        return MultiPeriodSyncResponse(
            success=True,
            message=f"增量数据同步已启动（最近{days_back}天）",
            data={
                "sync_type": "incremental",
                "periods": periods or ["daily"],
                "data_sources": data_sources or ["tushare", "akshare", "baostock"],
                "date_range": f"{start_date} 到 {end_date}",
                "days_back": days_back,
                "start_time": datetime.utcnow().isoformat()
            }
        )
    except Exception as e:
        logger.error(f"启动增量同步失败: {e}")
        raise HTTPException(status_code=500, detail=f"启动增量同步失败: {e}")

@router.get("/statistics")
async def get_sync_statistics():
    """获取多周期同步统计信息"""
    try:
        service = await get_multi_period_sync_service()
        stats = await service.get_sync_statistics()

        return {
            "success": True,
            "data": stats,
            "message": "统计信息获取成功"
        }
```

```
    except Exception as e:
        logger.error(f"获取同步统计失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取统计信息失败: {e}")

@router.get("/period-comparison/{symbol}")
async def compare_period_data(
    symbol: str,
    trade_date: str,
    data_source: str = "tushare"
):
    """
    对比同一股票不同周期的数据
    """
    try:
        from app.services.historical_data_service import get_historical_data_service
        service = await get_historical_data_service()

        periods = ["daily", "weekly", "monthly"]
        comparison = {}

        for period in periods:
            results = await service.get_historical_data(
                symbol=symbol,
                start_date=trade_date,
                end_date=trade_date,
                data_source=data_source,
                period=period,
                limit=1
            )

            if results:
                comparison[period] = results[0]
            else:
                comparison[period] = None

        return {
            "success": True,
            "data": {
                "symbol": symbol,
                "trade_date": trade_date,
                "data_source": data_source,
                "comparison": comparison,
                "available_periods": [k for k, v in comparison.items() if v is not None]
            },
            "message": "周期数据对比完成"
        }

    except Exception as e:
        logger.error(f"周期数据对比失败 {symbol}: {e}")
        raise HTTPException(status_code=500, detail=f"周期数据对比失败: {e}")

@router.get("/supported-periods")
async def get_supported_periods():
    """获取支持的数据周期"""
    return {
        "success": True,
        "data": {
            "periods": [
                {
                    "code": "daily",
                    "name": "日线",
                    "description": "每日交易数据",
                    "supported_sources": ["tushare", "akshare", "baostock"]
                },
                {
                    "code": "weekly",
                    "name": "周线",
                    "description": "每周交易数据",
                    "supported_sources": ["tushare", "akshare", "baostock"]
                },
                {
                    "code": "monthly",
                    "name": "月线",
                    "description": "每月交易数据",
                    "supported_sources": ["tushare", "akshare", "baostock"]
                }
            ],
            "data_sources": [
                {
                    "code": "tushare",
                    "name": "Tushare",
                    "description": "专业金融数据服务",
                    "supported_periods": ["daily", "weekly", "monthly"]
                },
                {
                    "code": "akshare",
                    "name": "AKShare",
                    "description": "免费开源金融数据",
                    "supported_periods": ["daily", "weekly", "monthly"]
                },
                {
                    "code": "baostock",
                    "name": "BaoStock",
                    "description": "免费证券数据平台",
                    "supported_periods": ["daily", "weekly", "monthly"]
                }
            ]
        },
        "message": "支持的周期信息获取成功"
    }

@router.get("/health")
async def health_check():
    """健康检查"""
    try:
        service = await get_multi_period_sync_service()
        stats = await service.get_sync_statistics()
```

```
        return {
            "success": True,
            "data": {
                "service": "多周期同步服务",
                "status": "healthy",
                "statistics": stats,
                "last_check": datetime.utcnow().isoformat()
            },
            "message": "服务正常"
        }

    except Exception as e:
        logger.error(f"健康检查失败: {e}")
        return {
            "success": False,
            "data": {
                "service": "多周期同步服务",
                "status": "unhealthy",
                "error": str(e),
                "last_check": datetime.utcnow().isoformat()
            },
            "message": "服务异常"
        }
```

FILE: app/routers/multi_source_sync.py

```
"""
Multi-source synchronization API routes
Provides endpoints for multi-source stock data synchronization
"""
import asyncio
import logging
from typing import Dict, List, Optional, Any, Union
from fastapi import APIRouter, HTTPException, Query
from pydantic import BaseModel
from app.services.multi_source_basics_sync_service import get_multi_source_sync_service
from app.services.data_sources.manager import DataSourceManager

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/api/sync/multi-source", tags=["Multi-Source Sync"])

class SyncRequest(BaseModel):
    """同步请求模型"""
    force: bool = False
    preferred_sources: Optional[List[str]] = None

class SyncResponse(BaseModel):
    """同步响应模型"""
    success: bool
    message: str
    data: Union[Dict[str, Any], List[Any], Any]

class DataSourceStatus(BaseModel):
    """数据源状态模型"""
    name: str
    priority: int
    available: bool
    description: str

@router.get("/sources/status")
async def get_data_sources_status():
    """获取所有数据源的状态"""
    try:
        manager = DataSourceManager()
        available_adapters = manager.get_available_adapters()
        all_adapters = manager.adapters
        status_list = []
        for adapter in all_adapters:
            is_available = adapter in available_adapters
            # 根据数据源类型提供描述
            descriptions = {
                "tushare": "专业金融数据API，提供高质量的A股数据和财务指标",
                "akshare": "开源金融数据库，提供基础的股票信息",
                "baostock": "免费开源的证券数据平台，提供历史数据"
            }
            status_item = {
                "name": adapter.name,
                "priority": adapter.priority,
                "available": is_available,
                "description": descriptions.get(adapter.name, f"{adapter.name}数据源")
            }
            # 添加 Token 来源信息（仅 Tushare）
            if adapter.name == "tushare" and is_available and hasattr(adapter, 'get_token_source'):
                token_source = adapter.get_token_source()
                if token_source:
                    status_item["token_source"] = token_source
                    if token_source == 'database':
                        status_item["description"] += " (Token来源: 数据库)"
                    elif token_source == 'env':
                        status_item["description"] += " (Token来源: .env)"
            status_list.append(status_item)
        return SyncResponse(
            success=True,
            message="Data sources status retrieved successfully",
            data=status_list
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Failed to get data sources status: {str(e)}")

@router.get("/sources/current")
async def get_current_data_source():
    """获取当前正在使用的数据源（优先级最高且可用的）"""
    try:
        manager = DataSourceManager()
        available_adapters = manager.get_available_adapters()
        if not available_adapters:
            return SyncResponse(
```

```
            success=False,
            message="No available data sources",
            data={"name": None, "priority": None}
        )
    # 获取优先级最高的可用数据源（优先级数字越大越高）
    current_adapter = max(available_adapters, key=lambda x: x.priority)
    # 根据数据源类型提供描述
    descriptions = {
        "tushare": "专业金融数据API",
        "akshare": "开源金融数据库",
        "baostock": "免费证券数据平台"
    }
    result = {
        "name": current_adapter.name,
        "priority": current_adapter.priority,
        "description": descriptions.get(current_adapter.name, current_adapter.name)
    }
    # 添加 Token 来源信息（仅 Tushare）
    if current_adapter.name == "tushare" and hasattr(current_adapter, 'get_token_source'):
        token_source = current_adapter.get_token_source()
        if token_source:
            result["token_source"] = token_source
            if token_source == 'database':
                result["token_source_display"] = "数据库配置"
            elif token_source == 'env':
                result["token_source_display"] = ".env 配置"
    return SyncResponse(
        success=True,
        message="Current data source retrieved successfully",
        data=result
    )
  except Exception as e:
    raise HTTPException(status_code=500, detail=f"Failed to get current data source: {str(e)}")

@router.get("/status")
async def get_sync_status():
  """获取多数据源同步状态"""
  try:
    service = get_multi_source_sync_service()
    status = await service.get_status()

    return SyncResponse(
        success=True,
        message="Status retrieved successfully",
        data=status
    )

  except Exception as e:
    raise HTTPException(status_code=500, detail=f"Failed to get sync status: {str(e)}")

@router.post("/stock_basics/run")
async def run_stock_basics_sync(
  force: bool = Query(False, description="是否强制运行同步"),
  preferred_sources: Optional[str] = Query(None, description="优先使用的数据源，用逗号分隔")
):
  """运行多数据源股票基础信息同步"""
  try:
    service = get_multi_source_sync_service()
    # 解析优先数据源
    sources_list = None
    if preferred_sources and isinstance(preferred_sources, str):
        sources_list = [s.strip() for s in preferred_sources.split(",") if s.strip()]
    # 运行同步（同步执行，前端已设置10分钟超时）
    result = await service.run_full_sync(force=force, preferred_sources=sources_list)
    # 判断是否成功
    success = result.get("status") in ["success", "success_with_errors"]
    message = "Synchronization completed successfully"
    if result.get("status") == "success_with_errors":
        message = f"Synchronization completed with {result.get('errors', 0)} errors"
    elif result.get("status") == "failed":
        message = f"Synchronization failed: {result.get('message', 'Unknown error')}"
        success = False
    elif result.get("status") == "running":
        message = "Synchronization is already running"
    return SyncResponse(
        success=success,
        message=message,
        data=result
    )
  except Exception as e:
    raise HTTPException(status_code=500, detail=f"Failed to run synchronization: {str(e)}")

async def _test_single_adapter(adapter) -> dict:
  """
  测试单个数据源适配器的连通性
  只做轻量级连通性测试，不获取完整数据
  """
  result = {
    "name": adapter.name,
    "priority": adapter.priority,
    "available": False,
    "message": "连接失败"
  }
  # 连通性测试超时时间（秒）
  test_timeout = 10
  try:
    # 测试连通性 - 强制重新连接以使用最新配置
    logger.info(f"🎤 测试 {adapter.name} 连通性 (超时: {test_timeout}秒)...")
    try:
      # 对于 Tushare，强制重新连接以使用最新的数据库配置
      if adapter.name == "tushare" and hasattr(adapter, '_provider'):
        logger.info(f"🔄 强制 {adapter.name} 重新连接以使用最新配置...")
        provider = adapter._provider
        if provider:
          # 重置连接状态
          provider.connected = False
```

```
                provider.token_source = None
                # 重新连接
                await asyncio.wait_for(
                    asyncio.to_thread(provider.connect_sync),
                    timeout=test_timeout
                )
            # 在线程池中运行 is_available() 检查
            is_available = await asyncio.wait_for(
                asyncio.to_thread(adapter.is_available),
                timeout=test_timeout
            )
            if is_available:
                result["available"] = True
                # 获取 Token 来源（仅 Tushare）
                token_source = None
                if adapter.name == "tushare" and hasattr(adapter, 'get_token_source'):
                    token_source = adapter.get_token_source()
                if token_source == 'database':
                    result["message"] = "✅ 连接成功 (Token来源: 数据库)"
                    result["token_source"] = "database"
                elif token_source == 'env':
                    result["message"] = "✅ 连接成功 (Token来源: .env)"
                    result["token_source"] = "env"
                else:
                    result["message"] = "✅ 连接成功"
                logger.info(f"✅ {adapter.name} 连通性测试成功，Token来源: {token_source}")
            else:
                result["available"] = False
                result["message"] = "❌ 数据源不可用"
                logger.warning(f"⚠️ {adapter.name} 不可用")
        except asyncio.TimeoutError:
            result["available"] = False
            result["message"] = f"❌ 连接超时 ({test_timeout}秒)"
            logger.warning(f"⚠️ {adapter.name} 连接超时")
        except Exception as e:
            result["available"] = False
            result["message"] = f"❌ 连接失败: {str(e)}"
            logger.error(f"❌ {adapter.name} 连接失败: {e}")
    except Exception as e:
        result["available"] = False
        result["message"] = f"❌ 测试异常: {str(e)}"
        logger.error(f"❌ 测试 {adapter.name} 时出错: {e}")
    return result

class TestSourceRequest(BaseModel):
    """测试数据源请求"""
    source_name: str | None = None

@router.post("/test-sources")
async def test_data_sources(request: TestSourceRequest = TestSourceRequest()):
    """
    测试数据源的连通性
    参数:
    - source_name: 可选，指定要测试的数据源名称。如果不指定，则测试所有数据源
    只做轻量级连通性测试，不获取完整数据
    - 测试超时: 10秒
    - 只获取1条数据验证连接
    - 快速返回结果
    """
    try:
        manager = DataSourceManager()
        all_adapters = manager.adapters
        # 从请求体中获取数据源名称
        source_name = request.source_name
        logger.info(f"📥 接收到测试请求，source_name={source_name}")
        # 如果指定了数据源名称，只测试该数据源
        if source_name:
            adapters_to_test = [a for a in all_adapters if a.name.lower() == source_name.lower()]
            if not adapters_to_test:
                raise HTTPException(
                    status_code=400,
                    detail=f"Data source '{source_name}' not found"
                )
            logger.info(f"🔍 开始测试数据源: {source_name}")
        else:
            adapters_to_test = all_adapters
            logger.info(f"🔍 开始测试 {len(all_adapters)} 个数据源的连通性...")
        # 并发测试适配器（在后台线程中执行）
        test_tasks = [_test_single_adapter(adapter) for adapter in adapters_to_test]
        test_results = await asyncio.gather(*test_tasks, return_exceptions=True)
        # 处理异常结果
        final_results = []
        for i, result in enumerate(test_results):
            if isinstance(result, Exception):
                logger.error(f"❌ 测试适配器 {adapters_to_test[i].name} 时出错: {result}")
                final_results.append({
                    "name": adapters_to_test[i].name,
                    "priority": adapters_to_test[i].priority,
                    "available": False,
                    "message": f"❌ 测试异常: {str(result)}"
                })
            else:
                final_results.append(result)
        # 统计结果
        available_count = sum(1 for r in final_results if r.get("available"))
        if source_name:
            logger.info(f"✅ 数据源 {source_name} 测试完成: {'可用' if available_count > 0 else '不可用'}")
        else:
            logger.info(f"✅ 数据源连通性测试完成: {available_count}/{len(final_results)} 可用")
        return SyncResponse(
            success=True,
            message=f"Tested {len(final_results)} data sources, {available_count} available",
            data={"test_results": final_results}
        )
    except HTTPException:
```

```
        raise
    except Exception as e:
        logger.error(f"❌ 测试数据源时出错: {e}", exc_info=True)
        raise HTTPException(status_code=500, detail=f"Failed to test data sources: {str(e)}")

@router.get("/recommendations")
async def get_sync_recommendations():
    """获取数据源使用建议"""
    try:
        manager = DataSourceManager()
        available_adapters = manager.get_available_adapters()

        recommendations = {
            "primary_source": None,
            "fallback_sources": [],
            "suggestions": [],
            "warnings": []
        }

        if available_adapters:
            # 推荐优先级最高的可用数据源作为主数据源
            primary = available_adapters[0]
            recommendations["primary_source"] = {
                "name": primary.name,
                "priority": primary.priority,
                "reason": "Highest priority available data source"
            }

            # 其他可用数据源作为备用
            for adapter in available_adapters[1:]:
                recommendations["fallback_sources"].append({
                    "name": adapter.name,
                    "priority": adapter.priority
                })

        # 生成建议
        if not available_adapters:
            recommendations["warnings"].append("No data sources are available. Please check your configuration.")
        elif len(available_adapters) == 1:
            recommendations["suggestions"].append("Consider configuring additional data sources for redundancy.")
        else:
            recommendations["suggestions"].append(f"You have {len(available_adapters)} data sources available, which provides good redundancy.")

        # 特定数据源的建议
        tushare_available = any(a.name == "tushare" for a in available_adapters)
        if not tushare_available:
            recommendations["suggestions"].append("Consider configuring Tushare for the most comprehensive financial data.")

        return SyncResponse(
            success=True,
            message="Recommendations generated successfully",
            data=recommendations
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Failed to generate recommendations: {str(e)}")

@router.get("/history")
async def get_sync_history(
    page: int = Query(1, ge=1, description="页码"),
    page_size: int = Query(10, ge=1, le=50, description="每页大小"),
    status: Optional[str] = Query(None, description="状态筛选")
):
    """获取同步历史记录"""
    try:
        from app.core.database import get_mongo_db
        db = get_mongo_db()
        # 构建查询条件
        query = {"job": "stock_basics_multi_source"}
        if status:
            query["status"] = status
        # 计算跳过的记录数
        skip = (page - 1) * page_size
        # 查询历史记录
        cursor = db.sync_status.find(query).sort("started_at", -1).skip(skip).limit(page_size)
        history_records = await cursor.to_list(length=page_size)
        # 获取总数
        total = await db.sync_status.count_documents(query)
        # 清理记录中的 _id 字段
        for record in history_records:
            record.pop("_id", None)
        return SyncResponse(
            success=True,
            message="History retrieved successfully",
            data={
                "records": history_records,
                "total": total,
                "page": page,
                "page_size": page_size,
                "has_more": skip + len(history_records) < total
            }
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Failed to get sync history: {str(e)}")

@router.delete("/cache")
async def clear_sync_cache():
    """清空同步相关的缓存"""
    try:
        service = get_multi_source_sync_service()
        # 清空同步状态缓存
        cleared_items = 0
        # 1. 清空同步状态
        try:
            from app.core.database import get_mongo_db
            db = get_mongo_db()
```

```
        # 删除同步状态记录
        result = await db.sync_status.delete_many({"job": "stock_basics_multi_source"})
        cleared_items += result.deleted_count
        # 重置服务状态
        service._running = False
    except Exception as e:
        logger.warning(f"Failed to clear sync status cache: {e}")
    # 2. 清空数据源缓存（如果有的话）
    try:
        manager = DataSourceManager()
        # 这里可以添加数据源特定的缓存清理逻辑
        # 目前数据源适配器没有持久化缓存，所以跳过
    except Exception as e:
        logger.warning(f"Failed to clear data source cache: {e}")
    return SyncResponse(
        success=True,
        message=f"Cache cleared successfully, {cleared_items} items removed",
        data={"cleared": True, "items_cleared": cleared_items}
    )
  except Exception as e:
    raise HTTPException(status_code=500, detail=f"Failed to clear cache: {str(e)}")
```

FILE: app/routers/news_data.py

```
"""
新闻数据API路由
提供新闻数据查询、同步和管理接口
"""
from fastapi import APIRouter, HTTPException, BackgroundTasks, Depends, Query, status
from typing import Optional, List, Dict, Any
from datetime import datetime, timedelta
from pydantic import BaseModel, Field
import logging
from app.routers.auth_db import get_current_user
from app.core.response import ok
from app.services.news_data_service import get_news_data_service, NewsQueryParams
from app.worker.news_data_sync_service import get_news_data_sync_service
router = APIRouter(prefix="/api/news-data", tags=["新闻数据"])
logger = logging.getLogger("webapi")
class NewsQueryRequest(BaseModel):
  """新闻查询请求"""
  symbol: Optional[str] = Field(None, description="股票代码")
  symbols: Optional[List[str]] = Field(None, description="多个股票代码")
  start_time: Optional[datetime] = Field(None, description="开始时间")
  end_time: Optional[datetime] = Field(None, description="结束时间")
  category: Optional[str] = Field(None, description="新闻类别")
  sentiment: Optional[str] = Field(None, description="情绪分析")
  importance: Optional[str] = Field(None, description="重要性")
  data_source: Optional[str] = Field(None, description="数据源")
  keywords: Optional[List[str]] = Field(None, description="关键词")
  limit: int = Field(50, description="返回数量限制")
  skip: int = Field(0, description="跳过数量")
class NewsSyncRequest(BaseModel):
  """新闻同步请求"""
  symbol: Optional[str] = Field(None, description="股票代码，为空则同步市场新闻")
  data_sources: Optional[List[str]] = Field(None, description="数据源列表")
  hours_back: int = Field(24, description="回溯小时数")
  max_news_per_source: int = Field(50, description="每个数据源最大新闻数量")
@router.get("/query/{symbol}", response_model=dict)
async def query_stock_news(
  symbol: str,
  hours_back: int = Query(24, description="回溯小时数"),
  limit: int = Query(20, description="返回数量限制"),
  category: Optional[str] = Query(None, description="新闻类别"),
  sentiment: Optional[str] = Query(None, description="情绪分析"),
  current_user: dict = Depends(get_current_user)
):
  """
  查询股票新闻（智能获取：优先数据库，无数据时实时获取）
  Args:
    symbol: 股票代码
    hours_back: 回溯小时数
    limit: 返回数量限制
    category: 新闻类别过滤
    sentiment: 情绪分析过滤
  Returns:
    dict: 新闻数据列表
  """
  try:
    service = await get_news_data_service()
    # 构建查询参数
    start_time = datetime.utcnow() - timedelta(hours=hours_back)
    params = NewsQueryParams(
      symbol=symbol,
      start_time=start_time,
      category=category,
      sentiment=sentiment,
      limit=limit,
      sort_by="publish_time",
      sort_order=-1
    )
    # 1. 先从数据库查询
    news_list = await service.query_news(params)
    data_source = "database"
    # 2. 如果数据库没有数据，实时获取
    if not news_list:
      logger.info(f"📰 数据库无新闻数据，实时获取: {symbol}")
      try:
        from app.worker.akshare_sync_service import get_akshare_sync_service
        sync_service = await get_akshare_sync_service()
        # 实时获取新闻
```

```
            news_data = await sync_service.provider.get_stock_news(
                symbol=symbol,
                limit=limit
            )
            if news_data:
                # 保存到数据库
                saved_count = await service.save_news_data(
                    news_data=news_data,
                    data_source="akshare",
                    market="CN"
                )
                logger.info(f"✅ 实时获取并保存 {saved_count} 条新闻")
                # 重新查询
                news_list = await service.query_news(params)
                data_source = "realtime"
            else:
                logger.warning(f"⚠️ 实时获取新闻失败: {symbol}")
        except Exception as e:
            logger.error(f"❌ 实时获取新闻异常: {e}")
    return ok(data={
        "symbol": symbol,
        "hours_back": hours_back,
        "total_count": len(news_list),
        "news": news_list,
        "data_source": data_source
    },
    message=f"查询成功，返回 {len(news_list)} 条新闻（来源：{data_source}）"
    )
  except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"查询股票新闻失败: {str(e)}"
    )
@router.post("/query", response_model=dict)
async def query_news_advanced(
  request: NewsQueryRequest,
  current_user: dict = Depends(get_current_user)
):
  """
  高级新闻查询

  Args:
    request: 查询请求参数

  Returns:
    dict: 新闻数据列表
  """
  try:
    service = await get_news_data_service()

    # 构建查询参数
    params = NewsQueryParams(
        symbol=request.symbol,
        symbols=request.symbols,
        start_time=request.start_time,
        end_time=request.end_time,
        category=request.category,
        sentiment=request.sentiment,
        importance=request.importance,
        data_source=request.data_source,
        keywords=request.keywords,
        limit=request.limit,
        skip=request.skip
    )

    # 查询新闻
    news_list = await service.query_news(params)

    return ok(data={
        "query_params": request.dict(),
        "total_count": len(news_list),
        "news": news_list
    },
    message=f"高级查询成功，返回 {len(news_list)} 条新闻"
    )

  except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"高级新闻查询失败: {str(e)}"
    )
@router.get("/latest", response_model=dict)
async def get_latest_news(
  symbol: Optional[str] = Query(None, description="股票代码，为空则获取所有新闻"),
  limit: int = Query(10, description="返回数量限制"),
  hours_back: int = Query(24, description="回溯小时数"),
  current_user: dict = Depends(get_current_user)
):
  """
  获取最新新闻

  Args:
    symbol: 股票代码，为空则获取所有新闻
    limit: 返回数量限制
    hours_back: 回溯小时数

  Returns:
    dict: 最新新闻列表
  """
  try:
    service = await get_news_data_service()

    # 获取最新新闻
    news_list = await service.get_latest_news(
```

```
            symbol=symbol,
            limit=limit,
            hours_back=hours_back
        )

        return ok(data={
                "symbol": symbol,
                "limit": limit,
                "hours_back": hours_back,
                "total_count": len(news_list),
                "news": news_list
            },
            message=f"获取最新新闻成功，返回 {len(news_list)} 条"
        )

    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取最新新闻失败: {str(e)}"
        )
@router.get("/search", response_model=dict)
async def search_news(
    query: str = Query(..., description="搜索关键词"),
    symbol: Optional[str] = Query(None, description="股票代码过滤"),
    limit: int = Query(20, description="返回数量限制"),
    current_user: dict = Depends(get_current_user)
):
    """
    全文搜索新闻

    Args:
        query: 搜索关键词
        symbol: 股票代码过滤
        limit: 返回数量限制

    Returns:
        dict: 搜索结果列表
    """
    try:
        service = await get_news_data_service()

        # 全文搜索
        news_list = await service.search_news(
            query_text=query,
            symbol=symbol,
            limit=limit
        )

        return ok(data={
                "query": query,
                "symbol": symbol,
                "total_count": len(news_list),
                "news": news_list
            },
            message=f"搜索成功，返回 {len(news_list)} 条结果"
        )

    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"新闻搜索失败: {str(e)}"
        )
@router.get("/statistics", response_model=dict)
async def get_news_statistics(
    symbol: Optional[str] = Query(None, description="股票代码"),
    days_back: int = Query(7, description="回溯天数"),
    current_user: dict = Depends(get_current_user)
):
    """
    获取新闻统计信息

    Args:
        symbol: 股票代码
        days_back: 回溯天数

    Returns:
        dict: 新闻统计信息
    """
    try:
        service = await get_news_data_service()

        # 计算时间范围
        start_time = datetime.utcnow() - timedelta(days=days_back)

        # 获取统计信息
        stats = await service.get_news_statistics(
            symbol=symbol,
            start_time=start_time
        )

        return ok(data={
                "symbol": symbol,
                "days_back": days_back,
                "statistics": {
                    "total_count": stats.total_count,
                    "sentiment_distribution": {
                        "positive": stats.positive_count,
                        "negative": stats.negative_count,
                        "neutral": stats.neutral_count
                    },
                    "importance_distribution": {
                        "high": stats.high_importance_count,
                        "medium": stats.medium_importance_count,
                        "low": stats.low_importance_count
                    },
```

```
                "categories": stats.categories,
                "sources": stats.sources
            }
        },
        message="获取新闻统计成功"
    )

    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取新闻统计失败: {str(e)}"
        )
@router.post("/sync/start", response_model=dict)
async def start_news_sync(
    request: NewsSyncRequest,
    background_tasks: BackgroundTasks,
    current_user: dict = Depends(get_current_user)
):
    """
    启动新闻同步任务

    Args:
        request: 同步请求参数
        background_tasks: 后台任务

    Returns:
        dict: 任务启动结果
    """
    try:
        sync_service = await get_news_data_sync_service()

        # 添加后台同步任务
        if request.symbol:
            background_tasks.add_task(
                _execute_stock_news_sync,
                sync_service,
                request
            )
            message = f"股票 {request.symbol} 新闻同步任务已启动"
        else:
            background_tasks.add_task(
                _execute_market_news_sync,
                sync_service,
                request
            )
            message = "市场新闻同步任务已启动"

        return ok(data={
                "sync_type": "stock" if request.symbol else "market",
                "symbol": request.symbol,
                "data_sources": request.data_sources,
                "hours_back": request.hours_back,
                "max_news_per_source": request.max_news_per_source
            },
            message=message
        )

    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"启动新闻同步失败: {str(e)}"
        )
@router.post("/sync/single", response_model=dict)
async def sync_single_stock_news(
    symbol: str,
    data_sources: Optional[List[str]] = None,
    hours_back: int = 24,
    max_news_per_source: int = 50,
    current_user: dict = Depends(get_current_user)
):
    """
    同步单只股票新闻（同步执行）

    Args:
        symbol: 股票代码
        data_sources: 数据源列表
        hours_back: 回溯小时数
        max_news_per_source: 每个数据源最大新闻数量

    Returns:
        dict: 同步结果
    """
    try:
        sync_service = await get_news_data_sync_service()

        # 执行同步
        stats = await sync_service.sync_stock_news(
            symbol=symbol,
            data_sources=data_sources,
            hours_back=hours_back,
            max_news_per_source=max_news_per_source
        )

        return ok(data={
                "symbol": symbol,
                "sync_stats": {
                    "total_processed": stats.total_processed,
                    "successful_saves": stats.successful_saves,
                    "failed_saves": stats.failed_saves,
                    "duplicate_skipped": stats.duplicate_skipped,
                    "sources_used": stats.sources_used,
                    "duration_seconds": stats.duration_seconds,
                    "success_rate": stats.success_rate
                }
            },
```

```
            message=f"股票 {symbol} 新闻同步完成，成功保存 {stats.successful_saves} 条"
        )

    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"同步股票新闻失败: {str(e)}"
        )
@router.delete("/cleanup", response_model=dict)
async def cleanup_old_news(
    days_to_keep: int = Query(90, description="保留天数"),
    current_user: dict = Depends(get_current_user)
):
    """
    清理过期新闻

    Args:
        days_to_keep: 保留天数

    Returns:
        dict: 清理结果
    """
    try:
        service = await get_news_data_service()

        # 删除过期新闻
        deleted_count = await service.delete_old_news(days_to_keep)

        return ok(data={
                "days_to_keep": days_to_keep,
                "deleted_count": deleted_count
            },
            message=f"清理完成，删除 {deleted_count} 条过期新闻"
        )

    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"清理过期新闻失败: {str(e)}"
        )
@router.get("/health", response_model=dict)
async def health_check():
    """健康检查"""
    try:
        service = await get_news_data_service()
        sync_service = await get_news_data_sync_service()

        return ok(data={
                "service_status": "healthy",
                "timestamp": datetime.utcnow().isoformat()
            },
            message="新闻数据服务运行正常"
        )

    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"健康检查失败: {str(e)}"
        )
# 后台任务执行函数
async def _execute_stock_news_sync(sync_service, request: NewsSyncRequest):
    """执行股票新闻同步"""
    try:
        await sync_service.sync_stock_news(
            symbol=request.symbol,
            data_sources=request.data_sources,
            hours_back=request.hours_back,
            max_news_per_source=request.max_news_per_source
        )
    except Exception as e:
        logger.error(f"❌ 后台股票新闻同步失败: {e}")
async def _execute_market_news_sync(sync_service, request: NewsSyncRequest):
    """执行市场新闻同步"""
    try:
        await sync_service.sync_market_news(
            data_sources=request.data_sources,
            hours_back=request.hours_back,
            max_news_per_source=request.max_news_per_source
        )
    except Exception as e:
        logger.error(f"❌ 后台市场新闻同步失败: {e}")
```

FILE: app/routers/notifications.py

```
"""
通知 REST API
"""
import logging
from typing import Optional
from fastapi import APIRouter, Depends, HTTPException, Query
from app.routers.auth_db import get_current_user
from app.core.response import ok
from app.core.database import get_redis_client
from app.services.notifications_service import get_notifications_service
router = APIRouter()
logger = logging.getLogger("webapi.notifications")
@router.get("/notifications")
async def list_notifications(
    status: Optional[str] = Query(None, description="状态: unread|read|all"),
    type: Optional[str] = Query(None, description="类型: analysis|alert|system"),
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
```

```
    user: dict = Depends(get_current_user)
):
    svc = get_notifications_service()
    s = status if status in ("read","unread") else None
    t = type if type in ("analysis","alert","system") else None
    data = await svc.list(user_id=user["id"], status=s, ntype=t, page=page, page_size=page_size)
    return ok(data=data.model_dump(), message="ok")

@router.get("/notifications/unread_count")
async def get_unread_count(user: dict = Depends(get_current_user)):
    svc = get_notifications_service()
    cnt = await svc.unread_count(user_id=user["id"])
    return ok(data={"count": cnt})

@router.post("/notifications/{notif_id}/read")
async def mark_read(notif_id: str, user: dict = Depends(get_current_user)):
    svc = get_notifications_service()
    ok_flag = await svc.mark_read(user_id=user["id"], notif_id=notif_id)
    if not ok_flag:
        raise HTTPException(status_code=404, detail="Notification not found")
    return ok()

@router.post("/notifications/read_all")
async def mark_all_read(user: dict = Depends(get_current_user)):
    svc = get_notifications_service()
    n = await svc.mark_all_read(user_id=user["id"])
    return ok(data={"updated": n})

@router.get("/notifications/debug/redis_pool")
async def debug_redis_pool(user: dict = Depends(get_current_user)):
    """调试端点：查看 Redis 连接池状态"""
    try:
        r = get_redis_client()
        pool = r.connection_pool
        # 获取连接池信息
        pool_info = {
            "max_connections": pool.max_connections,
            "connection_class": str(pool.connection_class),
            "available_connections": len(pool._available_connections) if hasattr(pool, '_available_connections') else "N/A",
            "in_use_connections": len(pool._in_use_connections) if hasattr(pool, '_in_use_connections') else "N/A",
        }
        # 获取 Redis 服务器信息
        info = await r.info("clients")
        redis_info = {
            "connected_clients": info.get("connected_clients", "N/A"),
            "client_recent_max_input_buffer": info.get("client_recent_max_input_buffer", "N/A"),
            "client_recent_max_output_buffer": info.get("client_recent_max_output_buffer", "N/A"),
            "blocked_clients": info.get("blocked_clients", "N/A"),
        }
        # 🔥 新增：获取 PubSub 频道信息
        try:
            pubsub_info = await r.execute_command("PUBSUB", "CHANNELS", "notifications:*")
            pubsub_channels = {
                "active_channels": len(pubsub_info) if pubsub_info else 0,
                "channels": pubsub_info if pubsub_info else []
            }
        except Exception as e:
            logger.warning(f"获取 PubSub 频道信息失败: {e}")
            pubsub_channels = {"error": str(e)}
        return ok(data={
            "pool": pool_info,
            "redis_server": redis_info,
            "pubsub": pubsub_channels
        })
    except Exception as e:
        logger.error(f"获取 Redis 连接池信息失败: {e}", exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))
```

FILE: app/routers/operation_logs.py

```
"""
操作日志API路由
"""
import logging
from typing import Dict, Any
from fastapi import APIRouter, Depends, HTTPException, status, Query, Request
from fastapi.responses import StreamingResponse
from app.routers.auth_db import get_current_user
from app.services.operation_log_service import get_operation_log_service
from app.models.operation_log import (
    OperationLogQuery,
    OperationLogListResponse,
    OperationLogStatsResponse,
    ClearLogsRequest,
    ClearLogsResponse,
    OperationLogCreate
)
router = APIRouter(prefix="/logs", tags=["操作日志"])
logger = logging.getLogger("webapi")

@router.get("/list", response_model=OperationLogListResponse)
async def get_operation_logs(
    page: int = Query(1, ge=1, description="页码"),
    page_size: int = Query(20, ge=1, le=100, description="每页数量"),
    start_date: str = Query(None, description="开始日期"),
    end_date: str = Query(None, description="结束日期"),
    action_type: str = Query(None, description="操作类型"),
    success: bool = Query(None, description="是否成功"),
    keyword: str = Query(None, description="关键词搜索"),
    current_user: dict = Depends(get_current_user)
):
    """获取操作日志列表"""
    try:
        logger.info(f"🔍 用户 {current_user['username']} 获取操作日志列表")

        service = get_operation_log_service()
        query = OperationLogQuery(
```

```
            page=page,
            page_size=page_size,
            start_date=start_date,
            end_date=end_date,
            action_type=action_type,
            success=success,
            keyword=keyword
        )

        logs, total = await service.get_logs(query)

        return OperationLogListResponse(
            success=True,
            data={
                "logs": [log.dict() for log in logs],
                "total": total,
                "page": page,
                "page_size": page_size,
                "total_pages": (total + page_size - 1) // page_size
            },
            message="获取操作日志列表成功"
        )

    except Exception as e:
        logger.error(f"获取操作日志列表失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取操作日志列表失败: {str(e)}"
        )
@router.get("/stats", response_model=OperationLogStatsResponse)
async def get_operation_log_stats(
    days: int = Query(30, ge=1, le=365, description="统计天数"),
    current_user: dict = Depends(get_current_user)
):
    """获取操作日志统计"""
    try:
        logger.info(f"📊 用户 {current_user['username']} 获取操作日志统计")

        service = get_operation_log_service()
        stats = await service.get_stats(days)

        return OperationLogStatsResponse(
            success=True,
            data=stats,
            message="获取操作日志统计成功"
        )

    except Exception as e:
        logger.error(f"获取操作日志统计失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取操作日志统计失败: {str(e)}"
        )
@router.get("/{log_id}")
async def get_operation_log_detail(
    log_id: str,
    current_user: dict = Depends(get_current_user)
):
    """获取操作日志详情"""
    try:
        logger.info(f"🔍 用户 {current_user['username']} 获取操作日志详情: {log_id}")

        service = get_operation_log_service()
        log = await service.get_log_by_id(log_id)

        if not log:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="操作日志不存在"
            )

        return {
            "success": True,
            "data": log.dict(),
            "message": "获取操作日志详情成功"
        }

    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"获取操作日志详情失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取操作日志详情失败: {str(e)}"
        )
@router.post("/clear", response_model=ClearLogsResponse)
async def clear_operation_logs(
    request: ClearLogsRequest,
    current_user: dict = Depends(get_current_user)
):
    """清空操作日志"""
    try:
        logger.info(f"🗑️ 用户 {current_user['username']} 清空操作日志")

        service = get_operation_log_service()
        result = await service.clear_logs(
            days=request.days,
            action_type=request.action_type
        )

        message = f"清空操作日志成功，删除了 {result['deleted_count']} 条记录"
        if request.days:
            message += f"（{request.days}天前的日志）"
        if request.action_type:
            message += f"（类型: {request.action_type}）"
```

```
        return ClearLogsResponse(
            success=True,
            data=result,
            message=message
        )

    except Exception as e:
        logger.error(f"清空操作日志失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"清空操作日志失败: {str(e)}"
        )
@router.post("/create")
async def create_operation_log(
    log_data: OperationLogCreate,
    request: Request,
    current_user: dict = Depends(get_current_user)
):
    """手动创建操作日志"""
    try:
        logger.info(f"📝 用户 {current_user['username']} 手动创建操作日志")

        service = get_operation_log_service()

        # 获取客户端信息
        ip_address = request.client.host if request.client else None
        user_agent = request.headers.get("user-agent")

        log_id = await service.create_log(
            user_id=current_user["id"],
            username=current_user["username"],
            log_data=log_data,
            ip_address=ip_address,
            user_agent=user_agent
        )

        return {
            "success": True,
            "data": {"log_id": log_id},
            "message": "创建操作日志成功"
        }

    except Exception as e:
        logger.error(f"创建操作日志失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"创建操作日志失败: {str(e)}"
        )
@router.get("/export/csv")
async def export_logs_csv(
    start_date: str = Query(None, description="开始日期"),
    end_date: str = Query(None, description="结束日期"),
    action_type: str = Query(None, description="操作类型"),
    current_user: dict = Depends(get_current_user)
):
    """导出操作日志为CSV"""
    try:
        logger.info(f"📤 用户 {current_user['username']} 导出操作日志CSV")

        service = get_operation_log_service()
        query = OperationLogQuery(
            page=1,
            page_size=10000,  # 导出时获取更多数据
            start_date=start_date,
            end_date=end_date,
            action_type=action_type
        )

        logs, _ = await service.get_logs(query)

        # 生成CSV内容
        import csv
        import io

        output = io.StringIO()
        writer = csv.writer(output)

        # 写入表头
        writer.writerow([
            "时间", "用户", "操作类型", "操作内容", "状态", "耗时(ms)", "IP地址", "错误信息"
        ])

        # 写入数据
        for log in logs:
            writer.writerow([
                log.timestamp.strftime("%Y-%m-%d %H:%M:%S"),
                log.username,
                log.action_type,
                log.action,
                "成功" if log.success else "失败",
                log.duration_ms or "",
                log.ip_address or "",
                log.error_message or ""
            ])

        output.seek(0)

        # 返回CSV文件
        from datetime import datetime
        filename = f"operation_logs_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"

        return StreamingResponse(
            io.BytesIO(output.getvalue().encode('utf-8-sig')),
            media_type="text/csv",
```

```
            headers={"Content-Disposition": f"attachment; filename={filename}"}
        )

    except Exception as e:
        logger.error(f"导出操作日志CSV失败: {e}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"导出操作日志CSV失败: {str(e)}"
        )
```

FILE: app/routers/paper.py

```
from fastapi import APIRouter, Depends, HTTPException, status, Query
from pydantic import BaseModel, Field
from typing import Literal, Optional, Dict, Any, List, Tuple
from datetime import datetime
import logging
import re
from app.routers.auth_db import get_current_user
from app.core.database import get_mongo_db
from app.core.response import ok
router = APIRouter(prefix="/paper", tags=["paper"])
logger = logging.getLogger("webapi")
# 每个市场的初始资金配置
INITIAL_CASH_BY_MARKET = {
    "CNY": 1_000_000.0,  # A股：100万人民币
    "HKD": 1_000_000.0,  # 港股：100万港币
    "USD": 100_000.0     # 美股：10万美元
}
class PlaceOrderRequest(BaseModel):
    code: str = Field(..., description="股票代码（支持A股/港股/美股）")
    side: Literal["buy", "sell"]
    quantity: int = Field(..., gt=0)
    market: Optional[str] = Field(None, description="市场类型 (CN/HK/US)，不传则自动识别")
    # 可选：关联的分析ID，便于从分析页面一键下单后追踪
    analysis_id: Optional[str] = None
def _detect_market_and_code(code: str) -> Tuple[str, str]:
    """
    检测股票代码的市场类型并标准化代码
    Returns:
        (market, normalized_code): 市场类型和标准化后的代码
            - CN: A股（6位数字）
            - HK: 港股（4-5位数字或带.HK后缀）
            - US: 美股（字母代码）
    """
    code = code.strip().upper()
    # 港股：带 .HK 后缀
    if code.endswith('.HK'):
        return ('HK', code[:-3].zfill(5))
    # 美股：纯字母
    if re.match(r'^[A-Z]+$', code):
        return ('US', code)
    # 港股：4-5位数字
    if re.match(r'^\d{4,5}$', code):
        return ('HK', code.zfill(5))
    # A股：6位数字
    if re.match(r'^\d{6}$', code):
        return ('CN', code)
    # 默认当作A股，补齐6位
    return ('CN', code.zfill(6))
async def _get_or_create_account(user_id: str) -> Dict[str, Any]:
    """获取或创建账户（多货币）"""
    db = get_mongo_db()
    acc = await db["paper_accounts"].find_one({"user_id": user_id})
    if not acc:
        now = datetime.utcnow().isoformat()
        acc = {
            "user_id": user_id,
            # 多货币现金账户
            "cash": {
                "CNY": INITIAL_CASH_BY_MARKET["CNY"],
                "HKD": INITIAL_CASH_BY_MARKET["HKD"],
                "USD": INITIAL_CASH_BY_MARKET["USD"]
            },
            # 多货币已实现盈亏
            "realized_pnl": {
                "CNY": 0.0,
                "HKD": 0.0,
                "USD": 0.0
            },
            # 账户设置
            "settings": {
                "auto_currency_conversion": False,
                "default_market": "CN"
            },
            "created_at": now,
            "updated_at": now,
        }
        await db["paper_accounts"].insert_one(acc)
    else:
        # 兼容旧账户结构：如果 cash 或 realized_pnl 仍为标量，迁移为多货币对象
        updates: Dict[str, Any] = {}
        try:
            cash_val = acc.get("cash")
            if not isinstance(cash_val, dict):
                base_cash = float(cash_val or 0.0)
                updates["cash"] = {"CNY": base_cash, "HKD": 0.0, "USD": 0.0}
            pnl_val = acc.get("realized_pnl")
            if not isinstance(pnl_val, dict):
                base_pnl = float(pnl_val or 0.0)
                updates["realized_pnl"] = {"CNY": base_pnl, "HKD": 0.0, "USD": 0.0}
            if updates:
```

```
            updates["updated_at"] = datetime.utcnow().isoformat()
            await db["paper_accounts"].update_one({"user_id": user_id}, {"$set": updates})
            # 重新读取迁移后的账户
            acc = await db["paper_accounts"].find_one({"user_id": user_id})
        except Exception as e:
            logger.error(f"❌ 账户结构迁移失败 user_id={user_id}: {e}")
    return acc

async def _get_market_rules(market: str) -> Optional[Dict[str, Any]]:
    """获取市场规则配置"""
    db = get_mongo_db()
    rules_doc = await db["paper_market_rules"].find_one({"market": market})
    if rules_doc:
        return rules_doc.get("rules", {})
    return None

def _calculate_commission(market: str, side: str, amount: float, rules: Dict[str, Any]) -> float:
    """计算手续费"""
    if not rules or "commission" not in rules:
        return 0.0
    commission_config = rules["commission"]
    commission = 0.0
    # 佣金
    comm_rate = commission_config.get("rate", 0.0)
    comm_min = commission_config.get("min", 0.0)
    commission += max(amount * comm_rate, comm_min)
    # 印花税（仅卖出）
    if side == "sell" and "stamp_duty_rate" in commission_config:
        commission += amount * commission_config["stamp_duty_rate"]
    # 其他费用（港股）
    if market == "HK":
        if "transaction_levy_rate" in commission_config:
            commission += amount * commission_config["transaction_levy_rate"]
        if "trading_fee_rate" in commission_config:
            commission += amount * commission_config["trading_fee_rate"]
        if "settlement_fee_rate" in commission_config:
            commission += amount * commission_config["settlement_fee_rate"]
    # SEC费用（美股，仅卖出）
    if market == "US" and side == "sell" and "sec_fee_rate" in commission_config:
        commission += amount * commission_config["sec_fee_rate"]
    return round(commission, 2)

async def _get_available_quantity(user_id: str, code: str, market: str) -> int:
    """获取可用数量（考虑T+1限制）"""
    db = get_mongo_db()
    pos = await db["paper_positions"].find_one({"user_id": user_id, "code": code})
    if not pos:
        return 0
    total_qty = pos.get("quantity", 0)
    # A股T+1：今天买入的不能卖出
    if market == "CN":
        # 获取市场规则
        rules = await _get_market_rules(market)
        if rules and rules.get("t_plus", 0) > 0:
            # 查询今天的买入数量
            today = datetime.utcnow().date().isoformat()
            pipeline = [
                {"$match": {
                    "user_id": user_id,
                    "code": code,
                    "side": "buy",
                    "timestamp": {"$gte": today}
                }},
                {"$group": {"_id": None, "total": {"$sum": "$quantity"}}}
            ]
            today_buy = await db["paper_trades"].aggregate(pipeline).to_list(1)
            today_buy_qty = today_buy[0]["total"] if today_buy else 0
            return max(0, total_qty - today_buy_qty)
    # 港股/美股T+0：全部可用
    return total_qty

async def _get_last_price(code: str, market: str) -> Optional[float]:
    """
    获取股票最新价格（支持多市场）
    Args:
        code: 股票代码
        market: 市场类型 (CN/HK/US)
    Returns:
        最新价格，如果获取失败返回 None
    """
    db = get_mongo_db()
    # A股：从数据库获取
    if market == "CN":
        # 1. 尝试从 market_quotes 获取
        q = await db["market_quotes"].find_one(
            {"$or": [{"code": code}, {"symbol": code}]},
            {"_id": 0, "close": 1}
        )
        if q and q.get("close") is not None:
            try:
                price = float(q["close"])
                if price > 0:
                    logger.debug(f"☑ 从 market_quotes 获取价格: {code} = {price}")
                    return price
            except Exception as e:
                logger.warning(f"⚠ market_quotes 价格转换失败 {code}: {e}")
        # 2. 回退到 stock_basic_info 的 current_price
        basic_info = await db["stock_basic_info"].find_one(
            {"$or": [{"code": code}, {"symbol": code}]},
            {"_id": 0, "current_price": 1}
        )
        if basic_info and basic_info.get("current_price") is not None:
            try:
                price = float(basic_info["current_price"])
                if price > 0:
                    logger.debug(f"☑ 从 stock_basic_info 获取价格: {code} = {price}")
```

```
            return price
        except Exception as e:
            logger.warning(f"⚠️ stock_basic_info 价格转换失败 {code}: {e}")
    logger.error(f"❌ 无法从数据库获取A股价格: {code}")
    return None
# 港股/美股：使用 ForeignStockService
elif market in ['HK', 'US']:
    try:
        from app.services.foreign_stock_service import ForeignStockService
        db = get_mongo_db()
        service = ForeignStockService(db=db)
        quote = await service.get_quote(market, code, force_refresh=False)
        if quote:
            # 尝试多个可能的价格字段
            price = quote.get("price") or quote.get("current_price") or quote.get("close")
            if price and float(price) > 0:
                logger.debug(f"✅ 从 ForeignStockService 获取{market}价格: {code} = {price}")
                return float(price)
    except Exception as e:
        logger.error(f"❌ 获取{market}股价格失败 {code}: {e}")
        return None
logger.error(f"❌ 无法获取股票价格: {code} (market={market})")
return None

def _zfill_code(code: str) -> str:
    s = str(code).strip()
    if len(s) == 6 and s.isdigit():
        return s
    return s.zfill(6)

@router.get("/account", response_model=dict)
async def get_account(current_user: dict = Depends(get_current_user)):
    """获取或创建纸上账户，返回资金与持仓估值汇总（支持多市场）"""
    db = get_mongo_db()
    acc = await _get_or_create_account(current_user["id"])
    # 聚合持仓估值（按货币分类）
    positions = await db["paper_positions"].find({"user_id": current_user["id"]}).to_list(None)
    positions_value_by_currency = {
        "CNY": 0.0,
        "HKD": 0.0,
        "USD": 0.0
    }
    detailed_positions: List[Dict[str, Any]] = []
    for p in positions:
        code = p.get("code")
        market = p.get("market", "CN")
        currency = p.get("currency", "CNY")
        qty = int(p.get("quantity", 0))
        avg_cost = float(p.get("avg_cost", 0.0))
        available_qty = p.get("available_qty", qty)
        # 获取最新价
        last = await _get_last_price(code, market)
        mkt_value = round((last or 0.0) * qty, 2)
        positions_value_by_currency[currency] += mkt_value
        detailed_positions.append({
            "code": code,
            "market": market,
            "currency": currency,
            "quantity": qty,
            "available_qty": available_qty,
            "avg_cost": avg_cost,
            "last_price": last,
            "market_value": mkt_value,
            "unrealized_pnl": None if last is None else round((last - avg_cost) * qty, 2)
        })
    # 计算总资产（按货币分别显示）
    cash = acc.get("cash", {})
    realized_pnl = acc.get("realized_pnl", {})
    # 兼容旧格式（单一现金）
    if not isinstance(cash, dict):
        cash = {"CNY": float(cash), "HKD": 0.0, "USD": 0.0}
    if not isinstance(realized_pnl, dict):
        realized_pnl = {"CNY": float(realized_pnl), "HKD": 0.0, "USD": 0.0}
    summary = {
        "cash": {
            "CNY": round(float(cash.get("CNY", 0.0)), 2),
            "HKD": round(float(cash.get("HKD", 0.0)), 2),
            "USD": round(float(cash.get("USD", 0.0)), 2)
        },
        "realized_pnl": {
            "CNY": round(float(realized_pnl.get("CNY", 0.0)), 2),
            "HKD": round(float(realized_pnl.get("HKD", 0.0)), 2),
            "USD": round(float(realized_pnl.get("USD", 0.0)), 2)
        },
        "positions_value": positions_value_by_currency,
        "equity": {
            "CNY": round(float(cash.get("CNY", 0.0)) + positions_value_by_currency["CNY"], 2),
            "HKD": round(float(cash.get("HKD", 0.0)) + positions_value_by_currency["HKD"], 2),
            "USD": round(float(cash.get("USD", 0.0)) + positions_value_by_currency["USD"], 2)
        },
        "updated_at": acc.get("updated_at"),
    }
    return ok({"account": summary, "positions": detailed_positions})

@router.post("/order", response_model=dict)
async def place_order(payload: PlaceOrderRequest, current_user: dict = Depends(get_current_user)):
    """提交市价单，按最新价即时成交（支持多市场）"""
    db = get_mongo_db()
    # 1. 识别市场类型
    if payload.market:
        market = payload.market.upper()
        normalized_code = payload.code
    else:
        market, normalized_code = _detect_market_and_code(payload.code)
    side = payload.side
```

```
qty = int(payload.quantity)
analysis_id = getattr(payload, "analysis_id", None)
# 2. 确定货币
currency_map = {
    "CN": "CNY",
    "HK": "HKD",
    "US": "USD"
}
currency = currency_map.get(market, "CNY")
# 3. 获取账户
acc = await _get_or_create_account(current_user["id"])
# 4. 获取价格
price = await _get_last_price(normalized_code, market)
if price is None or price <= 0:
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail=f"无法获取股票 {normalized_code} ({market}) 的最新价格"
    )
# 5. 计算金额
notional = round(price * qty, 2)
# 6. 获取市场规则并计算手续费
rules = await _get_market_rules(market)
commission = _calculate_commission(market, side, notional, rules) if rules else 0.0
total_cost = notional + commission
# 7. 获取持仓
pos = await db["paper_positions"].find_one({"user_id": current_user["id"], "code": normalized_code})
now_iso = datetime.utcnow().isoformat()
realized_pnl_delta = 0.0
# 8. 执行买卖逻辑
if side == "buy":
    # 资金检查（使用对应货币的账户）
    cash = acc.get("cash", {})
    if isinstance(cash, dict):
        available_cash = float(cash.get(currency, 0.0))
    else:
        # 兼容旧格式
        available_cash = float(cash) if currency == "CNY" else 0.0
    if available_cash < total_cost:
        raise HTTPException(
            status_code=400,
            detail=f"可用{currency}不足：需要 {total_cost:.2f}，可用 {available_cash:.2f}"
        )
    # 扣除资金（从对应货币账户）
    new_cash = round(available_cash - total_cost, 2)
    await db["paper_accounts"].update_one(
        {"user_id": current_user["id"]},
        {"$set": {f"cash.{currency}": new_cash, "updated_at": now_iso}}
    )
    # 更新/创建持仓：加权平均成本
    if not pos:
        new_pos = {
            "user_id": current_user["id"],
            "code": normalized_code,
            "market": market,
            "currency": currency,
            "quantity": qty,
            "available_qty": qty if market != "CN" else 0,  # A股T+1，今天买入不可用
            "frozen_qty": 0,
            "avg_cost": price,
            "updated_at": now_iso
        }
        await db["paper_positions"].insert_one(new_pos)
    else:
        old_qty = int(pos.get("quantity", 0))
        old_cost = float(pos.get("avg_cost", 0.0))
        new_qty = old_qty + qty
        new_avg = round((old_cost * old_qty + price * qty) / new_qty, 4) if new_qty > 0 else price
        # A股T+1：新买入的不可用
        if market == "CN":
            new_available = pos.get("available_qty", old_qty)  # 保持原有可用数量
        else:
            new_available = new_qty  # 港股/美股T+0，全部可用
        await db["paper_positions"].update_one(
            {"_id": pos["_id"]},
            {"$set": {
                "quantity": new_qty,
                "available_qty": new_available,
                "avg_cost": new_avg,
                "updated_at": now_iso
            }}
        )
else:  # sell
    # 检查可用数量（考虑T+1）
    available_qty = await _get_available_quantity(current_user["id"], normalized_code, market)
    if available_qty < qty:
        raise HTTPException(
            status_code=400,
            detail=f"可用持仓不足：需要 {qty}，可用 {available_qty}"
        )
    old_qty = int(pos.get("quantity", 0))
    avg_cost = float(pos.get("avg_cost", 0.0))
    new_qty = old_qty - qty
    pnl = round((price - avg_cost) * qty, 2)
    realized_pnl_delta = pnl
    # 卖出收入（加到对应货币账户，扣除手续费）
    net_proceeds = notional - commission
    await db["paper_accounts"].update_one(
        {"user_id": current_user["id"]},
        {
            "$inc": {
                f"cash.{currency}": net_proceeds,
                f"realized_pnl.{currency}": realized_pnl_delta
            },
```

```
            "$set": {"updated_at": now_iso}
        }
    )
    # 更新持仓
    if new_qty == 0:
        await db["paper_positions"].delete_one({"_id": pos["_id"]})
    else:
        new_available = max(0, pos.get("available_qty", old_qty) - qty)
        await db["paper_positions"].update_one(
            {"_id": pos["_id"]},
            {"$set": {
                "quantity": new_qty,
                "available_qty": new_available,
                "updated_at": now_iso
            }}
        )
# 9. 记录订单与成交（即成）
order_doc = {
    "user_id": current_user["id"],
    "code": normalized_code,
    "market": market,
    "currency": currency,
    "side": side,
    "quantity": qty,
    "price": price,
    "amount": notional,
    "commission": commission,
    "status": "filled",
    "created_at": now_iso,
    "filled_at": now_iso,
}
if analysis_id:
    order_doc["analysis_id"] = analysis_id
await db["paper_orders"].insert_one(order_doc)
trade_doc = {
    "user_id": current_user["id"],
    "code": normalized_code,
    "market": market,
    "currency": currency,
    "side": side,
    "quantity": qty,
    "price": price,
    "amount": notional,
    "commission": commission,
    "pnl": realized_pnl_delta if side == "sell" else 0.0,
    "timestamp": now_iso,
}
if analysis_id:
    trade_doc["analysis_id"] = analysis_id
await db["paper_trades"].insert_one(trade_doc)
return ok({"order": {k: v for k, v in order_doc.items() if k != "_id"}})

@router.get("/positions", response_model=dict)
async def list_positions(current_user: dict = Depends(get_current_user)):
    """获取持仓列表（支持多市场）"""
    db = get_mongo_db()
    items = await db["paper_positions"].find({"user_id": current_user["id"]}).to_list(None)
    enriched: List[Dict[str, Any]] = []
    for p in items:
        code = p.get("code")
        market = p.get("market", "CN")
        currency = p.get("currency", "CNY")
        qty = int(p.get("quantity", 0))
        available_qty = p.get("available_qty", qty)
        avg_cost = float(p.get("avg_cost", 0.0))
        last = await _get_last_price(code, market)
        mkt = round((last or 0.0) * qty, 2)
        enriched.append({
            "code": code,
            "market": market,
            "currency": currency,
            "quantity": qty,
            "available_qty": available_qty,
            "avg_cost": avg_cost,
            "last_price": last,
            "market_value": mkt,
            "unrealized_pnl": None if last is None else round((last - avg_cost) * qty, 2)
        })
    return ok({"items": enriched})

@router.get("/orders", response_model=dict)
async def list_orders(limit: int = Query(50, ge=1, le=200), current_user: dict = Depends(get_current_user)):
    db = get_mongo_db()
    cursor = db["paper_orders"].find({"user_id": current_user["id"]}).sort("created_at", -1).limit(limit)
    items = await cursor.to_list(None)
    # 去除 _id
    cleaned = [{k: v for k, v in it.items() if k != "_id"} for it in items]
    return ok({"items": cleaned})

@router.post("/reset", response_model=dict)
async def reset_account(confirm: bool = Query(False), current_user: dict = Depends(get_current_user)):
    """重置账户（支持多货币）"""
    if not confirm:
        raise HTTPException(status_code=400, detail="请设置 confirm=true 以确认重置")
    db = get_mongo_db()
    await db["paper_accounts"].delete_many({"user_id": current_user["id"]})
    await db["paper_positions"].delete_many({"user_id": current_user["id"]})
    await db["paper_orders"].delete_many({"user_id": current_user["id"]})
    await db["paper_trades"].delete_many({"user_id": current_user["id"]})
    # 重新创建账户
    acc = await _get_or_create_account(current_user["id"])
    return ok({"message": "账户已重置", "cash": acc.get("cash", {})})
```

FILE: app/routers/queue.py

```
from fastapi import APIRouter, Depends
```

```
from app.routers.auth_db import get_current_user
from app.services.queue_service import get_queue_service, QueueService
router = APIRouter()
@router.get("/stats")
async def queue_stats(user: dict = Depends(get_current_user), svc: QueueService = Depends(get_queue_service)):
    stats = await svc.stats()
    return {"user": user["id"], **stats}
```

FILE: app/routers/reports.py

```
"""
分析报告管理API路由
"""
import os
import json
from datetime import datetime, timedelta
from typing import List, Optional, Dict, Any
from pathlib import Path
from fastapi import APIRouter, Depends, HTTPException, Query, Response
from fastapi.responses import FileResponse, StreamingResponse
from pydantic import BaseModel
from .auth_db import get_current_user
from ..core.database import get_mongo_db
from ..utils.timezone import to_config_tz
import logging
logger = logging.getLogger("webapi")
# 股票名称缓存
_stock_name_cache = {}
def get_stock_name(stock_code: str) -> str:
    """
    获取股票名称
    优先级：缓存 -> MongoDB（按数据源优先级） -> 默认返回股票代码
    """
    global _stock_name_cache
    # 检查缓存
    if stock_code in _stock_name_cache:
        return _stock_name_cache[stock_code]
    try:
        # 从 MongoDB 获取股票名称
        from ..core.database import get_mongo_db_sync
        from ..core.unified_config import UnifiedConfigManager
        db = get_mongo_db_sync()
        code6 = str(stock_code).zfill(6)
        # 🔥 按数据源优先级查询
        config = UnifiedConfigManager()
        data_source_configs = config.get_data_source_configs()
        # 提取启用的数据源，按优先级排序
        enabled_sources = [
            ds.type.lower() for ds in data_source_configs
            if ds.enabled and ds.type.lower() in ['tushare', 'akshare', 'baostock']
        ]
        if not enabled_sources:
            enabled_sources = ['tushare', 'akshare', 'baostock']
        # 按数据源优先级查询
        stock_info = None
        for data_source in enabled_sources:
            stock_info = db.stock_basic_info.find_one(
                {"$or": [{"symbol": code6}, {"code": code6}], "source": data_source}
            )
            if stock_info:
                logger.debug(f"✅ 使用数据源 {data_source} 获取股票名称 {code6}")
                break
        # 如果所有数据源都没有，尝试不带 source 条件查询（兼容旧数据）
        if not stock_info:
            stock_info = db.stock_basic_info.find_one(
                {"$or": [{"symbol": code6}, {"code": code6}]}
            )
            if stock_info:
                logger.warning(f"⚠️ 使用旧数据（无 source 字段）获取股票名称 {code6}")
        if stock_info and stock_info.get("name"):
            stock_name = stock_info["name"]
            _stock_name_cache[stock_code] = stock_name
            return stock_name
        # 如果没有找到，返回股票代码
        _stock_name_cache[stock_code] = stock_code
        return stock_code
    except Exception as e:
        logger.warning(f"⚠️ 获取股票名称失败 {stock_code}: {e}")
        return stock_code
# 统一构建报告查询：支持 _id(ObjectId) / analysis_id / task_id 三种
def _build_report_query(report_id: str) -> Dict[str, Any]:
    ors = [
        {"analysis_id": report_id},
        {"task_id": report_id},
    ]
    try:
        from bson import ObjectId
        ors.append({"_id": ObjectId(report_id)})
    except Exception:
        pass
    return {"$or": ors}
router = APIRouter(prefix="/api/reports", tags=["reports"])
class ReportFilter(BaseModel):
    """报告筛选参数"""
    search_keyword: Optional[str] = None
    market_filter: Optional[str] = None
    start_date: Optional[str] = None
    end_date: Optional[str] = None
    stock_code: Optional[str] = None
    report_type: Optional[str] = None
class ReportListResponse(BaseModel):
```

```
    """报告列表响应"""
    reports: List[Dict[str, Any]]
    total: int
    page: int
    page_size: int
@router.get("/list", response_model=Dict[str, Any])
async def get_reports_list(
    page: int = Query(1, ge=1, description="页码"),
    page_size: int = Query(20, ge=1, le=100, description="每页数量"),
    search_keyword: Optional[str] = Query(None, description="搜索关键词"),
    market_filter: Optional[str] = Query(None, description="市场筛选（A股/港股/美股）"),
    start_date: Optional[str] = Query(None, description="开始日期"),
    end_date: Optional[str] = Query(None, description="结束日期"),
    stock_code: Optional[str] = Query(None, description="股票代码"),
    user: dict = Depends(get_current_user)
):
    """获取分析报告列表"""
    try:
        logger.info(f"🔍 获取报告列表: 用户={user['id']}, 页码={page}, 每页={page_size}, 市场={market_filter}")
        db = get_mongo_db()
        # 构建查询条件
        query = {}
        # 搜索关键词
        if search_keyword:
            query["$or"] = [
                {"stock_symbol": {"$regex": search_keyword, "$options": "i"}},
                {"analysis_id": {"$regex": search_keyword, "$options": "i"}},
                {"summary": {"$regex": search_keyword, "$options": "i"}}
            ]
        # 市场筛选
        if market_filter:
            query["market_type"] = market_filter
        # 股票代码筛选
        if stock_code:
            query["stock_symbol"] = stock_code
        # 日期范围筛选
        if start_date or end_date:
            date_query = {}
            if start_date:
                date_query["$gte"] = start_date
            if end_date:
                date_query["$lte"] = end_date
            query["analysis_date"] = date_query
        logger.info(f"📊 查询条件: {query}")
        # 计算总数
        total = await db.analysis_reports.count_documents(query)
        # 分页查询
        skip = (page - 1) * page_size
        cursor = db.analysis_reports.find(query).sort("created_at", -1).skip(skip).limit(page_size)
        reports = []
        async for doc in cursor:
            # 转换为前端需要的格式
            stock_code = doc.get("stock_symbol", "")
            # 🔥 优先使用MongoDB中保存的股票名称，如果没有则查询
            stock_name = doc.get("stock_name")
            if not stock_name:
                stock_name = get_stock_name(stock_code)
            # 🔥 获取市场类型，如果没有则根据股票代码推断
            market_type = doc.get("market_type")
            if not market_type:
                from tradingagents.utils.stock_utils import StockUtils
                market_info = StockUtils.get_market_info(stock_code)
                market_type_map = {
                    "china_a": "A股",
                    "hong_kong": "港股",
                    "us": "美股",
                    "unknown": "A股"
                }
                market_type = market_type_map.get(market_info.get("market", "unknown"), "A股")
            # 获取创建时间（数据库中是 UTC 时间，需要转换为 UTC+8）
            created_at = doc.get("created_at", datetime.utcnow())
            created_at_tz = to_config_tz(created_at)  # 转换为 UTC+8 并添加时区信息
            report = {
                "id": str(doc["_id"]),
                "analysis_id": doc.get("analysis_id", ""),
                "title": f"{stock_name}({stock_code}) 分析报告",
                "stock_code": stock_code,
                "stock_name": stock_name,
                "market_type": market_type,  # 🔥 添加市场类型字段
                "model_info": doc.get("model_info", "Unknown"),  # 🔥 添加模型信息字段
                "type": "single",  # 目前主要是单股分析
                "format": "markdown",  # 主要格式
                "status": doc.get("status", "completed"),
                "created_at": created_at_tz.isoformat() if created_at_tz else str(created_at),
                "analysis_date": doc.get("analysis_date", ""),
                "analysts": doc.get("analysts", []),
                "research_depth": doc.get("research_depth", 1),
                "summary": doc.get("summary", ""),
                "file_size": len(str(doc.get("reports", {}))),  # 估算大小
                "source": doc.get("source", "unknown"),
                "task_id": doc.get("task_id", "")
            }
            reports.append(report)
        logger.info(f"✅ 查询完成: 总数={total}, 返回={len(reports)}")
        return {
            "success": True,
            "data": {
                "reports": reports,
                "total": total,
                "page": page,
                "page_size": page_size
            },
            "message": "报告列表获取成功"
```

```
        }
    except Exception as e:
        logger.error(f"❌ 获取报告列表失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
@router.get("/{report_id}/detail")
async def get_report_detail(
    report_id: str,
    user: dict = Depends(get_current_user)
):
    """获取报告详情"""
    try:
        logger.info(f"🔍 获取报告详情: {report_id}")
        db = get_mongo_db()
        # 支持 ObjectId / analysis_id / task_id
        query = _build_report_query(report_id)
        doc = await db.analysis_reports.find_one(query)
        if not doc:
            # 兜底：从 analysis_tasks.result 中还原报告详情
            logger.info(f"⚠️ 未在analysis_reports找到，尝试从analysis_tasks还原: {report_id}")
            tasks_doc = await db.analysis_tasks.find_one(
                {"$or": [{"task_id": report_id}, {"result.analysis_id": report_id}]},
                {"result": 1, "task_id": 1, "stock_code": 1, "created_at": 1, "completed_at": 1}
            )
            if not tasks_doc or not tasks_doc.get("result"):
                raise HTTPException(status_code=404, detail="报告不存在")
            r = tasks_doc["result"] or {}
            created_at = tasks_doc.get("created_at")
            updated_at = tasks_doc.get("completed_at") or created_at
            # 转换时区：数据库中是 UTC 时间，转换为 UTC+8
            created_at_tz = to_config_tz(created_at)
            updated_at_tz = to_config_tz(updated_at)
            def to_iso(x):
                if hasattr(x, "isoformat"):
                    return x.isoformat()
                return x or ""
            stock_symbol = r.get("stock_symbol", r.get("stock_code", tasks_doc.get("stock_code", "")))
            stock_name = r.get("stock_name")
            if not stock_name:
                stock_name = get_stock_name(stock_symbol)
            report = {
                "id": tasks_doc.get("task_id", report_id),
                "analysis_id": r.get("analysis_id", ""),
                "stock_symbol": stock_symbol,
                "stock_name": stock_name,  # 🔥 添加股票名称字段
                "model_info": r.get("model_info", "Unknown"),  # 🔥 添加模型信息字段
                "analysis_date": r.get("analysis_date", ""),
                "status": r.get("status", "completed"),
                "created_at": to_iso(created_at_tz),
                "updated_at": to_iso(updated_at_tz),
                "analysts": r.get("analysts", []),
                "research_depth": r.get("research_depth", 1),
                "summary": r.get("summary", ""),
                "reports": r.get("reports", {}),
                "source": "analysis_tasks",
                "task_id": tasks_doc.get("task_id", report_id),
                "recommendation": r.get("recommendation", ""),
                "confidence_score": r.get("confidence_score", 0.0),
                "risk_level": r.get("risk_level", "中等"),
                "key_points": r.get("key_points", []),
                "execution_time": r.get("execution_time", 0),
                "tokens_used": r.get("tokens_used", 0)
            }
        else:
            # 转换为详细格式（analysis_reports 命中）
            stock_symbol = doc.get("stock_symbol", "")
            stock_name = doc.get("stock_name")
            if not stock_name:
                stock_name = get_stock_name(stock_symbol)
            # 获取时间（数据库中是 UTC 时间，需要转换为 UTC+8）
            created_at = doc.get("created_at", datetime.utcnow())
            updated_at = doc.get("updated_at", datetime.utcnow())
            # 转换时区：数据库中是 UTC 时间，转换为 UTC+8
            created_at_tz = to_config_tz(created_at)
            updated_at_tz = to_config_tz(updated_at)
            report = {
                "id": str(doc["_id"]),
                "analysis_id": doc.get("analysis_id", ""),
                "stock_symbol": stock_symbol,
                "stock_name": stock_name,  # 🔥 添加股票名称字段
                "model_info": doc.get("model_info", "Unknown"),  # 🔥 添加模型信息字段
                "analysis_date": doc.get("analysis_date", ""),
                "status": doc.get("status", "completed"),
                "created_at": created_at_tz.isoformat() if created_at_tz else str(created_at),
                "updated_at": updated_at_tz.isoformat() if updated_at_tz else str(updated_at),
                "analysts": doc.get("analysts", []),
                "research_depth": doc.get("research_depth", 1),
                "summary": doc.get("summary", ""),
                "reports": doc.get("reports", {}),
                "source": doc.get("source", "unknown"),
                "task_id": doc.get("task_id", ""),
                "recommendation": doc.get("recommendation", ""),
                "confidence_score": doc.get("confidence_score", 0.0),
                "risk_level": doc.get("risk_level", "中等"),
                "key_points": doc.get("key_points", []),
                "execution_time": doc.get("execution_time", 0),
                "tokens_used": doc.get("tokens_used", 0)
            }
        return {
            "success": True,
            "data": report,
            "message": "报告详情获取成功"
        }
    except HTTPException:
```

```
        raise
    except Exception as e:
        logger.error(f"❌ 获取报告详情失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
@router.get("/{report_id}/content/{module}")
async def get_report_module_content(
    report_id: str,
    module: str,
    user: dict = Depends(get_current_user)
):
    """获取报告特定模块的内容"""
    try:
        logger.info(f"🔍 获取报告模块内容: {report_id}/{module}")
        db = get_mongo_db()
        # 查询报告（支持多种ID）
        query = _build_report_query(report_id)
        doc = await db.analysis_reports.find_one(query)
        if not doc:
            raise HTTPException(status_code=404, detail="报告不存在")
        reports = doc.get("reports", {})
        if module not in reports:
            raise HTTPException(status_code=404, detail=f"模块 {module} 不存在")
        content = reports[module]
        return {
            "success": True,
            "data": {
                "module": module,
                "content": content,
                "content_type": "markdown" if isinstance(content, str) else "json"
            },
            "message": "模块内容获取成功"
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 获取报告模块内容失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
@router.delete("/{report_id}")
async def delete_report(
    report_id: str,
    user: dict = Depends(get_current_user)
):
    """删除报告"""
    try:
        logger.info(f"🗑️ 删除报告: {report_id}")
        db = get_mongo_db()
        # 查询报告（支持多种ID）
        query = _build_report_query(report_id)
        result = await db.analysis_reports.delete_one(query)
        if result.deleted_count == 0:
            raise HTTPException(status_code=404, detail="报告不存在")
        logger.info(f"✅ 报告删除成功: {report_id}")
        return {
            "success": True,
            "message": "报告删除成功"
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 删除报告失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
@router.get("/{report_id}/download")
async def download_report(
    report_id: str,
    format: str = Query("markdown", description="下载格式: markdown, json, pdf, docx"),
    user: dict = Depends(get_current_user)
):
    """下载报告
    支持的格式:
    - markdown: Markdown 格式（默认）
    - json: JSON 格式（包含完整数据）
    - docx: Word 文档格式（需要 pandoc）
    - pdf: PDF 格式（需要 pandoc 和 PDF 引擎）
    """
    try:
        logger.info(f"📥 下载报告: {report_id}, 格式: {format}")
        db = get_mongo_db()
        # 查询报告（支持多种ID）
        query = _build_report_query(report_id)
        doc = await db.analysis_reports.find_one(query)
        if not doc:
            raise HTTPException(status_code=404, detail="报告不存在")
        stock_symbol = doc.get("stock_symbol", "unknown")
        analysis_date = doc.get("analysis_date", datetime.now().strftime("%Y-%m-%d"))
        if format == "json":
            # JSON格式下载
            content = json.dumps(doc, ensure_ascii=False, indent=2, default=str)
            filename = f"{stock_symbol}_{analysis_date}_report.json"
            media_type = "application/json"
            # 返回文件流
            def generate():
                yield content.encode('utf-8')
            return StreamingResponse(
                generate(),
                media_type=media_type,
                headers={"Content-Disposition": f"attachment; filename={filename}"}
            )
        elif format == "markdown":
            # Markdown格式下载
            reports = doc.get("reports", {})
            content_parts = []
            # 添加标题
```

```
        content_parts.append(f"# {stock_symbol} 分析报告")
        content_parts.append(f"**分析日期**: {analysis_date}")
        content_parts.append(f"**分析师**: {', '.join(doc.get('analysts', []))}")
        content_parts.append(f"**研究深度**: {doc.get('research_depth', 1)}")
        content_parts.append("")
        # 添加摘要
        if doc.get("summary"):
            content_parts.append("## 执行摘要")
            content_parts.append(doc["summary"])
            content_parts.append("")
        # 添加各模块内容
        for module_name, module_content in reports.items():
            if isinstance(module_content, str) and module_content.strip():
                content_parts.append(f"## {module_name}")
                content_parts.append(module_content)
                content_parts.append("")
        content = "\n".join(content_parts)
        filename = f"{stock_symbol}_{analysis_date}_report.md"
        media_type = "text/markdown"
        # 返回文件流
        def generate():
            yield content.encode('utf-8')
        return StreamingResponse(
            generate(),
            media_type=media_type,
            headers={"Content-Disposition": f"attachment; filename={filename}"}
        )
    elif format == "docx":
        # Word 文档格式下载
        from app.utils.report_exporter import report_exporter
        if not report_exporter.pandoc_available:
            raise HTTPException(
                status_code=400,
                detail="Word 导出功能不可用。请安装 pandoc: pip install pypandoc"
            )
        try:
            # 生成 Word 文档
            docx_content = report_exporter.generate_docx_report(doc)
            filename = f"{stock_symbol}_{analysis_date}_report.docx"
            # 返回文件流
            def generate():
                yield docx_content
            return StreamingResponse(
                generate(),
                media_type="application/vnd.openxmlformats-officedocument.wordprocessingml.document",
                headers={"Content-Disposition": f"attachment; filename={filename}"}
            )
        except Exception as e:
            logger.error(f"❌ Word 文档生成失败: {e}")
            raise HTTPException(status_code=500, detail=f"Word 文档生成失败: {str(e)}")
    elif format == "pdf":
        # PDF 格式下载
        from app.utils.report_exporter import report_exporter
        if not report_exporter.pandoc_available:
            raise HTTPException(
                status_code=400,
                detail="PDF 导出功能不可用。请安装 pandoc 和 PDF 引擎（wkhtmltopdf 或 LaTeX）"
            )
        try:
            # 生成 PDF 文档
            pdf_content = report_exporter.generate_pdf_report(doc)
            filename = f"{stock_symbol}_{analysis_date}_report.pdf"
            # 返回文件流
            def generate():
                yield pdf_content
            return StreamingResponse(
                generate(),
                media_type="application/pdf",
                headers={"Content-Disposition": f"attachment; filename={filename}"}
            )
        except Exception as e:
            logger.error(f"❌ PDF 文档生成失败: {e}")
            raise HTTPException(status_code=500, detail=f"PDF 文档生成失败: {str(e)}")
    else:
        raise HTTPException(status_code=400, detail=f"不支持的下载格式: {format}")
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"❌ 下载报告失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
```

FILE: app/routers/scheduler.py

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
定时任务管理路由
提供定时任务的查询、暂停、恢复、手动触发等功能
"""
from fastapi import APIRouter, HTTPException, Depends, Query
from typing import List, Dict, Any, Optional
from datetime import datetime
from pydantic import BaseModel
from app.routers.auth_db import get_current_user
from app.services.scheduler_service import get_scheduler_service, SchedulerService
from app.core.response import ok
router = APIRouter(prefix="/api/scheduler", tags=["scheduler"])
class JobTriggerRequest(BaseModel):
    """手动触发任务请求"""
    job_id: str
    kwargs: Optional[Dict[str, Any]] = None
```

```
class JobUpdateRequest(BaseModel):
    """更新任务请求"""
    job_id: str
    enabled: Optional[bool] = None
    cron: Optional[str] = None

class JobMetadataUpdateRequest(BaseModel):
    """更新任务元数据请求"""
    display_name: Optional[str] = None
    description: Optional[str] = None

@router.get("/jobs")
async def list_jobs(
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    获取所有定时任务列表

    Returns:
        任务列表，包含任务ID、名称、状态、下次执行时间等信息
    """
    try:
        jobs = await service.list_jobs()
        return ok(data=jobs, message=f"获取到 {len(jobs)} 个定时任务")
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取任务列表失败: {str(e)}")

@router.put("/jobs/{job_id}/metadata")
async def update_job_metadata_route(
    job_id: str,
    request: JobMetadataUpdateRequest,
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    更新任务元数据（触发器名称和备注）

    Args:
        job_id: 任务ID
        request: 更新请求

    Returns:
        操作结果
    """
    # 检查管理员权限
    if not user.get("is_admin"):
        raise HTTPException(status_code=403, detail="仅管理员可以更新任务元数据")

    try:
        success = await service.update_job_metadata(
            job_id,
            display_name=request.display_name,
            description=request.description
        )
        if success:
            return ok(message=f"任务 {job_id} 元数据已更新")
        else:
            raise HTTPException(status_code=400, detail=f"更新任务 {job_id} 元数据失败")
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"更新任务元数据失败: {str(e)}")

@router.get("/jobs/{job_id}")
async def get_job_detail(
    job_id: str,
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    获取任务详情

    Args:
        job_id: 任务ID

    Returns:
        任务详细信息
    """
    try:
        job = await service.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"任务 {job_id} 不存在")
        return ok(data=job, message="获取任务详情成功")
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取任务详情失败: {str(e)}")

@router.post("/jobs/{job_id}/pause")
async def pause_job(
    job_id: str,
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    暂停任务

    Args:
        job_id: 任务ID

    Returns:
        操作结果
    """
    # 检查管理员权限
    if not user.get("is_admin"):
        raise HTTPException(status_code=403, detail="仅管理员可以暂停任务")

    try:
        success = await service.pause_job(job_id)
        if success:
            return ok(message=f"任务 {job_id} 已暂停")
        else:
```

```
        raise HTTPException(status_code=400, detail=f"暂停任务 {job_id} 失败")
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"暂停任务失败: {str(e)}")

@router.post("/jobs/{job_id}/resume")
async def resume_job(
    job_id: str,
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    恢复任务

    Args:
        job_id: 任务ID

    Returns:
        操作结果
    """
    # 检查管理员权限
    if not user.get("is_admin"):
        raise HTTPException(status_code=403, detail="仅管理员可以恢复任务")

    try:
        success = await service.resume_job(job_id)
        if success:
            return ok(message=f"任务 {job_id} 已恢复")
        else:
            raise HTTPException(status_code=400, detail=f"恢复任务 {job_id} 失败")
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"恢复任务失败: {str(e)}")

@router.post("/jobs/{job_id}/trigger")
async def trigger_job(
    job_id: str,
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service),
    force: bool = Query(False, description="是否强制执行（跳过交易时间检查等）")
):
    """
    手动触发任务执行
    Args:
        job_id: 任务ID
        force: 是否强制执行（跳过交易时间检查等），默认 False
    Returns:
        操作结果
    """
    # 检查管理员权限
    if not user.get("is_admin"):
        raise HTTPException(status_code=403, detail="仅管理员可以手动触发任务")
    try:
        # 为特定任务传递 force 参数
        kwargs = {}
        if force and job_id in ["tushare_quotes_sync", "akshare_quotes_sync"]:
            kwargs["force"] = True
        success = await service.trigger_job(job_id, kwargs=kwargs)
        if success:
            message = f"任务 {job_id} 已触发执行"
            if force:
                message += "（强制模式）"
            return ok(message=message)
        else:
            raise HTTPException(status_code=400, detail=f"触发任务 {job_id} 失败")
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"触发任务失败: {str(e)}")

@router.get("/jobs/{job_id}/history")
async def get_job_history(
    job_id: str,
    limit: int = Query(20, ge=1, le=100, description="返回数量限制"),
    offset: int = Query(0, ge=0, description="偏移量"),
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    获取任务执行历史

    Args:
        job_id: 任务ID
        limit: 返回数量限制
        offset: 偏移量

    Returns:
        任务执行历史记录
    """
    try:
        history = await service.get_job_history(job_id, limit=limit, offset=offset)
        total = await service.count_job_history(job_id)

        return ok(
            data={
                "history": history,
                "total": total,
                "limit": limit,
                "offset": offset
            },
            message=f"获取到 {len(history)} 条执行记录"
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取执行历史失败: {str(e)}")

@router.get("/history")
```

```
async def get_all_history(
    limit: int = Query(50, ge=1, le=200, description="返回数量限制"),
    offset: int = Query(0, ge=0, description="偏移量"),
    job_id: Optional[str] = Query(None, description="任务ID过滤"),
    status: Optional[str] = Query(None, description="状态过滤: success/failed"),
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    获取所有任务执行历史

    Args:
        limit: 返回数量限制
        offset: 偏移量
        job_id: 任务ID过滤
        status: 状态过滤

    Returns:
        所有任务执行历史记录
    """
    try:
        history = await service.get_all_history(
            limit=limit,
            offset=offset,
            job_id=job_id,
            status=status
        )
        total = await service.count_all_history(job_id=job_id, status=status)

        return ok(
            data={
                "history": history,
                "total": total,
                "limit": limit,
                "offset": offset
            },
            message=f"获取到 {len(history)} 条执行记录"
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取执行历史失败: {str(e)}")

@router.get("/stats")
async def get_scheduler_stats(
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    获取调度器统计信息

    Returns:
        调度器统计信息，包括任务总数、运行中任务数、暂停任务数等
    """
    try:
        stats = await service.get_stats()
        return ok(data=stats, message="获取统计信息成功")
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取统计信息失败: {str(e)}")

@router.get("/health")
async def scheduler_health_check(
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    调度器健康检查

    Returns:
        调度器健康状态
    """
    try:
        health = await service.health_check()
        return ok(data=health, message="调度器运行正常")
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"健康检查失败: {str(e)}")

@router.get("/executions")
async def get_job_executions(
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service),
    job_id: Optional[str] = Query(None, description="任务ID过滤"),
    status: Optional[str] = Query(None, description="状态过滤（success/failed/missed/running）"),
    is_manual: Optional[bool] = Query(None, description="是否手动触发（true=手动，false=自动，None=全部）"),
    limit: int = Query(50, ge=1, le=200, description="返回数量限制"),
    offset: int = Query(0, ge=0, description="偏移量")
):
    """
    获取任务执行历史

    Args:
        job_id: 任务ID过滤（可选）
        status: 状态过滤（可选）
        is_manual: 是否手动触发（可选）
        limit: 返回数量限制
        offset: 偏移量

    Returns:
        执行历史列表
    """
    try:
        executions = await service.get_job_executions(
            job_id=job_id,
            status=status,
            is_manual=is_manual,
            limit=limit,
            offset=offset
        )
        total = await service.count_job_executions(job_id=job_id, status=status, is_manual=is_manual)
        return ok(data={
            "items": executions,
            "total": total,
```

```
            "limit": limit,
            "offset": offset
        }, message=f"获取到 {len(executions)} 条执行记录")
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取执行历史失败: {str(e)}")

@router.get("/jobs/{job_id}/executions")
async def get_single_job_executions(
    job_id: str,
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service),
    status: Optional[str] = Query(None, description="状态过滤（success/failed/missed/running）"),
    is_manual: Optional[bool] = Query(None, description="是否手动触发（true=手动，false=自动，None=全部）"),
    limit: int = Query(50, ge=1, le=200, description="返回数量限制"),
    offset: int = Query(0, ge=0, description="偏移量")
):
    """
    获取指定任务的执行历史

    Args:
        job_id: 任务ID
        status: 状态过滤（可选）
        is_manual: 是否手动触发（可选）
        limit: 返回数量限制
        offset: 偏移量

    Returns:
        执行历史列表
    """
    try:
        executions = await service.get_job_executions(
            job_id=job_id,
            status=status,
            is_manual=is_manual,
            limit=limit,
            offset=offset
        )
        total = await service.count_job_executions(job_id=job_id, status=status, is_manual=is_manual)
        return ok(data={
            "items": executions,
            "total": total,
            "limit": limit,
            "offset": offset
        }, message=f"获取到 {len(executions)} 条执行记录")
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取执行历史失败: {str(e)}")

@router.get("/jobs/{job_id}/execution-stats")
async def get_job_execution_stats(
    job_id: str,
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    获取任务执行统计信息

    Args:
        job_id: 任务ID

    Returns:
        统计信息
    """
    try:
        stats = await service.get_job_execution_stats(job_id)
        return ok(data=stats, message="获取统计信息成功")
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取统计信息失败: {str(e)}")

@router.post("/executions/{execution_id}/cancel")
async def cancel_execution(
    execution_id: str,
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    取消/终止任务执行

    对于正在执行的任务，设置取消标记；
    对于已经退出但数据库中仍为running的任务，直接标记为failed

    Args:
        execution_id: 执行记录ID（MongoDB _id）

    Returns:
        操作结果
    """
    try:
        success = await service.cancel_job_execution(execution_id)
        if success:
            return ok(message="已设置取消标记，任务将在下次检查时停止")
        else:
            raise HTTPException(status_code=400, detail="取消任务失败")
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"取消任务失败: {str(e)}")

@router.post("/executions/{execution_id}/mark-failed")
async def mark_execution_failed(
    execution_id: str,
    reason: str = Query("用户手动标记为失败", description="失败原因"),
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    将执行记录标记为失败状态

    用于处理已经退出但数据库中仍为running的任务

    Args:
        execution_id: 执行记录ID（MongoDB _id）
        reason: 失败原因

    Returns:
        操作结果
    """
    try:
```

```
        success = await service.mark_execution_as_failed(execution_id, reason)
        if success:
            return ok(message="已标记为失败状态")
        else:
            raise HTTPException(status_code=400, detail="标记失败")
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"标记失败: {str(e)}")

@router.delete("/executions/{execution_id}")
async def delete_execution(
    execution_id: str,
    user: dict = Depends(get_current_user),
    service: SchedulerService = Depends(get_scheduler_service)
):
    """
    删除执行记录

    Args:
        execution_id: 执行记录ID（MongoDB _id）

    Returns:
        操作结果
    """
    try:
        success = await service.delete_execution(execution_id)
        if success:
            return ok(message="执行记录已删除")
        else:
            raise HTTPException(status_code=400, detail="删除失败")
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"删除执行记录失败: {str(e)}")
```

FILE: app/routers/screening.py

```
import logging
from fastapi import APIRouter, HTTPException, Depends
from pydantic import BaseModel, Field
from typing import List, Optional, Dict, Any
from app.routers.auth_db import get_current_user
from app.services.screening_service import ScreeningService, ScreeningParams
from app.services.enhanced_screening_service import get_enhanced_screening_service
from app.models.screening import (
    ScreeningCondition, ScreeningRequest as NewScreeningRequest,
    ScreeningResponse as NewScreeningResponse, FieldInfo, BASIC_FIELDS_INFO
)

router = APIRouter(tags=["screening"])
logger = logging.getLogger("webapi")

# 筛选字段配置响应模型
class FieldConfigResponse(BaseModel):
    """筛选字段配置响应"""
    fields: Dict[str, FieldInfo]
    categories: Dict[str, List[str]]

# 传统的请求/响应模型（保持向后兼容）
class OrderByItem(BaseModel):
    field: str
    direction: str = Field("desc", pattern=r"^(?i)(asc|desc)$")

class ScreeningRequest(BaseModel):
    market: str = Field("CN", description="市场：CN")
    date: Optional[str] = Field(None, description="交易日YYYY-MM-DD，缺省为最新")
    adj: str = Field("qfq", description="复权口径：qfq/hfq/none（P0占位）")
    conditions: Dict[str, Any] = Field(default_factory=dict)
    order_by: Optional[List[OrderByItem]] = None
    limit: int = Field(50, ge=1, le=500)
    offset: int = Field(0, ge=0)

class ScreeningResponse(BaseModel):
    total: int
    items: List[dict]

# 服务实例
svc = ScreeningService()
enhanced_svc = get_enhanced_screening_service()

@router.get("/fields", response_model=FieldConfigResponse)
async def get_screening_fields(user: dict = Depends(get_current_user)):
    """
    获取筛选字段配置
    返回所有可用的筛选字段及其配置信息
    """
    try:
        # 字段分类
        categories = {
            "basic": ["code", "name", "industry", "area", "market"],
            "market_value": ["total_mv", "circ_mv"],
            "financial": ["pe", "pb", "pe_ttm", "pb_mrq", "roe"],
            "trading": ["turnover_rate", "volume_ratio"],
            "price": ["close", "pct_chg", "amount"],
            "technical": ["ma20", "rsi14", "kdj_k", "kdj_d", "kdj_j", "dif", "dea", "macd_hist"]
        }
        return FieldConfigResponse(
            fields=BASIC_FIELDS_INFO,
            categories=categories
        )
    except Exception as e:
        logger.error(f"[get_screening_fields] 获取字段配置失败: {e}", exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))

def _convert_legacy_conditions_to_new_format(legacy_conditions: Dict[str, Any]) -> List[ScreeningCondition]:
    """
    将传统格式的筛选条件转换为新格式

    传统格式示例:
    {
        "logic": "AND",
        "children": [
```

```
            {"field": "market_cap", "op": "between", "value": [5000000, 9007199254740991]}
        ]
    }
    新格式:
    [
        ScreeningCondition(field="total_mv", operator="between", value=[50, 90071992547])
    ]
    """
    conditions = []
    # 字段名映射（前端可能使用的旧字段名 -> 统一的后端字段名）
    field_mapping = {
        "market_cap": "total_mv",     # 市值（兼容旧字段名）
        "pe_ratio": "pe",             # 市盈率（兼容旧字段名）
        "pb_ratio": "pb",             # 市净率（兼容旧字段名）
        "turnover": "turnover_rate",  # 换手率（兼容旧字段名）
        "change_percent": "pct_chg",  # 涨跌幅（兼容旧字段名）
        "price": "close",             # 价格（兼容旧字段名）
    }
    # 操作符映射
    operator_mapping = {
        "between": "between",
        "gt": ">",
        "lt": "<",
        "gte": ">=",
        "lte": "<=",
        "eq": "==",
        "ne": "!=",
        "in": "in",
        "contains": "contains"
    }
    if isinstance(legacy_conditions, dict):
        children = legacy_conditions.get("children", [])
        for child in children:
            if isinstance(child, dict):
                field = child.get("field")
                op = child.get("op")
                value = child.get("value")
                if field and op and value is not None:
                    # 映射字段名
                    mapped_field = field_mapping.get(field, field)
                    # 映射操作符
                    mapped_op = operator_mapping.get(op, op)
                    # 处理市值单位转换（前端传入的是万元，数据库存储的是亿元）
                    if mapped_field == "total_mv" and isinstance(value, list):
                        # 将万元转换为亿元
                        converted_value = [v / 10000 for v in value if isinstance(v, (int, float))]
                        logger.info(f"[screening] 市值单位转换: {value} 万元 -> {converted_value} 亿元")
                        value = converted_value
                    elif mapped_field == "total_mv" and isinstance(value, (int, float)):
                        value = value / 10000
                        logger.info(f"[screening] 市值单位转换: {child.get('value')} 万元 -> {value} 亿元")
                    # 创建筛选条件
                    condition = ScreeningCondition(
                        field=mapped_field,
                        operator=mapped_op,
                        value=value
                    )
                    conditions.append(condition)
                    logger.info(f"[screening] 转换条件: {field}({op}) -> {mapped_field}({mapped_op}), 值: {value}")
    return conditions
# 传统筛选接口（保持向后兼容，但使用增强服务）
@router.post("/run", response_model=ScreeningResponse)
async def run_screening(req: ScreeningRequest, user: dict = Depends(get_current_user)):
    try:
        logger.info(f"[screening] 请求条件: {req.conditions}")
        logger.info(f"[screening] 排序与分页: order_by={req.order_by}, limit={req.limit}, offset={req.offset}")
        # 转换传统格式的条件为新格式
        conditions = _convert_legacy_conditions_to_new_format(req.conditions)
        logger.info(f"[screening] 转换后的条件: {conditions}")
        # 使用增强筛选服务
        result = await enhanced_svc.screen_stocks(
            conditions=conditions,
            market=req.market,
            date=req.date,
            adj=req.adj,
            limit=req.limit,
            offset=req.offset,
            order_by=[{"field": o.field, "direction": o.direction} for o in (req.order_by or [])],
            use_database_optimization=True
        )
        logger.info(f"[screening] 筛选完成: total={result.get('total')}, "
                    f"took={result.get('took_ms')}ms, optimization={result.get('optimization_used')}")
        if result.get('items'):
            sample = result['items'][:3]
            logger.info(f"[screening] 返回样例(前3条): {sample}")
        return ScreeningResponse(total=result["total"], items=result["items"])
    except Exception as e:
        logger.error(f"[screening] 处理失败: {e}", exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))
# 新的优化筛选接口
@router.post("/enhanced", response_model=NewScreeningResponse)
async def enhanced_screening(req: NewScreeningRequest, user: dict = Depends(get_current_user)):
    """
    增强的股票筛选接口
    - 支持更丰富的筛选条件格式
    - 自动选择最优的筛选策略（数据库优化 vs 传统方法）
    - 提供详细的性能统计信息
    """
    try:
        logger.info(f"[enhanced_screening] 筛选条件: {len(req.conditions)}个")
        logger.info(f"[enhanced_screening] 排序与分页: order_by={req.order_by}, limit={req.limit}, offset={req.offset}")
        # 执行增强筛选
```

```
        result = await enhanced_svc.screen_stocks(
            conditions=req.conditions,
            market=req.market,
            date=req.date,
            adj=req.adj,
            limit=req.limit,
            offset=req.offset,
            order_by=req.order_by,
            use_database_optimization=req.use_database_optimization
        )
        logger.info(f"[enhanced_screening] 筛选完成: total={result.get('total')}, "
                    f"took={result.get('took_ms')}ms, optimization={result.get('optimization_used')}")
        return NewScreeningResponse(
            total=result["total"],
            items=result["items"],
            took_ms=result.get("took_ms"),
            optimization_used=result.get("optimization_used"),
            source=result.get("source")
        )
    except Exception as e:
        logger.error(f"[enhanced_screening] 筛选失败: {e}")
        raise HTTPException(status_code=500, detail=f"增强筛选失败: {str(e)}")

# 获取支持的字段信息
@router.get("/fields", response_model=List[Dict[str, Any]])
async def get_supported_fields(user: dict = Depends(get_current_user)):
    """获取所有支持的筛选字段信息"""
    try:
        fields = await enhanced_svc.get_all_supported_fields()
        return fields
    except Exception as e:
        logger.error(f"[screening] 获取字段信息失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取字段信息失败: {str(e)}")

# 获取单个字段的详细信息
@router.get("/fields/{field_name}", response_model=Dict[str, Any])
async def get_field_info(field_name: str, user: dict = Depends(get_current_user)):
    """获取指定字段的详细信息"""
    try:
        field_info = await enhanced_svc.get_field_info(field_name)
        if not field_info:
            raise HTTPException(status_code=404, detail=f"字段 '{field_name}' 不存在")
        return field_info
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"[screening] 获取字段信息失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取字段信息失败: {str(e)}")

# 验证筛选条件
@router.post("/validate", response_model=Dict[str, Any])
async def validate_conditions(conditions: List[ScreeningCondition], user: dict = Depends(get_current_user)):
    """验证筛选条件的有效性"""
    try:
        validation_result = await enhanced_svc.validate_conditions(conditions)
        return validation_result
    except Exception as e:
        logger.error(f"[screening] 验证条件失败: {e}")
        raise HTTPException(status_code=500, detail=f"验证条件失败: {str(e)}")
# 重复定义的旧端点移除（保留带日志的版本）

@router.get("/industries")
async def get_industries(user: dict = Depends(get_current_user)):
    """
    获取数据库中所有可用的行业列表
    根据系统配置的数据源优先级，从优先级最高的数据源获取行业分类数据
    返回按股票数量排序的行业列表
    """
    try:
        from app.core.database import get_mongo_db
        from app.core.unified_config import UnifiedConfigManager

        db = get_mongo_db()
        collection = db["stock_basic_info"]
        # 🔥 获取数据源优先级配置（使用统一配置管理器的异步方法）
        config = UnifiedConfigManager()
        data_source_configs = await config.get_data_source_configs_async()
        # 提取启用的数据源，按优先级排序（已排序）
        enabled_sources = [
            ds.type.lower() for ds in data_source_configs
            if ds.enabled and ds.type.lower() in ['tushare', 'akshare', 'baostock']
        ]
        if not enabled_sources:
            # 如果没有配置，使用默认顺序
            enabled_sources = ['tushare', 'akshare', 'baostock']

        logger.info(f"[get_industries] 数据源优先级: {enabled_sources}")
        # 🔥 按优先级查询：优先使用优先级最高的数据源
        preferred_source = enabled_sources[0] if enabled_sources else 'tushare'
        # 聚合查询：按行业分组并统计股票数量（只查询指定数据源）
        pipeline = [
            {
                "$match": {
                    "source": preferred_source,  # 🔥 只查询优先级最高的数据源
                    "industry": {"$ne": None, "$ne": ""}  # 过滤空行业
                }
            },
            {
                "$group": {
                    "_id": "$industry",
                    "count": {"$sum": 1}
                }
            },
            {"$sort": {"count": -1}},  # 按股票数量降序排序
            {
                "$project": {
                    "industry": "$_id",
                    "count": 1,
                    "_id": 0
```

```
            }
        }
    ]
    industries = []
    async for doc in collection.aggregate(pipeline):
        # 清洗字段，避免 NaN/Inf 导致 JSON 序列化失败
        raw_industry = doc.get("industry")
        safe_industry = ""
        try:
            if raw_industry is None:
                safe_industry = ""
            elif isinstance(raw_industry, float):
                if raw_industry != raw_industry or raw_industry in (float("inf"), float("-inf")):
                    safe_industry = ""
                else:
                    safe_industry = str(raw_industry)
            else:
                safe_industry = str(raw_industry)
        except Exception:
            safe_industry = ""
        raw_count = doc.get("count", 0)
        safe_count = 0
        try:
            if isinstance(raw_count, float):
                if raw_count != raw_count or raw_count in (float("inf"), float("-inf")):
                    safe_count = 0
                else:
                    safe_count = int(raw_count)
            else:
                safe_count = int(raw_count)
        except Exception:
            safe_count = 0
        industries.append({
            "value": safe_industry,
            "label": safe_industry,
            "count": safe_count,
        })
    logger.info(f"[get_industries] 从数据源 {preferred_source} 返回 {len(industries)} 个行业")
    return {
        "industries": industries,
        "total": len(industries),
        "source": preferred_source  # 🔥 返回数据来源
    }
except Exception as e:
    logger.error(f"[get_industries] 获取行业列表失败: {e}", exc_info=True)
    raise HTTPException(status_code=500, detail=str(e))
```

FILE: app/routers/social_media.py

```
"""
社媒消息数据API路由
提供社媒消息的查询、搜索和统计接口
"""
from typing import Optional, List, Dict, Any
from datetime import datetime, timedelta
from fastapi import APIRouter, HTTPException, BackgroundTasks, Query
from pydantic import BaseModel, Field
from app.services.social_media_service import (
    get_social_media_service,
    SocialMediaQueryParams,
    SocialMediaStats
)
from app.core.response import ok
router = APIRouter(prefix="/api/social-media", tags=["social-media"])
class SocialMediaMessage(BaseModel):
    """社媒消息模型"""
    message_id: str
    platform: str
    message_type: str = "post"
    content: str
    media_urls: Optional[List[str]] = []
    hashtags: Optional[List[str]] = []
    author: Dict[str, Any]
    engagement: Dict[str, Any]
    publish_time: datetime
    sentiment: Optional[str] = "neutral"
    sentiment_score: Optional[float] = 0.0
    keywords: Optional[List[str]] = []
    topics: Optional[List[str]] = []
    importance: Optional[str] = "low"
    credibility: Optional[str] = "medium"
    location: Optional[Dict[str, str]] = None
    language: str = "zh-CN"
    data_source: str
    crawler_version: str = "1.0"
class SocialMediaBatchRequest(BaseModel):
    """批量保存社媒消息请求"""
    symbol: str = Field(..., description="股票代码")
    messages: List[SocialMediaMessage] = Field(..., description="社媒消息列表")
class SocialMediaQueryRequest(BaseModel):
    """社媒消息查询请求"""
    symbol: Optional[str] = None
    symbols: Optional[List[str]] = None
    platform: Optional[str] = None
    message_type: Optional[str] = None
    start_time: Optional[datetime] = None
    end_time: Optional[datetime] = None
    sentiment: Optional[str] = None
    importance: Optional[str] = None
    min_influence_score: Optional[float] = None
    min_engagement_rate: Optional[float] = None
    verified_only: bool = False
```

```
    keywords: Optional[List[str]] = None
    hashtags: Optional[List[str]] = None
    limit: int = Field(50, ge=1, le=1000)
    skip: int = Field(0, ge=0)

@router.post("/save", response_model=dict)
async def save_social_media_messages(request: SocialMediaBatchRequest):
    """批量保存社媒消息"""
    try:
        service = await get_social_media_service()
        # 转换消息格式并添加股票代码
        messages = []
        for msg in request.messages:
            message_dict = msg.dict()
            message_dict["symbol"] = request.symbol
            messages.append(message_dict)
        # 保存消息
        result = await service.save_social_media_messages(messages)
        return ok(
            data=result,
            message=f"成功保存 {result['saved']} 条社媒消息"
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"保存社媒消息失败: {str(e)}")

@router.post("/query", response_model=dict)
async def query_social_media_messages(request: SocialMediaQueryRequest):
    """查询社媒消息"""
    try:
        service = await get_social_media_service()
        # 构建查询参数
        params = SocialMediaQueryParams(
            symbol=request.symbol,
            symbols=request.symbols,
            platform=request.platform,
            message_type=request.message_type,
            start_time=request.start_time,
            end_time=request.end_time,
            sentiment=request.sentiment,
            importance=request.importance,
            min_influence_score=request.min_influence_score,
            min_engagement_rate=request.min_engagement_rate,
            verified_only=request.verified_only,
            keywords=request.keywords,
            hashtags=request.hashtags,
            limit=request.limit,
            skip=request.skip
        )
        # 执行查询
        messages = await service.query_social_media_messages(params)
        return ok(
            data={
                "messages": messages,
                "count": len(messages),
                "params": request.dict()
            },
            message=f"查询到 {len(messages)} 条社媒消息"
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"查询社媒消息失败: {str(e)}")

@router.get("/latest/{symbol}", response_model=dict)
async def get_latest_messages(
    symbol: str,
    platform: Optional[str] = Query(None, description="平台类型"),
    limit: int = Query(20, ge=1, le=100, description="返回数量")
):
    """获取最新社媒消息"""
    try:
        service = await get_social_media_service()
        messages = await service.get_latest_messages(symbol, platform, limit)

        return ok(data={
                "messages": messages,
                "count": len(messages),
                "symbol": symbol,
                "platform": platform
            },
            message=f"获取到 {len(messages)} 条最新消息"
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取最新消息失败: {str(e)}")

@router.get("/search", response_model=dict)
async def search_messages(
    query: str = Query(..., description="搜索关键词"),
    symbol: Optional[str] = Query(None, description="股票代码"),
    platform: Optional[str] = Query(None, description="平台类型"),
    limit: int = Query(50, ge=1, le=200, description="返回数量")
):
    """全文搜索社媒消息"""
    try:
        service = await get_social_media_service()
        messages = await service.search_messages(query, symbol, platform, limit)
        return ok(
            data={
                "messages": messages,
                "count": len(messages),
                "query": query,
                "symbol": symbol,
                "platform": platform
            },
            message=f"搜索到 {len(messages)} 条相关消息"
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"搜索消息失败: {str(e)}")
```

```
@router.get("/statistics", response_model=dict)
async def get_statistics(
    symbol: Optional[str] = Query(None, description="股票代码"),
    hours_back: int = Query(24, ge=1, le=168, description="回溯小时数")
):
    """获取社媒消息统计信息"""
    try:
        service = await get_social_media_service()

        # 计算时间范围
        end_time = datetime.utcnow()
        start_time = end_time - timedelta(hours=hours_back)

        stats = await service.get_social_media_statistics(symbol, start_time, end_time)

        return ok(data={
                "statistics": stats.__dict__,
                "symbol": symbol,
                "time_range": {
                    "start_time": start_time,
                    "end_time": end_time,
                    "hours_back": hours_back
                }
            },
            message="统计信息获取成功"
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取统计信息失败: {str(e)}")
@router.get("/platforms", response_model=dict)
async def get_supported_platforms():
    """获取支持的社媒平台列表"""
    platforms = [
        {
            "code": "weibo",
            "name": "微博",
            "description": "新浪微博社交平台"
        },
        {
            "code": "wechat",
            "name": "微信",
            "description": "微信公众号和朋友圈"
        },
        {
            "code": "douyin",
            "name": "抖音",
            "description": "字节跳动短视频平台"
        },
        {
            "code": "xiaohongshu",
            "name": "小红书",
            "description": "生活方式分享平台"
        },
        {
            "code": "zhihu",
            "name": "知乎",
            "description": "知识问答社区"
        },
        {
            "code": "twitter",
            "name": "Twitter",
            "description": "国际社交媒体平台"
        },
        {
            "code": "reddit",
            "name": "Reddit",
            "description": "国际论坛社区"
        }
    ]

    return ok(data={
            "platforms": platforms,
            "count": len(platforms)
        },
        message="支持的平台列表获取成功"
    )
@router.get("/sentiment-analysis/{symbol}", response_model=dict)
async def get_sentiment_analysis(
    symbol: str,
    platform: Optional[str] = Query(None, description="平台类型"),
    hours_back: int = Query(24, ge=1, le=168, description="回溯小时数")
):
    """获取股票的社媒情绪分析"""
    try:
        service = await get_social_media_service()

        # 计算时间范围
        end_time = datetime.utcnow()
        start_time = end_time - timedelta(hours=hours_back)

        # 查询消息
        params = SocialMediaQueryParams(
            symbol=symbol,
            platform=platform,
            start_time=start_time,
            end_time=end_time,
            limit=1000
        )

        messages = await service.query_social_media_messages(params)

        # 分析情绪分布
        sentiment_counts = {"positive": 0, "negative": 0, "neutral": 0}
        platform_sentiment = {}
        hourly_sentiment = {}
```

```
        for msg in messages:
            sentiment = msg.get("sentiment", "neutral")
            sentiment_counts[sentiment] += 1

            # 按平台统计
            msg_platform = msg.get("platform", "unknown")
            if msg_platform not in platform_sentiment:
                platform_sentiment[msg_platform] = {"positive": 0, "negative": 0, "neutral": 0}
            platform_sentiment[msg_platform][sentiment] += 1

            # 按小时统计
            publish_time = msg.get("publish_time")
            if publish_time:
                hour_key = publish_time.strftime("%Y-%m-%d %H:00")
                if hour_key not in hourly_sentiment:
                    hourly_sentiment[hour_key] = {"positive": 0, "negative": 0, "neutral": 0}
                hourly_sentiment[hour_key][sentiment] += 1

        # 计算情绪指数 (positive: +1, neutral: 0, negative: -1)
        total_messages = len(messages)
        sentiment_score = 0
        if total_messages > 0:
            sentiment_score = (sentiment_counts["positive"] - sentiment_counts["negative"]) / total_messages

        return ok(data={
                "symbol": symbol,
                "total_messages": total_messages,
                "sentiment_distribution": sentiment_counts,
                "sentiment_score": sentiment_score,
                "platform_sentiment": platform_sentiment,
                "hourly_sentiment": hourly_sentiment,
                "time_range": {
                    "start_time": start_time,
                    "end_time": end_time,
                    "hours_back": hours_back
                }
            },
            message=f"情绪分析完成，共分析 {total_messages} 条消息"
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"情绪分析失败: {str(e)}")
@router.get("/health", response_model=dict)
async def health_check():
    """健康检查"""
    try:
        service = await get_social_media_service()

        # 简单的连接测试
        collection = await service._get_collection()
        count = await collection.estimated_document_count()

        return ok(data={
                "status": "healthy",
                "total_messages": count,
                "service": "social_media_service"
            },
            message="社媒消息服务运行正常"
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"健康检查失败: {str(e)}")
```

FILE: app/routers/sse.py

```
from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import StreamingResponse
import asyncio
import json
import logging
import time
from app.routers.auth_db import get_current_user
from app.core.database import get_redis_client
from app.core.config import settings
from app.services.queue_service import get_queue_service, QueueService
router = APIRouter()
logger = logging.getLogger("webapi.sse")
async def task_progress_generator(task_id: str, user_id: str):
    """Generate SSE events for task progress updates"""
    r = get_redis_client()
    pubsub = None
    channel = f"task_progress:{task_id}"
    try:
        # Load dynamic SSE settings
        try:
            from app.services.config_provider import provider as config_provider
            eff = await config_provider.get_effective_system_settings()
            poll_timeout = float(eff.get("sse_poll_timeout_seconds", 1.0))
            heartbeat_every = int(eff.get("sse_heartbeat_interval_seconds", 10))
            max_idle_seconds = int(eff.get("sse_task_max_idle_seconds", 300))
        except Exception:
            poll_timeout = float(getattr(settings, "SSE_POLL_TIMEOUT_SECONDS", 1.0))
            heartbeat_every = int(getattr(settings, "SSE_HEARTBEAT_INTERVAL_SECONDS", 10))
            max_idle_seconds = int(getattr(settings, "SSE_TASK_MAX_IDLE_SECONDS", 300))
        # 🔥 修复：创建 PubSub 连接
        pubsub = r.pubsub()
        logger.info(f"📡 [SSE-Task] 创建 PubSub 连接: task={task_id}, user={user_id}")
        # 🔥 修复：订阅频道（可能失败，需要确保 pubsub 被清理）
        try:
            await pubsub.subscribe(channel)
            logger.info(f"✅ [SSE-Task] 订阅频道成功: {channel}")
            # Send initial connection confirmation
```

```
            yield f"event: connected\ndata: {{\"task_id\": \"{task_id}\", \"message\": \"已连接进度流\"}}\n\n"
        except Exception as subscribe_error:
            # 🔥 订阅失败时立即清理 pubsub 连接
            logger.error(f"❌ [SSE-Task] 订阅频道失败: {subscribe_error}")
            try:
                await pubsub.close()
                logger.info(f"🧹 [SSE-Task] 订阅失败后已关闭 PubSub 连接")
            except Exception as close_error:
                logger.error(f"❌ [SSE-Task] 关闭 PubSub 连接失败: {close_error}")
            # 重新抛出异常，让外层 except 处理
            raise
        # Listen for progress updates
        idle_elapsed = 0.0
        last_hb = time.monotonic()
        while idle_elapsed < max_idle_seconds:
            try:
                message = await asyncio.wait_for(pubsub.get_message(ignore_subscribe_messages=True), timeout=poll_timeout)
                if message and message['type'] == 'message':
                    # Reset idle timer on valid message
                    idle_elapsed = 0.0
                    try:
                        progress_data = json.loads(message['data'])
                        yield f"event: progress\ndata: {json.dumps(progress_data, ensure_ascii=False)}\n\n"
                    except json.JSONDecodeError:
                        logger.warning(f"Invalid JSON in progress message: {message['data']}")
                else:
                    # No update: accumulate idle time and send heartbeat if due
                    idle_elapsed += poll_timeout
                    now = time.monotonic()
                    if now - last_hb >= heartbeat_every:
                        yield f"event: heartbeat\ndata: {{\"timestamp\": \"{asyncio.get_event_loop().time()}\"}}\n\n"
                        last_hb = now
            except asyncio.TimeoutError:
                idle_elapsed += poll_timeout
                continue
    except Exception as e:
        logger.exception(f"SSE error for task {task_id}: {e}")
        yield f"event: error\ndata: {{\"error\": \"连接异常: {str(e)}\"}}\n\n"
    finally:
        # 🔥 修复：确保在所有情况下都释放连接
        if pubsub:
            logger.info(f"🧹 [SSE-Task] 清理 PubSub 连接: task={task_id}")
            # 分步骤关闭，确保即使 unsubscribe 失败也能关闭连接
            try:
                await pubsub.unsubscribe(channel)
                logger.debug(f"✅ [SSE-Task] 已取消订阅频道: {channel}")
            except Exception as e:
                logger.warning(f"⚠️ [SSE-Task] 取消订阅失败（将继续关闭连接）: {e}")
            try:
                await pubsub.close()
                logger.info(f"✅ [SSE-Task] PubSub 连接已关闭: task={task_id}")
            except Exception as e:
                logger.error(f"❌ [SSE-Task] 关闭 PubSub 连接失败: {e}", exc_info=True)
                # 即使关闭失败，也尝试重置连接
                try:
                    await pubsub.reset()
                    logger.info(f"🔄 [SSE-Task] PubSub 连接已重置: task={task_id}")
                except Exception as reset_error:
                    logger.error(f"❌ [SSE-Task] 重置 PubSub 连接也失败: {reset_error}")
async def batch_progress_generator(batch_id: str, user_id: str):
    """Generate SSE events for batch progress updates"""
    svc = get_queue_service()
    try:
        # Load dynamic SSE settings for batch stream
        try:
            from app.services.config_provider import provider as config_provider
            eff = await config_provider.get_effective_system_settings()
            batch_poll_interval = float(eff.get("sse_batch_poll_interval_seconds", 2))
            batch_max_idle_seconds = int(eff.get("sse_batch_max_idle_seconds", 600))
        except Exception:
            batch_poll_interval = float(getattr(settings, "SSE_BATCH_POLL_INTERVAL_SECONDS", 2.0))
            batch_max_idle_seconds = int(getattr(settings, "SSE_BATCH_MAX_IDLE_SECONDS", 600))
        # Send initial connection confirmation
        yield f"event: connected\ndata: {{\"batch_id\": \"{batch_id}\", \"message\": \"已连接批次进度流\"}}\n\n"
        idle_elapsed = 0.0
        while idle_elapsed < batch_max_idle_seconds:
            try:
                # Get current batch status
                batch_data = await svc.get_batch(batch_id)
                if not batch_data:
                    yield f"event: error\ndata: {{\"error\": \"批次不存在\"}}\n\n"
                    break
                # Check if batch belongs to user
                if batch_data.get("user") != user_id:
                    yield f"event: error\ndata: {{\"error\": \"无权限访问此批次\"}}\n\n"
                    break
                # Calculate batch progress based on task statuses
                task_ids = batch_data.get("tasks", [])
                if not task_ids:
                    yield f"event: progress\ndata: {{\"batch_id\": \"{batch_id}\", \"message\": \"批次无任务\", \"progress\": 0}}\n\n"
                    await asyncio.sleep(batch_poll_interval)
                    idle_elapsed += batch_poll_interval
                    continue
                completed_count = 0
                failed_count = 0
                processing_count = 0
                for task_id in task_ids:
                    task_data = await svc.get_task(task_id)
                    if task_data:
                        status = task_data.get("status", "queued")
                        if status == "completed":
                            completed_count += 1
                        elif status == "failed":
```

```
                failed_count += 1
            elif status == "processing":
                processing_count += 1
        total_tasks = len(task_ids)
        finished_tasks = completed_count + failed_count
        progress = round((finished_tasks / total_tasks) * 100, 1) if total_tasks > 0 else 0
        # Determine batch status
        if finished_tasks == total_tasks:
            if failed_count == 0:
                batch_status = "completed"
                message = f"批次完成: {completed_count}/{total_tasks} 成功"
            elif completed_count == 0:
                batch_status = "failed"
                message = f"批次失败: {failed_count}/{total_tasks} 失败"
            else:
                batch_status = "partial"
                message = f"批次部分成功: {completed_count} 成功, {failed_count} 失败"
        elif processing_count > 0 or finished_tasks < total_tasks:
            batch_status = "processing"
            message = f"批次处理中: {finished_tasks}/{total_tasks} 已完成, {processing_count} 处理中"
        else:
            batch_status = "queued"
            message = f"批次排队中: {total_tasks} 任务待处理"
        progress_data = {
            "batch_id": batch_id,
            "status": batch_status,
            "message": message,
            "progress": progress,
            "total_tasks": total_tasks,
            "completed": completed_count,
            "failed": failed_count,
            "processing": processing_count,
            "timestamp": asyncio.get_event_loop().time()
        }
        yield f"event: progress\ndata: {json.dumps(progress_data, ensure_ascii=False)}\n\n"
        # Break if batch is finished
        if batch_status in ["completed", "failed", "partial"]:
            yield f"event: finished\ndata: {{\"batch_id\": \"{batch_id}\", \"final_status\": \"{batch_status}\"}}\n\n"
            break
        # Wait before next update
        await asyncio.sleep(batch_poll_interval)
        idle_elapsed += batch_poll_interval
    except Exception as e:
        logger.exception(f"Batch progress error: {e}")
        yield f"event: error\ndata: {{\"error\": \"获取批次状态失败: {str(e)}\"}}\n\n"
        break
except Exception as e:
    logger.exception(f"SSE batch error for {batch_id}: {e}")
    yield f"event: error\ndata: {{\"error\": \"连接异常: {str(e)}\"}}\n\n"
@router.get("/tasks/{task_id}")
async def stream_task_progress(task_id: str, user: dict = Depends(get_current_user), svc: QueueService = Depends(get_queue_service)):
    """Stream real-time progress updates for a specific task"""
    # Verify task exists and belongs to user
    task_data = await svc.get_task(task_id)
    if not task_data or task_data.get("user") != user["id"]:
        raise HTTPException(status_code=404, detail="Task not found")
    return StreamingResponse(
        task_progress_generator(task_id, user["id"]),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
            "X-Accel-Buffering": "no"  # Disable nginx buffering
        }
    )
@router.get("/batches/{batch_id}")
async def stream_batch_progress(batch_id: str, user: dict = Depends(get_current_user), svc: QueueService = Depends(get_queue_service)):
    """Stream real-time progress updates for a batch"""
    # Verify batch exists and belongs to user
    batch_data = await svc.get_batch(batch_id)
    if not batch_data or batch_data.get("user") != user["id"]:
        raise HTTPException(status_code=404, detail="Batch not found")
    return StreamingResponse(
        batch_progress_generator(batch_id, user["id"]),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
            "X-Accel-Buffering": "no"
        }
    )
```

FILE: app/routers/stock_data.py

```
"""
股票数据API路由 - 基于扩展数据模型
提供标准化的股票数据访问接口
"""
from typing import Optional, List
from fastapi import APIRouter, Depends, HTTPException, Query
from fastapi import status
from app.routers.auth_db import get_current_user
from app.services.stock_data_service import get_stock_data_service
from app.models import (
    StockBasicInfoResponse,
    MarketQuotesResponse,
    StockListResponse,
    StockBasicInfoExtended,
    MarketQuotesExtended,
    MarketType
)
router = APIRouter(prefix="/api/stock-data", tags=["股票数据"])
```

```
@router.get("/basic-info/{symbol}", response_model=StockBasicInfoResponse)
async def get_stock_basic_info(
    symbol: str,
    current_user: dict = Depends(get_current_user)
):
    """
    获取股票基础信息
    Args:
        symbol: 股票代码 (支持6位A股代码)
    Returns:
        StockBasicInfoResponse: 包含扩展字段的股票基础信息
    """
    try:
        service = get_stock_data_service()
        stock_info = await service.get_stock_basic_info(symbol)
        if not stock_info:
            return StockBasicInfoResponse(
                success=False,
                message=f"未找到股票代码 {symbol} 的基础信息"
            )
        return StockBasicInfoResponse(
            success=True,
            data=stock_info,
            message="获取成功"
        )
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取股票基础信息失败: {str(e)}"
        )
@router.get("/quotes/{symbol}", response_model=MarketQuotesResponse)
async def get_market_quotes(
    symbol: str,
    current_user: dict = Depends(get_current_user)
):
    """
    获取实时行情数据
    Args:
        symbol: 股票代码 (支持6位A股代码)
    Returns:
        MarketQuotesResponse: 包含扩展字段的实时行情数据
    """
    try:
        service = get_stock_data_service()
        quotes = await service.get_market_quotes(symbol)
        if not quotes:
            return MarketQuotesResponse(
                success=False,
                message=f"未找到股票代码 {symbol} 的行情数据"
            )
        return MarketQuotesResponse(
            success=True,
            data=quotes,
            message="获取成功"
        )
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取实时行情失败: {str(e)}"
        )
@router.get("/list", response_model=StockListResponse)
async def get_stock_list(
    market: Optional[str] = Query(None, description="市场筛选"),
    industry: Optional[str] = Query(None, description="行业筛选"),
    page: int = Query(1, ge=1, description="页码"),
    page_size: int = Query(20, ge=1, le=100, description="每页大小"),
    current_user: dict = Depends(get_current_user)
):
    """
    获取股票列表

    Args:
        market: 市场筛选 (可选)
        industry: 行业筛选 (可选)
        page: 页码 (从1开始)
        page_size: 每页大小 (1-100)

    Returns:
        StockListResponse: 股票列表数据
    """
    try:
        service = get_stock_data_service()
        stock_list = await service.get_stock_list(
            market=market,
            industry=industry,
            page=page,
            page_size=page_size
        )

        # 计算总数 (简化实现，实际应该单独查询)
        total = len(stock_list)

        return StockListResponse(
            success=True,
            data=stock_list,
            total=total,
            page=page,
            page_size=page_size,
            message="获取成功"
        )

    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
```

```
        detail=f"获取股票列表失败: {str(e)}"
    )
@router.get("/combined/{symbol}")
async def get_combined_stock_data(
    symbol: str,
    current_user: dict = Depends(get_current_user)
):
    """
    获取股票综合数据 (基础信息 + 实时行情)
    Args:
        symbol: 股票代码
    Returns:
        dict: 包含基础信息和实时行情的综合数据
    """
    try:
        service = get_stock_data_service()
        # 并行获取基础信息和行情数据
        import asyncio
        basic_info_task = service.get_stock_basic_info(symbol)
        quotes_task = service.get_market_quotes(symbol)
        basic_info, quotes = await asyncio.gather(
            basic_info_task,
            quotes_task,
            return_exceptions=True
        )
        # 处理异常
        if isinstance(basic_info, Exception):
            basic_info = None
        if isinstance(quotes, Exception):
            quotes = None
        if not basic_info and not quotes:
            return {
                "success": False,
                "message": f"未找到股票代码 {symbol} 的任何数据"
            }
        return {
            "success": True,
            "data": {
                "basic_info": basic_info.dict() if basic_info else None,
                "quotes": quotes.dict() if quotes else None,
                "symbol": symbol,
                "timestamp": quotes.updated_at if quotes else None
            },
            "message": "获取成功"
        }
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取股票综合数据失败: {str(e)}"
        )
@router.get("/search")
async def search_stocks(
    keyword: str = Query(..., min_length=1, description="搜索关键词"),
    limit: int = Query(10, ge=1, le=50, description="返回数量限制"),
    current_user: dict = Depends(get_current_user)
):
    """
    搜索股票

    Args:
        keyword: 搜索关键词 (股票代码或名称)
        limit: 返回数量限制

    Returns:
        dict: 搜索结果
    """
    try:
        from app.core.database import get_mongo_db
        from app.core.unified_config import UnifiedConfigManager
        db = get_mongo_db()
        collection = db.stock_basic_info
        # 🔥 获取数据源优先级配置
        config = UnifiedConfigManager()
        data_source_configs = await config.get_data_source_configs_async()
        # 提取启用的数据源，按优先级排序
        enabled_sources = [
            ds.type.lower() for ds in data_source_configs
            if ds.enabled and ds.type.lower() in ['tushare', 'akshare', 'baostock']
        ]
        if not enabled_sources:
            enabled_sources = ['tushare', 'akshare', 'baostock']
        preferred_source = enabled_sources[0] if enabled_sources else 'tushare'
        # 构建搜索条件
        search_conditions = []
        # 如果是6位数字，按代码精确匹配
        if keyword.isdigit() and len(keyword) == 6:
            search_conditions.append({"symbol": keyword})
        else:
            # 按名称模糊匹配
            search_conditions.append({"name": {"$regex": keyword, "$options": "i"}})
            # 如果包含数字，也尝试代码匹配
            if any(c.isdigit() for c in keyword):
                search_conditions.append({"symbol": {"$regex": keyword}})
        # 🔥 添加数据源筛选，只查询优先级最高的数据源
        query = {
            "$and": [
                {"$or": search_conditions},
                {"source": preferred_source}
            ]
        }
        # 执行搜索
        cursor = collection.find(query, {"_id": 0}).limit(limit)
```

```
        results = await cursor.to_list(length=limit)
        # 数据标准化
        service = get_stock_data_service()
        standardized_results = []
        for doc in results:
            standardized_doc = service._standardize_basic_info(doc)
            standardized_results.append(standardized_doc)
        return {
            "success": True,
            "data": standardized_results,
            "total": len(standardized_results),
            "keyword": keyword,
            "source": preferred_source,  # 🔥 返回数据来源
            "message": "搜索完成"
        }

    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"搜索股票失败: {str(e)}"
        )
@router.get("/markets")
async def get_market_summary(
    current_user: dict = Depends(get_current_user)
):
    """
    获取市场概览
    Returns:
        dict: 各市场的股票数量统计
    """
    try:
        from app.core.database import get_mongo_db
        db = get_mongo_db()
        collection = db.stock_basic_info
        # 统计各市场股票数量
        pipeline = [
            {
                "$group": {
                    "_id": "$market",
                    "count": {"$sum": 1}
                }
            },
            {
                "$sort": {"count": -1}
            }
        ]
        cursor = collection.aggregate(pipeline)
        market_stats = await cursor.to_list(length=None)
        # 总数统计
        total_count = await collection.count_documents({})
        return {
            "success": True,
            "data": {
                "total_stocks": total_count,
                "market_breakdown": market_stats,
                "supported_markets": ["CN"],  # 当前支持的市场
                "last_updated": None  # 可以从数据中获取最新更新时间
            },
            "message": "获取成功"
        }
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取市场概览失败: {str(e)}"
        )
@router.get("/sync-status/quotes")
async def get_quotes_sync_status(
    current_user: dict = Depends(get_current_user)
):
    """
    获取实时行情同步状态
    Returns:
        dict: {
            "success": True,
            "data": {
                "last_sync_time": "2025-10-28 15:06:00",
                "last_sync_time_iso": "2025-10-28T15:06:00+08:00",
                "interval_seconds": 360,
                "interval_minutes": 6,
                "data_source": "tushare",
                "success": True,
                "records_count": 5440,
                "error_message": None
            },
            "message": "获取成功"
        }
    """
    try:
        from app.services.quotes_ingestion_service import QuotesIngestionService
        service = QuotesIngestionService()
        status_data = await service.get_sync_status()
        return {
            "success": True,
            "data": status_data,
            "message": "获取成功"
        }
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"获取同步状态失败: {str(e)}"
        )
```

FILE: app/routers/stock_sync.py

```
---------------------------------------------------------------------------------------------------------------------------------
"""
股票数据同步API路由
支持单个股票或批量股票的历史数据和财务数据同步
"""
from typing import List, Optional
from fastapi import APIRouter, Depends, HTTPException, BackgroundTasks
from pydantic import BaseModel, Field
from app.routers.auth_db import get_current_user
from app.core.response import ok
from app.core.database import get_mongo_db
from app.worker.tushare_sync_service import get_tushare_sync_service
from app.worker.akshare_sync_service import get_akshare_sync_service
from app.worker.financial_data_sync_service import get_financial_sync_service
import logging
import asyncio
from datetime import datetime, timedelta
logger = logging.getLogger("webapi")
router = APIRouter(prefix="/api/stock-sync", tags=["股票数据同步"])
async def _sync_latest_to_market_quotes(symbol: str) -> None:
    """
    将 stock_daily_quotes 中的最新数据同步到 market_quotes
    智能判断逻辑：
    - 如果 market_quotes 中已有更新的数据（trade_date 更新），则不覆盖
    - 如果 market_quotes 中没有数据或数据较旧，则更新
    Args:
        symbol: 股票代码（6位）
    """
    db = get_mongo_db()
    symbol6 = str(symbol).zfill(6)
    # 从 stock_daily_quotes 获取最新数据
    latest_doc = await db.stock_daily_quotes.find_one(
        {"symbol": symbol6},
        sort=[("trade_date", -1)]
    )
    if not latest_doc:
        logger.warning(f"⚠️ {symbol6}: stock_daily_quotes 中没有数据")
        return
    historical_trade_date = latest_doc.get("trade_date")
    # 🔥 检查 market_quotes 中是否已有更新的数据
    existing_quote = await db.market_quotes.find_one({"code": symbol6})
    if existing_quote:
        existing_trade_date = existing_quote.get("trade_date")
        # 如果 market_quotes 中的数据日期更新或相同，则不覆盖
        if existing_trade_date and historical_trade_date:
            # 比较日期字符串（格式：YYYY-MM-DD 或 YYYYMMDD）
            existing_date_str = str(existing_trade_date).replace("-", "")
            historical_date_str = str(historical_trade_date).replace("-", "")
            if existing_date_str >= historical_date_str:
                # 🔥 日期相同或更新时，都不覆盖（避免用历史数据覆盖实时数据）
                logger.info(
                    f"⏭️ {symbol6}: market_quotes 中的数据日期 >= 历史数据日期 "
                    f"(market_quotes: {existing_trade_date}, historical: {historical_trade_date})，跳过覆盖"
                )
                return
    # 提取需要的字段
    quote_data = {
        "code": symbol6,
        "symbol": symbol6,
        "close": latest_doc.get("close"),
        "open": latest_doc.get("open"),
        "high": latest_doc.get("high"),
        "low": latest_doc.get("low"),
        "volume": latest_doc.get("volume"),  # 已经转换过单位
        "amount": latest_doc.get("amount"),  # 已经转换过单位
        "pct_chg": latest_doc.get("pct_chg"),
        "pre_close": latest_doc.get("pre_close"),
        "trade_date": latest_doc.get("trade_date"),
        "updated_at": datetime.utcnow()
    }
    # 🔥 日志：记录同步的成交量
    logger.info(
        f"📊 [同步到market_quotes] {symbol6} - "
        f"volume={quote_data['volume']}, amount={quote_data['amount']}, trade_date={quote_data['trade_date']}"
    )
    # 更新 market_quotes
    await db.market_quotes.update_one(
        {"code": symbol6},
        {"$set": quote_data},
        upsert=True
    )
class SingleStockSyncRequest(BaseModel):
    """单股票同步请求"""
    symbol: str = Field(..., description="股票代码（6位）")
    sync_realtime: bool = Field(False, description="是否同步实时行情")
    sync_historical: bool = Field(True, description="是否同步历史数据")
    sync_financial: bool = Field(True, description="是否同步财务数据")
    sync_basic: bool = Field(False, description="是否同步基础数据")
    data_source: str = Field("tushare", description="数据源: tushare/akshare")
    days: int = Field(30, description="历史数据天数", ge=1, le=3650)
class BatchStockSyncRequest(BaseModel):
    """批量股票同步请求"""
    symbols: List[str] = Field(..., description="股票代码列表")
    sync_historical: bool = Field(True, description="是否同步历史数据")
    sync_financial: bool = Field(True, description="是否同步财务数据")
    sync_basic: bool = Field(False, description="是否同步基础数据")
    data_source: str = Field("tushare", description="数据源: tushare/akshare")
    days: int = Field(30, description="历史数据天数", ge=1, le=3650)
@router.post("/single")
async def sync_single_stock(
    request: SingleStockSyncRequest,
```

```
    background_tasks: BackgroundTasks,
    current_user: dict = Depends(get_current_user)
):
    """
    同步单个股票的历史数据、财务数据和实时行情
    - **symbol**: 股票代码（6位）
    - **sync_realtime**: 是否同步实时行情
    - **sync_historical**: 是否同步历史数据
    - **sync_financial**: 是否同步财务数据
    - **data_source**: 数据源（tushare/akshare）
    - **days**: 历史数据天数
    """
    try:
        logger.info(f"📊 开始同步单个股票: {request.symbol} (数据源: {request.data_source})")
        result = {
            "symbol": request.symbol,
            "realtime_sync": None,
            "historical_sync": None,
            "financial_sync": None,
            "basic_sync": None
        }
        # 同步实时行情
        if request.sync_realtime:
            try:
                # 🔥 单个股票实时行情同步：优先使用 AKShare（避免 Tushare 接口限制）
                actual_data_source = request.data_source
                if request.data_source == "tushare":
                    logger.info(f"💡 单个股票实时行情同步，自动切换到 AKShare 数据源（避免 Tushare 接口限制）")
                    actual_data_source = "akshare"
                if actual_data_source == "tushare":
                    service = await get_tushare_sync_service()
                elif actual_data_source == "akshare":
                    service = await get_akshare_sync_service()
                else:
                    raise ValueError(f"不支持的数据源: {actual_data_source}")
                # 同步实时行情（只同步指定的股票）
                realtime_result = await service.sync_realtime_quotes(
                    symbols=[request.symbol],
                    force=True  # 强制执行，跳过交易时间检查
                )
                # 🔥 如果 AKShare 同步失败，回退到 Tushare 全量同步
                if actual_data_source == "akshare" and realtime_result.get("success_count", 0) == 0:
                    logger.warning(f"⚠️ AKShare 同步失败，回退到 Tushare 全量同步")
                    logger.info(f"💡 Tushare 只支持全量同步，将同步所有股票的实时行情")
                    tushare_service = await get_tushare_sync_service()
                    if tushare_service:
                        # 使用 Tushare 全量同步（不指定 symbols，同步所有股票）
                        realtime_result = await tushare_service.sync_realtime_quotes(
                            symbols=None,  # 全量同步
                            force=True
                        )
                        logger.info(f"✅ Tushare 全量同步完成: 成功 {realtime_result.get('success_count', 0)} 只")
                    else:
                        logger.error(f"❌ Tushare 服务不可用，无法回退")
                        realtime_result["fallback_failed"] = True
                success = realtime_result.get("success_count", 0) > 0
                # 🔥 如果切换了数据源，在消息中说明
                message = f"实时行情同步{'成功' if success else '失败'}"
                if request.data_source == "tushare" and actual_data_source == "akshare":
                    message += "（已自动切换到 AKShare 数据源）"
                result["realtime_sync"] = {
                    "success": success,
                    "message": message,
                    "data_source_used": actual_data_source  # 🔥 返回实际使用的数据源
                }
                logger.info(f"✅ {request.symbol} 实时行情同步完成: {success}")
            except Exception as e:
                logger.error(f"❌ {request.symbol} 实时行情同步失败: {e}")
                result["realtime_sync"] = {
                    "success": False,
                    "error": str(e)
                }

        # 同步历史数据
        if request.sync_historical:
            try:
                if request.data_source == "tushare":
                    service = await get_tushare_sync_service()
                elif request.data_source == "akshare":
                    service = await get_akshare_sync_service()
                else:
                    raise ValueError(f"不支持的数据源: {request.data_source}")
                # 计算日期范围
                end_date = datetime.now().strftime('%Y-%m-%d')
                start_date = (datetime.now() - timedelta(days=request.days)).strftime('%Y-%m-%d')
                # 同步历史数据
                hist_result = await service.sync_historical_data(
                    symbols=[request.symbol],
                    start_date=start_date,
                    end_date=end_date,
                    incremental=False
                )
                result["historical_sync"] = {
                    "success": hist_result.get("success_count", 0) > 0,
                    "records": hist_result.get("total_records", 0),
                    "message": f"同步了 {hist_result.get('total_records', 0)} 条历史记录"
                }
                logger.info(f"✅ {request.symbol} 历史数据同步完成: {hist_result.get('total_records', 0)} 条记录")
                # 🔥 同步最新历史数据到 market_quotes
                if hist_result.get("success_count", 0) > 0:
                    try:
                        await _sync_latest_to_market_quotes(request.symbol)
                        logger.info(f"✅ {request.symbol} 最新数据已同步到 market_quotes")
```

```
            except Exception as e:
                logger.warning(f"⚠️ {request.symbol} 同步到 market_quotes 失败: {e}")
        # 🔥 【已禁用】如果没有勾选实时行情，但在交易时间内，自动同步实时行情
        # 用户反馈：不希望自动同步实时行情，应该严格按照用户的选择
        # if not request.sync_realtime:
        #     from app.utils.trading_time import is_trading_time
        #     if is_trading_time():
        #         logger.info(f"📊 {request.symbol} 当前在交易时间内，自动同步实时行情")
        #         try:
        #             realtime_result = await service.sync_realtime_quotes(
        #                 symbols=[request.symbol],
        #                 force=True
        #             )
        #             if realtime_result.get("success_count", 0) > 0:
        #                 logger.info(f"✅ {request.symbol} 实时行情自动同步成功")
        #                 result["realtime_sync"] = {
        #                     "success": True,
        #                     "message": "实时行情自动同步成功（交易时间内）"
        #                 }
        #         except Exception as e:
        #             logger.warning(f"⚠️ {request.symbol} 实时行情自动同步失败: {e}")
    except Exception as e:
        logger.error(f"❌ {request.symbol} 历史数据同步失败: {e}")
        result["historical_sync"] = {
            "success": False,
            "error": str(e)
        }

# 同步财务数据
if request.sync_financial:
    try:
        financial_service = await get_financial_sync_service()

        # 同步财务数据
        fin_result = await financial_service.sync_single_stock(
            symbol=request.symbol,
            data_sources=[request.data_source]
        )

        success = fin_result.get(request.data_source, False)
        result["financial_sync"] = {
            "success": success,
            "message": "财务数据同步成功" if success else "财务数据同步失败"
        }
        logger.info(f"✅ {request.symbol} 财务数据同步完成: {success}")

    except Exception as e:
        logger.error(f"❌ {request.symbol} 财务数据同步失败: {e}")
        result["financial_sync"] = {
            "success": False,
            "error": str(e)
        }
# 同步基础数据
if request.sync_basic:
    try:
        # 🔥 同步单个股票的基础数据
        # 参考 basics_sync_service 的实现逻辑
        if request.data_source == "tushare":
            from app.services.basics_sync import (
                fetch_stock_basic_df,
                find_latest_trade_date,
                fetch_daily_basic_mv_map,
                fetch_latest_roe_map,
            )
            db = get_mongo_db()
            symbol6 = str(request.symbol).zfill(6)
            # Step 1: 获取股票基础信息
            stock_df = await asyncio.to_thread(fetch_stock_basic_df)
            if stock_df is None or stock_df.empty:
                result["basic_sync"] = {
                    "success": False,
                    "error": "Tushare 返回空数据"
                }
            else:
                # 筛选出目标股票
                stock_row = None
                for _, row in stock_df.iterrows():
                    ts_code = row.get("ts_code", "")
                    if isinstance(ts_code, str) and ts_code.startswith(symbol6):
                        stock_row = row
                        break
                if stock_row is None:
                    result["basic_sync"] = {
                        "success": False,
                        "error": f"未找到股票 {symbol6} 的基础信息"
                    }
                else:
                    # Step 2: 获取最新交易日和财务指标
                    latest_trade_date = await asyncio.to_thread(find_latest_trade_date)
                    daily_data_map = await asyncio.to_thread(fetch_daily_basic_mv_map, latest_trade_date)
                    roe_map = await asyncio.to_thread(fetch_latest_roe_map)
                    # Step 3: 构建文档（参考 basics_sync_service 的逻辑）
                    # 🔥 先获取当前时间，避免作用域问题
                    now_iso = datetime.utcnow().isoformat()
                    name = stock_row.get("name") or ""
                    area = stock_row.get("area") or ""
                    industry = stock_row.get("industry") or ""
                    market = stock_row.get("market") or ""
                    list_date = stock_row.get("list_date") or ""
                    ts_code = stock_row.get("ts_code") or ""
                    # 提取6位代码
                    if isinstance(ts_code, str) and "." in ts_code:
                        code = ts_code.split(".")[0]
                    else:
```

```
            code = symbol6
        # 判断交易所
        if isinstance(ts_code, str):
            if ts_code.endswith(".SH"):
                sse = "上海证券交易所"
            elif ts_code.endswith(".SZ"):
                sse = "深圳证券交易所"
            elif ts_code.endswith(".BJ"):
                sse = "北京证券交易所"
            else:
                sse = "未知"
        else:
            sse = "未知"
        # 生成 full_symbol
        full_symbol = ts_code
        # 提取财务指标
        daily_metrics = {}
        if isinstance(ts_code, str) and ts_code in daily_data_map:
            daily_metrics = daily_data_map[ts_code]
        # 市值转换（万元 -> 亿元）
        total_mv_yi = None
        circ_mv_yi = None
        if "total_mv" in daily_metrics:
            try:
                total_mv_yi = float(daily_metrics["total_mv"]) / 10000.0
            except Exception:
                pass
        if "circ_mv" in daily_metrics:
            try:
                circ_mv_yi = float(daily_metrics["circ_mv"]) / 10000.0
            except Exception:
                pass
        # 构建文档
        doc = {
            "code": code,
            "symbol": code,
            "name": name,
            "area": area,
            "industry": industry,
            "market": market,
            "list_date": list_date,
            "sse": sse,
            "sec": "stock_cn",
            "source": "tushare",
            "updated_at": now_iso,
            "full_symbol": full_symbol,
        }
        # 添加市值
        if total_mv_yi is not None:
            doc["total_mv"] = total_mv_yi
        if circ_mv_yi is not None:
            doc["circ_mv"] = circ_mv_yi
        # 添加估值指标
        for field in ["pe", "pb", "ps", "pe_ttm", "pb_mrq", "ps_ttm"]:
            if field in daily_metrics:
                doc[field] = daily_metrics[field]
        # 添加 ROE
        if isinstance(ts_code, str) and ts_code in roe_map:
            roe_val = roe_map[ts_code].get("roe")
            if roe_val is not None:
                doc["roe"] = roe_val
        # 添加交易指标
        for field in ["turnover_rate", "volume_ratio"]:
            if field in daily_metrics:
                doc[field] = daily_metrics[field]
        # 添加股本信息
        for field in ["total_share", "float_share"]:
            if field in daily_metrics:
                doc[field] = daily_metrics[field]
        # Step 4: 更新数据库
        await db.stock_basic_info.update_one(
            {"code": code, "source": "tushare"},
            {"$set": doc},
            upsert=True
        )
        result["basic_sync"] = {
            "success": True,
            "message": "基础数据同步成功"
        }
        logger.info(f"✅ {request.symbol} 基础数据同步完成")
elif request.data_source == "akshare":
    # 🔥 AKShare 数据源的基础数据同步
    db = get_mongo_db()
    symbol6 = str(request.symbol).zfill(6)
    # 获取 AKShare 同步服务
    service = await get_akshare_sync_service()
    # 获取股票基础信息
    basic_info = await service.provider.get_stock_basic_info(symbol6)
    if basic_info:
        # 转换为字典格式
        if hasattr(basic_info, 'model_dump'):
            basic_data = basic_info.model_dump()
        elif hasattr(basic_info, 'dict'):
            basic_data = basic_info.dict()
        else:
            basic_data = basic_info
        # 确保必要字段
        basic_data["code"] = symbol6
        basic_data["symbol"] = symbol6
        basic_data["source"] = "akshare"
        basic_data["updated_at"] = datetime.utcnow().isoformat()
        # 更新到数据库
        await db.stock_basic_info.update_one(
```

```
                    {"code": symbol6, "source": "akshare"},
                    {"$set": basic_data},
                    upsert=True
                )
                result["basic_sync"] = {
                    "success": True,
                    "message": "基础数据同步成功"
                }
                logger.info(f"☑ {request.symbol} 基础数据同步完成 (AKShare)")
            else:
                result["basic_sync"] = {
                    "success": False,
                    "error": "未获取到基础数据"
                }
        else:
            result["basic_sync"] = {
                "success": False,
                "error": f"基础数据同步仅支持 Tushare/AKShare 数据源，当前数据源: {request.data_source}"
            }
    except Exception as e:
        logger.error(f"❌ {request.symbol} 基础数据同步失败: {e}")
        result["basic_sync"] = {
            "success": False,
            "error": str(e)
        }
# 判断整体是否成功
overall_success = (
    (not request.sync_realtime or result["realtime_sync"].get("success", False)) and
    (not request.sync_historical or result["historical_sync"].get("success", False)) and
    (not request.sync_financial or result["financial_sync"].get("success", False)) and
    (not request.sync_basic or result["basic_sync"].get("success", False))
)
# 添加整体成功标志到结果中
result["overall_success"] = overall_success
return ok(
    data=result,
    message=f"股票 {request.symbol} 数据同步{'成功' if overall_success else '部分失败'}"
)

except Exception as e:
    logger.error(f"❌ 同步单个股票失败: {e}")
    raise HTTPException(status_code=500, detail=f"同步失败: {str(e)}")

@router.post("/batch")
async def sync_batch_stocks(
    request: BatchStockSyncRequest,
    background_tasks: BackgroundTasks,
    current_user: dict = Depends(get_current_user)
):
    """
    批量同步多个股票的历史数据和财务数据

    - **symbols**: 股票代码列表
    - **sync_historical**: 是否同步历史数据
    - **sync_financial**: 是否同步财务数据
    - **data_source**: 数据源（tushare/akshare）
    - **days**: 历史数据天数
    """
    try:
        logger.info(f"📊 开始批量同步 {len(request.symbols)} 只股票 (数据源: {request.data_source})")

        result = {
            "total": len(request.symbols),
            "symbols": request.symbols,
            "historical_sync": None,
            "financial_sync": None,
            "basic_sync": None
        }

        # 同步历史数据
        if request.sync_historical:
            try:
                if request.data_source == "tushare":
                    service = await get_tushare_sync_service()
                elif request.data_source == "akshare":
                    service = await get_akshare_sync_service()
                else:
                    raise ValueError(f"不支持的数据源: {request.data_source}")
                # 计算日期范围
                end_date = datetime.now().strftime('%Y-%m-%d')
                start_date = (datetime.now() - timedelta(days=request.days)).strftime('%Y-%m-%d')

                # 批量同步历史数据
                hist_result = await service.sync_historical_data(
                    symbols=request.symbols,
                    start_date=start_date,
                    end_date=end_date,
                    incremental=False
                )

                result["historical_sync"] = {
                    "success_count": hist_result.get("success_count", 0),
                    "error_count": hist_result.get("error_count", 0),
                    "total_records": hist_result.get("total_records", 0),
                    "message": f"成功同步 {hist_result.get('success_count', 0)}/{len(request.symbols)} 只股票，共 {hist_result.get('total_records', 0)} 条记录"
                }
                logger.info(f"☑ 批量历史数据同步完成: {hist_result.get('success_count', 0)}/{len(request.symbols)}")

            except Exception as e:
                logger.error(f"❌ 批量历史数据同步失败: {e}")
                result["historical_sync"] = {
                    "success_count": 0,
                    "error_count": len(request.symbols),
                    "error": str(e)
                }
```

```
# 同步财务数据
if request.sync_financial:
    try:
        financial_service = await get_financial_sync_service()

        # 批量同步财务数据
        fin_results = await financial_service.sync_financial_data(
            symbols=request.symbols,
            data_sources=[request.data_source],
            batch_size=10
        )

        source_stats = fin_results.get(request.data_source)
        if source_stats:
            result["financial_sync"] = {
                "success_count": source_stats.success_count,
                "error_count": source_stats.error_count,
                "total_symbols": source_stats.total_symbols,
                "message": f"成功同步 {source_stats.success_count}/{source_stats.total_symbols} 只股票的财务数据"
            }
        else:
            result["financial_sync"] = {
                "success_count": 0,
                "error_count": len(request.symbols),
                "message": "财务数据同步失败"
            }

        logger.info(f"☑ 批量财务数据同步完成: {result['financial_sync']['success_count']}/{len(request.symbols)}")

    except Exception as e:
        logger.error(f"❌ 批量财务数据同步失败: {e}")
        result["financial_sync"] = {
            "success_count": 0,
            "error_count": len(request.symbols),
            "error": str(e)
        }
# 同步基础数据
if request.sync_basic:
    try:
        # 💧 批量同步基础数据
        # 注意：基础数据同步服务目前只支持 Tushare 数据源
        if request.data_source == "tushare":
            from tradingagents.dataflows.providers.china.tushare import TushareProvider
            tushare_provider = TushareProvider()
            if tushare_provider.is_available():
                success_count = 0
                error_count = 0
                for symbol in request.symbols:
                    try:
                        basic_info = await tushare_provider.get_stock_basic_info(symbol)
                        if basic_info:
                            # 保存到 MongoDB
                            db = get_mongo_db()
                            symbol6 = str(symbol).zfill(6)
                            # 添加必要字段
                            basic_info["code"] = symbol6
                            basic_info["source"] = "tushare"
                            basic_info["updated_at"] = datetime.utcnow()
                            await db.stock_basic_info.update_one(
                                {"code": symbol6, "source": "tushare"},
                                {"$set": basic_info},
                                upsert=True
                            )
                            success_count += 1
                            logger.info(f"☑ {symbol} 基础数据同步成功")
                        else:
                            error_count += 1
                            logger.warning(f"⚠️ {symbol} 未获取到基础数据")
                    except Exception as e:
                        error_count += 1
                        logger.error(f"❌ {symbol} 基础数据同步失败: {e}")
                result["basic_sync"] = {
                    "success_count": success_count,
                    "error_count": error_count,
                    "total_symbols": len(request.symbols),
                    "message": f"成功同步 {success_count}/{len(request.symbols)} 只股票的基础数据"
                }
                logger.info(f"☑ 批量基础数据同步完成: {success_count}/{len(request.symbols)}")
            else:
                result["basic_sync"] = {
                    "success_count": 0,
                    "error_count": len(request.symbols),
                    "error": "Tushare 数据源不可用"
                }
        else:
            result["basic_sync"] = {
                "success_count": 0,
                "error_count": len(request.symbols),
                "error": f"基础数据同步仅支持 Tushare 数据源，当前数据源: {request.data_source}"
            }
    except Exception as e:
        logger.error(f"❌ 批量基础数据同步失败: {e}")
        result["basic_sync"] = {
            "success_count": 0,
            "error_count": len(request.symbols),
            "error": str(e)
        }
# 判断整体是否成功
hist_success = result["historical_sync"].get("success_count", 0) if request.sync_historical else 0
fin_success = result["financial_sync"].get("success_count", 0) if request.sync_financial else 0
basic_success = result["basic_sync"].get("success_count", 0) if request.sync_basic else 0
total_success = max(hist_success, fin_success, basic_success)
# 添加统计信息到结果中
result["total_success"] = total_success
```

```
        result["total_symbols"] = len(request.symbols)
        return ok(
            data=result,
            message=f"批量同步完成: {total_success}/{len(request.symbols)} 只股票成功"
        )

    except Exception as e:
        logger.error(f"❌ 批量同步失败: {e}")
        raise HTTPException(status_code=500, detail=f"批量同步失败: {str(e)}")
@router.get("/status/{symbol}")
async def get_sync_status(
    symbol: str,
    current_user: dict = Depends(get_current_user)
):
    """
    获取股票的同步状态

    返回最后同步时间、数据条数等信息
    """
    try:
        from app.core.database import get_mongo_db

        db = get_mongo_db()

        # 查询历史数据最后同步时间
        hist_doc = await db.historical_data.find_one(
            {"symbol": symbol},
            sort=[("date", -1)]
        )

        # 查询财务数据最后同步时间
        fin_doc = await db.stock_financial_data.find_one(
            {"symbol": symbol},
            sort=[("updated_at", -1)]
        )

        # 统计历史数据条数
        hist_count = await db.historical_data.count_documents({"symbol": symbol})

        # 统计财务数据条数
        fin_count = await db.stock_financial_data.count_documents({"symbol": symbol})

        return ok(data={
            "symbol": symbol,
            "historical_data": {
                "last_sync": hist_doc.get("updated_at") if hist_doc else None,
                "last_date": hist_doc.get("date") if hist_doc else None,
                "total_records": hist_count
            },
            "financial_data": {
                "last_sync": fin_doc.get("updated_at") if fin_doc else None,
                "last_report_period": fin_doc.get("report_period") if fin_doc else None,
                "total_records": fin_count
            }
        })

    except Exception as e:
        logger.error(f"❌ 获取同步状态失败: {e}")
        raise HTTPException(status_code=500, detail=f"获取同步状态失败: {str(e)}")
```

FILE: app/routers/stocks.py

```
"""
股票详情相关API
- 统一响应包: {success, data, message, timestamp}
- 所有端点均需鉴权 (Bearer Token)
- 路径前缀在 main.py 中挂载为 /api，当前路由自身前缀为 /stocks
"""
from typing import Optional, Dict, Any, List, Tuple
from fastapi import APIRouter, Depends, HTTPException, status, Query
import logging
import re
from app.routers.auth_db import get_current_user
from app.core.database import get_mongo_db
from app.core.response import ok
logger = logging.getLogger(__name__)
router = APIRouter(prefix="/stocks", tags=["stocks"])
def _zfill_code(code: str) -> str:
    try:
        s = str(code).strip()
        if len(s) == 6 and s.isdigit():
            return s
        return s.zfill(6)
    except Exception:
        return str(code)
def _detect_market_and_code(code: str) -> Tuple[str, str]:
    """
    检测股票代码的市场类型并标准化代码
    Args:
        code: 股票代码
    Returns:
        (market, normalized_code): 市场类型和标准化后的代码
            - CN: A股（6位数字）
            - HK: 港股（4-5位数字或带.HK后缀）
            - US: 美股（字母代码）
    """
    code = code.strip().upper()
    # 港股：带.HK后缀
    if code.endswith('.HK'):
        return ('HK', code[:-3].zfill(5))  # 移除.HK，补齐到5位
    # 美股：纯字母
    if re.match(r'^[A-Z]+$', code):
```

```
        return ('US', code)
    # 港股：4-5位数字
    if re.match(r'^\d{4,5}$', code):
        return ('HK', code.zfill(5))  # 补齐到5位
    # A股：6位数字
    if re.match(r'^\d{6}$', code):
        return ('CN', code)
    # 默认当作A股处理
    return ('CN', _zfill_code(code))

@router.get("/{code}/quote", response_model=dict)
async def get_quote(
    code: str,
    force_refresh: bool = Query(False, description="是否强制刷新（跳过缓存）"),
    current_user: dict = Depends(get_current_user)
):
    """
    获取股票实时行情（支持A股/港股/美股）
    自动识别市场类型：
    - 6位数字 → A股
    - 4位数字或.HK → 港股
    - 纯字母 → 美股
    参数：
    - code: 股票代码
    - force_refresh: 是否强制刷新（跳过缓存）
    返回字段（data内，蛇形命名）：
      - code, name, market
      - price(close), change_percent(pct_chg), amount, prev_close(估算)
      - turnover_rate, amplitude（振幅，替代量比）
      - trade_date, updated_at
    """
    # 检测市场类型
    market, normalized_code = _detect_market_and_code(code)
    # 港股和美股：使用新服务
    if market in ['HK', 'US']:
        from app.services.foreign_stock_service import ForeignStockService
        db = get_mongo_db()  # 不需要 await，直接返回数据库对象
        service = ForeignStockService(db=db)
        try:
            quote = await service.get_quote(market, normalized_code, force_refresh)
            return ok(data=quote)
        except Exception as e:
            logger.error(f"获取{market}股票{code}行情失败: {e}")
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail=f"获取行情失败: {str(e)}"
            )
    # A股：使用现有逻辑
    db = get_mongo_db()
    code6 = normalized_code
    # 行情
    q = await db["market_quotes"].find_one({"code": code6}, {"_id": 0})
    # 🔥 调试日志：查看查询结果
    logger.info(f"🔍 查询 market_quotes: code={code6}")
    if q:
        logger.info(f"  ✅ 找到数据: volume={q.get('volume')}, amount={q.get('amount')}, volume_ratio={q.get('volume_ratio')}")
    else:
        logger.info(f"  ❌ 未找到数据")
    # 🔥 基础信息 - 按数据源优先级查询
    from app.core.unified_config import UnifiedConfigManager
    config = UnifiedConfigManager()
    data_source_configs = await config.get_data_source_configs_async()
    # 提取启用的数据源，按优先级排序
    enabled_sources = [
        ds.type.lower() for ds in data_source_configs
        if ds.enabled and ds.type.lower() in ['tushare', 'akshare', 'baostock']
    ]
    if not enabled_sources:
        enabled_sources = ['tushare', 'akshare', 'baostock']
    # 按优先级查询基础信息
    b = None
    for src in enabled_sources:
        b = await db["stock_basic_info"].find_one({"code": code6, "source": src}, {"_id": 0})
        if b:
            break
    # 如果所有数据源都没有，尝试不带 source 条件查询（兼容旧数据）
    if not b:
        b = await db["stock_basic_info"].find_one({"code": code6}, {"_id": 0})
    if not q and not b:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="未找到该股票的任何信息")
    close = (q or {}).get("close")
    pct = (q or {}).get("pct_chg")
    pre_close_saved = (q or {}).get("pre_close")
    prev_close = pre_close_saved
    if prev_close is None:
        try:
            if close is not None and pct is not None:
                prev_close = round(float(close) / (1.0 + float(pct) / 100.0), 4)
        except Exception:
            prev_close = None
    # 🔥 优先从 market_quotes 获取 turnover_rate（实时数据）
    # 如果 market_quotes 中没有，再从 stock_basic_info 获取（日度数据）
    turnover_rate = (q or {}).get("turnover_rate")
    turnover_rate_date = None
    if turnover_rate is None:
        turnover_rate = (b or {}).get("turnover_rate")
        turnover_rate_date = (b or {}).get("trade_date")  # 来自日度数据
    else:
        turnover_rate_date = (q or {}).get("trade_date")  # 来自实时数据
    # 🔥 计算振幅（amplitude）替代量比（volume_ratio）
    # 振幅 = (最高价 - 最低价) / 昨收价 × 100%
    amplitude = None
    amplitude_date = None
```

```
    try:
        high = (q or {}).get("high")
        low = (q or {}).get("low")
        logger.info(f"🔍 计算振幅: high={high}, low={low}, prev_close={prev_close}")
        if high is not None and low is not None and prev_close is not None and prev_close > 0:
            amplitude = round((float(high) - float(low)) / float(prev_close) * 100, 2)
            amplitude_date = (q or {}).get("trade_date")  # 来自实时数据
            logger.info(f"✅ 振幅计算成功: {amplitude}%")
        else:
            logger.warning(f"⚠️ 数据不完整，无法计算振幅")
    except Exception as e:
        logger.warning(f"❌ 计算振幅失败: {e}")
        amplitude = None
    data = {
        "code": code6,
        "name": (b or {}).get("name"),
        "market": (b or {}).get("market"),
        "price": close,
        "change_percent": pct,
        "amount": (q or {}).get("amount"),
        "volume": (q or {}).get("volume"),
        "open": (q or {}).get("open"),
        "high": (q or {}).get("high"),
        "low": (q or {}).get("low"),
        "prev_close": prev_close,
        # 🔥 优先使用实时数据，降级到日度数据
        "turnover_rate": turnover_rate,
        "amplitude": amplitude,  # 🔥 新增：振幅（替代量比）
        "turnover_rate_date": turnover_rate_date,  # 🔥 新增：换手率数据日期
        "amplitude_date": amplitude_date,  # 🔥 新增：振幅数据日期
        "trade_date": (q or {}).get("trade_date"),
        "updated_at": (q or {}).get("updated_at"),
    }
    return ok(data)

@router.get("/{code}/fundamentals", response_model=dict)
async def get_fundamentals(
    code: str,
    source: Optional[str] = Query(None, description="数据源 (tushare/akshare/baostock/multi_source)"),
    force_refresh: bool = Query(False, description="是否强制刷新（跳过缓存）"),
    current_user: dict = Depends(get_current_user)
):
    """
    获取基础面快照（支持A股/港股/美股）
    数据来源优先级：
    1. stock_basic_info 集合（基础信息、估值指标）
    2. stock_financial_data 集合（财务指标：ROE、负债率等）
    参数：
    - code: 股票代码
    - source: 数据源（可选），默认按优先级：tushare > multi_source > akshare > baostock
    - force_refresh: 是否强制刷新（跳过缓存）
    """
    # 检测市场类型
    market, normalized_code = _detect_market_and_code(code)
    # 港股和美股：使用新服务
    if market in ['HK', 'US']:
        from app.services.foreign_stock_service import ForeignStockService
        db = get_mongo_db()  # 不需要 await，直接返回数据库对象
        service = ForeignStockService(db=db)
        try:
            info = await service.get_basic_info(market, normalized_code, force_refresh)
            return ok(data=info)
        except Exception as e:
            logger.error(f"获取{market}股票{code}基础信息失败: {e}")
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail=f"获取基础信息失败: {str(e)}"
            )
    # A股：使用现有逻辑
    db = get_mongo_db()
    code6 = normalized_code
    # 1. 获取基础信息（支持数据源筛选）
    query = {"code": code6}
    if source:
        # 指定数据源
        query["source"] = source
        b = await db["stock_basic_info"].find_one(query, {"_id": 0})
        if not b:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"未找到该股票在数据源 {source} 中的基础信息"
            )
    else:
        # 🔥 未指定数据源，按优先级查询
        source_priority = ["tushare", "multi_source", "akshare", "baostock"]
        b = None
        for src in source_priority:
            query_with_source = {"code": code6, "source": src}
            b = await db["stock_basic_info"].find_one(query_with_source, {"_id": 0})
            if b:
                logger.info(f"✅ 使用数据源: {src} 查询股票 {code6}")
                break
        # 如果所有数据源都没有，尝试不带 source 条件查询（兼容旧数据）
        if not b:
            b = await db["stock_basic_info"].find_one({"code": code6}, {"_id": 0})
            if b:
                logger.warning(f"⚠️ 使用旧数据（无 source 字段）: {code6}")
        if not b:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="未找到该股票的基础信息")
    # 2. 尝试从 stock_financial_data 获取最新财务指标
    # 🔥 按数据源优先级查询，而不是按时间戳，避免混用不同数据源的数据
    financial_data = None
    try:
        # 获取数据源优先级配置
```

```
        from app.core.unified_config import UnifiedConfigManager
        config = UnifiedConfigManager()
        data_source_configs = await config.get_data_source_configs_async()
        # 提取启用的数据源，按优先级排序
        enabled_sources = [
            ds.type.lower() for ds in data_source_configs
            if ds.enabled and ds.type.lower() in ['tushare', 'akshare', 'baostock']
        ]
        if not enabled_sources:
            enabled_sources = ['tushare', 'akshare', 'baostock']
        # 按数据源优先级查询财务数据
        for data_source in enabled_sources:
            financial_data = await db["stock_financial_data"].find_one(
                {"$or": [{"symbol": code6}, {"code": code6}], "data_source": data_source},
                {"_id": 0},
                sort=[("report_period", -1)]  # 按报告期降序，获取该数据源的最新数据
            )
            if financial_data:
                logger.info(f"☑ 使用数据源 {data_source} 的财务数据 (报告期: {financial_data.get('report_period')})")
                break
        if not financial_data:
            logger.warning(f"⚠      未找到 {code6} 的财务数据")
    except Exception as e:
        logger.error(f"获取财务数据失败: {e}")
    # 3. 获取实时PE/PB（优先使用实时计算）
    from tradingagents.dataflows.realtime_metrics import get_pe_pb_with_fallback
    import asyncio
    # 在线程池中执行同步的实时计算
    realtime_metrics = await asyncio.to_thread(
        get_pe_pb_with_fallback,
        code6,
        db.client
    )
    # 4. 构建返回数据
    # 💧 优先使用实时市值，降级到 stock_basic_info 的静态市值
    realtime_market_cap = realtime_metrics.get("market_cap")  # 实时市值（亿元）
    total_mv = realtime_market_cap if realtime_market_cap else b.get("total_mv")
    data = {
        "code": code6,
        "name": b.get("name"),
        "industry": b.get("industry"),  # 行业（如：银行、软件服务）
        "market": b.get("market"),      # 交易所（如：主板、创业板）
        # 板块信息：使用 market 字段（主板/创业板/科创板/北交所等）
        "sector": b.get("market"),
        # 估值指标（优先使用实时计算，降级到 stock_basic_info）
        "pe": realtime_metrics.get("pe") or b.get("pe"),
        "pb": realtime_metrics.get("pb") or b.get("pb"),
        "pe_ttm": realtime_metrics.get("pe_ttm") or b.get("pe_ttm"),
        "pb_mrq": realtime_metrics.get("pb_mrq") or b.get("pb_mrq"),
        # 💧 市销率（PS）- 动态计算（使用实时市值）
        "ps": None,
        "ps_ttm": None,
        # PE/PB 数据来源标识
        "pe_source": realtime_metrics.get("source", "unknown"),
        "pe_is_realtime": realtime_metrics.get("is_realtime", False),
        "pe_updated_at": realtime_metrics.get("updated_at"),
        # ROE（优先从 stock_financial_data 获取，其次从 stock_basic_info）
        "roe": None,
        # 负债率（从 stock_financial_data 获取）
        "debt_ratio": None,
        # 市值：优先使用实时市值，降级到静态市值
        "total_mv": total_mv,
        "circ_mv": b.get("circ_mv"),
        # 💧 市值来源标识
        "mv_is_realtime": bool(realtime_market_cap),
        # 交易指标（可能为空）
        "turnover_rate": b.get("turnover_rate"),
        "volume_ratio": b.get("volume_ratio"),
        "updated_at": b.get("updated_at"),
    }
    # 5. 从财务数据中提取 ROE、负债率和计算 PS
    if financial_data:
        # ROE（净资产收益率）
        if financial_data.get("financial_indicators"):
            indicators = financial_data["financial_indicators"]
            data["roe"] = indicators.get("roe")
            data["debt_ratio"] = indicators.get("debt_to_assets")
        # 如果 financial_indicators 中没有，尝试从顶层字段获取
        if data["roe"] is None:
            data["roe"] = financial_data.get("roe")
        if data["debt_ratio"] is None:
            data["debt_ratio"] = financial_data.get("debt_to_assets")
        # 💧 动态计算 PS（市销率）- 使用实时市值
        # 优先使用 TTM 营业收入，如果没有则使用单期营业收入
        revenue_ttm = financial_data.get("revenue_ttm")
        revenue = financial_data.get("revenue")
        revenue_for_ps = revenue_ttm if revenue_ttm and revenue_ttm > 0 else revenue
        if revenue_for_ps and revenue_for_ps > 0:
            # 💧 使用实时市值（如果有），否则使用静态市值
            if total_mv and total_mv > 0:
                # 营业收入单位：元，需要转换为亿元
                revenue_yi = revenue_for_ps / 100000000
                ps_calculated = total_mv / revenue_yi
                data["ps"] = round(ps_calculated, 2)
                data["ps_ttm"] = round(ps_calculated, 2) if revenue_ttm else None
    # 6. 如果财务数据中没有 ROE，使用 stock_basic_info 中的
    if data["roe"] is None:
        data["roe"] = b.get("roe")
    return ok(data)
@router.get("/{code}/kline", response_model=dict)
async def get_kline(
    code: str,
```

```
    period: str = "day",
    limit: int = 120,
    adj: str = "none",
    force_refresh: bool = Query(False, description="是否强制刷新（跳过缓存）"),
    current_user: dict = Depends(get_current_user)
):
    """
    获取K线数据（支持A股/港股/美股）
    period: day/week/month/5m/15m/30m/60m
    adj: none/qfq/hfq
    force_refresh: 是否强制刷新（跳过缓存）
    🔥 新增功能：当天实时K线数据
    - 交易时间内（09:30-15:00）：从 market_quotes 获取实时数据
    - 收盘后：检查历史数据是否有当天数据，没有则从 market_quotes 获取
    """
    import logging
    from datetime import datetime, timedelta, time as dtime
    from zoneinfo import ZoneInfo
    logger = logging.getLogger(__name__)
    valid_periods = {"day","week","month","5m","15m","30m","60m"}
    if period not in valid_periods:
        raise HTTPException(status_code=400, detail=f"不支持的period: {period}")
    # 检测市场类型
    market, normalized_code = _detect_market_and_code(code)
    # 港股和美股：使用新服务
    if market in ['HK', 'US']:
        from app.services.foreign_stock_service import ForeignStockService
        db = get_mongo_db()  # 不需要 await，直接返回数据库对象
        service = ForeignStockService(db=db)
        try:
            kline_data = await service.get_kline(market, normalized_code, period, limit, force_refresh)
            return ok(data={
                'code': normalized_code,
                'period': period,
                'items': kline_data,
                'source': 'cache_or_api'
            })
        except Exception as e:
            logger.error(f"获取{market}股票{code}K线数据失败: {e}")
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail=f"获取K线数据失败: {str(e)}"
            )
    # A股：使用现有逻辑
    code_padded = normalized_code
    adj_norm = None if adj in (None, "none", "", "null") else adj
    items = None
    source = None
    # 周期映射：前端 -> MongoDB
    period_map = {
        "day": "daily",
        "week": "weekly",
        "month": "monthly",
        "5m": "5min",
        "15m": "15min",
        "30m": "30min",
        "60m": "60min"
    }
    mongodb_period = period_map.get(period, "daily")
    # 获取当前时间（北京时间）
    from app.core.config import settings
    tz = ZoneInfo(settings.TIMEZONE)
    now = datetime.now(tz)
    today_str_yyyymmdd = now.strftime("%Y%m%d")  # 格式：20251028（用于查询）
    today_str_formatted = now.strftime("%Y-%m-%d")  # 格式：2025-10-28（用于返回）
    # 1. 优先从 MongoDB 缓存获取
    try:
        from tradingagents.dataflows.cache.mongodb_cache_adapter import get_mongodb_cache_adapter
        adapter = get_mongodb_cache_adapter()
        # 计算日期范围
        end_date = now.strftime("%Y-%m-%d")
        start_date = (now - timedelta(days=limit * 2)).strftime("%Y-%m-%d")
        logger.info(f"🔍 尝试从 MongoDB 获取 K 线数据: {code_padded}, period={period} (MongoDB: {mongodb_period}), limit={limit}")
        df = adapter.get_historical_data(code_padded, start_date, end_date, period=mongodb_period)
        if df is not None and not df.empty:
            # 转换 DataFrame 为列表格式
            items = []
            for _, row in df.tail(limit).iterrows():
                items.append({
                    "time": row.get("trade_date", row.get("date", "")),  # 前端期望 time 字段
                    "open": float(row.get("open", 0)),
                    "high": float(row.get("high", 0)),
                    "low": float(row.get("low", 0)),
                    "close": float(row.get("close", 0)),
                    "volume": float(row.get("volume", row.get("vol", 0))),
                    "amount": float(row.get("amount", 0)) if "amount" in row else None,
                })
            source = "mongodb"
            logger.info(f"✅ 从 MongoDB 获取到 {len(items)} 条 K 线数据")
    except Exception as e:
        logger.warning(f"⚠️ MongoDB 获取 K 线失败: {e}")
    # 2. 如果 MongoDB 没有数据，降级到外部 API（带超时保护）
    if not items:
        logger.info(f"📡 MongoDB 无数据，降级到外部 API")
        try:
            import asyncio
            from app.services.data_sources.manager import DataSourceManager
            mgr = DataSourceManager()
            # 添加 10 秒超时保护
            items, source = await asyncio.wait_for(
                asyncio.to_thread(mgr.get_kline_with_fallback, code_padded, period, limit, adj_norm),
                timeout=10.0
            )
```

```
        except asyncio.TimeoutError:
            logger.error(f"❌ 外部 API 获取 K 线超时（10秒）")
            raise HTTPException(status_code=504, detail="获取K线数据超时，请稍后重试")
        except Exception as e:
            logger.error(f"❌ 外部 API 获取 K 线失败: {e}")
            raise HTTPException(status_code=500, detail=f"获取K线数据失败: {str(e)}")
    # 🔥 3. 检查是否需要添加当天实时数据（仅针对日线）
    if period == "day" and items:
        try:
            # 检查历史数据中是否已有当天的数据（支持两种日期格式）
            has_today_data = any(
                item.get("time") in [today_str_yyyymmdd, today_str_formatted]
                for item in items
            )
            # 判断是否在交易时间内或收盘后缓冲期
            current_time = now.time()
            is_weekday = now.weekday() < 5  # 周一到周五
            # 交易时间：9:30-11:30, 13:00-15:00
            # 收盘后缓冲期：15:00-15:30（确保获取到收盘价）
            is_trading_time = (
                is_weekday and (
                    (dtime(9, 30) <= current_time <= dtime(11, 30)) or
                    (dtime(13, 0) <= current_time <= dtime(15, 30))
                )
            )
            # 🔥 只在交易时间或收盘后缓冲期内才添加实时数据
            # 非交易日（周末、节假日）不添加实时数据
            should_fetch_realtime = is_trading_time
            if should_fetch_realtime:
                logger.info(f"🔥 尝试从 market_quotes 获取当天实时数据: {code_padded} (交易时间: {is_trading_time}, 已有当天数据: {has_today_data})")
                db = get_mongo_db()
                market_quotes_coll = db["market_quotes"]
                # 查询当天的实时行情
                realtime_quote = await market_quotes_coll.find_one({"code": code_padded})
                if realtime_quote:
                    # 🔥 构造当天的K线数据（使用统一的日期格式 YYYY-MM-DD）
                    today_kline = {
                        "time": today_str_formatted,  # 🔥 使用 YYYY-MM-DD 格式，与历史数据保持一致
                        "open": float(realtime_quote.get("open", 0)),
                        "high": float(realtime_quote.get("high", 0)),
                        "low": float(realtime_quote.get("low", 0)),
                        "close": float(realtime_quote.get("close", 0)),
                        "volume": float(realtime_quote.get("volume", 0)),
                        "amount": float(realtime_quote.get("amount", 0)),
                    }
                    # 如果历史数据中已有当天数据，替换；否则追加
                    if has_today_data:
                        # 替换最后一条数据（假设最后一条是当天的）
                        items[-1] = today_kline
                        logger.info(f"✅ 替换当天K线数据: {code_padded}")
                    else:
                        # 追加到末尾
                        items.append(today_kline)
                        logger.info(f"✅ 追加当天K线数据: {code_padded}")
                    source = f"{source}+market_quotes"
                else:
                    logger.warning(f"⚠️ market_quotes 中未找到当天数据: {code_padded}")
        except Exception as e:
            logger.warning(f"⚠️ 获取当天实时数据失败（忽略）: {e}")
    data = {
        "code": code_padded,
        "period": period,
        "limit": limit,
        "adj": adj if adj else "none",
        "source": source,
        "items": items or []
    }
    return ok(data)

@router.get("/{code}/news", response_model=dict)
async def get_news(code: str, days: int = 30, limit: int = 50, include_announcements: bool = True, current_user: dict = Depends(get_current_user)):
    """获取新闻与公告（支持A股、港股、美股）"""
    from app.services.foreign_stock_service import ForeignStockService
    from app.services.news_data_service import get_news_data_service, NewsQueryParams
    # 检测股票类型
    market, normalized_code = _detect_market_and_code(code)
    if market == 'US':
        # 美股：使用 ForeignStockService
        service = ForeignStockService()
        result = await service.get_us_news(normalized_code, days=days, limit=limit)
        return ok(result)
    elif market == 'HK':
        # 港股：暂时返回空数据（TODO: 实现港股新闻）
        data = {
            "code": normalized_code,
            "days": days,
            "limit": limit,
            "source": "none",
            "items": []
        }
        return ok(data)
    else:
        # A股：直接调用同步服务的查询方法（包含智能回退逻辑）
        try:
            logger.info(f"=" * 80)
            logger.info(f"📰 开始获取新闻: code={code}, normalized_code={normalized_code}, days={days}, limit={limit}")
            # 直接使用 news_data 路由的查询逻辑
            from app.services.news_data_service import get_news_data_service, NewsQueryParams
            from datetime import datetime, timedelta
            from app.worker.akshare_sync_service import get_akshare_sync_service
            service = await get_news_data_service()
            sync_service = await get_akshare_sync_service()
            # 计算时间范围
            hours_back = days * 24
```

```
        # 🔥 不设置 start_time 限制，直接查询最新的 N 条新闻
        # 因为数据库中的新闻可能不是最近几天的，而是历史数据
        params = NewsQueryParams(
            symbol=normalized_code,
            limit=limit,
            sort_by="publish_time",
            sort_order=-1
        )
        logger.info(f"🔍 查询参数: symbol={params.symbol}, limit={params.limit} (不限制时间范围)")
        # 1. 先从数据库查询
        logger.info(f"📊 步骤1: 从数据库查询新闻...")
        news_list = await service.query_news(params)
        logger.info(f"📊 数据库查询结果: 返回 {len(news_list)} 条新闻")
        data_source = "database"
        # 2. 如果数据库没有数据，调用同步服务
        if not news_list:
            logger.info(f"⚠️ 数据库无新闻数据，调用同步服务获取: {normalized_code}")
            try:
                # 🔥 调用同步服务，传入单个股票代码列表
                logger.info(f"🔄 步骤2: 调用同步服务...")
                await sync_service.sync_news_data(
                    symbols=[normalized_code],
                    max_news_per_stock=limit,
                    force_update=False,
                    favorites_only=False
                )
                # 重新查询
                logger.info(f"🔄 步骤3: 重新从数据库查询...")
                news_list = await service.query_news(params)
                logger.info(f"📊 重新查询结果: 返回 {len(news_list)} 条新闻")
                data_source = "realtime"
            except Exception as e:
                logger.error(f"❌ 同步服务异常: {e}", exc_info=True)
        # 转换为旧格式（兼容前端）
        logger.info(f"🔄 步骤4: 转换数据格式...")
        items = []
        for news in news_list:
            # 🔥 将 datetime 对象转换为 ISO 字符串
            publish_time = news.get("publish_time", "")
            if isinstance(publish_time, datetime):
                publish_time = publish_time.isoformat()
            items.append({
                "title": news.get("title", ""),
                "source": news.get("source", ""),
                "time": publish_time,
                "url": news.get("url", ""),
                "type": "news",
                "content": news.get("content", ""),
                "summary": news.get("summary", "")
            })
        logger.info(f"✅ 转换完成: {len(items)} 条新闻")
        data = {
            "code": normalized_code,
            "days": days,
            "limit": limit,
            "include_announcements": include_announcements,
            "source": data_source,
            "items": items
        }
        logger.info(f"📤 最终返回: source={data_source}, items_count={len(items)}")
        logger.info(f"=" * 80)
        return ok(data)
    except Exception as e:
        logger.error(f"❌ 获取新闻失败: {e}", exc_info=True)
        data = {
            "code": normalized_code,
            "days": days,
            "limit": limit,
            "include_announcements": include_announcements,
            "source": None,
            "items": []
        }
        return ok(data)
```

FILE: app/routers/sync.py

```
"""
Sync router for stock basics synchronization
- POST /api/sync/stock_basics/run -> trigger full sync
- GET  /api/sync/stock_basics/status -> get last status
Requires MongoDB initialized by app lifespan.
"""
from __future__ import annotations
from fastapi import APIRouter, HTTPException
from app.services.basics_sync_service import get_basics_sync_service
router = APIRouter(prefix="/api/sync", tags=["sync"])
@router.post("/stock_basics/run")
async def run_stock_basics_sync(force: bool = False):
    try:
        service = get_basics_sync_service()
        result = await service.run_full_sync(force=force)
        return {"success": True, "data": result}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
@router.get("/stock_basics/status")
async def get_stock_basics_status():
    service = get_basics_sync_service()
    status = await service.get_status()
    return {"success": True, "data": status}
```

FILE: app/routers/system_config.py

```
from fastapi import APIRouter, Depends, HTTPException, status
from typing import Any, Dict
import re
import logging
from app.core.config import settings
from app.routers.auth_db import get_current_user
router = APIRouter()
logger = logging.getLogger("webapi")
SENSITIVE_KEYS = {
    "MONGODB_PASSWORD",
    "REDIS_PASSWORD",
    "JWT_SECRET",
    "CSRF_SECRET",
    "STOCK_DATA_API_KEY",
    "REFRESH_TOKEN_EXPIRE_DAYS",  # not sensitive itself, but keep for completeness
}
MASK = "***"
def _mask_value(key: str, value: Any) -> Any:
    if value is None:
        return None
    if key in SENSITIVE_KEYS:
        return MASK
    # Mask URLs that may contain credentials
    if key in {"MONGO_URI", "REDIS_URL"} and isinstance(value, str):
        v = value
        # mongodb://user:pass@host:port/db?...
        v = re.sub(r"(mongodb://[^:/?#]+):([^@/]+)@", r"\1:***@", v)
        # redis://:pass@host:port/db
        v = re.sub(r"(redis://:)[^@/]+@", r"\1***@", v)
        return v
    return value
def _build_summary() -> Dict[str, Any]:
    raw = settings.model_dump()
    # Attach derived URLs
    raw["MONGO_URI"] = settings.MONGO_URI
    raw["REDIS_URL"] = settings.REDIS_URL
    summary: Dict[str, Any] = {}
    for k, v in raw.items():
        summary[k] = _mask_value(k, v)
    return summary
@router.get("/config/summary", tags=["system"], summary="配置概要（已屏蔽敏感项，需管理员）")
async def get_config_summary(current_user: dict = Depends(get_current_user)) -> Dict[str, Any]:
    """
    返回当前生效的设置概要。敏感字段将以 *** 掩码显示。
    访问控制：需管理员身份。
    """
    if not current_user.get("is_admin", False):
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Admin privileges required")
    return {"settings": _build_summary()}
@router.get("/config/validate", tags=["system"], summary="验证配置完整性")
async def validate_config():
    """
    验证系统配置的完整性和有效性。
    返回验证结果，包括缺少的配置项和无效的配置。
    验证内容：
    1. 环境变量配置（.env 文件）
    2. MongoDB 中存储的配置（大模型、数据源等）
    注意：此接口会先从 MongoDB 重载配置到环境变量，然后再验证。
    """
    from app.core.startup_validator import StartupValidator
    from app.core.config_bridge import bridge_config_to_env
    from app.services.config_service import config_service
    try:
        # 🔧 步骤1: 重载配置 - 从 MongoDB 读取配置并桥接到环境变量
        try:
            bridge_config_to_env()
            logger.info("✅ 配置已从 MongoDB 重载到环境变量")
        except Exception as e:
            logger.warning(f"⚠️ 配置重载失败: {e}，将验证 .env 文件中的配置")
        # 🔍 步骤2: 验证环境变量配置
        validator = StartupValidator()
        env_result = validator.validate()
        # 🔍 步骤3: 验证 MongoDB 中的配置（厂家级别）
        mongodb_validation = {
            "llm_providers": [],
            "data_source_configs": [],
            "warnings": []
        }
        try:
            from app.utils.api_key_utils import (
                is_valid_api_key,
                get_env_api_key_for_provider
            )
            # 🔥 修改：直接从数据库读取原始数据，避免使用 get_llm_providers() 返回的已修改数据
            # get_llm_providers() 会将环境变量的 Key 赋值给 provider.api_key，导致无法区分来源
            from pymongo import MongoClient
            from app.core.config import settings
            from app.models.config import LLMProvider
            # 创建同步 MongoDB 客户端
            client = MongoClient(settings.MONGO_URI)
            db = client[settings.MONGO_DB]
            providers_collection = db.llm_providers
            # 查询所有厂家配置（原始数据）
            providers_data = list(providers_collection.find())
            llm_providers = [LLMProvider(**data) for data in providers_data]
            # 关闭同步客户端
            client.close()
            logger.info(f"🔍 获取到 {len(llm_providers)} 个大模型厂家")
            for provider in llm_providers:
                # 只验证已启用的厂家
```

```
        if not provider.is_active:
            continue
        validation_item = {
            "name": provider.name,
            "display_name": provider.display_name,
            "is_active": provider.is_active,
            "has_api_key": False,
            "status": "未配置",
            "source": None,  # 标识配置来源（database/environment）
            "mongodb_configured": False,  # MongoDB 是否配置
            "env_configured": False  # 环境变量是否配置
        }
        # 🔥 关键：检查数据库中的原始 API Key 是否有效
        db_key_valid = is_valid_api_key(provider.api_key)
        validation_item["mongodb_configured"] = db_key_valid
        # 检查环境变量中的 API Key 是否有效
        env_key = get_env_api_key_for_provider(provider.name)
        env_key_valid = env_key is not None
        validation_item["env_configured"] = env_key_valid
        if db_key_valid:
            # MongoDB 中有有效的 API Key（优先级最高）
            validation_item["has_api_key"] = True
            validation_item["status"] = "已配置"
            validation_item["source"] = "database"
        elif env_key_valid:
            # MongoDB 中没有，但环境变量中有有效的 API Key
            validation_item["has_api_key"] = True
            validation_item["status"] = "已配置（环境变量）"
            validation_item["source"] = "environment"
            # 用黄色警告提示用户可以在数据库中配置
            mongodb_validation["warnings"].append(
                f"大模型厂家 {provider.display_name} 使用环境变量配置，建议在数据库中配置以便统一管理"
            )
        else:
            # MongoDB 和环境变量都没有有效的 API Key
            validation_item["status"] = "未配置"
            mongodb_validation["warnings"].append(
                f"大模型厂家 {provider.display_name} 已启用但未配置有效的 API Key（数据库和环境变量中都未找到）"
            )
        mongodb_validation["llm_providers"].append(validation_item)
    # 验证数据源配置
    from app.utils.api_key_utils import (
        is_valid_api_key,
        get_env_api_key_for_datasource
    )
    system_config = await config_service.get_system_config()
    if system_config and system_config.data_source_configs:
        logger.info(f"🔍 获取到 {len(system_config.data_source_configs)} 个数据源配置")
        for ds_config in system_config.data_source_configs:
            # 只验证已启用的数据源
            if not ds_config.enabled:
                continue
            validation_item = {
                "name": ds_config.name,
                "type": ds_config.type,
                "enabled": ds_config.enabled,
                "has_api_key": False,
                "status": "未配置",
                "source": None,  # 标识配置来源（database/environment/builtin）
                "mongodb_configured": False,  # 新增：MongoDB 是否配置
                "env_configured": False  # 新增：环境变量是否配置
            }
            # 某些数据源不需要 API Key（如 AKShare）
            if ds_config.type in ["akshare", "yahoo"]:
                validation_item["has_api_key"] = True
                validation_item["status"] = "已配置（无需密钥）"
                validation_item["source"] = "builtin"
                validation_item["mongodb_configured"] = True
                validation_item["env_configured"] = True
            else:
                # 检查数据库中的 API Key 是否有效
                db_key_valid = is_valid_api_key(ds_config.api_key)
                validation_item["mongodb_configured"] = db_key_valid
                # 检查环境变量中的 API Key 是否有效
                ds_type = ds_config.type.value if hasattr(ds_config.type, 'value') else ds_config.type
                env_key = get_env_api_key_for_datasource(ds_type)
                env_key_valid = env_key is not None
                validation_item["env_configured"] = env_key_valid
                if db_key_valid:
                    # MongoDB 中有有效的 API Key（优先级最高）
                    validation_item["has_api_key"] = True
                    validation_item["status"] = "已配置"
                    validation_item["source"] = "database"
                elif env_key_valid:
                    # MongoDB 中没有，但环境变量中有有效的 API Key
                    validation_item["has_api_key"] = True
                    validation_item["status"] = "已配置（环境变量）"
                    validation_item["source"] = "environment"
                    # 用黄色警告提示用户可以在数据库中配置
                    mongodb_validation["warnings"].append(
                        f"数据源 {ds_config.name} 使用环境变量配置，建议在数据库中配置以便统一管理"
                    )
                else:
                    # MongoDB 和环境变量都没有有效的 API Key
                    validation_item["status"] = "未配置"
                    mongodb_validation["warnings"].append(
                        f"数据源 {ds_config.name} 已启用但未配置有效的 API Key（数据库和环境变量中都未找到）"
                    )
            mongodb_validation["data_source_configs"].append(validation_item)
except Exception as e:
    logger.error(f"验证 MongoDB 配置失败: {e}", exc_info=True)
    mongodb_validation["warnings"].append(f"MongoDB 配置验证失败: {str(e)}")
# 合并验证结果
```

```
        logger.info(f"🔍 MongoDB 验证结果: {len(mongodb_validation['llm_providers'])} 个大模型厂家, {len(mongodb_validation['data_source_configs'])} 个数据源, {len(mongodb_validation['warnings'])} 个警告")
        # 🔥 修改：只有必需配置有问题时才认为验证失败
        # MongoDB 配置警告（推荐配置）不影响总体验证结果
        # 只有环境变量中的必需配置缺失或无效时才显示红色错误
        overall_success = env_result.success
        return {
            "success": True,
            "data": {
                # 环境变量验证结果
                "env_validation": {
                    "success": env_result.success,
                    "missing_required": [
                        {"key": config.key, "description": config.description}
                        for config in env_result.missing_required
                    ],
                    "missing_recommended": [
                        {"key": config.key, "description": config.description}
                        for config in env_result.missing_recommended
                    ],
                    "invalid_configs": [
                        {"key": config.key, "error": config.description}
                        for config in env_result.invalid_configs
                    ],
                    "warnings": env_result.warnings
                },
                # MongoDB 配置验证结果
                "mongodb_validation": mongodb_validation,
                # 总体验证结果（只考虑必需配置）
                "success": overall_success
            },
            "message": "配置验证完成"
        }
    except Exception as e:
        logger.error(f"配置验证失败: {e}", exc_info=True)
        return {
            "success": False,
            "data": None,
            "message": f"配置验证失败: {str(e)}"
        }
```

FILE: app/routers/tags.py

```
"""
标签管理 API
"""
from typing import Optional, List
from fastapi import APIRouter, Depends, HTTPException, status
from pydantic import BaseModel, Field
from app.routers.auth_db import get_current_user
from app.core.response import ok
from app.services.tags_service import tags_service
router = APIRouter(prefix="/tags", tags=["标签管理"])
class TagCreate(BaseModel):
    name: str = Field(..., min_length=1, max_length=30)
    color: Optional[str] = Field(default="#409EFF", max_length=20)
    sort_order: int = 0
class TagUpdate(BaseModel):
    name: Optional[str] = Field(default=None, min_length=1, max_length=30)
    color: Optional[str] = Field(default=None, max_length=20)
    sort_order: Optional[int] = None
class TagResponse(BaseModel):
    id: str
    name: str
    color: str
    sort_order: int
    created_at: str
    updated_at: str
@router.get("/", response_model=dict)
async def list_tags(current_user: dict = Depends(get_current_user)):
    try:
        tags = await tags_service.list_tags(current_user["id"])
        return ok(tags)
    except Exception as e:
        raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail=f"获取标签失败: {e}")
@router.post("/", response_model=dict)
async def create_tag(payload: TagCreate, current_user: dict = Depends(get_current_user)):
    try:
        tag = await tags_service.create_tag(
            user_id=current_user["id"],
            name=payload.name,
            color=payload.color,
            sort_order=payload.sort_order,
        )
        return ok(tag, "创建成功")
    except Exception as e:
        # 可能违反唯一索引（同名），返回400
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=f"创建标签失败: {e}")
@router.put("/{tag_id}", response_model=dict)
async def update_tag(tag_id: str, payload: TagUpdate, current_user: dict = Depends(get_current_user)):
    try:
        success = await tags_service.update_tag(
            user_id=current_user["id"],
            tag_id=tag_id,
            name=payload.name,
            color=payload.color,
            sort_order=payload.sort_order,
        )
        if not success:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="标签不存在")
        return ok({"id": tag_id}, "更新成功")
    except HTTPException:
        raise
```

```
    except Exception as e:
        raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail=f"更新标签失败: {e}")

@router.delete("/{tag_id}", response_model=dict)
async def delete_tag(tag_id: str, current_user: dict = Depends(get_current_user)):
    try:
        success = await tags_service.delete_tag(current_user["id"], tag_id)
        if not success:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="标签不存在")
        return ok({"id": tag_id}, "删除成功")
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail=f"删除标签失败: {e}")
```

FILE: app/routers/tushare_init.py

```
"""
Tushare数据初始化API路由
提供Web界面的数据初始化功能
"""
import asyncio
from datetime import datetime
from typing import Dict, Any, Optional
from fastapi import APIRouter, HTTPException, BackgroundTasks, Depends
from pydantic import BaseModel, Field
from app.routers.auth_db import get_current_user
from app.core.database import get_mongo_db
from app.worker.tushare_init_service import get_tushare_init_service
from app.core.response import ok

router = APIRouter(prefix="/api/tushare-init", tags=["Tushare初始化"])

class InitializationRequest(BaseModel):
    """初始化请求模型"""
    historical_days: int = Field(default=365, ge=1, le=3650, description="历史数据天数")
    skip_if_exists: bool = Field(default=True, description="如果数据已存在是否跳过")
    force_update: bool = Field(default=False, description="强制更新已有数据")

class DatabaseStatusResponse(BaseModel):
    """数据库状态响应模型"""
    basic_info_count: int = Field(description="基础信息数量")
    quotes_count: int = Field(description="行情数据数量")
    extended_coverage: float = Field(description="扩展字段覆盖率")
    latest_basic_update: Optional[datetime] = Field(description="基础信息最新更新时间")
    latest_quotes_update: Optional[datetime] = Field(description="行情数据最新更新时间")
    needs_initialization: bool = Field(description="是否需要初始化")

class InitializationStatusResponse(BaseModel):
    """初始化状态响应模型"""
    is_running: bool = Field(description="是否正在运行")
    current_step: Optional[str] = Field(description="当前步骤")
    progress: Optional[str] = Field(description="进度")
    started_at: Optional[datetime] = Field(description="开始时间")
    estimated_completion: Optional[datetime] = Field(description="预计完成时间")

# 全局初始化状态跟踪
_initialization_status = {
    "is_running": False,
    "current_step": None,
    "progress": None,
    "started_at": None,
    "task": None
}

@router.get("/status", response_model=dict)
async def get_database_status(
    current_user: dict = Depends(get_current_user)
):
    """
    获取数据库状态
    检查当前数据库中的数据情况，判断是否需要初始化
    """
    try:
        db = get_mongo_db()

        # 检查各集合状态
        basic_count = await db.stock_basic_info.count_documents({})
        quotes_count = await db.market_quotes.count_documents({})

        # 检查扩展字段覆盖率
        extended_count = 0
        extended_coverage = 0.0
        if basic_count > 0:
            extended_count = await db.stock_basic_info.count_documents({
                "full_symbol": {"$exists": True},
                "market_info": {"$exists": True}
            })
            extended_coverage = extended_count / basic_count

        # 检查最新更新时间
        latest_basic = await db.stock_basic_info.find_one(
            {}, sort=[("updated_at", -1)]
        )
        latest_quotes = await db.market_quotes.find_one(
            {}, sort=[("updated_at", -1)]
        )

        # 判断是否需要初始化
        needs_initialization = (
            basic_count == 0 or
            extended_coverage < 0.5
        )

        status = DatabaseStatusResponse(
            basic_info_count=basic_count,
            quotes_count=quotes_count,
            extended_coverage=extended_coverage,
            latest_basic_update=latest_basic.get("updated_at") if latest_basic else None,
```

```
            latest_quotes_update=latest_quotes.get("updated_at") if latest_quotes else None,
            needs_initialization=needs_initialization
        )

        return ok(data=status,
            message="数据库状态获取成功"
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取数据库状态失败: {str(e)}")
@router.get("/initialization-status", response_model=dict)
async def get_initialization_status(
    current_user: dict = Depends(get_current_user)
):
    """
    获取初始化状态
    检查当前是否有初始化任务在运行
    """
    try:
        status = InitializationStatusResponse(
            is_running=_initialization_status["is_running"],
            current_step=_initialization_status["current_step"],
            progress=_initialization_status["progress"],
            started_at=_initialization_status["started_at"],
            estimated_completion=None  # TODO: 可以根据历史数据估算
        )

        return ok(data=status,
            message="初始化状态获取成功"
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"获取初始化状态失败: {str(e)}")
@router.post("/start-basic", response_model=dict)
async def start_basic_initialization(
    background_tasks: BackgroundTasks,
    current_user: dict = Depends(get_current_user)
):
    """
    启动基础信息初始化
    仅同步股票基础信息，适合快速初始化
    """
    if _initialization_status["is_running"]:
        raise HTTPException(status_code=400, detail="初始化任务已在运行中")

    try:
        # 启动后台任务
        background_tasks.add_task(_run_basic_initialization)

        return ok(data={"message": "基础信息初始化已启动"},
            message="基础信息初始化任务已在后台启动"
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"启动基础信息初始化失败: {str(e)}")
@router.post("/start-full", response_model=dict)
async def start_full_initialization(
    request: InitializationRequest,
    background_tasks: BackgroundTasks,
    current_user: dict = Depends(get_current_user)
):
    """
    启动完整数据初始化
    包括基础信息、历史数据、财务数据、行情数据的完整同步
    """
    if _initialization_status["is_running"]:
        raise HTTPException(status_code=400, detail="初始化任务已在运行中")

    try:
        # 启动后台任务
        background_tasks.add_task(
            _run_full_initialization,
            request.historical_days,
            not request.skip_if_exists or request.force_update
        )

        return ok(data={
                "message": "完整数据初始化已启动",
                "historical_days": request.historical_days,
                "force_update": not request.skip_if_exists or request.force_update
            },
            message="完整数据初始化任务已在后台启动"
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"启动完整数据初始化失败: {str(e)}")
@router.post("/stop", response_model=dict)
async def stop_initialization(
    current_user: dict = Depends(get_current_user)
):
    """
    停止初始化任务
    尝试取消正在运行的初始化任务
    """
    if not _initialization_status["is_running"]:
        raise HTTPException(status_code=400, detail="没有正在运行的初始化任务")

    try:
        # 尝试取消任务
        if _initialization_status["task"]:
            _initialization_status["task"].cancel()

        # 重置状态
        _initialization_status.update({
            "is_running": False,
```

```
        "current_step": None,
        "progress": None,
        "started_at": None,
        "task": None
    })

    return ok(data={"message": "初始化任务已停止"},
        message="初始化任务停止成功"
    )

  except Exception as e:
    raise HTTPException(status_code=500, detail=f"停止初始化任务失败: {str(e)}")
async def _run_basic_initialization():
  """运行基础信息初始化（后台任务）"""
  _initialization_status.update({
    "is_running": True,
    "current_step": "基础信息初始化",
    "progress": "0/1",
    "started_at": datetime.utcnow()
  })

  try:
    service = await get_tushare_init_service()
    result = await service.sync_service.sync_stock_basic_info(force_update=True)

    _initialization_status.update({
      "is_running": False,
      "current_step": "完成",
      "progress": "1/1"
    })

  except Exception as e:
    _initialization_status.update({
      "is_running": False,
      "current_step": f"失败: {str(e)}",
      "progress": "错误"
    })
async def _run_full_initialization(historical_days: int, force_update: bool):
  """运行完整数据初始化（后台任务）"""
  _initialization_status.update({
    "is_running": True,
    "current_step": "准备初始化",
    "progress": "0/6",
    "started_at": datetime.utcnow()
  })

  try:
    service = await get_tushare_init_service()

    # 创建一个任务来跟踪进度
    async def progress_tracker():
      while _initialization_status["is_running"]:
        if hasattr(service, 'stats') and service.stats:
          _initialization_status.update({
            "current_step": service.stats.current_step,
            "progress": f"{service.stats.completed_steps}/{service.stats.total_steps}"
          })
        await asyncio.sleep(1)

    # 启动进度跟踪
    tracker_task = asyncio.create_task(progress_tracker())
    _initialization_status["task"] = tracker_task

    # 运行初始化
    result = await service.run_full_initialization(
      historical_days=historical_days,
      skip_if_exists=not force_update
    )

    # 停止进度跟踪
    tracker_task.cancel()

    _initialization_status.update({
      "is_running": False,
      "current_step": "完成" if result["success"] else "部分完成",
      "progress": result["progress"],
      "task": None
    })

  except Exception as e:
    _initialization_status.update({
      "is_running": False,
      "current_step": f"失败: {str(e)}",
      "progress": "错误",
      "task": None
    })
```

FILE: app/routers/usage_statistics.py

```
"""
使用统计 API 路由
"""
import logging
from datetime import datetime
from typing import Optional, List, Dict, Any
from fastapi import APIRouter, Depends, Query, HTTPException
from app.routers.auth_db import get_current_user
from app.models.config import UsageRecord, UsageStatistics
from app.services.usage_statistics_service import usage_statistics_service
logger = logging.getLogger("app.routers.usage_statistics")
router = APIRouter(prefix="/api/usage", tags=["使用统计"])
@router.get("/records", summary="获取使用记录")
async def get_usage_records(
```

```
    provider: Optional[str] = Query(None, description="供应商"),
    model_name: Optional[str] = Query(None, description="模型名称"),
    start_date: Optional[str] = Query(None, description="开始日期(ISO格式)"),
    end_date: Optional[str] = Query(None, description="结束日期(ISO格式)"),
    limit: int = Query(100, ge=1, le=1000, description="返回记录数"),
    current_user: dict = Depends(get_current_user)
) -> Dict[str, Any]:
    """获取使用记录"""
    try:
        # 解析日期
        start_dt = datetime.fromisoformat(start_date) if start_date else None
        end_dt = datetime.fromisoformat(end_date) if end_date else None

        # 获取记录
        records = await usage_statistics_service.get_usage_records(
            provider=provider,
            model_name=model_name,
            start_date=start_dt,
            end_date=end_dt,
            limit=limit
        )

        return {
            "success": True,
            "message": "获取使用记录成功",
            "data": {
                "records": [record.model_dump() for record in records],
                "total": len(records)
            }
        }
    except Exception as e:
        logger.error(f"获取使用记录失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/statistics", summary="获取使用统计")
async def get_usage_statistics(
    days: int = Query(7, ge=1, le=365, description="统计天数"),
    provider: Optional[str] = Query(None, description="供应商"),
    model_name: Optional[str] = Query(None, description="模型名称"),
    current_user: dict = Depends(get_current_user)
) -> Dict[str, Any]:
    """获取使用统计"""
    try:
        stats = await usage_statistics_service.get_usage_statistics(
            days=days,
            provider=provider,
            model_name=model_name
        )

        return {
            "success": True,
            "message": "获取使用统计成功",
            "data": stats.model_dump()
        }
    except Exception as e:
        logger.error(f"获取使用统计失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/cost/by-provider", summary="按供应商统计成本")
async def get_cost_by_provider(
    days: int = Query(7, ge=1, le=365, description="统计天数"),
    current_user: dict = Depends(get_current_user)
) -> Dict[str, Any]:
    """按供应商统计成本"""
    try:
        cost_data = await usage_statistics_service.get_cost_by_provider(days=days)

        return {
            "success": True,
            "message": "获取成本统计成功",
            "data": cost_data
        }
    except Exception as e:
        logger.error(f"获取成本统计失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/cost/by-model", summary="按模型统计成本")
async def get_cost_by_model(
    days: int = Query(7, ge=1, le=365, description="统计天数"),
    current_user: dict = Depends(get_current_user)
) -> Dict[str, Any]:
    """按模型统计成本"""
    try:
        cost_data = await usage_statistics_service.get_cost_by_model(days=days)

        return {
            "success": True,
            "message": "获取成本统计成功",
            "data": cost_data
        }
    except Exception as e:
        logger.error(f"获取成本统计失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/cost/daily", summary="每日成本统计")
async def get_daily_cost(
    days: int = Query(7, ge=1, le=365, description="统计天数"),
    current_user: dict = Depends(get_current_user)
) -> Dict[str, Any]:
    """每日成本统计"""
    try:
        cost_data = await usage_statistics_service.get_daily_cost(days=days)

        return {
            "success": True,
            "message": "获取每日成本成功",
            "data": cost_data
        }
    except Exception as e:
        logger.error(f"获取每日成本失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@router.delete("/records/old", summary="删除旧记录")
async def delete_old_records(
```

```
    days: int = Query(90, ge=30, le=365, description="保留天数"),
    current_user: dict = Depends(get_current_user)
) -> Dict[str, Any]:
    """删除旧记录"""
    try:
        deleted_count = await usage_statistics_service.delete_old_records(days=days)
        return {
            "success": True,
            "message": f"删除旧记录成功",
            "data": {"deleted_count": deleted_count}
        }
    except Exception as e:
        logger.error(f"删除旧记录失败: {e}")
        raise HTTPException(status_code=500, detail=str(e))
```

FILE: app/routers/websocket_notifications.py

```
"""
WebSocket 通知系统
替代 SSE + Redis PubSub，解决连接泄漏问题
"""
import asyncio
import json
import logging
from typing import Dict, Set
from fastapi import APIRouter, WebSocket, WebSocketDisconnect, Query, HTTPException
from datetime import datetime
from app.services.auth_service import AuthService
router = APIRouter()
logger = logging.getLogger("webapi.websocket")
# 🔥 全局 WebSocket 连接管理器
class ConnectionManager:
    """WebSocket 连接管理器"""

    def __init__(self):
        # user_id -> Set[WebSocket]
        self.active_connections: Dict[str, Set[WebSocket]] = {}
        self._lock = asyncio.Lock()

    async def connect(self, websocket: WebSocket, user_id: str):
        """连接 WebSocket"""
        await websocket.accept()

        async with self._lock:
            if user_id not in self.active_connections:
                self.active_connections[user_id] = set()
            self.active_connections[user_id].add(websocket)

            total_connections = sum(len(conns) for conns in self.active_connections.values())
            logger.info(f"✅ [WS] 新连接: user={user_id}, "
                        f"该用户连接数={len(self.active_connections[user_id])}, "
                        f"总连接数={total_connections}")

    async def disconnect(self, websocket: WebSocket, user_id: str):
        """断开 WebSocket"""
        async with self._lock:
            if user_id in self.active_connections:
                self.active_connections[user_id].discard(websocket)
                if not self.active_connections[user_id]:
                    del self.active_connections[user_id]

            total_connections = sum(len(conns) for conns in self.active_connections.values())
            logger.info(f"💡 [WS] 断开连接: user={user_id}, 总连接数={total_connections}")

    async def send_personal_message(self, message: dict, user_id: str):
        """发送消息给指定用户的所有连接"""
        async with self._lock:
            if user_id not in self.active_connections:
                logger.debug(f"⚠️ [WS] 用户 {user_id} 没有活跃连接")
                return

            connections = list(self.active_connections[user_id])

        # 在锁外发送消息，避免阻塞
        message_json = json.dumps(message, ensure_ascii=False)
        dead_connections = []

        for connection in connections:
            try:
                await connection.send_text(message_json)
                logger.debug(f"📤 [WS] 发送消息给 user={user_id}")
            except Exception as e:
                logger.warning(f"❌ [WS] 发送消息失败: {e}")
                dead_connections.append(connection)

        # 清理死连接
        if dead_connections:
            async with self._lock:
                if user_id in self.active_connections:
                    for conn in dead_connections:
                        self.active_connections[user_id].discard(conn)
                    if not self.active_connections[user_id]:
                        del self.active_connections[user_id]

    async def broadcast(self, message: dict):
        """广播消息给所有连接"""
        async with self._lock:
            all_connections = []
            for connections in self.active_connections.values():
                all_connections.extend(connections)

        message_json = json.dumps(message, ensure_ascii=False)

        for connection in all_connections:
```

```
            try:
                await connection.send_text(message_json)
            except Exception as e:
                logger.warning(f"❌ [WS] 广播消息失败: {e}")

    def get_stats(self) -> dict:
        """获取连接统计"""
        return {
            "total_users": len(self.active_connections),
            "total_connections": sum(len(conns) for conns in self.active_connections.values()),
            "users": {user_id: len(conns) for user_id, conns in self.active_connections.items()}
        }
# 全局连接管理器实例
manager = ConnectionManager()

@router.websocket("/ws/notifications")
async def websocket_notifications_endpoint(
    websocket: WebSocket,
    token: str = Query(...)
):
    """
    WebSocket 通知端点

    客户端连接: ws://localhost:8000/api/ws/notifications?token=<jwt_token>

    消息格式:
    {
        "type": "notification",  // 消息类型: notification, heartbeat, connected
        "data": {
            "id": "...",
            "title": "...",
            "content": "...",
            "type": "analysis",
            "link": "/stocks/000001",
            "source": "analysis",
            "created_at": "2025-10-23T12:00:00",
            "status": "unread"
        }
    }
    """
    # 验证 token
    token_data = AuthService.verify_token(token)
    if not token_data:
        await websocket.close(code=1008, reason="Unauthorized")
        return

    user_id = "admin"  # 从 token_data 中获取

    # 连接 WebSocket
    await manager.connect(websocket, user_id)

    # 发送连接确认
    await websocket.send_json({
        "type": "connected",
        "data": {
            "user_id": user_id,
            "timestamp": datetime.utcnow().isoformat(),
            "message": "WebSocket 连接成功"
        }
    })

    try:
        # 心跳任务
        async def send_heartbeat():
            while True:
                try:
                    await asyncio.sleep(30)  # 每 30 秒发送一次心跳
                    await websocket.send_json({
                        "type": "heartbeat",
                        "data": {
                            "timestamp": datetime.utcnow().isoformat()
                        }
                    })
                except Exception as e:
                    logger.debug(f"💔 [WS] 心跳发送失败: {e}")
                    break

        # 启动心跳任务
        heartbeat_task = asyncio.create_task(send_heartbeat())

        # 接收客户端消息（主要用于保持连接）
        while True:
            try:
                data = await websocket.receive_text()
                # 可以处理客户端发送的消息（如 ping/pong）
                logger.debug(f"📥 [WS] 收到客户端消息: user={user_id}, data={data}")
            except WebSocketDisconnect:
                logger.info(f"🔌 [WS] 客户端主动断开: user={user_id}")
                break
            except Exception as e:
                logger.error(f"❌ [WS] 接收消息错误: {e}")
                break

    finally:
        # 取消心跳任务
        if 'heartbeat_task' in locals():
            heartbeat_task.cancel()
            try:
                await heartbeat_task
            except asyncio.CancelledError:
                pass

        # 断开连接
        await manager.disconnect(websocket, user_id)

@router.websocket("/ws/tasks/{task_id}")
async def websocket_task_progress_endpoint(
```

```
    websocket: WebSocket,
    task_id: str,
    token: str = Query(...)
):
    """
    WebSocket 任务进度端点

    客户端连接: ws://localhost:8000/api/ws/tasks/<task_id>?token=<jwt_token>

    消息格式:
    {
        "type": "progress",  // 消息类型: progress, completed, error, heartbeat
        "data": {
            "task_id": "...",
            "message": "正在分析...",
            "step": 1,
            "total_steps": 5,
            "progress": 20.0,
            "timestamp": "2025-10-23T12:00:00"
        }
    }
    """
    # 验证 token
    token_data = AuthService.verify_token(token)
    if not token_data:
        await websocket.close(code=1008, reason="Unauthorized")
        return

    user_id = "admin"
    channel = f"task_progress:{task_id}"

    # 连接 WebSocket
    await websocket.accept()
    logger.info(f"✅ [WS-Task] 新连接: task={task_id}, user={user_id}")

    # 发送连接确认
    await websocket.send_json({
        "type": "connected",
        "data": {
            "task_id": task_id,
            "timestamp": datetime.utcnow().isoformat(),
            "message": "已连接任务进度流"
        }
    })

    try:
        # 这里可以从 Redis 或数据库获取任务进度
        # 暂时保持连接，等待任务完成
        while True:
            try:
                data = await websocket.receive_text()
                logger.debug(f"📥 [WS-Task] 收到客户端消息: task={task_id}, data={data}")
            except WebSocketDisconnect:
                logger.info(f"💡 [WS-Task] 客户端主动断开: task={task_id}")
                break
            except Exception as e:
                logger.error(f"❌ [WS-Task] 接收消息错误: {e}")
                break

    finally:
        logger.info(f"💡 [WS-Task] 断开连接: task={task_id}")
@router.get("/ws/stats")
async def get_websocket_stats():
    """获取 WebSocket 连接统计"""
    return manager.get_stats()
# 🔥 辅助函数：供其他模块调用，发送通知
async def send_notification_via_websocket(user_id: str, notification: dict):
    """
    通过 WebSocket 发送通知

    Args:
        user_id: 用户 ID
        notification: 通知数据
    """
    message = {
        "type": "notification",
        "data": notification
    }
    await manager.send_personal_message(message, user_id)
async def send_task_progress_via_websocket(task_id: str, progress_data: dict):
    """
    通过 WebSocket 发送任务进度

    Args:
        task_id: 任务 ID
        progress_data: 进度数据
    """
    # 注意：这里需要知道任务属于哪个用户
    # 可以从数据库查询或在 progress_data 中传递
    # 暂时简化处理
    message = {
        "type": "progress",
        "data": progress_data
    }
    # 广播给所有连接（生产环境应该只发给任务所属用户）
    await manager.broadcast(message)
```

FILE: app/schemas/__init__.py

```
"""
Pydantic schemas for API request/response models
"""
```

FILE: app/scripts/init_providers.py

```
#!/usr/bin/env python3
"""
初始化大模型厂家数据脚本
"""
import asyncio
import sys
import os
from datetime import datetime
# 添加项目根目录到Python路径
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..', '..'))
from app.core.database import init_db, get_mongo_db
from app.models.config import LLMProvider
async def init_providers():
    """初始化大模型厂家数据"""
    print("🚀 开始初始化大模型厂家数据...")

    # 初始化数据库连接
    await init_db()
    db = get_mongo_db()
    providers_collection = db.llm_providers

    # 预设厂家数据
    providers_data = [
        {
            "name": "openai",
            "display_name": "OpenAI",
            "description": "OpenAI是人工智能领域的领先公司，提供GPT系列模型",
            "website": "https://openai.com",
            "api_doc_url": "https://platform.openai.com/docs",
            "default_base_url": "https://api.openai.com/v1",
            "is_active": True,
            "supported_features": ["chat", "completion", "embedding", "image", "vision", "function_calling", "streaming"]
        },
        {
            "name": "anthropic",
            "display_name": "Anthropic",
            "description": "Anthropic专注于AI安全研究，提供Claude系列模型",
            "website": "https://anthropic.com",
            "api_doc_url": "https://docs.anthropic.com",
            "default_base_url": "https://api.anthropic.com",
            "is_active": True,
            "supported_features": ["chat", "completion", "function_calling", "streaming"]
        },
        {
            "name": "google",
            "display_name": "Google AI",
            "description": "Google的人工智能平台，提供Gemini系列模型",
            "website": "https://ai.google.dev",
            "api_doc_url": "https://ai.google.dev/docs",
            "default_base_url": "https://generativelanguage.googleapis.com/v1beta",
            "is_active": True,
            "supported_features": ["chat", "completion", "embedding", "vision", "function_calling", "streaming"]
        },
        {
            "name": "zhipu",
            "display_name": "智谱AI",
            "description": "智谱AI提供GLM系列中文大模型",
            "website": "https://zhipuai.cn",
            "api_doc_url": "https://open.bigmodel.cn/doc",
            "default_base_url": "https://open.bigmodel.cn/api/paas/v4",
            "is_active": True,
            "supported_features": ["chat", "completion", "embedding", "function_calling", "streaming"]
        },
        {
            "name": "deepseek",
            "display_name": "DeepSeek",
            "description": "DeepSeek提供高性能的AI推理服务",
            "website": "https://www.deepseek.com",
            "api_doc_url": "https://platform.deepseek.com/api-docs",
            "default_base_url": "https://api.deepseek.com",
            "is_active": True,
            "supported_features": ["chat", "completion", "function_calling", "streaming"]
        },
        {
            "name": "dashscope",
            "display_name": "阿里云百炼",
            "description": "阿里云百炼大模型服务平台，提供通义千问等模型",
            "website": "https://bailian.console.aliyun.com",
            "api_doc_url": "https://help.aliyun.com/zh/dashscope/",
            "default_base_url": "https://dashscope.aliyuncs.com/compatible-mode/v1",
            "is_active": True,
            "supported_features": ["chat", "completion", "embedding", "function_calling", "streaming"]
        },
        {
            "name": "siliconflow",
            "display_name": "硅基流动",
            "description": "硅基流动提供高性价比的AI推理服务，支持多种开源模型",
            "website": "https://siliconflow.cn",
            "api_doc_url": "https://docs.siliconflow.cn",
            "default_base_url": "https://api.siliconflow.cn/v1",
            "is_active": True,
            "supported_features": ["chat", "completion", "embedding", "function_calling", "streaming"]
        },
        {
            "name": "302ai",
            "display_name": "302.AI",
            "description": "302.AI是企业级AI聚合平台，提供多种主流大模型的统一接口",
            "website": "https://302.ai",
            "api_doc_url": "https://doc.302.ai",
            "default_base_url": "https://api.302.ai/v1",
            "is_active": True,
            "supported_features": ["chat", "completion", "embedding", "image", "vision", "function_calling", "streaming"]
```

```
        }
    ]

    # 清除现有数据
    await providers_collection.delete_many({})
    print("🧹 清除现有厂家数据")

    # 插入新数据
    for provider_data in providers_data:
        provider_data["created_at"] = datetime.utcnow()
        provider_data["updated_at"] = datetime.utcnow()

        result = await providers_collection.insert_one(provider_data)
        print(f"✅ 添加厂家: {provider_data['display_name']} (ID: {result.inserted_id})")

    print(f"🎉 成功初始化 {len(providers_data)} 个厂家数据")
if __name__ == "__main__":
    asyncio.run(init_providers())
```

FILE: app/services/__init__.py

```
"""
Service layer for business logic and integrations
"""
```

FILE: app/services/analysis/__init__.py

```
"""Analysis service subpackage.

This package contains utilities split out from the monolithic analysis_service.py
without changing the public API of AnalysisService.
"""
```

FILE: app/services/analysis/status_update_utils.py

```
"""Utilities for updating analysis task status.

Extracted from AnalysisService to reduce file size and improve modularity
without changing external behavior.
"""
from __future__ import annotations
from datetime import datetime
from typing import Optional, Dict, Any
from app.core.database import get_mongo_db
from app.core.redis_client import get_redis_service, RedisKeys
from app.models.analysis import AnalysisStatus, AnalysisResult

async def perform_update_task_status(
    task_id: str,
    status: AnalysisStatus,
    progress: int,
    result: Optional[AnalysisResult] = None,
) -> None:
    """Update a task's status in MongoDB and Redis.
    Mirrors the original logic in AnalysisService._update_task_status.
    """
    db = get_mongo_db()
    redis_service = get_redis_service()
    update_data: Dict[str, Any] = {
        "status": status,
        "progress": progress,
        "updated_at": datetime.utcnow(),
    }
    if status == AnalysisStatus.PROCESSING and "started_at" not in update_data:
        update_data["started_at"] = datetime.utcnow()
    elif status in [AnalysisStatus.COMPLETED, AnalysisStatus.FAILED]:
        update_data["completed_at"] = datetime.utcnow()
        if result:
            update_data["result"] = result.dict()
    await db.analysis_tasks.update_one({"task_id": task_id}, {"$set": update_data})
    progress_key = RedisKeys.TASK_PROGRESS.format(task_id=task_id)
    await redis_service.set_json(
        progress_key,
        {
            "task_id": task_id,
            "status": status,
            "progress": progress,
            "updated_at": datetime.utcnow().isoformat(),
        },
        ttl=3600,
    )
async def perform_update_task_status_with_tracker(
    task_id: str,
    status: AnalysisStatus,
    progress_tracker,  # RedisProgressTracker
    result: Optional[AnalysisResult] = None,
) -> None:
    """Update task status using detailed data from a progress tracker.
    Mirrors the original logic in AnalysisService._update_task_status_with_tracker.
    """
    db = get_mongo_db()
    redis_service = get_redis_service()
    progress_data = progress_tracker.to_dict()
    update_data: Dict[str, Any] = {
        "status": status,
        "progress": progress_data["progress"],
        "current_step": progress_data["current_step"],
        "message": progress_data["message"],
        "updated_at": datetime.utcnow(),
    }
    if status == AnalysisStatus.PROCESSING and "started_at" not in update_data:
```

```
        update_data["started_at"] = datetime.utcnow()
    elif status in [AnalysisStatus.COMPLETED, AnalysisStatus.FAILED]:
        update_data["completed_at"] = datetime.utcnow()
        if result:
            update_data["result"] = result.dict()
    await db.analysis_tasks.update_one({"task_id": task_id}, {"$set": update_data})
    progress_key = RedisKeys.TASK_PROGRESS.format(task_id=task_id)
    await redis_service.set_json(
        progress_key,
        {
            "task_id": task_id,
            "status": status.value if hasattr(status, "value") else status,
            "progress": progress_data["progress"],
            "current_step": progress_data["current_step"],
            "message": progress_data["message"],
            "elapsed_time": progress_data["elapsed_time"],
            "remaining_time": progress_data["remaining_time"],
            "steps": progress_data["steps"],
            "updated_at": datetime.utcnow().isoformat(),
        },
        ttl=3600,
    )
```

FILE: app/services/analysis_service.py

```
"""
股票分析服务
将现有的TradingAgents分析功能包装成API服务
"""
import asyncio
import uuid
import json
import logging
from datetime import datetime
from typing import Dict, Any, List, Optional, Callable
from pathlib import Path
import sys
# 添加项目根目录到路径
project_root = Path(__file__).parent.parent.parent
sys.path.insert(0, str(project_root))
# 初始化TradingAgents日志系统
from tradingagents.utils.logging_init import init_logging
init_logging()
from tradingagents.graph.trading_graph import TradingAgentsGraph
from tradingagents.default_config import DEFAULT_CONFIG
from app.services.simple_analysis_service import create_analysis_config, get_provider_by_model_name
from app.models.analysis import (
    AnalysisParameters, AnalysisResult, AnalysisTask, AnalysisBatch,
    AnalysisStatus, BatchStatus, SingleAnalysisRequest, BatchAnalysisRequest
)
from app.models.user import PyObjectId
from bson import ObjectId
from app.core.database import get_mongo_db
from app.core.redis_client import get_redis_service, RedisKeys
from app.services.queue_service import QueueService
from app.core.database import get_redis_client
from app.services.redis_progress_tracker import RedisProgressTracker
from app.services.config_provider import provider as config_provider
from app.services.queue import DEFAULT_USER_CONCURRENT_LIMIT, GLOBAL_CONCURRENT_LIMIT, VISIBILITY_TIMEOUT_SECONDS
from app.services.usage_statistics_service import UsageStatisticsService
from app.models.config import UsageRecord
import logging
logger = logging.getLogger(__name__)
class AnalysisService:
    """股票分析服务类"""
    def __init__(self):
        # 获取Redis客户端
        redis_client = get_redis_client()
        self.queue_service = QueueService(redis_client)
        # 初始化使用统计服务
        self.usage_service = UsageStatisticsService()
        self._trading_graph_cache = {}
        # 进度跟踪器缓存
        self._progress_trackers: Dict[str, RedisProgressTracker] = {}
    def _convert_user_id(self, user_id: str) -> PyObjectId:
        """将字符串用户ID转换为PyObjectId"""
        try:
            logger.info(f"🔄 开始转换用户ID: {user_id} (类型: {type(user_id)})")
            # 如果是admin用户，使用固定的ObjectId
            if user_id == "admin":
                # 使用固定的ObjectId作为admin用户ID
                admin_object_id = ObjectId("507f1f77bcf86cd799439011")
                logger.info(f"🔄 转换admin用户ID: {user_id} -> {admin_object_id}")
                return PyObjectId(admin_object_id)
            else:
                # 尝试将字符串转换为ObjectId
                object_id = ObjectId(user_id)
                logger.info(f"🔄 转换用户ID: {user_id} -> {object_id}")
                return PyObjectId(object_id)
        except Exception as e:
            logger.error(f"❌ 用户ID转换失败: {user_id} -> {e}")
            # 如果转换失败，生成一个新的ObjectId
            new_object_id = ObjectId()
            logger.warning(f"⚠️ 生成新的用户ID: {new_object_id}")
            return PyObjectId(new_object_id)

    def _get_trading_graph(self, config: Dict[str, Any]) -> TradingAgentsGraph:
        """获取或创建TradingAgents图实例（带缓存）- 与单股分析保持一致"""
        config_key = json.dumps(config, sort_keys=True)
        if config_key not in self._trading_graph_cache:
            # 直接使用完整配置，不再合并DEFAULT_CONFIG（因为create_analysis_config已经处理了）
            # 这与单股分析服务和web目录的方式一致
```

```
        self._trading_graph_cache[config_key] = TradingAgentsGraph(
            selected_analysts=config.get("selected_analysts", ["market", "fundamentals"]),
            debug=config.get("debug", False),
            config=config
        )
        logger.info(f"创建新的TradingAgents实例: {config.get('llm_provider', 'default')}")
    return self._trading_graph_cache[config_key]
def _execute_analysis_sync_with_progress(self, task: AnalysisTask, progress_tracker: RedisProgressTracker) -> AnalysisResult:
    """同步执行分析任务（在线程池中运行，带进度跟踪）"""
    try:
        # 在线程中重新初始化日志系统
        from tradingagents.utils.logging_init import init_logging, get_logger
        init_logging()
        thread_logger = get_logger('analysis_thread')
        thread_logger.info(f"🔄 [线程池] 开始执行分析任务: {task.task_id} - {task.symbol}")
        logger.info(f"🔄 [线程池] 开始执行分析任务: {task.task_id} - {task.symbol}")
        # 环境检查
        progress_tracker.update_progress("🔧 检查环境配置")
        # 使用标准配置函数创建完整配置
        from app.core.unified_config import unified_config
        quick_model = getattr(task.parameters, 'quick_analysis_model', None) or unified_config.get_quick_analysis_model()
        deep_model = getattr(task.parameters, 'deep_analysis_model', None) or unified_config.get_deep_analysis_model()
        # 🔧 从 MongoDB 数据库读取模型的完整配置参数（而不是从 JSON 文件）
        quick_model_config = None
        deep_model_config = None
        try:
            from pymongo import MongoClient
            from app.core.config import settings
            # 使用同步 MongoDB 客户端
            client = MongoClient(settings.MONGO_URI)
            db = client[settings.MONGO_DB]
            collection = db.system_configs
            # 查询最新的活跃配置
            doc = collection.find_one({"is_active": True}, sort=[("version", -1)])
            if doc and "llm_configs" in doc:
                llm_configs = doc["llm_configs"]
                logger.info(f"✅ 从 MongoDB 读取到 {len(llm_configs)} 个模型配置")
                for llm_config in llm_configs:
                    if llm_config.get("model_name") == quick_model:
                        quick_model_config = {
                            "max_tokens": llm_config.get("max_tokens", 4000),
                            "temperature": llm_config.get("temperature", 0.7),
                            "timeout": llm_config.get("timeout", 180),
                            "retry_times": llm_config.get("retry_times", 3),
                            "api_base": llm_config.get("api_base")
                        }
                        logger.info(f"✅ 读取快速模型配置: {quick_model}")
                        logger.info(f"   max_tokens={quick_model_config['max_tokens']}, temperature={quick_model_config['temperature']}")
                        logger.info(f"   timeout={quick_model_config['timeout']}, retry_times={quick_model_config['retry_times']}")
                        logger.info(f"   api_base={quick_model_config['api_base']}")
                    if llm_config.get("model_name") == deep_model:
                        deep_model_config = {
                            "max_tokens": llm_config.get("max_tokens", 4000),
                            "temperature": llm_config.get("temperature", 0.7),
                            "timeout": llm_config.get("timeout", 180),
                            "retry_times": llm_config.get("retry_times", 3),
                            "api_base": llm_config.get("api_base")
                        }
                        logger.info(f"✅ 读取深度模型配置: {deep_model} - {deep_model_config}")
            else:
                logger.warning("⚠️ MongoDB 中没有找到系统配置，将使用默认参数")
        except Exception as e:
            logger.warning(f"⚠️ 从 MongoDB 读取模型配置失败: {e}，将使用默认参数")
        # 成本估算
        progress_tracker.update_progress("💰 预估分析成本")
        # 根据模型名称动态查找供应商（同步版本）
        llm_provider = "dashscope"  # 默认使用dashscope
        # 参数配置
        progress_tracker.update_progress("⚙️ 配置分析参数")
        # 使用标准配置函数创建完整配置
        from app.services.simple_analysis_service import create_analysis_config
        config = create_analysis_config(
            research_depth=task.parameters.research_depth,
            selected_analysts=task.parameters.selected_analysts or ["market", "fundamentals"],
            quick_model=quick_model,
            deep_model=deep_model,
            llm_provider=llm_provider,
            market_type=getattr(task.parameters, 'market_type', "A股"),
            quick_model_config=quick_model_config,  # 传递模型配置
            deep_model_config=deep_model_config    # 传递模型配置
        )
        # 启动引擎
        progress_tracker.update_progress("🚀 初始化AI分析引擎")
        # 获取TradingAgents实例
        trading_graph = self._get_trading_graph(config)
        # 执行分析
        from datetime import timezone
        start_time = datetime.now(timezone.utc)
        analysis_date = task.parameters.analysis_date or datetime.now().strftime("%Y-%m-%d")
        # 创建进度回调函数
        def progress_callback(message: str):
            progress_tracker.update_progress(message)
        # 调用现有的分析方法（同步调用，传递进度回调）
        _, decision = trading_graph.propagate(task.symbol, analysis_date, progress_callback)
        execution_time = (datetime.now(timezone.utc) - start_time).total_seconds()
        # 生成报告
        progress_tracker.update_progress("📊 生成分析报告")
        # 从决策中提取模型信息
        model_info = decision.get('model_info', 'Unknown') if isinstance(decision, dict) else 'Unknown'
        # 构建结果
        result = AnalysisResult(
            analysis_id=str(uuid.uuid4()),
```

```
            summary=decision.get("summary", ""),
            recommendation=decision.get("recommendation", ""),
            confidence_score=decision.get("confidence_score", 0.0),
            risk_level=decision.get("risk_level", "中等"),
            key_points=decision.get("key_points", []),
            detailed_analysis=decision,
            execution_time=execution_time,
            tokens_used=decision.get("tokens_used", 0),
            model_info=model_info  # 🤖 添加模型信息字段
        )
        logger.info(f"✅ [线程池] 分析任务完成: {task.task_id} - 耗时{execution_time:.2f}秒")
        return result
    except Exception as e:
        logger.error(f"❌ [线程池] 执行分析任务失败: {task.task_id} - {e}")
        raise
def _execute_analysis_sync(self, task: AnalysisTask) -> AnalysisResult:
    """同步执行分析任务（在线程池中运行）"""
    try:
        logger.info(f"🔄 [线程池] 开始执行分析任务: {task.task_id} - {task.symbol}")
        # 使用标准配置函数创建完整配置
        from app.core.unified_config import unified_config
        quick_model = getattr(task.parameters, 'quick_analysis_model', None) or unified_config.get_quick_analysis_model()
        deep_model = getattr(task.parameters, 'deep_analysis_model', None) or unified_config.get_deep_analysis_model()
        # 🔧 从 MongoDB 数据库读取模型的完整配置参数（而不是从 JSON 文件）
        quick_model_config = None
        deep_model_config = None
        try:
            from pymongo import MongoClient
            from app.core.config import settings
            # 使用同步 MongoDB 客户端
            client = MongoClient(settings.MONGO_URI)
            db = client[settings.MONGO_DB]
            collection = db.system_configs
            # 查询最新的活跃配置
            doc = collection.find_one({"is_active": True}, sort=[("version", -1)])
            if doc and "llm_configs" in doc:
                llm_configs = doc["llm_configs"]
                logger.info(f"✅ 从 MongoDB 读取到 {len(llm_configs)} 个模型配置")
                for llm_config in llm_configs:
                    if llm_config.get("model_name") == quick_model:
                        quick_model_config = {
                            "max_tokens": llm_config.get("max_tokens", 4000),
                            "temperature": llm_config.get("temperature", 0.7),
                            "timeout": llm_config.get("timeout", 180),
                            "retry_times": llm_config.get("retry_times", 3),
                            "api_base": llm_config.get("api_base")
                        }
                        logger.info(f"✅ 读取快速模型配置: {quick_model}")
                        logger.info(f"   max_tokens={quick_model_config['max_tokens']}, temperature={quick_model_config['temperature']}")
                        logger.info(f"   timeout={quick_model_config['timeout']}, retry_times={quick_model_config['retry_times']}")
                        logger.info(f"   api_base={quick_model_config['api_base']}")
                    if llm_config.get("model_name") == deep_model:
                        deep_model_config = {
                            "max_tokens": llm_config.get("max_tokens", 4000),
                            "temperature": llm_config.get("temperature", 0.7),
                            "timeout": llm_config.get("timeout", 180),
                            "retry_times": llm_config.get("retry_times", 3),
                            "api_base": llm_config.get("api_base")
                        }
                        logger.info(f"✅ 读取深度模型配置: {deep_model} - {deep_model_config}")
            else:
                logger.warning("⚠️ MongoDB 中没有找到系统配置，将使用默认参数")
        except Exception as e:
            logger.warning(f"⚠️ 从 MongoDB 读取模型配置失败: {e}，将使用默认参数")
        # 根据模型名称动态查找供应商（同步版本）
        llm_provider = "dashscope"  # 默认使用dashscope
        # 使用标准配置函数创建完整配置
        from app.services.simple_analysis_service import create_analysis_config
        config = create_analysis_config(
            research_depth=task.parameters.research_depth,
            selected_analysts=task.parameters.selected_analysts or ["market", "fundamentals"],
            quick_model=quick_model,
            deep_model=deep_model,
            llm_provider=llm_provider,
            market_type=getattr(task.parameters, 'market_type', "A股"),
            quick_model_config=quick_model_config,  # 传递模型配置
            deep_model_config=deep_model_config    # 传递模型配置
        )
        # 获取TradingAgents实例
        trading_graph = self._get_trading_graph(config)
        # 执行分析
        from datetime import timezone
        start_time = datetime.now(timezone.utc)
        analysis_date = task.parameters.analysis_date or datetime.now().strftime("%Y-%m-%d")
        # 调用现有的分析方法（同步调用）
        _, decision = trading_graph.propagate(task.symbol, analysis_date)
        execution_time = (datetime.now(timezone.utc) - start_time).total_seconds()
        # 从决策中提取模型信息
        model_info = decision.get('model_info', 'Unknown') if isinstance(decision, dict) else 'Unknown'
        # 构建结果
        result = AnalysisResult(
            analysis_id=str(uuid.uuid4()),
            summary=decision.get("summary", ""),
            recommendation=decision.get("recommendation", ""),
            confidence_score=decision.get("confidence_score", 0.0),
            risk_level=decision.get("risk_level", "中等"),
            key_points=decision.get("key_points", []),
            detailed_analysis=decision,
            execution_time=execution_time,
            tokens_used=decision.get("tokens_used", 0),
            model_info=model_info  # 🤖 添加模型信息字段
        )
        logger.info(f"✅ [线程池] 分析任务完成: {task.task_id} - 耗时{execution_time:.2f}秒")
```

```
        return result
    except Exception as e:
        logger.error(f"❌ [线程池] 执行分析任务失败: {task.task_id} - {e}")
        raise
async def _execute_single_analysis_async(self, task: AnalysisTask):
    """异步执行单股分析任务（在后台运行，不阻塞主线程）"""
    progress_tracker = None
    try:
        logger.info(f"🔄 开始执行分析任务: {task.task_id} - {task.symbol}")
        # 创建进度跟踪器
        progress_tracker = RedisProgressTracker(
            task_id=task.task_id,
            analysts=task.parameters.selected_analysts or ["market", "fundamentals"],
            research_depth=task.parameters.research_depth or "标准",
            llm_provider="dashscope"
        )
        # 缓存进度跟踪器
        self._progress_trackers[task.task_id] = progress_tracker
        # 初始化进度
        progress_tracker.update_progress("🚀 开始股票分析")
        await self._update_task_status_with_tracker(task.task_id, AnalysisStatus.PROCESSING, progress_tracker)
        # 在线程池中执行分析，避免阻塞事件循环
        import asyncio
        import concurrent.futures
        loop = asyncio.get_event_loop()
        # 使用线程池执行器运行同步的分析代码
        with concurrent.futures.ThreadPoolExecutor() as executor:
            result = await loop.run_in_executor(
                executor,
                self._execute_analysis_sync_with_progress,
                task,
                progress_tracker
            )
        # 标记完成
        progress_tracker.mark_completed("✅ 分析完成")
        await self._update_task_status_with_tracker(task.task_id, AnalysisStatus.COMPLETED, progress_tracker, result)
        # 记录 token 使用
        try:
            # 获取使用的模型信息
            quick_model = getattr(task.parameters, 'quick_analysis_model', None)
            deep_model = getattr(task.parameters, 'deep_analysis_model', None)
            # 优先使用深度分析模型，如果没有则使用快速分析模型
            model_name = deep_model or quick_model or "qwen-plus"
            # 根据模型名称确定供应商
            from app.services.simple_analysis_service import get_provider_by_model_name
            provider = get_provider_by_model_name(model_name)
            # 记录使用情况
            await self._record_token_usage(task, result, provider, model_name)
        except Exception as e:
            logger.error(f"⚠️ 记录 token 使用失败: {e}")
        logger.info(f"✅ 分析任务完成: {task.task_id}")
    except Exception as e:
        logger.error(f"❌ 分析任务失败: {task.task_id} - {e}")
        # 标记失败
        if progress_tracker:
            progress_tracker.mark_failed(str(e))
            await self._update_task_status_with_tracker(task.task_id, AnalysisStatus.FAILED, progress_tracker)
        else:
            await self._update_task_status(task.task_id, AnalysisStatus.FAILED, 0, str(e))
    finally:
        # 清理进度跟踪器缓存
        if task.task_id in self._progress_trackers:
            del self._progress_trackers[task.task_id]
async def submit_single_analysis(
    self,
    user_id: str,
    request: SingleAnalysisRequest
) -> Dict[str, Any]:
    """提交单股分析任务"""
    try:
        logger.info(f"📝 开始提交单股分析任务")
        logger.info(f"👤 用户ID: {user_id} (类型: {type(user_id)})")
        # 获取股票代码 (兼容旧字段)
        stock_symbol = request.get_symbol()
        logger.info(f"📊 股票代码: {stock_symbol}")
        logger.info(f"⚙️ 分析参数: {request.parameters}")
        # 生成任务ID
        task_id = str(uuid.uuid4())
        logger.info(f"🆔 生成任务ID: {task_id}")
        # 转换用户ID
        converted_user_id = self._convert_user_id(user_id)
        logger.info(f"🔄 转换后的用户ID: {converted_user_id} (类型: {type(converted_user_id)})")
        # 创建分析任务
        logger.info(f"🏗️ 开始创建AnalysisTask对象...")
        # 读取合并后的系统设置（ENV 优先 → DB），用于填充模型与并发/超时配置
        try:
            effective_settings = await config_provider.get_effective_system_settings()
        except Exception:
            effective_settings = {}
        # 填充分析参数中的模型（若请求未显式提供）
        params = request.parameters or AnalysisParameters()
        if not getattr(params, 'quick_analysis_model', None):
            params.quick_analysis_model = effective_settings.get("quick_analysis_model", "qwen-turbo")
        if not getattr(params, 'deep_analysis_model', None):
            params.deep_analysis_model = effective_settings.get("deep_analysis_model", "qwen-max")
        # 应用系统级并发与可见性超时（若提供）
        try:
            self.queue_service.user_concurrent_limit = int(effective_settings.get("max_concurrent_tasks", DEFAULT_USER_CONCURRENT_LIMIT))
            self.queue_service.global_concurrent_limit = int(effective_settings.get("max_concurrent_tasks", GLOBAL_CONCURRENT_LIMIT))
            self.queue_service.visibility_timeout = int(effective_settings.get("default_analysis_timeout", VISIBILITY_TIMEOUT_SECONDS))
        except Exception:
            # 使用默认值即可
```

```
            pass
        task = AnalysisTask(
            task_id=task_id,
            user_id=converted_user_id,
            symbol=stock_symbol,
            stock_code=stock_symbol,  # 兼容字段
            parameters=params,
            status=AnalysisStatus.PENDING
        )
        logger.info(f"✅ AnalysisTask对象创建成功")
        # 保存任务到数据库
        logger.info(f"💾 开始保存任务到数据库...")
        db = get_mongo_db()
        task_dict = task.model_dump(by_alias=True)
        logger.info(f"📄 任务字典: {task_dict}")
        await db.analysis_tasks.insert_one(task_dict)
        logger.info(f"✅ 任务已保存到数据库")
        # 单股分析：直接在后台执行（不阻塞API响应）
        logger.info(f"🚀 开始在后台执行分析任务...")
        # 创建后台任务，不等待完成
        import asyncio
        background_task = asyncio.create_task(
            self._execute_single_analysis_async(task)
        )
        # 不等待任务完成，让它在后台运行
        logger.info(f"✅ 后台任务已启动，任务ID: {task_id}")
        logger.info(f"🎉 单股分析任务提交完成: {task_id} - {stock_symbol}")
        return {
            "task_id": task_id,
            "symbol": stock_symbol,
            "stock_code": stock_symbol,  # 兼容字段
            "status": AnalysisStatus.PENDING,
            "message": "任务已在后台启动"
        }

    except Exception as e:
        logger.error(f"提交单股分析任务失败: {e}")
        raise

async def submit_batch_analysis(
    self,
    user_id: str,
    request: BatchAnalysisRequest
) -> Dict[str, Any]:
    """提交批量分析任务"""
    try:
        # 生成批次ID
        batch_id = str(uuid.uuid4())

        # 转换用户ID
        converted_user_id = self._convert_user_id(user_id)
        # 读取系统设置，填充模型参数并应用并发/超时配置
        try:
            effective_settings = await config_provider.get_effective_system_settings()
        except Exception:
            effective_settings = {}
        params = request.parameters or AnalysisParameters()
        if not getattr(params, 'quick_analysis_model', None):
            params.quick_analysis_model = effective_settings.get("quick_analysis_model", "qwen-turbo")
        if not getattr(params, 'deep_analysis_model', None):
            params.deep_analysis_model = effective_settings.get("deep_analysis_model", "qwen-max")
        try:
            self.queue_service.user_concurrent_limit = int(effective_settings.get("max_concurrent_tasks", DEFAULT_USER_CONCURRENT_LIMIT))
            self.queue_service.global_concurrent_limit = int(effective_settings.get("max_concurrent_tasks", GLOBAL_CONCURRENT_LIMIT))
            self.queue_service.visibility_timeout = int(effective_settings.get("default_analysis_timeout", VISIBILITY_TIMEOUT_SECONDS))
        except Exception:
            pass
        # 创建批次记录
        # 获取股票代码列表 (兼容旧字段)
        stock_symbols = request.get_symbols()
        batch = AnalysisBatch(
            batch_id=batch_id,
            user_id=converted_user_id,
            title=request.title,
            description=request.description,
            total_tasks=len(stock_symbols),
            parameters=params,
            status=BatchStatus.PENDING
        )
        # 创建任务列表
        tasks = []
        for symbol in stock_symbols:
            task_id = str(uuid.uuid4())
            task = AnalysisTask(
                task_id=task_id,
                batch_id=batch_id,
                user_id=converted_user_id,
                symbol=symbol,
                stock_code=symbol,  # 兼容字段
                parameters=batch.parameters,
                status=AnalysisStatus.PENDING
            )
            tasks.append(task)

        # 保存到数据库
        db = get_mongo_db()
        await db.analysis_batches.insert_one(batch.dict(by_alias=True))
        await db.analysis_tasks.insert_many([task.dict(by_alias=True) for task in tasks])

        # 提交任务到队列
        for task in tasks:
            # 准备队列参数（直接传递分析参数，不嵌套）
            queue_params = task.parameters.dict() if task.parameters else {}
            # 添加任务元数据
```

```
            queue_params.update({
                "task_id": task.task_id,
                "symbol": task.symbol,
                "stock_code": task.symbol,  # 兼容字段
                "user_id": str(task.user_id),
                "batch_id": task.batch_id,
                "created_at": task.created_at.isoformat() if task.created_at else None
            })
            # 调用队列服务
            await self.queue_service.enqueue_task(
                user_id=str(converted_user_id),
                symbol=task.symbol,
                params=queue_params,
                batch_id=task.batch_id
            )

        logger.info(f"批量分析任务已提交: {batch_id} - {len(tasks)}个股票")

        return {
            "batch_id": batch_id,
            "total_tasks": len(tasks),
            "status": BatchStatus.PENDING,
            "message": f"已提交{len(tasks)}个分析任务到队列"
        }

    except Exception as e:
        logger.error(f"提交批量分析任务失败: {e}")
        raise

async def execute_analysis_task(
    self,
    task: AnalysisTask,
    progress_callback: Optional[Callable[[int, str], None]] = None
) -> AnalysisResult:
    """执行单个分析任务"""
    try:
        logger.info(f"开始执行分析任务: {task.task_id} - {task.symbol}")

        # 更新任务状态
        await self._update_task_status(task.task_id, AnalysisStatus.PROCESSING, 0)

        if progress_callback:
            progress_callback(10, "初始化分析引擎...")

        # 使用标准配置函数创建完整配置 - 与单股分析保持一致
        from app.core.unified_config import unified_config
        quick_model = getattr(task.parameters, 'quick_analysis_model', None) or unified_config.get_quick_analysis_model()
        deep_model = getattr(task.parameters, 'deep_analysis_model', None) or unified_config.get_deep_analysis_model()
        # 🔧 从数据库读取模型的完整配置参数
        quick_model_config = None
        deep_model_config = None
        llm_configs = unified_config.get_llm_configs()
        for llm_config in llm_configs:
            if llm_config.model_name == quick_model:
                quick_model_config = {
                    "max_tokens": llm_config.max_tokens,
                    "temperature": llm_config.temperature,
                    "timeout": llm_config.timeout,
                    "retry_times": llm_config.retry_times,
                    "api_base": llm_config.api_base
                }
            if llm_config.model_name == deep_model:
                deep_model_config = {
                    "max_tokens": llm_config.max_tokens,
                    "temperature": llm_config.temperature,
                    "timeout": llm_config.timeout,
                    "retry_times": llm_config.retry_times,
                    "api_base": llm_config.api_base
                }
        # 根据模型名称动态查找供应商
        llm_provider = await get_provider_by_model_name(quick_model)
        # 使用标准配置函数创建完整配置
        config = create_analysis_config(
            research_depth=task.parameters.research_depth,
            selected_analysts=task.parameters.selected_analysts or ["market", "fundamentals"],
            quick_model=quick_model,
            deep_model=deep_model,
            llm_provider=llm_provider,
            market_type=getattr(task.parameters, 'market_type', "A股"),
            quick_model_config=quick_model_config,  # 传递模型配置
            deep_model_config=deep_model_config     # 传递模型配置
        )

        if progress_callback:
            progress_callback(30, "创建分析图...")

        # 获取TradingAgents实例
        trading_graph = self._get_trading_graph(config)

        if progress_callback:
            progress_callback(50, "执行股票分析...")

        # 执行分析
        start_time = datetime.utcnow()
        analysis_date = task.parameters.analysis_date or datetime.now().strftime("%Y-%m-%d")

        # 调用现有的分析方法
        _, decision = trading_graph.propagate(task.symbol, analysis_date)

        execution_time = (datetime.utcnow() - start_time).total_seconds()

        if progress_callback:
            progress_callback(80, "处理分析结果...")
        # 从决策中提取模型信息
        model_info = decision.get('model_info', 'Unknown') if isinstance(decision, dict) else 'Unknown'
```

```
        # 构建结果
        result = AnalysisResult(
            analysis_id=str(uuid.uuid4()),
            summary=decision.get("summary", ""),
            recommendation=decision.get("recommendation", ""),
            confidence_score=decision.get("confidence_score", 0.0),
            risk_level=decision.get("risk_level", "中等"),
            key_points=decision.get("key_points", []),
            detailed_analysis=decision,
            execution_time=execution_time,
            tokens_used=decision.get("tokens_used", 0),
            model_info=model_info  # 🔥 添加模型信息字段
        )
        if progress_callback:
            progress_callback(100, "分析完成")
        # 更新任务状态
        await self._update_task_status(task.task_id, AnalysisStatus.COMPLETED, 100, result)
        # 记录 token 使用
        try:
            # 记录使用情况
            await self._record_token_usage(task, result, llm_provider, deep_model or quick_model)
        except Exception as e:
            logger.error(f"⚠️ 记录 token 使用失败: {e}")
        logger.info(f"分析任务完成: {task.task_id} - 耗时{execution_time:.2f}秒")
        return result

    except Exception as e:
        logger.error(f"执行分析任务失败: {task.task_id} - {e}")

        # 更新任务状态为失败
        error_result = AnalysisResult(error_message=str(e))
        await self._update_task_status(task.task_id, AnalysisStatus.FAILED, 0, error_result)

        raise

async def _update_task_status(
    self,
    task_id: str,
    status: AnalysisStatus,
    progress: int,
    result: Optional[AnalysisResult] = None,
) -> None:
    """更新任务状态（委托至拆分的工具函数）"""
    try:
        from app.services.analysis.status_update_utils import perform_update_task_status
        await perform_update_task_status(task_id, status, progress, result)
    except Exception as e:
        logger.error(f"更新任务状态失败: {task_id} - {e}")
async def _update_task_status_with_tracker(
    self,
    task_id: str,
    status: AnalysisStatus,
    progress_tracker: RedisProgressTracker,
    result: Optional[AnalysisResult] = None,
) -> None:
    """使用进度跟踪器更新任务状态（委托至拆分的工具函数）"""
    try:
        from app.services.analysis.status_update_utils import perform_update_task_status_with_tracker
        await perform_update_task_status_with_tracker(task_id, status, progress_tracker, result)
    except Exception as e:
        logger.error(f"更新任务状态失败: {task_id} - {e}")
async def get_task_status(self, task_id: str) -> Optional[Dict[str, Any]]:
    """获取任务状态"""
    try:
        # 先检查内存中的进度跟踪器
        if task_id in self._progress_trackers:
            progress_tracker = self._progress_trackers[task_id]
            progress_data = progress_tracker.to_dict()
            # 从数据库获取任务基本信息
            db = get_mongo_db()
            task = await db.analysis_tasks.find_one({"task_id": task_id})
            if task:
                # 合并数据库信息和进度跟踪器信息
                return {
                    "task_id": task_id,
                    "user_id": task.get("user_id"),
                    "symbol": task.get("stock_symbol") or task.get("symbol"),
                    "stock_code": task.get("stock_symbol") or task.get("symbol"),  # 兼容字段
                    "status": progress_data["status"],
                    "progress": progress_data["progress"],
                    "current_step": progress_data["current_step"],
                    "message": progress_data["message"],
                    "elapsed_time": progress_data["elapsed_time"],
                    "remaining_time": progress_data["remaining_time"],
                    "estimated_total_time": progress_data.get("estimated_total_time", 0),
                    "steps": progress_data["steps"],
                    "start_time": progress_data["start_time"],
                    "end_time": None,
                    "last_update": progress_data["last_update"],
                    "parameters": task.get("parameters", {}),
                    "execution_time": None,
                    "tokens_used": None,
                    "result_data": task.get("result"),
                    "error_message": None
                }
        # 从Redis缓存获取
        redis_service = get_redis_service()
        progress_key = RedisKeys.TASK_PROGRESS.format(task_id=task_id)
        cached_status = await redis_service.get_json(progress_key)
        if cached_status:
            return cached_status
        # 从数据库获取
        db = get_mongo_db()
        task = await db.analysis_tasks.find_one({"task_id": task_id})
```

```
        if task:
            # 计算已用时间
            elapsed_time = 0
            remaining_time = 0
            estimated_total_time = 0
            if task.get("started_at"):
                from datetime import datetime
                start_time = task.get("started_at")
                if task.get("completed_at"):
                    # 任务已完成
                    elapsed_time = (task.get("completed_at") - start_time).total_seconds()
                    estimated_total_time = elapsed_time  # 已完成任务的总时长就是已用时间
                    remaining_time = 0
                else:
                    # 任务进行中
                    elapsed_time = (datetime.utcnow() - start_time).total_seconds()
                    # 使用任务的预估时长，如果没有则使用默认值（5分钟）
                    estimated_total_time = task.get("estimated_duration", 300)
                    # 预计剩余 = 预估总时长 - 已用时间
                    remaining_time = max(0, estimated_total_time - elapsed_time)
            return {
                "task_id": task_id,
                "status": task.get("status"),
                "progress": task.get("progress", 0),
                "current_step": task.get("current_step", ""),
                "message": task.get("message", ""),
                "elapsed_time": elapsed_time,
                "remaining_time": remaining_time,
                "estimated_total_time": estimated_total_time,
                "start_time": task.get("started_at").isoformat() if task.get("started_at") else None,
                "updated_at": task.get("updated_at", "").isoformat() if task.get("updated_at") else None,
                "result_data": task.get("result")
            }
        return None
    except Exception as e:
        logger.error(f"获取任务状态失败: {task_id} - {e}")
        return None

async def cancel_task(self, task_id: str) -> bool:
    """取消任务"""
    try:
        # 更新任务状态
        await self._update_task_status(task_id, AnalysisStatus.CANCELLED, 0)

        # 从队列中移除（如果还在队列中）
        await self.queue_service.remove_task(task_id)

        logger.info(f"任务已取消: {task_id}")
        return True

    except Exception as e:
        logger.error(f"取消任务失败: {task_id} - {e}")
        return False
async def _record_token_usage(
    self,
    task: AnalysisTask,
    result: AnalysisResult,
    provider: str,
    model_name: str
):
    """记录 token 使用情况"""
    try:
        # 从结果中提取 token 使用信息
        # 注意：这里需要从 LLM 响应中获取实际的 token 使用量
        # 目前使用估算值
        input_tokens = result.tokens_used // 2 if result.tokens_used > 0 else 0
        output_tokens = result.tokens_used - input_tokens if result.tokens_used > 0 else 0
        # 如果没有 token 使用信息，使用默认估算
        if result.tokens_used == 0:
            # 根据分析类型估算
            input_tokens = 2000  # 默认输入 token
            output_tokens = 1000  # 默认输出 token
        # 获取模型价格配置
        from app.services.config_service import config_service
        config = await config_service.get_system_config()
        # 查找对应的 LLM 配置
        llm_config = None
        if config and config.llm_configs:
            for cfg in config.llm_configs:
                if cfg.provider == provider and cfg.model_name == model_name:
                    llm_config = cfg
                    break
        # 计算成本
        cost = 0.0
        currency = "CNY"  # 默认货币单位
        if llm_config:
            input_price = llm_config.input_price_per_1k or 0.0
            output_price = llm_config.output_price_per_1k or 0.0
            cost = (input_tokens / 1000 * input_price) + (output_tokens / 1000 * output_price)
            currency = llm_config.currency or "CNY"
        # 创建使用记录
        usage_record = UsageRecord(
            timestamp=datetime.now().isoformat(),
            provider=provider,
            model_name=model_name,
            input_tokens=input_tokens,
            output_tokens=output_tokens,
            cost=cost,
            currency=currency,
            session_id=task.task_id,
            analysis_type="stock_analysis",
            stock_code=task.symbol
        )
        # 保存到数据库
```

```
        success = await self.usage_service.add_usage_record(usage_record)
        if success:
            logger.info(f"⏱️ 记录使用成本: {provider}/{model_name} - ¥{cost:.4f}")
        else:
            logger.warning(f"⚠️ 记录使用成本失败")
    except Exception as e:
        logger.error(f"❌ 记录 token 使用失败: {e}")
# 全局分析服务实例（延迟初始化）
analysis_service: Optional[AnalysisService] = None
def get_analysis_service() -> AnalysisService:
    """获取分析服务实例（延迟初始化）"""
    global analysis_service
    if analysis_service is None:
        analysis_service = AnalysisService()
    return analysis_service
```

FILE: app/services/auth_service.py

```
import time
from datetime import datetime, timedelta, timezone
from app.utils.timezone import now_tz
from typing import Optional
import jwt
from pydantic import BaseModel
from app.core.config import settings
class TokenData(BaseModel):
    sub: str
    exp: int
class AuthService:
    @staticmethod
    def create_access_token(sub: str, expires_minutes: int | None = None, expires_delta: int | None = None) -> str:
        if expires_delta:
            # 如果指定了秒数，使用秒数
            expire = now_tz() + timedelta(seconds=expires_delta)
        else:
            # 否则使用分钟数
            expire = now_tz() + timedelta(minutes=expires_minutes or settings.ACCESS_TOKEN_EXPIRE_MINUTES)
        payload = {"sub": sub, "exp": expire}
        token = jwt.encode(payload, settings.JWT_SECRET, algorithm=settings.JWT_ALGORITHM)
        return token
    @staticmethod
    def verify_token(token: str) -> Optional[TokenData]:
        import logging
        logger = logging.getLogger(__name__)
        try:
            logger.debug(f"🔍 开始验证token")
            logger.debug(f"📝 Token长度: {len(token)}")
            logger.debug(f"🔑 JWT密钥: {settings.JWT_SECRET[:10]}...")
            logger.debug(f"🔧 JWT算法: {settings.JWT_ALGORITHM}")
            payload = jwt.decode(token, settings.JWT_SECRET, algorithms=[settings.JWT_ALGORITHM])
            logger.debug(f"✅ Token解码成功")
            logger.debug(f"📋 Payload: {payload}")
            token_data = TokenData(sub=payload.get("sub"), exp=int(payload.get("exp", time.time())))
            logger.debug(f"🎯 Token数据: sub={token_data.sub}, exp={token_data.exp}")
            # 检查是否过期
            current_time = int(time.time())
            if token_data.exp < current_time:
                logger.warning(f"⏰ Token已过期: exp={token_data.exp}, now={current_time}")
                return None
            logger.debug(f"✅ Token验证成功")
            return token_data
        except jwt.ExpiredSignatureError:
            logger.warning("⏰ Token已过期")
            return None
        except jwt.InvalidTokenError as e:
            logger.warning(f"❌ Token无效: {str(e)}")
            return None
        except Exception as e:
            logger.error(f"❌ Token验证异常: {str(e)}")
            return None
```

FILE: app/services/basics_sync/__init__.py

```
"""
基础数据同步子包：封装与股票基础信息同步相关的阻塞调用与处理函数。
- utils.py：与 Tushare 的阻塞式获取函数（股票列表、最新交易日、日度基础数据）
- processing.py：共享的文档构建/指标处理函数
"""
from .utils import (
    fetch_stock_basic_df,
    find_latest_trade_date,
    fetch_daily_basic_mv_map,
    fetch_latest_roe_map,
)
from .processing import add_financial_metrics
```

FILE: app/services/basics_sync/processing.py

```
"""
共享的文档指标处理函数
- add_financial_metrics: 将日度基础指标（市值/估值/交易）追加到文档中
"""
from typing import Dict
def add_financial_metrics(doc: Dict, daily_metrics: Dict) -> None:
    """
    将财务与交易指标写入 doc（就地修改）。
    - 市值：total_mv/circ_mv（从万元转换为亿元）
    - 估值：pe/pb/pe_ttm/pb_mrq/ps/ps_ttm（过滤 NaN/None）
    - 交易：turnover_rate/volume_ratio（过滤 NaN/None）
```

```
    - 股本：total_share/float_share（万股，过滤 NaN/None）
    """
    # 市值（万元 -> 亿元）
    if "total_mv" in daily_metrics and daily_metrics["total_mv"] is not None:
        doc["total_mv"] = daily_metrics["total_mv"] / 10000
    if "circ_mv" in daily_metrics and daily_metrics["circ_mv"] is not None:
        doc["circ_mv"] = daily_metrics["circ_mv"] / 10000
    # 估值指标（🔥 新增 ps 和 ps_ttm）
    for field in ["pe", "pb", "pe_ttm", "pb_mrq", "ps", "ps_ttm"]:
        if field in daily_metrics and daily_metrics[field] is not None:
            try:
                value = float(daily_metrics[field])
                if not (value != value):  # 过滤 NaN
                    doc[field] = value
            except (ValueError, TypeError):
                pass
    # 交易指标
    for field in ["turnover_rate", "volume_ratio"]:
        if field in daily_metrics and daily_metrics[field] is not None:
            try:
                value = float(daily_metrics[field])
                if not (value != value):  # 过滤 NaN
                    doc[field] = value
            except (ValueError, TypeError):
                pass
    # 🔥 股本数据（万股）
    for field in ["total_share", "float_share"]:
        if field in daily_metrics and daily_metrics[field] is not None:
            try:
                value = float(daily_metrics[field])
                if not (value != value):  # 过滤 NaN
                    doc[field] = value
            except (ValueError, TypeError):
                pass
```

FILE: app/services/basics_sync/utils.py

```
"""
与 Tushare 相关的阻塞式工具函数：
- fetch_stock_basic_df：获取股票列表（确保 Tushare 已连接）
- find_latest_trade_date：探测最近可用交易日（YYYYMMDD）
- fetch_daily_basic_mv_map：根据交易日获取日度基础指标映射（市值/估值/交易）
"""
from __future__ import annotations
from datetime import datetime, timedelta
from typing import Dict

def fetch_stock_basic_df():
    """
    从 Tushare 获取股票基础列表（DataFrame格式），要求已正确配置并连接。
    依赖环境变量：TUSHARE_ENABLED=true 且 .env 中提供 TUSHARE_TOKEN。
    注意：这是一个同步函数，会等待 Tushare 连接完成。
    """
    import time
    import logging
    from tradingagents.dataflows.providers.china.tushare import get_tushare_provider
    from app.core.config import settings
    logger = logging.getLogger(__name__)
    # 检查 Tushare 是否启用
    if not settings.TUSHARE_ENABLED:
        logger.error("❌ Tushare 数据源已禁用 (TUSHARE_ENABLED=false)")
        logger.error("💡 请在 .env 文件中设置 TUSHARE_ENABLED=true 或使用多数据源同步服务")
        raise RuntimeError(
            "Tushare is disabled (TUSHARE_ENABLED=false). "
            "Set TUSHARE_ENABLED=true in .env or use MultiSourceBasicsSyncService."
        )
    provider = get_tushare_provider()
    # 等待连接完成（最多等待 5 秒）
    max_wait_seconds = 5
    wait_interval = 0.1
    elapsed = 0.0
    logger.info(f"⏳ 等待 Tushare 连接...")
    while not getattr(provider, "connected", False) and elapsed < max_wait_seconds:
        time.sleep(wait_interval)
        elapsed += wait_interval
    # 检查连接状态和API可用性
    if not getattr(provider, "connected", False) or provider.api is None:
        logger.error(f"❌ Tushare 连接失败（等待 {max_wait_seconds}s 后超时）")
        logger.error(f"💡 请检查：")
        logger.error(f"   1. .env 文件中配置了有效的 TUSHARE_TOKEN")
        logger.error(f"   2. Tushare Token 未过期且有足够的积分")
        logger.error(f"   3. 网络连接正常")
        raise RuntimeError(
            f"Tushare not connected after waiting {max_wait_seconds}s. "
            "Check TUSHARE_TOKEN in .env and ensure it's valid."
        )
    logger.info(f"✅ Tushare 已连接，开始获取股票列表...")
    # 直接调用 Tushare API 获取 DataFrame
    try:
        df = provider.api.stock_basic(
            list_status='L',
            fields='ts_code,symbol,name,area,industry,market,exchange,list_date,is_hs'
        )
        # 🔧 增强错误诊断
        if df is None:
            logger.error(f"❌ Tushare API 返回 None")
            logger.error(f"💡 可能原因：")
            logger.error(f"   1. Tushare Token 无效或过期")
            logger.error(f"   2. API 积分不足")
            logger.error(f"   3. 网络连接问题")
            raise RuntimeError("Tushare API returned None. Check token validity and API credits.")
        if hasattr(df, 'empty') and df.empty:
            logger.error(f"❌ Tushare API 返回空 DataFrame")
```

```
        logger.error(f"💡 可能原因：")
        logger.error(f"   1. list_status='L' 参数可能不正确")
        logger.error(f"   2. Tushare 数据源暂时不可用")
        logger.error(f"   3. API 调用限制（请检查积分和调用频率）")
        raise RuntimeError("Tushare API returned empty DataFrame. Check API parameters and data availability.")
    logger.info(f"☑ 成功获取 {len(df)} 条股票数据")
    return df
except Exception as e:
    logger.error(f"✕ 调用 Tushare API 失败: {e}")
    raise RuntimeError(f"Failed to fetch stock basic DataFrame: {e}")

def find_latest_trade_date() -> str:
    """
    探测最近可用的交易日（YYYYMMDD）。
    - 从今天起回溯最多 5 天；
    - 如都不可用，回退为昨天日期。
    """
    from tradingagents.dataflows.providers.china.tushare import get_tushare_provider
    provider = get_tushare_provider()
    api = provider.api
    if api is None:
        raise RuntimeError("Tushare API unavailable")
    today = datetime.now()
    for delta in range(0, 6):
        d = (today - timedelta(days=delta)).strftime("%Y%m%d")
        try:
            db = api.daily_basic(trade_date=d, fields="ts_code,total_mv")
            if db is not None and not db.empty:
                return d
        except Exception:
            continue
    return (today - timedelta(days=1)).strftime("%Y%m%d")

def fetch_daily_basic_mv_map(trade_date: str) -> Dict[str, Dict[str, float]]:
    """
    根据交易日获取日度基础指标映射。
    覆盖字段：total_mv/circ_mv/pe/pb/ps/turnover_rate/volume_ratio/pe_ttm/pb_mrq/ps_ttm
    """
    from tradingagents.dataflows.providers.china.tushare import get_tushare_provider
    provider = get_tushare_provider()
    api = provider.api
    if api is None:
        raise RuntimeError("Tushare API unavailable")
    # 🔥 新增：添加 ps、ps_ttm、total_share、float_share 字段
    fields = "ts_code,total_mv,circ_mv,pe,pb,ps,turnover_rate,volume_ratio,pe_ttm,pb_mrq,ps_ttm,total_share,float_share"
    db = api.daily_basic(trade_date=trade_date, fields=fields)
    data_map: Dict[str, Dict[str, float]] = {}
    if db is not None and not db.empty:
        for _, row in db.iterrows():  # type: ignore
            ts_code = row.get("ts_code")
            if ts_code is not None:
                try:
                    metrics = {}
                    # 🔥 新增：添加 ps、ps_ttm、total_share、float_share 到字段列表
                    for field in [
                        "total_mv",
                        "circ_mv",
                        "pe",
                        "pb",
                        "ps",
                        "turnover_rate",
                        "volume_ratio",
                        "pe_ttm",
                        "pb_mrq",
                        "ps_ttm",
                        "total_share",
                        "float_share",
                    ]:
                        value = row.get(field)
                        if value is not None and str(value).lower() not in ["nan", "none", ""]:
                            metrics[field] = float(value)
                    if metrics:
                        data_map[str(ts_code)] = metrics
                except Exception:
                    pass
    return data_map

def fetch_latest_roe_map() -> Dict[str, Dict[str, float]]:
    """
    获取最近一个可用财报期的 ROE 映射（ts_code -> {"roe": float}）。
    优先按最近季度的 end_date 逆序探测，找到第一期非空数据。
    """
    from tradingagents.dataflows.providers.china.tushare import get_tushare_provider
    from datetime import datetime
    provider = get_tushare_provider()
    api = provider.api
    if api is None:
        raise RuntimeError("Tushare API unavailable")
    # 生成最近若干个财政季度的期末日期，格式 YYYYMMDD
    def quarter_ends(now: datetime):
        y = now.year
        q_dates = [
            f"{y}0331",
            f"{y}0630",
            f"{y}0930",
            f"{y}1231",
        ]
        # 包含上一年，增加成功概率
        py = y - 1
        q_dates_prev = [
            f"{py}1231",
            f"{py}0930",
            f"{py}0630",
            f"{py}0331",
        ]
        # 近6期即可
```

```
        return q_dates_prev + q_dates
    candidates = quarter_ends(datetime.now())
    data_map: Dict[str, Dict[str, float]] = {}
    for end_date in candidates:
        try:
            df = api.fina_indicator(end_date=end_date, fields="ts_code,end_date,roe")
            if df is not None and not df.empty:
                for _, row in df.iterrows():  # type: ignore
                    ts_code = row.get("ts_code")
                    val = row.get("roe")
                    if ts_code is None or val is None:
                        continue
                    try:
                        v = float(val)
                    except Exception:
                        continue
                    data_map[str(ts_code)] = {"roe": v}
                if data_map:
                    break  # 找到最近一期即可
        except Exception:
            continue
    return data_map
```

FILE: app/services/basics_sync_service.py

```
"""
Stock basics synchronization service
- Fetches A-share stock basic info from Tushare
- Enriches with latest market cap (total_mv)
- Upserts into MongoDB collection `stock_basic_info`
- Persists status in collection `sync_status` with key `stock_basics`
- Provides a singleton accessor for reuse across routers/scheduler
This module is async-friendly and offloads blocking IO (Tushare/pandas) to a thread.
"""
from __future__ import annotations
import asyncio
import logging
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Any, Dict, List, Optional
from motor.motor_asyncio import AsyncIOMotorDatabase
from pymongo import UpdateOne
from app.core.database import get_mongo_db
from app.core.config import settings
from app.services.basics_sync import (
    fetch_stock_basic_df as _fetch_stock_basic_df_util,
    find_latest_trade_date as _find_latest_trade_date_util,
    fetch_daily_basic_mv_map as _fetch_daily_basic_mv_map_util,
    fetch_latest_roe_map as _fetch_latest_roe_map_util,
)
logger = logging.getLogger(__name__)
STATUS_COLLECTION = "sync_status"
DATA_COLLECTION = "stock_basic_info"
JOB_KEY = "stock_basics"
@dataclass
class SyncStats:
    started_at: Optional[str] = None
    finished_at: Optional[str] = None
    status: str = "idle"  # idle|running|success|failed
    total: int = 0
    inserted: int = 0
    updated: int = 0
    errors: int = 0
    message: str = ""
    last_trade_date: Optional[str] = None  # YYYYMMDD
class BasicsSyncService:
    def __init__(self) -> None:
        self._lock = asyncio.Lock()
        self._running = False
        self._last_status: Optional[Dict[str, Any]] = None
        self._indexes_ensured = False
    async def _ensure_indexes(self, db: AsyncIOMotorDatabase) -> None:
        """确保必要的索引存在"""
        if self._indexes_ensured:
            return
        try:
            collection = db[DATA_COLLECTION]
            logger.info("📊 检查并创建股票基础信息索引...")
            # 1. 复合唯一索引：股票代码+数据源（用于 upsert）
            await collection.create_index([
                ("code", 1),
                ("source", 1)
            ], unique=True, name="code_source_unique", background=True)
            # 2. 股票代码索引（查询所有数据源）
            await collection.create_index[("code", 1)], name="code_index", background=True)
            # 3. 数据源索引（按数据源筛选）
            await collection.create_index[("source", 1)], name="source_index", background=True)
            # 4. 股票名称索引（按名称搜索）
            await collection.create_index[("name", 1)], name="name_index", background=True)
            # 5. 行业索引（按行业筛选）
            await collection.create_index[("industry", 1)], name="industry_index", background=True)
            # 6. 市场索引（按市场筛选）
            await collection.create_index[("market", 1)], name="market_index", background=True)
            # 7. 总市值索引（按市值排序）
            await collection.create_index[("total_mv", -1)], name="total_mv_desc", background=True)
            # 8. 流通市值索引（按流通市值排序）
            await collection.create_index[("circ_mv", -1)], name="circ_mv_desc", background=True)
            # 9. 更新时间索引（数据维护）
            await collection.create_index[("updated_at", -1)], name="updated_at_desc", background=True)
            # 10. PE索引（按估值筛选）
```

```
        await collection.create_index([("pe", 1)], name="pe_index", background=True)
        # 11. PB索引（按估值筛选）
        await collection.create_index([("pb", 1)], name="pb_index", background=True)
        # 12. 换手率索引（按活跃度筛选）
        await collection.create_index([("turnover_rate", -1)], name="turnover_rate_desc", background=True)
        self._indexes_ensured = True
        logger.info("☑ 股票基础信息索引检查完成")
    except Exception as e:
        # 索引创建失败不应该阻止服务启动
        logger.warning(f"⚠ 创建索引时出现警告（可能已存在）: {e}")
async def get_status(self, db: Optional[AsyncIOMotorDatabase] = None) -> Dict[str, Any]:
    """Return last persisted status; falls back to in-memory snapshot."""
    try:
        db = db or get_mongo_db()
        doc = await db[STATUS_COLLECTION].find_one({"job": JOB_KEY})
        if doc:
            doc.pop("_id", None)
            return doc
    except Exception as e:
        logger.warning(f"Failed to load sync status from DB: {e}")
    return self._last_status or {"job": JOB_KEY, "status": "idle"}
async def _persist_status(self, db: AsyncIOMotorDatabase, stats: Dict[str, Any]) -> None:
    stats["job"] = JOB_KEY
    await db[STATUS_COLLECTION].update_one({"job": JOB_KEY}, {"$set": stats}, upsert=True)
    self._last_status = {k: v for k, v in stats.items() if k != "_id"}
async def _execute_bulk_write_with_retry(
    self,
    db: AsyncIOMotorDatabase,
    operations: List,
    max_retries: int = 3
) -> tuple:
    """
    执行批量写入，带重试机制
    Args:
        db: MongoDB数据库实例
        operations: 批量操作列表
        max_retries: 最大重试次数
    Returns:
        (新增数量, 更新数量)
    """
    inserted = 0
    updated = 0
    retry_count = 0
    while retry_count < max_retries:
        try:
            result = await db[DATA_COLLECTION].bulk_write(operations, ordered=False)
            inserted = len(result.upserted_ids) if result.upserted_ids else 0
            updated = result.modified_count or 0
            logger.debug(f"☑ 批量写入成功: 新增 {inserted}, 更新 {updated}")
            return inserted, updated
        except asyncio.TimeoutError as e:
            retry_count += 1
            if retry_count < max_retries:
                wait_time = 2 ** retry_count  # 指数退避：2秒、4秒、8秒
                logger.warning(f"⚠ 批量写入超时 (第{retry_count}次重试)，等待{wait_time}秒后重试...")
                await asyncio.sleep(wait_time)
            else:
                logger.error(f"✖ 批量写入失败，已重试{max_retries}次: {e}")
                return 0, 0
        except Exception as e:
            logger.error(f"✖ 批量写入失败: {e}")
            return 0, 0
    return inserted, updated
async def run_full_sync(self, force: bool = False) -> Dict[str, Any]:
    """Run a full sync. If already running, return current status unless force."""
    async with self._lock:
        if self._running and not force:
            logger.info("Stock basics sync already running; skip start")
            return await self.get_status()
        self._running = True
    db = get_mongo_db()
    # 🔥 确保索引存在（提升查询和 upsert 性能）
    await self._ensure_indexes(db)
    stats = SyncStats()
    stats.started_at = datetime.utcnow().isoformat()
    stats.status = "running"
    await self._persist_status(db, stats.__dict__.copy())
    try:
        # Step 0: Check if Tushare is enabled
        if not settings.TUSHARE_ENABLED:
            error_msg = (
                "✖ Tushare 数据源已禁用 (TUSHARE_ENABLED=false)\n"
                "💡 此服务仅支持 Tushare 数据源\n"
                "📋 解决方案：\n"
                "  1. 在 .env 文件中设置 TUSHARE_ENABLED=true 并配置 TUSHARE_TOKEN\n"
                "  2. 系统已自动切换到多数据源同步服务（支持 AKShare/BaoStock）"
            )
            logger.warning(error_msg)
            raise RuntimeError(error_msg)
        # Step 1: Fetch stock basic list from Tushare (blocking -> thread)
        stock_df = await asyncio.to_thread(self._fetch_stock_basic_df)
        if stock_df is None or getattr(stock_df, "empty", True):
            raise RuntimeError("Tushare returned empty stock_basic list")
        # Step 2: Determine latest trade_date and fetch daily_basic for financial metrics (blocking -> thread)
        latest_trade_date = await asyncio.to_thread(self._find_latest_trade_date)
        stats.last_trade_date = latest_trade_date
        daily_data_map = await asyncio.to_thread(self._fetch_daily_basic_mv_map, latest_trade_date)
        # Step 2b: Fetch latest ROE snapshot from fina_indicator (blocking -> thread)
        roe_map = await asyncio.to_thread(self._fetch_latest_roe_map)
        # Step 3: Upsert into MongoDB (batched bulk writes)
        ops: List[UpdateOne] = []
        now_iso = datetime.utcnow().isoformat()
        for _, row in stock_df.iterrows():  # type: ignore
```

```
    name = row.get("name") or ""
    area = row.get("area") or ""
    industry = row.get("industry") or ""
    market = row.get("market") or ""
    list_date = row.get("list_date") or ""
    ts_code = row.get("ts_code") or ""
    # Extract 6-digit stock code from ts_code (e.g., "000001.SZ" -> "000001")
    if isinstance(ts_code, str) and "." in ts_code:
        code = ts_code.split(".")[0]  # Keep the 6-digit format
    else:
        # Fallback to symbol with zero-padding if ts_code is invalid
        symbol = row.get("symbol") or ""
        code = str(symbol).zfill(6) if symbol else ""
    # 根据 ts_code 判断交易所
    if isinstance(ts_code, str):
        if ts_code.endswith(".SH"):
            sse = "上海证券交易所"
        elif ts_code.endswith(".SZ"):
            sse = "深圳证券交易所"
        elif ts_code.endswith(".BJ"):
            sse = "北京证券交易所"
        else:
            sse = "未知"
    else:
        sse = "未知"
    category = "stock_cn"
    # Extract daily financial metrics - use ts_code directly for matching
    daily_metrics = {}
    if isinstance(ts_code, str) and ts_code in daily_data_map:
        daily_metrics = daily_data_map[ts_code]
    # Process market cap (convert from 万元 to 亿元)
    total_mv_yi = None
    circ_mv_yi = None
    if "total_mv" in daily_metrics:
        try:
            total_mv_yi = float(daily_metrics["total_mv"]) / 10000.0
        except Exception:
            pass
    if "circ_mv" in daily_metrics:
        try:
            circ_mv_yi = float(daily_metrics["circ_mv"]) / 10000.0
        except Exception:
            pass
    # 生成 full_symbol（完整标准化代码）
    full_symbol = self._generate_full_symbol(code)
    doc = {
        "code": code,
        "symbol": code,  # 添加 symbol 字段（标准化字段）
        "name": name,
        "area": area,
        "industry": industry,
        "market": market,
        "list_date": list_date,
        "sse": sse,
        "sec": category,
        "source": "tushare",  # 🔥 数据源标识
        "updated_at": now_iso,
        "full_symbol": full_symbol,  # 添加完整标准化代码
    }
    # Add market cap fields
    if total_mv_yi is not None:
        doc["total_mv"] = total_mv_yi
    if circ_mv_yi is not None:
        doc["circ_mv"] = circ_mv_yi
    # Add financial ratios (🔥 新增 ps 和 ps_ttm)
    for field in ["pe", "pb", "ps", "pe_ttm", "pb_mrq", "ps_ttm"]:
        if field in daily_metrics:
            doc[field] = daily_metrics[field]
    # ROE from fina_indicator snapshot
    if isinstance(ts_code, str) and ts_code in roe_map:
        roe_val = roe_map[ts_code].get("roe")
        if roe_val is not None:
            doc["roe"] = roe_val
    # Add trading metrics
    for field in ["turnover_rate", "volume_ratio"]:
        if field in daily_metrics:
            doc[field] = daily_metrics[field]
    # 🔥 Add share capital fields (total_share, float_share)
    for field in ["total_share", "float_share"]:
        if field in daily_metrics:
            doc[field] = daily_metrics[field]
    # 🔥 使用 (code, source) 联合查询条件
    ops.append(
        UpdateOne({"code": code, "source": "tushare"}, {"$set": doc}, upsert=True)
    )
inserted = 0
updated = 0
errors = 0
# Execute in chunks to avoid oversized batches
BATCH = 1000
for i in range(0, len(ops), BATCH):
    batch = ops[i : i + BATCH]
    batch_inserted, batch_updated = await self._execute_bulk_write_with_retry(db, batch)
    if batch_inserted > 0 or batch_updated > 0:
        inserted += batch_inserted
        updated += batch_updated
    else:
        errors += 1
        logger.error(f"Bulk write error on batch {i//BATCH}")
stats.total = len(ops)
stats.inserted = inserted
stats.updated = updated
stats.errors = errors
```

```
        stats.status = "success" if errors == 0 else "success_with_errors"
        stats.finished_at = datetime.utcnow().isoformat()
        await self._persist_status(db, stats.__dict__.copy())
        logger.info(
            f"Stock basics sync finished: total={stats.total} inserted={inserted} updated={updated} errors={errors} trade_date={latest_trade_date}"
        )
        return stats.__dict__
    except Exception as e:
        stats.status = "failed"
        stats.message = str(e)
        stats.finished_at = datetime.utcnow().isoformat()
        await self._persist_status(db, stats.__dict__.copy())
        logger.exception(f"Stock basics sync failed: {e}")
        return stats.__dict__
    finally:
        async with self._lock:
            self._running = False
# ---- Blocking helpers (run in thread) ----
def _fetch_stock_basic_df(self):
    """委托到 basics_sync.utils 的阻塞式实现"""
    return _fetch_stock_basic_df_util()
def _find_latest_trade_date(self) -> str:
    """Delegate to basics_sync.utils (blocking)"""
    return _find_latest_trade_date_util()
def _fetch_daily_basic_mv_map(self, trade_date: str) -> Dict[str, Dict[str, float]]:
    """Delegate to basics_sync.utils (blocking)"""
    return _fetch_daily_basic_mv_map_util(trade_date)
def _fetch_latest_roe_map(self) -> Dict[str, Dict[str, float]]:
    """Delegate to basics_sync.utils (blocking)"""
    return _fetch_latest_roe_map_util()
def _generate_full_symbol(self, code: str) -> str:
    """
    根据股票代码生成完整标准化代码
    Args:
        code: 6位股票代码
    Returns:
        完整标准化代码（如 000001.SZ），如果代码无效则返回原始代码（确保不为空）
    """
    # 确保 code 不为空
    if not code:
        return ""
    # 标准化为字符串并去除空格
    code = str(code).strip()
    # 如果长度不是 6，返回原始代码（避免返回 None）
    if len(code) != 6:
        return code
    # 根据代码判断交易所
    if code.startswith(('60', '68', '90')):
        return f"{code}.SS"  # 上海证券交易所
    elif code.startswith(('00', '30', '20')):
        return f"{code}.SZ"  # 深圳证券交易所
    elif code.startswith(('8', '4')):
        return f"{code}.BJ"  # 北京证券交易所
    else:
        # 无法识别的代码，返回原始代码（确保不为空）
        return code if code else ""
# Singleton accessor
_basics_sync_service: Optional[BasicsSyncService] = None
def get_basics_sync_service() -> BasicsSyncService:
    global _basics_sync_service
    if _basics_sync_service is None:
        _basics_sync_service = BasicsSyncService()
    return _basics_sync_service
```

FILE: app/services/config_provider.py

```
from __future__ import annotations
from datetime import datetime, timedelta
from typing import Any, Dict, Optional
import os
from app.services.config_service import config_service
class ConfigProvider:
    """Effective configuration provider with simple env→DB merge and TTL cache.
    - Priority: ENV > DB
    - Cache TTL: configurable (default 60s)
    - Invalidate on writes: caller should invoke `invalidate()` after writes
    """
    def __init__(self, ttl_seconds: int = 60) -> None:
        self._ttl = timedelta(seconds=ttl_seconds)
        self._cache_settings: Optional[Dict[str, Any]] = None
        self._cache_time: Optional[datetime] = None
    def invalidate(self) -> None:
        self._cache_settings = None
        self._cache_time = None
    def _is_cache_valid(self) -> bool:
        return (
            self._cache_settings is not None
            and self._cache_time is not None
            and __import__("datetime").datetime.now(__import__("datetime").timezone.utc) - self._cache_time < self._ttl
        )
    async def get_effective_system_settings(self) -> Dict[str, Any]:
        if self._is_cache_valid():
            return dict(self._cache_settings or {})
        # Load DB settings
        cfg = await config_service.get_system_config()
        base: Dict[str, Any] = {}
        if cfg and getattr(cfg, "system_settings", None):
            try:
                base = dict(cfg.system_settings)
            except Exception:
```

```
            base = {}
        # Merge ENV over DB (best-effort heuristics):
        # - if ENV with exact key exists -> override
        # - try uppercased and dot/space to underscore variants
        merged: Dict[str, Any] = dict(base)
        for k, v in list(base.items()):
            candidates = [
                k,
                k.upper(),
                str(k).replace(".", "_").replace(" ", "_").upper(),
            ]
            found = None
            for ek in candidates:
                if ek in os.environ:
                    found = os.environ.get(ek)
                    break
            if found is not None:
                merged[k] = found
        # Optionally: allow whitelisting additional env-only keys via prefix
        # For now, keep minimal behavior to avoid surprising surfaces.
        # Cache
        self._cache_settings = dict(merged)
        self._cache_time = __import__("datetime").datetime.now(__import__("datetime").timezone.utc)
        return dict(merged)
    async def get_system_settings_meta(self) -> Dict[str, Dict[str, Any]]:
        """Return metadata for system settings keys including sensitivity, editability and source.
        Fields per key:
          - sensitive: bool (by keyword patterns)
          - editable: bool (False if sensitive or source is environment; True otherwise)
          - source: 'environment' | 'database' | 'default'
          - has_value: bool (effective value is not None/empty)
        """
        # Load DB settings raw
        cfg = await config_service.get_system_config()
        db_settings: Dict[str, Any] = {}
        if cfg and getattr(cfg, "system_settings", None):
            try:
                db_settings = dict(cfg.system_settings)
            except Exception:
                db_settings = {}
        def _env_override_for_key(key: str) -> Optional[Any]:
            candidates = [
                key,
                key.upper(),
                str(key).replace(".", "_").replace(" ", "_").upper(),
            ]
            for ek in candidates:
                if ek in os.environ:
                    return os.environ.get(ek)
            return None
        sens_patterns = ("key", "secret", "password", "token", "client_secret")
        meta: Dict[str, Dict[str, Any]] = {}
        for k, v in db_settings.items():
            env_v = _env_override_for_key(k)
            source = "environment" if env_v is not None else ("database" if v is not None else "default")
            sensitive = isinstance(k, str) and any(p in k.lower() for p in sens_patterns)
            editable = not sensitive and source != "environment"
            effective_val = env_v if env_v is not None else v
            has_value = effective_val not in (None, "")
            meta[k] = {
                "sensitive": bool(sensitive),
                "editable": bool(editable),
                "source": source,
                "has_value": bool(has_value),
            }
        return meta

# Module-level singleton
provider = ConfigProvider(ttl_seconds=60)
```

FILE: app/services/config_service.py

```
"""
配置管理服务
"""
import time
import asyncio
import logging
from typing import List, Optional, Dict, Any
from datetime import datetime
from app.utils.timezone import now_tz
from bson import ObjectId
from app.core.database import get_mongo_db
from app.core.unified_config import unified_config
from app.models.config import (
    SystemConfig, LLMConfig, DataSourceConfig, DatabaseConfig,
    ModelProvider, DataSourceType, DatabaseType, LLMProvider,
    MarketCategory, DataSourceGrouping, ModelCatalog, ModelInfo
)
logger = logging.getLogger(__name__)
class ConfigService:
    """配置管理服务类"""
    def __init__(self, db_manager=None):
        self.db = None
        self.db_manager = db_manager
    async def _get_db(self):
        """获取数据库连接"""
        if self.db is None:
            if self.db_manager and self.db_manager.mongo_db is not None:
                # 如果有DatabaseManager实例，直接使用
                self.db = self.db_manager.mongo_db
            else:
                # 否则使用全局函数
```

```
        self.db = get_mongo_db()
    return self.db
# ==================== 市场分类管理 ====================
async def get_market_categories(self) -> List[MarketCategory]:
    """获取所有市场分类"""
    try:
        db = await self._get_db()
        categories_collection = db.market_categories
        categories_data = await categories_collection.find({}).to_list(length=None)
        categories = [MarketCategory(**data) for data in categories_data]
        # 如果没有分类，创建默认分类
        if not categories:
            categories = await self._create_default_market_categories()
        # 按排序顺序排列
        categories.sort(key=lambda x: x.sort_order)
        return categories
    except Exception as e:
        print(f"❌ 获取市场分类失败: {e}")
        return []
async def _create_default_market_categories(self) -> List[MarketCategory]:
    """创建默认市场分类"""
    default_categories = [
        MarketCategory(
            id="a_shares",
            name="a_shares",
            display_name="A股",
            description="中国A股市场数据源",
            enabled=True,
            sort_order=1
        ),
        MarketCategory(
            id="us_stocks",
            name="us_stocks",
            display_name="美股",
            description="美国股票市场数据源",
            enabled=True,
            sort_order=2
        ),
        MarketCategory(
            id="hk_stocks",
            name="hk_stocks",
            display_name="港股",
            description="香港股票市场数据源",
            enabled=True,
            sort_order=3
        ),
        MarketCategory(
            id="crypto",
            name="crypto",
            display_name="数字货币",
            description="数字货币市场数据源",
            enabled=True,
            sort_order=4
        ),
        MarketCategory(
            id="futures",
            name="futures",
            display_name="期货",
            description="期货市场数据源",
            enabled=True,
            sort_order=5
        )
    ]
    # 保存到数据库
    db = await self._get_db()
    categories_collection = db.market_categories
    for category in default_categories:
        await categories_collection.insert_one(category.model_dump())
    return default_categories
async def add_market_category(self, category: MarketCategory) -> bool:
    """添加市场分类"""
    try:
        db = await self._get_db()
        categories_collection = db.market_categories
        # 检查ID是否已存在
        existing = await categories_collection.find_one({"id": category.id})
        if existing:
            return False
        await categories_collection.insert_one(category.model_dump())
        return True
    except Exception as e:
        print(f"❌ 添加市场分类失败: {e}")
        return False
async def update_market_category(self, category_id: str, updates: Dict[str, Any]) -> bool:
    """更新市场分类"""
    try:
        db = await self._get_db()
        categories_collection = db.market_categories
        updates["updated_at"] = now_tz()
        result = await categories_collection.update_one(
            {"id": category_id},
            {"$set": updates}
        )
        return result.modified_count > 0
    except Exception as e:
        print(f"❌ 更新市场分类失败: {e}")
        return False
async def delete_market_category(self, category_id: str) -> bool:
    """删除市场分类"""
    try:
        db = await self._get_db()
        categories_collection = db.market_categories
        groupings_collection = db.datasource_groupings
```

```
        # 检查是否有数据源使用此分类
        groupings_count = await groupings_collection.count_documents(
            {"market_category_id": category_id}
        )
        if groupings_count > 0:
            return False

        result = await categories_collection.delete_one({"id": category_id})
        return result.deleted_count > 0
    except Exception as e:
        print(f"❌ 删除市场分类失败: {e}")
        return False

# ==================== 数据源分组管理 ====================

async def get_datasource_groupings(self) -> List[DataSourceGrouping]:
    """获取所有数据源分组关系"""
    try:
        db = await self._get_db()
        groupings_collection = db.datasource_groupings

        groupings_data = await groupings_collection.find({}).to_list(length=None)
        return [DataSourceGrouping(**data) for data in groupings_data]
    except Exception as e:
        print(f"❌ 获取数据源分组关系失败: {e}")
        return []

async def add_datasource_to_category(self, grouping: DataSourceGrouping) -> bool:
    """将数据源添加到分类"""
    try:
        db = await self._get_db()
        groupings_collection = db.datasource_groupings

        # 检查是否已存在
        existing = await groupings_collection.find_one({
            "data_source_name": grouping.data_source_name,
            "market_category_id": grouping.market_category_id
        })
        if existing:
            return False

        await groupings_collection.insert_one(grouping.model_dump())
        return True
    except Exception as e:
        print(f"❌ 添加数据源到分类失败: {e}")
        return False

async def remove_datasource_from_category(self, data_source_name: str, category_id: str) -> bool:
    """从分类中移除数据源"""
    try:
        db = await self._get_db()
        groupings_collection = db.datasource_groupings

        result = await groupings_collection.delete_one({
            "data_source_name": data_source_name,
            "market_category_id": category_id
        })
        return result.deleted_count > 0
    except Exception as e:
        print(f"❌ 从分类中移除数据源失败: {e}")
        return False

async def update_datasource_grouping(self, data_source_name: str, category_id: str, updates: Dict[str, Any]) -> bool:
    """更新数据源分组关系
    🔥 重要：同时更新 datasource_groupings 和 system_configs 两个集合
    - datasource_groupings: 用于前端展示和管理
    - system_configs.data_source_configs: 用于实际数据获取时的优先级判断
    """
    try:
        db = await self._get_db()
        groupings_collection = db.datasource_groupings
        config_collection = db.system_configs

        # 1. 更新 datasource_groupings 集合
        updates["updated_at"] = now_tz()
        result = await groupings_collection.update_one(
            {
                "data_source_name": data_source_name,
                "market_category_id": category_id
            },
            {"$set": updates}
        )

        # 2. 🔥 如果更新了优先级，同步更新 system_configs 集合
        if "priority" in updates and result.modified_count > 0:
            # 获取当前激活的配置
            config_data = await config_collection.find_one(
                {"is_active": True},
                sort=[("version", -1)]
            )
            if config_data:
                data_source_configs = config_data.get("data_source_configs", [])

                # 查找并更新对应的数据源配置
                # 注意：data_source_name 可能是 "AKShare"，而 config 中的 name 也是 "AKShare"
                # 但是 type 字段是小写的 "akshare"
                updated = False
                for ds_config in data_source_configs:
                    # 尝试匹配 name 字段（优先）或 type 字段
                    if (ds_config.get("name") == data_source_name or
                        ds_config.get("type") == data_source_name.lower()):
                        ds_config["priority"] = updates["priority"]
                        updated = True
                        logger.info(f"✅ [优先级同步] 更新 system_configs 中的数据源: {data_source_name}, 新优先级: {updates['priority']}")
                        break

                if updated:
                    # 更新配置版本
                    version = config_data.get("version", 0)
                    await config_collection.update_one(
                        {"_id": config_data["_id"]},
                        {
                            "$set": {
                                "data_source_configs": data_source_configs,
                                "version": version + 1,
                                "updated_at": now_tz()
                            }
```

```
                }
            )
            logger.info(f"☑ [优先级同步] system_configs 版本更新: {version} -> {version + 1}")
        else:
            logger.warning(f"⚠ [优先级同步] 未找到匹配的数据源配置: {data_source_name}")
    return result.modified_count > 0
except Exception as e:
    logger.error(f"✗ 更新数据源分组关系失败: {e}")
    return False
async def update_category_datasource_order(self, category_id: str, ordered_datasources: List[Dict[str, Any]]) -> bool:
    """更新分类中数据源的排序
    🔥 重要：同时更新 datasource_groupings 和 system_configs 两个集合
    - datasource_groupings: 用于前端展示和管理
    - system_configs.data_source_configs: 用于实际数据获取时的优先级判断
    """
    try:
        db = await self._get_db()
        groupings_collection = db.datasource_groupings
        config_collection = db.system_configs

        # 1. 批量更新 datasource_groupings 集合中的优先级
        for item in ordered_datasources:
            await groupings_collection.update_one(
                {
                    "data_source_name": item["name"],
                    "market_category_id": category_id
                },
                {
                    "$set": {
                        "priority": item["priority"],
                        "updated_at": now_tz()
                    }
                }
            )

        # 2. 🔥 同步更新 system_configs 集合中的 data_source_configs
        # 获取当前激活的配置
        config_data = await config_collection.find_one(
            {"is_active": True},
            sort=[("version", -1)]
        )

        if config_data:
            # 构建数据源名称到优先级的映射
            priority_map = {item["name"]: item["priority"] for item in ordered_datasources}

            # 更新 data_source_configs 中对应数据源的优先级
            data_source_configs = config_data.get("data_source_configs", [])
            updated = False

            for ds_config in data_source_configs:
                ds_name = ds_config.get("name")
                if ds_name in priority_map:
                    ds_config["priority"] = priority_map[ds_name]
                    updated = True
                    print(f"📊 [优先级同步] 更新数据源 {ds_name} 的优先级为 {priority_map[ds_name]}")

            # 如果有更新，保存回数据库
            if updated:
                await config_collection.update_one(
                    {"_id": config_data["_id"]},
                    {
                        "$set": {
                            "data_source_configs": data_source_configs,
                            "updated_at": now_tz(),
                            "version": config_data.get("version", 0) + 1
                        }
                    }
                )
                print(f"☑ [优先级同步] 已同步更新 system_configs 集合，新版本: {config_data.get('version', 0) + 1}")
            else:
                print(f"⚠ [优先级同步] 没有找到需要更新的数据源配置")
        else:
            print(f"⚠ [优先级同步] 未找到激活的系统配置")

        return True
    except Exception as e:
        print(f"✗ 更新分类数据源排序失败: {e}")
        import traceback
        traceback.print_exc()
        return False
async def get_system_config(self) -> Optional[SystemConfig]:
    """获取系统配置 - 优先从数据库获取最新数据"""
    try:
        # 直接从数据库获取最新配置，避免缓存问题
        db = await self._get_db()
        config_collection = db.system_configs

        config_data = await config_collection.find_one(
            {"is_active": True},
            sort=[("version", -1)]
        )

        if config_data:
            print(f"📊 从数据库获取配置，版本: {config_data.get('version', 0)}, LLM配置数量: {len(config_data.get('llm_configs', []))}")
            return SystemConfig(**config_data)

        # 如果没有配置，创建默认配置
        print("⚠ 数据库中没有配置，创建默认配置")
        return await self._create_default_config()

    except Exception as e:
        print(f"✗ 从数据库获取配置失败: {e}")
        # 作为最后的回退，尝试从统一配置管理器获取
        try:
            unified_system_config = await unified_config.get_unified_system_config()
            if unified_system_config:
                print("🔄 回退到统一配置管理器")
                return unified_system_config
        except Exception as e2:
            print(f"从统一配置获取也失败: {e2}")

        return None

async def _create_default_config(self) -> SystemConfig:
```

```
"""创建默认系统配置"""
default_config = SystemConfig(
    config_name="默认配置",
    config_type="system",
    llm_configs=[
        LLMConfig(
            provider=ModelProvider.OPENAI,
            model_name="gpt-3.5-turbo",
            api_key="your-openai-api-key",
            api_base="https://api.openai.com/v1",
            max_tokens=4000,
            temperature=0.7,
            enabled=False,
            description="OpenAI GPT-3.5 Turbo模型"
        ),
        LLMConfig(
            provider=ModelProvider.ZHIPU,
            model_name="glm-4",
            api_key="your-zhipu-api-key",
            api_base="https://open.bigmodel.cn/api/paas/v4",
            max_tokens=4000,
            temperature=0.7,
            enabled=True,
            description="智谱AI GLM-4模型（推荐）"
        ),
        LLMConfig(
            provider=ModelProvider.QWEN,
            model_name="qwen-turbo",
            api_key="your-qwen-api-key",
            api_base="https://dashscope.aliyuncs.com/compatible-mode/v1",
            max_tokens=4000,
            temperature=0.7,
            enabled=False,
            description="阿里云通义千问模型"
        )
    ],
    default_llm="glm-4",
    data_source_configs=[
        DataSourceConfig(
            name="AKShare",
            type=DataSourceType.AKSHARE,
            endpoint="https://akshare.akfamily.xyz",
            timeout=30,
            rate_limit=100,
            enabled=True,
            priority=1,
            description="AKShare开源金融数据接口"
        ),
        DataSourceConfig(
            name="Tushare",
            type=DataSourceType.TUSHARE,
            api_key="your-tushare-token",
            endpoint="http://api.tushare.pro",
            timeout=30,
            rate_limit=200,
            enabled=False,
            priority=2,
            description="Tushare专业金融数据接口"
        )
    ],
    default_data_source="AKShare",
    database_configs=[
        DatabaseConfig(
            name="MongoDB主库",
            type=DatabaseType.MONGODB,
            host="localhost",
            port=27017,
            database="tradingagents",
            enabled=True,
            description="MongoDB主数据库"
        ),
        DatabaseConfig(
            name="Redis缓存",
            type=DatabaseType.REDIS,
            host="localhost",
            port=6379,
            database="0",
            enabled=True,
            description="Redis缓存数据库"
        )
    ],
    system_settings={
        "max_concurrent_tasks": 3,
        "default_analysis_timeout": 300,
        "enable_cache": True,
        "cache_ttl": 3600,
        "log_level": "INFO",
        "enable_monitoring": True,
        # Worker/Queue intervals
        "worker_heartbeat_interval_seconds": 30,
        "queue_poll_interval_seconds": 1.0,
        "queue_cleanup_interval_seconds": 60.0,
        # SSE intervals
        "sse_poll_timeout_seconds": 1.0,
        "sse_heartbeat_interval_seconds": 10,
        "sse_task_max_idle_seconds": 300,
        "sse_batch_poll_interval_seconds": 2.0,
        "sse_batch_max_idle_seconds": 600,
        # TradingAgents runtime intervals (optional; DB-managed)
        "ta_hk_min_request_interval_seconds": 2.0,
        "ta_hk_timeout_seconds": 60,
        "ta_hk_max_retries": 3,
        "ta_hk_rate_limit_wait_seconds": 60,
        "ta_hk_cache_ttl_seconds": 86400,
        # 新增：TradingAgents 数据来源策略
        # 是否优先从 app 缓存(Mongo 集合 stock_basic_info / market_quotes) 读取
```

```
        "ta_use_app_cache": False,
        "ta_china_min_api_interval_seconds": 0.5,
        "ta_us_min_api_interval_seconds": 1.0,
        "ta_google_news_sleep_min_seconds": 2.0,
        "ta_google_news_sleep_max_seconds": 6.0,
        "app_timezone": "Asia/Shanghai"
    }
  )

  # 保存到数据库
  await self.save_system_config(default_config)
  return default_config

async def save_system_config(self, config: SystemConfig) -> bool:
  """保存系统配置到数据库"""
  try:
    print(f"💾 开始保存配置，LLM配置数量: {len(config.llm_configs)}")
    # 保存到数据库
    db = await self._get_db()
    config_collection = db.system_configs
    # 更新时间戳和版本
    config.updated_at = now_tz()
    config.version += 1
    # 将当前激活的配置设为非激活
    update_result = await config_collection.update_many(
      {"is_active": True},
      {"$set": {"is_active": False}}
    )
    print(f"📝 禁用旧配置数量: {update_result.modified_count}")
    # 插入新配置 - 移除_id字段让MongoDB自动生成新的
    config_dict = config.model_dump(by_alias=True)
    if '_id' in config_dict:
      del config_dict['_id']  # 移除旧的_id，让MongoDB生成新的
    # 打印即将保存的 system_settings
    system_settings = config_dict.get('system_settings', {})
    print(f"📝 即将保存的 system_settings 包含 {len(system_settings)} 项")
    if 'quick_analysis_model' in system_settings:
      print(f"  ✓ 包含 quick_analysis_model: {system_settings['quick_analysis_model']}")
    else:
      print(f"  ⚠️ 不包含 quick_analysis_model")
    if 'deep_analysis_model' in system_settings:
      print(f"  ✓ 包含 deep_analysis_model: {system_settings['deep_analysis_model']}")
    else:
      print(f"  ⚠️ 不包含 deep_analysis_model")
    insert_result = await config_collection.insert_one(config_dict)
    print(f"📝 新配置ID: {insert_result.inserted_id}")
    # 验证保存结果
    saved_config = await config_collection.find_one({"_id": insert_result.inserted_id})
    if saved_config:
      print(f"✅ 配置保存成功，验证LLM配置数量: {len(saved_config.get('llm_configs', []))}")
      # 暂时跳过统一配置同步，避免冲突
      # unified_config.sync_to_legacy_format(config)
      return True
    else:
      print("❌ 配置保存验证失败")
      return False
  except Exception as e:
    print(f"❌ 保存配置失败: {e}")
    import traceback
    traceback.print_exc()
    return False
async def delete_llm_config(self, provider: str, model_name: str) -> bool:
  """删除大模型配置"""
  try:
    print(f"🗑️ 删除大模型配置 - provider: {provider}, model_name: {model_name}")
    config = await self.get_system_config()
    if not config:
      print("❌ 系统配置为空")
      return False
    print(f"📊 当前大模型配置数量: {len(config.llm_configs)}")
    # 打印所有现有配置
    for i, llm in enumerate(config.llm_configs):
      print(f"  {i+1}. provider: {llm.provider.value}, model_name: {llm.model_name}")
    # 查找并删除指定的LLM配置
    original_count = len(config.llm_configs)
    # 使用更宽松的匹配条件
    config.llm_configs = [
      llm for llm in config.llm_configs
      if not (str(llm.provider.value).lower() == provider.lower() and llm.model_name == model_name)
    ]
    new_count = len(config.llm_configs)
    print(f"🔄 删除后配置数量: {new_count} (原来: {original_count})")
    if new_count == original_count:
      print(f"❌ 没有找到匹配的配置: {provider}/{model_name}")
      return False  # 没有找到要删除的配置
    # 保存更新后的配置
    save_result = await self.save_system_config(config)
    print(f"💾 保存结果: {save_result}")
    return save_result
  except Exception as e:
    print(f"❌ 删除LLM配置失败: {e}")
    import traceback
    traceback.print_exc()
    return False
async def set_default_llm(self, model_name: str) -> bool:
  """设置默认大模型"""
  try:
    config = await self.get_system_config()
    if not config:
      return False
    # 检查指定的模型是否存在
    model_exists = any(
```

```
            llm.model_name == model_name for llm in config.llm_configs
        )
        if not model_exists:
            return False
        config.default_llm = model_name
        return await self.save_system_config(config)
    except Exception as e:
        print(f"设置默认LLM失败: {e}")
        return False
async def set_default_data_source(self, data_source_name: str) -> bool:
    """设置默认数据源"""
    try:
        config = await self.get_system_config()
        if not config:
            return False
        # 检查指定的数据源是否存在
        source_exists = any(
            ds.name == data_source_name for ds in config.data_source_configs
        )
        if not source_exists:
            return False
        config.default_data_source = data_source_name
        return await self.save_system_config(config)
    except Exception as e:
        print(f"设置默认数据源失败: {e}")
        return False
async def update_system_settings(self, settings: Dict[str, Any]) -> bool:
    """更新系统设置"""
    try:
        config = await self.get_system_config()
        if not config:
            return False
        # 打印更新前的系统设置
        print(f"📝 更新前 system_settings 包含 {len(config.system_settings)} 项")
        if 'quick_analysis_model' in config.system_settings:
            print(f"  ✓ 更新前包含 quick_analysis_model: {config.system_settings['quick_analysis_model']}")
        else:
            print(f"  ⚠️  更新前不包含 quick_analysis_model")
        # 更新系统设置
        config.system_settings.update(settings)
        # 打印更新后的系统设置
        print(f"📝 更新后 system_settings 包含 {len(config.system_settings)} 项")
        if 'quick_analysis_model' in config.system_settings:
            print(f"  ✓ 更新后包含 quick_analysis_model: {config.system_settings['quick_analysis_model']}")
        else:
            print(f"  ⚠️  更新后不包含 quick_analysis_model")
        if 'deep_analysis_model' in config.system_settings:
            print(f"  ✓ 更新后包含 deep_analysis_model: {config.system_settings['deep_analysis_model']}")
        else:
            print(f"  ⚠️  更新后不包含 deep_analysis_model")
        result = await self.save_system_config(config)
        # 同步到文件系统（供 unified_config 使用）
        if result:
            try:
                from app.core.unified_config import unified_config
                unified_config.sync_to_legacy_format(config)
                print(f"✅ 系统设置已同步到文件系统")
            except Exception as e:
                print(f"⚠️  同步系统设置到文件系统失败: {e}")
        return result
    except Exception as e:
        print(f"更新系统设置失败: {e}")
        return False
async def get_system_settings(self) -> Dict[str, Any]:
    """获取系统设置"""
    try:
        config = await self.get_system_config()
        if not config:
            return {}
        return config.system_settings
    except Exception as e:
        print(f"获取系统设置失败: {e}")
        return {}
async def export_config(self) -> Dict[str, Any]:
    """导出配置"""
    try:
        config = await self.get_system_config()
        if not config:
            return {}
        # 转换为可序列化的字典格式
        # 方案A：导出时对敏感字段脱敏/清空
        def _llm_sanitize(x: LLMConfig):
            d = x.model_dump()
            d["api_key"] = ""
            # 确保必填字段有默认值（防止导出 None 或空字符串）
            # 注意：max_tokens 在 system_configs 中已经有正确的值，直接使用
            if not d.get("max_tokens") or d.get("max_tokens") == "":
                d["max_tokens"] = 4000
            if not d.get("temperature") and d.get("temperature") != 0:
                d["temperature"] = 0.7
            if not d.get("timeout") or d.get("timeout") == "":
                d["timeout"] = 180
            if not d.get("retry_times") or d.get("retry_times") == "":
                d["retry_times"] = 3
            return d
        def _ds_sanitize(x: DataSourceConfig):
            d = x.model_dump()
            d["api_key"] = ""
            d["api_secret"] = ""
            return d
        def _db_sanitize(x: DatabaseConfig):
            d = x.model_dump()
            d["password"] = ""
```

```
            return d
        export_data = {
            "config_name": config.config_name,
            "config_type": config.config_type,
            "llm_configs": [_llm_sanitize(llm) for llm in config.llm_configs],
            "default_llm": config.default_llm,
            "data_source_configs": [_ds_sanitize(ds) for ds in config.data_source_configs],
            "default_data_source": config.default_data_source,
            "database_configs": [_db_sanitize(db) for db in config.database_configs],
            # 方案A：导出时对 system_settings 中的敏感键做脱敏
            "system_settings": {k: (None if any(p in k.lower() for p in ("key","secret","password","token","client_secret")) else v) for k, v in (config.system_settings or {}).items()},
            "exported_at": now_tz().isoformat(),
            "version": config.version
        }
        return export_data
    except Exception as e:
        print(f"导出配置失败: {e}")
        return {}
async def import_config(self, config_data: Dict[str, Any]) -> bool:
    """导入配置"""
    try:
        # 验证配置数据格式
        if not self._validate_config_data(config_data):
            return False
        # 创建新的系统配置（方案A：导入时忽略敏感字段）
        def _llm_sanitize_in(llm: Dict[str, Any]):
            d = dict(llm or {})
            d.pop("api_key", None)
            d["api_key"] = ""
            # 清理空字符串，让 Pydantic 使用默认值
            if d.get("max_tokens") == "" or d.get("max_tokens") is None:
                d.pop("max_tokens", None)
            if d.get("temperature") == "" or d.get("temperature") is None:
                d.pop("temperature", None)
            if d.get("timeout") == "" or d.get("timeout") is None:
                d.pop("timeout", None)
            if d.get("retry_times") == "" or d.get("retry_times") is None:
                d.pop("retry_times", None)
            return LLMConfig(**d)
        def _ds_sanitize_in(ds: Dict[str, Any]):
            d = dict(ds or {})
            d.pop("api_key", None)
            d.pop("api_secret", None)
            d["api_key"] = ""
            d["api_secret"] = ""
            return DataSourceConfig(**d)
        def _db_sanitize_in(db: Dict[str, Any]):
            d = dict(db or {})
            d.pop("password", None)
            d["password"] = ""
            return DatabaseConfig(**d)
        new_config = SystemConfig(
            config_name=config_data.get("config_name", "导入的配置"),
            config_type="imported",
            llm_configs=[_llm_sanitize_in(llm) for llm in config_data.get("llm_configs", [])],
            default_llm=config_data.get("default_llm"),
            data_source_configs=[_ds_sanitize_in(ds) for ds in config_data.get("data_source_configs", [])],
            default_data_source=config_data.get("default_data_source"),
            database_configs=[_db_sanitize_in(db) for db in config_data.get("database_configs", [])],
            system_settings=config_data.get("system_settings", {})
        )
        return await self.save_system_config(new_config)
    except Exception as e:
        print(f"导入配置失败: {e}")
        return False
def _validate_config_data(self, config_data: Dict[str, Any]) -> bool:
    """验证配置数据格式"""
    try:
        required_fields = ["llm_configs", "data_source_configs", "database_configs", "system_settings"]
        for field in required_fields:
            if field not in config_data:
                print(f"配置数据缺少必需字段: {field}")
                return False
        return True
    except Exception as e:
        print(f"验证配置数据失败: {e}")
        return False
async def migrate_legacy_config(self) -> bool:
    """迁移传统配置"""
    try:
        # 这里可以调用迁移脚本的逻辑
        # 或者直接在这里实现迁移逻辑
        from scripts.migrate_config_to_webapi import ConfigMigrator
        migrator = ConfigMigrator()
        return await migrator.migrate_all_configs()
    except Exception as e:
        print(f"迁移传统配置失败: {e}")
        return False

async def update_llm_config(self, llm_config: LLMConfig) -> bool:
    """更新大模型配置"""
    try:
        # 直接保存到统一配置管理器
        success = unified_config.save_llm_config(llm_config)
        if not success:
            return False
        # 同时更新数据库配置
        config = await self.get_system_config()
        if not config:
            return False
        # 查找并更新对应的LLM配置
        for i, existing_config in enumerate(config.llm_configs):
            if existing_config.model_name == llm_config.model_name:
                config.llm_configs[i] = llm_config
```

```
            break
        else:
            # 如果不存在，添加新配置
            config.llm_configs.append(llm_config)
        return await self.save_system_config(config)
    except Exception as e:
        print(f"更新LLM配置失败: {e}")
        return False

async def test_llm_config(self, llm_config: LLMConfig) -> Dict[str, Any]:
    """测试大模型配置 - 真实调用API进行验证"""
    start_time = time.time()
    try:
        import requests
        # 获取 provider 字符串值（兼容枚举和字符串）
        provider_str = llm_config.provider.value if hasattr(llm_config.provider, 'value') else str(llm_config.provider)
        logger.info(f"🔧 测试大模型配置: {provider_str} - {llm_config.model_name}")
        logger.info(f"📍 API基础URL (模型配置): {llm_config.api_base}")
        # 获取厂家配置（用于获取 API Key 和 default_base_url）
        db = await self._get_db()
        providers_collection = db.llm_providers
        provider_data = await providers_collection.find_one({"name": provider_str})
        # 1. 确定 API 基础 URL
        api_base = llm_config.api_base
        if not api_base:
            # 如果模型配置没有 api_base，从厂家配置获取 default_base_url
            if provider_data and provider_data.get("default_base_url"):
                api_base = provider_data["default_base_url"]
                logger.info(f"✅ 从厂家配置获取 API 基础 URL: {api_base}")
            else:
                return {
                    "success": False,
                    "message": f"模型配置和厂家配置都未设置 API 基础 URL",
                    "response_time": time.time() - start_time,
                    "details": None
                }
        # 2. 验证 API Key
        api_key = None
        if llm_config.api_key:
            api_key = llm_config.api_key
        else:
            # 从厂家配置获取 API Key
            if provider_data and provider_data.get("api_key"):
                api_key = provider_data["api_key"]
                logger.info(f"✅ 从厂家配置获取到API密钥")
            else:
                # 尝试从环境变量获取
                api_key = self._get_env_api_key(provider_str)
                if api_key:
                    logger.info(f"✅ 从环境变量获取到API密钥")
        if not api_key or not self._is_valid_api_key(api_key):
            return {
                "success": False,
                "message": f"{provider_str} 未配置有效的API密钥",
                "response_time": time.time() - start_time,
                "details": None
            }
        # 3. 根据厂家类型选择测试方法
        if provider_str == "google":
            # Google AI 使用专门的测试方法
            logger.info(f"🔍 使用 Google AI 专用测试方法")
            result = self._test_google_api(api_key, f"{provider_str} {llm_config.model_name}", api_base, llm_config.model_name)
            result["response_time"] = time.time() - start_time
            return result
        elif provider_str == "deepseek":
            # DeepSeek 使用专门的测试方法
            logger.info(f"🔍 使用 DeepSeek 专用测试方法")
            result = self._test_deepseek_api(api_key, f"{provider_str} {llm_config.model_name}", llm_config.model_name)
            result["response_time"] = time.time() - start_time
            return result
        elif provider_str == "dashscope":
            # DashScope 使用专门的测试方法
            logger.info(f"🔍 使用 DashScope 专用测试方法")
            result = self._test_dashscope_api(api_key, f"{provider_str} {llm_config.model_name}", llm_config.model_name)
            result["response_time"] = time.time() - start_time
            return result
        else:
            # 其他厂家使用 OpenAI 兼容的测试方法
            logger.info(f"🔍 使用 OpenAI 兼容测试方法")
            # 构建测试请求
            api_base_normalized = api_base.rstrip("/")
            # 🔧 智能版本号处理：只有在没有版本号的情况下才添加 /v1
            # 避免对已有版本号的URL（如智谱AI的 /v4）重复添加 /v1
            import re
            if not re.search(r'/v\d+$', api_base_normalized):
                # URL末尾没有版本号，添加 /v1（OpenAI标准）
                api_base_normalized = api_base_normalized + "/v1"
                logger.info(f"🔧 添加 /v1 版本号: {api_base_normalized}")
            else:
                # URL已包含版本号（如 /v4），不添加
                logger.info(f"✅ 检测到已有版本号，保持原样: {api_base_normalized}")
            url = f"{api_base_normalized}/chat/completions"
            headers = {
                "Content-Type": "application/json",
                "Authorization": f"Bearer {api_key}"
            }
            data = {
                "model": llm_config.model_name,
                "messages": [
                    {"role": "user", "content": "Hello, please respond with 'OK' if you can read this."}
                ],
                "max_tokens": 200,  # 增加到200，给推理模型（如o1/gpt-5）足够空间
                "temperature": 0.1
            }
```

```
        logger.info(f"🌐 发送测试请求到: {url}")
        logger.info(f"📦 使用模型: {llm_config.model_name}")
        logger.info(f"📦 请求数据: {data}")
        # 发送测试请求
        response = requests.post(url, json=data, headers=headers, timeout=15)
        response_time = time.time() - start_time
        logger.info(f"📡 收到响应: HTTP {response.status_code}")
        # 处理响应（仅用于 OpenAI 兼容的厂家）
        if response.status_code == 200:
            try:
                result = response.json()
                logger.info(f"📦 响应JSON: {result}")
                if "choices" in result and len(result["choices"]) > 0:
                    content = result["choices"][0]["message"]["content"]
                    logger.info(f"📝 响应内容: {content}")
                    if content and len(content.strip()) > 0:
                        logger.info(f"✅ 测试成功: {content[:50]}")
                        return {
                            "success": True,
                            "message": f"成功连接到 {provider_str} {llm_config.model_name}",
                            "response_time": response_time,
                            "details": {
                                "provider": provider_str,
                                "model": llm_config.model_name,
                                "api_base": api_base,
                                "response_preview": content[:100]
                            }
                        }
                    else:
                        logger.warning(f"⚠️ API响应内容为空")
                        return {
                            "success": False,
                            "message": "API响应内容为空",
                            "response_time": response_time,
                            "details": None
                        }
                else:
                    logger.warning(f"⚠️ API响应格式异常，缺少 choices 字段")
                    logger.warning(f"📦 响应内容: {result}")
                    return {
                        "success": False,
                        "message": "API响应格式异常",
                        "response_time": response_time,
                        "details": None
                    }
            except Exception as e:
                logger.error(f"❌ 解析响应失败: {e}")
                logger.error(f"📦 响应文本: {response.text[:500]}")
                return {
                    "success": False,
                    "message": f"解析响应失败: {str(e)}",
                    "response_time": response_time,
                    "details": None
                }
        elif response.status_code == 401:
            return {
                "success": False,
                "message": "API密钥无效或已过期",
                "response_time": response_time,
                "details": None
            }
        elif response.status_code == 403:
            return {
                "success": False,
                "message": "API权限不足或配额已用完",
                "response_time": response_time,
                "details": None
            }
        elif response.status_code == 404:
            return {
                "success": False,
                "message": f"API端点不存在，请检查API基础URL是否正确: {url}",
                "response_time": response_time,
                "details": None
            }
        else:
            try:
                error_detail = response.json()
                error_msg = error_detail.get("error", {}).get("message", f"HTTP {response.status_code}")
                return {
                    "success": False,
                    "message": f"API测试失败: {error_msg}",
                    "response_time": response_time,
                    "details": None
                }
            except:
                return {
                    "success": False,
                    "message": f"API测试失败: HTTP {response.status_code}",
                    "response_time": response_time,
                    "details": None
                }
    except requests.exceptions.Timeout:
        response_time = time.time() - start_time
        return {
            "success": False,
            "message": "连接超时，请检查API基础URL是否正确或网络是否可达",
            "response_time": response_time,
            "details": None
        }
    except requests.exceptions.ConnectionError as e:
        response_time = time.time() - start_time
        return {
            "success": False,
            "message": f"连接失败，请检查API基础URL是否正确: {str(e)}",
```

```
            "response_time": response_time,
            "details": None
        }
    except Exception as e:
        response_time = time.time() - start_time
        logger.error(f"❌ 测试大模型配置失败: {e}")
        return {
            "success": False,
            "message": f"连接失败: {str(e)}",
            "response_time": response_time,
            "details": None
        }

def _truncate_api_key(self, api_key: str, prefix_len: int = 6, suffix_len: int = 6) -> str:
    """
    截断 API Key 用于显示
    Args:
        api_key: 完整的 API Key
        prefix_len: 保留前缀长度
        suffix_len: 保留后缀长度
    Returns:
        截断后的 API Key，例如：0f229a...c550ec
    """
    if not api_key or len(api_key) <= prefix_len + suffix_len:
        return api_key
    return f"{api_key[:prefix_len]}...{api_key[-suffix_len:]}"
async def test_data_source_config(self, ds_config: DataSourceConfig) -> Dict[str, Any]:
    """测试数据源配置 - 真实调用API进行验证"""
    start_time = time.time()
    try:
        import requests
        import os
        ds_type = ds_config.type.value if hasattr(ds_config.type, 'value') else str(ds_config.type)
        logger.info(f"🔧 [TEST] Testing data source config: {ds_config.name} ({ds_type})")
        # 🔥 优先使用配置中的 API Key，如果没有或被截断，则从数据库获取
        api_key = ds_config.api_key
        used_db_credentials = False
        used_env_credentials = False
        logger.info(f"🔍 [TEST] Received API Key from config: {repr(api_key)} (type: {type(api_key).__name__}, length: {len(api_key) if api_key else 0})")
        # 根据不同的数据源类型进行测试
        if ds_type == "tushare":
            # 🔥 如果配置中的 API Key 包含 "..."（截断标记），需要验证是否是未修改的原值
            if api_key and "..." in api_key:
                logger.info(f"🔍 [TEST] API Key contains '...' (truncated), checking if it matches database value")
                # 从数据库中获取完整的 API Key
                system_config = await self.get_system_config()
                db_config = None
                if system_config:
                    for ds in system_config.data_source_configs:
                        if ds.name == ds_config.name:
                            db_config = ds
                            break
                if db_config and db_config.api_key:
                    # 对数据库中的完整 API Key 进行相同的截断处理
                    truncated_db_key = self._truncate_api_key(db_config.api_key)
                    logger.info(f"🔍 [TEST] Database API Key truncated: {truncated_db_key}")
                    logger.info(f"🔍 [TEST] Received API Key: {api_key}")
                    # 比较截断后的值
                    if api_key == truncated_db_key:
                        # 相同，说明用户没有修改，使用数据库中的完整值
                        api_key = db_config.api_key
                        used_db_credentials = True
                        logger.info(f"✅ [TEST] Truncated values match, using complete API Key from database (length: {len(api_key)})")
                    else:
                        # 不同，说明用户修改了但修改得不完整
                        logger.error(f"❌ [TEST] Truncated API Key doesn't match database value, user may have modified it incorrectly")
                        return {
                            "success": False,
                            "message": "API Key 格式错误：检测到截断标记但与数据库中的值不匹配，请输入完整的 API Key",
                            "response_time": time.time() - start_time,
                            "details": {
                                "error": "truncated_key_mismatch",
                                "received": api_key,
                                "expected": truncated_db_key
                            }
                        }
                else:
                    # 数据库中没有有效的 API Key，尝试从环境变量获取
                    logger.info(f"⚠️ [TEST] No valid API Key in database, trying environment variable")
                    env_token = os.getenv('TUSHARE_TOKEN')
                    if env_token:
                        api_key = env_token.strip().strip('"').strip("'")
                        used_env_credentials = True
                        logger.info(f"🔑 [TEST] Using TUSHARE_TOKEN from environment (length: {len(api_key)})")
                    else:
                        logger.error(f"❌ [TEST] No valid API Key in database or environment")
                        return {
                            "success": False,
                            "message": "API Key 无效：数据库和环境变量中均未配置有效的 Token",
                            "response_time": time.time() - start_time,
                            "details": None
                        }
            # 如果 API Key 为空，尝试从数据库或环境变量获取
            elif not api_key:
                logger.info(f"⚠️ [TEST] API Key is empty, trying to get from database")
                # 从数据库中获取完整的 API Key
                system_config = await self.get_system_config()
                db_config = None
                if system_config:
                    for ds in system_config.data_source_configs:
                        if ds.name == ds_config.name:
                            db_config = ds
                            break
```

```
        if db_config and db_config.api_key and "..." not in db_config.api_key:
            api_key = db_config.api_key
            used_db_credentials = True
            logger.info(f"🔗 [TEST] Using API Key from database (length: {len(api_key)})")
        else:
            # 如果数据库中也没有，尝试从环境变量获取
            logger.info(f"⚠️ [TEST] No valid API Key in database, trying environment variable")
            env_token = os.getenv('TUSHARE_TOKEN')
            if env_token:
                api_key = env_token.strip().strip('"').strip("'")
                used_env_credentials = True
                logger.info(f"🔗 [TEST] Using TUSHARE_TOKEN from environment (length: {len(api_key)})")
            else:
                logger.error(f"❌ [TEST] No valid API Key in config, database, or environment")
                return {
                    "success": False,
                    "message": "API Key 无效：配置、数据库和环境变量中均未配置有效的 Token",
                    "response_time": time.time() - start_time,
                    "details": None
                }
    else:
        # API Key 是完整的，直接使用
        logger.info(f"✅ [TEST] Using complete API Key from config (length: {len(api_key)})")
    # 测试 Tushare API
    try:
        logger.info(f"🧪 [TEST] Calling Tushare API with token (length: {len(api_key)})")
        import tushare as ts
        ts.set_token(api_key)
        pro = ts.pro_api()
        # 获取交易日历（轻量级测试）
        df = pro.trade_cal(exchange='SSE', start_date='20240101', end_date='20240101')
        if df is not None and len(df) > 0:
            response_time = time.time() - start_time
            logger.info(f"✅ [TEST] Tushare API call successful (response time: {response_time:.2f}s)")
            # 构建消息，说明使用了哪个来源的凭证
            credential_source = "配置"
            if used_db_credentials:
                credential_source = "数据库"
            elif used_env_credentials:
                credential_source = "环境变量"
            return {
                "success": True,
                "message": f"成功连接到 Tushare 数据源（使用{credential_source}中的凭证）",
                "response_time": response_time,
                "details": {
                    "type": ds_type,
                    "test_result": "获取交易日历成功",
                    "credential_source": credential_source,
                    "used_db_credentials": used_db_credentials,
                    "used_env_credentials": used_env_credentials
                }
            }
        else:
            logger.error(f"❌ [TEST] Tushare API returned empty data")
            return {
                "success": False,
                "message": "Tushare API 返回数据为空",
                "response_time": time.time() - start_time,
                "details": None
            }
    except ImportError:
        logger.error(f"❌ [TEST] Tushare library not installed")
        return {
            "success": False,
            "message": "Tushare 库未安装，请运行: pip install tushare",
            "response_time": time.time() - start_time,
            "details": None
        }
    except Exception as e:
        logger.error(f"❌ [TEST] Tushare API call failed: {e}")
        return {
            "success": False,
            "message": f"Tushare API 调用失败: {str(e)}",
            "response_time": time.time() - start_time,
            "details": None
        }
elif ds_type == "akshare":
    # AKShare 不需要 API Key，直接测试
    try:
        import akshare as ak
        # 使用更轻量级的接口测试 - 获取交易日历
        # 这个接口数据量小，响应快，更适合测试连接
        df = ak.tool_trade_date_hist_sina()
        if df is not None and len(df) > 0:
            response_time = time.time() - start_time
            return {
                "success": True,
                "message": f"成功连接到 AKShare 数据源",
                "response_time": response_time,
                "details": {
                    "type": ds_type,
                    "test_result": f"获取交易日历成功（{len(df)} 条记录）"
                }
            }
        else:
            return {
                "success": False,
                "message": "AKShare API 返回数据为空",
                "response_time": time.time() - start_time,
                "details": None
            }
    except ImportError:
        return {
            "success": False,
            "message": "AKShare 库未安装，请运行: pip install akshare",
```

```
            "response_time": time.time() - start_time,
            "details": None
        }
    except Exception as e:
        return {
            "success": False,
            "message": f"AKShare API 调用失败: {str(e)}",
            "response_time": time.time() - start_time,
            "details": None
        }
elif ds_type == "baostock":
    # BaoStock 不需要 API Key，直接测试登录
    try:
        import baostock as bs
        # 测试登录
        lg = bs.login()
        if lg.error_code == '0':
            # 登录成功，测试获取数据
            try:
                # 获取交易日历（轻量级测试）
                rs = bs.query_trade_dates(start_date="2024-01-01", end_date="2024-01-01")
                if rs.error_code == '0':
                    response_time = time.time() - start_time
                    bs.logout()
                    return {
                        "success": True,
                        "message": f"成功连接到 BaoStock 数据源",
                        "response_time": response_time,
                        "details": {
                            "type": ds_type,
                            "test_result": "登录成功，获取交易日历成功"
                        }
                    }
                else:
                    bs.logout()
                    return {
                        "success": False,
                        "message": f"BaoStock 数据获取失败: {rs.error_msg}",
                        "response_time": time.time() - start_time,
                        "details": None
                    }
            except Exception as e:
                bs.logout()
                return {
                    "success": False,
                    "message": f"BaoStock 数据获取异常: {str(e)}",
                    "response_time": time.time() - start_time,
                    "details": None
                }
        else:
            return {
                "success": False,
                "message": f"BaoStock 登录失败: {lg.error_msg}",
                "response_time": time.time() - start_time,
                "details": None
            }
    except ImportError:
        return {
            "success": False,
            "message": "BaoStock 库未安装，请运行: pip install baostock",
            "response_time": time.time() - start_time,
            "details": None
        }
    except Exception as e:
        return {
            "success": False,
            "message": f"BaoStock API 调用失败: {str(e)}",
            "response_time": time.time() - start_time,
            "details": None
        }
elif ds_type == "yahoo_finance":
    # Yahoo Finance 测试
    if not ds_config.endpoint:
        ds_config.endpoint = "https://query1.finance.yahoo.com"
    try:
        url = f"{ds_config.endpoint}/v8/finance/chart/AAPL"
        params = {"interval": "1d", "range": "1d"}
        response = requests.get(url, params=params, timeout=10)
        if response.status_code == 200:
            data = response.json()
            if "chart" in data and "result" in data["chart"]:
                response_time = time.time() - start_time
                return {
                    "success": True,
                    "message": f"成功连接到 Yahoo Finance 数据源",
                    "response_time": response_time,
                    "details": {
                        "type": ds_type,
                        "endpoint": ds_config.endpoint,
                        "test_result": "获取 AAPL 数据成功"
                    }
                }
        return {
            "success": False,
            "message": f"Yahoo Finance API 返回错误: HTTP {response.status_code}",
            "response_time": time.time() - start_time,
            "details": None
        }
    except Exception as e:
        return {
            "success": False,
            "message": f"Yahoo Finance API 调用失败: {str(e)}",
            "response_time": time.time() - start_time,
            "details": None
        }
```

```
elif ds_type == "alpha_vantage":
    # 🔥 如果配置中的 API Key 包含 "..."（截断标记），需要验证是否是未修改的原值
    if api_key and "..." in api_key:
        logger.info(f"🔍 [TEST] API Key contains '...' (truncated), checking if it matches database value")
        # 从数据库中获取完整的 API Key
        system_config = await self.get_system_config()
        db_config = None
        if system_config:
            for ds in system_config.data_source_configs:
                if ds.name == ds_config.name:
                    db_config = ds
                    break
        if db_config and db_config.api_key:
            # 对数据库中的完整 API Key 进行相同的截断处理
            truncated_db_key = self._truncate_api_key(db_config.api_key)
            logger.info(f"🔍 [TEST] Database API Key truncated: {truncated_db_key}")
            logger.info(f"🔍 [TEST] Received API Key: {api_key}")
            # 比较截断后的值
            if api_key == truncated_db_key:
                # 相同，说明用户没有修改，使用数据库中的完整值
                api_key = db_config.api_key
                used_db_credentials = True
                logger.info(f"✅ [TEST] Truncated values match, using complete API Key from database (length: {len(api_key)})")
            else:
                # 不同，说明用户修改了但修改得不完整
                logger.error(f"❌ [TEST] Truncated API Key doesn't match database value")
                return {
                    "success": False,
                    "message": "API Key 格式错误：检测到截断标记但与数据库中的值不匹配，请输入完整的 API Key",
                    "response_time": time.time() - start_time,
                    "details": {
                        "error": "truncated_key_mismatch",
                        "received": api_key,
                        "expected": truncated_db_key
                    }
                }
        else:
            # 数据库中没有有效的 API Key，尝试从环境变量获取
            logger.info(f"⚠️ [TEST] No valid API Key in database, trying environment variable")
            env_key = os.getenv('ALPHA_VANTAGE_API_KEY')
            if env_key:
                api_key = env_key.strip().strip('"').strip("'")
                used_env_credentials = True
                logger.info(f"🔑 [TEST] Using ALPHA_VANTAGE_API_KEY from environment (length: {len(api_key)})")
            else:
                logger.error(f"❌ [TEST] No valid API Key in database or environment")
                return {
                    "success": False,
                    "message": "API Key 无效：数据库和环境变量中均未配置有效的 API Key",
                    "response_time": time.time() - start_time,
                    "details": None
                }
    # 如果 API Key 为空，尝试从数据库或环境变量获取
    elif not api_key:
        logger.info(f"⚠️ [TEST] API Key is empty, trying to get from database")
        # 从数据库中获取完整的 API Key
        system_config = await self.get_system_config()
        db_config = None
        if system_config:
            for ds in system_config.data_source_configs:
                if ds.name == ds_config.name:
                    db_config = ds
                    break
        if db_config and db_config.api_key and "..." not in db_config.api_key:
            api_key = db_config.api_key
            used_db_credentials = True
            logger.info(f"🔑 [TEST] Using API Key from database (length: {len(api_key)})")
        else:
            # 如果数据库中也没有，尝试从环境变量获取
            logger.info(f"⚠️ [TEST] No valid API Key in database, trying environment variable")
            env_key = os.getenv('ALPHA_VANTAGE_API_KEY')
            if env_key:
                api_key = env_key.strip().strip('"').strip("'")
                used_env_credentials = True
                logger.info(f"🔑 [TEST] Using ALPHA_VANTAGE_API_KEY from environment (length: {len(api_key)})")
            else:
                logger.error(f"❌ [TEST] No valid API Key in config, database, or environment")
                return {
                    "success": False,
                    "message": "API Key 无效：配置、数据库和环境变量中均未配置有效的 API Key",
                    "response_time": time.time() - start_time,
                    "details": None
                }
    else:
        # API Key 是完整的，直接使用
        logger.info(f"✅ [TEST] Using complete API Key from config (length: {len(api_key)})")
    # 测试 Alpha Vantage API
    endpoint = ds_config.endpoint or "https://www.alphavantage.co"
    url = f"{endpoint}/query"
    params = {
        "function": "TIME_SERIES_INTRADAY",
        "symbol": "IBM",
        "interval": "5min",
        "apikey": api_key
    }
    try:
        logger.info(f"💡 [TEST] Calling Alpha Vantage API with key (length: {len(api_key)})")
        response = requests.get(url, params=params, timeout=10)
        if response.status_code == 200:
            data = response.json()
            if "Time Series (5min)" in data or "Meta Data" in data:
                response_time = time.time() - start_time
                logger.info(f"✅ [TEST] Alpha Vantage API call successful (response time: {response_time:.2f}s)")
                # 构建消息，说明使用了哪个来源的凭证
```

```
                credential_source = "配置"
                if used_db_credentials:
                    credential_source = "数据库"
                elif used_env_credentials:
                    credential_source = "环境变量"
                return {
                    "success": True,
                    "message": f"成功连接到 Alpha Vantage 数据源（使用{credential_source}中的凭证）",
                    "response_time": response_time,
                    "details": {
                        "type": ds_type,
                        "endpoint": endpoint,
                        "test_result": "API 密钥有效",
                        "credential_source": credential_source,
                        "used_db_credentials": used_db_credentials,
                        "used_env_credentials": used_env_credentials
                    }
                }
            elif "Error Message" in data:
                return {
                    "success": False,
                    "message": f"Alpha Vantage API 错误: {data['Error Message']}",
                    "response_time": time.time() - start_time,
                    "details": None
                }
            elif "Note" in data:
                return {
                    "success": False,
                    "message": "API 调用频率超限，请稍后再试",
                    "response_time": time.time() - start_time,
                    "details": None
                }
        return {
            "success": False,
            "message": f"Alpha Vantage API 返回错误: HTTP {response.status_code}",
            "response_time": time.time() - start_time,
            "details": None
        }
    except Exception as e:
        return {
            "success": False,
            "message": f"Alpha Vantage API 调用失败: {str(e)}",
            "response_time": time.time() - start_time,
            "details": None
        }
else:
    # 其他数据源类型 - 尝试从环境变量获取 API Key（如果需要）
    # 支持的环境变量映射
    env_key_map = {
        "finnhub": "FINNHUB_API_KEY",
        "polygon": "POLYGON_API_KEY",
        "iex": "IEX_API_KEY",
        "quandl": "QUANDL_API_KEY",
    }
    # 如果配置中没有 API Key，尝试从环境变量获取
    if ds_type in env_key_map and (not api_key or "..." in api_key):
        env_var_name = env_key_map[ds_type]
        env_key = os.getenv(env_var_name)
        if env_key:
            api_key = env_key.strip()
            used_env_credentials = True
            logger.info(f"🔗 使用环境变量中的 {ds_type.upper()} API Key ({env_var_name})")
    # 基本的端点测试
    if ds_config.endpoint:
        try:
            # 如果有 API Key，添加到请求中
            headers = {}
            params = {}
            if api_key:
                # 根据不同数据源的认证方式添加 API Key
                if ds_type == "finnhub":
                    params["token"] = api_key
                elif ds_type in ["polygon", "alpha_vantage"]:
                    params["apiKey"] = api_key
                elif ds_type == "iex":
                    params["token"] = api_key
                else:
                    # 默认使用 header 认证
                    headers["Authorization"] = f"Bearer {api_key}"
            response = requests.get(ds_config.endpoint, params=params, headers=headers, timeout=10)
            response_time = time.time() - start_time
            if response.status_code < 500:
                return {
                    "success": True,
                    "message": f"成功连接到数据源 {ds_config.name}",
                    "response_time": response_time,
                    "details": {
                        "type": ds_type,
                        "endpoint": ds_config.endpoint,
                        "status_code": response.status_code,
                        "used_env_credentials": used_env_credentials
                    }
                }
            else:
                return {
                    "success": False,
                    "message": f"数据源返回服务器错误: HTTP {response.status_code}",
                    "response_time": response_time,
                    "details": None
                }
        except Exception as e:
            return {
                "success": False,
                "message": f"连接失败: {str(e)}",
                "response_time": time.time() - start_time,
```

```
                "details": None
            }
        else:
            return {
                "success": False,
                "message": f"不支持的数据源类型: {ds_type}，且未配置端点",
                "response_time": time.time() - start_time,
                "details": None
            }
    except Exception as e:
        response_time = time.time() - start_time
        logger.error(f"❌ 测试数据源配置失败: {e}")
        return {
            "success": False,
            "message": f"连接失败: {str(e)}",
            "response_time": response_time,
            "details": None
        }

async def test_database_config(self, db_config: DatabaseConfig) -> Dict[str, Any]:
    """测试数据库配置 - 真实连接测试"""
    start_time = time.time()
    try:
        db_type = db_config.type.value if hasattr(db_config.type, 'value') else str(db_config.type)
        logger.info(f"🔧 测试数据库配置: {db_config.name} ({db_type})")
        logger.info(f"📍 连接地址: {db_config.host}:{db_config.port}")
        # 根据不同的数据库类型进行测试
        if db_type == "mongodb":
            try:
                from motor.motor_asyncio import AsyncIOMotorClient
                import os
                # 🔥 优先使用环境变量中的完整连接信息（包括host、用户名、密码）
                host = db_config.host
                port = db_config.port
                username = db_config.username
                password = db_config.password
                database = db_config.database
                auth_source = None
                used_env_config = False
                # 检测是否在 Docker 环境中
                is_docker = os.path.exists('/.dockerenv') or os.getenv('DOCKER_CONTAINER') == 'true'
                # 如果配置中没有用户名密码，尝试从环境变量获取完整配置
                if not username or not password:
                    env_host = os.getenv('MONGODB_HOST')
                    env_port = os.getenv('MONGODB_PORT')
                    env_username = os.getenv('MONGODB_USERNAME')
                    env_password = os.getenv('MONGODB_PASSWORD')
                    env_auth_source = os.getenv('MONGODB_AUTH_SOURCE', 'admin')
                    if env_username and env_password:
                        username = env_username
                        password = env_password
                        auth_source = env_auth_source
                        used_env_config = True
                        # 如果环境变量中有 host 配置，也使用它
                        if env_host:
                            host = env_host
                            # 🐳 Docker 环境下，将 localhost 替换为 mongodb
                            if is_docker and host == 'localhost':
                                host = 'mongodb'
                                logger.info(f"🐳 检测到 Docker 环境，将 host 从 localhost 改为 mongodb")
                        if env_port:
                            port = int(env_port)
                        logger.info(f"🔑 使用环境变量中的 MongoDB 配置 (host={host}, port={port}, authSource={auth_source})")
                # 如果配置中没有数据库名，尝试从环境变量获取
                if not database:
                    env_database = os.getenv('MONGODB_DATABASE')
                    if env_database:
                        database = env_database
                        logger.info(f"📦 使用环境变量中的数据库名: {database}")
                # 从连接参数中获取 authSource（如果有）
                if not auth_source and db_config.connection_params:
                    auth_source = db_config.connection_params.get('authSource')
                # 构建连接字符串
                if username and password:
                    connection_string = f"mongodb://{username}:{password}@{host}:{port}"
                else:
                    connection_string = f"mongodb://{host}:{port}"
                if database:
                    connection_string += f"/{database}"
                # 添加连接参数
                params_list = []
                # 如果有 authSource，添加到参数中
                if auth_source:
                    params_list.append(f"authSource={auth_source}")
                # 添加其他连接参数
                if db_config.connection_params:
                    for k, v in db_config.connection_params.items():
                        if k != 'authSource':  # authSource 已经添加过了
                            params_list.append(f"{k}={v}")
                if params_list:
                    connection_string += f"?{'&'.join(params_list)}"
                logger.info(f"🔗 连接字符串: {connection_string.replace(password or '', '***') if password else connection_string}")
                # 创建客户端并测试连接
                client = AsyncIOMotorClient(
                    connection_string,
                    serverSelectionTimeoutMS=5000  # 5秒超时
                )
                # 如果指定了数据库，测试该数据库的访问权限
                if database:
                    # 测试指定数据库的访问（不需要管理员权限）
                    db = client[database]
                    # 尝试列出集合（如果没有权限会报错）
                    collections = await db.list_collection_names()
```

```
            test_result = f"数据库 '{database}' 可访问，包含 {len(collections)} 个集合"
        else:
            # 如果没有指定数据库，只执行 ping 命令
            await client.admin.command('ping')
            test_result = "连接成功"
        response_time = time.time() - start_time
        # 关闭连接
        client.close()
        return {
            "success": True,
            "message": f"成功连接到 MongoDB 数据库",
            "response_time": response_time,
            "details": {
                "type": db_type,
                "host": host,
                "port": port,
                "database": database,
                "auth_source": auth_source,
                "test_result": test_result,
                "used_env_config": used_env_config
            }
        }
    except ImportError:
        return {
            "success": False,
            "message": "Motor 库未安装，请运行: pip install motor",
            "response_time": time.time() - start_time,
            "details": None
        }
    except Exception as e:
        error_msg = str(e)
        logger.error(f"❌ MongoDB 连接测试失败: {error_msg}")
        if "Authentication failed" in error_msg or "auth failed" in error_msg.lower():
            message = "认证失败，请检查用户名和密码"
        elif "requires authentication" in error_msg.lower():
            message = "需要认证，请配置用户名和密码"
        elif "not authorized" in error_msg.lower():
            message = "权限不足，请检查用户权限配置"
        elif "Connection refused" in error_msg:
            message = "连接被拒绝，请检查主机地址和端口"
        elif "timed out" in error_msg.lower():
            message = "连接超时，请检查网络和防火墙设置"
        elif "No servers found" in error_msg:
            message = "找不到服务器，请检查主机地址和端口"
        else:
            message = f"连接失败: {error_msg}"
        return {
            "success": False,
            "message": message,
            "response_time": time.time() - start_time,
            "details": None
        }
elif db_type == "redis":
    try:
        import redis.asyncio as aioredis
        import os
        # 🔥 优先使用环境变量中的完整 Redis 配置（包括host、密码）
        host = db_config.host
        port = db_config.port
        password = db_config.password
        database = db_config.database
        used_env_config = False
        # 检测是否在 Docker 环境中
        is_docker = os.path.exists('/.dockerenv') or os.getenv('DOCKER_CONTAINER') == 'true'
        # 如果配置中没有密码，尝试从环境变量获取完整配置
        if not password:
            env_host = os.getenv('REDIS_HOST')
            env_port = os.getenv('REDIS_PORT')
            env_password = os.getenv('REDIS_PASSWORD')
            if env_password:
                password = env_password
                used_env_config = True
                # 如果环境变量中有 host 配置，也使用它
                if env_host:
                    host = env_host
                    # 🔥 Docker 环境下，将 localhost 替换为 redis
                    if is_docker and host == 'localhost':
                        host = 'redis'
                        logger.info(f"🐳 检测到 Docker 环境，将 Redis host 从 localhost 改为 redis")
                if env_port:
                    port = int(env_port)
                logger.info(f"🔧 使用环境变量中的 Redis 配置 (host={host}, port={port})")
        # 如果配置中没有数据库编号，尝试从环境变量获取
        if database is None:
            env_db = os.getenv('REDIS_DB')
            if env_db:
                database = int(env_db)
                logger.info(f"📦 使用环境变量中的 Redis 数据库编号: {database}")
        # 构建连接参数
        redis_params = {
            "host": host,
            "port": port,
            "decode_responses": True,
            "socket_connect_timeout": 5
        }
        if password:
            redis_params["password"] = password
        if database is not None:
            redis_params["db"] = int(database)
        # 创建连接并测试
        redis_client = await aioredis.from_url(
            f"redis://{host}:{port}",
            **redis_params
```

```
        )
        # 执行 PING 命令
        await redis_client.ping()
        # 获取服务器信息
        info = await redis_client.info("server")
        response_time = time.time() - start_time
        # 关闭连接
        await redis_client.close()
        return {
            "success": True,
            "message": f"成功连接到 Redis 数据库",
            "response_time": response_time,
            "details": {
                "type": db_type,
                "host": host,
                "port": port,
                "database": database,
                "redis_version": info.get("redis_version", "unknown"),
                "used_env_config": used_env_config
            }
        }
    except ImportError:
        return {
            "success": False,
            "message": "Redis 库未安装，请运行: pip install redis",
            "response_time": time.time() - start_time,
            "details": None
        }
    except Exception as e:
        error_msg = str(e)
        if "WRONGPASS" in error_msg or "Authentication" in error_msg:
            message = "认证失败，请检查密码"
        elif "Connection refused" in error_msg:
            message = "连接被拒绝，请检查主机地址和端口"
        elif "timed out" in error_msg.lower():
            message = "连接超时，请检查网络和防火墙设置"
        else:
            message = f"连接失败: {error_msg}"
        return {
            "success": False,
            "message": message,
            "response_time": time.time() - start_time,
            "details": None
        }
elif db_type == "mysql":
    try:
        import aiomysql
        # 创建连接
        conn = await aiomysql.connect(
            host=db_config.host,
            port=db_config.port,
            user=db_config.username,
            password=db_config.password,
            db=db_config.database,
            connect_timeout=5
        )
        # 执行测试查询
        async with conn.cursor() as cursor:
            await cursor.execute("SELECT VERSION()")
            version = await cursor.fetchone()
        response_time = time.time() - start_time
        # 关闭连接
        conn.close()
        return {
            "success": True,
            "message": f"成功连接到 MySQL 数据库",
            "response_time": response_time,
            "details": {
                "type": db_type,
                "host": db_config.host,
                "port": db_config.port,
                "database": db_config.database,
                "version": version[0] if version else "unknown"
            }
        }
    except ImportError:
        return {
            "success": False,
            "message": "aiomysql 库未安装，请运行: pip install aiomysql",
            "response_time": time.time() - start_time,
            "details": None
        }
    except Exception as e:
        error_msg = str(e)
        if "Access denied" in error_msg:
            message = "访问被拒绝，请检查用户名和密码"
        elif "Unknown database" in error_msg:
            message = f"数据库 '{db_config.database}' 不存在"
        elif "Can't connect" in error_msg:
            message = "无法连接，请检查主机地址和端口"
        else:
            message = f"连接失败: {error_msg}"
        return {
            "success": False,
            "message": message,
            "response_time": time.time() - start_time,
            "details": None
        }
elif db_type == "postgresql":
    try:
        import asyncpg
        # 创建连接
        conn = await asyncpg.connect(
```

```
                host=db_config.host,
                port=db_config.port,
                user=db_config.username,
                password=db_config.password,
                database=db_config.database,
                timeout=5
            )
            # 执行测试查询
            version = await conn.fetchval("SELECT version()")
            response_time = time.time() - start_time
            # 关闭连接
            await conn.close()
            return {
                "success": True,
                "message": f"成功连接到 PostgreSQL 数据库",
                "response_time": response_time,
                "details": {
                    "type": db_type,
                    "host": db_config.host,
                    "port": db_config.port,
                    "database": db_config.database,
                    "version": version.split()[1] if version else "unknown"
                }
            }
        except ImportError:
            return {
                "success": False,
                "message": "asyncpg 库未安装，请运行: pip install asyncpg",
                "response_time": time.time() - start_time,
                "details": None
            }
        except Exception as e:
            error_msg = str(e)
            if "password authentication failed" in error_msg:
                message = "密码认证失败，请检查用户名和密码"
            elif "does not exist" in error_msg:
                message = f"数据库 '{db_config.database}' 不存在"
            elif "Connection refused" in error_msg:
                message = "连接被拒绝，请检查主机地址和端口"
            else:
                message = f"连接失败: {error_msg}"
            return {
                "success": False,
                "message": message,
                "response_time": time.time() - start_time,
                "details": None
            }
    elif db_type == "sqlite":
        try:
            import aiosqlite
            # SQLite 使用文件路径，不需要 host/port
            db_path = db_config.database or db_config.host
            # 创建连接
            async with aiosqlite.connect(db_path, timeout=5) as conn:
                # 执行测试查询
                async with conn.execute("SELECT sqlite_version()") as cursor:
                    version = await cursor.fetchone()
            response_time = time.time() - start_time
            return {
                "success": True,
                "message": f"成功连接到 SQLite 数据库",
                "response_time": response_time,
                "details": {
                    "type": db_type,
                    "database": db_path,
                    "version": version[0] if version else "unknown"
                }
            }
        except ImportError:
            return {
                "success": False,
                "message": "aiosqlite 库未安装，请运行: pip install aiosqlite",
                "response_time": time.time() - start_time,
                "details": None
            }
        except Exception as e:
            return {
                "success": False,
                "message": f"连接失败: {str(e)}",
                "response_time": time.time() - start_time,
                "details": None
            }
    else:
        return {
            "success": False,
            "message": f"不支持的数据库类型: {db_type}",
            "response_time": time.time() - start_time,
            "details": None
        }
except Exception as e:
    response_time = time.time() - start_time
    logger.error(f"❌ 测试数据库配置失败: {e}")
    return {
        "success": False,
        "message": f"连接失败: {str(e)}",
        "response_time": response_time,
        "details": None
    }
# ========== 数据库配置管理 ==========
async def add_database_config(self, db_config: DatabaseConfig) -> bool:
    """添加数据库配置"""
    try:
        logger.info(f"➕ 添加数据库配置: {db_config.name}")
```

```
        config = await self.get_system_config()
        if not config:
            logger.error("❌ 系统配置为空")
            return False
        # 检查是否已存在同名配置
        for existing_db in config.database_configs:
            if existing_db.name == db_config.name:
                logger.error(f"❌ 数据库配置 '{db_config.name}' 已存在")
                return False
        # 添加新配置
        config.database_configs.append(db_config)
        # 保存配置
        result = await self.save_system_config(config)
        if result:
            logger.info(f"☑ 数据库配置 '{db_config.name}' 添加成功")
        else:
            logger.error(f"❌ 数据库配置 '{db_config.name}' 添加失败")
        return result
    except Exception as e:
        logger.error(f"❌ 添加数据库配置失败: {e}")
        import traceback
        traceback.print_exc()
        return False
async def update_database_config(self, db_config: DatabaseConfig) -> bool:
    """更新数据库配置"""
    try:
        logger.info(f"🔄 更新数据库配置: {db_config.name}")
        config = await self.get_system_config()
        if not config:
            logger.error("❌ 系统配置为空")
            return False
        # 查找并更新配置
        found = False
        for i, existing_db in enumerate(config.database_configs):
            if existing_db.name == db_config.name:
                config.database_configs[i] = db_config
                found = True
                break
        if not found:
            logger.error(f"❌ 数据库配置 '{db_config.name}' 不存在")
            return False
        # 保存配置
        result = await self.save_system_config(config)
        if result:
            logger.info(f"☑ 数据库配置 '{db_config.name}' 更新成功")
        else:
            logger.error(f"❌ 数据库配置 '{db_config.name}' 更新失败")
        return result
    except Exception as e:
        logger.error(f"❌ 更新数据库配置失败: {e}")
        import traceback
        traceback.print_exc()
        return False
async def delete_database_config(self, db_name: str) -> bool:
    """删除数据库配置"""
    try:
        logger.info(f"🗑 删除数据库配置: {db_name}")
        config = await self.get_system_config()
        if not config:
            logger.error("❌ 系统配置为空")
            return False
        # 记录原始数量
        original_count = len(config.database_configs)
        # 删除指定配置
        config.database_configs = [
            db for db in config.database_configs
            if db.name != db_name
        ]
        new_count = len(config.database_configs)
        if new_count == original_count:
            logger.error(f"❌ 数据库配置 '{db_name}' 不存在")
            return False
        # 保存配置
        result = await self.save_system_config(config)
        if result:
            logger.info(f"☑ 数据库配置 '{db_name}' 删除成功")
        else:
            logger.error(f"❌ 数据库配置 '{db_name}' 删除失败")
        return result
    except Exception as e:
        logger.error(f"❌ 删除数据库配置失败: {e}")
        import traceback
        traceback.print_exc()
        return False
async def get_database_config(self, db_name: str) -> Optional[DatabaseConfig]:
    """获取指定的数据库配置"""
    try:
        config = await self.get_system_config()
        if not config:
            return None
        for db in config.database_configs:
            if db.name == db_name:
                return db
        return None
    except Exception as e:
        logger.error(f"❌ 获取数据库配置失败: {e}")
        return None
async def get_database_configs(self) -> List[DatabaseConfig]:
    """获取所有数据库配置"""
    try:
        config = await self.get_system_config()
        if not config:
```

```
        return []
    return config.database_configs
    except Exception as e:
        logger.error(f"❌ 获取数据库配置列表失败: {e}")
        return []
# ========== 模型目录管理 ==========
async def get_model_catalog(self) -> List[ModelCatalog]:
    """获取所有模型目录"""
    try:
        db = await self._get_db()
        catalog_collection = db.model_catalog
        catalogs = []
        async for doc in catalog_collection.find():
            catalogs.append(ModelCatalog(**doc))
        return catalogs
    except Exception as e:
        print(f"获取模型目录失败: {e}")
        return []
async def get_provider_models(self, provider: str) -> Optional[ModelCatalog]:
    """获取指定厂家的模型目录"""
    try:
        db = await self._get_db()
        catalog_collection = db.model_catalog
        doc = await catalog_collection.find_one({"provider": provider})
        if doc:
            return ModelCatalog(**doc)
        return None
    except Exception as e:
        print(f"获取厂家模型目录失败: {e}")
        return None
async def save_model_catalog(self, catalog: ModelCatalog) -> bool:
    """保存或更新模型目录"""
    try:
        db = await self._get_db()
        catalog_collection = db.model_catalog
        catalog.updated_at = now_tz()
        # 更新或插入
        result = await catalog_collection.replace_one(
            {"provider": catalog.provider},
            catalog.model_dump(by_alias=True, exclude={"id"}),
            upsert=True
        )
        return result.acknowledged
    except Exception as e:
        print(f"保存模型目录失败: {e}")
        return False
async def delete_model_catalog(self, provider: str) -> bool:
    """删除模型目录"""
    try:
        db = await self._get_db()
        catalog_collection = db.model_catalog
        result = await catalog_collection.delete_one({"provider": provider})
        return result.deleted_count > 0
    except Exception as e:
        print(f"删除模型目录失败: {e}")
        return False
async def init_default_model_catalog(self) -> bool:
    """初始化默认模型目录"""
    try:
        db = await self._get_db()
        catalog_collection = db.model_catalog
        # 检查是否已有数据
        count = await catalog_collection.count_documents({})
        if count > 0:
            print("模型目录已存在，跳过初始化")
            return True
        # 创建默认目录
        default_catalogs = self._get_default_model_catalog()
        for catalog_data in default_catalogs:
            catalog = ModelCatalog(**catalog_data)
            await self.save_model_catalog(catalog)
        print(f"✅ 初始化了 {len(default_catalogs)} 个厂家的模型目录")
        return True
    except Exception as e:
        print(f"初始化模型目录失败: {e}")
        return False
def _get_default_model_catalog(self) -> List[Dict[str, Any]]:
    """获取默认模型目录数据"""
    return [
        {
            "provider": "dashscope",
            "provider_name": "通义千问",
            "models": [
                {
                    "name": "qwen-turbo",
                    "display_name": "Qwen Turbo - 快速经济 (1M上下文)",
                    "input_price_per_1k": 0.0003,
                    "output_price_per_1k": 0.0003,
                    "context_length": 1000000,
                    "currency": "CNY",
                    "description": "Qwen2.5-Turbo，支持100万tokens超长上下文"
                },
                {
                    "name": "qwen-plus",
                    "display_name": "Qwen Plus - 平衡推荐",
                    "input_price_per_1k": 0.0008,
                    "output_price_per_1k": 0.002,
                    "context_length": 32768,
                    "currency": "CNY"
                },
                {
                    "name": "qwen-plus-latest",
                    "display_name": "Qwen Plus Latest - 最新平衡",
```

```
        "input_price_per_1k": 0.0008,
        "output_price_per_1k": 0.002,
        "context_length": 32768,
        "currency": "CNY"
      },
      {
        "name": "qwen-max",
        "display_name": "Qwen Max - 最强性能",
        "input_price_per_1k": 0.02,
        "output_price_per_1k": 0.06,
        "context_length": 8192,
        "currency": "CNY"
      },
      {
        "name": "qwen-max-latest",
        "display_name": "Qwen Max Latest - 最新旗舰",
        "input_price_per_1k": 0.02,
        "output_price_per_1k": 0.06,
        "context_length": 8192,
        "currency": "CNY"
      },
      {
        "name": "qwen-long",
        "display_name": "Qwen Long - 长文本",
        "input_price_per_1k": 0.0005,
        "output_price_per_1k": 0.002,
        "context_length": 1000000,
        "currency": "CNY"
      },
      {
        "name": "qwen-vl-plus",
        "display_name": "Qwen VL Plus - 视觉理解",
        "input_price_per_1k": 0.008,
        "output_price_per_1k": 0.008,
        "context_length": 8192,
        "currency": "CNY"
      },
      {
        "name": "qwen-vl-max",
        "display_name": "Qwen VL Max - 视觉旗舰",
        "input_price_per_1k": 0.02,
        "output_price_per_1k": 0.02,
        "context_length": 8192,
        "currency": "CNY"
      }
    ]
  },
  {
    "provider": "openai",
    "provider_name": "OpenAI",
    "models": [
      {
        "name": "gpt-4o",
        "display_name": "GPT-4o - 最新旗舰",
        "input_price_per_1k": 0.005,
        "output_price_per_1k": 0.015,
        "context_length": 128000,
        "currency": "USD"
      },
      {
        "name": "gpt-4o-mini",
        "display_name": "GPT-4o Mini - 轻量旗舰",
        "input_price_per_1k": 0.00015,
        "output_price_per_1k": 0.0006,
        "context_length": 128000,
        "currency": "USD"
      },
      {
        "name": "gpt-4-turbo",
        "display_name": "GPT-4 Turbo - 强化版",
        "input_price_per_1k": 0.01,
        "output_price_per_1k": 0.03,
        "context_length": 128000,
        "currency": "USD"
      },
      {
        "name": "gpt-4",
        "display_name": "GPT-4 - 经典版",
        "input_price_per_1k": 0.03,
        "output_price_per_1k": 0.06,
        "context_length": 8192,
        "currency": "USD"
      },
      {
        "name": "gpt-3.5-turbo",
        "display_name": "GPT-3.5 Turbo - 经济版",
        "input_price_per_1k": 0.0005,
        "output_price_per_1k": 0.0015,
        "context_length": 16385,
        "currency": "USD"
      }
    ]
  },
  {
    "provider": "google",
    "provider_name": "Google Gemini",
    "models": [
      {
        "name": "gemini-2.5-pro",
        "display_name": "Gemini 2.5 Pro - 最新旗舰",
        "input_price_per_1k": 0.00125,
        "output_price_per_1k": 0.005,
        "context_length": 1000000,
        "currency": "USD"
      },
      {
```

```
            "name": "gemini-2.5-flash",
            "display_name": "Gemini 2.5 Flash - 最新快速",
            "input_price_per_1k": 0.000075,
            "output_price_per_1k": 0.0003,
            "context_length": 1000000,
            "currency": "USD"
        },
        {
            "name": "gemini-1.5-pro",
            "display_name": "Gemini 1.5 Pro - 专业版",
            "input_price_per_1k": 0.00125,
            "output_price_per_1k": 0.005,
            "context_length": 2000000,
            "currency": "USD"
        },
        {
            "name": "gemini-1.5-flash",
            "display_name": "Gemini 1.5 Flash - 快速版",
            "input_price_per_1k": 0.000075,
            "output_price_per_1k": 0.0003,
            "context_length": 1000000,
            "currency": "USD"
        }
    ]
  },
  {
    "provider": "deepseek",
    "provider_name": "DeepSeek",
    "models": [
        {
            "name": "deepseek-chat",
            "display_name": "DeepSeek Chat - 通用对话",
            "input_price_per_1k": 0.0001,
            "output_price_per_1k": 0.0002,
            "context_length": 32768,
            "currency": "CNY"
        },
        {
            "name": "deepseek-coder",
            "display_name": "DeepSeek Coder - 代码专用",
            "input_price_per_1k": 0.0001,
            "output_price_per_1k": 0.0002,
            "context_length": 16384,
            "currency": "CNY"
        }
    ]
  },
  {
    "provider": "anthropic",
    "provider_name": "Anthropic Claude",
    "models": [
        {
            "name": "claude-3-5-sonnet-20241022",
            "display_name": "Claude 3.5 Sonnet - 当前旗舰",
            "input_price_per_1k": 0.003,
            "output_price_per_1k": 0.015,
            "context_length": 200000,
            "currency": "USD"
        },
        {
            "name": "claude-3-5-sonnet-20240620",
            "display_name": "Claude 3.5 Sonnet (旧版)",
            "input_price_per_1k": 0.003,
            "output_price_per_1k": 0.015,
            "context_length": 200000,
            "currency": "USD"
        },
        {
            "name": "claude-3-opus-20240229",
            "display_name": "Claude 3 Opus - 强大性能",
            "input_price_per_1k": 0.015,
            "output_price_per_1k": 0.075,
            "context_length": 200000,
            "currency": "USD"
        },
        {
            "name": "claude-3-sonnet-20240229",
            "display_name": "Claude 3 Sonnet - 平衡版",
            "input_price_per_1k": 0.003,
            "output_price_per_1k": 0.015,
            "context_length": 200000,
            "currency": "USD"
        },
        {
            "name": "claude-3-haiku-20240307",
            "display_name": "Claude 3 Haiku - 快速版",
            "input_price_per_1k": 0.00025,
            "output_price_per_1k": 0.00125,
            "context_length": 200000,
            "currency": "USD"
        }
    ]
  },
  {
    "provider": "qianfan",
    "provider_name": "百度千帆",
    "models": [
        {
            "name": "ernie-3.5-8k",
            "display_name": "ERNIE 3.5 8K - 快速高效",
            "input_price_per_1k": 0.0012,
            "output_price_per_1k": 0.0012,
            "context_length": 8192,
            "currency": "CNY"
        },
        {
```

```
                "name": "ernie-4.0-turbo-8k",
                "display_name": "ERNIE 4.0 Turbo 8K - 强大推理",
                "input_price_per_1k": 0.03,
                "output_price_per_1k": 0.09,
                "context_length": 8192,
                "currency": "CNY"
            },
            {
                "name": "ERNIE-Speed-8K",
                "display_name": "ERNIE Speed 8K - 极速响应",
                "input_price_per_1k": 0.0004,
                "output_price_per_1k": 0.0004,
                "context_length": 8192,
                "currency": "CNY"
            },
            {
                "name": "ERNIE-Lite-8K",
                "display_name": "ERNIE Lite 8K - 轻量经济",
                "input_price_per_1k": 0.0003,
                "output_price_per_1k": 0.0006,
                "context_length": 8192,
                "currency": "CNY"
            }
        ]
    },
    {
        "provider": "zhipu",
        "provider_name": "智谱AI",
        "models": [
            {
                "name": "glm-4",
                "display_name": "GLM-4 - 旗舰版",
                "input_price_per_1k": 0.1,
                "output_price_per_1k": 0.1,
                "context_length": 128000,
                "currency": "CNY"
            },
            {
                "name": "glm-4-plus",
                "display_name": "GLM-4 Plus - 增强版",
                "input_price_per_1k": 0.05,
                "output_price_per_1k": 0.05,
                "context_length": 128000,
                "currency": "CNY"
            },
            {
                "name": "glm-3-turbo",
                "display_name": "GLM-3 Turbo - 快速版",
                "input_price_per_1k": 0.001,
                "output_price_per_1k": 0.001,
                "context_length": 128000,
                "currency": "CNY"
            }
        ]
    }
]
async def get_available_models(self) -> List[Dict[str, Any]]:
    """获取可用的模型列表（从数据库读取，如果为空则返回默认数据）"""
    try:
        catalogs = await self.get_model_catalog()
        # 如果数据库中没有数据，初始化默认目录
        if not catalogs:
            print("📦 模型目录为空，初始化默认目录...")
            await self.init_default_model_catalog()
            catalogs = await self.get_model_catalog()
        # 转换为API响应格式
        result = []
        for catalog in catalogs:
            result.append({
                "provider": catalog.provider,
                "provider_name": catalog.provider_name,
                "models": [
                    {
                        "name": model.name,
                        "display_name": model.display_name,
                        "description": model.description,
                        "context_length": model.context_length,
                        "input_price_per_1k": model.input_price_per_1k,
                        "output_price_per_1k": model.output_price_per_1k,
                        "is_deprecated": model.is_deprecated
                    }
                    for model in catalog.models
                ]
            })
        return result
    except Exception as e:
        print(f"获取模型列表失败: {e}")
        # 失败时返回默认数据
        return self._get_default_model_catalog()
async def set_default_llm(self, model_name: str) -> bool:
    """设置默认大模型"""
    try:
        config = await self.get_system_config()
        if not config:
            return False
        # 检查模型是否存在
        model_exists = any(
            llm.model_name == model_name
            for llm in config.llm_configs
        )
        if not model_exists:
            return False
        config.default_llm = model_name
        return await self.save_system_config(config)
```

```
    except Exception as e:
        print(f"设置默认LLM失败: {e}")
        return False
async def set_default_data_source(self, source_name: str) -> bool:
    """设置默认数据源"""
    try:
        config = await self.get_system_config()
        if not config:
            return False
        # 检查数据源是否存在
        source_exists = any(
            ds.name == source_name
            for ds in config.data_source_configs
        )
        if not source_exists:
            return False
        config.default_data_source = source_name
        return await self.save_system_config(config)
    except Exception as e:
        print(f"设置默认数据源失败: {e}")
        return False
# ========== 大模型厂家管理 ==========
async def get_llm_providers(self) -> List[LLMProvider]:
    """获取所有大模型厂家（合并环境变量配置）"""
    try:
        db = await self._get_db()
        providers_collection = db.llm_providers
        providers_data = await providers_collection.find().to_list(length=None)
        providers = []
        logger.info(f"🔍 [get_llm_providers] 从数据库获取到 {len(providers_data)} 个供应商")
        for provider_data in providers_data:
            provider = LLMProvider(**provider_data)
            # 🔥 判断数据库中的 API Key 是否有效
            db_key_valid = self._is_valid_api_key(provider.api_key)
            logger.info(f"🔍 [get_llm_providers] 供应商 {provider.display_name} ({provider.name}): 数据库密钥有效={db_key_valid}")
            # 初始化 extra_config
            provider.extra_config = provider.extra_config or {}
            if not db_key_valid:
                # 数据库中的 Key 无效，尝试从环境变量获取
                logger.info(f"🔍 [get_llm_providers] 尝试从环境变量获取 {provider.name} 的 API 密钥...")
                env_key = self._get_env_api_key(provider.name)
                if env_key:
                    provider.api_key = env_key
                    provider.extra_config["source"] = "environment"
                    provider.extra_config["has_api_key"] = True
                    logger.info(f"✅ [get_llm_providers] 从环境变量为厂家 {provider.display_name} 获取API密钥")
                else:
                    provider.extra_config["has_api_key"] = False
                    logger.warning(f"⚠️ [get_llm_providers] 厂家 {provider.display_name} 的数据库配置和环境变量都未配置有效的API密钥")
            else:
                # 数据库中的 Key 有效，使用数据库配置
                provider.extra_config["source"] = "database"
                provider.extra_config["has_api_key"] = True
                logger.info(f"✅ [get_llm_providers] 使用数据库配置的 {provider.display_name} API密钥")
            providers.append(provider)
        logger.info(f"🔍 [get_llm_providers] 返回 {len(providers)} 个供应商")
        return providers
    except Exception as e:
        logger.error(f"❌ [get_llm_providers] 获取厂家列表失败: {e}", exc_info=True)
        return []
def _is_valid_api_key(self, api_key: Optional[str]) -> bool:
    """
    判断 API Key 是否有效
    有效条件：
    1. Key 不为空
    2. Key 不是占位符（不以 'your_' 或 'your-' 开头，不以 '_here' 结尾）
    3. Key 不是截断的密钥（不包含 '...'）
    4. Key 长度 > 10（基本的格式验证）
    Args:
        api_key: 待验证的 API Key
    Returns:
        bool: True 表示有效，False 表示无效
    """
    if not api_key:
        return False
    # 去除首尾空格
    api_key = api_key.strip()
    # 检查是否为空
    if not api_key:
        return False
    # 检查是否为占位符（前缀）
    if api_key.startswith('your_') or api_key.startswith('your-'):
        return False
    # 检查是否为占位符（后缀）
    if api_key.endswith('_here') or api_key.endswith('-here'):
        return False
    # 🔥 检查是否为截断的密钥（包含 '...'）
    if '...' in api_key:
        return False
    # 检查长度（大多数 API Key 都 > 10 个字符）
    if len(api_key) <= 10:
        return False
    return True
def _get_env_api_key(self, provider_name: str) -> Optional[str]:
    """从环境变量获取API密钥"""
    import os
    # 环境变量映射表
    env_key_mapping = {
        "openai": "OPENAI_API_KEY",
        "anthropic": "ANTHROPIC_API_KEY",
        "google": "GOOGLE_API_KEY",
        "zhipu": "ZHIPU_API_KEY",
```

```
        "deepseek": "DEEPSEEK_API_KEY",
        "dashscope": "DASHSCOPE_API_KEY",
        "qianfan": "QIANFAN_API_KEY",
        "azure": "AZURE_OPENAI_API_KEY",
        "siliconflow": "SILICONFLOW_API_KEY",
        "openrouter": "OPENROUTER_API_KEY",
        # 🆕 聚合渠道
        "302ai": "AI302_API_KEY",
        "oneapi": "ONEAPI_API_KEY",
        "newapi": "NEWAPI_API_KEY",
        "custom_aggregator": "CUSTOM_AGGREGATOR_API_KEY"
    }
    env_var = env_key_mapping.get(provider_name)
    if env_var:
        api_key = os.getenv(env_var)
        # 使用统一的验证方法
        if self._is_valid_api_key(api_key):
            return api_key
    return None
async def add_llm_provider(self, provider: LLMProvider) -> str:
    """添加大模型厂家"""
    try:
        db = await self._get_db()
        providers_collection = db.llm_providers
        # 检查厂家名称是否已存在
        existing = await providers_collection.find_one({"name": provider.name})
        if existing:
            raise ValueError(f"厂家 {provider.name} 已存在")
        provider.created_at = now_tz()
        provider.updated_at = now_tz()
        # 修复：删除 _id 字段，让 MongoDB 自动生成 ObjectId
        provider_data = provider.model_dump(by_alias=True, exclude_unset=True)
        if "_id" in provider_data:
            del provider_data["_id"]
        result = await providers_collection.insert_one(provider_data)
        return str(result.inserted_id)
    except Exception as e:
        print(f"添加厂家失败: {e}")
        raise
async def update_llm_provider(self, provider_id: str, update_data: Dict[str, Any]) -> bool:
    """更新大模型厂家"""
    try:
        db = await self._get_db()
        providers_collection = db.llm_providers
        update_data["updated_at"] = now_tz()
        # 兼容处理：尝试 ObjectId 和字符串两种类型
        # 原因：历史数据可能混用了 ObjectId 和字符串作为 _id
        try:
            # 先尝试作为 ObjectId 查询
            result = await providers_collection.update_one(
                {"_id": ObjectId(provider_id)},
                {"$set": update_data}
            )
            # 如果没有匹配到，再尝试作为字符串查询
            if result.matched_count == 0:
                result = await providers_collection.update_one(
                    {"_id": provider_id},
                    {"$set": update_data}
                )
        except Exception:
            # 如果 ObjectId 转换失败，直接用字符串查询
            result = await providers_collection.update_one(
                {"_id": provider_id},
                {"$set": update_data}
            )
        # 修复：matched_count > 0 表示找到了记录（即使没有修改）
        # modified_count > 0 只有在实际修改了字段时才为真
        # 如果记录存在但值相同，modified_count 为 0，但这不应该返回 404
        return result.matched_count > 0
    except Exception as e:
        print(f"更新厂家失败: {e}")
        import traceback
        traceback.print_exc()
        return False
async def delete_llm_provider(self, provider_id: str) -> bool:
    """删除大模型厂家"""
    try:
        print(f"🗑️ 删除厂家 - provider_id: {provider_id}")
        print(f"🔍 ObjectId类型: {type(ObjectId(provider_id))}")
        db = await self._get_db()
        providers_collection = db.llm_providers
        print(f"📊 数据库: {db.name}, 集合: {providers_collection.name}")
        # 先列出所有厂家的ID，看看格式
        all_providers = await providers_collection.find({}, {"_id": 1, "display_name": 1}).to_list(length=None)
        print(f"📋 数据库中所有厂家ID:")
        for p in all_providers:
            print(f"  - {p['_id']} ({type(p['_id'])}) - {p.get('display_name')}")
            if str(p['_id']) == provider_id:
                print(f"  ✅ 找到匹配的ID!")
        # 尝试不同的查找方式
        print(f"🔍 尝试用ObjectId查找...")
        existing1 = await providers_collection.find_one({"_id": ObjectId(provider_id)})
        print(f"🔍 尝试用字符串查找...")
        existing2 = await providers_collection.find_one({"_id": provider_id})
        print(f"🔍 ObjectId查找结果: {existing1 is not None}")
        print(f"🔍 字符串查找结果: {existing2 is not None}")
        existing = existing1 or existing2
        if not existing:
            print(f"❌ 两种方式都找不到厂家: {provider_id}")
            return False
        print(f"✅ 找到厂家: {existing.get('display_name')}")
        # 使用找到的方式进行删除
        if existing1:
```

```
            result = await providers_collection.delete_one({"_id": ObjectId(provider_id)})
        else:
            result = await providers_collection.delete_one({"_id": provider_id})
        success = result.deleted_count > 0
        print(f"🗑️ 删除结果: {success}, deleted_count: {result.deleted_count}")
        return success
    except Exception as e:
        print(f"❌ 删除厂家失败: {e}")
        import traceback
        traceback.print_exc()
        return False
async def toggle_llm_provider(self, provider_id: str, is_active: bool) -> bool:
    """切换大模型厂家状态"""
    try:
        db = await self._get_db()
        providers_collection = db.llm_providers
        # 兼容处理：尝试 ObjectId 和字符串两种类型
        try:
            # 先尝试作为 ObjectId 查询
            result = await providers_collection.update_one(
                {"_id": ObjectId(provider_id)},
                {"$set": {"is_active": is_active, "updated_at": now_tz()}}
            )
            # 如果没有匹配到，再尝试作为字符串查询
            if result.matched_count == 0:
                result = await providers_collection.update_one(
                    {"_id": provider_id},
                    {"$set": {"is_active": is_active, "updated_at": now_tz()}}
                )
        except Exception:
            # 如果 ObjectId 转换失败，直接用字符串查询
            result = await providers_collection.update_one(
                {"_id": provider_id},
                {"$set": {"is_active": is_active, "updated_at": now_tz()}}
            )
        return result.matched_count > 0
    except Exception as e:
        print(f"切换厂家状态失败: {e}")
        return False
async def init_aggregator_providers(self) -> Dict[str, Any]:
    """
    初始化聚合渠道厂家配置
    Returns:
        初始化结果统计
    """
    from app.constants.model_capabilities import AGGREGATOR_PROVIDERS
    try:
        db = await self._get_db()
        providers_collection = db.llm_providers
        added_count = 0
        skipped_count = 0
        updated_count = 0
        for provider_name, config in AGGREGATOR_PROVIDERS.items():
            # 从环境变量获取 API Key
            api_key = self._get_env_api_key(provider_name)
            # 检查是否已存在
            existing = await providers_collection.find_one({"name": provider_name})
            if existing:
                # 如果已存在但没有 API Key，且环境变量中有，则更新
                if not existing.get("api_key") and api_key:
                    update_data = {
                        "api_key": api_key,
                        "is_active": True,  # 有 API Key 则自动启用
                        "updated_at": now_tz()
                    }
                    await providers_collection.update_one(
                        {"name": provider_name},
                        {"$set": update_data}
                    )
                    updated_count += 1
                    print(f"✅ 更新聚合渠道 {config['display_name']} 的 API Key")
                else:
                    skipped_count += 1
                    print(f"⏭️ 聚合渠道 {config['display_name']} 已存在，跳过")
                continue
            # 创建聚合渠道厂家配置
            provider_data = {
                "name": provider_name,
                "display_name": config["display_name"],
                "description": config["description"],
                "website": config.get("website"),
                "api_doc_url": config.get("api_doc_url"),
                "default_base_url": config["default_base_url"],
                "is_active": bool(api_key),  # 有 API Key 则自动启用
                "supported_features": ["chat", "completion", "function_calling", "streaming"],
                "api_key": api_key or "",
                "extra_config": {
                    "supported_providers": config.get("supported_providers", []),
                    "source": "environment" if api_key else "manual"
                },
                # 🔗 聚合渠道标识
                "is_aggregator": True,
                "aggregator_type": "openai_compatible",
                "model_name_format": config.get("model_name_format", "{provider}/{model}"),
                "created_at": now_tz(),
                "updated_at": now_tz()
            }
            provider = LLMProvider(**provider_data)
            # 修复：删除 _id 字段，让 MongoDB 自动生成 ObjectId
            insert_data = provider.model_dump(by_alias=True, exclude_unset=True)
            if "_id" in insert_data:
                del insert_data["_id"]
            await providers_collection.insert_one(insert_data)
```

```
            added_count += 1
            if api_key:
                print(f"✅ 添加聚合渠道: {config['display_name']} (已从环境变量获取 API Key)")
            else:
                print(f"✅ 添加聚合渠道: {config['display_name']} (需手动配置 API Key)")
        message_parts = []
        if added_count > 0:
            message_parts.append(f"成功添加 {added_count} 个聚合渠道")
        if updated_count > 0:
            message_parts.append(f"更新 {updated_count} 个")
        if skipped_count > 0:
            message_parts.append(f"跳过 {skipped_count} 个已存在的")
        return {
            "success": True,
            "added": added_count,
            "updated": updated_count,
            "skipped": skipped_count,
            "message": "，".join(message_parts) if message_parts else "无变更"
        }
    except Exception as e:
        print(f"❌ 初始化聚合渠道失败: {e}")
        import traceback
        traceback.print_exc()
        return {
            "success": False,
            "error": str(e),
            "message": "初始化聚合渠道失败"
        }
async def migrate_env_to_providers(self) -> Dict[str, Any]:
    """将环境变量配置迁移到厂家管理"""
    import os
    try:
        db = await self._get_db()
        providers_collection = db.llm_providers
        # 预设厂家配置
        default_providers = [
            {
                "name": "openai",
                "display_name": "OpenAI",
                "description": "OpenAI是人工智能领域的领先公司，提供GPT系列模型",
                "website": "https://openai.com",
                "api_doc_url": "https://platform.openai.com/docs",
                "default_base_url": "https://api.openai.com/v1",
                "supported_features": ["chat", "completion", "embedding", "image", "vision", "function_calling", "streaming"]
            },
            {
                "name": "anthropic",
                "display_name": "Anthropic",
                "description": "Anthropic专注于AI安全研究，提供Claude系列模型",
                "website": "https://anthropic.com",
                "api_doc_url": "https://docs.anthropic.com",
                "default_base_url": "https://api.anthropic.com",
                "supported_features": ["chat", "completion", "function_calling", "streaming"]
            },
            {
                "name": "dashscope",
                "display_name": "阿里云百炼",
                "description": "阿里云百炼大模型服务平台，提供通义千问等模型",
                "website": "https://bailian.console.aliyun.com",
                "api_doc_url": "https://help.aliyun.com/zh/dashscope/",
                "default_base_url": "https://dashscope.aliyuncs.com/compatible-mode/v1",
                "supported_features": ["chat", "completion", "embedding", "function_calling", "streaming"]
            },
            {
                "name": "deepseek",
                "display_name": "DeepSeek",
                "description": "DeepSeek提供高性能的AI推理服务",
                "website": "https://www.deepseek.com",
                "api_doc_url": "https://platform.deepseek.com/api-docs",
                "default_base_url": "https://api.deepseek.com",
                "supported_features": ["chat", "completion", "function_calling", "streaming"]
            }
        ]
        migrated_count = 0
        updated_count = 0
        skipped_count = 0
        for provider_config in default_providers:
            # 从环境变量获取API密钥
            api_key = self._get_env_api_key(provider_config["name"])
            # 检查是否已存在
            existing = await providers_collection.find_one({"name": provider_config["name"]})
            if existing:
                # 如果已存在但没有API密钥，且环境变量中有密钥，则更新
                if not existing.get("api_key") and api_key:
                    update_data = {
                        "api_key": api_key,
                        "is_active": True,
                        "extra_config": {"migrated_from": "environment"},
                        "updated_at": now_tz()
                    }
                    await providers_collection.update_one(
                        {"name": provider_config["name"]},
                        {"$set": update_data}
                    )
                    updated_count += 1
                    print(f"✅ 更新厂家 {provider_config['display_name']} 的API密钥")
                else:
                    skipped_count += 1
                    print(f"⏭️ 跳过厂家 {provider_config['display_name']} (已有配置)")
                continue
            # 创建新厂家配置
            provider_data = {
                **provider_config,
                "api_key": api_key,
```

```
            "is_active": bool(api_key),  # 有密钥的自动启用
            "extra_config": {"migrated_from": "environment"} if api_key else {},
            "created_at": now_tz(),
            "updated_at": now_tz()
        }
        await providers_collection.insert_one(provider_data)
        migrated_count += 1
        print(f"✅ 创建厂家 {provider_config['display_name']}")
    total_changes = migrated_count + updated_count
    message_parts = []
    if migrated_count > 0:
        message_parts.append(f"新建 {migrated_count} 个厂家")
    if updated_count > 0:
        message_parts.append(f"更新 {updated_count} 个厂家的API密钥")
    if skipped_count > 0:
        message_parts.append(f"跳过 {skipped_count} 个已配置的厂家")
    if total_changes > 0:
        message = "迁移完成：" + "，".join(message_parts)
    else:
        message = "所有厂家都已配置，无需迁移"
    return {
        "success": True,
        "migrated_count": migrated_count,
        "updated_count": updated_count,
        "skipped_count": skipped_count,
        "message": message
    }
  except Exception as e:
    print(f"环境变量迁移失败: {e}")
    return {
        "success": False,
        "error": str(e),
        "message": "环境变量迁移失败"
    }
async def test_provider_api(self, provider_id: str) -> dict:
  """测试厂家API密钥"""
  try:
    print(f"🔍 测试厂家API - provider_id: {provider_id}")
    db = await self._get_db()
    providers_collection = db.llm_providers
    # 兼容处理：尝试 ObjectId 和字符串两种类型
    from bson import ObjectId
    provider_data = None
    try:
        # 先尝试作为 ObjectId 查询
        provider_data = await providers_collection.find_one({"_id": ObjectId(provider_id)})
    except Exception:
        pass
    # 如果没有找到，再尝试作为字符串查询
    if not provider_data:
        provider_data = await providers_collection.find_one({"_id": provider_id})
    if not provider_data:
        return {
            "success": False,
            "message": f"厂家不存在 (ID: {provider_id})"
        }
    provider_name = provider_data.get("name")
    api_key = provider_data.get("api_key")
    display_name = provider_data.get("display_name", provider_name)
    # 🔥 判断数据库中的 API Key 是否有效
    if not self._is_valid_api_key(api_key):
        # 数据库中的 Key 无效，尝试从环境变量读取
        env_api_key = self._get_env_api_key(provider_name)
        if env_api_key:
            api_key = env_api_key
            print(f"✅ 数据库配置无效，从环境变量读取到 {display_name} 的 API Key")
        else:
            return {
                "success": False,
                "message": f"{display_name} 未配置有效的API密钥（数据库和环境变量中都未找到）"
            }
    else:
        print(f"✅ 使用数据库配置的 {display_name} API密钥")
    # 根据厂家类型调用相应的测试函数
    test_result = await self._test_provider_connection(provider_name, api_key, display_name)
    return test_result
  except Exception as e:
    print(f"测试厂家API失败: {e}")
    return {
        "success": False,
        "message": f"测试失败: {str(e)}"
    }
async def _test_provider_connection(self, provider_name: str, api_key: str, display_name: str) -> dict:
  """测试具体厂家的连接"""
  import asyncio
  try:
    # 聚合渠道（使用 OpenAI 兼容 API）
    if provider_name in ["302ai", "oneapi", "newapi", "custom_aggregator"]:
        # 获取厂家的 base_url
        db = await self._get_db()
        providers_collection = db.llm_providers
        provider_data = await providers_collection.find_one({"name": provider_name})
        base_url = provider_data.get("default_base_url") if provider_data else None
        return await asyncio.get_event_loop().run_in_executor(
            None, self._test_openai_compatible_api, api_key, display_name, base_url, provider_name
        )
    elif provider_name == "google":
        # 获取厂家的 base_url
        db = await self._get_db()
        providers_collection = db.llm_providers
        provider_data = await providers_collection.find_one({"name": provider_name})
        base_url = provider_data.get("default_base_url") if provider_data else None
        return await asyncio.get_event_loop().run_in_executor(None, self._test_google_api, api_key, display_name, base_url)
```

```
        elif provider_name == "deepseek":
            return await asyncio.get_event_loop().run_in_executor(None, self._test_deepseek_api, api_key, display_name)
        elif provider_name == "dashscope":
            return await asyncio.get_event_loop().run_in_executor(None, self._test_dashscope_api, api_key, display_name)
        elif provider_name == "openrouter":
            return await asyncio.get_event_loop().run_in_executor(None, self._test_openrouter_api, api_key, display_name)
        elif provider_name == "openai":
            return await asyncio.get_event_loop().run_in_executor(None, self._test_openai_api, api_key, display_name)
        elif provider_name == "anthropic":
            return await asyncio.get_event_loop().run_in_executor(None, self._test_anthropic_api, api_key, display_name)
        elif provider_name == "qianfan":
            return await asyncio.get_event_loop().run_in_executor(None, self._test_qianfan_api, api_key, display_name)
        else:
            # 🔧 对于未知的自定义厂家，使用 OpenAI 兼容 API 测试
            logger.info(f"🔍 使用 OpenAI 兼容 API 测试自定义厂家: {provider_name}")
            # 获取厂家的 base_url
            db = await self._get_db()
            providers_collection = db.llm_providers
            provider_data = await providers_collection.find_one({"name": provider_name})
            base_url = provider_data.get("default_base_url") if provider_data else None
            if not base_url:
                return {
                    "success": False,
                    "message": f"自定义厂家 {display_name} 未配置 API 基础 URL"
                }
            return await asyncio.get_event_loop().run_in_executor(
                None, self._test_openai_compatible_api, api_key, display_name, base_url, provider_name
            )
    except Exception as e:
        return {
            "success": False,
            "message": f"{display_name} 连接测试失败: {str(e)}"
        }
def _test_google_api(self, api_key: str, display_name: str, base_url: str = None, model_name: str = None) -> dict:
    """测试Google AI API"""
    try:
        import requests
        # 如果没有指定模型，使用默认模型
        if not model_name:
            model_name = "gemini-2.0-flash-exp"
            logger.info(f"⚠️ 未指定模型，使用默认模型: {model_name}")
        logger.info(f"🔍 [Google AI 测试] 开始测试")
        logger.info(f"   display_name: {display_name}")
        logger.info(f"   model_name: {model_name}")
        logger.info(f"   base_url (原始): {base_url}")
        logger.info(f"   api_key 长度: {len(api_key) if api_key else 0}")
        # 使用配置的 base_url 或默认值
        if not base_url:
            base_url = "https://generativelanguage.googleapis.com/v1beta"
            logger.info(f"⚠️ base_url 为空，使用默认值: {base_url}")
        # 移除末尾的斜杠
        base_url = base_url.rstrip('/')
        logger.info(f"   base_url (去除斜杠): {base_url}")
        # 如果 base_url 以 /v1 结尾，替换为 /v1beta（Google AI 的正确端点）
        if base_url.endswith('/v1'):
            base_url = base_url[:-3] + '/v1beta'
            logger.info(f"✅ 将 /v1 替换为 /v1beta: {base_url}")
        # 构建完整的 API 端点（使用用户配置的模型）
        url = f"{base_url}/models/{model_name}:generateContent?key={api_key}"
        logger.info(f"🔗 [Google AI 测试] 最终请求 URL: {url.replace(api_key, '***')}")
        headers = {
            "Content-Type": "application/json"
        }
        # 🔧 增加 token 限制到 2000，避免思考模式消耗导致无输出
        data = {
            "contents": [{
                "parts": [{
                    "text": "Hello, please respond with 'OK' if you can read this."
                }]
            }],
            "generationConfig": {
                "maxOutputTokens": 2000,
                "temperature": 0.1
            }
        }
        response = requests.post(url, json=data, headers=headers, timeout=15)
        print(f"📥 [Google AI 测试] 响应状态码: {response.status_code}")
        if response.status_code == 200:
            # 打印完整的响应内容用于调试
            print(f"📥 [Google AI 测试] 响应内容（前1000字符）: {response.text[:1000]}")
            result = response.json()
            print(f"📥 [Google AI 测试] 解析后的 JSON 结构:")
            print(f"   - 顶层键: {list(result.keys())}")
            print(f"   - 是否包含 'candidates': {'candidates' in result}")
            if "candidates" in result:
                print(f"   - candidates 长度: {len(result['candidates'])}")
                if len(result['candidates']) > 0:
                    print(f"   - candidates[0] 的键: {list(result['candidates'][0].keys())}")
            if "candidates" in result and len(result["candidates"]) > 0:
                candidate = result["candidates"][0]
                print(f"📥 [Google AI 测试] candidate 结构: {candidate}")
                # 检查 finishReason
                finish_reason = candidate.get("finishReason", "")
                print(f"📥 [Google AI 测试] finishReason: {finish_reason}")
                if "content" in candidate:
                    content = candidate["content"]
                    # 检查是否有 parts
                    if "parts" in content and len(content["parts"]) > 0:
                        text = content["parts"][0].get("text", "")
                        print(f"📥 [Google AI 测试] 提取的文本: {text}")
                        if text and len(text.strip()) > 0:
                            return {
                                "success": True,
```

```
                    "message": f"{display_name} API连接测试成功"
                }
            else:
                print(f"❌ [Google AI 测试] 文本为空")
                return {
                    "success": False,
                    "message": f"{display_name} API响应内容为空"
                }
        else:
            # content 中没有 parts，可能是因为 MAX_TOKENS 或其他原因
            print(f"❌ [Google AI 测试] content 中没有 parts")
            print(f"   content 的键: {list(content.keys())}")
            if finish_reason == "MAX_TOKENS":
                return {
                    "success": False,
                    "message": f"{display_name} API响应被截断（MAX_TOKENS），请增加 maxOutputTokens 配置"
                }
            else:
                return {
                    "success": False,
                    "message": f"{display_name} API响应格式异常（缺少 parts，finishReason: {finish_reason}）"
                }
    else:
        print(f"❌ [Google AI 测试] candidate 中缺少 'content'")
        print(f"   candidate 的键: {list(candidate.keys())}")
        return {
            "success": False,
            "message": f"{display_name} API响应格式异常（缺少 content）"
        }
else:
    print(f"❌ [Google AI 测试] 缺少 candidates 或 candidates 为空")
    return {
        "success": False,
        "message": f"{display_name} API无有效候选响应"
    }
elif response.status_code == 400:
    print(f"❌ [Google AI 测试] 400 错误，响应内容: {response.text[:500]}")
    try:
        error_detail = response.json()
        error_msg = error_detail.get("error", {}).get("message", "未知错误")
        return {
            "success": False,
            "message": f"{display_name} API请求错误: {error_msg}"
        }
    except:
        return {
            "success": False,
            "message": f"{display_name} API请求格式错误"
        }
elif response.status_code == 403:
    print(f"❌ [Google AI 测试] 403 错误，响应内容: {response.text[:500]}")
    return {
        "success": False,
        "message": f"{display_name} API密钥无效或权限不足"
    }
elif response.status_code == 503:
    print(f"❌ [Google AI 测试] 503 错误，响应内容: {response.text[:500]}")
    try:
        error_detail = response.json()
        error_code = error_detail.get("code", "")
        error_msg = error_detail.get("message", "服务暂时不可用")
        if error_code == "NO_KEYS_AVAILABLE":
            return {
                "success": False,
                "message": f"{display_name} 中转服务暂时无可用密钥，请稍后重试或联系中转服务提供商"
            }
        else:
            return {
                "success": False,
                "message": f"{display_name} 服务暂时不可用: {error_msg}"
            }
    except:
        return {
            "success": False,
            "message": f"{display_name} 服务暂时不可用 (HTTP 503)"
        }
else:
    print(f"❌ [Google AI 测试] {response.status_code} 错误，响应内容: {response.text[:500]}")
    return {
        "success": False,
        "message": f"{display_name} API测试失败: HTTP {response.status_code}"
    }
except Exception as e:
    return {
        "success": False,
        "message": f"{display_name} API测试异常: {str(e)}"
    }
def _test_deepseek_api(self, api_key: str, display_name: str, model_name: str = None) -> dict:
    """测试DeepSeek API"""
    try:
        import requests
        # 如果没有指定模型，使用默认模型
        if not model_name:
            model_name = "deepseek-chat"
            logger.info(f"⚠️ 未指定模型，使用默认模型: {model_name}")
        logger.info(f"🔍 [DeepSeek 测试] 使用模型: {model_name}")
        url = "https://api.deepseek.com/chat/completions"
        headers = {
            "Content-Type": "application/json",
            "Authorization": f"Bearer {api_key}"
        }
        data = {
            "model": model_name,
            "messages": [
```

```
                {"role": "user", "content": "你好，请简单介绍一下你自己。"}
            ],
            "max_tokens": 50,
            "temperature": 0.1
        }
        response = requests.post(url, json=data, headers=headers, timeout=10)
        if response.status_code == 200:
            result = response.json()
            if "choices" in result and len(result["choices"]) > 0:
                content = result["choices"][0]["message"]["content"]
                if content and len(content.strip()) > 0:
                    return {
                        "success": True,
                        "message": f"{display_name} API连接测试成功"
                    }
                else:
                    return {
                        "success": False,
                        "message": f"{display_name} API响应为空"
                    }
            else:
                return {
                    "success": False,
                    "message": f"{display_name} API响应格式异常"
                }
        else:
            return {
                "success": False,
                "message": f"{display_name} API测试失败: HTTP {response.status_code}"
            }
    except Exception as e:
        return {
            "success": False,
            "message": f"{display_name} API测试异常: {str(e)}"
        }
def _test_dashscope_api(self, api_key: str, display_name: str, model_name: str = None) -> dict:
    """测试阿里云百炼API"""
    try:
        import requests
        # 如果没有指定模型，使用默认模型
        if not model_name:
            model_name = "qwen-turbo"
            logger.info(f"⚠️ 未指定模型，使用默认模型: {model_name}")
        logger.info(f"🔍 [DashScope 测试] 使用模型: {model_name}")
        # 使用阿里云百炼的OpenAI兼容接口
        url = "https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions"
        headers = {
            "Content-Type": "application/json",
            "Authorization": f"Bearer {api_key}"
        }
        data = {
            "model": model_name,
            "messages": [
                {"role": "user", "content": "你好，请简单介绍一下你自己。"}
            ],
            "max_tokens": 50,
            "temperature": 0.1
        }
        response = requests.post(url, json=data, headers=headers, timeout=10)
        if response.status_code == 200:
            result = response.json()
            if "choices" in result and len(result["choices"]) > 0:
                content = result["choices"][0]["message"]["content"]
                if content and len(content.strip()) > 0:
                    return {
                        "success": True,
                        "message": f"{display_name} API连接测试成功"
                    }
                else:
                    return {
                        "success": False,
                        "message": f"{display_name} API响应为空"
                    }
            else:
                return {
                    "success": False,
                    "message": f"{display_name} API响应格式异常"
                }
        else:
            return {
                "success": False,
                "message": f"{display_name} API测试失败: HTTP {response.status_code}"
            }
    except Exception as e:
        return {
            "success": False,
            "message": f"{display_name} API测试异常: {str(e)}"
        }
def _test_openrouter_api(self, api_key: str, display_name: str) -> dict:
    """测试OpenRouter API"""
    try:
        import requests
        url = "https://openrouter.ai/api/v1/chat/completions"
        headers = {
            "Content-Type": "application/json",
            "Authorization": f"Bearer {api_key}",
            "HTTP-Referer": "https://tradingagents.cn",  # OpenRouter要求
            "X-Title": "TradingAgents-CN"
        }
        data = {
            "model": "meta-llama/llama-3.2-3b-instruct:free",  # 使用免费模型
            "messages": [
                {"role": "user", "content": "你好，请简单介绍一下你自己。"}
```

```
            ],
            "max_tokens": 50,
            "temperature": 0.1
        }
        response = requests.post(url, json=data, headers=headers, timeout=15)
        if response.status_code == 200:
            result = response.json()
            if "choices" in result and len(result["choices"]) > 0:
                content = result["choices"][0]["message"]["content"]
                if content and len(content.strip()) > 0:
                    return {
                        "success": True,
                        "message": f"{display_name} API连接测试成功"
                    }
                else:
                    return {
                        "success": False,
                        "message": f"{display_name} API响应为空"
                    }
            else:
                return {
                    "success": False,
                    "message": f"{display_name} API响应格式异常"
                }
        else:
            return {
                "success": False,
                "message": f"{display_name} API测试失败: HTTP {response.status_code}"
            }
    except Exception as e:
        return {
            "success": False,
            "message": f"{display_name} API测试异常: {str(e)}"
        }
def _test_openai_api(self, api_key: str, display_name: str) -> dict:
    """测试OpenAI API"""
    try:
        import requests
        url = "https://api.openai.com/v1/chat/completions"
        headers = {
            "Content-Type": "application/json",
            "Authorization": f"Bearer {api_key}"
        }
        data = {
            "model": "gpt-3.5-turbo",
            "messages": [
                {"role": "user", "content": "你好，请简单介绍一下你自己。"}
            ],
            "max_tokens": 50,
            "temperature": 0.1
        }
        response = requests.post(url, json=data, headers=headers, timeout=10)
        if response.status_code == 200:
            result = response.json()
            if "choices" in result and len(result["choices"]) > 0:
                content = result["choices"][0]["message"]["content"]
                if content and len(content.strip()) > 0:
                    return {
                        "success": True,
                        "message": f"{display_name} API连接测试成功"
                    }
                else:
                    return {
                        "success": False,
                        "message": f"{display_name} API响应为空"
                    }
            else:
                return {
                    "success": False,
                    "message": f"{display_name} API响应格式异常"
                }
        else:
            return {
                "success": False,
                "message": f"{display_name} API测试失败: HTTP {response.status_code}"
            }
    except Exception as e:
        return {
            "success": False,
            "message": f"{display_name} API测试异常: {str(e)}"
        }
def _test_anthropic_api(self, api_key: str, display_name: str) -> dict:
    """测试Anthropic API"""
    try:
        import requests
        url = "https://api.anthropic.com/v1/messages"
        headers = {
            "Content-Type": "application/json",
            "x-api-key": api_key,
            "anthropic-version": "2023-06-01"
        }
        data = {
            "model": "claude-3-haiku-20240307",
            "max_tokens": 50,
            "messages": [
                {"role": "user", "content": "你好，请简单介绍一下你自己。"}
            ]
        }
        response = requests.post(url, json=data, headers=headers, timeout=10)
        if response.status_code == 200:
            result = response.json()
            if "content" in result and len(result["content"]) > 0:
                content = result["content"][0]["text"]
```

```
            if content and len(content.strip()) > 0:
                return {
                    "success": True,
                    "message": f"{display_name} API连接测试成功"
                }
            else:
                return {
                    "success": False,
                    "message": f"{display_name} API响应为空"
                }
        else:
            return {
                "success": False,
                "message": f"{display_name} API响应格式异常"
            }
    else:
        return {
            "success": False,
            "message": f"{display_name} API测试失败: HTTP {response.status_code}"
        }
  except Exception as e:
    return {
        "success": False,
        "message": f"{display_name} API测试异常: {str(e)}"
    }
def _test_qianfan_api(self, api_key: str, display_name: str) -> dict:
  """测试百度千帆API"""
  try:
    import requests
    # 千帆新一代API使用OpenAI兼容接口
    url = "https://qianfan.baidubce.com/v2/chat/completions"
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {api_key}"
    }
    data = {
        "model": "ernie-3.5-8k",
        "messages": [
            {"role": "user", "content": "你好，请简单介绍一下你自己。"}
        ],
        "max_tokens": 50,
        "temperature": 0.1
    }
    response = requests.post(url, json=data, headers=headers, timeout=15)
    if response.status_code == 200:
        result = response.json()
        if "choices" in result and len(result["choices"]) > 0:
            content = result["choices"][0]["message"]["content"]
            if content and len(content.strip()) > 0:
                return {
                    "success": True,
                    "message": f"{display_name} API连接测试成功"
                }
            else:
                return {
                    "success": False,
                    "message": f"{display_name} API响应为空"
                }
        else:
            return {
                "success": False,
                "message": f"{display_name} API响应格式异常"
            }
    elif response.status_code == 401:
        return {
            "success": False,
            "message": f"{display_name} API密钥无效或已过期"
        }
    elif response.status_code == 403:
        return {
            "success": False,
            "message": f"{display_name} API权限不足或配额已用完"
        }
    else:
        try:
            error_detail = response.json()
            error_msg = error_detail.get("error", {}).get("message", f"HTTP {response.status_code}")
            return {
                "success": False,
                "message": f"{display_name} API测试失败: {error_msg}"
            }
        except:
            return {
                "success": False,
                "message": f"{display_name} API测试失败: HTTP {response.status_code}"
            }
  except Exception as e:
    return {
        "success": False,
        "message": f"{display_name} API测试异常: {str(e)}"
    }
async def fetch_provider_models(self, provider_id: str) -> dict:
  """从厂家 API 获取模型列表"""
  try:
    print(f"🔍 获取厂家模型列表 - provider_id: {provider_id}")
    db = await self._get_db()
    providers_collection = db.llm_providers
    # 兼容处理：尝试 ObjectId 和字符串两种类型
    from bson import ObjectId
    provider_data = None
    try:
        provider_data = await providers_collection.find_one({"_id": ObjectId(provider_id)})
    except Exception:
        pass
```

```
        if not provider_data:
            provider_data = await providers_collection.find_one({"_id": provider_id})
        if not provider_data:
            return {
                "success": False,
                "message": f"厂家不存在 (ID: {provider_id})"
            }
        provider_name = provider_data.get("name")
        api_key = provider_data.get("api_key")
        base_url = provider_data.get("default_base_url")
        display_name = provider_data.get("display_name", provider_name)
        # 💧 判断数据库中的 API Key 是否有效
        if not self._is_valid_api_key(api_key):
            # 数据库中的 Key 无效，尝试从环境变量读取
            env_api_key = self._get_env_api_key(provider_name)
            if env_api_key:
                api_key = env_api_key
                print(f"✅ 数据库配置无效，从环境变量读取到 {display_name} 的 API Key")
            else:
                # 某些聚合平台（如 OpenRouter）的 /models 端点不需要 API Key
                print(f"⚠️  {display_name} 未配置有效的API密钥，尝试无认证访问")
        else:
            print(f"✅ 使用数据库配置的 {display_name} API密钥")
        if not base_url:
            return {
                "success": False,
                "message": f"{display_name} 未配置 API 基础地址 (default_base_url)"
            }
        # 调用 OpenAI 兼容的 /v1/models 端点
        import asyncio
        result = await asyncio.get_event_loop().run_in_executor(
            None, self._fetch_models_from_api, api_key, base_url, display_name
        )
        return result
    except Exception as e:
        print(f"获取模型列表失败: {e}")
        import traceback
        traceback.print_exc()
        return {
            "success": False,
            "message": f"获取模型列表失败: {str(e)}"
        }
def _fetch_models_from_api(self, api_key: str, base_url: str, display_name: str) -> dict:
    """从 API 获取模型列表"""
    try:
        import requests
        # 🔧 智能版本号处理：只有在没有版本号的情况下才添加 /v1
        # 避免对已有版本号的URL（如智谱AI的 /v4）重复添加 /v1
        import re
        base_url = base_url.rstrip("/")
        if not re.search(r'/v\d+$', base_url):
            # URL末尾没有版本号，添加 /v1（OpenAI标准）
            base_url = base_url + "/v1"
            logger.info(f"  [获取模型列表] 添加 /v1 版本号: {base_url}")
        else:
            # URL已包含版本号（如 /v4），不添加
            logger.info(f"  [获取模型列表] 检测到已有版本号，保持原样: {base_url}")
        url = f"{base_url}/models"
        # 构建请求头
        headers = {}
        if api_key:
            headers["Authorization"] = f"Bearer {api_key}"
            print(f"🔍 请求 URL: {url} (with API Key)")
        else:
            print(f"🔍 请求 URL: {url} (without API Key)")
        response = requests.get(url, headers=headers, timeout=15)
        print(f"📊 响应状态码: {response.status_code}")
        print(f"📊 响应内容: {response.text[:500]}...")
        if response.status_code == 200:
            result = response.json()
            print(f"📊 响应 JSON 结构: {list(result.keys())}")
            if "data" in result and isinstance(result["data"], list):
                all_models = result["data"]
                print(f"📊 API 返回 {len(all_models)} 个模型")
                # 打印前几个模型的完整结构（用于调试价格字段）
                if all_models:
                    print(f"🔍 第一个模型的完整结构:")
                    import json
                    print(json.dumps(all_models[0], indent=2, ensure_ascii=False))
                # 打印所有 Anthropic 模型（用于调试）
                anthropic_models = [m for m in all_models if "anthropic" in m.get("id", "").lower()]
                if anthropic_models:
                    print(f"🔍 Anthropic 模型列表 ({len(anthropic_models)} 个):")
                    for m in anthropic_models[:20]:  # 只打印前 20 个
                        print(f"  - {m.get('id')}")
                # 过滤：只保留主流大厂的常用模型
                filtered_models = self._filter_popular_models(all_models)
                print(f"✅ 过滤后保留 {len(filtered_models)} 个常用模型")
                # 转换模型格式，包含价格信息
                formatted_models = self._format_models_with_pricing(filtered_models)
                return {
                    "success": True,
                    "models": formatted_models,
                    "message": f"成功获取 {len(formatted_models)} 个常用模型（已过滤）"
                }
            else:
                print(f"❌ 响应格式异常，期望 'data' 字段为列表")
                return {
                    "success": False,
                    "message": f"{display_name} API 响应格式异常（缺少 data 字段或格式不正确）"
                }
        elif response.status_code == 401:
            return {
```

```
            "success": False,
            "message": f"{display_name} API密钥无效或已过期"
        }
    elif response.status_code == 403:
        return {
            "success": False,
            "message": f"{display_name} API权限不足"
        }
    else:
        try:
            error_detail = response.json()
            error_msg = error_detail.get("error", {}).get("message", f"HTTP {response.status_code}")
            print(f"❌ API 错误: {error_msg}")
            return {
                "success": False,
                "message": f"{display_name} API请求失败: {error_msg}"
            }
        except:
            print(f"❌ HTTP 错误: {response.status_code}")
            return {
                "success": False,
                "message": f"{display_name} API请求失败: HTTP {response.status_code}, 响应: {response.text[:200]}"
            }
  except Exception as e:
    print(f"❌ 异常: {e}")
    import traceback
    traceback.print_exc()
    return {
        "success": False,
        "message": f"{display_name} API请求异常: {str(e)}"
    }
def _format_models_with_pricing(self, models: list) -> list:
  """
  格式化模型列表，包含价格信息
  支持多种价格格式：
  1. OpenRouter: pricing.prompt/completion (USD per token)
  2. 302.ai: price.prompt/completion 或 price.input/output
  3. 其他: 可能没有价格信息
  """
  formatted = []
  for model in models:
    model_id = model.get("id", "")
    model_name = model.get("name", model_id)
    # 尝试从多个字段获取价格信息
    input_price_per_1k = None
    output_price_per_1k = None
    # 方式1：OpenRouter 格式 (pricing.prompt/completion)
    pricing = model.get("pricing", {})
    if pricing:
        prompt_price = pricing.get("prompt", "0")  # USD per token
        completion_price = pricing.get("completion", "0")  # USD per token
        try:
            if prompt_price and float(prompt_price) > 0:
                input_price_per_1k = float(prompt_price) * 1000
            if completion_price and float(completion_price) > 0:
                output_price_per_1k = float(completion_price) * 1000
        except (ValueError, TypeError):
            pass
    # 方式2：302.ai 格式 (price.prompt/completion 或 price.input/output)
    if not input_price_per_1k and not output_price_per_1k:
        price = model.get("price", {})
        if price and isinstance(price, dict):
            # 尝试 prompt/completion 字段
            prompt_price = price.get("prompt") or price.get("input")
            completion_price = price.get("completion") or price.get("output")
            try:
                if prompt_price and float(prompt_price) > 0:
                    # 假设是 per token，转换为 per 1K tokens
                    input_price_per_1k = float(prompt_price) * 1000
                if completion_price and float(completion_price) > 0:
                    output_price_per_1k = float(completion_price) * 1000
            except (ValueError, TypeError):
                pass
    # 获取上下文长度
    context_length = model.get("context_length")
    if not context_length:
        # 尝试从 top_provider 获取
        top_provider = model.get("top_provider", {})
        context_length = top_provider.get("context_length")
    # 如果还是没有，尝试从 max_completion_tokens 推断
    if not context_length:
        max_tokens = model.get("max_completion_tokens")
        if max_tokens and max_tokens > 0:
            # 通常上下文长度是最大输出的 4-8 倍
            context_length = max_tokens * 4
    formatted_model = {
        "id": model_id,
        "name": model_name,
        "context_length": context_length,
        "input_price_per_1k": input_price_per_1k,
        "output_price_per_1k": output_price_per_1k,
    }
    formatted.append(formatted_model)
    # 打印价格信息（用于调试）
    if input_price_per_1k or output_price_per_1k:
        print(f"💰 {model_id}: 输入=${input_price_per_1k:.6f}/1K, 输出=${output_price_per_1k:.6f}/1K")
  return formatted
def _filter_popular_models(self, models: list) -> list:
  """过滤模型列表，只保留主流大厂的常用模型"""
  import re
  # 只保留三大厂：OpenAI、Anthropic、Google
  popular_providers = [
    "openai",     # OpenAI
```

```
        "anthropic",   # Anthropic
        "google",      # Google
    ]
    # 常见模型名称前缀（用于识别不带厂商前缀的模型）
    model_prefixes = {
        "gpt-": "openai",          # gpt-3.5-turbo, gpt-4, gpt-4o
        "o1-": "openai",           # o1-preview, o1-mini
        "claude-": "anthropic",    # claude-3-opus, claude-3-sonnet
        "gemini-": "google",       # gemini-pro, gemini-1.5-pro
        "gemini": "google",        # gemini (不带连字符)
    }
    # 排除的关键词
    exclude_keywords = [
        "preview",
        "experimental",
        "alpha",
        "beta",
        "free",
        "extended",
        "nitro",
        ":free",
        ":extended",
        "online",  # 排除带在线搜索的版本
        "instruct",  # 排除 instruct 版本
    ]
    # 日期格式正则表达式（匹配 2024-05-13 这种格式）
    date_pattern = re.compile(r'\d{4}-\d{2}-\d{2}')
    filtered = []
    for model in models:
        model_id = model.get("id", "").lower()
        model_name = model.get("name", "").lower()
        # 检查是否属于三大厂
        # 方式1：模型ID中包含厂商名称（如 openai/gpt-4）
        is_popular_provider = any(provider in model_id for provider in popular_providers)
        # 方式2：模型ID以常见前缀开头（如 gpt-4, claude-3-sonnet）
        if not is_popular_provider:
            for prefix, provider in model_prefixes.items():
                if model_id.startswith(prefix):
                    is_popular_provider = True
                    print(f"🔍 识别模型前缀: {model_id} -> {provider}")
                    break
        if not is_popular_provider:
            continue
        # 检查是否包含日期（排除带日期的旧版本）
        if date_pattern.search(model_id):
            print(f"⏭️  跳过带日期的旧版本: {model_id}")
            continue
        # 检查是否包含排除关键词
        has_exclude_keyword = any(keyword in model_id or keyword in model_name for keyword in exclude_keywords)
        if has_exclude_keyword:
            print(f"⏭️  跳过排除关键词: {model_id}")
            continue
        # 保留该模型
        print(f"✅ 保留模型: {model_id}")
        filtered.append(model)
    return filtered
def _test_openai_compatible_api(self, api_key: str, display_name: str, base_url: str = None, provider_name: str = None) -> dict:
    """测试 OpenAI 兼容 API（用于聚合渠道和自定义厂家）"""
    try:
        import requests
        # 如果没有提供 base_url，使用默认值
        if not base_url:
            return {
                "success": False,
                "message": f"{display_name} 未配置 API 基础地址 (default_base_url)"
            }
        # 🔧 智能版本号处理：只有在没有版本号的情况下才添加 /v1
        # 避免对已有版本号的URL（如智谱AI的 /v4）重复添加 /v1
        import re
        logger.info(f"   [测试API] 原始 base_url: {base_url}")
        base_url = base_url.rstrip("/")
        logger.info(f"   [测试API] 去除斜杠后: {base_url}")
        if not re.search(r'/v\d+$', base_url):
            # URL末尾没有版本号，添加 /v1（OpenAI标准）
            base_url = base_url + "/v1"
            logger.info(f"   [测试API] 添加 /v1 版本号: {base_url}")
        else:
            # URL已包含版本号（如 /v4），不添加
            logger.info(f"   [测试API] 检测到已有版本号，保持原样: {base_url}")
        url = f"{base_url}/chat/completions"
        logger.info(f"   [测试API] 最终请求URL: {url}")
        headers = {
            "Content-Type": "application/json",
            "Authorization": f"Bearer {api_key}"
        }
        # 🎯 根据不同厂家选择合适的测试模型
        test_model = "gpt-3.5-turbo"  # 默认模型
        if provider_name == "siliconflow":
            # 硅基流动使用免费的 Qwen 模型进行测试
            test_model = "Qwen/Qwen2.5-7B-Instruct"
            logger.info(f"🔍 硅基流动使用测试模型: {test_model}")
        elif provider_name == "zhipu":
            # 智谱AI使用 glm-4 模型进行测试
            test_model = "glm-4"
            logger.info(f"🔍 智谱AI使用测试模型: {test_model}")
        # 使用一个通用的模型名称进行测试
        # 聚合渠道通常支持多种模型，这里使用 gpt-3.5-turbo 作为测试
        data = {
            "model": test_model,
            "messages": [
                {"role": "user", "content": "Hello, please respond with 'OK' if you can read this."}
            ],
            "max_tokens": 200,  # 增加到200，给推理模型（如o1/gpt-5）足够空间
```

```
            "temperature": 0.1
        }
        response = requests.post(url, json=data, headers=headers, timeout=15)
        if response.status_code == 200:
            result = response.json()
            if "choices" in result and len(result["choices"]) > 0:
                content = result["choices"][0]["message"]["content"]
                if content and len(content.strip()) > 0:
                    return {
                        "success": True,
                        "message": f"{display_name} API连接测试成功"
                    }
                else:
                    return {
                        "success": False,
                        "message": f"{display_name} API响应为空"
                    }
            else:
                return {
                    "success": False,
                    "message": f"{display_name} API响应格式异常"
                }
        elif response.status_code == 401:
            return {
                "success": False,
                "message": f"{display_name} API密钥无效或已过期"
            }
        elif response.status_code == 403:
            return {
                "success": False,
                "message": f"{display_name} API权限不足或配额已用完"
            }
        else:
            try:
                error_detail = response.json()
                error_msg = error_detail.get("error", {}).get("message", f"HTTP {response.status_code}")
                logger.error(f"❌ [{display_name}] API测试失败")
                logger.error(f"   请求URL: {url}")
                logger.error(f"   状态码: {response.status_code}")
                logger.error(f"   错误详情: {error_detail}")
                return {
                    "success": False,
                    "message": f"{display_name} API测试失败: {error_msg}"
                }
            except:
                logger.error(f"❌ [{display_name}] API测试失败")
                logger.error(f"   请求URL: {url}")
                logger.error(f"   状态码: {response.status_code}")
                logger.error(f"   响应内容: {response.text[:500]}")
                return {
                    "success": False,
                    "message": f"{display_name} API测试失败: HTTP {response.status_code}"
                }
    except Exception as e:
        return {
            "success": False,
            "message": f"{display_name} API测试异常: {str(e)}"
        }
# 创建全局实例
config_service = ConfigService()
```

FILE: app/services/data_consistency_checker.py

```
"""
数据一致性检查和处理服务
处理多数据源之间的数据不一致性问题
"""
import logging
import pandas as pd
from typing import Dict, List, Optional, Tuple, Any
from dataclasses import dataclass
from datetime import datetime
import numpy as np
logger = logging.getLogger(__name__)
@dataclass
class DataConsistencyResult:
    """数据一致性检查结果"""
    is_consistent: bool
    primary_source: str
    secondary_source: str
    differences: Dict[str, Any]
    confidence_score: float
    recommended_action: str
    details: Dict[str, Any]
@dataclass
class FinancialMetricComparison:
    """财务指标比较结果"""
    metric_name: str
    primary_value: Optional[float]
    secondary_value: Optional[float]
    difference_pct: Optional[float]
    is_significant: bool
    tolerance: float
class DataConsistencyChecker:
    """数据一致性检查器"""

    def __init__(self):
        # 设置各种指标的容忍度阈值
        self.tolerance_thresholds = {
            'pe': 0.05,      # PE允许5%差异
            'pb': 0.05,      # PB允许5%差异
            'total_mv': 0.02, # 市值允许2%差异
            'price': 0.01,   # 股价允许1%差异
```

```
        'volume': 0.10,  # 成交量允许10%差异
        'turnover_rate': 0.05  # 换手率允许5%差异
    }

    # 关键指标权重（用于计算置信度分数）
    self.metric_weights = {
        'pe': 0.25,
        'pb': 0.25,
        'total_mv': 0.20,
        'price': 0.15,
        'volume': 0.10,
        'turnover_rate': 0.05
    }

def check_daily_basic_consistency(
    self,
    primary_data: pd.DataFrame,
    secondary_data: pd.DataFrame,
    primary_source: str,
    secondary_source: str
) -> DataConsistencyResult:
    """
    检查daily_basic数据的一致性

    Args:
        primary_data: 主数据源数据
        secondary_data: 次数据源数据
        primary_source: 主数据源名称
        secondary_source: 次数据源名称
    """
    try:
        logger.info(f"🔍 检查数据一致性: {primary_source} vs {secondary_source}")

        # 1. 基础检查
        if primary_data.empty or secondary_data.empty:
            return DataConsistencyResult(
                is_consistent=False,
                primary_source=primary_source,
                secondary_source=secondary_source,
                differences={'error': 'One or both datasets are empty'},
                confidence_score=0.0,
                recommended_action='use_primary_only',
                details={'reason': 'Empty dataset detected'}
            )

        # 2. 股票代码匹配
        common_stocks = self._find_common_stocks(primary_data, secondary_data)
        if len(common_stocks) == 0:
            return DataConsistencyResult(
                is_consistent=False,
                primary_source=primary_source,
                secondary_source=secondary_source,
                differences={'error': 'No common stocks found'},
                confidence_score=0.0,
                recommended_action='use_primary_only',
                details={'reason': 'No overlapping stocks'}
            )

        logger.info(f"📊 找到{len(common_stocks)}只共同股票进行比较")

        # 3. 逐指标比较
        metric_comparisons = []
        for metric in ['pe', 'pb', 'total_mv']:
            comparison = self._compare_metric(
                primary_data, secondary_data, common_stocks, metric
            )
            if comparison:
                metric_comparisons.append(comparison)

        # 4. 计算整体一致性
        consistency_result = self._calculate_overall_consistency(
            metric_comparisons, primary_source, secondary_source
        )

        return consistency_result

    except Exception as e:
        logger.error(f"❌ 数据一致性检查失败: {e}")
        return DataConsistencyResult(
            is_consistent=False,
            primary_source=primary_source,
            secondary_source=secondary_source,
            differences={'error': str(e)},
            confidence_score=0.0,
            recommended_action='use_primary_only',
            details={'exception': str(e)}
        )

def _find_common_stocks(self, df1: pd.DataFrame, df2: pd.DataFrame) -> List[str]:
    """找到两个数据集中的共同股票"""
    # 尝试不同的股票代码列名
    code_cols = ['ts_code', 'symbol', 'code', 'stock_code']

    df1_codes = set()
    df2_codes = set()

    for col in code_cols:
        if col in df1.columns:
            df1_codes.update(df1[col].dropna().astype(str).tolist())
        if col in df2.columns:
            df2_codes.update(df2[col].dropna().astype(str).tolist())

    return list(df1_codes.intersection(df2_codes))

def _compare_metric(
    self,
```

```
    df1: pd.DataFrame,
    df2: pd.DataFrame,
    common_stocks: List[str],
    metric: str
) -> Optional[FinancialMetricComparison]:
    """比较特定指标"""
    try:
        if metric not in df1.columns or metric not in df2.columns:
            return None

        # 获取共同股票的指标值
        df1_values = []
        df2_values = []

        for stock in common_stocks[:100]:  # 限制比较数量
            val1 = self._get_stock_metric_value(df1, stock, metric)
            val2 = self._get_stock_metric_value(df2, stock, metric)

            if val1 is not None and val2 is not None:
                df1_values.append(val1)
                df2_values.append(val2)

        if len(df1_values) == 0:
            return None

        # 计算平均值和差异
        avg1 = np.mean(df1_values)
        avg2 = np.mean(df2_values)

        if avg1 != 0:
            diff_pct = abs(avg2 - avg1) / abs(avg1)
        else:
            diff_pct = float('inf') if avg2 != 0 else 0

        tolerance = self.tolerance_thresholds.get(metric, 0.1)
        is_significant = diff_pct > tolerance

        return FinancialMetricComparison(
            metric_name=metric,
            primary_value=avg1,
            secondary_value=avg2,
            difference_pct=diff_pct,
            is_significant=is_significant,
            tolerance=tolerance
        )

    except Exception as e:
        logger.warning(f"⚠️ 比较指标{metric}失败: {e}")
        return None

def _get_stock_metric_value(self, df: pd.DataFrame, stock_code: str, metric: str) -> Optional[float]:
    """获取特定股票的指标值"""
    try:
        # 尝试不同的匹配方式
        for code_col in ['ts_code', 'symbol', 'code']:
            if code_col in df.columns:
                mask = df[code_col].astype(str) == stock_code
                if mask.any():
                    value = df.loc[mask, metric].iloc[0]
                    if pd.notna(value) and value != 0:
                        return float(value)
        return None
    except:
        return None

def _calculate_overall_consistency(
    self,
    comparisons: List[FinancialMetricComparison],
    primary_source: str,
    secondary_source: str
) -> DataConsistencyResult:
    """计算整体一致性结果"""
    if not comparisons:
        return DataConsistencyResult(
            is_consistent=False,
            primary_source=primary_source,
            secondary_source=secondary_source,
            differences={'error': 'No valid metric comparisons'},
            confidence_score=0.0,
            recommended_action='use_primary_only',
            details={'reason': 'No comparable metrics'}
        )

    # 计算加权置信度分数
    total_weight = 0
    weighted_score = 0
    differences = {}

    for comp in comparisons:
        weight = self.metric_weights.get(comp.metric_name, 0.1)
        total_weight += weight

        # 一致性分数：差异越小分数越高
        if comp.difference_pct is not None and comp.difference_pct != float('inf'):
            consistency_score = max(0, 1 - (comp.difference_pct / comp.tolerance))
        else:
            consistency_score = 0

        weighted_score += weight * consistency_score

        # 记录差异
        differences[comp.metric_name] = {
            'primary_value': comp.primary_value,
            'secondary_value': comp.secondary_value,
            'difference_pct': comp.difference_pct,
            'is_significant': comp.is_significant,
```

```
            'tolerance': comp.tolerance
        }

    confidence_score = weighted_score / total_weight if total_weight > 0 else 0

    # 判断整体一致性
    significant_differences = sum(1 for comp in comparisons if comp.is_significant)
    is_consistent = significant_differences <= len(comparisons) * 0.3  # 允许30%的指标有显著差异

    # 推荐行动
    if confidence_score > 0.8:
        recommended_action = 'use_either'  # 数据高度一致，可以使用任一数据源
    elif confidence_score > 0.6:
        recommended_action = 'use_primary_with_warning'  # 使用主数据源但发出警告
    elif confidence_score > 0.3:
        recommended_action = 'use_primary_only'  # 仅使用主数据源
    else:
        recommended_action = 'investigate_sources'  # 需要调查数据源问题

    return DataConsistencyResult(
        is_consistent=is_consistent,
        primary_source=primary_source,
        secondary_source=secondary_source,
        differences=differences,
        confidence_score=confidence_score,
        recommended_action=recommended_action,
        details={
            'total_comparisons': len(comparisons),
            'significant_differences': significant_differences,
            'consistency_threshold': 0.3
        }
    )
def resolve_data_conflicts(
    self,
    primary_data: pd.DataFrame,
    secondary_data: pd.DataFrame,
    consistency_result: DataConsistencyResult
) -> Tuple[pd.DataFrame, str]:
    """
    根据一致性检查结果解决数据冲突

    Returns:
        Tuple[pd.DataFrame, str]: (最终数据, 解决策略说明)
    """
    action = consistency_result.recommended_action

    if action == 'use_either':
        logger.info("☑ 数据高度一致，使用主数据源")
        return primary_data, "数据源高度一致，使用主数据源"

    elif action == 'use_primary_with_warning':
        logger.warning("⚠ 数据存在差异但在可接受范围内，使用主数据源")
        return primary_data, f"数据存在轻微差异（置信度: {consistency_result.confidence_score:.2f}），使用主数据源"

    elif action == 'use_primary_only':
        logger.warning("⚠ 数据差异较大，仅使用主数据源")
        return primary_data, f"数据差异显著（置信度: {consistency_result.confidence_score:.2f}），仅使用主数据源"

    else:  # investigate_sources
        logger.error("❌ 数据源存在严重问题，需要人工调查")
        return primary_data, f"数据源存在严重不一致（置信度: {consistency_result.confidence_score:.2f}），建议检查数据源"
```

FILE: app/services/data_sources/__init__.py

```
"""
Data sources subpackage.
Expose adapters and manager for backward-compatible imports.
"""
from .base import DataSourceAdapter
from .tushare_adapter import TushareAdapter
from .akshare_adapter import AKShareAdapter
from .baostock_adapter import BaoStockAdapter
from .manager import DataSourceManager
```

FILE: app/services/data_sources/akshare_adapter.py

```
"""
AKShare data source adapter
"""
from typing import Optional, Dict
import logging
from datetime import datetime, timedelta
import pandas as pd
from .base import DataSourceAdapter
logger = logging.getLogger(__name__)
class AKShareAdapter(DataSourceAdapter):
    """AKShare数据源适配器"""
    def __init__(self):
        super().__init__()  # 调用父类初始化
    @property
    def name(self) -> str:
        return "akshare"
    def _get_default_priority(self) -> int:
        return 2  # 数字越大优先级越高
    def is_available(self) -> bool:
        """检查AKShare是否可用"""
        try:
            import akshare as ak  # noqa: F401
            return True
        except ImportError:
            return False
```

```
def get_stock_list(self) -> Optional[pd.DataFrame]:
    """获取股票列表（使用 AKShare 的 stock_info_a_code_name 接口获取真实股票名称）"""
    if not self.is_available():
        return None
    try:
        import akshare as ak
        logger.info("AKShare: Fetching stock list with real names from stock_info_a_code_name()...")
        # 使用 AKShare 的 stock_info_a_code_name 接口获取股票代码和名称
        df = ak.stock_info_a_code_name()
        if df is None or df.empty:
            logger.warning("AKShare: stock_info_a_code_name() returned empty data")
            return None
        # 标准化列名（AKShare 返回的列名可能是中文）
        # 通常返回的列：code（代码）、name（名称）
        df = df.rename(columns={
            'code': 'symbol',
            '代码': 'symbol',
            'name': 'name',
            '名称': 'name'
        })
        # 确保有必需的列
        if 'symbol' not in df.columns or 'name' not in df.columns:
            logger.error(f"AKShare: Unexpected column names: {df.columns.tolist()}")
            return None
        # 生成 ts_code 和其他字段
        def generate_ts_code(code: str) -> str:
            """根据股票代码生成 ts_code"""
            if not code:
                return ""
            code = str(code).zfill(6)
            if code.startswith(('60', '68', '90')):
                return f"{code}.SH"
            elif code.startswith(('00', '30', '20')):
                return f"{code}.SZ"
            elif code.startswith(('8', '4')):
                return f"{code}.BJ"
            else:
                return f"{code}.SZ"  # 默认深圳
        def get_market(code: str) -> str:
            """根据股票代码判断市场"""
            if not code:
                return ""
            code = str(code).zfill(6)
            if code.startswith('000'):
                return '主板'
            elif code.startswith('002'):
                return '中小板'
            elif code.startswith('300'):
                return '创业板'
            elif code.startswith('60'):
                return '主板'
            elif code.startswith('688'):
                return '科创板'
            elif code.startswith('8'):
                return '北交所'
            elif code.startswith('4'):
                return '新三板'
            else:
                return '未知'
        # 添加 ts_code 和 market 字段
        df['ts_code'] = df['symbol'].apply(generate_ts_code)
        df['market'] = df['symbol'].apply(get_market)
        df['area'] = ''
        df['industry'] = ''
        df['list_date'] = ''
        logger.info(f"AKShare: Successfully fetched {len(df)} stocks with real names")
        return df
    except Exception as e:
        logger.error(f"AKShare: Failed to fetch stock list: {e}")
        return None
def get_daily_basic(self, trade_date: str) -> Optional[pd.DataFrame]:
    """获取每日基础财务数据（快速版）"""
    if not self.is_available():
        return None
    try:
        import akshare as ak  # noqa: F401
        logger.info(f"AKShare: Attempting to get basic financial data for {trade_date}")
        stock_df = self.get_stock_list()
        if stock_df is None or stock_df.empty:
            logger.warning("AKShare: No stock list available")
            return None
        max_stocks = 10
        stock_list = stock_df.head(max_stocks)
        basic_data = []
        processed_count = 0
        import time
        start_time = time.time()
        timeout_seconds = 30
        for _, stock in stock_list.iterrows():
            if time.time() - start_time > timeout_seconds:
                logger.warning(f"AKShare: Timeout reached, processed {processed_count} stocks")
                break
            try:
                symbol = stock.get('symbol', '')
                name = stock.get('name', '')
                ts_code = stock.get('ts_code', '')
                if not symbol:
                    continue
                info_data = ak.stock_individual_info_em(symbol=symbol)
                if info_data is not None and not info_data.empty:
                    info_dict = {}
                    for _, row in info_data.iterrows():
                        item = row.get('item', '')
                        value = row.get('value', '')
```

```
                info_dict[item] = value
            latest_price = self._safe_float(info_dict.get('最新', 0))
            # 🔥 AKShare 的"总市值"单位是万元，需要转换为亿元（与 Tushare 一致）
            total_mv_wan = self._safe_float(info_dict.get('总市值', 0))  # 万元
            total_mv_yi = total_mv_wan / 10000 if total_mv_wan else None  # 转换为亿元
            basic_data.append({
                'ts_code': ts_code,
                'trade_date': trade_date,
                'name': name,
                'close': latest_price,
                'total_mv': total_mv_yi,  # 亿元（与 Tushare 一致）
                'turnover_rate': None,
                'pe': None,
                'pb': None,
            })
            processed_count += 1
            if processed_count % 5 == 0:
                logger.debug(f"AKShare: Processed {processed_count} stocks in {time.time() - start_time:.1f}s")
        except Exception as e:
            logger.debug(f"AKShare: Failed to get data for {symbol}: {e}")
            continue
    if basic_data:
        df = pd.DataFrame(basic_data)
        logger.info(f"AKShare: Successfully fetched basic data for {trade_date}, {len(df)} records")
        return df
    else:
        logger.warning("AKShare: No basic data collected")
        return None
  except Exception as e:
    logger.error(f"AKShare: Failed to fetch basic data for {trade_date}: {e}")
    return None
def _safe_float(self, value) -> Optional[float]:
  try:
    if value is None or value == '' or value == 'None':
        return None
    return float(value)
  except (ValueError, TypeError):
    return None
def get_realtime_quotes(self, source: str = "eastmoney"):
  """
  获取全市场实时快照，返回以6位代码为键的字典
  Args:
    source: 数据源选择，"eastmoney"（东方财富）或 "sina"（新浪财经）
  Returns:
    Dict[str, Dict]: {code: {close, pct_chg, amount, ...}}
  """
  if not self.is_available():
    return None
  try:
    import akshare as ak  # type: ignore
    # 根据 source 参数选择接口
    if source == "sina":
        df = ak.stock_zh_a_spot()  # 新浪财经接口
        logger.info("使用 AKShare 新浪财经接口获取实时行情")
    else:  # 默认使用东方财富
        df = ak.stock_zh_a_spot_em()  # 东方财富接口
        logger.info("使用 AKShare 东方财富接口获取实时行情")
    if df is None or getattr(df, "empty", True):
        logger.warning(f"AKShare {source} 返回空数据")
        return None
    # 列名兼容（两个接口的列名可能不同）
    code_col = next((c for c in ["代码", "code", "symbol", "股票代码"] if c in df.columns), None)
    price_col = next((c for c in ["最新价", "现价", "最新价(元)", "price", "最新", "trade"] if c in df.columns), None)
    pct_col = next((c for c in ["涨跌幅", "涨跌幅(%)", "涨幅", "pct_chg", "changepercent"] if c in df.columns), None)
    amount_col = next((c for c in ["成交额", "成交额(元)", "amount", "成交额(万元)", "amount(万元)"] if c in df.columns), None)
    open_col = next((c for c in ["今开", "开盘", "open", "今开(元)"] if c in df.columns), None)
    high_col = next((c for c in ["最高", "high"] if c in df.columns), None)
    low_col = next((c for c in ["最低", "low"] if c in df.columns), None)
    pre_close_col = next((c for c in ["昨收", "昨收(元)", "pre_close", "昨收价", "settlement"] if c in df.columns), None)
    volume_col = next((c for c in ["成交量", "成交量(手)", "volume", "成交量(股)", "vol"] if c in df.columns), None)
    if not code_col or not price_col:
        logger.error(f"AKShare {source} 缺少必要列: code={code_col}, price={price_col}, columns={list(df.columns)}")
        return None
    result: Dict[str, Dict[str, Optional[float]]] = {}
    for _, row in df.iterrows():  # type: ignore
        code_raw = row.get(code_col)
        if not code_raw:
            continue
        # 标准化股票代码：处理交易所前缀（如 sz000001, sh600036）
        code_str = str(code_raw).strip()
        # 如果代码长度超过6位，去掉前面的交易所前缀（如 sz, sh）
        if len(code_str) > 6:
            # 去掉前面的非数字字符（通常是2个字符的交易所代码）
            code_str = ''.join(filter(str.isdigit, code_str))
        # 如果是纯数字，移除前导0后补齐到6位
        if code_str.isdigit():
            code_clean = code_str.lstrip('0') or '0'  # 移除前导0，如果全是0则保留一个0
            code = code_clean.zfill(6)  # 补齐到6位
        else:
            # 如果不是纯数字，尝试提取数字部分
            code_digits = ''.join(filter(str.isdigit, code_str))
            if code_digits:
                code = code_digits.zfill(6)
            else:
                # 无法提取有效代码，跳过
                continue
        close = self._safe_float(row.get(price_col))
        pct = self._safe_float(row.get(pct_col)) if pct_col else None
        amt = self._safe_float(row.get(amount_col)) if amount_col else None
        op = self._safe_float(row.get(open_col)) if open_col else None
        hi = self._safe_float(row.get(high_col)) if high_col else None
        lo = self._safe_float(row.get(low_col)) if low_col else None
        pre = self._safe_float(row.get(pre_close_col)) if pre_close_col else None
```

```
            vol = self._safe_float(row.get(volume_col)) if volume_col else None
            # 🔥 日志：记录AKShare返回的成交量
            if code in ["300750", "000001", "600000"]:  # 只记录几个示例股票
                logger.info(f"📊 [AKShare实时] {code} - volume_col={volume_col}, vol={vol}, amount={amt}")
            result[code] = {
                "close": close,
                "pct_chg": pct,
                "amount": amt,
                "volume": vol,
                "open": op,
                "high": hi,
                "low": lo,
                "pre_close": pre
            }
        logger.info(f"✅ AKShare {source} 获取到 {len(result)} 只股票的实时行情")
        return result
    except Exception as e:
        logger.error(f"获取AKShare {source} 实时快照失败: {e}")
        return None
def get_kline(self, code: str, period: str = "day", limit: int = 120, adj: Optional[str] = None):
    """AKShare K-line as fallback. Try daily/week/month via stock_zh_a_hist; minutes via stock_zh_a_minute."""
    if not self.is_available():
        return None
    try:
        import akshare as ak
        code6 = str(code).zfill(6)
        items = []
        if period in ("day", "week", "month"):
            period_map = {"day": "daily", "week": "weekly", "month": "monthly"}
            adjust_map = {None: "", "qfq": "qfq", "hfq": "hfq"}
            df = ak.stock_zh_a_hist(symbol=code6, period=period_map[period], adjust=adjust_map.get(adj, ""))
            if df is None or getattr(df, 'empty', True):
                return None
            df = df.tail(limit)
            for _, row in df.iterrows():
                items.append({
                    "time": str(row.get('日期') or row.get('date') or ''),
                    "open": self._safe_float(row.get('开盘') or row.get('open')),
                    "high": self._safe_float(row.get('最高') or row.get('high')),
                    "low": self._safe_float(row.get('最低') or row.get('low')),
                    "close": self._safe_float(row.get('收盘') or row.get('close')),
                    "volume": self._safe_float(row.get('成交量') or row.get('volume')),
                    "amount": self._safe_float(row.get('成交额') or row.get('amount')),
                })
            return items
        else:
            # minutes
            per_map = {"5m": "5", "15m": "15", "30m": "30", "60m": "60"}
            if period not in per_map:
                return None
            df = ak.stock_zh_a_minute(symbol=code6, period=per_map[period], adjust=adj if adj in ("qfq", "hfq") else "")
            if df is None or getattr(df, 'empty', True):
                return None
            df = df.tail(limit)
            for _, row in df.iterrows():
                items.append({
                    "time": str(row.get('时间') or row.get('day') or ''),
                    "open": self._safe_float(row.get('开盘') or row.get('open')),
                    "high": self._safe_float(row.get('最高') or row.get('high')),
                    "low": self._safe_float(row.get('最低') or row.get('low')),
                    "close": self._safe_float(row.get('收盘') or row.get('close')),
                    "volume": self._safe_float(row.get('成交量') or row.get('volume')),
                    "amount": self._safe_float(row.get('成交额') or row.get('amount')),
                })
            return items
    except Exception as e:
        logger.error(f"AKShare get_kline failed: {e}")
        return None
def get_news(self, code: str, days: int = 2, limit: int = 50, include_announcements: bool = True):
    """AKShare-based news/announcements fallback"""
    if not self.is_available():
        return None
    try:
        import akshare as ak
        code6 = str(code).zfill(6)
        items = []
        # news
        try:
            dfn = ak.stock_news_em(symbol=code6)
            if dfn is not None and not dfn.empty:
                for _, row in dfn.head(limit).iterrows():
                    items.append({
                        # AkShare 将字段标准化为中文列名：新闻标题 / 文章来源 / 发布时间 / 新闻链接
                        "title": str(row.get('新闻标题') or row.get('标题') or row.get('title') or ''),
                        "source": str(row.get('文章来源') or row.get('来源') or row.get('source') or 'akshare'),
                        "time": str(row.get('发布时间') or row.get('time') or ''),
                        "url": str(row.get('新闻链接') or row.get('url') or ''),
                        "type": "news",
                    })
        except Exception:
            pass
        # announcements
        try:
            if include_announcements:
                dfa = ak.stock_announcement_em(symbol=code6)
                if dfa is not None and not dfa.empty:
                    for _, row in dfa.head(max(0, limit - len(items))).iterrows():
                        items.append({
                            "title": str(row.get('公告标题') or row.get('title') or ''),
                            "source": "akshare",
                            "time": str(row.get('公告时间') or row.get('time') or ''),
                            "url": str(row.get('公告链接') or row.get('url') or ''),
                            "type": "announcement",
                        })
        except Exception:
```

```
            pass
        return items if items else None
    except Exception as e:
        logger.error(f"AKShare get_news failed: {e}")
        return None
def find_latest_trade_date(self) -> Optional[str]:
    yesterday = (datetime.now() - timedelta(days=1)).strftime("%Y%m%d")
    logger.info(f"AKShare: Using yesterday as trade date: {yesterday}")
    return yesterday
```

FILE: app/services/data_sources/baostock_adapter.py

```
"""
BaoStock data source adapter
"""
from typing import Optional
import logging
from datetime import datetime, timedelta
import pandas as pd
from .base import DataSourceAdapter
logger = logging.getLogger(__name__)
class BaoStockAdapter(DataSourceAdapter):
    """BaoStockdata source adapter"""
    def __init__(self):
        super().__init__()  # 调用父类初始化
    @property
    def name(self) -> str:
        return "baostock"
    def _get_default_priority(self) -> int:
        return 1  # lowest priority (数字越大优先级越高)
    def is_available(self) -> bool:
        try:
            import baostock as bs  # noqa: F401
            return True
        except ImportError:
            return False
    def get_stock_list(self) -> Optional[pd.DataFrame]:
        if not self.is_available():
            return None
        try:
            import baostock as bs
            lg = bs.login()
            if lg.error_code != '0':
                logger.error(f"BaoStock: Login failed: {lg.error_msg}")
                return None
            try:
                logger.info("BaoStock: Querying stock basic info...")
                rs = bs.query_stock_basic()
                if rs.error_code != '0':
                    logger.error(f"BaoStock: Query failed: {rs.error_msg}")
                    return None
                data_list = []
                while (rs.error_code == '0') & rs.next():
                    data_list.append(rs.get_row_data())
                if not data_list:
                    return None
                df = pd.DataFrame(data_list, columns=rs.fields)
                df = df[df['type'] == '1']
                df['symbol'] = df['code'].str.replace(r'^(sh|sz)\.', '', regex=True)
                df['ts_code'] = (
                    df['code'].str.replace('sh.', '').str.replace('sz.', '')
                    + df['code'].str.extract(r'^(sh|sz)\.').iloc[:, 0].str.upper().str.replace('SH', '.SH').str.replace('SZ', '.SZ')
                )
                df['name'] = df['code_name']
                df['area'] = ''
                # 获取行业信息
                logger.info("BaoStock: Querying stock industry info...")
                industry_rs = bs.query_stock_industry()
                if industry_rs.error_code == '0':
                    industry_list = []
                    while (industry_rs.error_code == '0') & industry_rs.next():
                        industry_list.append(industry_rs.get_row_data())
                    if industry_list:
                        industry_df = pd.DataFrame(industry_list, columns=industry_rs.fields)
                        # 去掉行业编码前缀（如 "I65软件和信息技术服务业" -> "软件和信息技术服务业"）
                        def clean_industry_name(industry_str):
                            if not industry_str or pd.isna(industry_str):
                                return ''
                            # 使用正则表达式去掉前面的字母和数字编码（如 I65、C31 等）
                            import re
                            cleaned = re.sub(r'^[A-Z]\d+', '', str(industry_str))
                            return cleaned.strip()
                        industry_df['industry_clean'] = industry_df['industry'].apply(clean_industry_name)
                        # 创建行业映射字典 {code: industry_clean}
                        industry_map = dict(zip(industry_df['code'], industry_df['industry_clean']))
                        # 将行业信息合并到主DataFrame
                        df['industry'] = df['code'].map(industry_map).fillna('')
                        logger.info(f"BaoStock: Successfully mapped industry info for {len(industry_map)} stocks")
                    else:
                        df['industry'] = ''
                        logger.warning("BaoStock: No industry data returned")
                else:
                    df['industry'] = ''
                    logger.warning(f"BaoStock: Failed to query industry info: {industry_rs.error_msg}")
                df['market'] = '\u4e3b\u677f'
                df['list_date'] = ''
                logger.info(f"BaoStock: Successfully fetched {len(df)} stocks")
                return df[['symbol', 'name', 'ts_code', 'area', 'industry', 'market', 'list_date']]
            finally:
                bs.logout()
        except Exception as e:
            logger.error(f"BaoStock: Failed to fetch stock list: {e}")
```

```
        return None
    def get_daily_basic(self, trade_date: str, max_stocks: int = None) -> Optional[pd.DataFrame]:
        """
        获取每日基础数据（包含PE、PB、总市值等）
        Args:
            trade_date: 交易日期 (YYYYMMDD)
            max_stocks: 最大处理股票数量，None表示处理所有股票
        """
        if not self.is_available():
            return None
        try:
            import baostock as bs
            logger.info(f"BaoStock: Attempting to get valuation data for {trade_date}")
            lg = bs.login()
            if lg.error_code != '0':
                logger.error(f"BaoStock: Login failed: {lg.error_msg}")
                return None
            try:
                logger.info("BaoStock: Querying stock basic info...")
                rs = bs.query_stock_basic()
                if rs.error_code != '0':
                    logger.error(f"BaoStock: Query stock list failed: {rs.error_msg}")
                    return None
                stock_list = []
                while (rs.error_code == '0') & rs.next():
                    stock_list.append(rs.get_row_data())
                if not stock_list:
                    logger.warning("BaoStock: No stocks found")
                    return None
                total_stocks = len([s for s in stock_list if len(s) > 5 and s[4] == '1' and s[5] == '1'])
                logger.info(f"📊 BaoStock: 找到 {total_stocks} 只活跃股票，开始处理{'全部' if max_stocks is None else f'前 {max_stocks} 只'}...")
                basic_data = []
                processed_count = 0
                failed_count = 0
                for stock in stock_list:
                    if max_stocks and processed_count >= max_stocks:
                        break
                    code = stock[0] if len(stock) > 0 else ''
                    name = stock[1] if len(stock) > 1 else ''
                    stock_type = stock[4] if len(stock) > 4 else '0'
                    status = stock[5] if len(stock) > 5 else '0'
                    if stock_type == '1' and status == '1':
                        try:
                            formatted_date = f"{trade_date[:4]}-{trade_date[4:6]}-{trade_date[6:8]}"
                            # 🔥 获取估值数据和总股本
                            rs_valuation = bs.query_history_k_data_plus(
                                code,
                                "date,code,close,peTTM,pbMRQ,psTTM,pcfNcfTTM,isST",
                                start_date=formatted_date,
                                end_date=formatted_date,
                                frequency="d",
                                adjustflag="3",
                            )
                            if rs_valuation.error_code == '0':
                                valuation_data = []
                                while (rs_valuation.error_code == '0') & rs_valuation.next():
                                    valuation_data.append(rs_valuation.get_row_data())
                                if valuation_data:
                                    row = valuation_data[0]
                                    symbol = code.replace('sh.', '').replace('sz.', '')
                                    ts_code = f"{symbol}.SH" if code.startswith('sh.') else f"{symbol}.SZ"
                                    pe_ttm = self._safe_float(row[3]) if len(row) > 3 else None
                                    pb_mrq = self._safe_float(row[4]) if len(row) > 4 else None
                                    ps_ttm = self._safe_float(row[5]) if len(row) > 5 else None
                                    pcf_ttm = self._safe_float(row[6]) if len(row) > 6 else None
                                    close_price = self._safe_float(row[2]) if len(row) > 2 else None
                                    # 🔥 BaoStock 不直接提供总市值和总股本
                                    # 为了避免同步超时，这里不调用额外的 API 获取总股本
                                    # total_mv 留空，后续可以通过其他数据源补充
                                    total_mv = None
                                    basic_data.append({
                                        'ts_code': ts_code,
                                        'trade_date': trade_date,
                                        'name': name,
                                        'pe': pe_ttm,  # 🔥 市盈率（TTM）
                                        'pb': pb_mrq,  # 🔥 市净率（MRQ）
                                        'ps': ps_ttm,  # 市销率
                                        'pcf': pcf_ttm,  # 市现率
                                        'close': close_price,
                                        'total_mv': total_mv,  # ⚠️ BaoStock 不提供，留空
                                        'turnover_rate': None,  # ⚠️ BaoStock 不提供
                                    })
                                    processed_count += 1
                                    # 🔥 每处理50只股票输出一次进度日志
                                    if processed_count % 50 == 0:
                                        progress_pct = (processed_count / total_stocks) * 100
                                        logger.info(f"📊 BaoStock 同步进度: {processed_count}/{total_stocks} ({progress_pct:.1f}%) - 最新: {name}({ts_code})")
                                else:
                                    failed_count += 1
                            else:
                                failed_count += 1
                        except Exception as e:
                            failed_count += 1
                            if failed_count % 50 == 0:
                                logger.warning(f"⚠️ BaoStock: 已有 {failed_count} 只股票获取失败")
                            logger.debug(f"BaoStock: Failed to get valuation for {code}: {e}")
                            continue
                if basic_data:
                    df = pd.DataFrame(basic_data)
                    logger.info(f"✅ BaoStock 同步完成: 成功 {len(df)} 只，失败 {failed_count} 只，日期 {trade_date}")
                    return df
                else:
                    logger.warning(f"⚠️ BaoStock: 未获取到任何估值数据（失败 {failed_count} 只）")
                    return None
            finally:
```

```
            bs.logout()
    except Exception as e:
        logger.error(f"BaoStock: Failed to fetch valuation data for {trade_date}: {e}")
        return None

def _safe_float(self, value) -> Optional[float]:
    try:
        if value is None or value == '' or value == 'None':
            return None
        return float(value)
    except (ValueError, TypeError):
        return None

def get_realtime_quotes(self):
    """Placeholder: BaoStock does not provide full-market realtime snapshot in our adapter.
    Return None to allow fallback to higher-priority sources.
    """
    if not self.is_available():
        return None
    return None

def get_kline(self, code: str, period: str = "day", limit: int = 120, adj: Optional[str] = None):
    """BaoStock not used for K-line here; return None to allow fallback"""
    if not self.is_available():
        return None
    return None

def get_news(self, code: str, days: int = 2, limit: int = 50, include_announcements: bool = True):
    """BaoStock does not provide news in this adapter; return None"""
    if not self.is_available():
        return None
    return None
    """
```